DEEP LEARNING

AI · ML · DS / REVISION NOTES

UDEMY · THE LAZY PROGRAMMER

PyTorch: Deep Learning and Artificial Intelligence

ANNs, CNNs, RNNs, NLP, recommenders, transfer learning, GANs and deep RL in PyTorch

Volume 3 of 3 · Sections 94–131

38 SECTIONS

377 FIGURES

350 PAGES

Compiled October 2026

github.com/ankitgupta1729/AI-ML-DS

Contents

LECTURE 94 Finding the Optimal Policy 4
LECTURE 95 The Solution 15
LECTURE 96 Generalized Policy Iteration 29
LECTURE 97 Policy as a Distribution 49
LECTURE 98 Temporal Difference Methods 56
LECTURE 99 Using V(s) for Policy Improvement 75
LECTURE 100 Training Neural Networks Is Inherently Unstable 97
LECTURE 101 Reinforcement Learning 112
LECTURE 102 A Matter of Perspective 119
LECTURE 103 Actions (Simplified) 125
LECTURE 104 Using the Buffer (During Training) 135
LECTURE 105 Using the Environment 143
LECTURE 106 Max Size and Set of States 148
LECTURE 107 Stock 151
LECTURE 108 Batch Size 154
LECTURE 109 Plot Rl Rewards and Portfolio Value 158
LECTURE 110 Stock Prices and Trained Agent 160
LECTURE 111 Review: Maximum Likelihood Estimation (MLE) 166
LECTURE 112 Standard Deviation 177
LECTURE 113 Section Outline 179
LECTURE 114 Contrastive Loss 184
LECTURE 115 Siamese Networks Code Outline 200
LECTURE 116 File Paths 210
LECTURE 117 Test Images 212
LECTURE 118 Subject and Set of Indices 214
LECTURE 119 Batch and Positive Samples 217
LECTURE 120 Non Matching Pairs and Test Generator 219
LECTURE 121 Distances 220

LECTURE 122 Facial Recognition Section Summary 222
LECTURE 123 Back to Regression 231
LECTURE 124 The Bernoulli 241
LECTURE 125 One-Hot Encoding Is Inefficient 250
LECTURE 126 Why Do We Need Gradient Descent? 263
LECTURE 127 Stochastic Gradient Descent 273
LECTURE 128 Momentum Analogy 279
LECTURE 129 Variable Learning Rates 287
LECTURE 130 Review So Far 309
LECTURE 131 One More Step to Adam 334

LECTURE 94

Finding the Optimal Policy

In this lecture, I'm going to discuss what it means to learn in a reinforcement learning problem and reinforcement learning, there are two main types of tasks. Previously, we discuss what I call the prediction problem that is given a policy pi. Find the associated value function V of s. The second type of task is called the control problem.

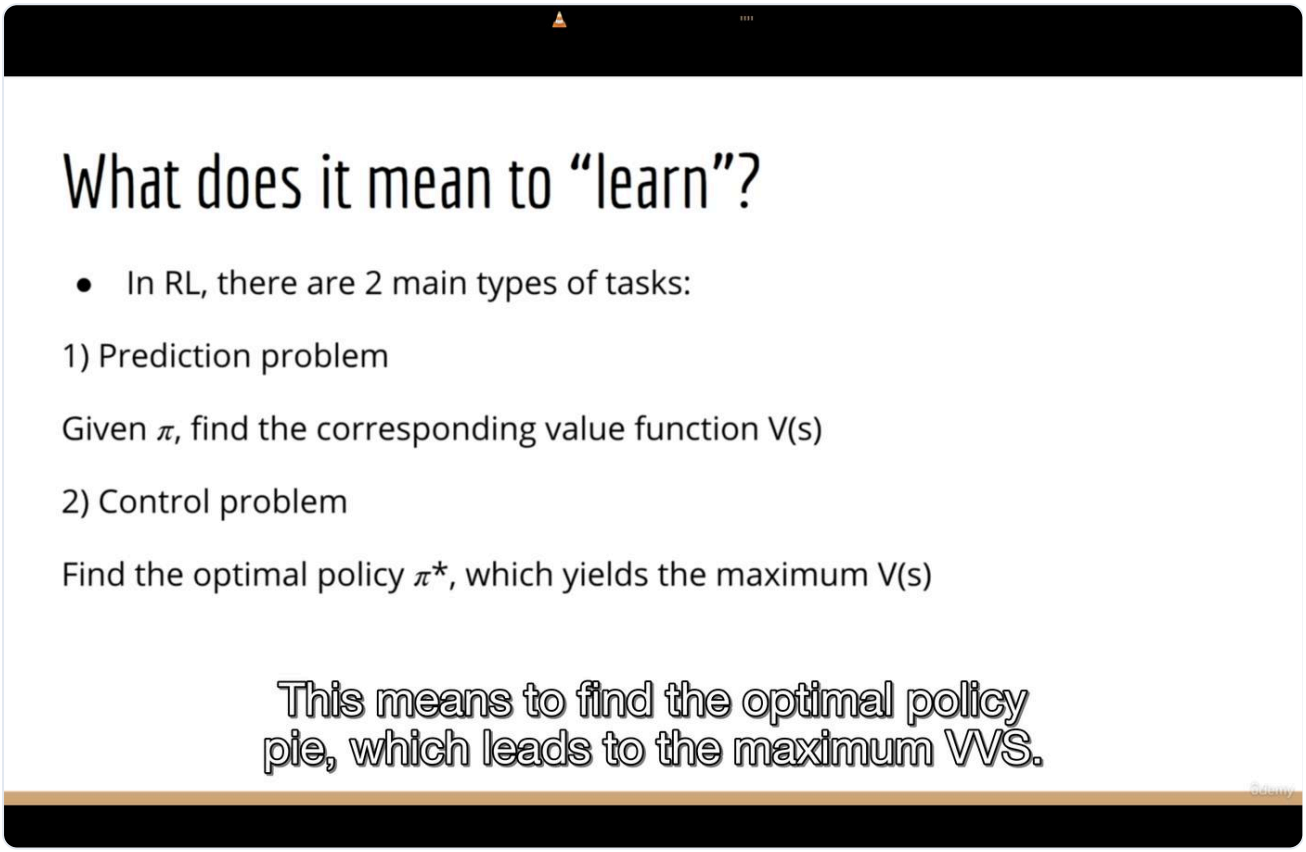

FIGURE 1.1 | Given Z, Find the Corresponding Value Function V(s)

This means to find the optimal policy pi, which leads to the maximum V of s. In other words, the control problem is maximizing the sum of future rewards. You've already known it since the beginning of the section that this is our goal. Now we can accurately define what this means and thus find a solution. First, we need a little bit more math and a few more symbols to accurately describe what we are doing. You already know about the value function given a state V of s, to be more precise, we refer to this as the state value function.

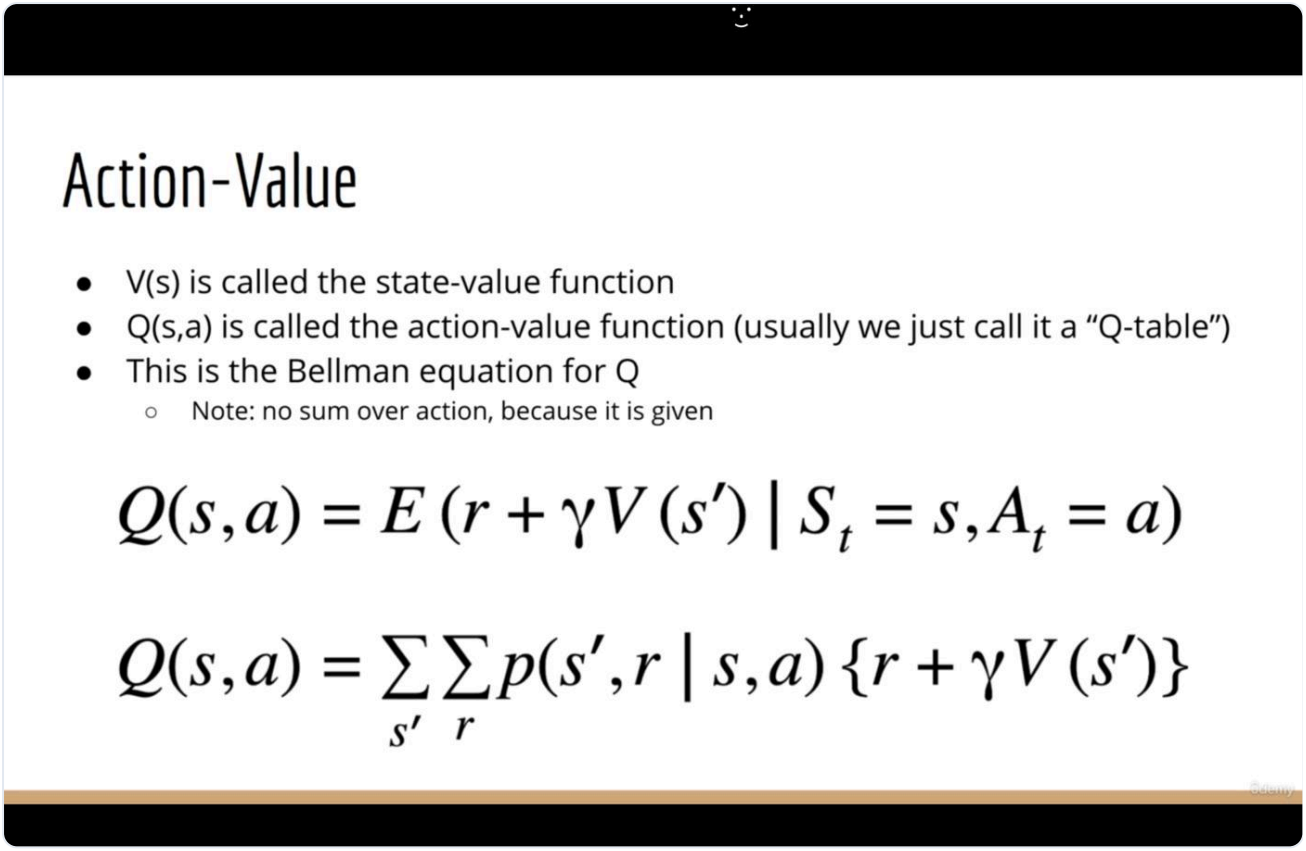

FIGURE 1.2

There is another quantity known as the action value function, where the value depends both on the state s and the action we denote it with the symbol Q As you can see, it has almost the same definition as V of s, except that it's also conditioned on the action. And so in the bellman equation, since A is given, we do not sum over the policy distribution. One interesting question to consider from a computer science perspective is how much space is required to store the value functions? Let's consider the scenario where both states and actions are discrete and that we have a finite number of them. So let's say we have big S states and big A actions. In this case, we can store the state value function in an array of size big S.

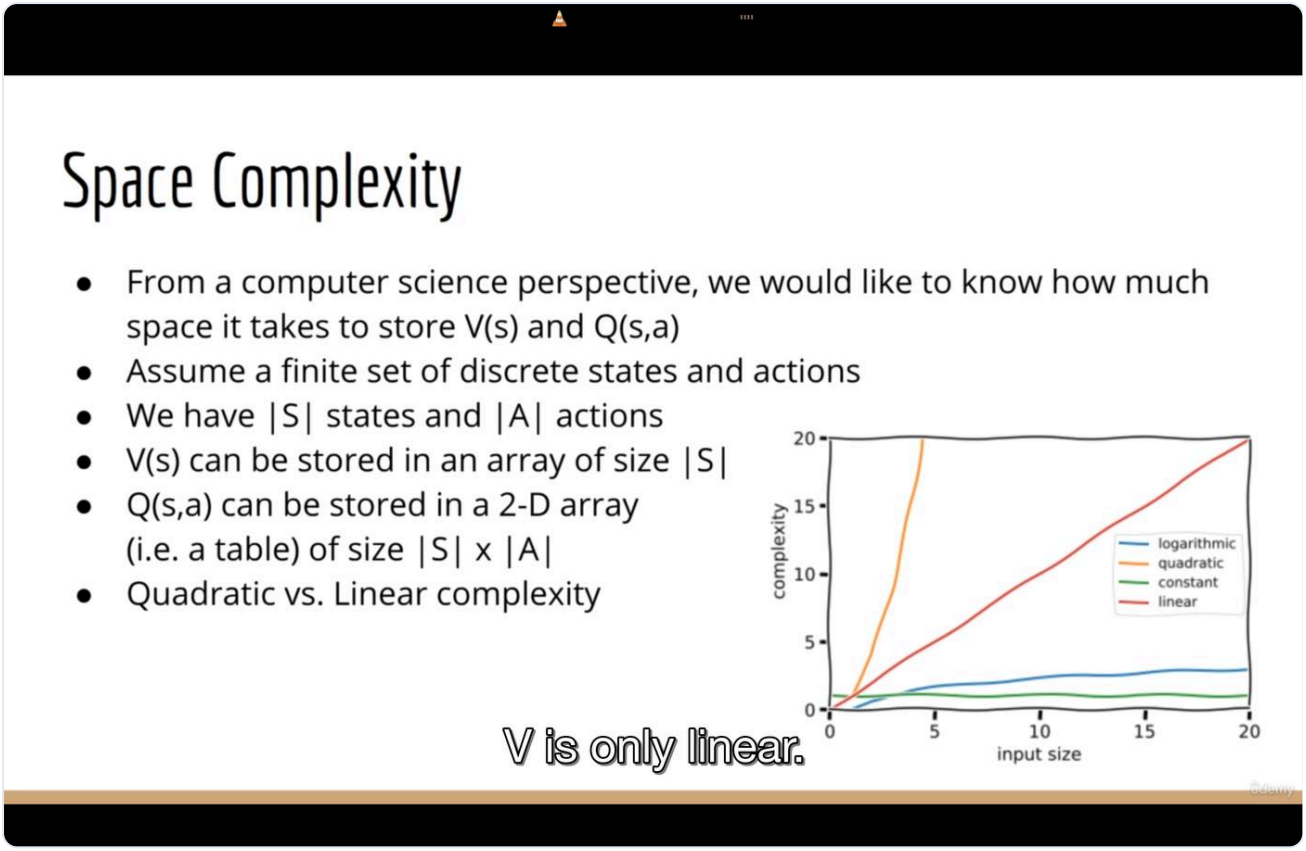

FIGURE 1.3 | Assume a Finite Set of Discrete States and Actions

But for the action value function, we'll have to store it in a two dimensional array of size big S times big A, and thus the storage required for Q is quadratic, whereas the storage required for V is only linear. OK, so why do we need this concept of the action value? Well, this is going to help us find the optimal policy. Remember, some policies may be good and some policies may be bad. The optimal policy is the best policy, the one that maximizes the value. First, let's just think about how to compare two different policies.

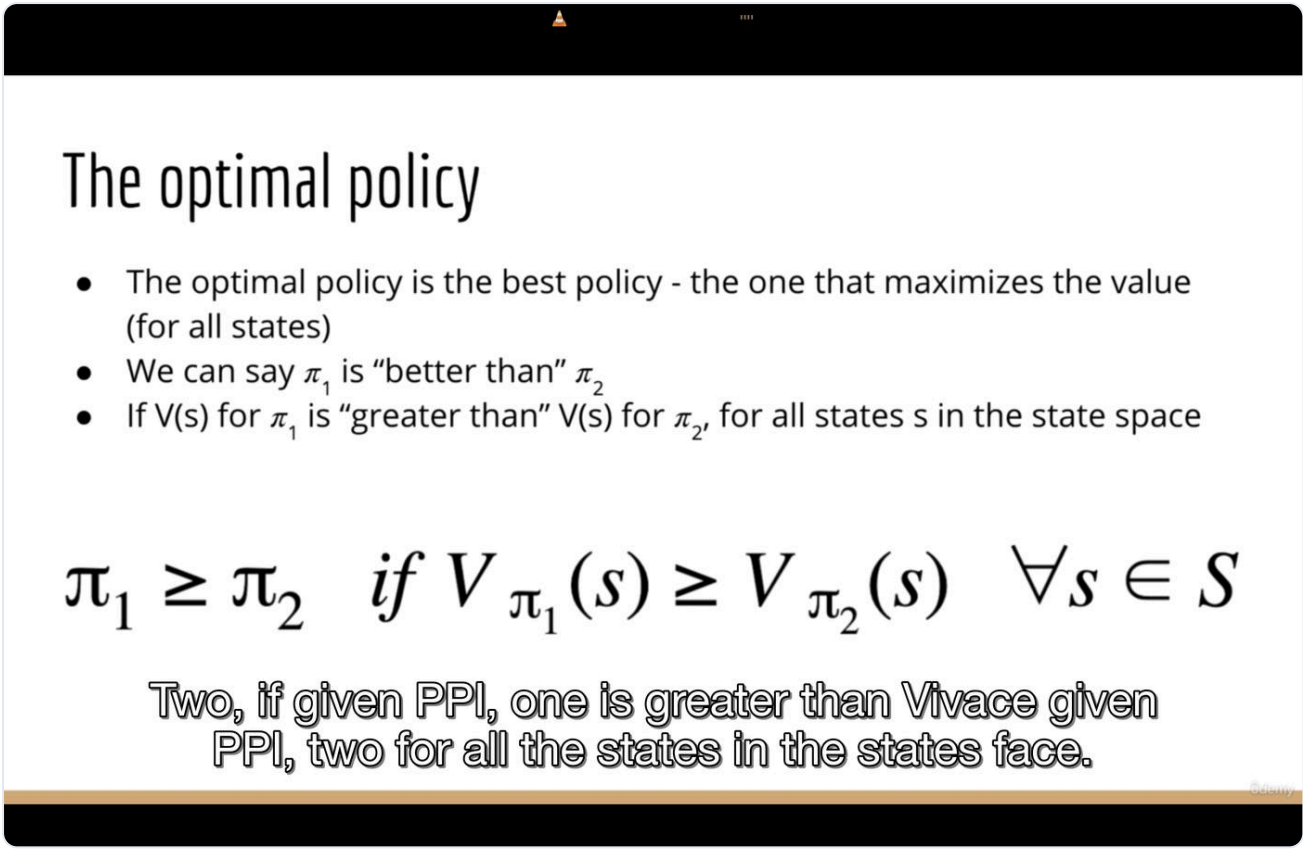

FIGURE 1.4 | The Optimal Policy

We can say that policy one is better than policy two, if V of s given pi one is greater than V of s given pi two for all the states in the state space. So this helps us describe the relative good ness of different policies.

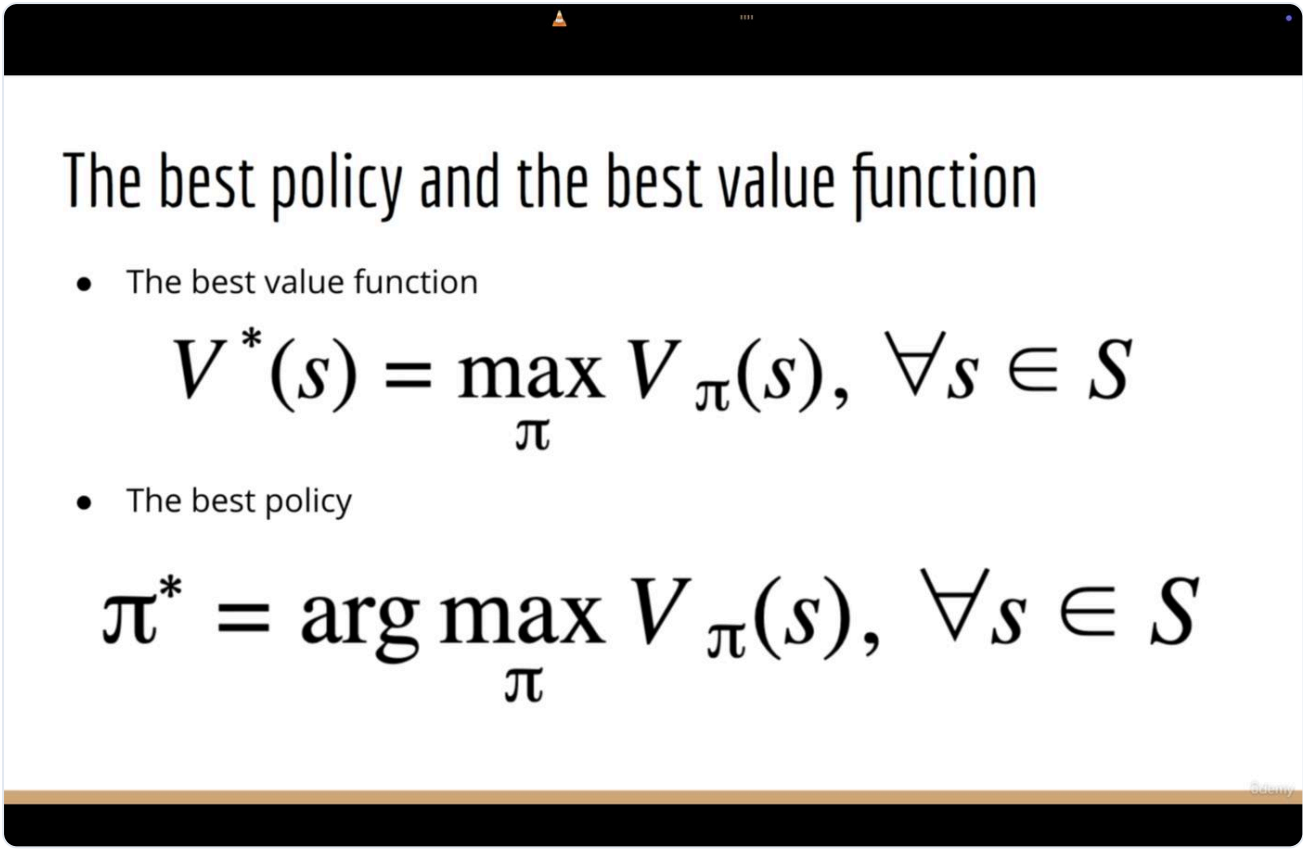

FIGURE 1.5 | The Best Policy and the Best Value Function

From here, we can define the best policy and the best value function, the best value function is the value function for which there is no greater value function. It's the max over all possible policies of V of s given pi. So we'll denote it with the symbol V star. Similarly, the best policy will be the argmax over all policies pi. So we call that pi star. So far, we haven't needed to invoke the action value function, but let's see how it's related. First, the optimal action value is defined similarly to the optimal state value. It's the max over all possible policies pi of Q given pi, we'll call this Q star. This must hold over all states and all actions.

Furthermore, the optimal state value is equal to the max over all actions from the optimal action value.

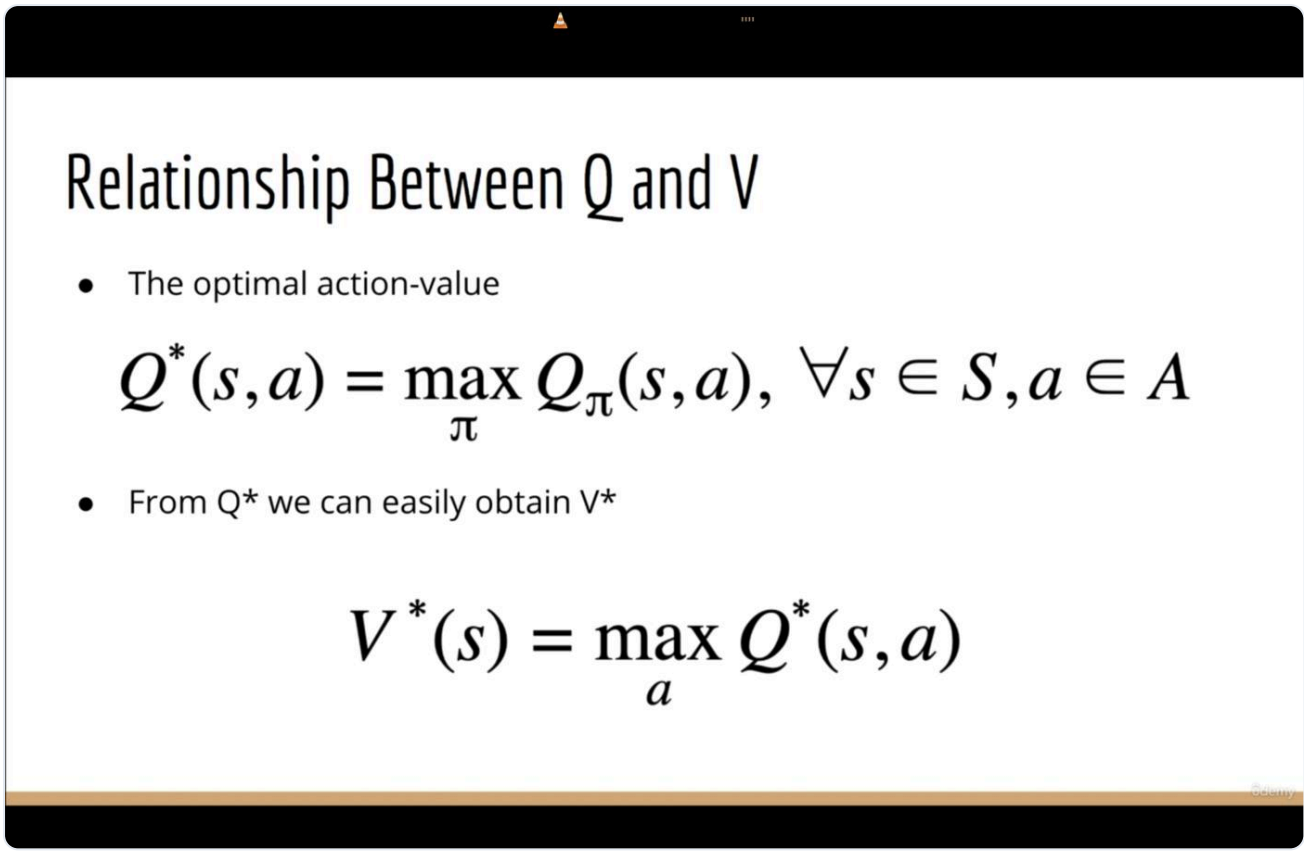

FIGURE 1.6 | Relationship Between Q and V

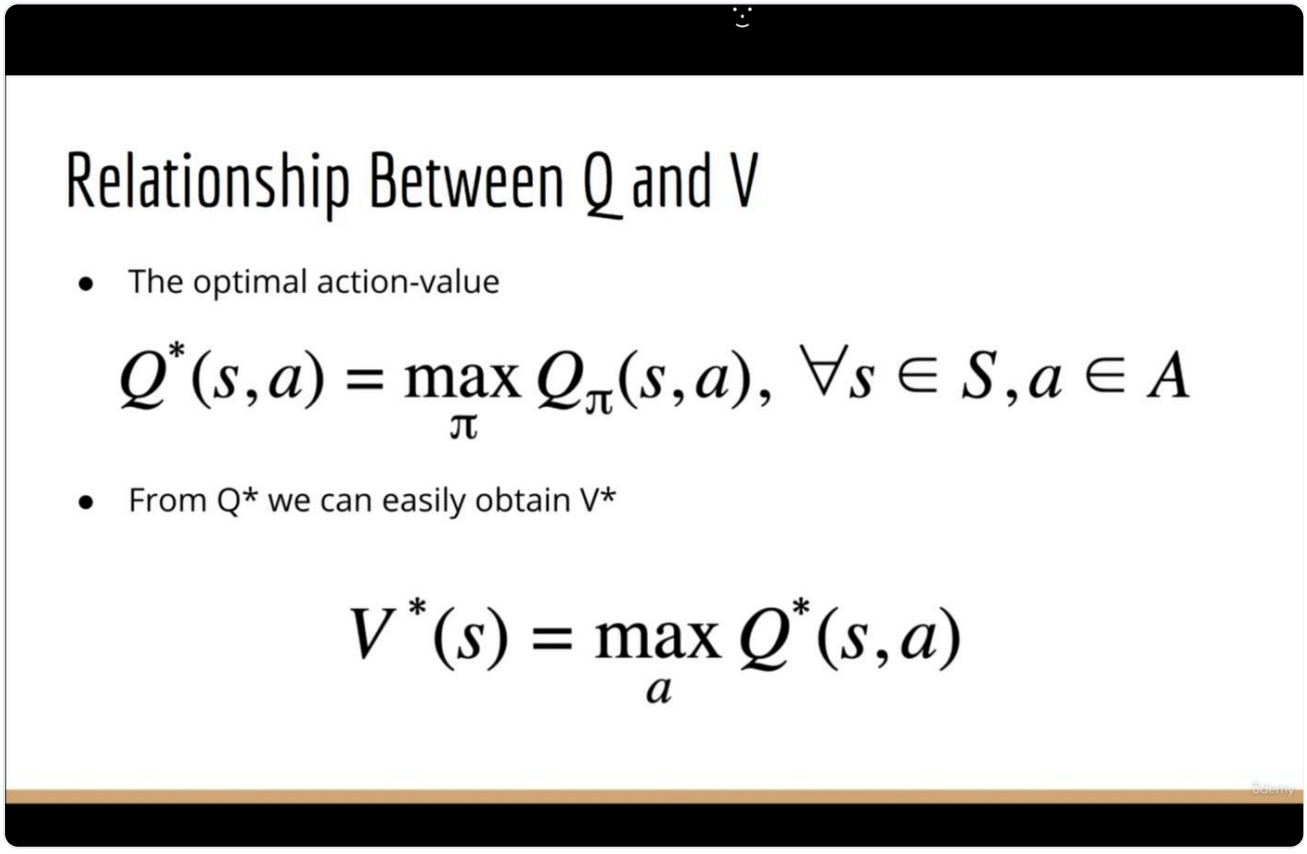

FIGURE 1.7 | Relationship Between Q and V

So from this, we can define the relationship between Q and V. The real value, no pun intended, of the action value function is this. Suppose we are playing some game and we would like to know what's the best action to perform right now? Well, we have a dictionary telling us exactly what to do. All we have to do is find the argmax of Q given a state s.

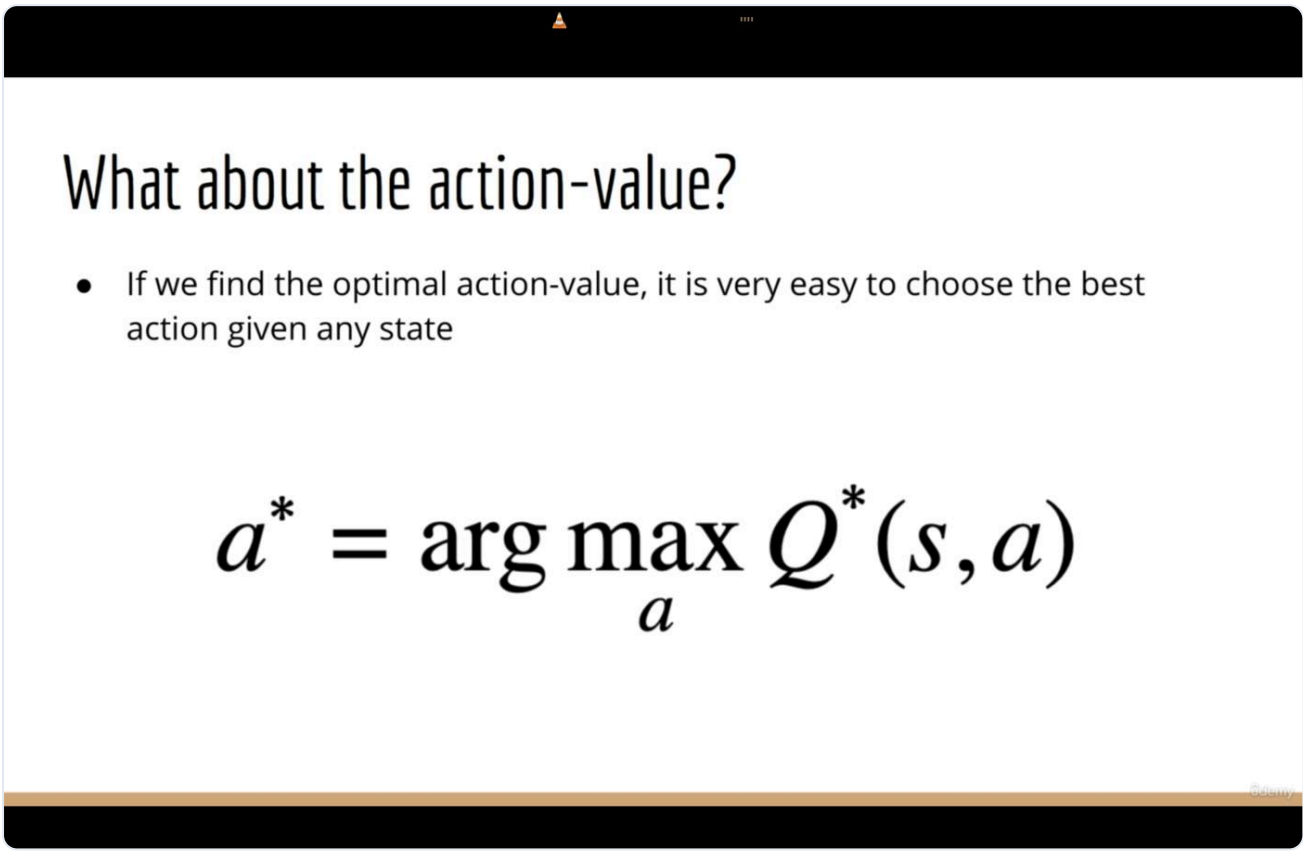

FIGURE 1.8 | What About the Action-value?

In other words, the best action to perform becomes a simple dictionary. Look-Up. As an aside, notice how the optimal policy is not unique. It could be there.

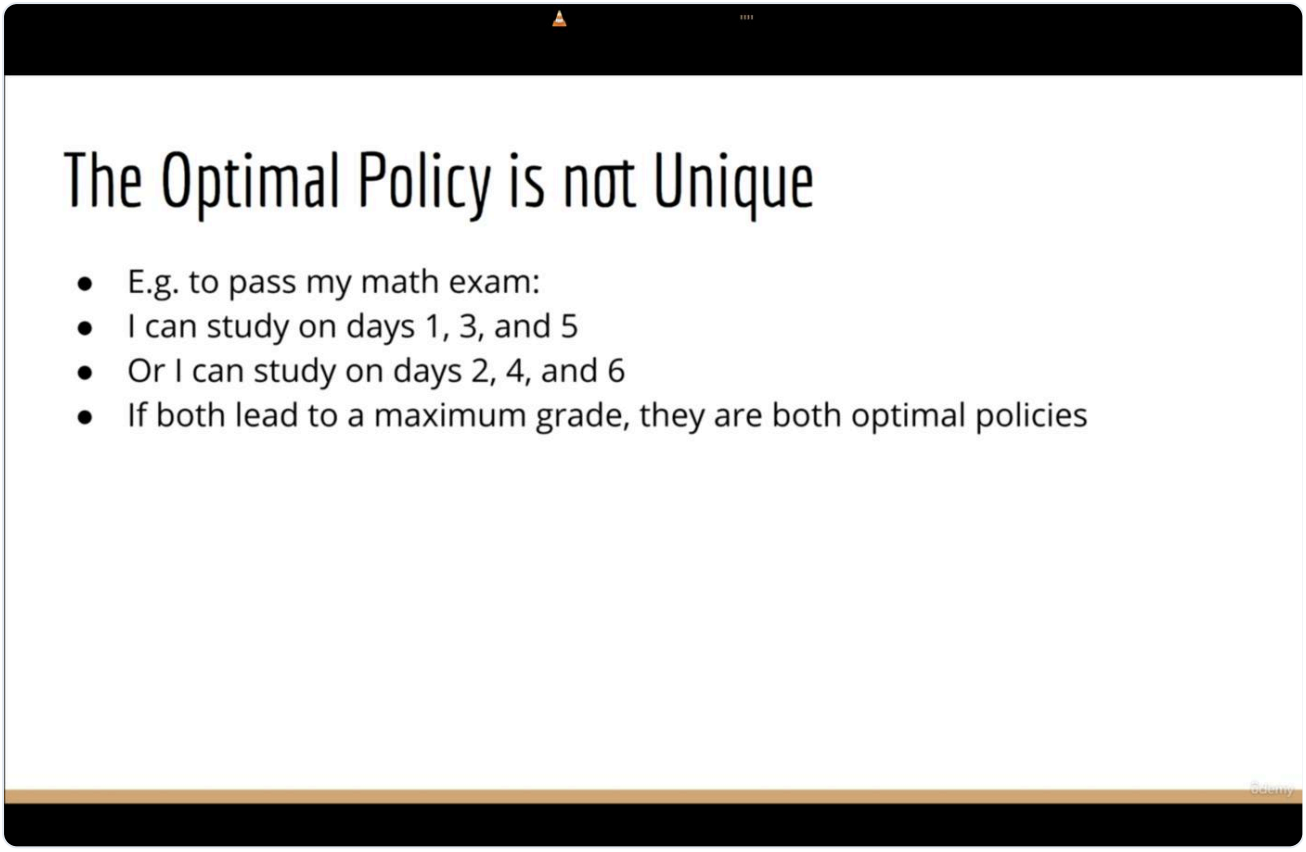

FIGURE 1.9 | The Optimal Policy Is Not Unique

Multiple different policies lead to the same best value function. In this case, it suffices to find just one of them. On the other hand, the optimal value function is unique because if there are two different value functions, then logically one of them must be greater than the other one. Before moving on, let's think about what a good first approach might be to actually finding an optimal policy. Remember, this is called the control problem.

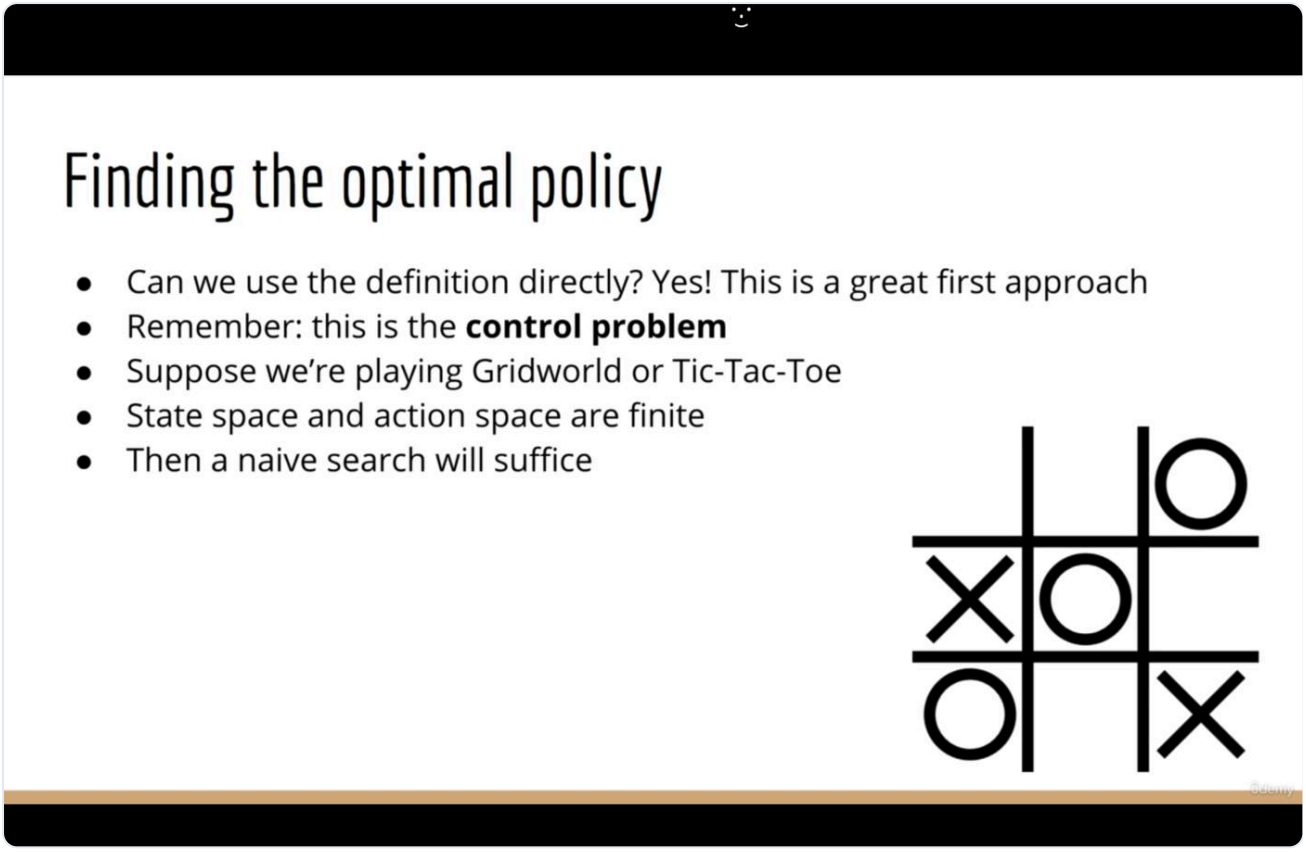

FIGURE 1.10 | Finding the Optimal Policy

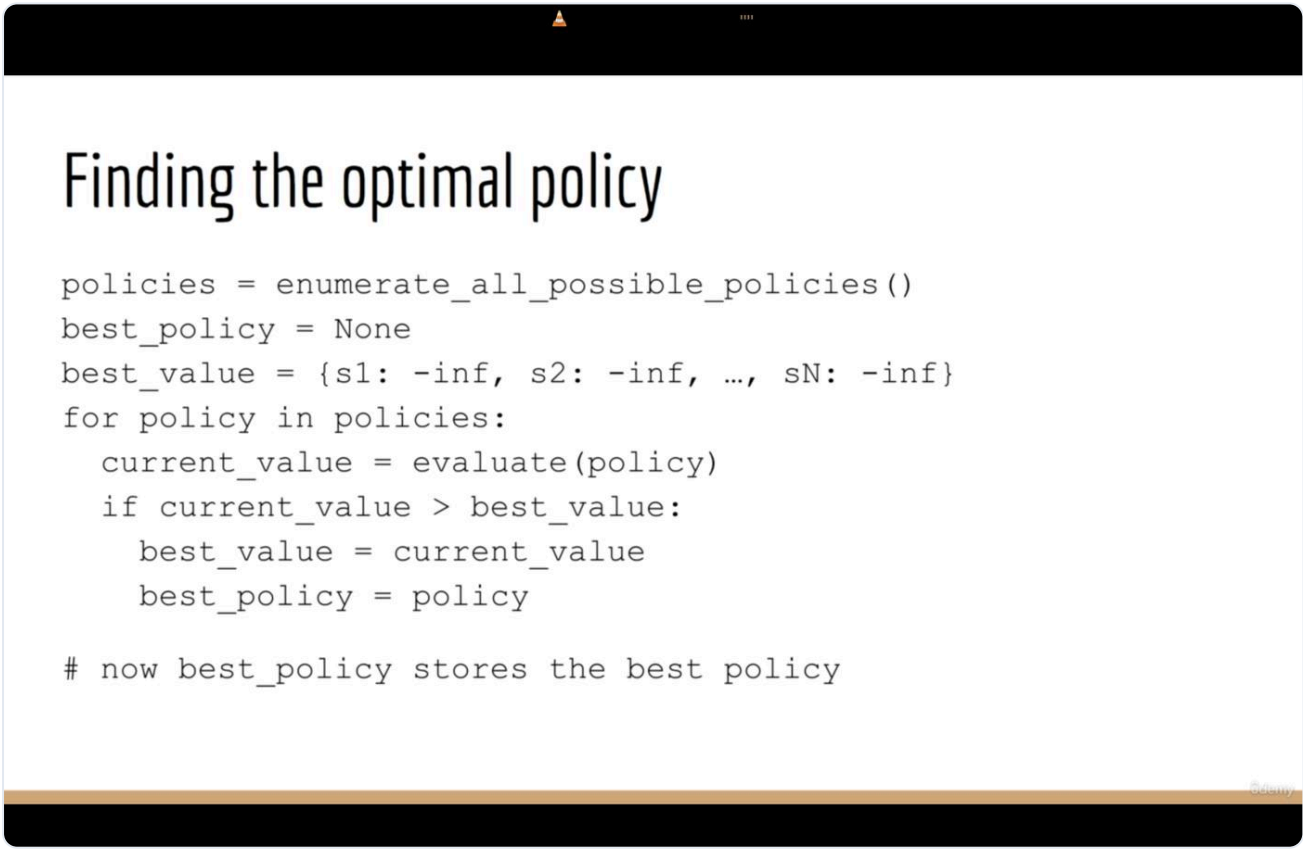

FIGURE 1.11 | Finding the Optimal Policy

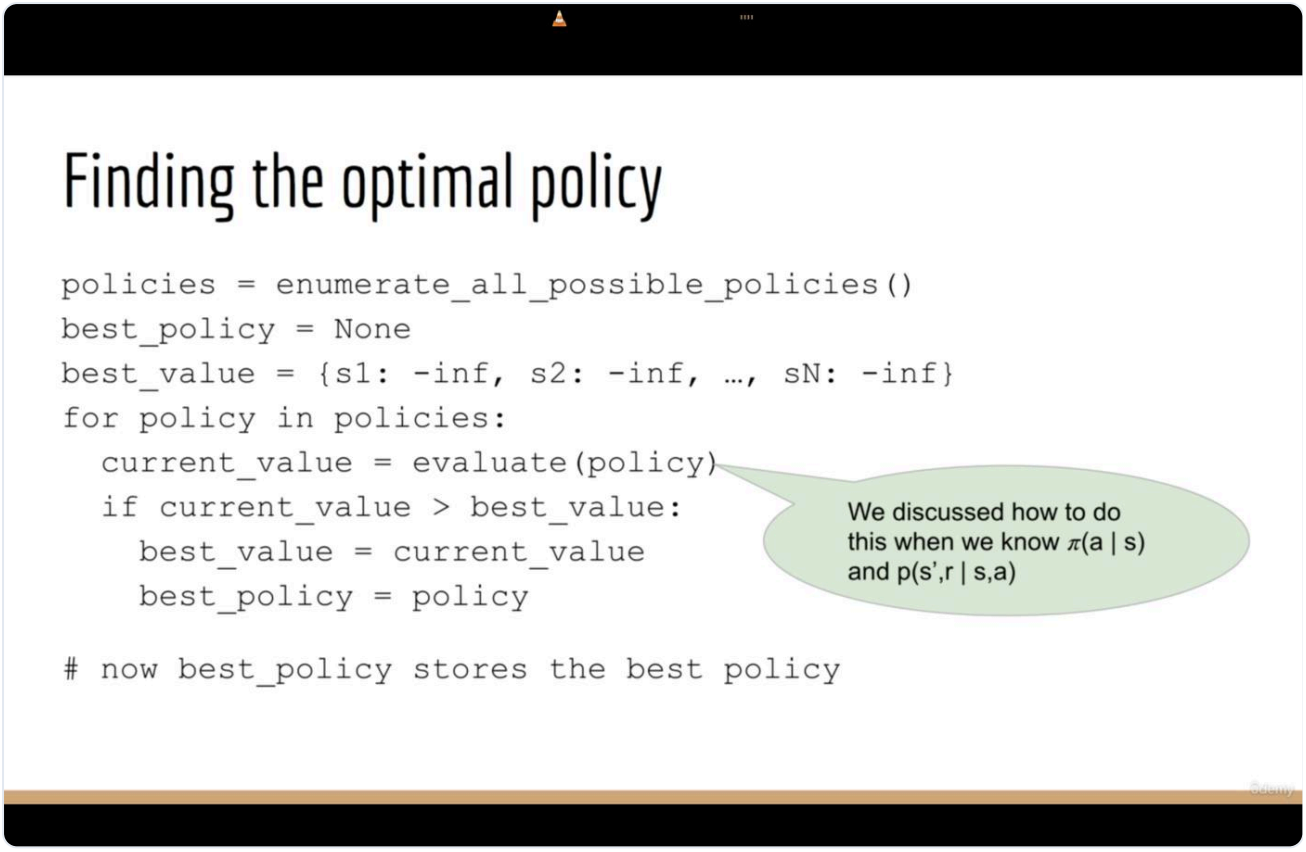

FIGURE 1.12 | Finding the Optimal Policy

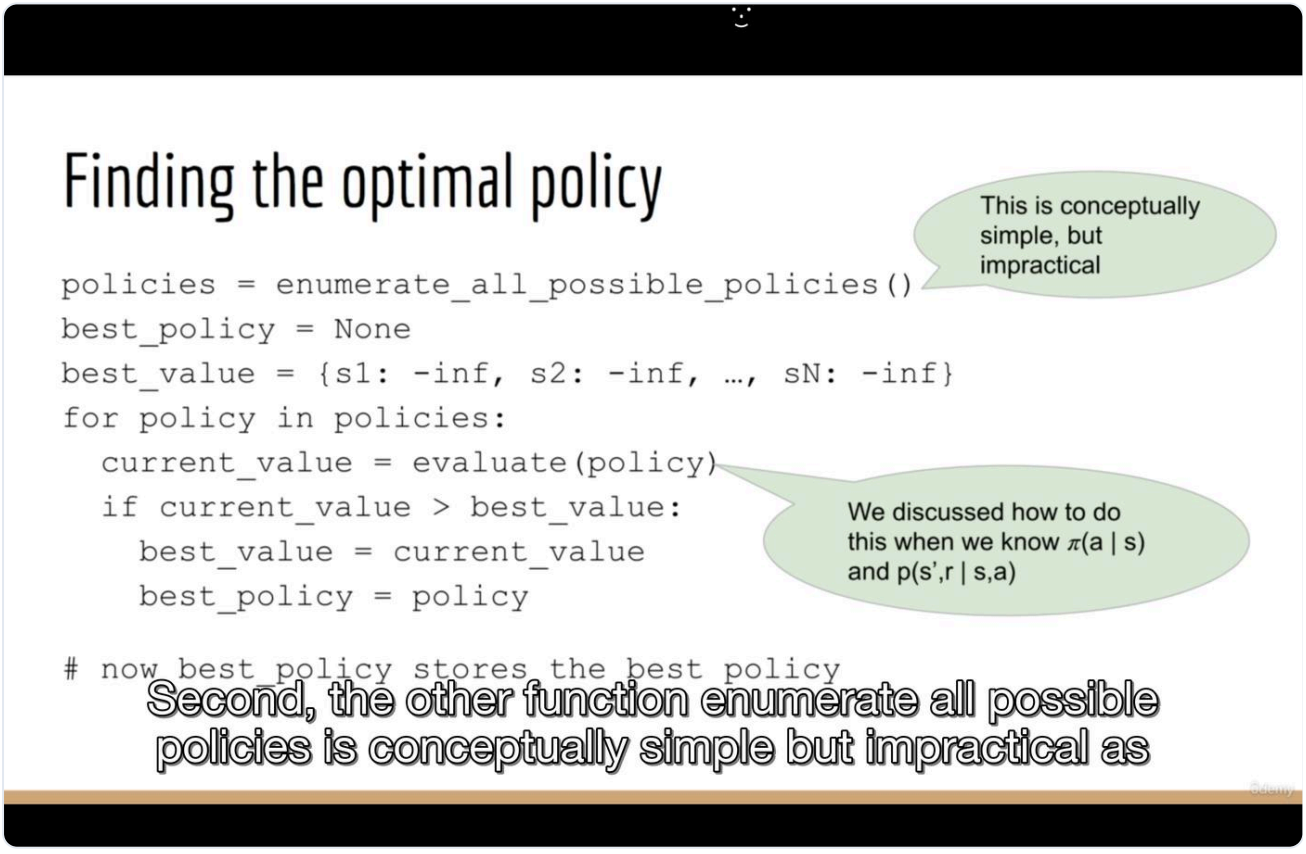

FIGURE 1.13 | Finding the Optimal Policy Rhs Is Conceptual

Suppose we are playing a game like Grid World or tic tac toe where our state space and action space are both finite. In this case, a naive search can solve this problem. First, let's create a list of all possible policies that can exist, obviously, some may be bad and some may be good, but one or more of them can be defined to be the best. Then in a loop, we can test each policy, so first we call a function to find the value for the current policy. So that's the evaluate function. Remember that as we discussed earlier, this is just a linear algebra problem. Then once we have the value function, we can compare it to our current best value function. If the new value function is better, then we make this the new best value function and make this policy the new best policy.

When we are done looping through all the policies, we have found the optimal policy. In the next lecture, we'll discuss these two functions here in greater detail. You may have noticed that we don't yet really know how to implement them. What I would like you to know now is this. First, the evaluate function. Finding V of s given a policy is not all that hard. In fact, we already discussed how we would do this. If you knew both the policy distribution and the environment dynamics, it becomes a simple system of linear equations. In the next lecture, we'll look at a more practical way of implementing this. Second, the other function enumerate all possible policies is conceptually simple but impractical as an exercise, you may want to try implementing this in code for a simple example like Grid World.

In other words, while this method here appears to be nice and simple, it is not actually practical to do.

LECTURE 95

The Solution

In this lecture, we are going to finally dive into some real reinforcement learning algorithms. Previously, we looked at some simple and naive solutions to reinforcement learning. Let's recap what they were.

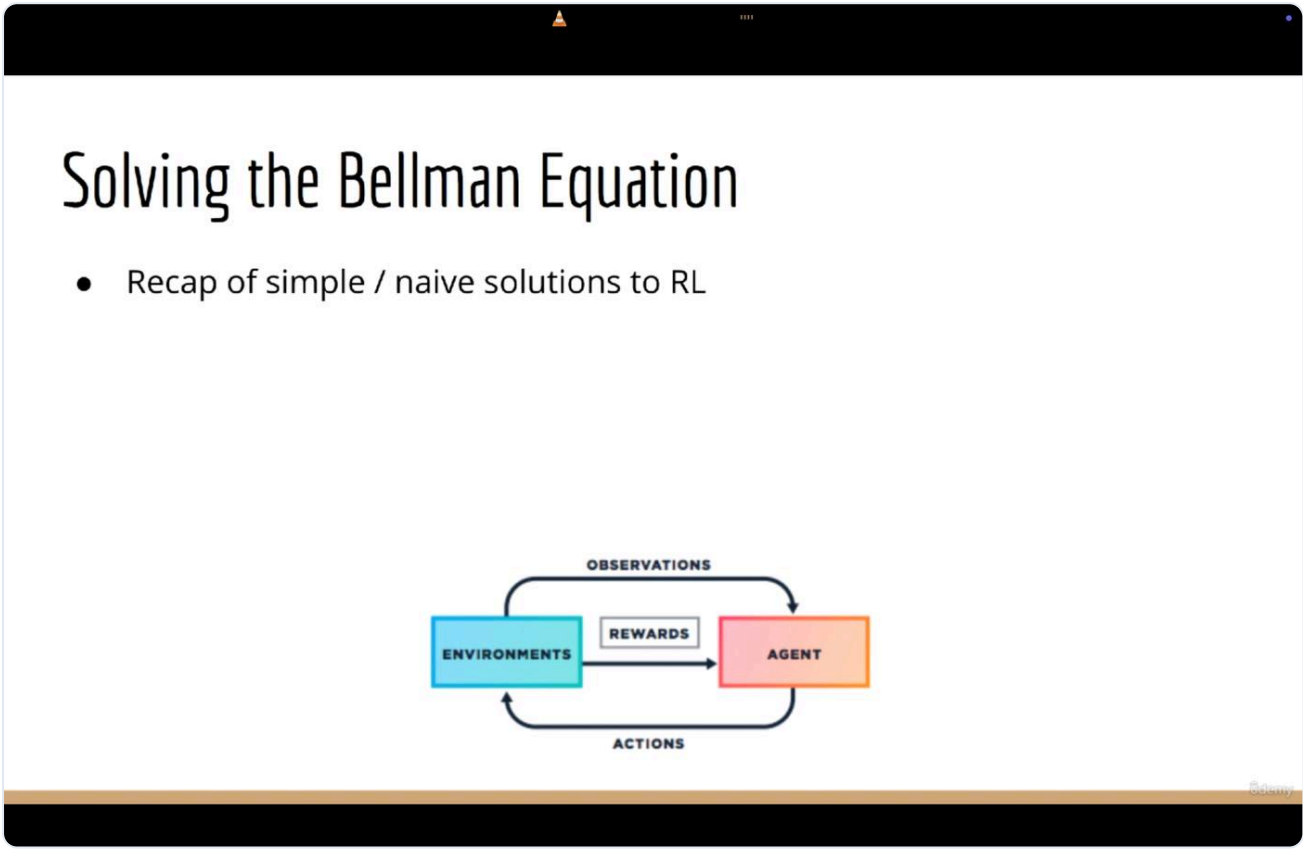

FIGURE 2.1

First, we recall that there are two types of problems, the prediction problem and the control problem. The prediction problem means, given a policy, find V of s. The control problem means find the best policy, which yields the maximum V of s to solve the prediction problem.

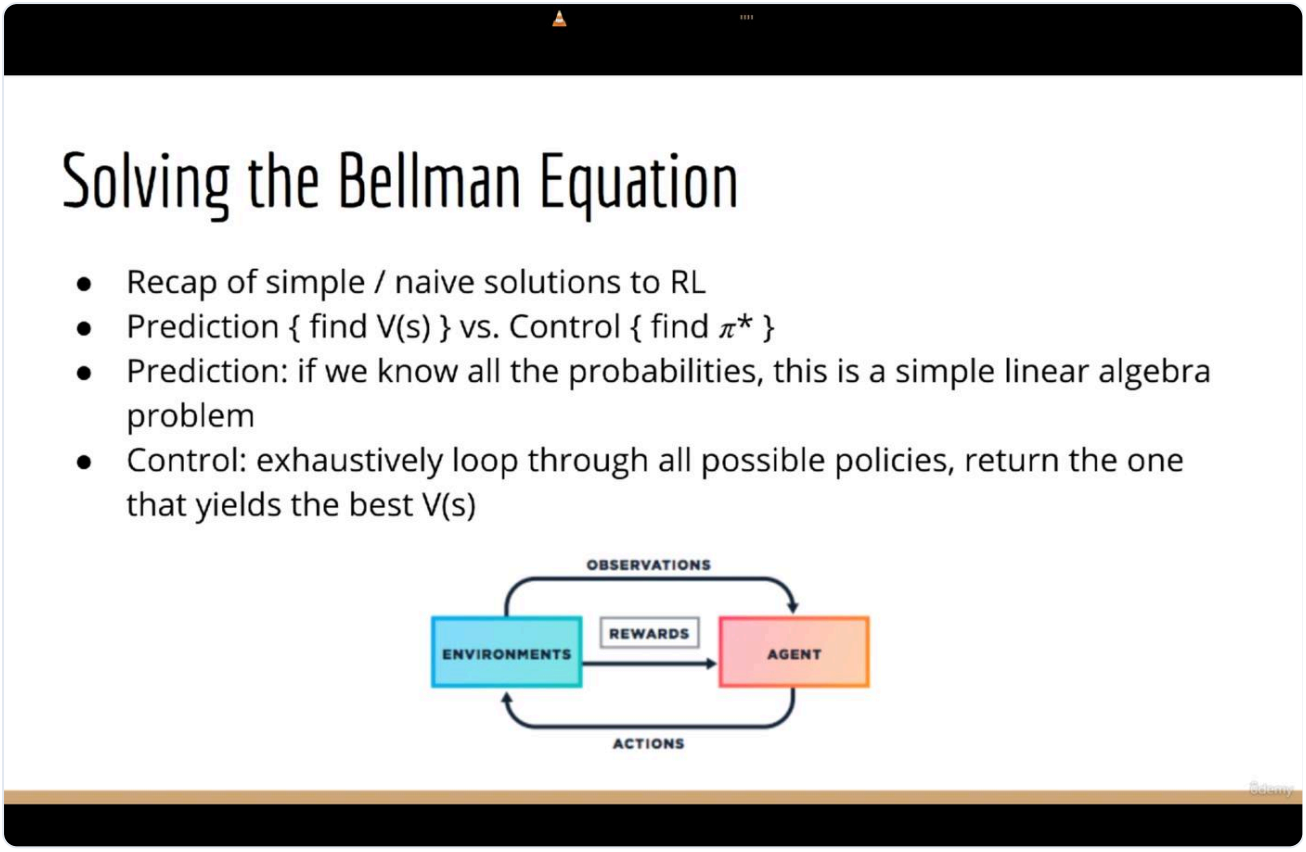

FIGURE 2.2

We noted that if we have the policy distribution and the state transition probabilities, it becomes a simple linear algebra problem. There are many algorithms and functions we can use to solve a system of linear equations for the control problem. We considered the naive method of simply looping through all the possible policies and finding which one is the best policy. Let's consider now why both of these approaches are somewhat unrealistic. First, let's consider the prediction problem. Previously, the solution required us to know both the policy distribution and the environment dynamics. But the reality is, if you imagine, say, any Atari game, we won't know the environment dynamics.

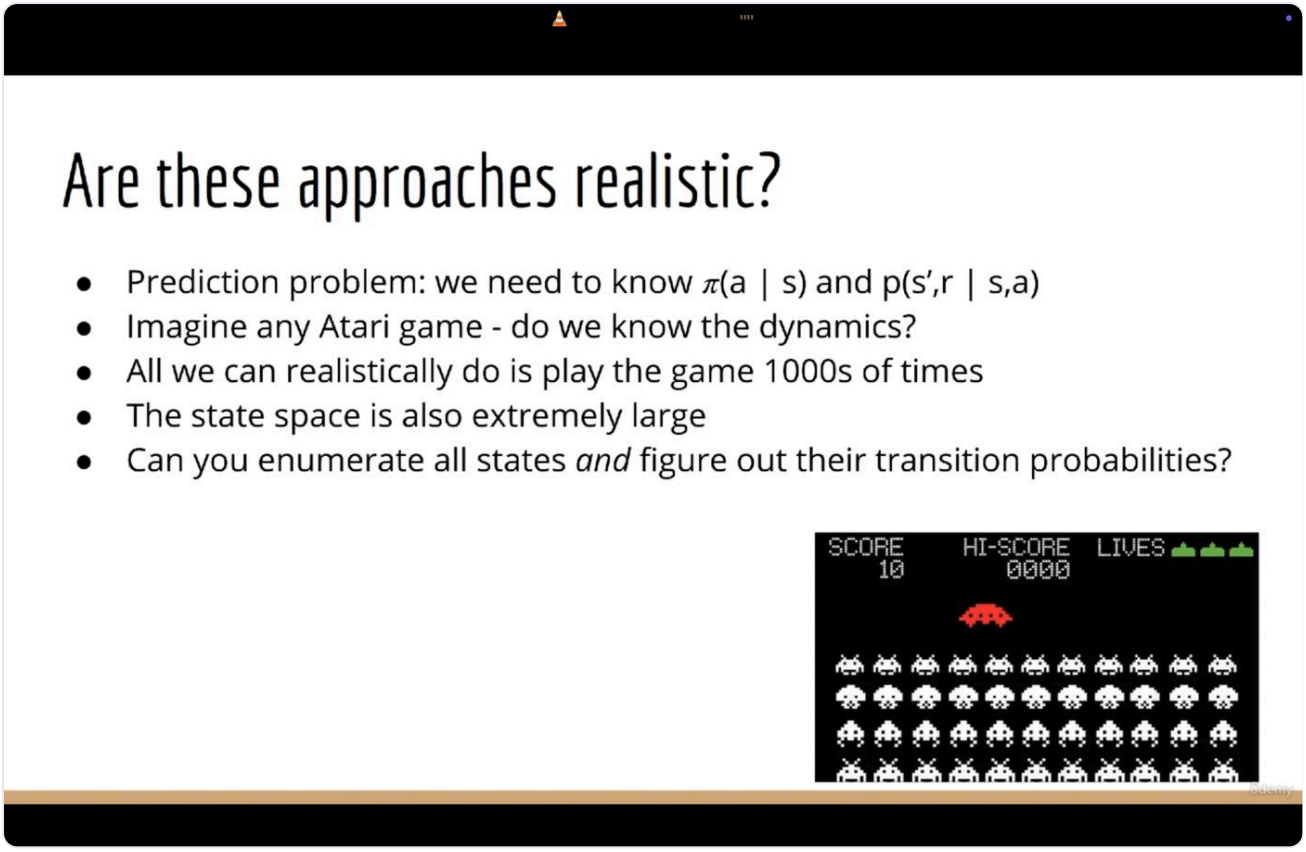

FIGURE 2.3 | Are These Approaches Realistic?

All we can really do, practically speaking, is play the game thousands of times. But usually the state space is so large that it would be infeasible to measure the state transition probabilities. And so unless you're playing with a toy environment, you would not be told these probabilities either. Now, let's consider the control problem, is it really possible to enumerate all possible policies?

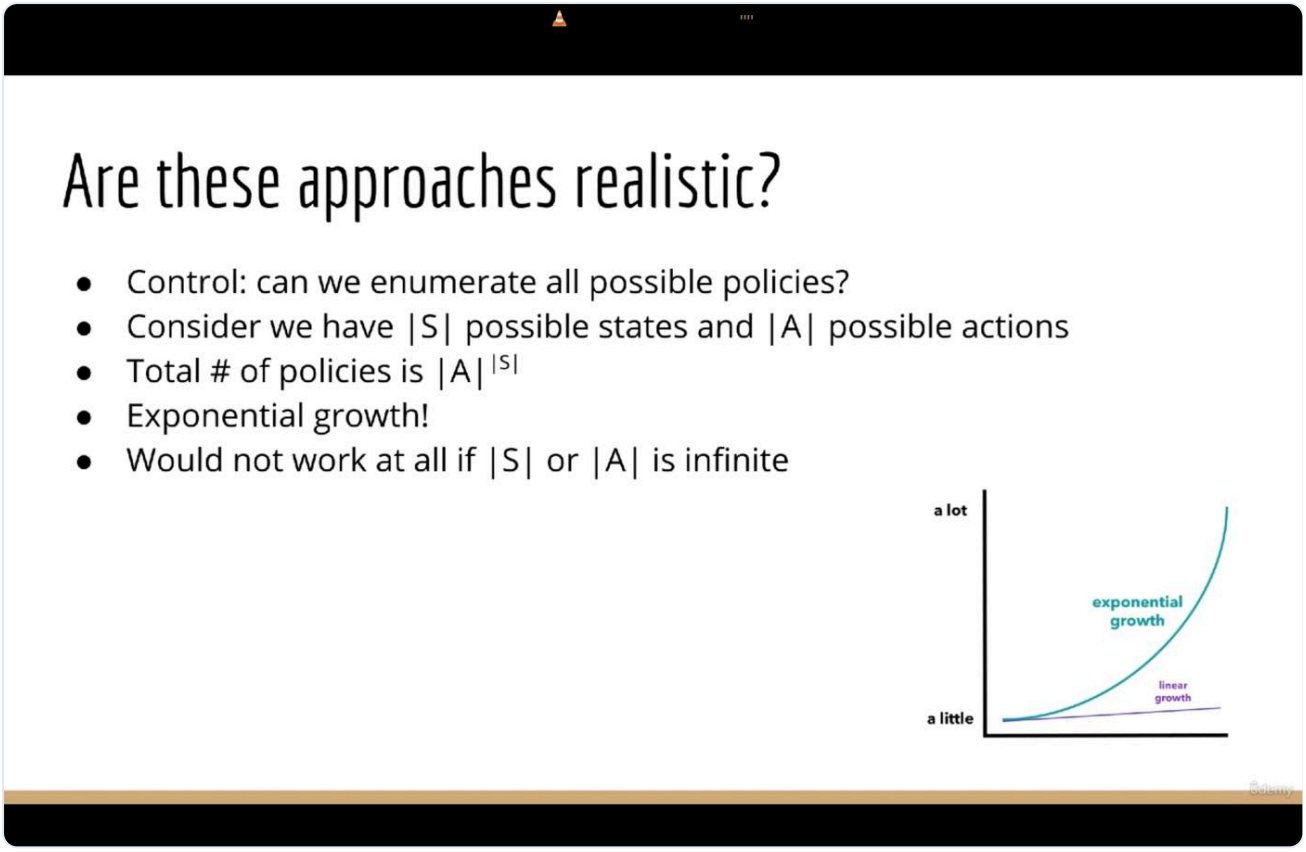

FIGURE 2.4 | Are These Approaches Realistic?

Consider if we have big S possible states and big A possible actions in this case, the total number of possible policies is big A to the power big S. In other words, this grows exponentially and hence it is not feasible to enumerate all possible policies for most practical problems. Furthermore, this would not even work in the case where the state space or action space is infinitely large. All right, so now that we've realized there is a problem, what is the solution? The trick to this is to remember the relationship between the expected value and the mean. The problem with the expected value is that it requires us to know the probability distribution of the random variable in question.

But importantly, there is a way for us to estimate the mean or equivalently the expected value. This is called the sample mean. This is the basis for many scientific experiments.

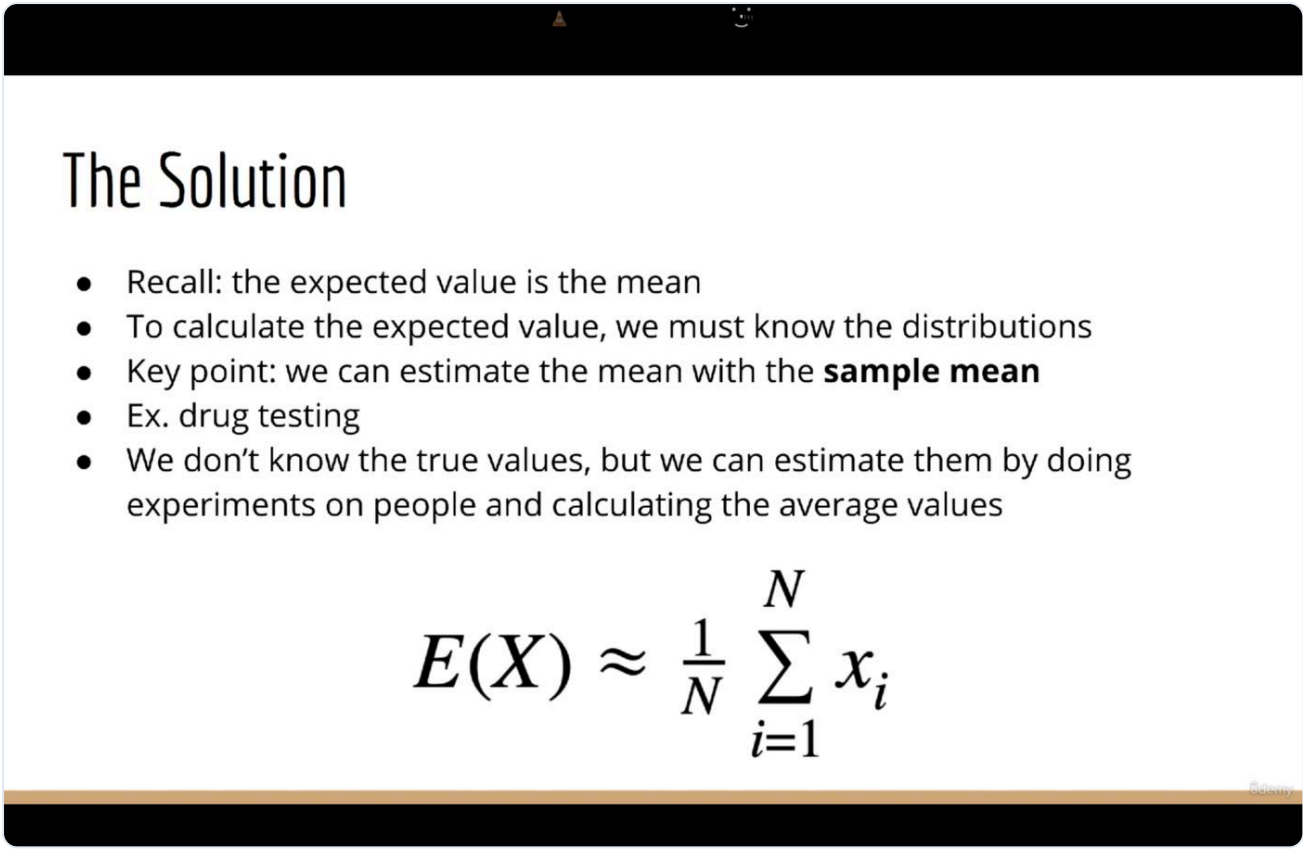

FIGURE 2.5 | The Solution

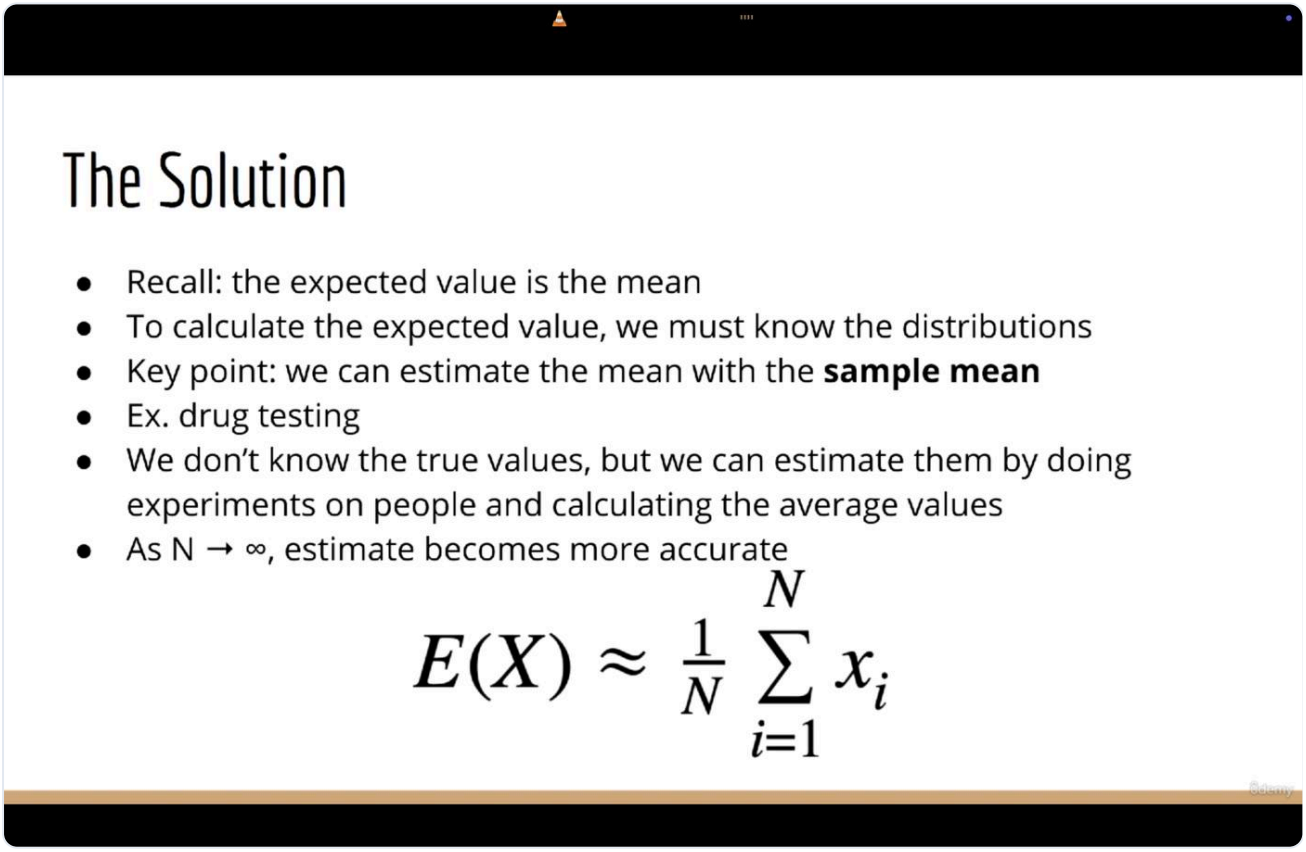

FIGURE 2.6 | The Solution

For example, if we want to test a drug, we don't know the true values, but we can do an experiment on a set number of people and calculate the average values. The sample mean is simply the sum of all the samples we collect divided by the number of samples. The idea is that as N approaches infinity, this estimate will become more and more accurate. OK, so what does this have to do with reinforcement learning? Well, remember that the value function is simply the expected return. Therefore, using the sampling approach, what we can do is sample a set of returns for each state in the state space and take the average that will give us an estimate of the value of each state.

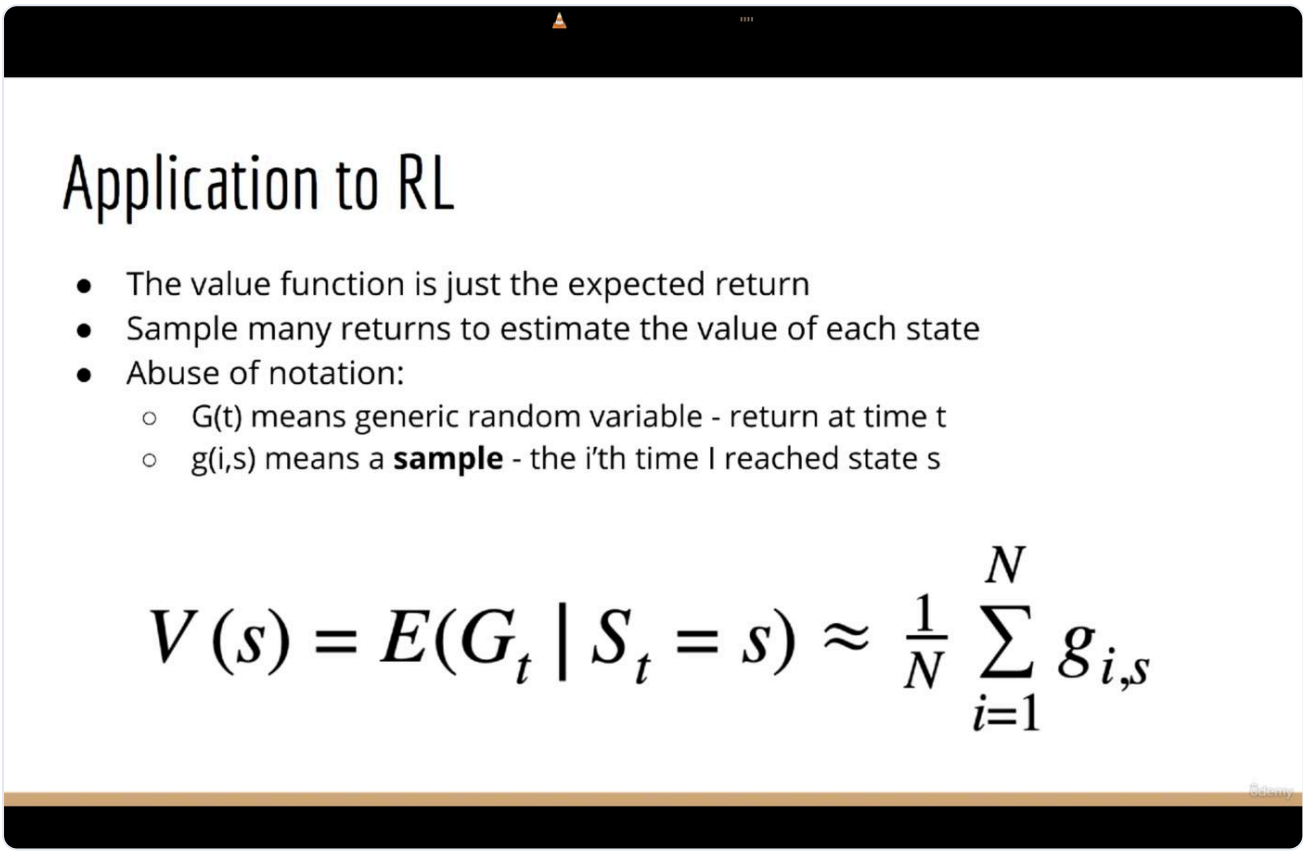

FIGURE 2.7

You'll notice that I've abused notation a little bit here by using different indices for the return g. When I say G of T, I mean a generic return G the return for the state at time t, but when I index G using I and s what I mean is this is a sample of the return. It's the ith sample return from the state s. Now, one obvious, but maybe not so obvious point, where do these samples come from? You may recall that when we're in NumPy and we want a sample from, say, the standard normal, we can simply call a function like np dot random dot randn. But what does it mean to sample a return?

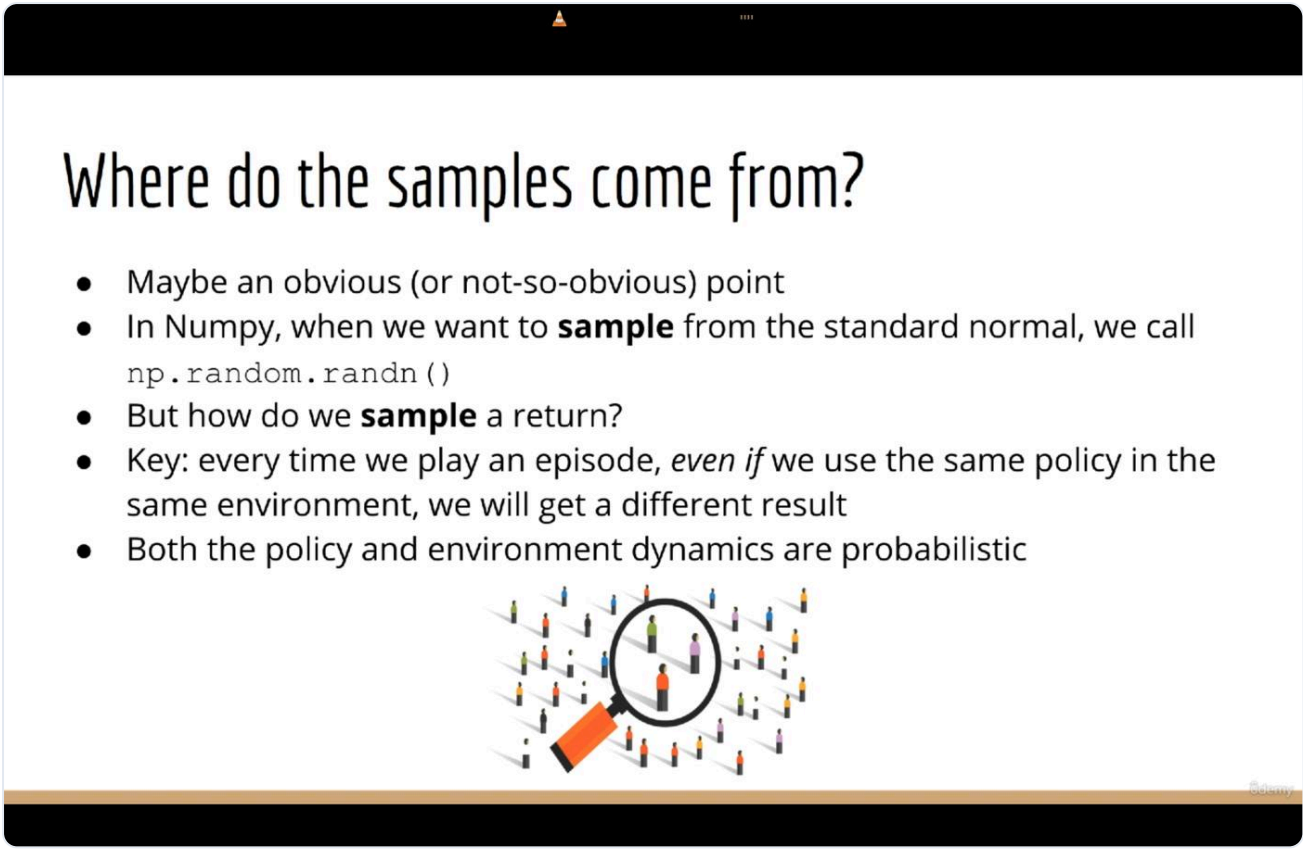

FIGURE 2.8 | Where Do the Samples Come from?

Well, remember that when we play an episode, even if we use the exact same policy and the exact same environment, the result will be different. This is because both the policy and the environment dynamics are probabilistic. So just playing the game itself results in collecting samples.

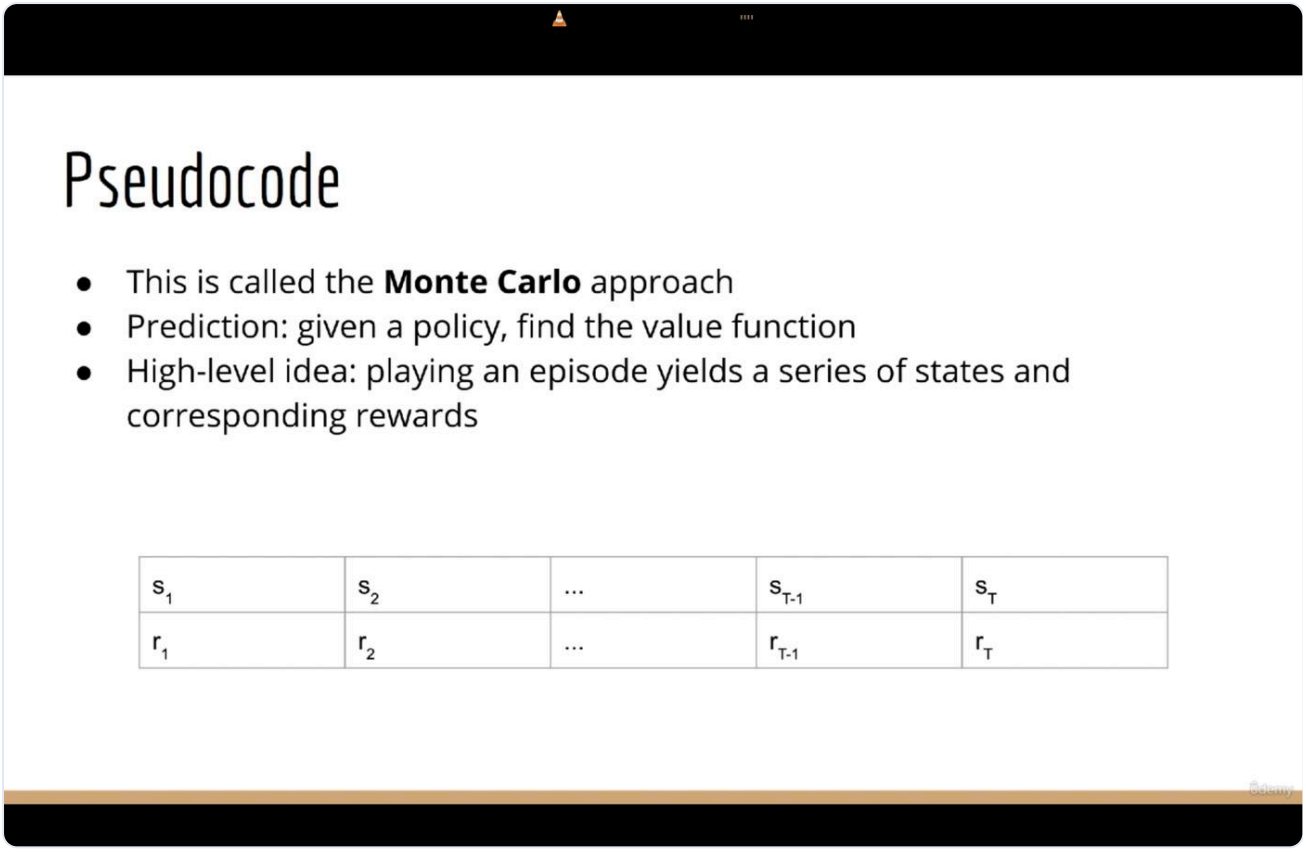

FIGURE 2.9 | Prediction: Given a Policy, Find the Value Function

Using this concept, let's now attempt to write some pseudocode to describe this algorithm, by the way, this approach is called the Monte Carlo approach, since this is a form of Monte Carlo sampling. First, we're going to describe the prediction problem or in other words, given a policy, find the value function. Let's just think about this at a high level. First, suppose we play one episode of a game. This consists of entering a series of states and corresponding rewards so we can call them S1 up to ST and R1 up to RT from this. How do we calculate the return of each state? Well, it's helpful to actually go backwards, for example, G of big T is zero importantly, note that the return is only the sum of future rewards.

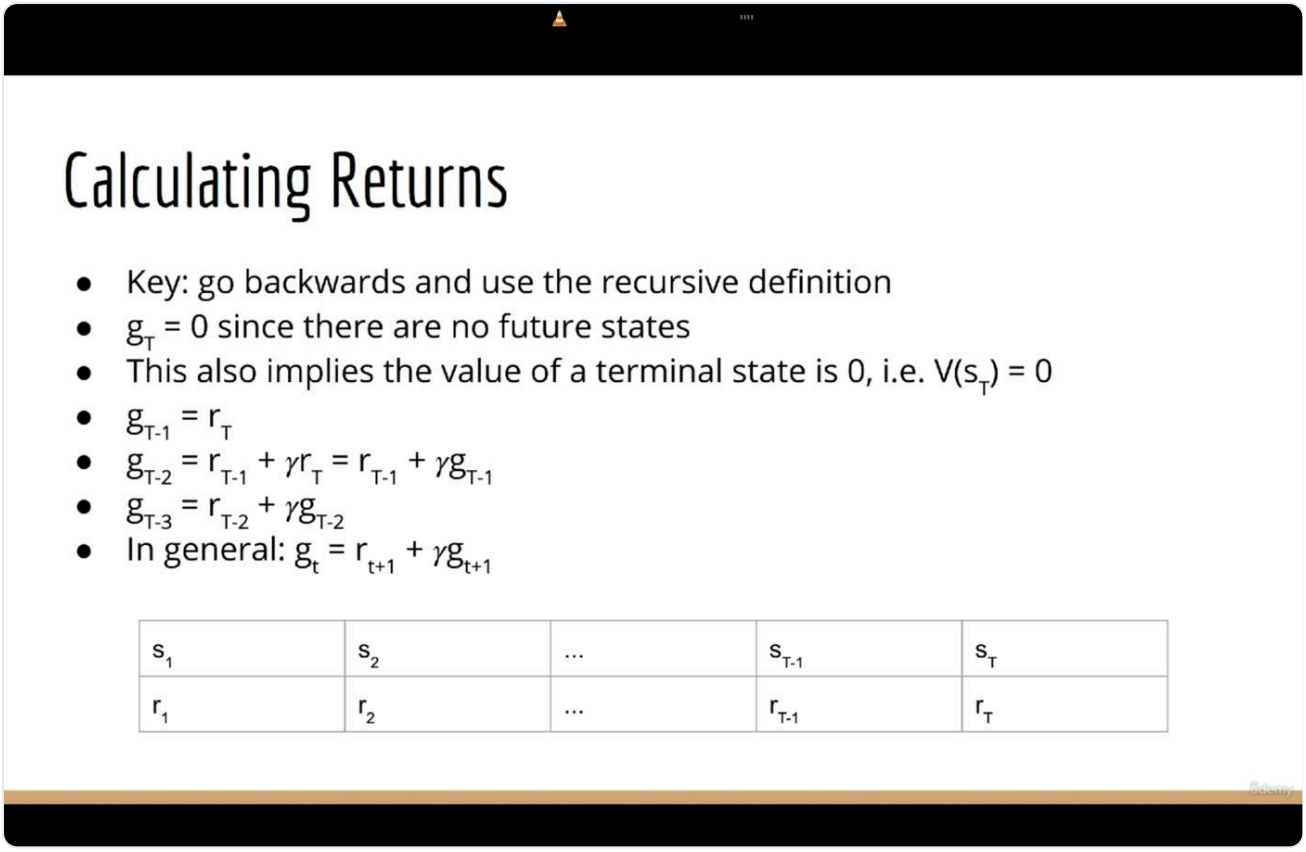

FIGURE 2.10 | Key: Go Backwards and Use the Recursive Definition

Since a terminal state is the end of an episode, that means there are no future states, no future states means no future rewards. Therefore, the return of a terminal state is always zero and hence the value of a terminal state is also always zero. In any case, let's continue. How do we calculate the return of the previous timestep? Well, by definition, G of Big T minus one is equal to R of Big T.. OK, the next one, how about G of big T minus two, it's G of Big T minus two equal to R of Big T minus one plus gamma times R t. But importantly, remember that the return is recursive. So this is also equal to R of big T minus one plus gamma times G of Big T minus one.

We can repeat this pattern. So G of Big T minus three is equal to R of Big T minus two plus gamma times G of Big T minus two. And of course this follows for any value of T, so we can say G of T is equal to R of T plus one plus gamma times G of T plus one. So you can keep repeating the same pattern until you calculate the return for each state.

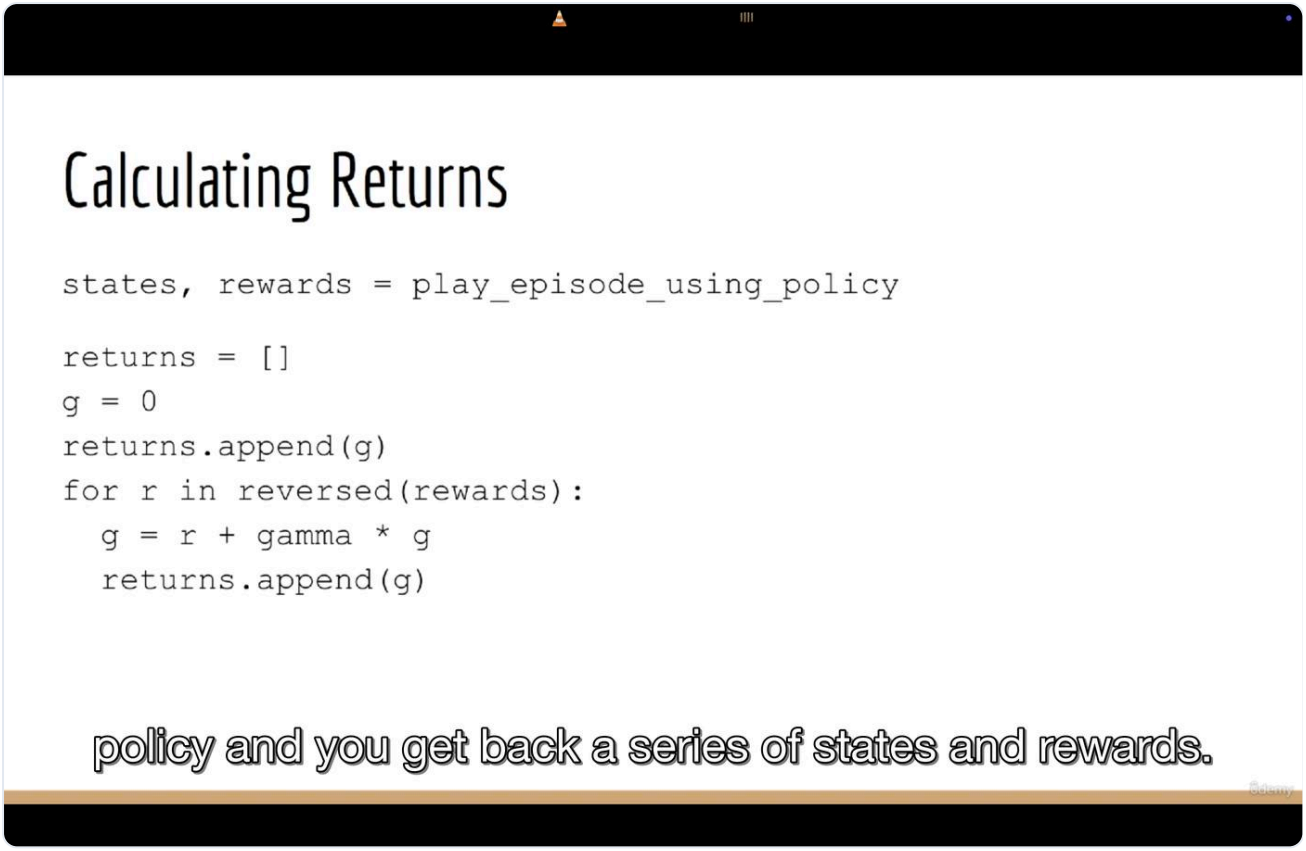

FIGURE 2.11

Practically speaking, here's how you would do it in code first, you would play an episode using your policy and you get back a series of states and rewards. Next, we initialize a list of returns to an empty list and we initialize the return to zero then and this is the important part. You loop through the rewards in reverse inside the loop. We just use the recursive definition of G. G equals R plus gamma times G.

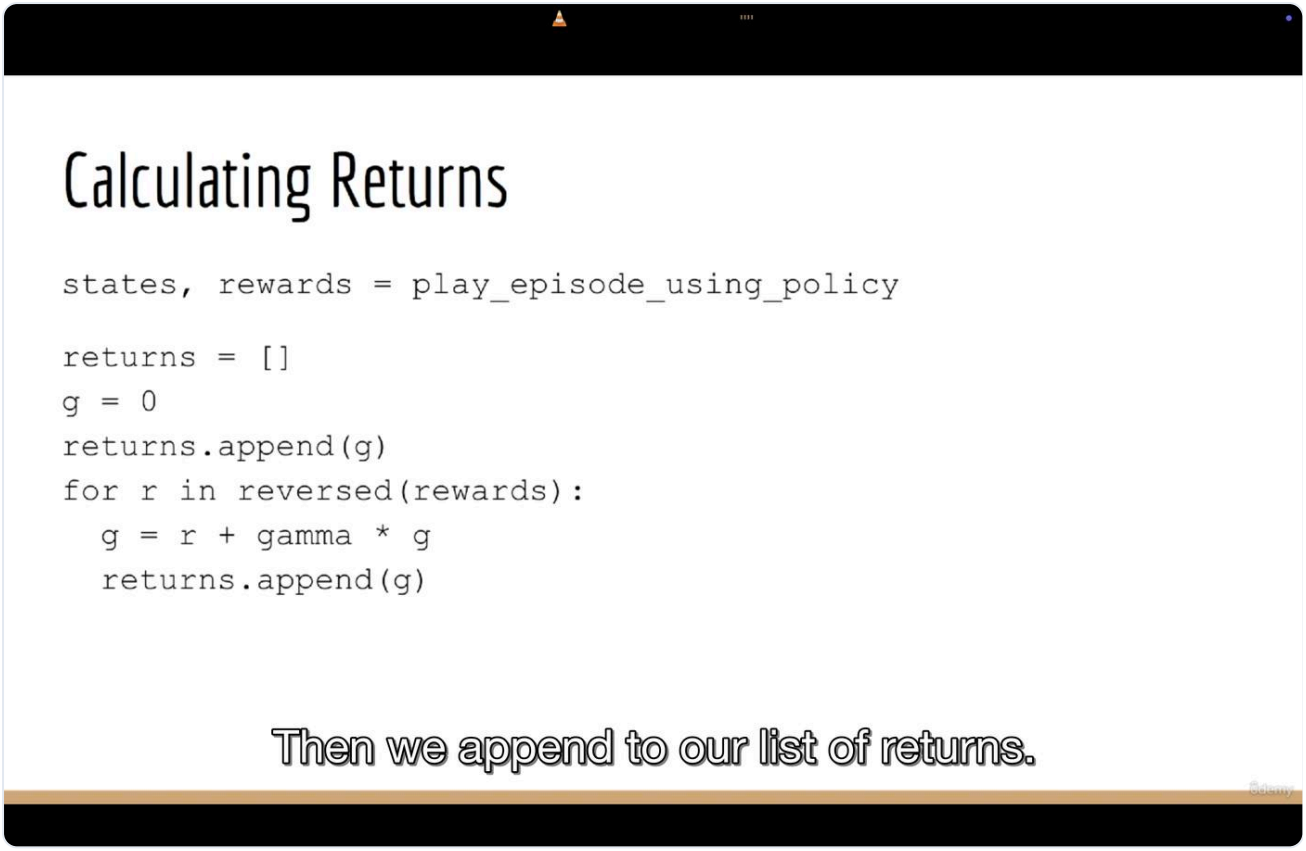

FIGURE 2.12

Then we append to our list of returns. All right, so how do we put the above algorithm into pseudocode? Well, that's simple. Everything we just did, now we just do it in a loop, do the thing we just did a several hundred or several thousand times then for each state, take the average return. First we assume we are given some policy. Then we instantiate an empty dictionary to store our sample returns. The key of this dictionary will be the state and the value will be a list of returns encountered throughout each episode.

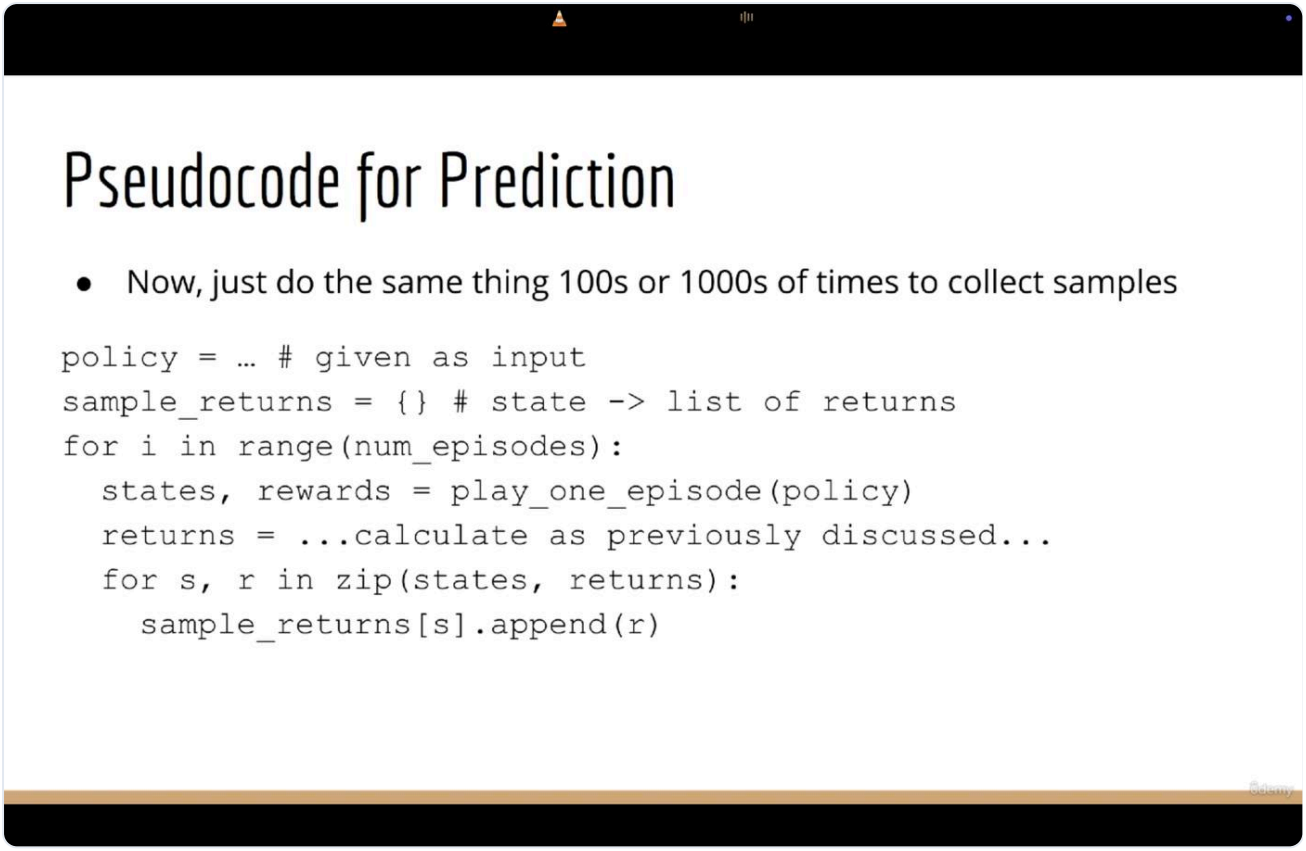

FIGURE 2.13 | Pseudocode for Prediction

Then we enter a loop which goes on for a predetermined number of episodes. Inside the Loop, we first play one episode using the given policy. This returns us a list of states and corresponding rewards. From that, we can calculate the return for each state as previously described. Next, we loop through each state and return in corresponding order for each return we encounter. We append that to the list of returns for that state. This loop is how we collect our samples.

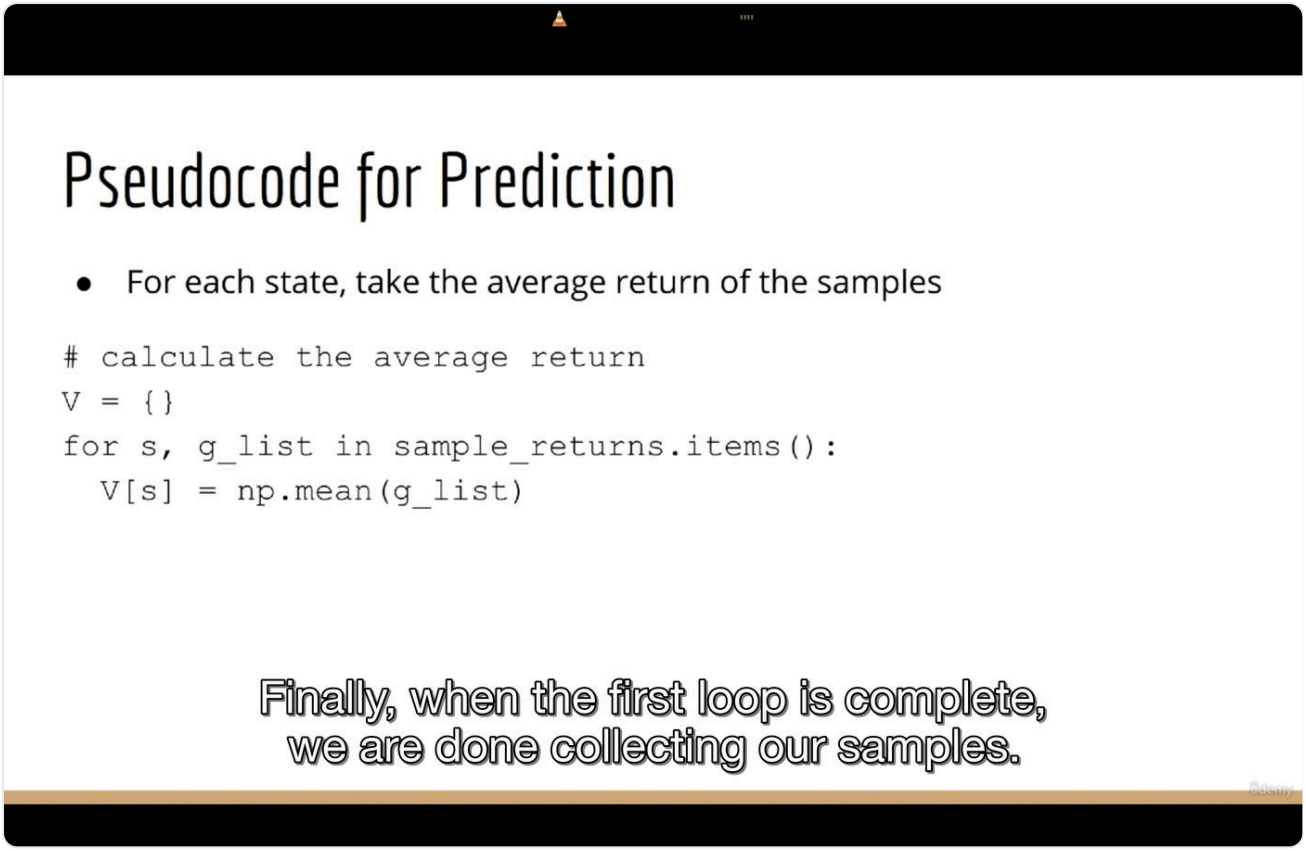

FIGURE 2.14 | Pseudocode for Prediction

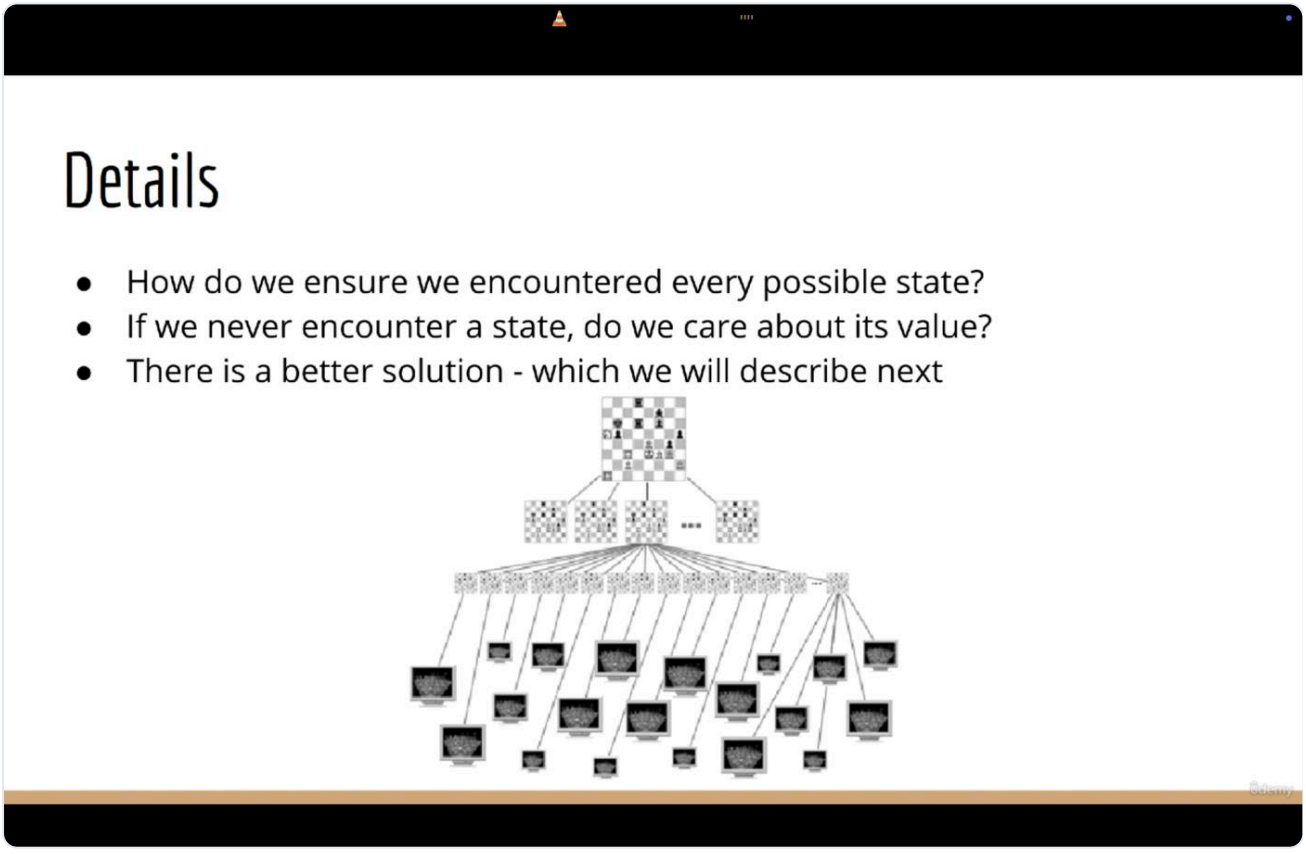

FIGURE 2.15 | How Do We Ensure We Encountered Every Possible State?

Finally, when the first loop is complete, we are done collecting our samples. Now we can calculate the average return, which is by definition the value function. So we loop through each of the items in our sample returns dictionary for each iteration. We get the state s and a list of sample returns G list inside the loop V of s is simply assigned the sample mean of G list. It's important to realize that there are some complications with the algorithm as described first, because we're only sampling, how do we ensure that we actually encounter every possible state in the number of episodes we played? In fact, we cannot, although we may surmise that because we didn't encounter a particular state, we don't need to know its value because our policy does not allow us to go there.

You could simply ignore the value of those states in any subsequent function you plug the value function into, but there is a better solution to understand it. We're going to move on to the second problem, the problem of control or finding the best policy.

LECTURE 96

Generalized Policy Iteration

As a general pattern. It note that for the prediction problem, usually we are interested in finding V of s, which is what we just did. For the control problem, we are usually interested in working with the action value Q of s a. The reason for this is, if you recall, the Q function makes it easy to look up what action to take given a state s. We always take the argmax over all possible actions. We call this the greedy action.

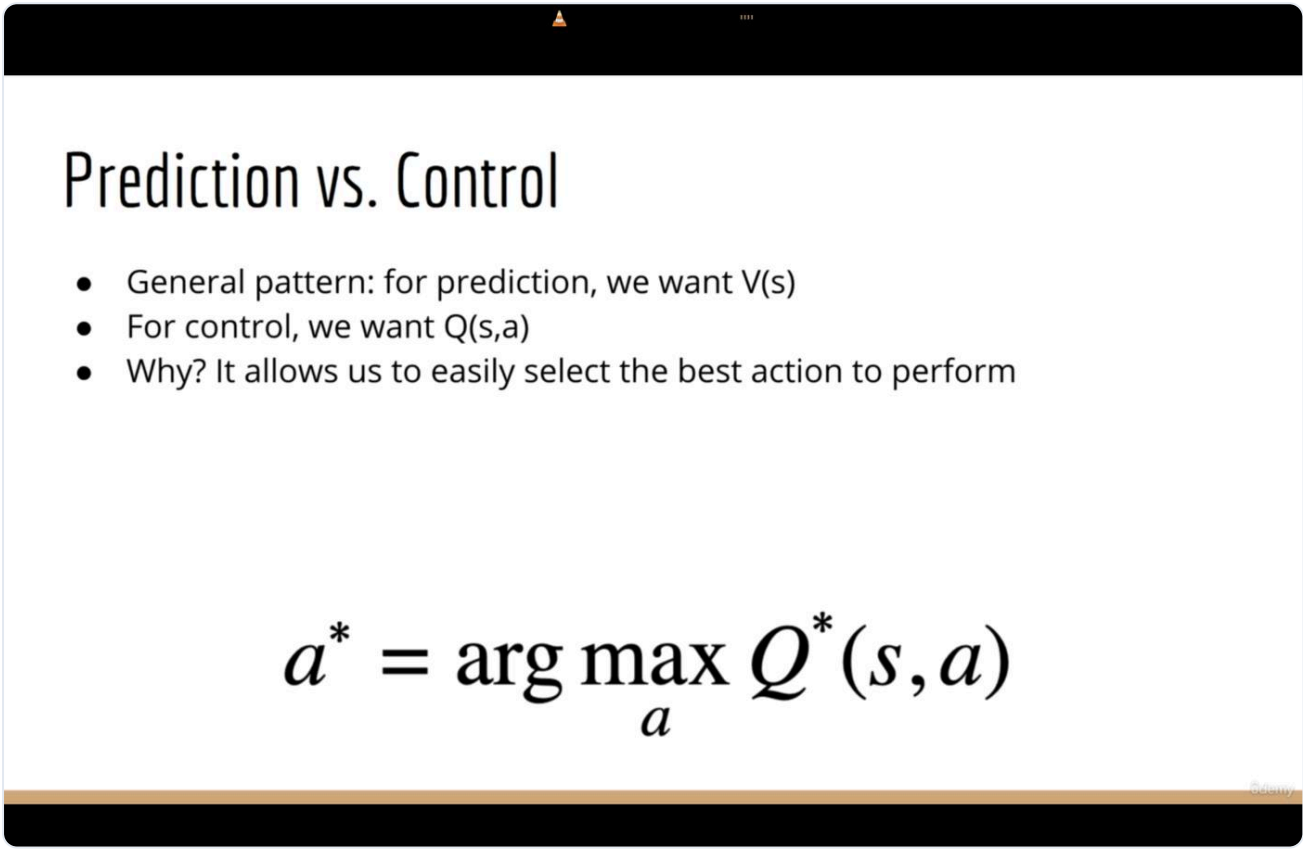

FIGURE 3.1 | Prediction vs. Control

In this way, we maximise what we believe to be the sum of future rewards. So just keep this general pattern in mind for the prediction problem. We work with V of s for the control problem we work with Q of s a. In order to understand how to apply the Monte Carlo approach to the control problem, we first have to understand the principle of policy, iteration and policy improvement. Let's consider two basic facts.

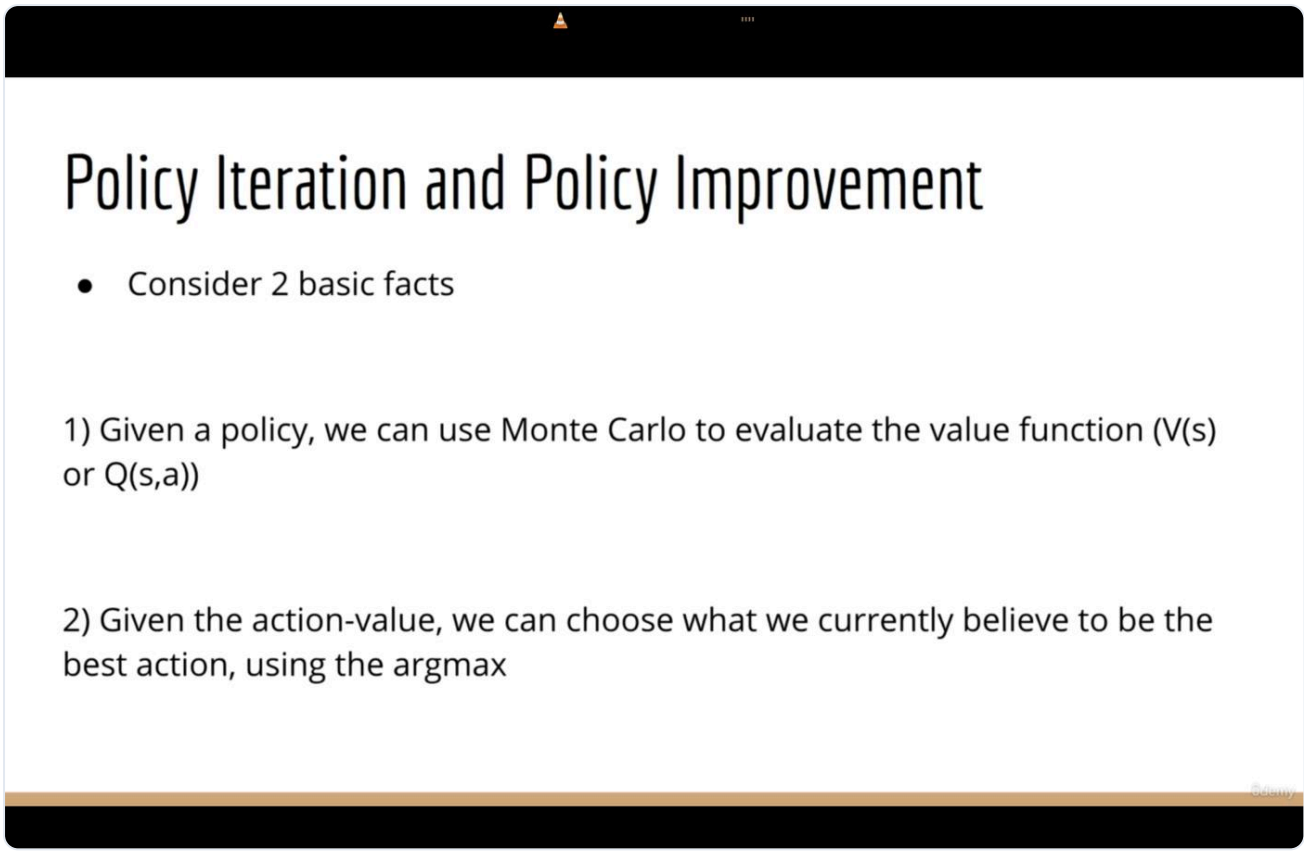

FIGURE 3.2 | Policy Iteration and Policy Improvement

Number one, given a policy, we can use Monte Carlo to evaluate the value function, whether that be the state value or the action value. We just saw that in the previous lecture. Number two, given an action value function, we can always choose based on this what we believe to be the best action given the current state. This is just the argmax over all possible actions A. Well, it turns out that these two facts are interdependent. Given a policy, we can find its corresponding action value. And from that, we can take the argmax to find a possibly better policy. But what if this policy is different from the original policy?

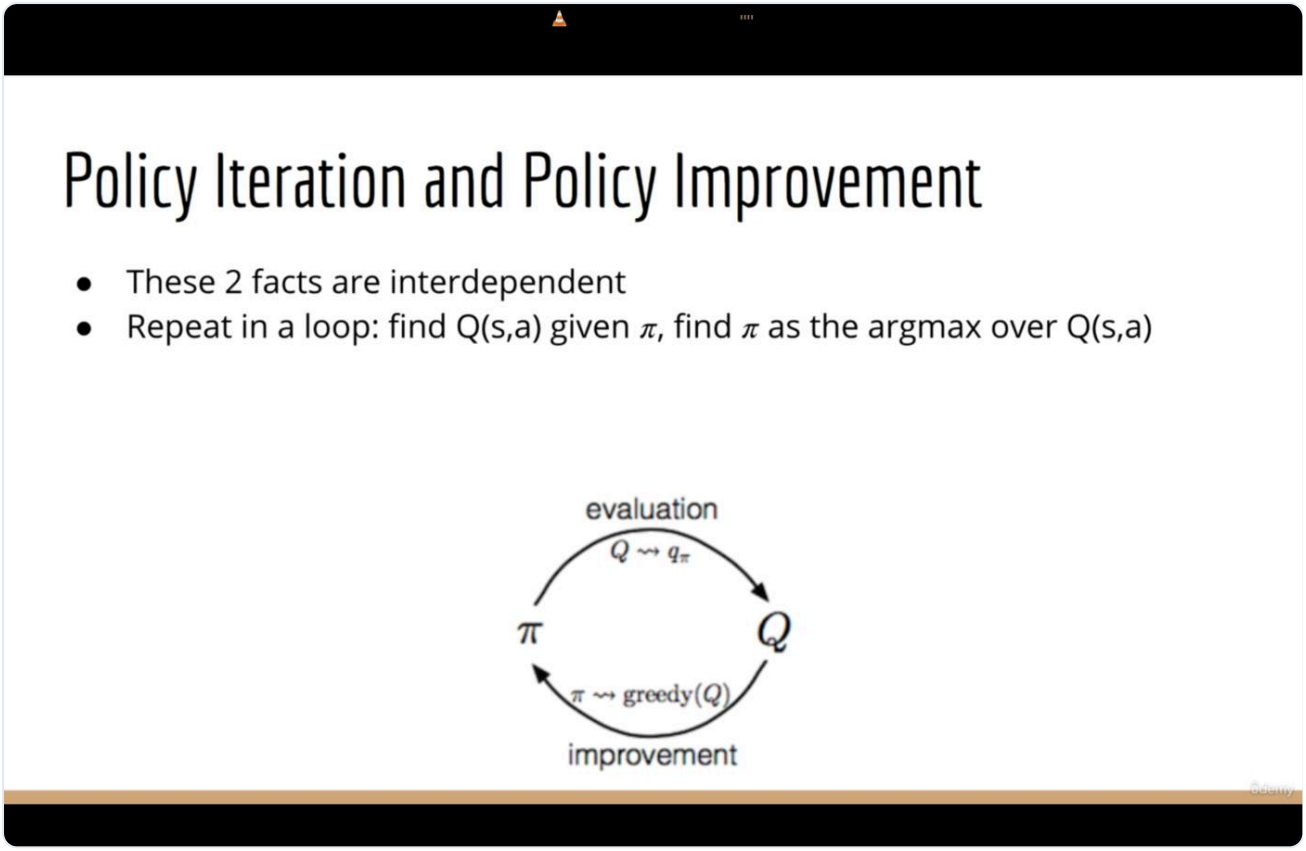

FIGURE 3.3 | Policy Iteration and Policy Improvement

Then we can find the value function for this new policy. And from that, we can take the argmax again to find yet another new and possibly better, but at least as good policy. From there we can find the value function again. So you see, this is just a loop where we go back and forth, finding the value function, a given a policy and improving the policy. Given that value function. You can see that we have named these two steps back to finding the value function is called the evaluation step and the act of finding the best policy given that value function is called the improvement step. And just to be clear, the reason why this is a loop is because they both change each other.

So by doing step one, you change the policy and by doing step two, you change the value.

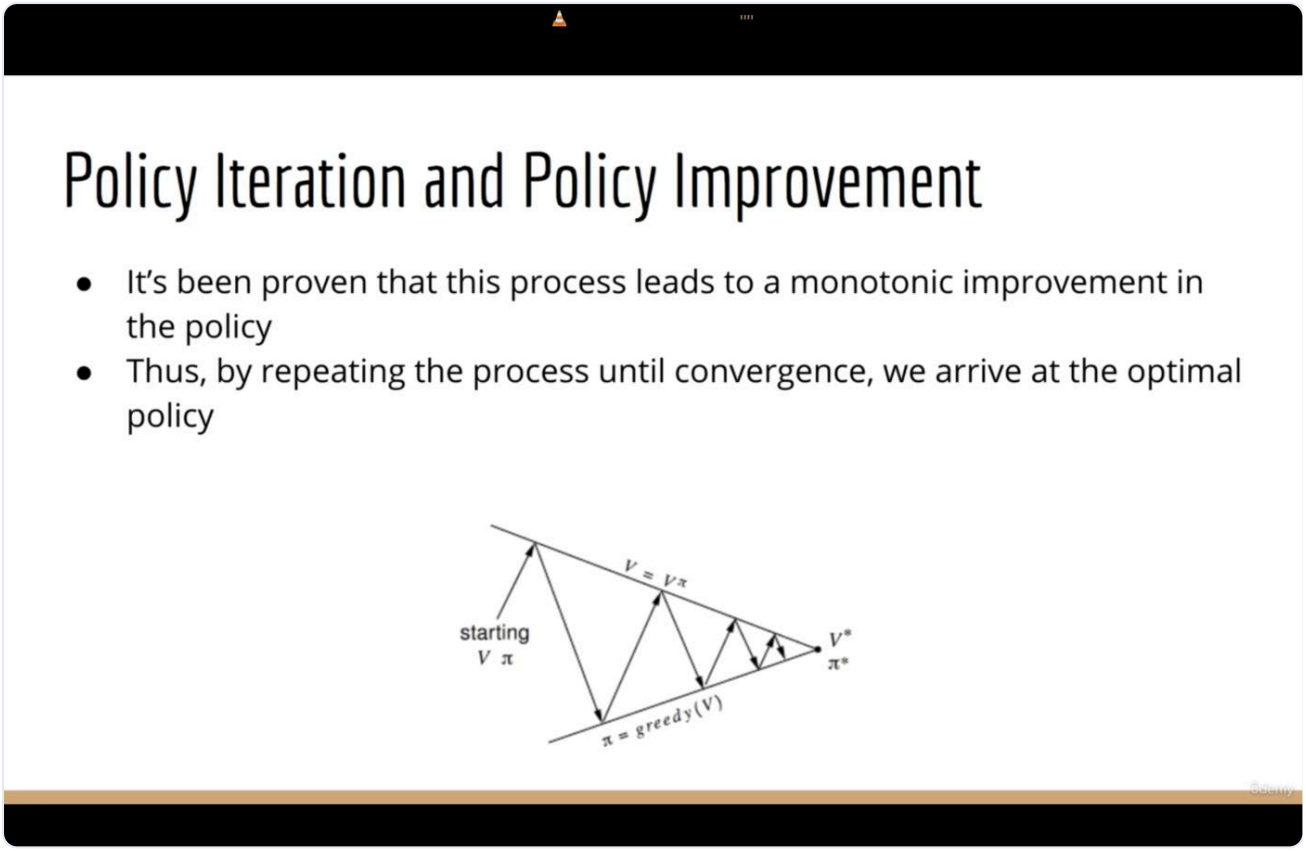

FIGURE 3.4 | Policy Iteration and Policy Improvement

It's been proven, although we won't discuss it here, that performing these steps leads to an monotonic improvement in the policy. Therefore, if we just keep doing this process, eventually we will end up at the optimal policy. So how can we apply this to Monte Carlo? Here's a rough outline. First, we start by initializing Q of s a and a policy to both be random. Next, we enter a loop that goes for a predetermined at number of episodes inside the loop.

FIGURE 3.5 | Monte Carlo Approach

We first evaluate the policy by finding Q of s a for the given policy. We call this the policy evaluation step. Once we've done that, we find a new policy where for each step, we take the action to be the argmax over all actions for Q of s a with a given state. This is called the policy improvement step. This is pretty simple and not unlike the prediction problem, except for one small difference. Note that earlier when we discussed the evaluation problem, we discussed how to find V of s, but now we have to find Q of s a in order to find Q of s a where we evaluate our policy. We need to keep track of not only the states and rewards, but also the actions.

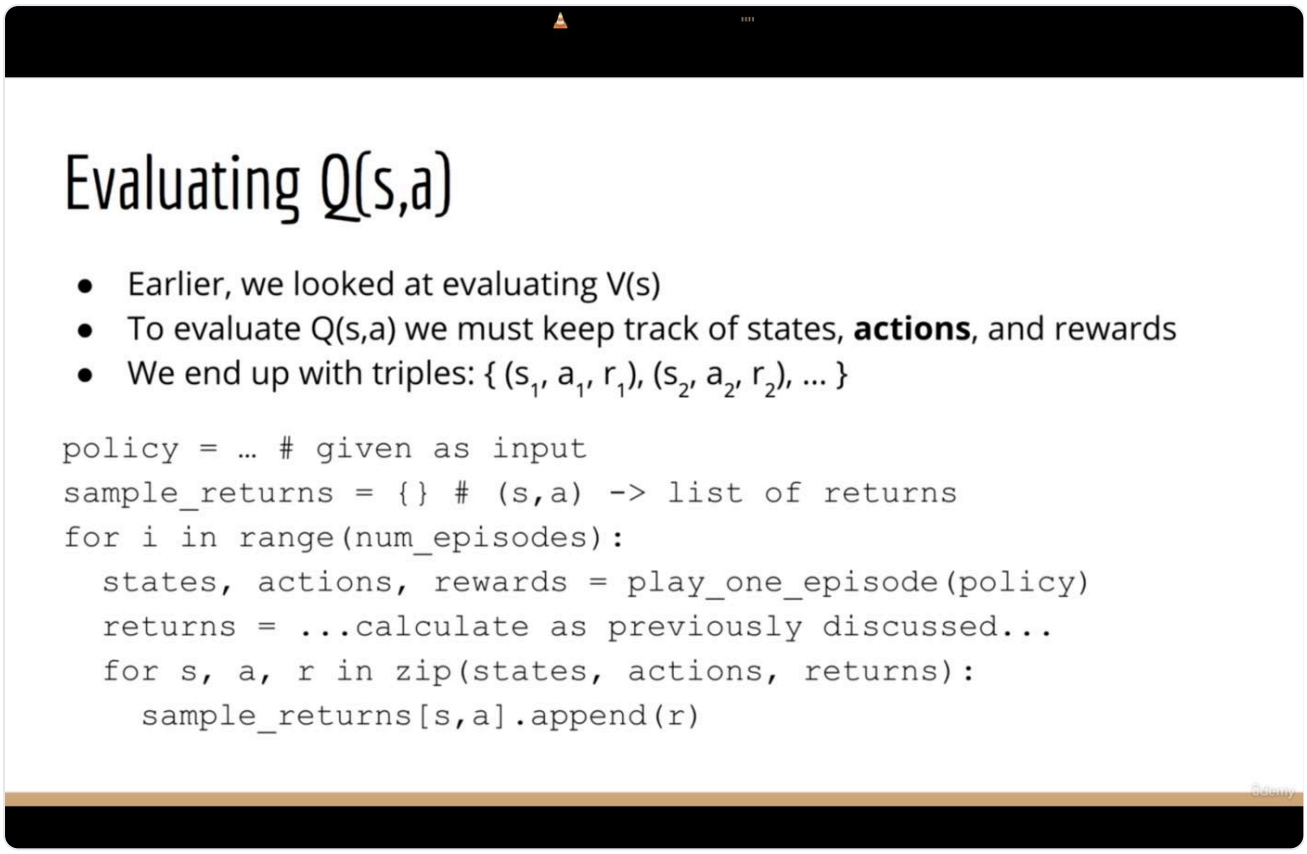

FIGURE 3.6

So we're going to record triples of states actions and rewards. So S1, A1, R1, S2, A2, R2, and so on. Here's what the pseudocode for the evaluation step would look like. As you can see, this is pretty similar to calculating V of s except when we store the sample returns. We index the dictionary by both state and action. In the second part. Q is again a dictionary, but now the key is a state action tuple.

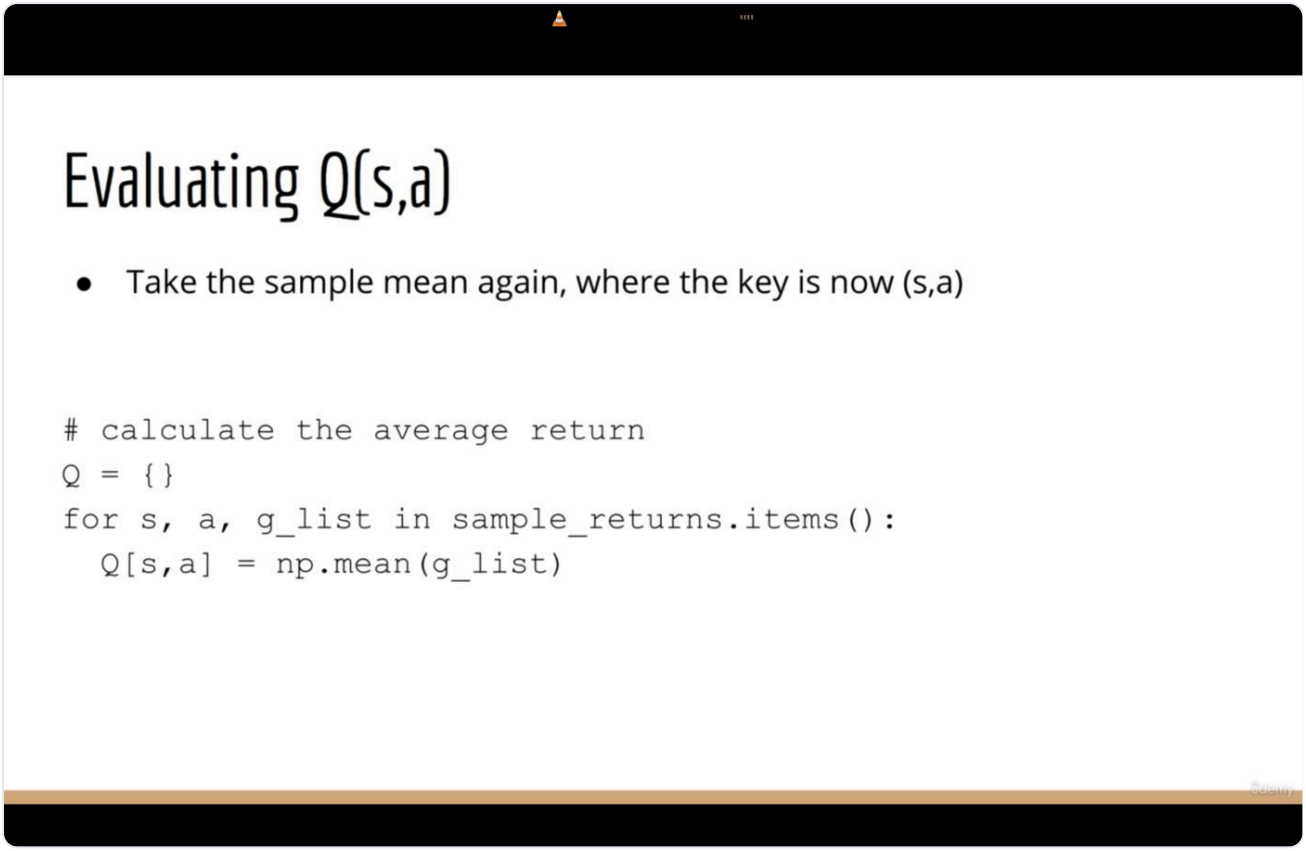

FIGURE 3.7

Other than that, the process is exactly the same. We still calculate the sample mean of each list of returns for a given key. At this point, our solution works, but it's not an ideal solution. Let's think about why. First, if you recall V of s only stores big S values but Q of s a stores big S times big A values. As you know, with Monte Carlo sampling.

FIGURE 3.8 | Improving Our Solution

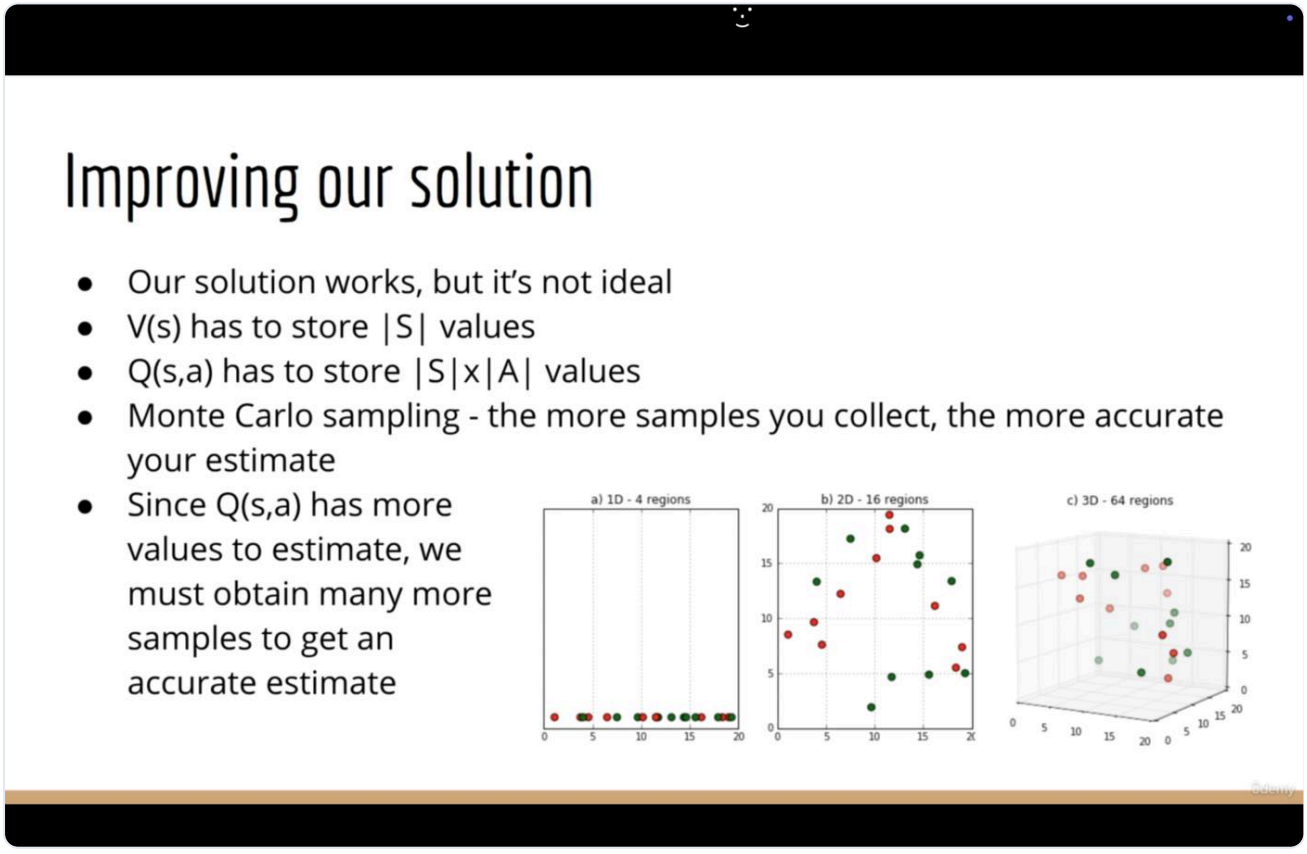

FIGURE 3.9 | Improving Our Solution

The more samples you collect, the more accurate your answer becomes. When we use Q of s a, we have many more values to estimate and therefore we have to collect many more samples in order to get an accurate estimate. But there is a second problem. Each policy evaluation step is a monte Carlo estimation, which means that we must calculate a large number of samples. But now we've nested this loop inside another loop. What's the effect of this? Well, let's say our policy iteration loop needs to run 1000 times to find the optimal policy. Now, let's say our inner loop needs to run 1000 times in order to get an accurate estimate of Q of s a. How many episodes do we have to play?

The answer is 1000 times 1000, which is 1 million.

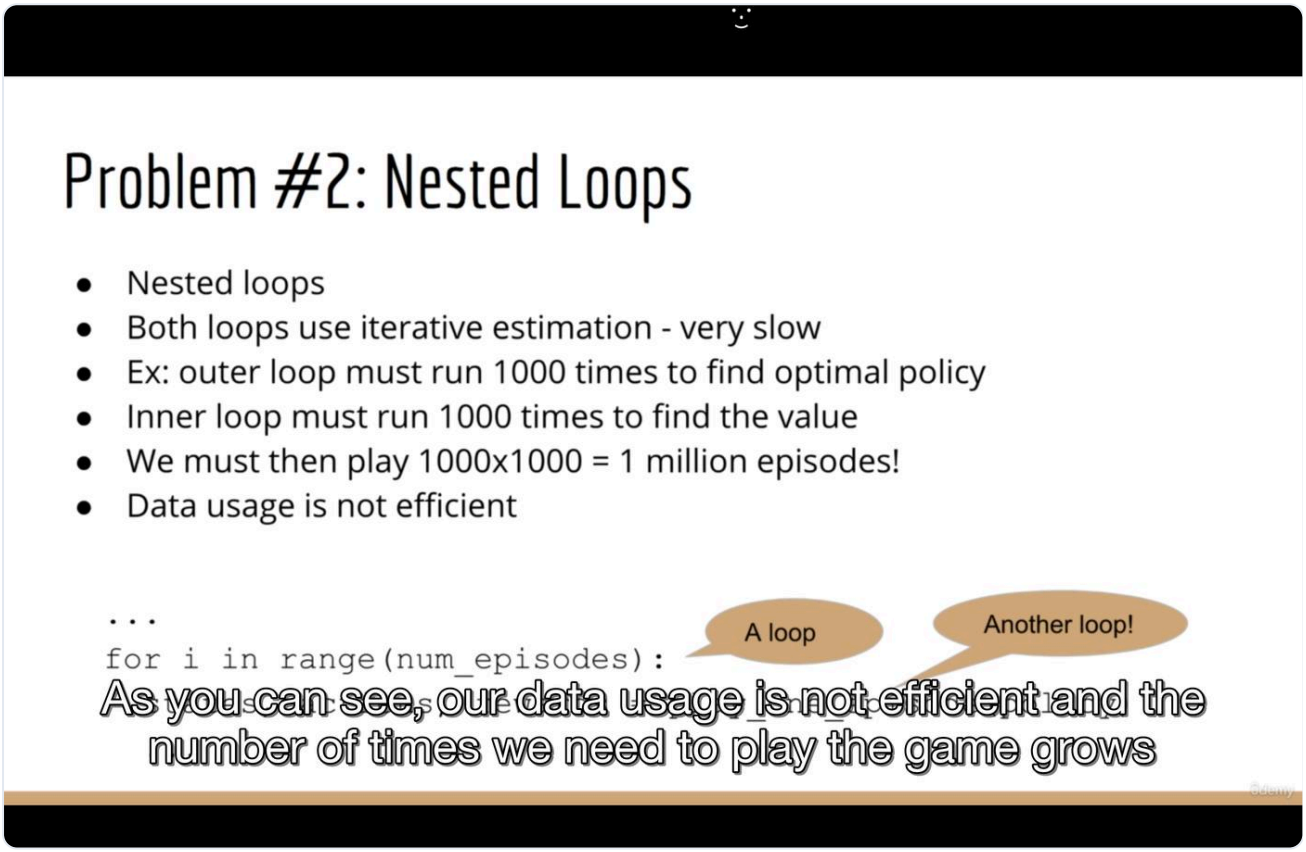

FIGURE 3.10

As you can see, our data usage is not efficient and the number of times we need to play the game grows quite fast. A better solution is to use what's called generalized policy iteration. At first, this approach may seem kind of wacky, but in fact it's been shown to work.

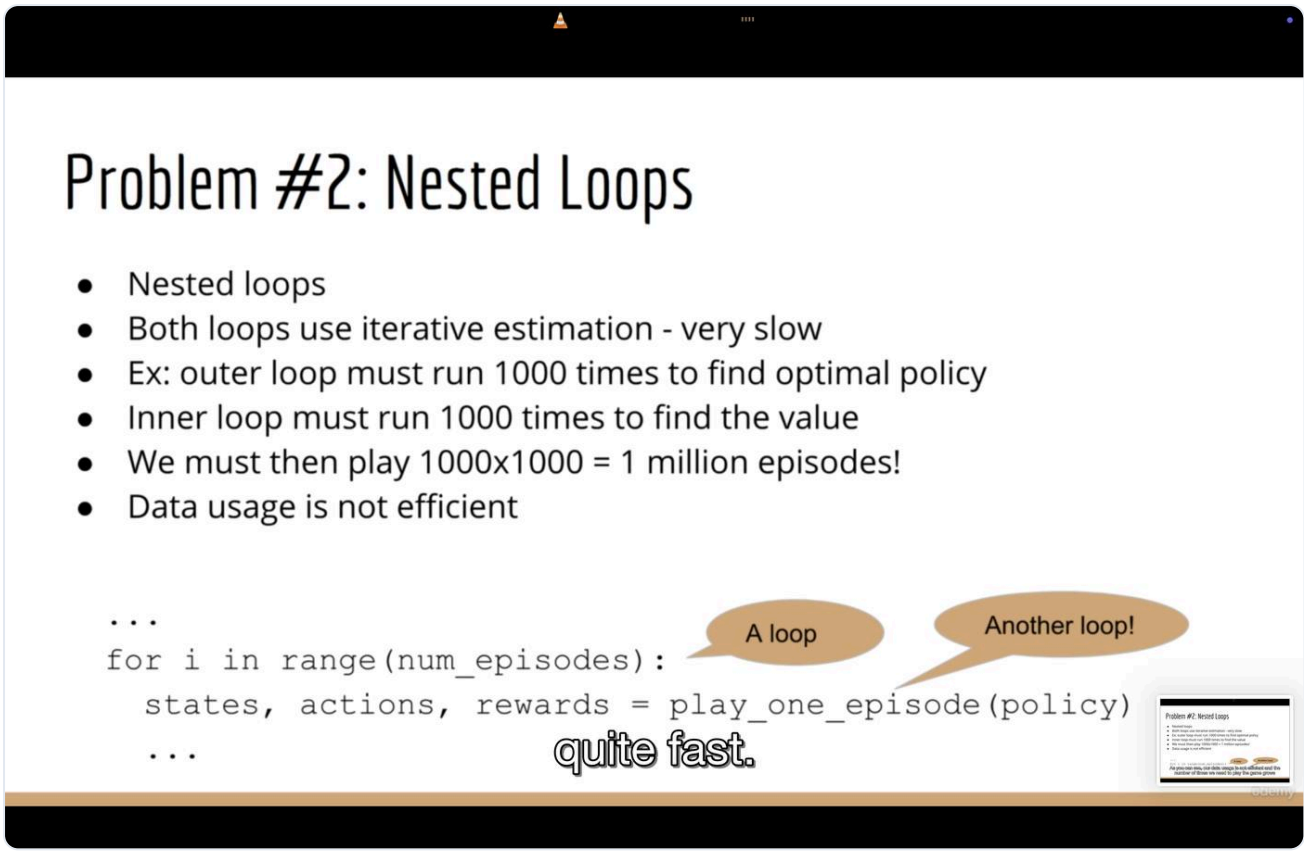

FIGURE 3.11

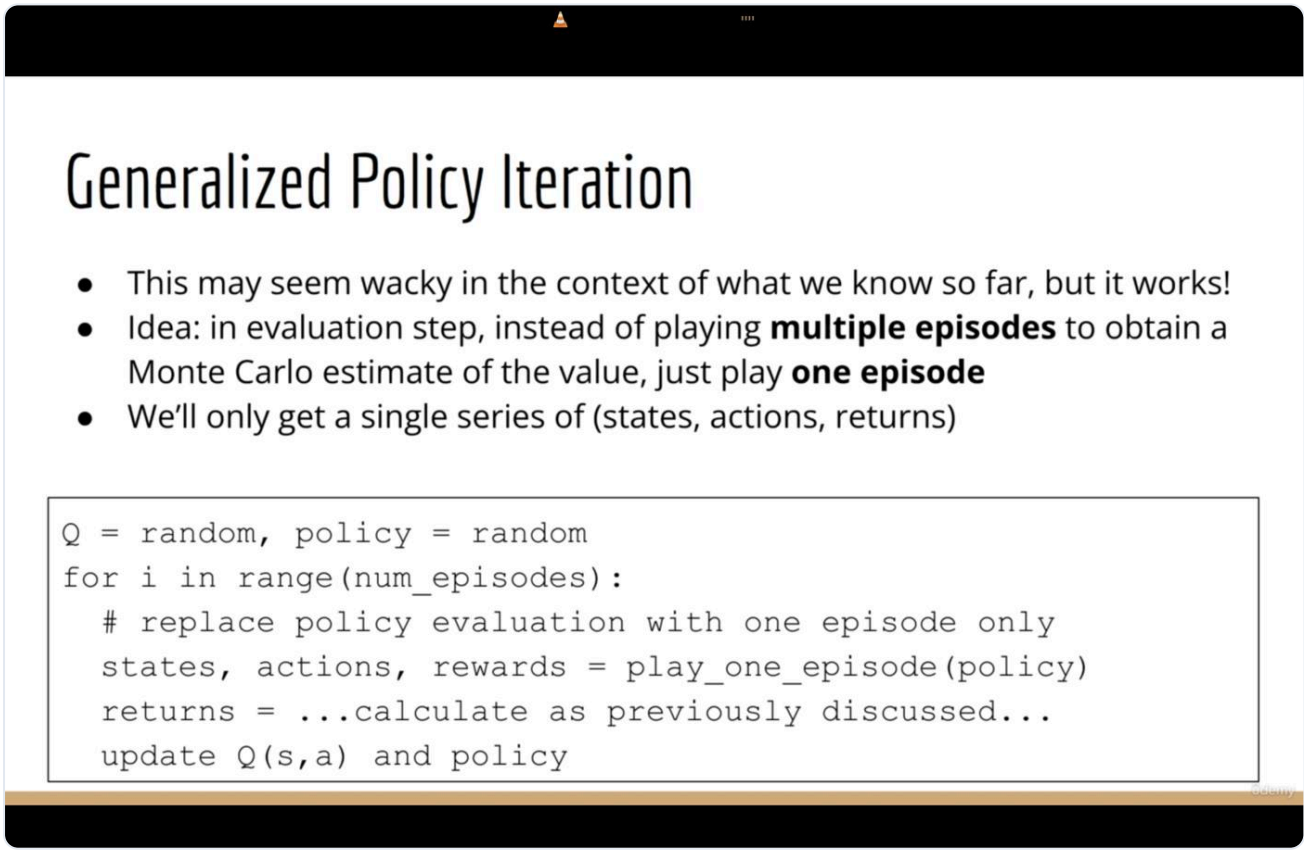

FIGURE 3.12 | Generalized Policy Iteration

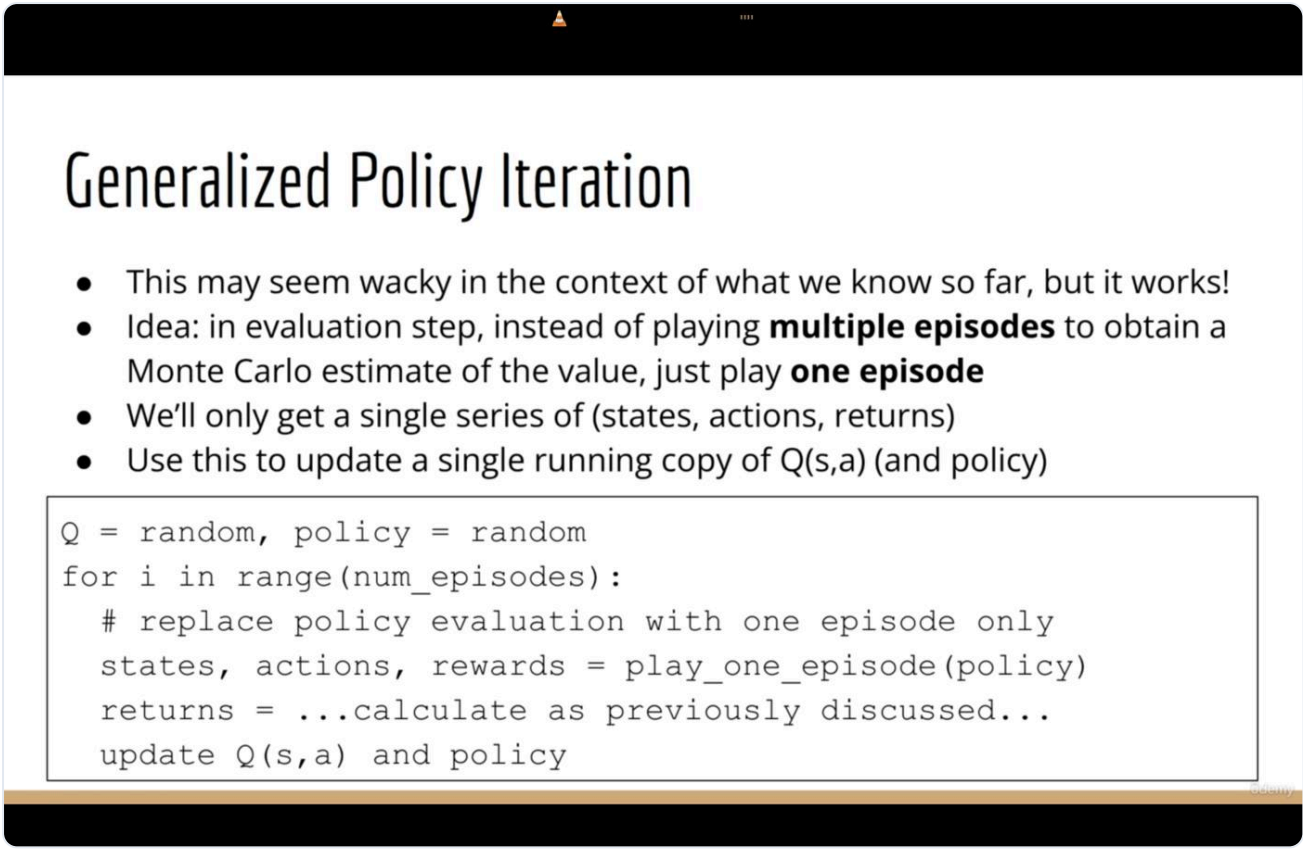

FIGURE 3.13 | Generalized Policy Iteration

The idea is this for the policy evaluation step, instead of playing multiple episodes, in order to get a good estimate of the value of our policy, we are only going to play one episode. After playing this one episode, we'll get a series of states actions and rewards from which we can calculate the corresponding returns. From that, we can loop through each state action in return and use the latest sample of the return to update Q of s a. Now you'll notice that I've just put some pseudocode here. Update Q of s a.

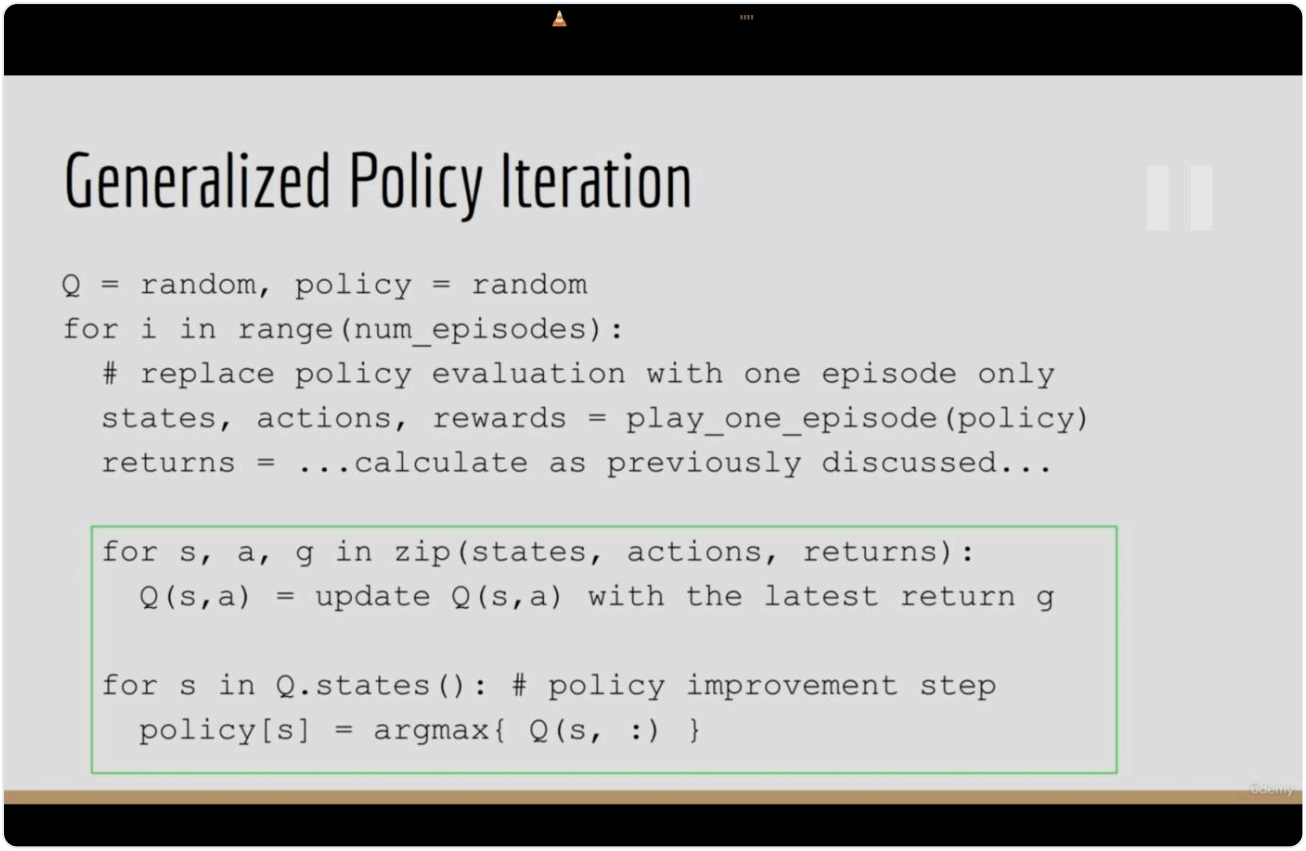

FIGURE 3.14 | Generalized Policy Iteration

But I haven't told you exactly how we're going to do that. We'll be discussing this shortly. The important thing to note right now is that we'll only keep a single running copy of Q of s a in the next and final step is the same as before. We update the policy by taking the argmax over Q for a given state over all actions. So what's going on with this line here?

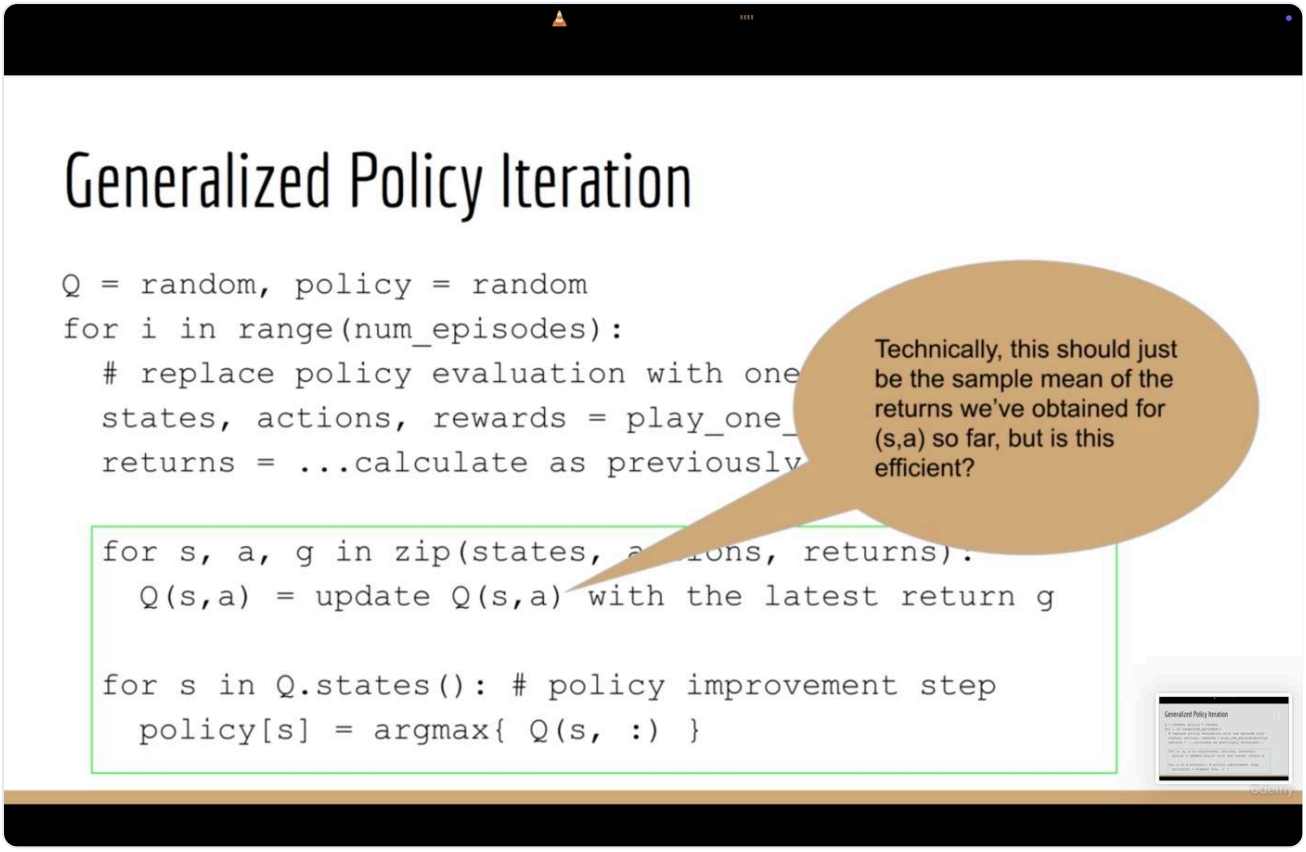

FIGURE 3.15 | Generalized Policy Iteration

How do we update Q of s a given the old estimate of Q of s a? And for this we need to take a short digression back to sample means and expected values yet again. As I'm sure you're tired of me repeating by now, this is the expression for the sample mean we sum up all the samples and we divide by the total number of samples. The question we have to ask is, is this an efficient calculation? The answer is no.

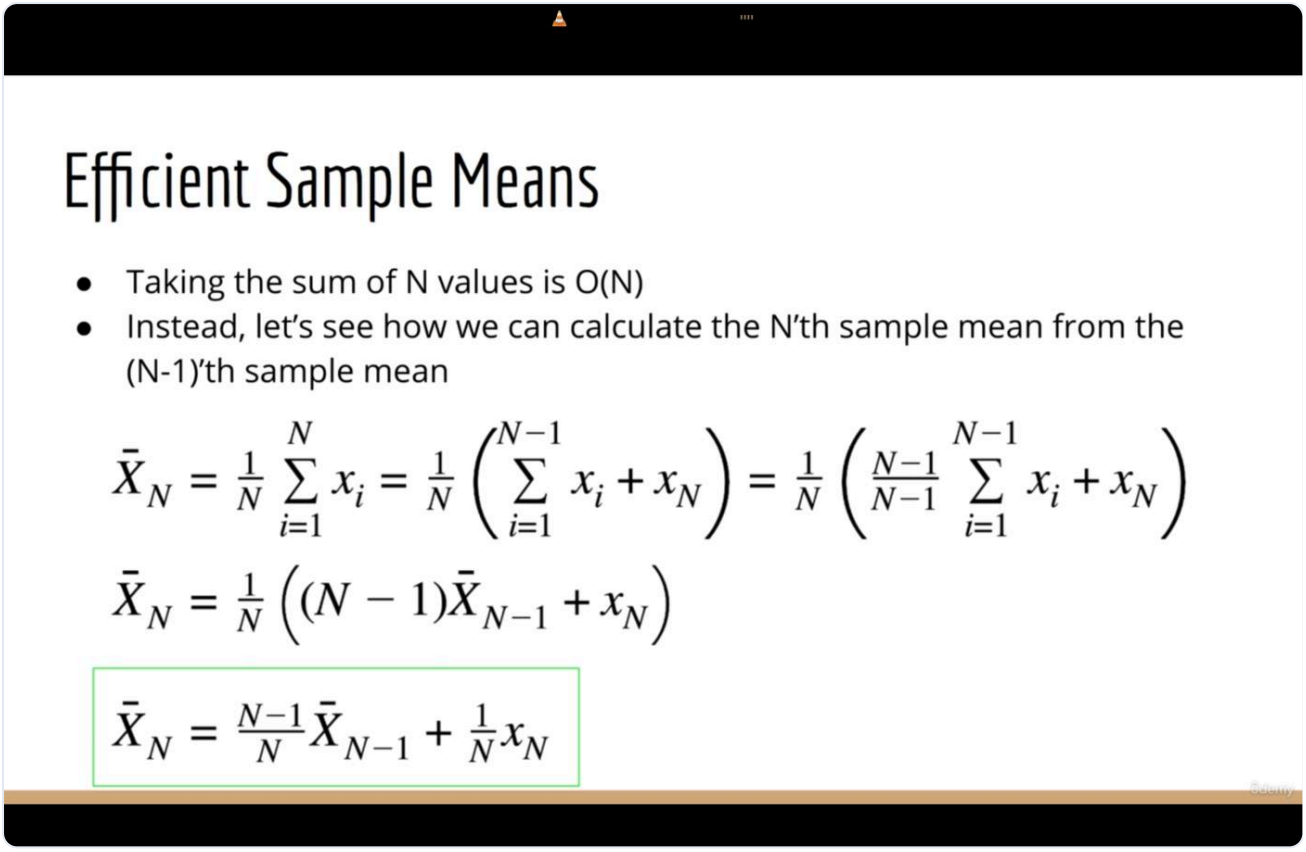

FIGURE 3.16 | Efficient Sample Means

Summing up N values is O of N. The more values we store, the more time it takes to calculate. This is not good. So the question becomes, is there a way to reduce the time it takes to calculate the sample mean? Every time I collect a new sample to see how this is done.

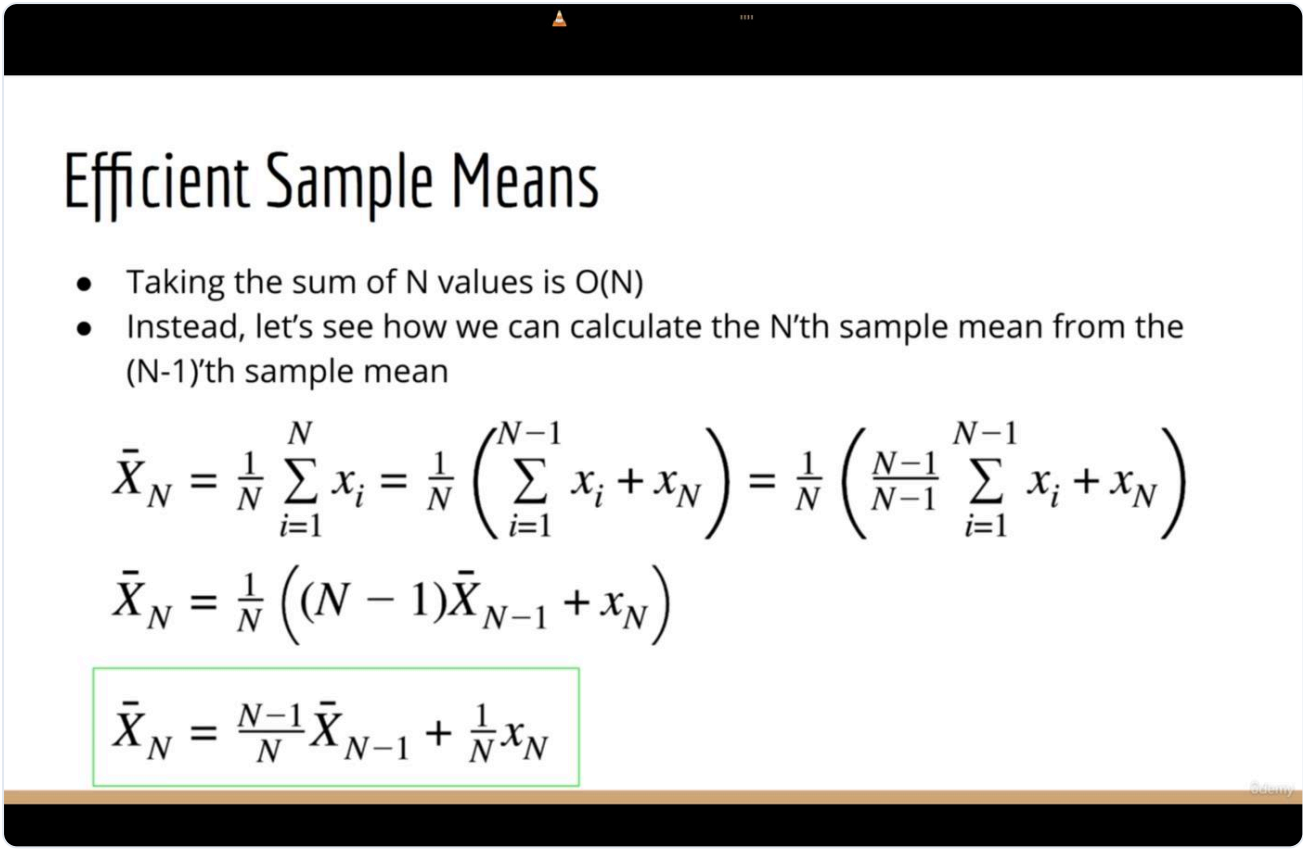

FIGURE 3.17 | Efficient Sample Means

Let's express the sample mean in terms of the previous sample mean and the latest sample. Before we move on, please make sure you can derive this yourself on paper. In the derivation you see here, x bar n means the sample mean using the first n samples thus x bar n minus one means the sample mean using the first n minus one samples. The trick is we can turn the above expression into something that looks like and in fact is gradient descent.

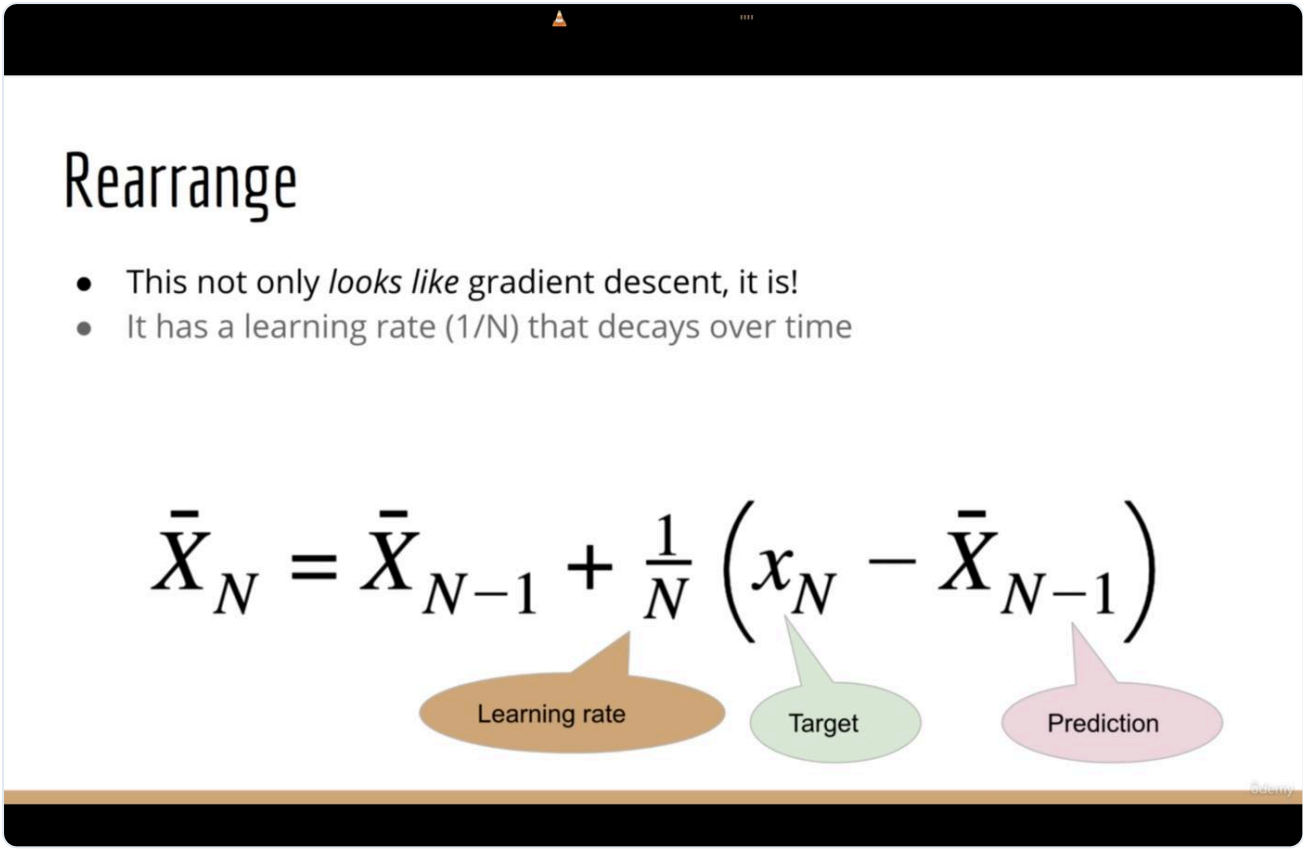

FIGURE 3.18

We can write it so it looks a little more familiar. The new estimate is equal to the old estimate plus one over n times, the sample minus the old estimate. In this scenario, the target is the latest sample we've collected, while the old estimate is our prediction. One over n is the learning rate. So this learning rate is one that decays over time by using such a learning rate.

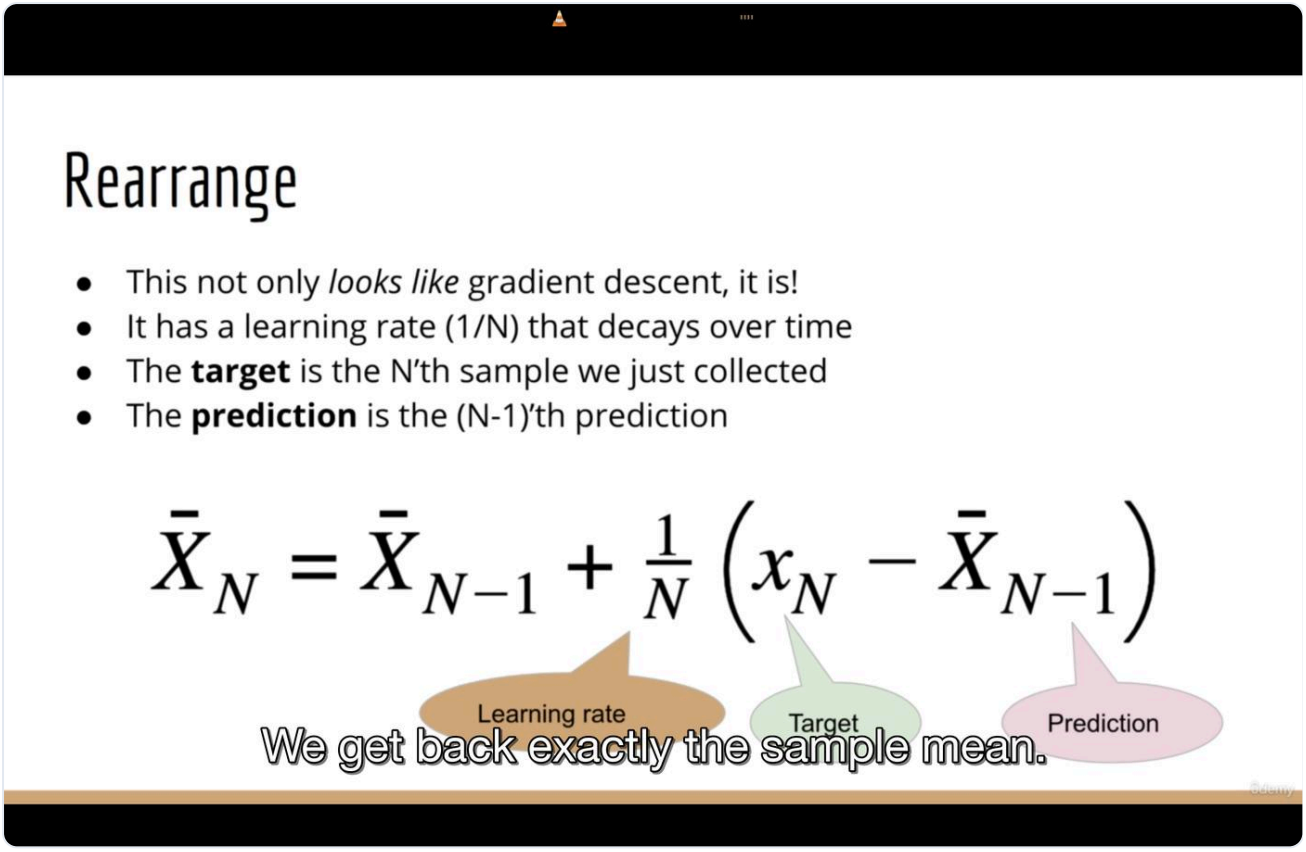

FIGURE 3.19 | The Target Is the N'th Sample We Just Collected

We get back exactly the sample mean. Remember, all we've done so far is basic algebra. Here's what it would look like if we translated it into our Monte Carlo reinforcement learning code to update queue. The new estimate of Q of s a is updated as the old estimate plus one over n times the latest return minus the old estimate. Just note that the equal sign here means assignment.

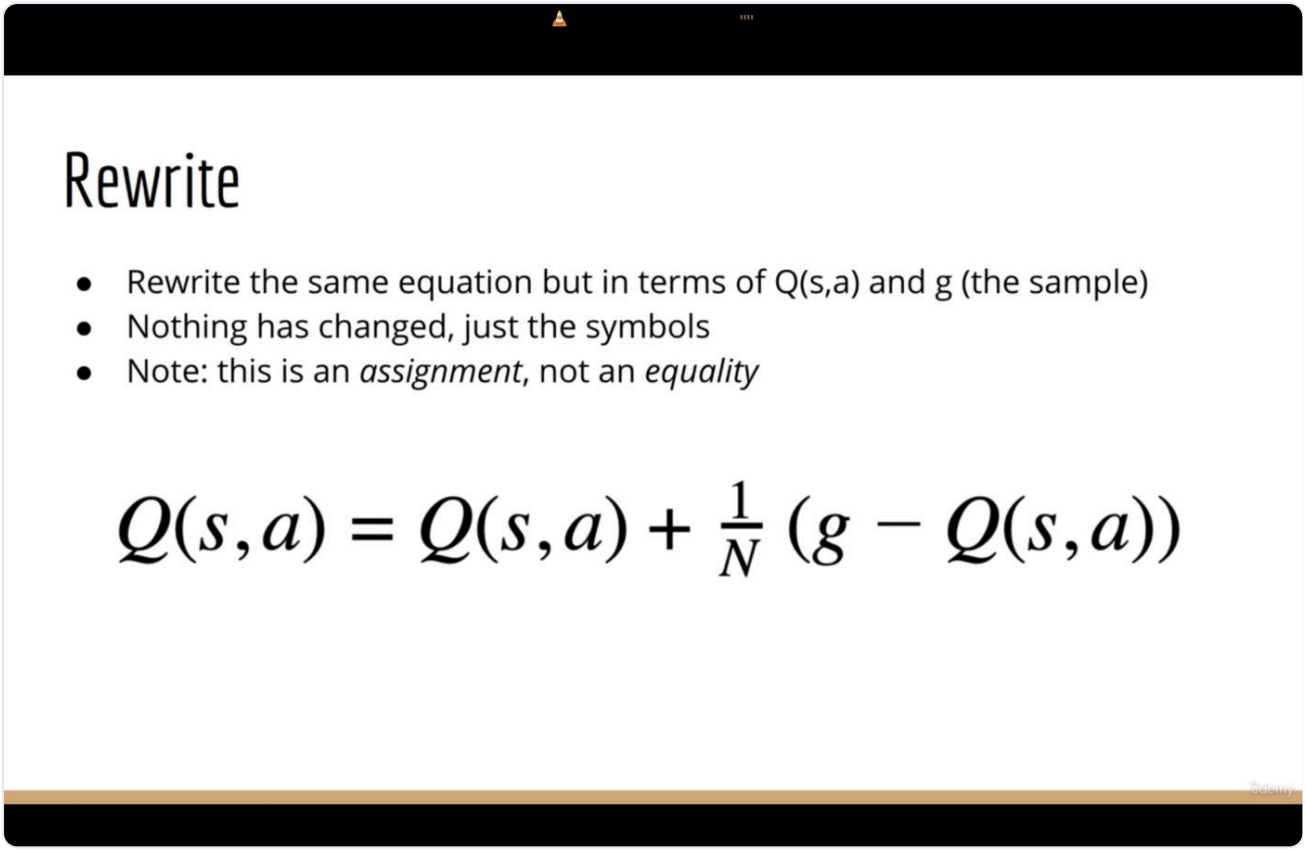

FIGURE 3.20 | Note

We're not saying that both sides are equal. The left side is the new estimate and the right side holds the old estimate. We just haven't bothered to subscript them. However, we are not done yet because we are going to make yet another modification to this. Remember that in our latest generalized policy iteration loop we are updating the same Q dictionary using different policies.

Wait!

- We only have 1 copy of Q - all the samples we collect on each iteration come from **different policies** (because the policy is updated each step)
- The samples which we take the *sample mean* of do not come from the same distribution
- Intuition:
  - Oldest samples come from oldest policies - they matter less
  - Newest samples come from newest policies - they matter more

```
Q = random, policy = random
for i in range(num_episodes):
  states, actions, returns = ... get sample ...
  update Q(s,a) and policy
```

FIGURE 3.21

Constant learning rate

- 1/N learning rate gives us the standard (equally weighted) average
- Constant learning rate gives us the exponentially decaying average

$$Q(s,a) = Q(s,a) + \eta\,(g - Q(s,a))$$

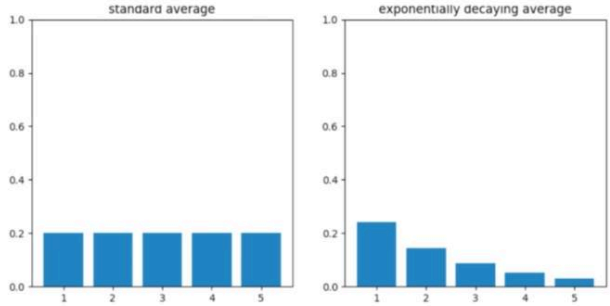

FIGURE 3.22 | Constant Learning Rate

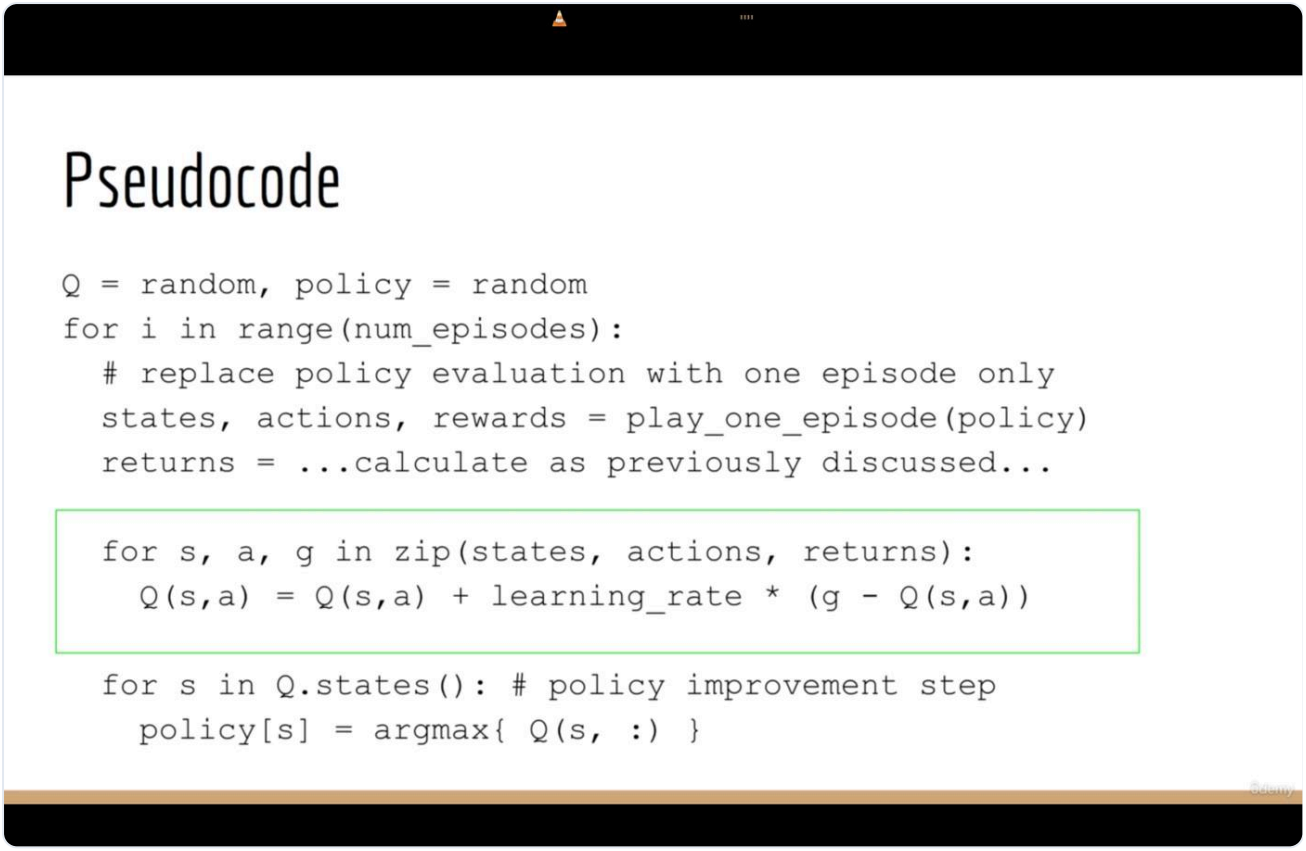

FIGURE 3.23

This policy is being updated on each step and therefore the samples which we are using to calculate Q of s a are not coming from the same distribution. In this case we don't want to use exactly the sample mean intuitively. The oldest samples come from the oldest policies. They don't really matter that much. The newest samples come from the newest policies and they matter more. The key is to use a constant learning rate instead of one that decays over time. Doing so gives us what is called the exponentially decaying average. The idea is when you take the conventional sample mean each sample is weighted equally. As we said, we don't want that because old values come from old policies and so they come from a different distribution than the one we are now interested in.

On the other hand, the exponentially decaying average weights each sample in an exponentially decaying fashion. The most recent sample matters most, and the weights for each sample decay exponentially as you go backwards in the order they were collected. So now we can express our generalized policy iteration code again, but this time we can fill in this little missing detail about how to update Q of s a. So this is the exact same pseudocode as before, except I've replaced the middle block of code with the actual update. We can use the update queue. Unfortunately, at this point there is a subtle but important detail we have yet to discuss. So we are not quite done specifying our algorithm, but we've laid out most of the groundwork.

LECTURE 97

Policy as a Distribution

Earlier in this section I told you about the value of treating the policy as a probability distribution. First it allows us to describe the entire MDP as a set of two probabilities one representing the environment dynamics and one representing the agent. This allows us to reason about the MDP mathematically and find a solution to both the prediction and control problems as we just did.

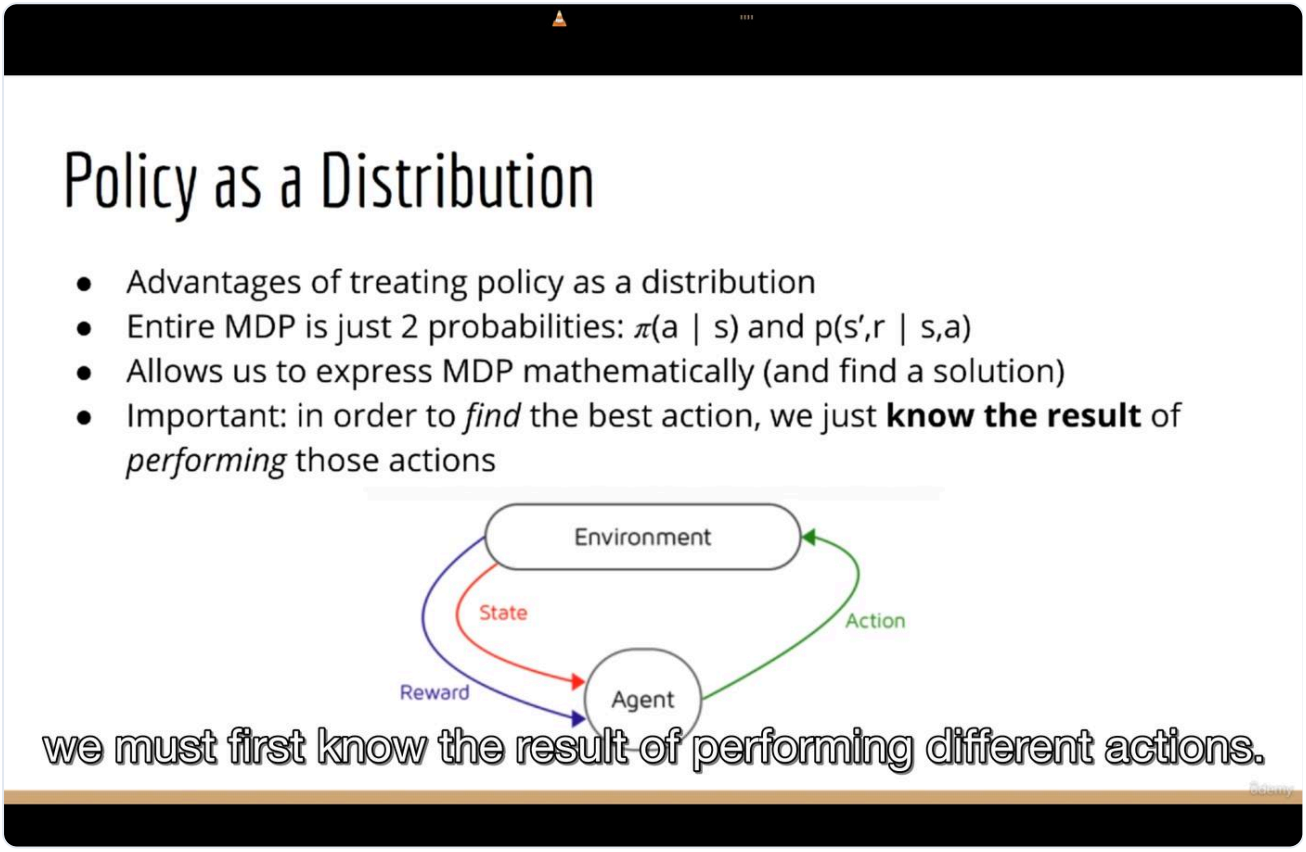

FIGURE 4.1 | Policy as a Distribution

The other important reason which I mentioned briefly earlier is that in order to find the best action we must first know the result of performing different actions. In other words we must have collected those samples. Here is the problem when we take the argmax over all actions Q of s a the Q value here is not changing that much from one episode to the next. Let's say we have three actions so we want to choose from Q of s a1 Q of s a2 and Q of s a3. While perhaps we initialize Q of s a1 and Q of s a2 to 0 while we initialize Q of s a3 to 1 also assume that all the rewards are positive during the process of playing the game. Maybe Q of s a3 gets updated to a value of 2.

We will now always choose action 3 because that gives us the best Q value out of the three possible actions.

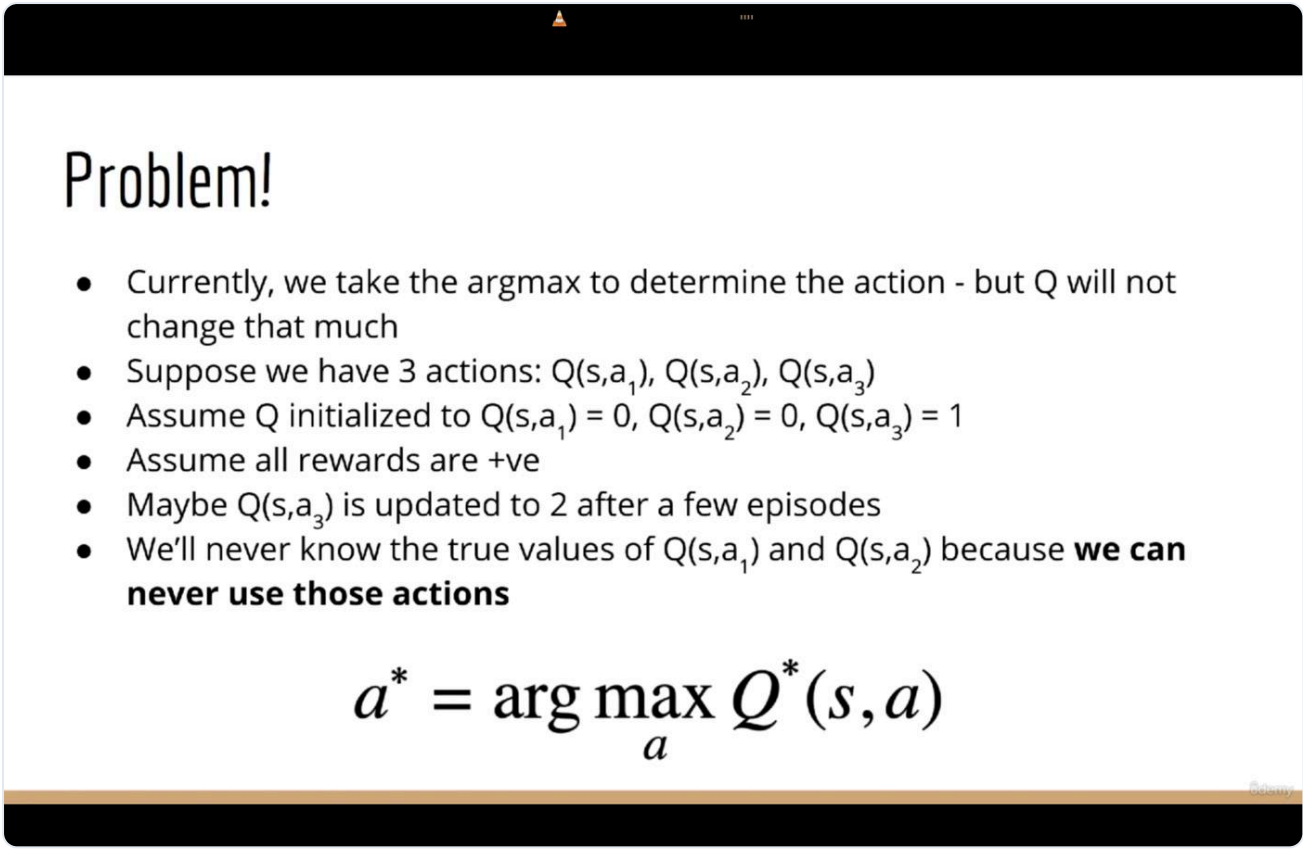

FIGURE 4.2

The problem is we don't know the true values of Q of s a1 and Q of s a2 because we never collected data from performing those actions. How do we know the values of the other actions. If we have never actually performed those actions. In fact we don't know those values at all. We just happen to initialize them to zero because of that. There is no way for us to confidently choose the right action. This is a more general problem known as the explore exploit dilemma. Suppose you go to a casino and you're playing a simplified version of slot machines. You have several slot machines to choose from and you don't know which one you should play. However you do know that not all these slot machines yield the same reward.

As I said these slot machines are simplified so there are only two possible outcomes. You can either win or you lose if you win you get one dollar and if you lose you get zero dollars.

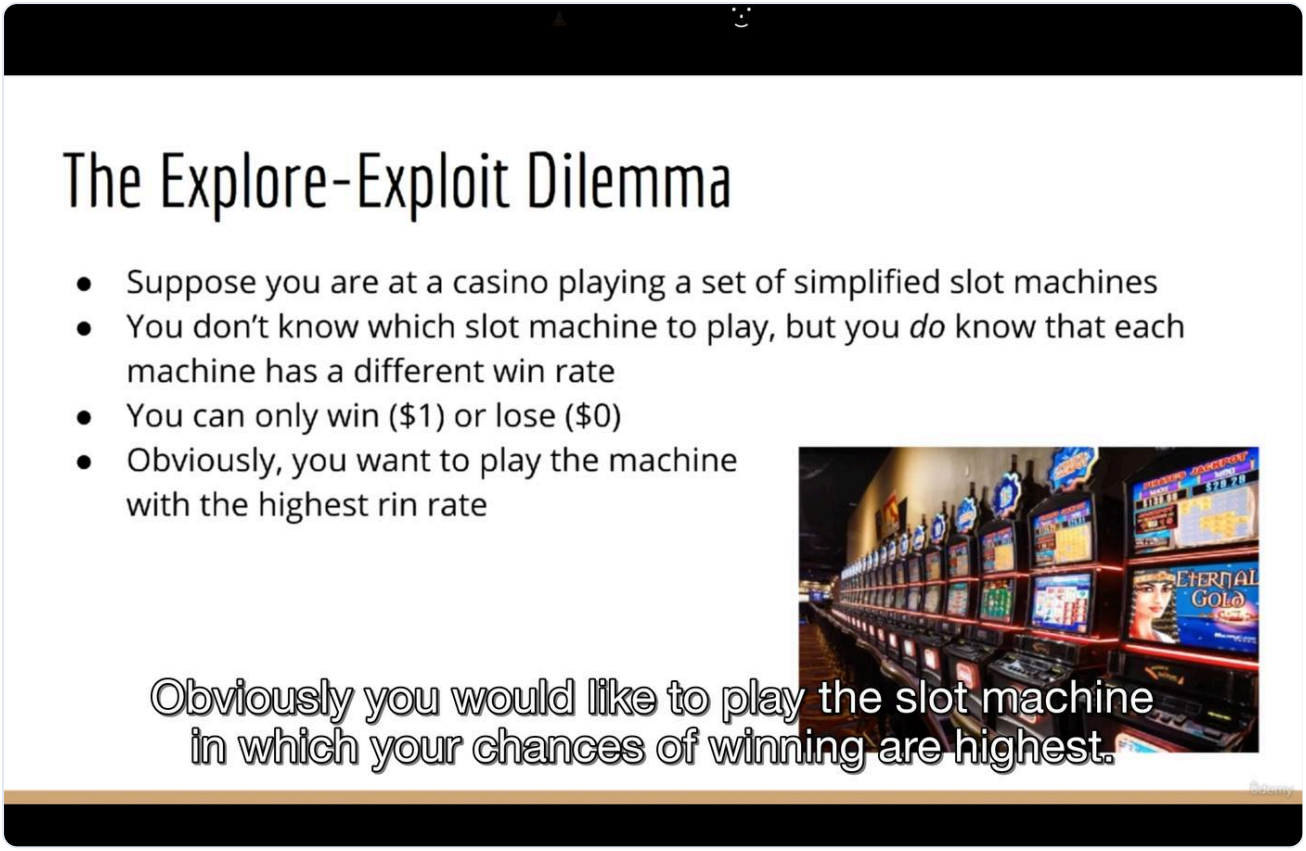

FIGURE 4.3

Obviously you would like to play the slot machine in which your chances of winning are highest. OK. So how can you figure out which slot machine will give you the best chance of winning. Well you cannot ask the manager of the casino because he wants to make money. He will not share that information with you. So what do you do. Your only method of figuring out which slot machine is best is to collect some data.

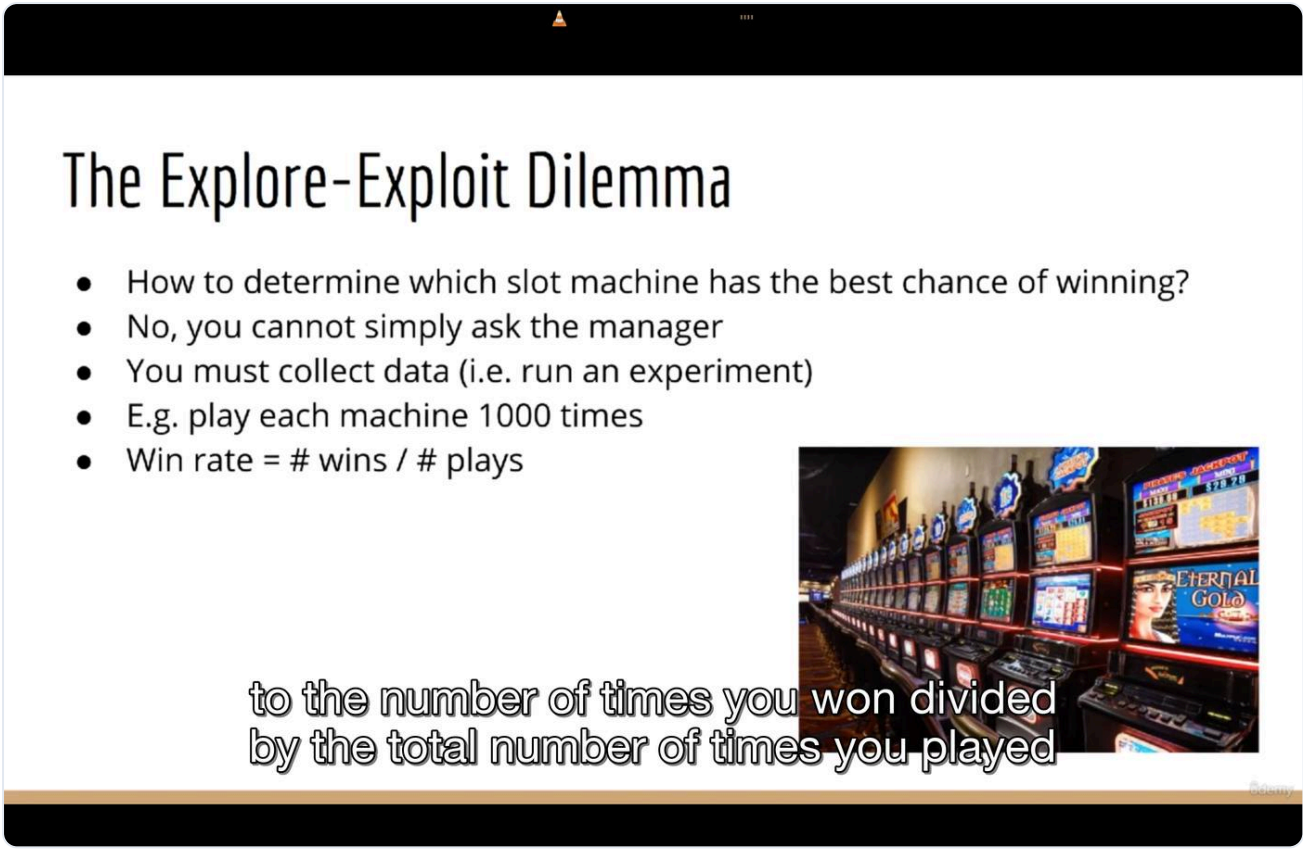

FIGURE 4.4 | No, You Cannot Simply Ask the Manager

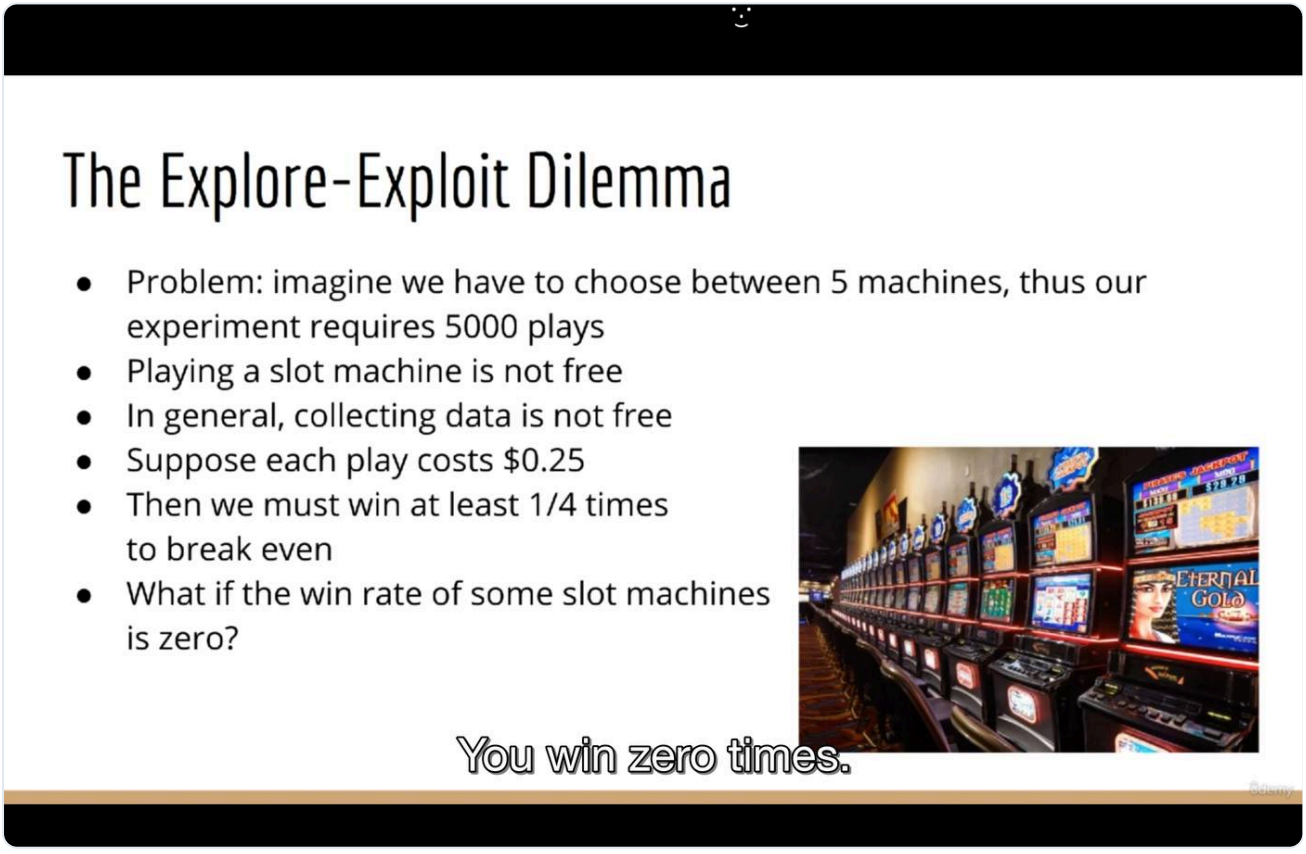

FIGURE 4.5 | Playing a Slot Machine Is Not Free

So let's say you play each slot machine 1000 times the win rate of the slot machine is simply equal to the number of times you won divided by the total number of times you played but what's the problem with what we just did let's say you have to choose between five slot machines if you played a slot machine 1000 times that means you've played 5000 games. Now let me remind you that playing a slot machine at a casino is not free. In general collecting data is not free. It usually requires time resources or both. Let's say each time you play a slot machine you spend 25 cents. You need to win at least one out of every four times in order to break even.

Imagine that for some of these slot machines. You win zero times. Now it becomes clear that playing those slot machines 1000 times would be a huge waste of both time and money here's another question to consider. Why should we collect 1000 samples per slot machine. Why not one hundred Why not ten.

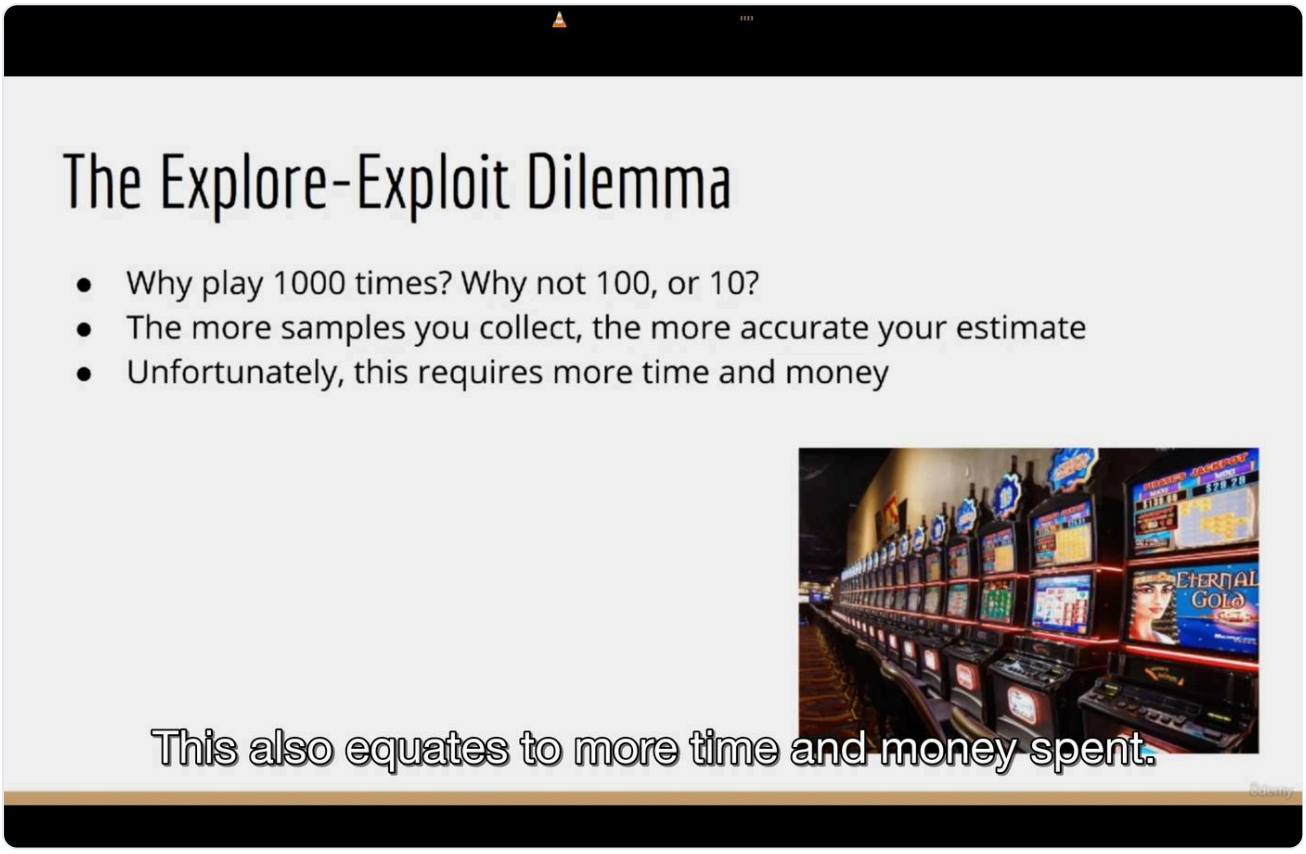

FIGURE 4.6

As you know the more samples you collect the better and more accurate your estimate becomes unfortunately. This also equates to more time and money spent. That's why we call this a dilemma. The explore exploit dilemma means we are trying to balance both exploration and exploitation. We want to explore so that we can collect more data to find out which slot machine is truly the best. On the other hand we want to exploit because we want to play what we believe is the best slot machine in order to win more often and to make more money.

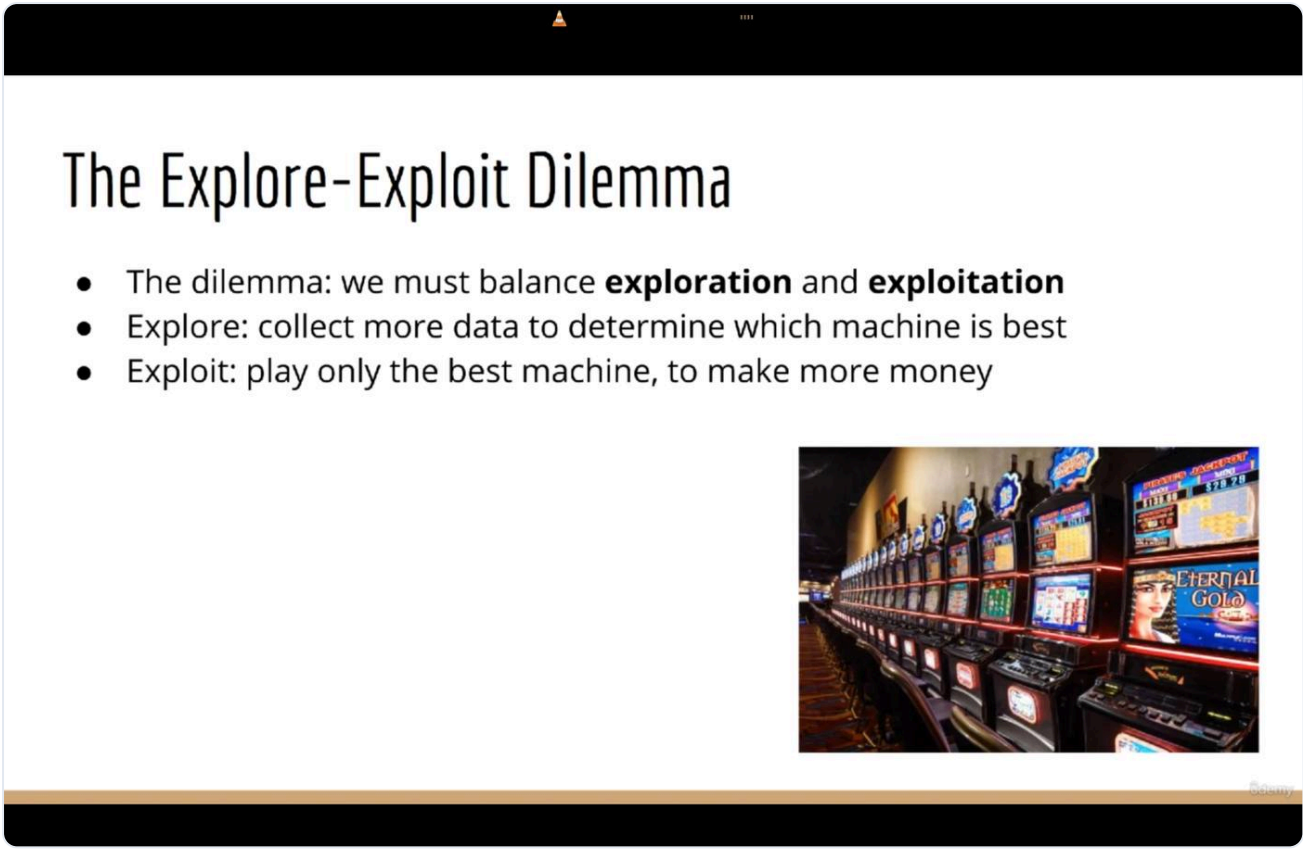

FIGURE 4.7

These two forces always oppose each other the typical answer to this problem in reinforcement learning is called Epsilon greedy. This allows us to be greedy and exploit what we think will yield the highest future sum of rewards. But we have a small probability epsilon of choosing a random value so that we can explore and collect samples for our Q table to make our estimates of Q more accurate. And remember they do need to be somewhat accurate in order for us to confidently choose the best action here's how we will do this in code instead of always performing the action specified by the policy we will generate a random number between 0 and 1. If this random number is less than epsilon we will choose an action at random.

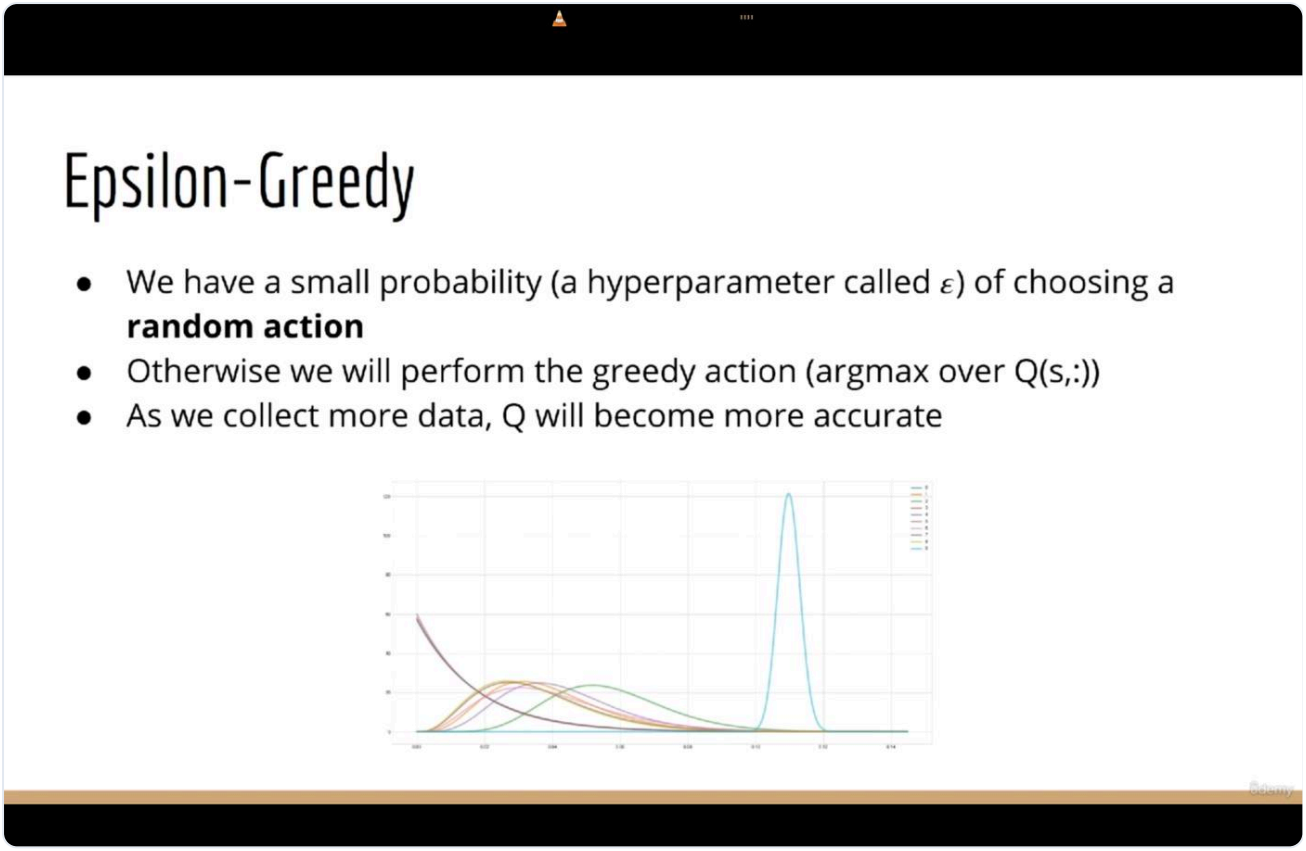

FIGURE 4.8

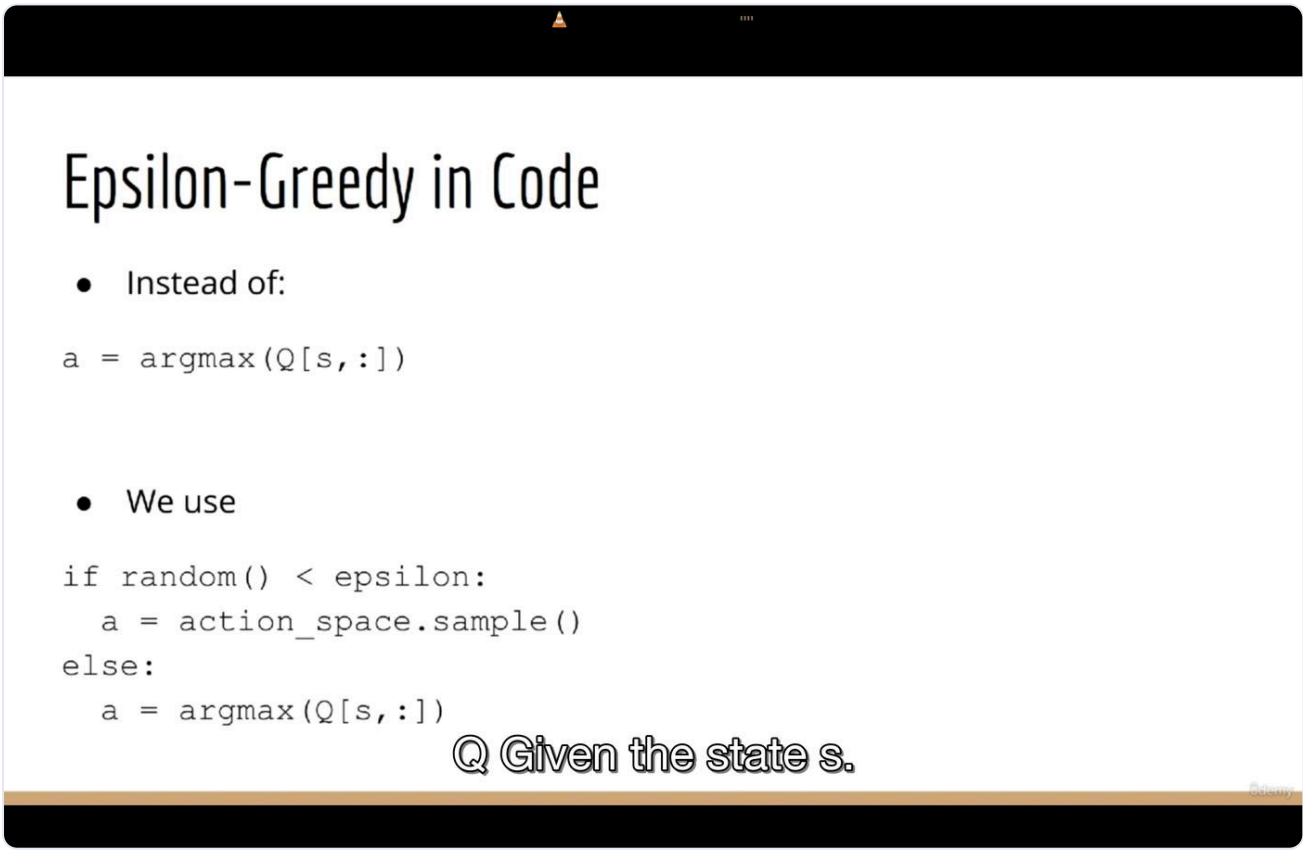

FIGURE 4.9

Otherwise we will perform the action specified by the greedy policy which is greedy with respect to Q Given the state s.

LECTURE 98

Temporal Difference Methods

At last we are ready to study the famous Q learning algorithm. Let's recap what we've done so far because it's been quite a long process to even get to Q learning.

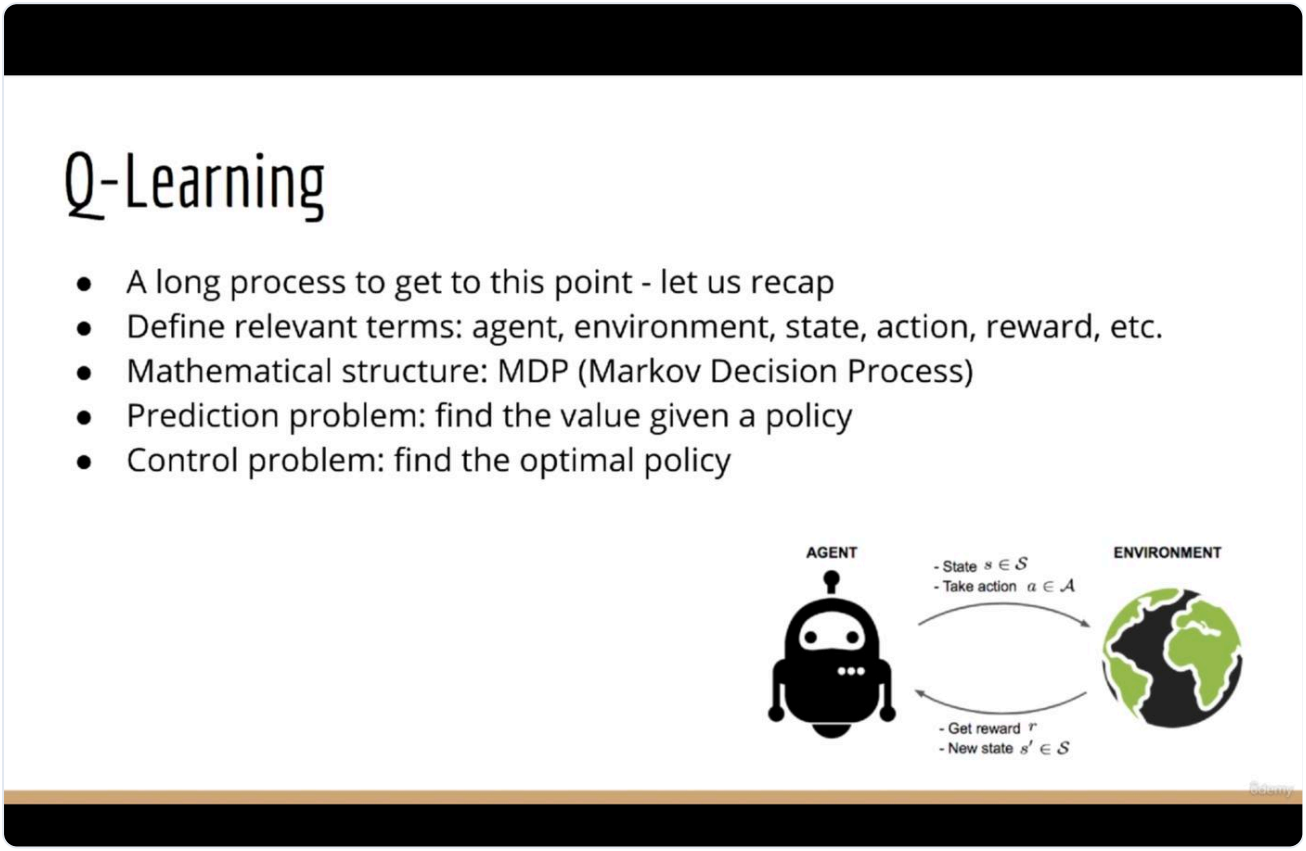

FIGURE 5.1 | Define Relevant Terms: Agent, Environment, State, Action, Reward, Etc

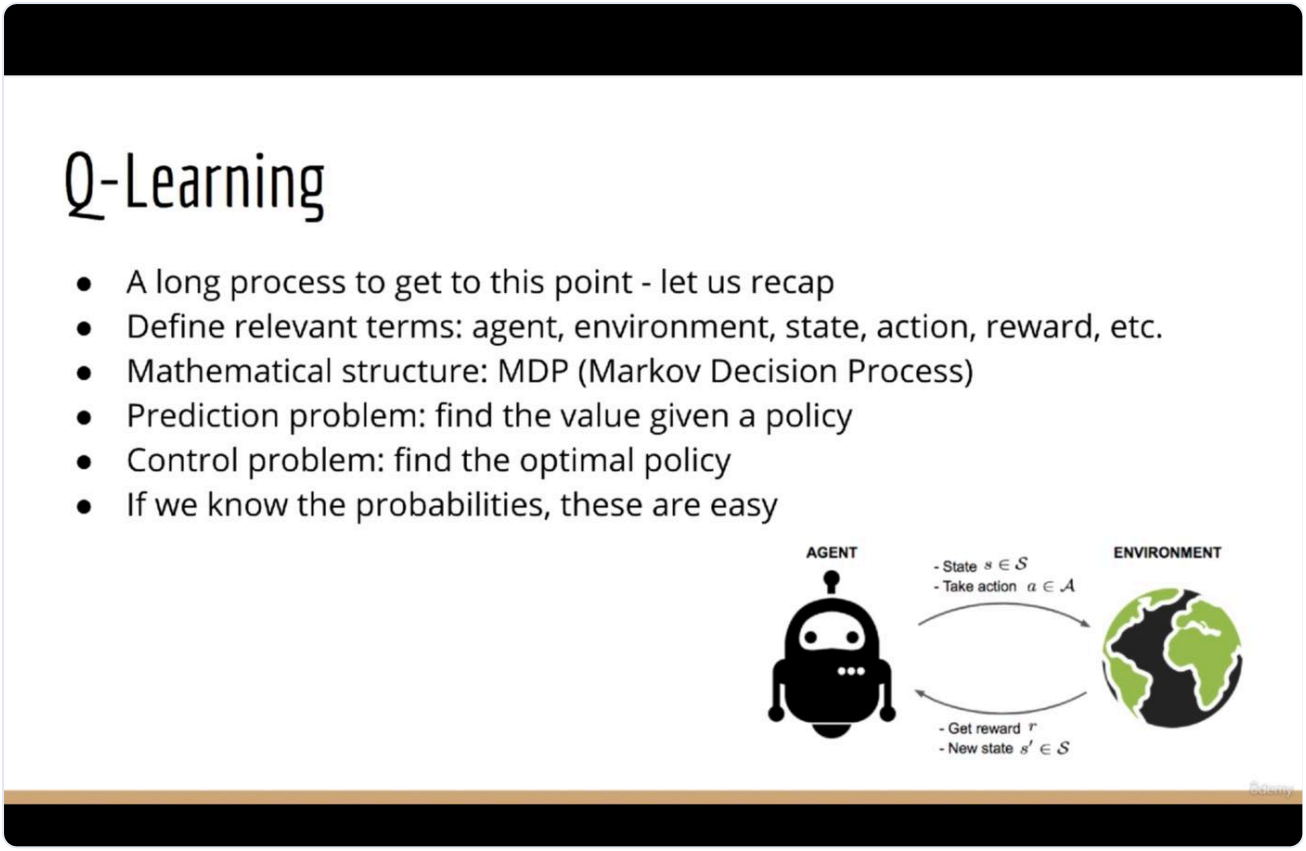

FIGURE 5.2 | Define Relevant Terms: Agent, Environment, State, Action, Reward, Etc

First we defined all the relevant terms such as agent environment state action reward and so forth. Having these definitions allowed us to structure our problem mathematically. Specifically we can model reinforcement learning problems as MDPs Markov decision processes. Next we considered how we would solve an MDP meaning two things. First we solve the problem of prediction finding the value function given a policy. Second we solve the problem of control finding the best policy in a given environment. We learn that if we know all the probabilities in an MDP this is quite easy.

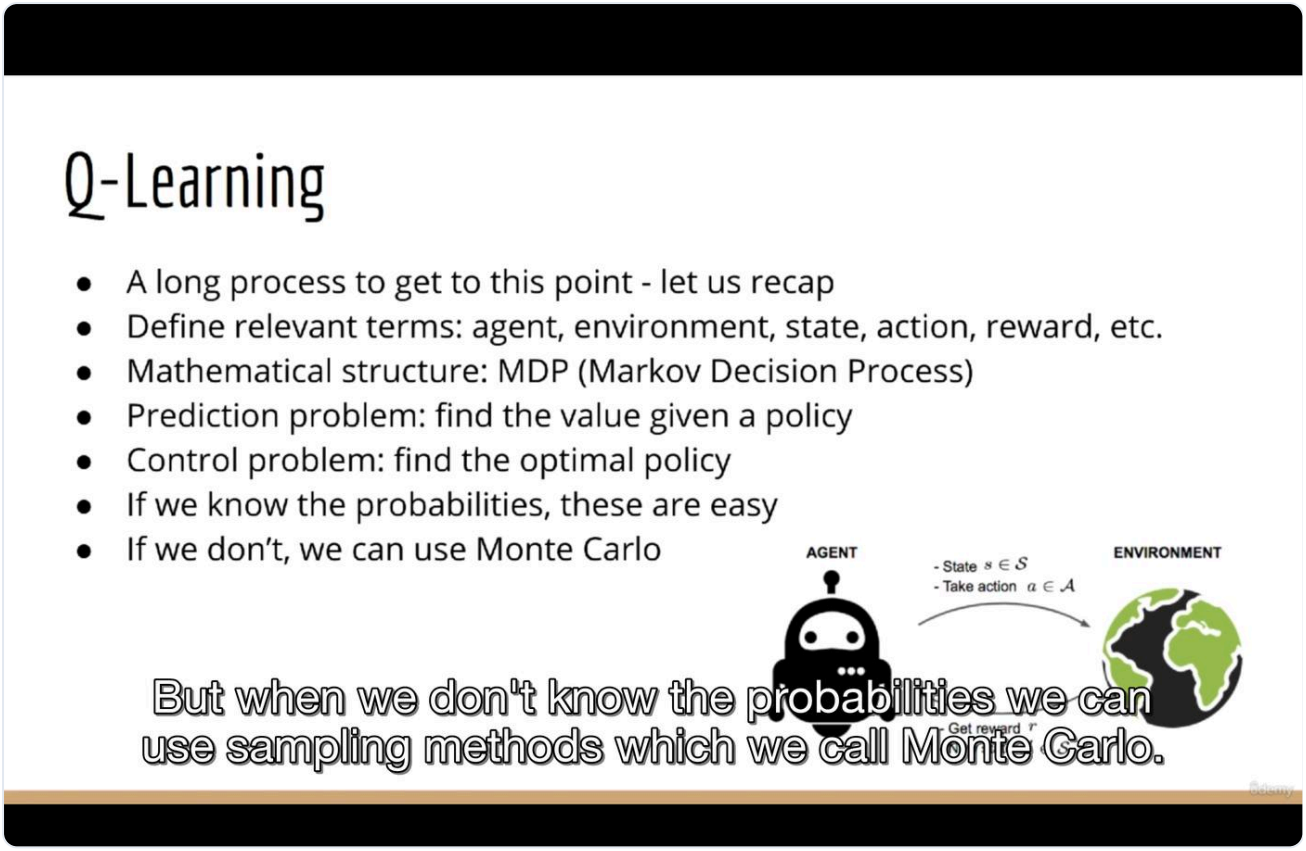

FIGURE 5.3 | Define Relevant Terms: Agent, Environment, State, Action, Reward, Etc

But when we don't know the probabilities we can use sampling methods which we call Monte Carlo.

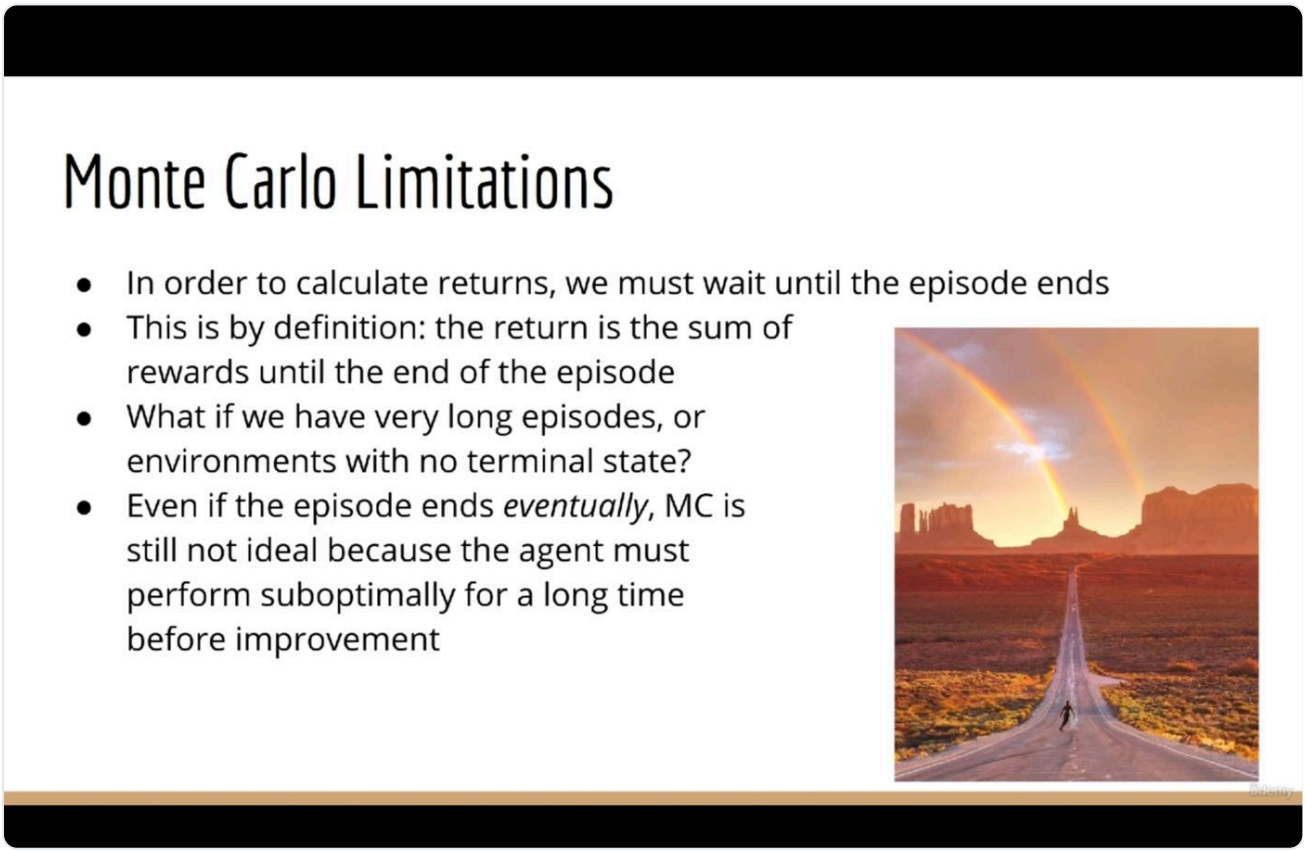

FIGURE 5.4

There is one major problem with Monte Carlo is that in order to calculate returns we must wait until an episode is over. This is because the return is defined as the sum of all future rewards. We can't know the sum of future rewards until we have collected them and therefore we must reach a terminal state before calculating the sample returns. Why is this a problem. Well imagine the scenario where you have very long episodes or the scenario where there is no terminal state. In the latter case Monte Carlo is not an option in the former. It's still not ideal because that means your agent has to spend a very long time performing suboptimally even though it has already collected a lot of data from which it may improve the answer to this is temporal difference methods.

If you recall I said earlier that one of the most important features of the return is that it can be defined recursively the return at time t can be expressed in terms of the return at time T plus 1. You saw how this helped us both create and solve the Bellman equation. Now it's going to help us once again.

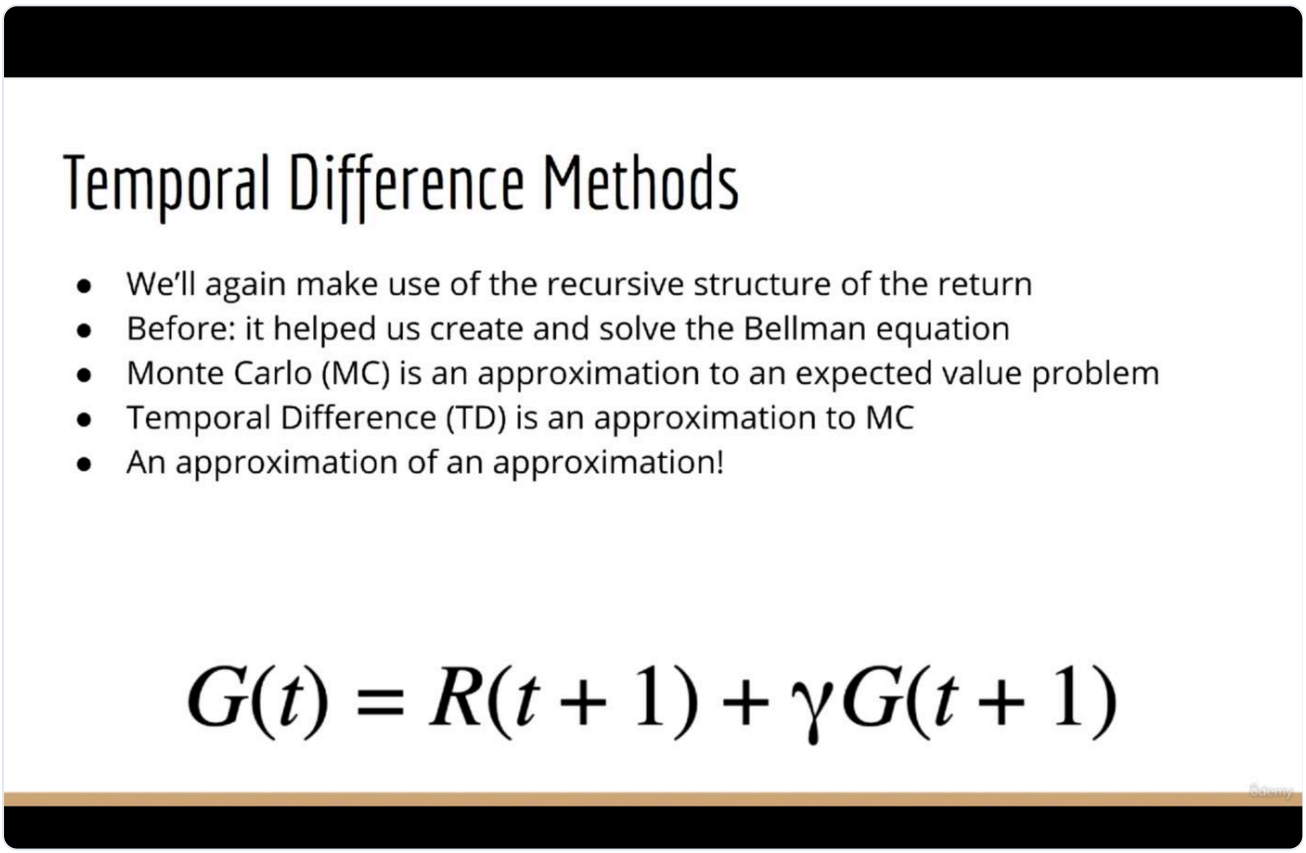

FIGURE 5.5 | Temporal Difference Methods

One simple way to think about it is this Monte Carlo methods were an approximation to an expected value problem Temporal Difference methods are simply an approximation to Monte Carlo. So in other words they are an approximation of an approximation recall this one neat trick with Monte Carlo methods when we're updating Q or V which is the sample mean of all the samples we've collected.

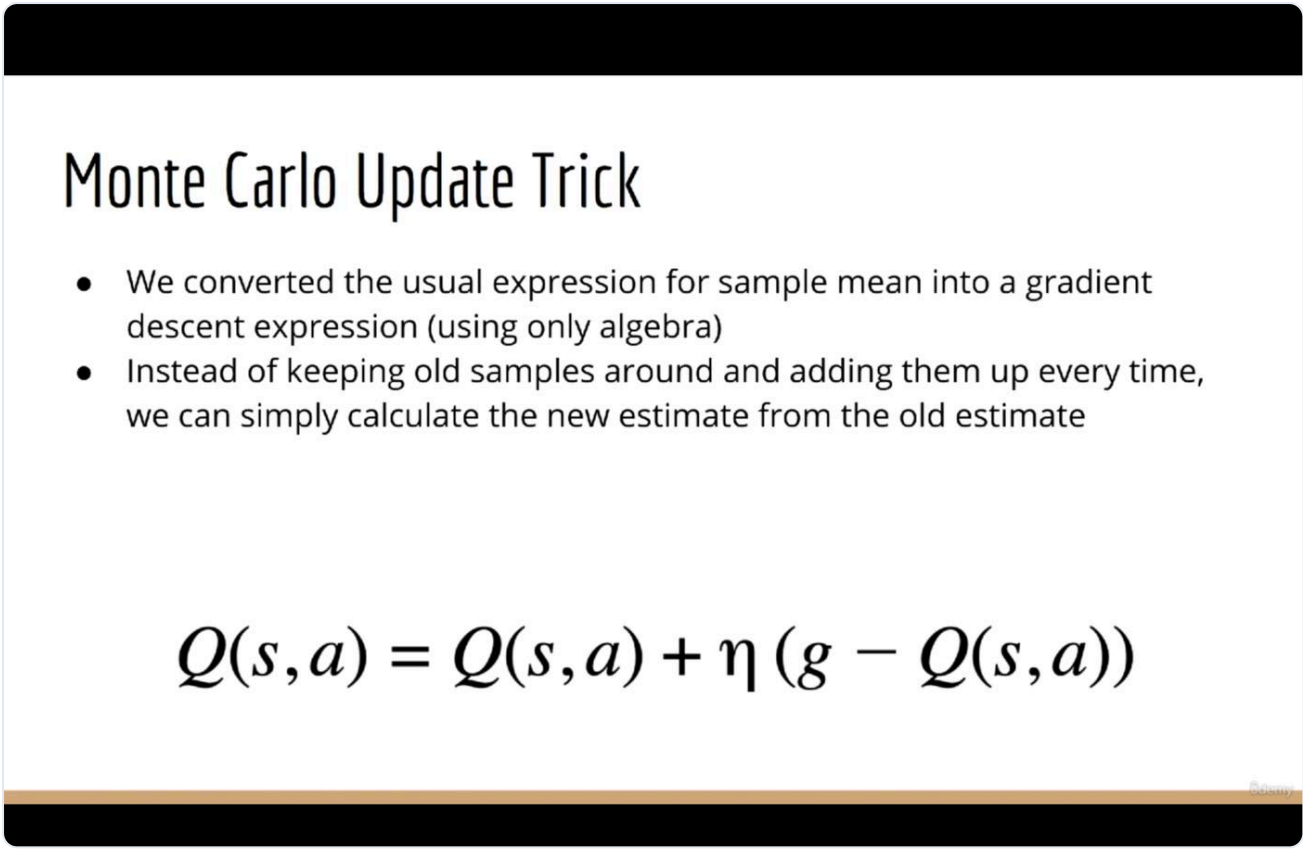

FIGURE 5.6 | Monte Carlo Update Trick

We don't have to express the update as a sample mean that would mean we have to keep all the samples around which can take up lots of memory and take lots of time to calculate. Instead we can express the current sample mean in terms of the past the sample mean we noted that this looks a lot like gradient descent Well why not test out this theory.

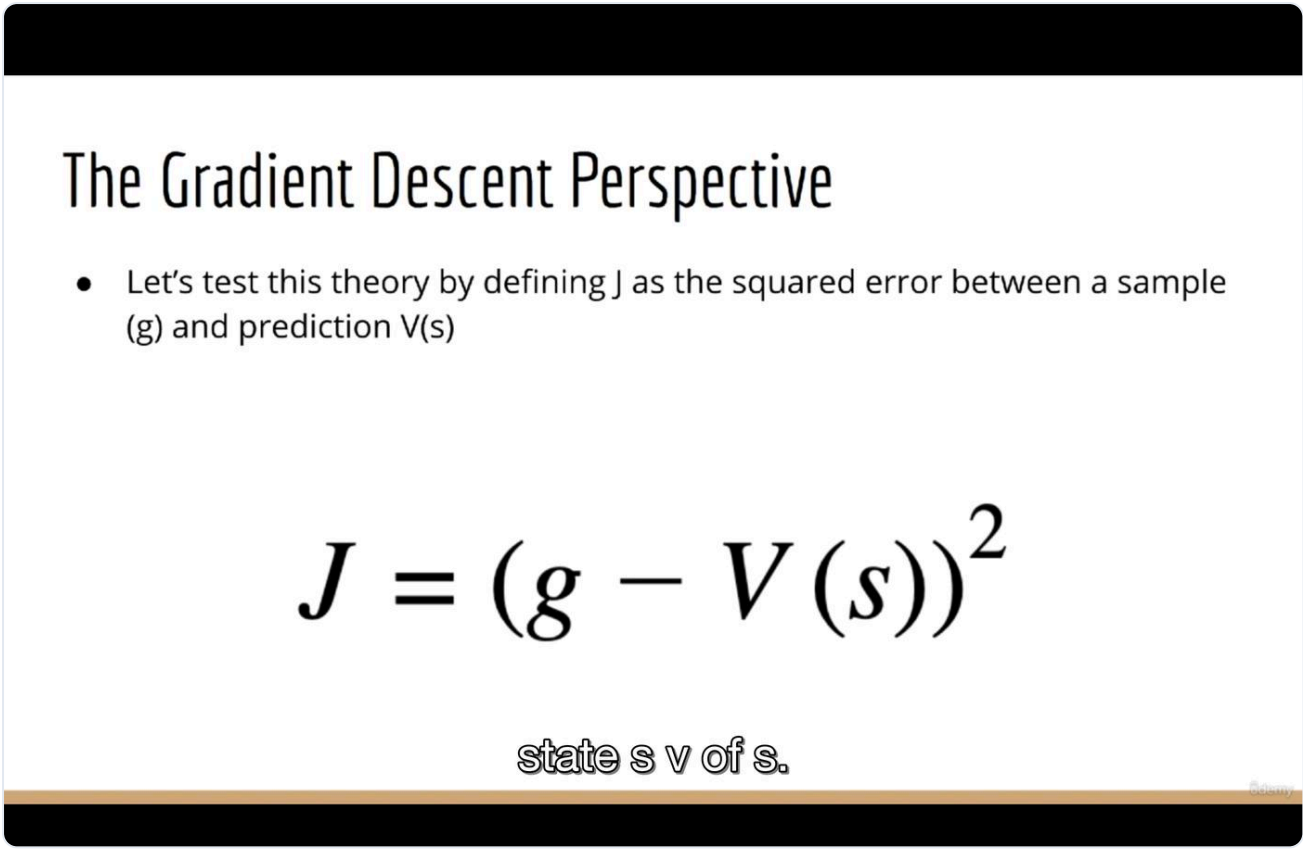

FIGURE 5.7 | The Gradient Descent Perspective

Let's get our squared error J to be the squared error between my true target G and my value for the state s v of s. Now let's say I want to update V of s using the latest sample G which I've just collected in order to perform the update. I'm going to use gradient descent set V of s to be the old value of V of s minus the learning rate times the gradient. Well what is the gradient if we plug this gradient into our gradient descent update.

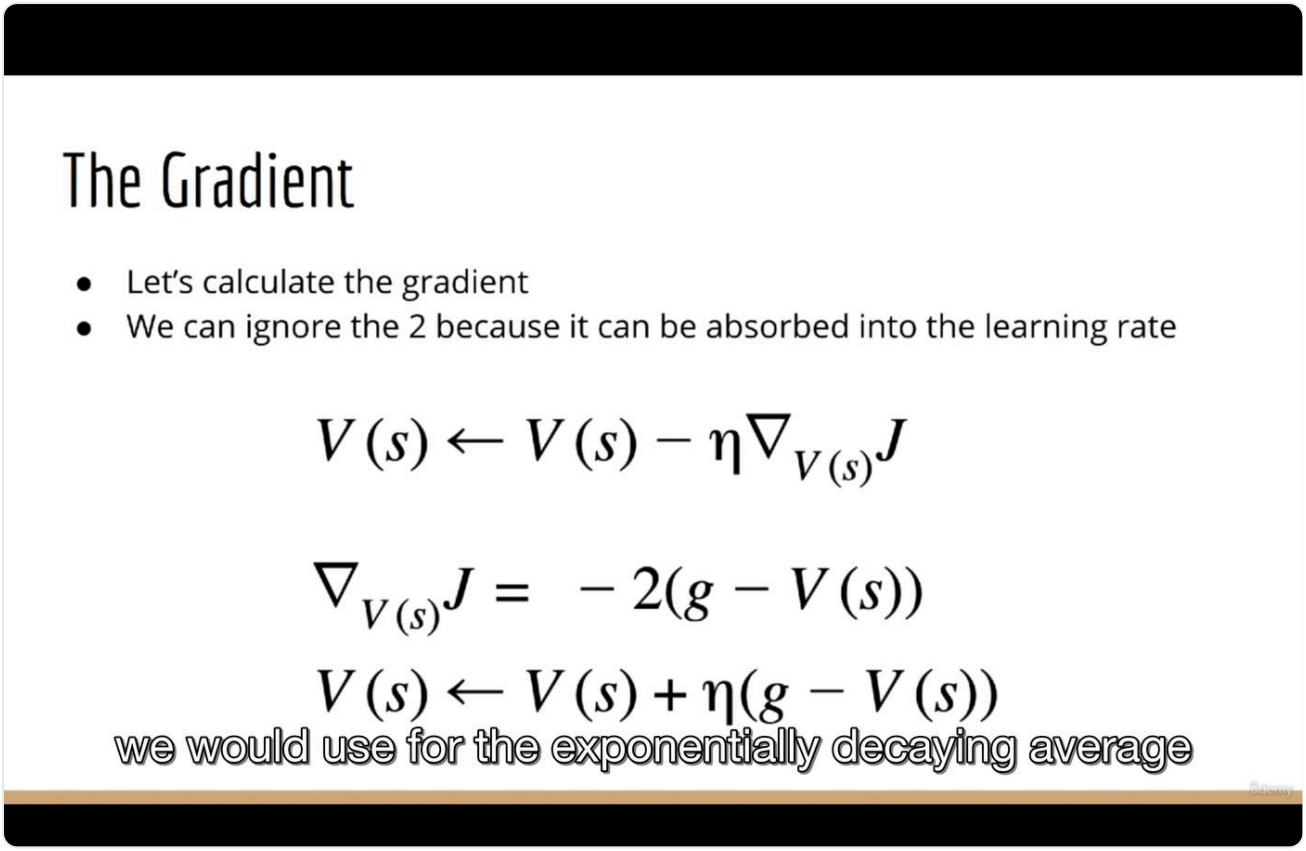

FIGURE 5.8 | The Gradient

And ignoring the two since it can be absorbed into the learning rate we get back the exact opposite equation we would use for the exponentially decaying average as a side note. I want to mention that there is ultimately no difference whether we call what we are doing gradient descent or gradient ascent because there's a plus sign here. You might think of it as gradient ascent. The plus sign is natural to use if you derive this expression from the sample mean update.

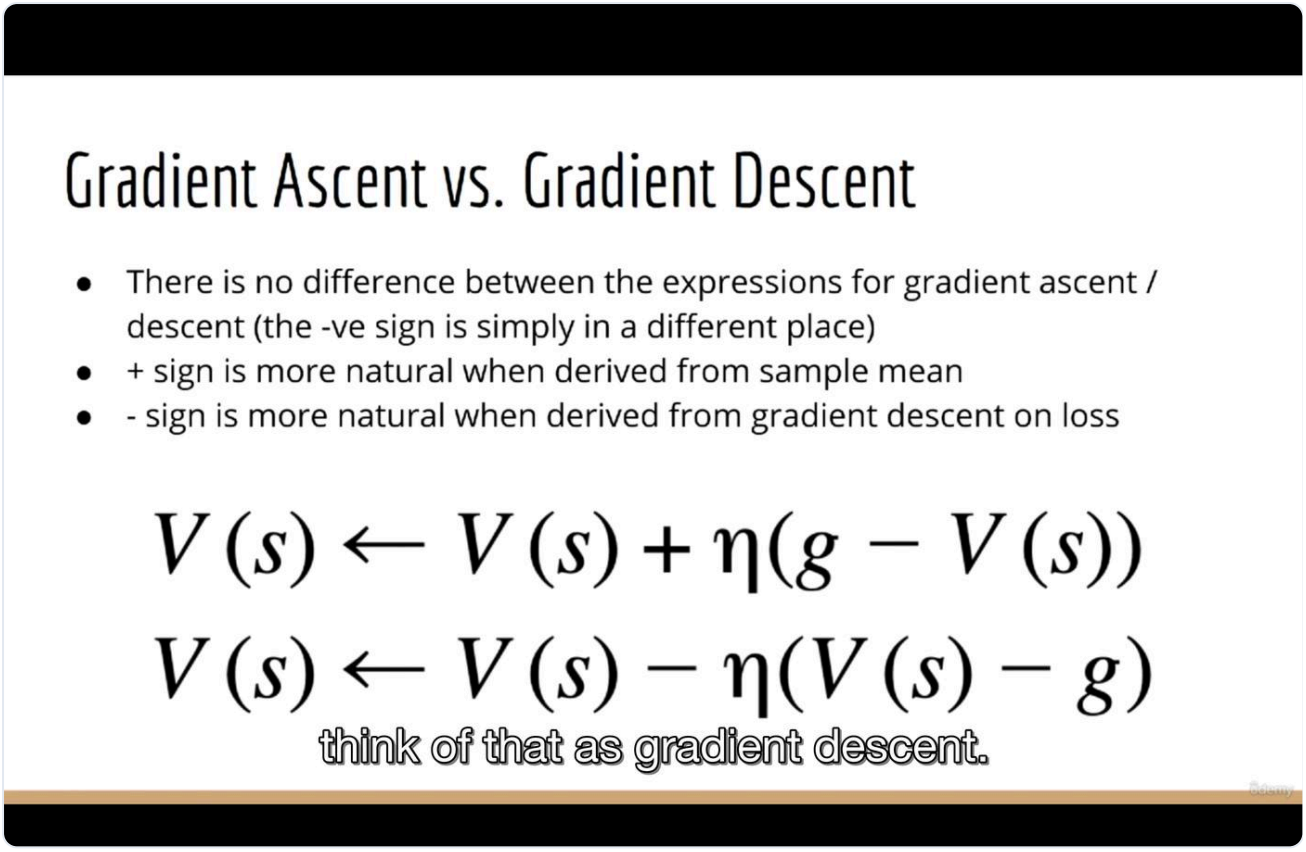

FIGURE 5.9 | Gradient Ascent vs. Gradient Descent

But if you derive this expression from the squared error you will get a negative sign and you might think of that as gradient descent.

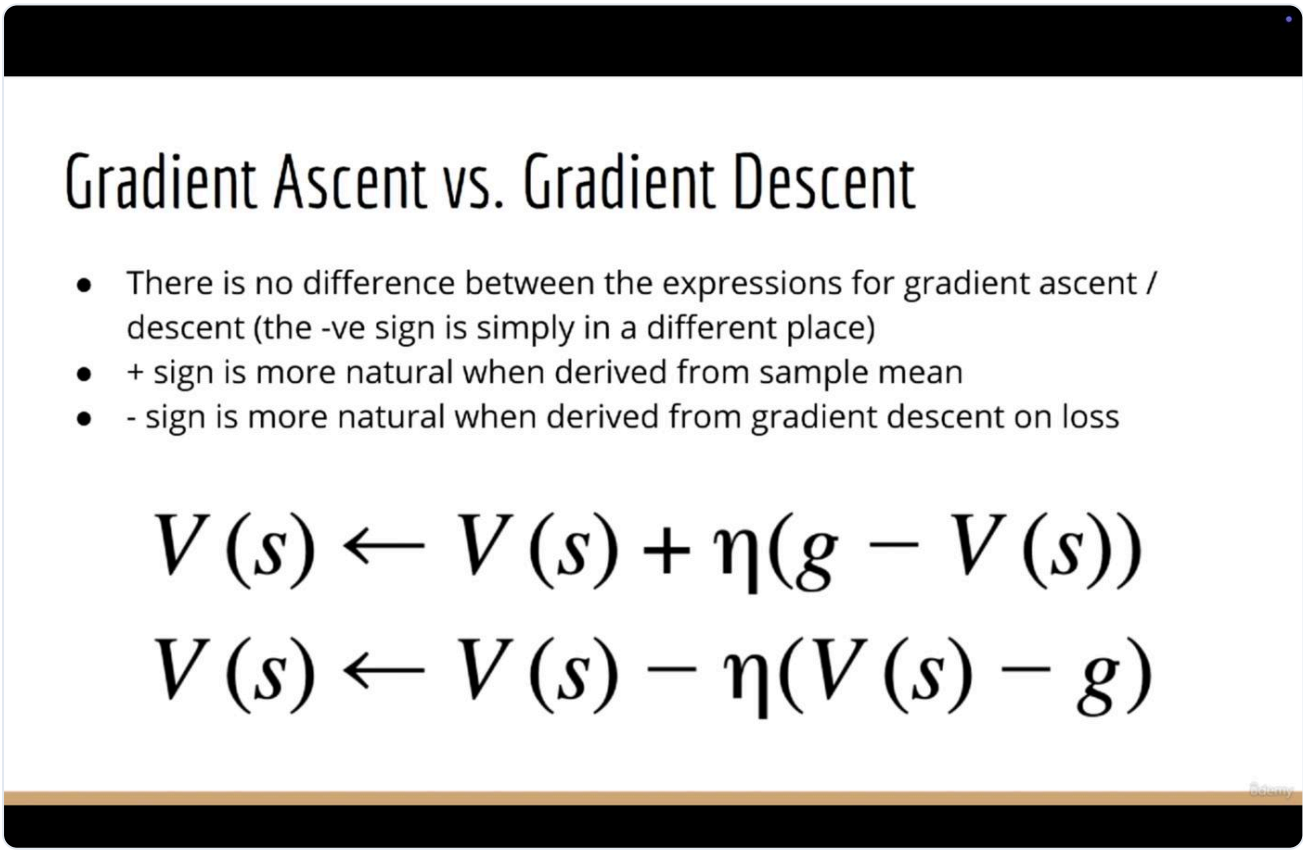

FIGURE 5.10 | Gradient Ascent vs. Gradient Descent

Ultimately however this is just basic algebra and you should confirm to yourself that both expressions are equivalent so what's so significant about this.

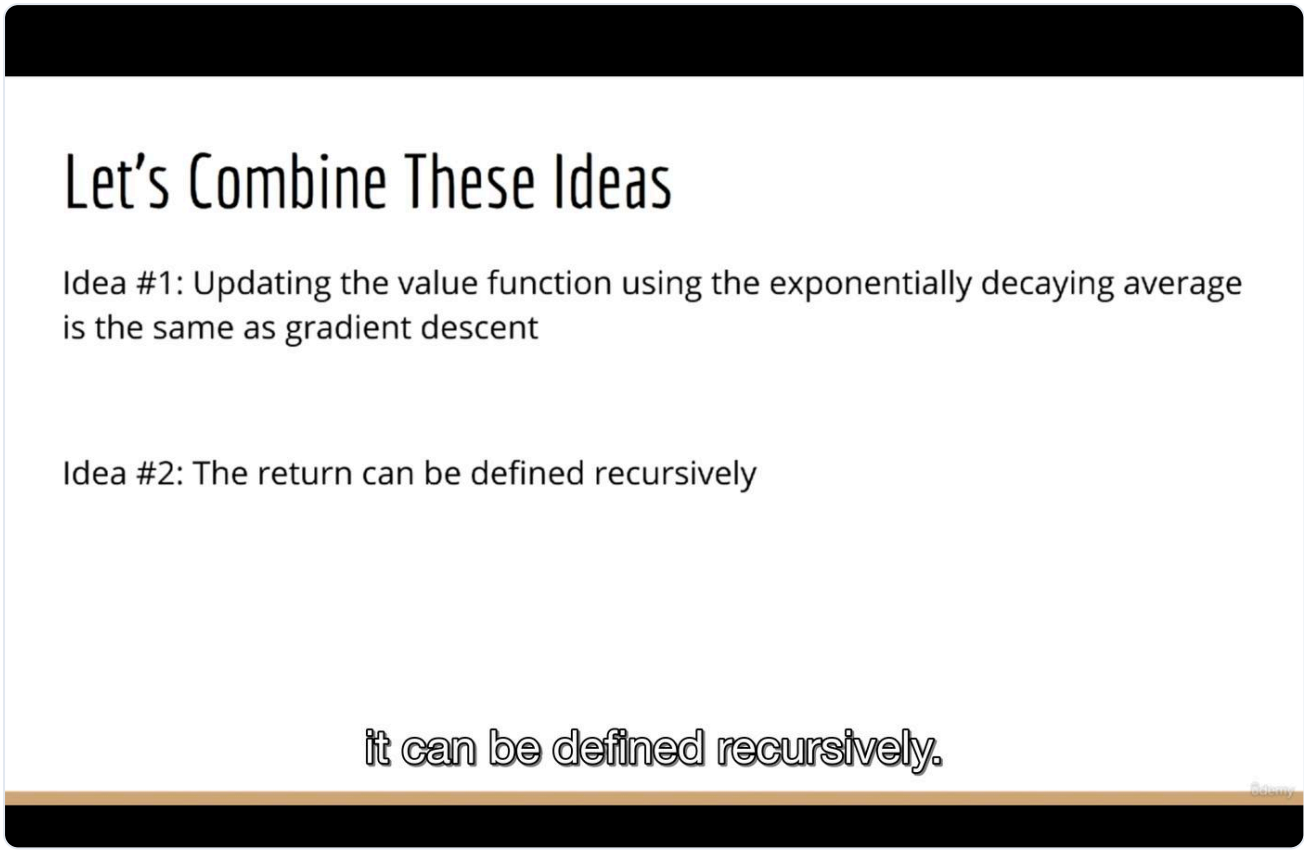

FIGURE 5.11

Well now we're going to put together these two ideas idea number one is that updating the value function using the exponentially decaying average is just like gradient descent idea number two is that the return it can be defined recursively. So how about this. We're going to continue using gradient descent but instead of using the full return we are going to estimate the return we collect the next reward. Ah but instead of waiting to collect any future rewards we simply guess that they will be close to V of s prime. The value of the state where we landed. So instead of using G equal to R Plus gamma times the next reward plus gamma squared times the next reward.

And so on. We recognize that G is just equal to r plus gamma times the next return g prime but g prime has the expected value V of s prime so we instead just say G is approximately equal to r plus gamma times V of s prime. In this way we only have to wait a 1 step before updating our model.

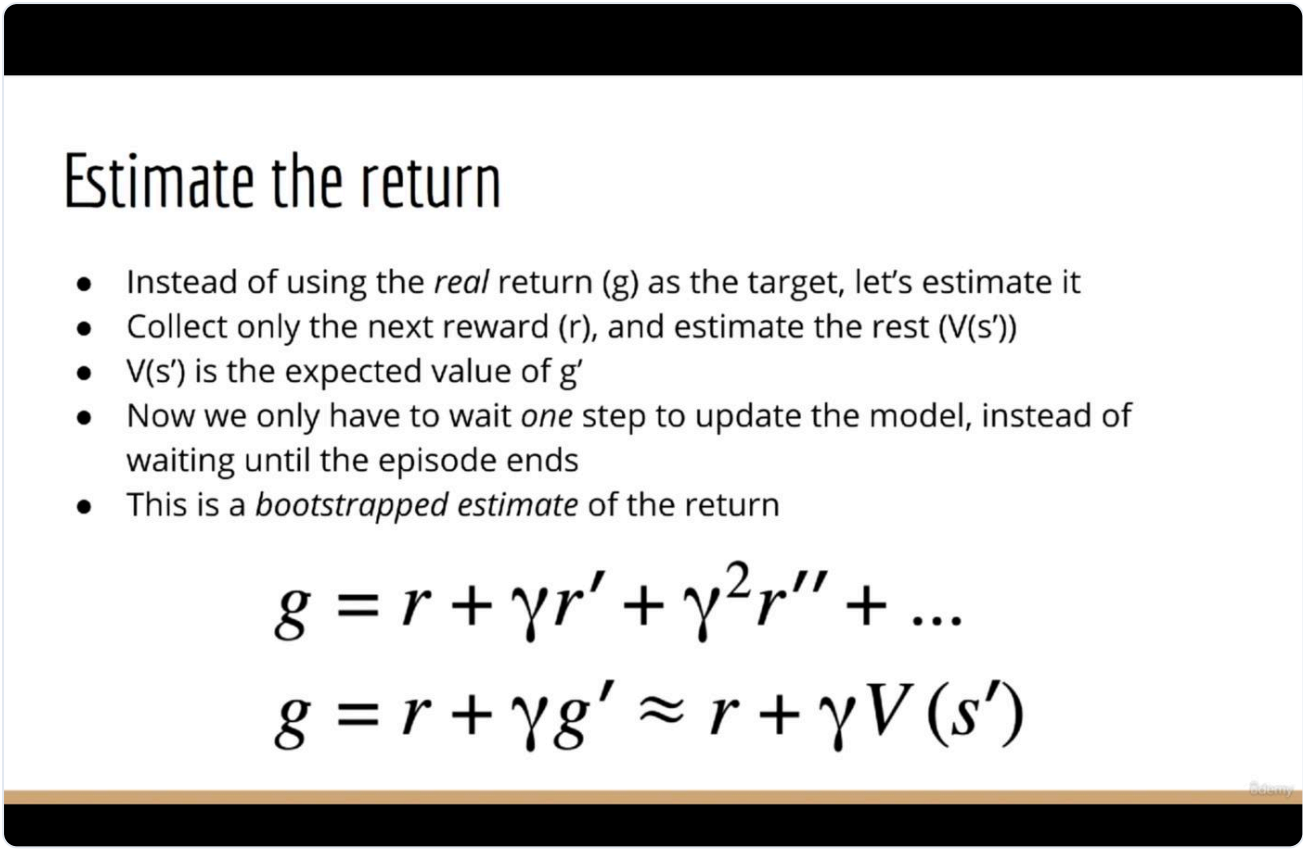

FIGURE 5.12 | Estimate the Return

We no longer have to wait until the end of the episode we call r plus gamma times V of s prime a bootstrapped estimate of the return. It allows us to update V of s immediately after obtaining the next reward R instead of having to wait until we've collected all the future rewards. This method is called the temporal difference method Here's some pseudocode so you can conceptualize how this will work as a side note. This should also give you some insight into what goes on inside our play episode function which we haven't really discussed yet. And this pseudocode. We don't have any need to abstract away the play episode function because playing the episode is part of this loop.

We have things to do on each step of the episode and therefore we can encapsulate it or delegated to some other function to start. Assume we are given some environment and some policy we initialize the value function it to be random then we enter a loop which plays a predetermined number of episodes. Alternatively you could also run the loop until you find that V of s converges or in other words settles on some value and doesn't deviate from it. Inside the loop we begin to playing in episode. The first thing we do is call env dot reset which resets the environment and puts us back into the initial state and returns that initial state we'll call it s.

Next we initialize a done boolean flag to false this boolean flag will get set to true when we've completed an episode. Next we enter a while loop that completes when done becomes true inside the loop. We grab the action from our policy we then perform the action in the environment by calling env dot step. This returns three things the next state s prime the reward R and the next. Done flag. Note that I've designed this pseudocode to be similar to the OpenAI Gym API which

has become somewhat standard over the past few years. It's easy to understand and it will help you in the future. If you ever do start using OpenAI Gym the reverse is also true. If you've had experience with OpenAI Gym in the past they should make things easier to understand.

Next we do the big update. The temporal difference update we've been discussing throughout this lecture. Finally and this is important not to forget. We must update the variable s for the next iteration of the loop. What is currently the next state s prime will become the current state s in the next iteration.

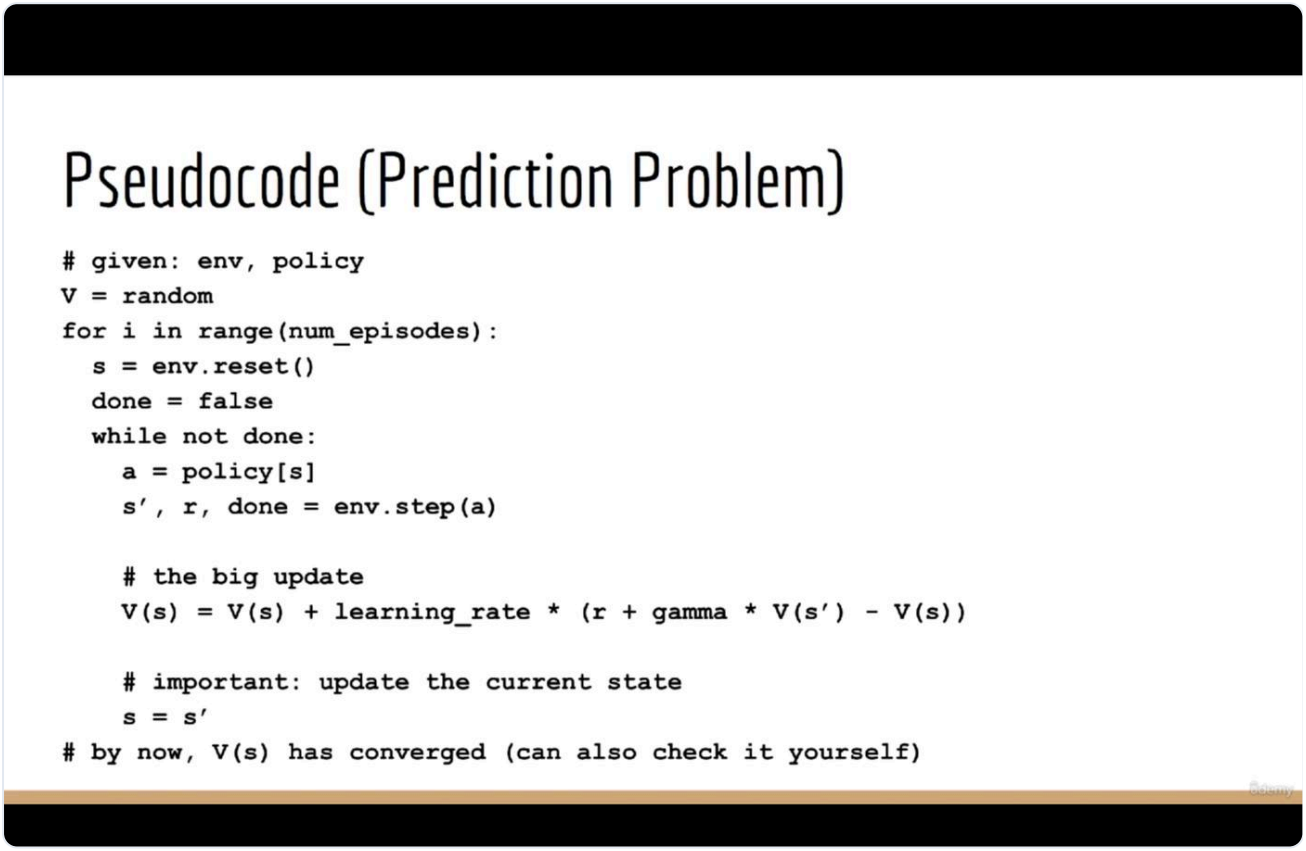

FIGURE 5.13 | Pseudocode (Prediction Problem)

Lastly when the loop is complete V of s has converged there is one odd thing about temporal difference learning. Look carefully at this so-called gradient descent update the target is r plus gamma times V of s prime. The prediction is V of s if we relate this back to supervised learning we notice something strange in supervised learning.

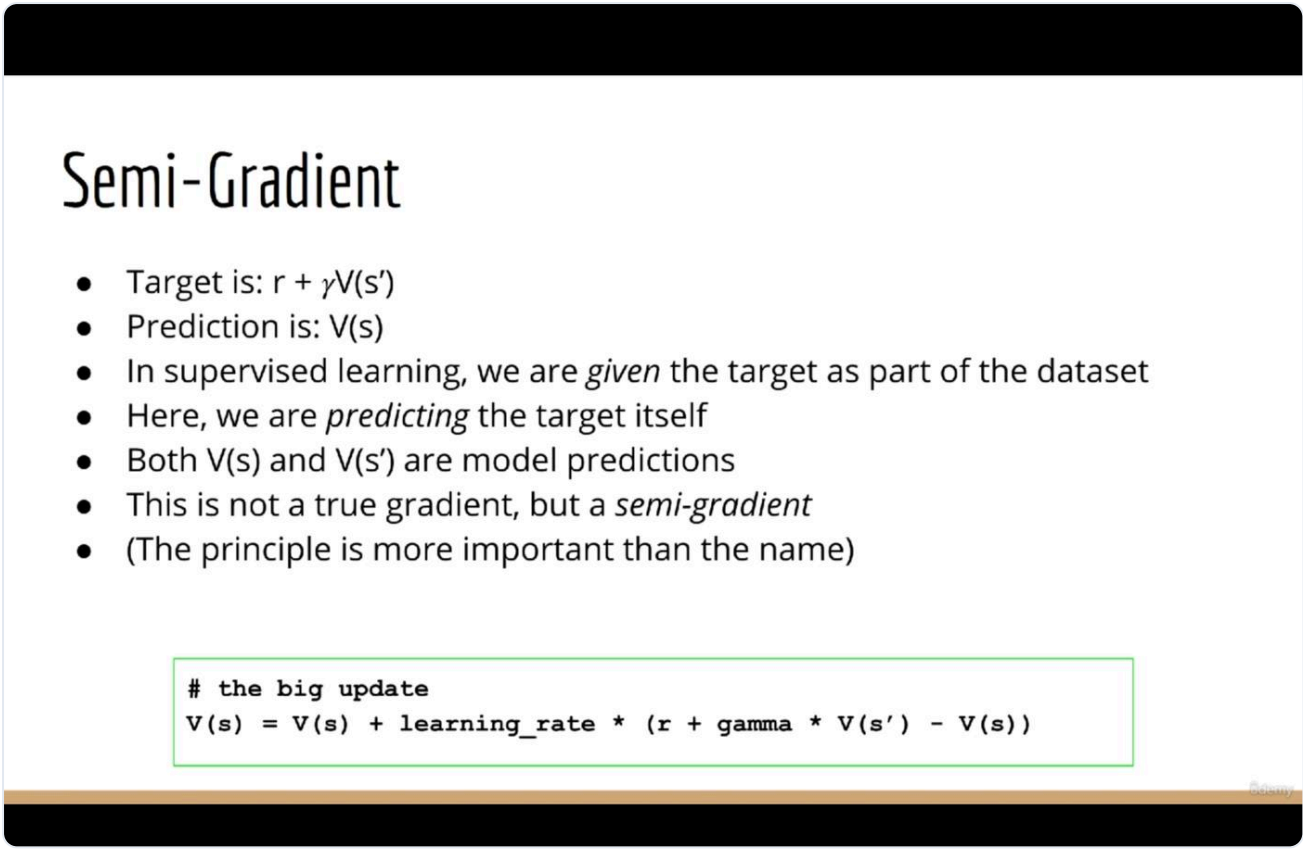

FIGURE 5.14

We are given the target as part of the dataset but here we are doing something surprising. We are predicting the target itself part of the target is given. That's the reward R but the other part V of s prime is actually a model prediction. Thus it's more correct to say that what we are doing is not quite gradient descent. It's called a semi gradient instead but this is just the name.

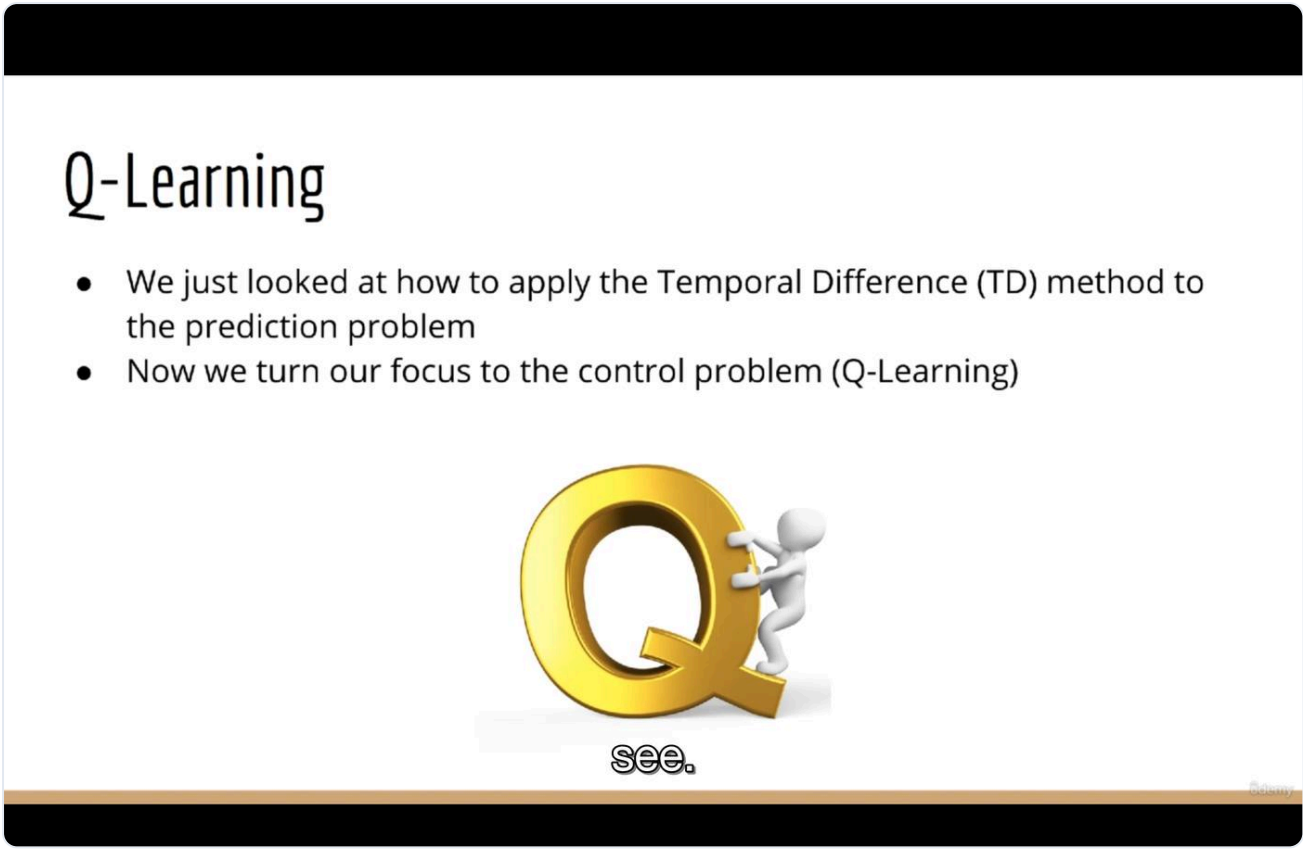

FIGURE 5.15

It is the principle that's important. The principle is we don't know the true target. We are estimating it. And this makes it very different from supervised learning.

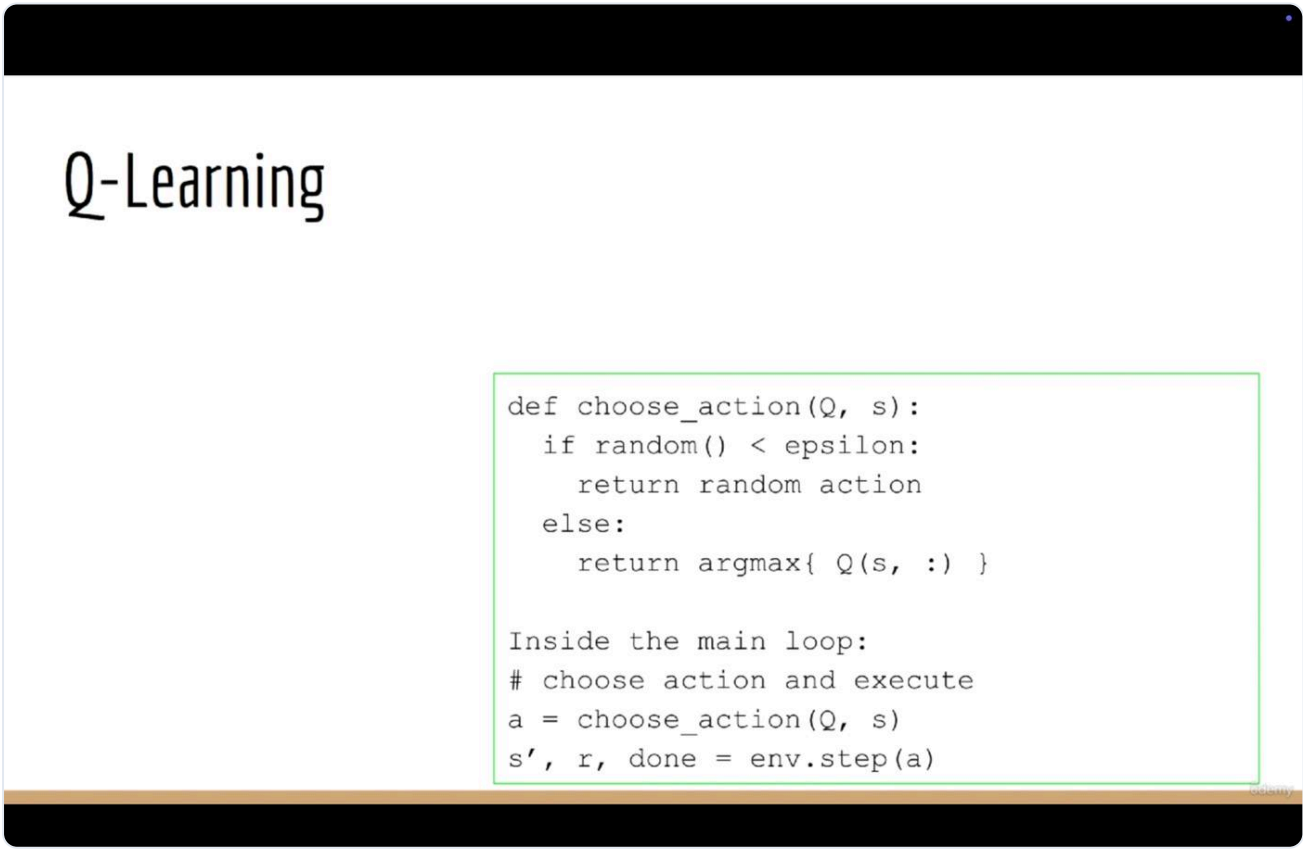

FIGURE 5.16

All right. So what we looked at so far was the prediction problem. Now it's finally time for the big reveal. We are going to solve the control problem using the famous Q learning algorithm at this point we spent so long building up the prerequisites for Q learning that you should hardly be surprised that what you see. Nonetheless let's have a look as before because Q learning is a control algorithm we are interested in updating Q rather than updating V at a high level.

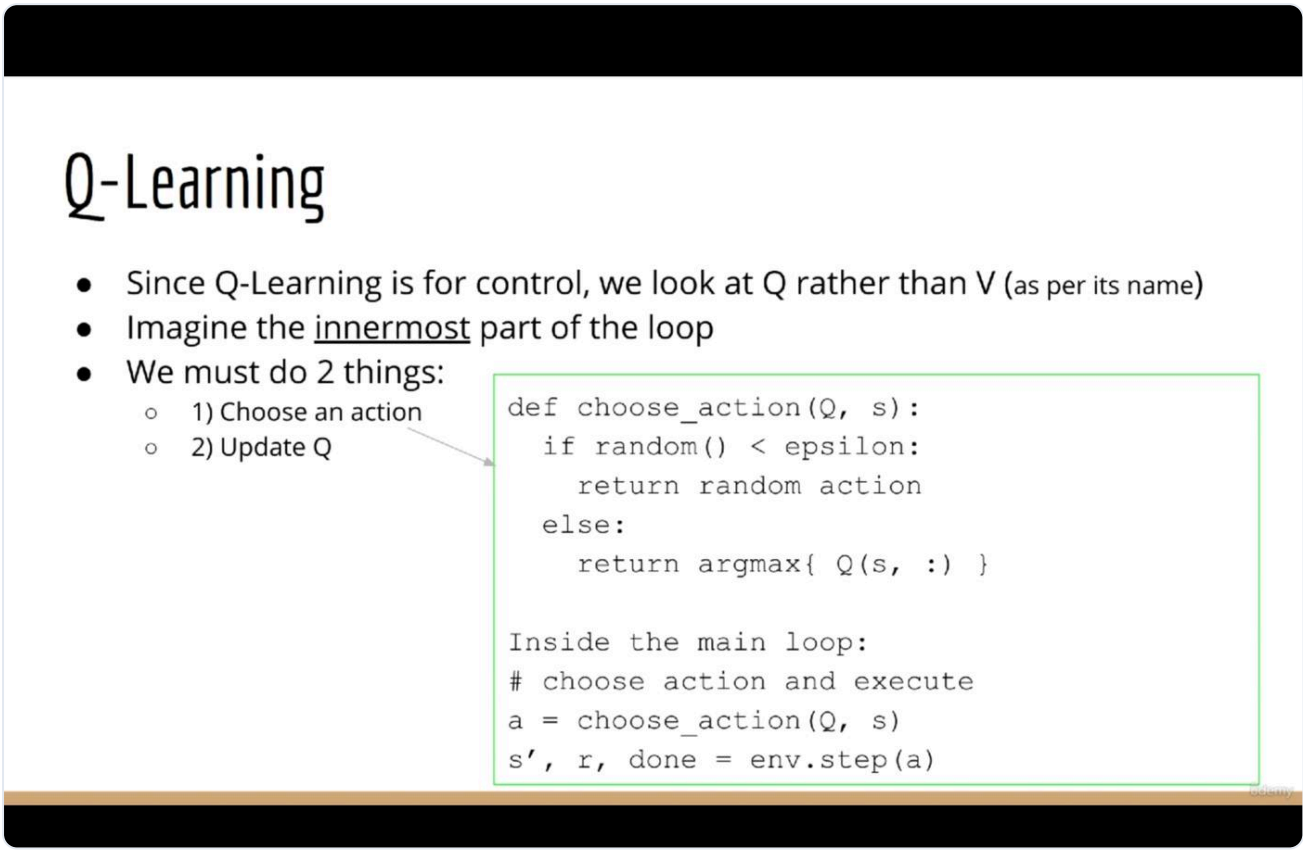

FIGURE 5.17

We are mostly interested in the innermost part of the loop. That is where we choose and action and take a step in the environment and where we update the Q table. So these are the two pieces we are going to focus on here when we choose an action and we are again going to use an epsilon greedy approach. So with a small probability Epsilon we will choose a random action.

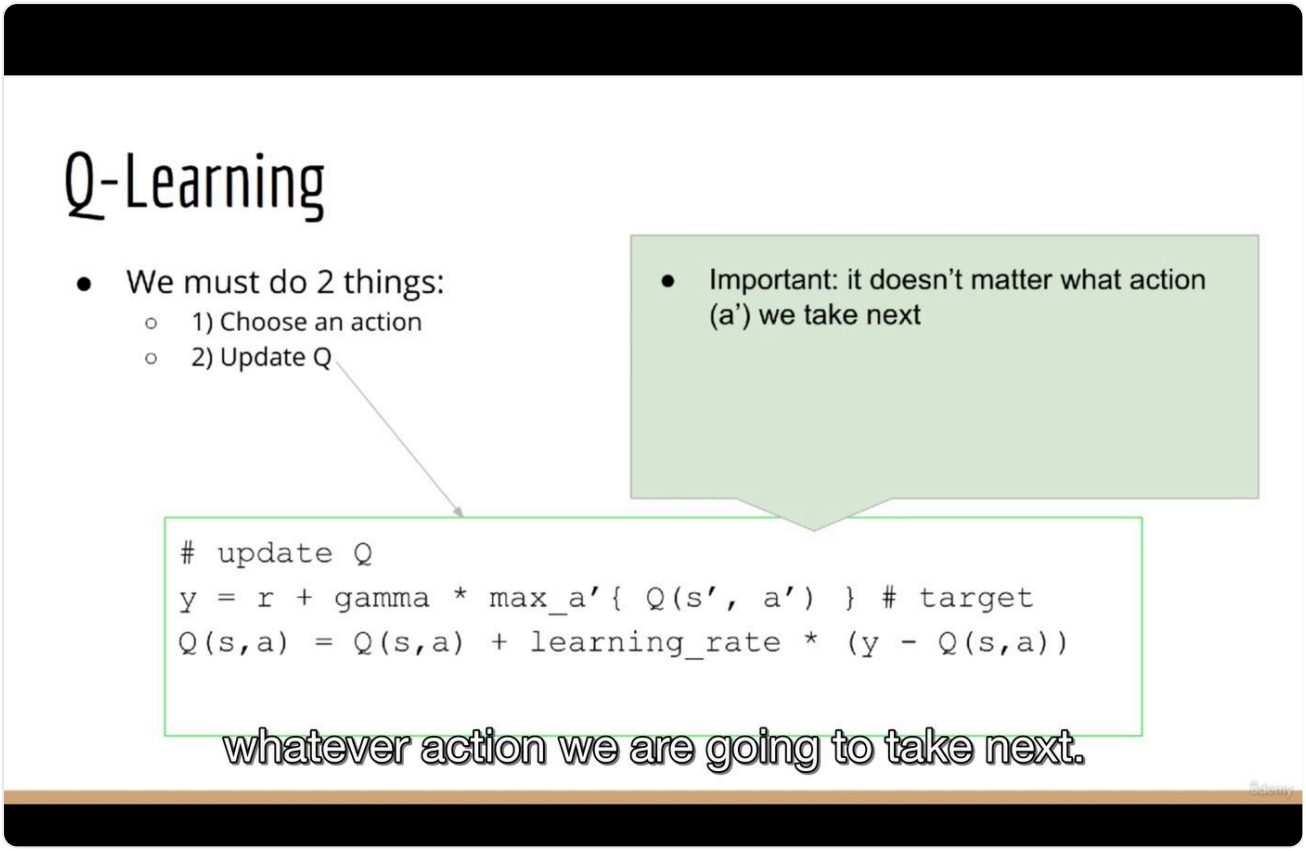

FIGURE 5.18

Otherwise we'll take the argmax over Q given a state s once we choose our action we then take that action in the environment when we update Q we do something very subtle but important that is when we calculate the target we ignore whatever action we are going to take next. Instead we assume that we will take the greedy action and take the max over Q Given the state s prime. This has two advantages. First it means that in order to update Q We don't have to wait until we obtain the next action a prime in the next iteration of the loop. Second it makes Q learning what is called an off policy algorithm.

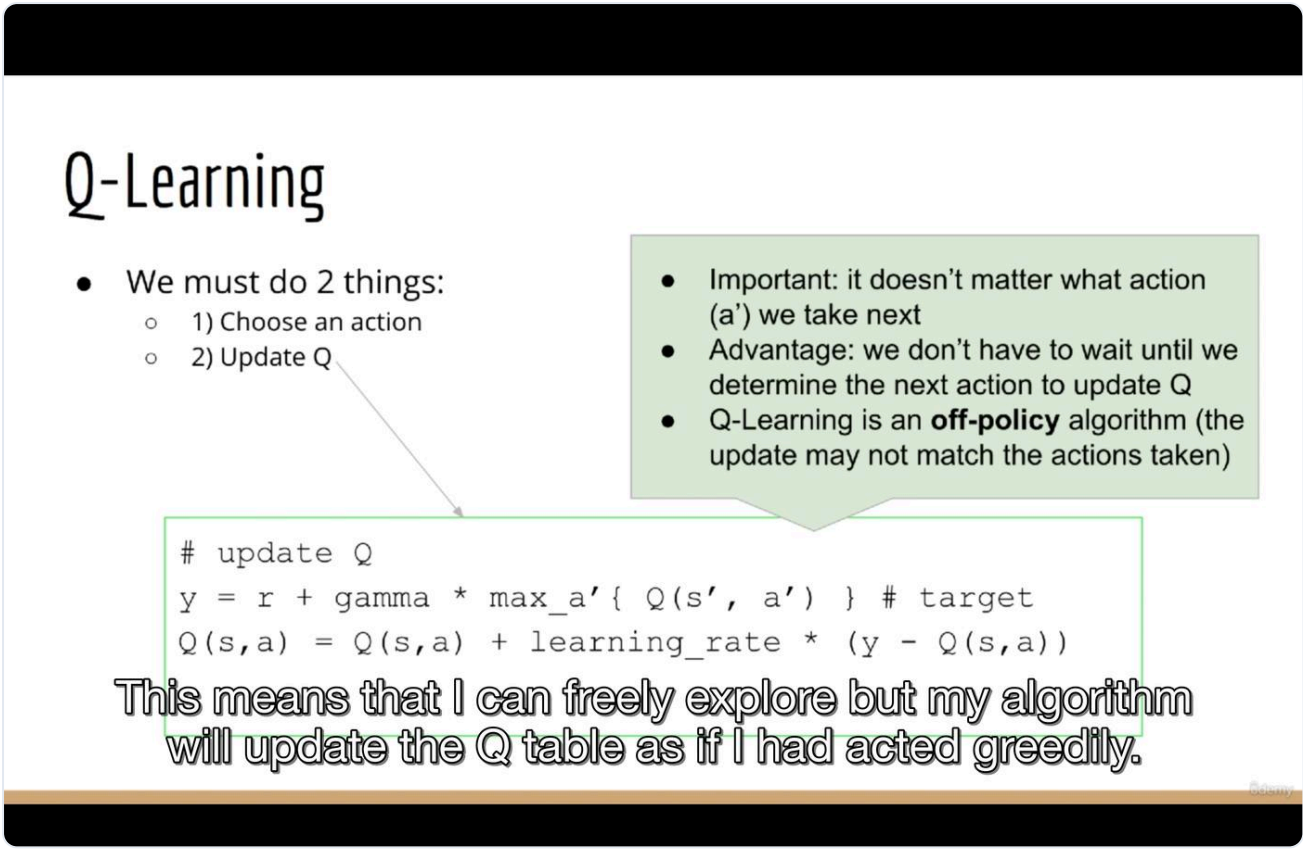

FIGURE 5.19

This means that I can freely explore but my algorithm will update the Q table as if I had acted greedily. So here's what Q learning looks like when we put it all together. Luckily it looks quite similar to the prediction problem pseudocode. First we are given some environment object. Then we initialize Q with random values. Then we enter a loop that goes for a predetermined number of episodes Optionally you can keep going until Q or the policy converges next. It's the same as what we had before we reset the environment and start back at the initial state. We initialize the done flag to false. Then we enter a loop for the episode quitting only when we are done.

Next we use epsilon greedy to choose an action. Then we take a step in the environment. We get back the state s prime the reward R and the done flag. Next we create the target for the Q update. Taking the max over Q Given the state s prime next we update Q of s a using the target we just calculated.

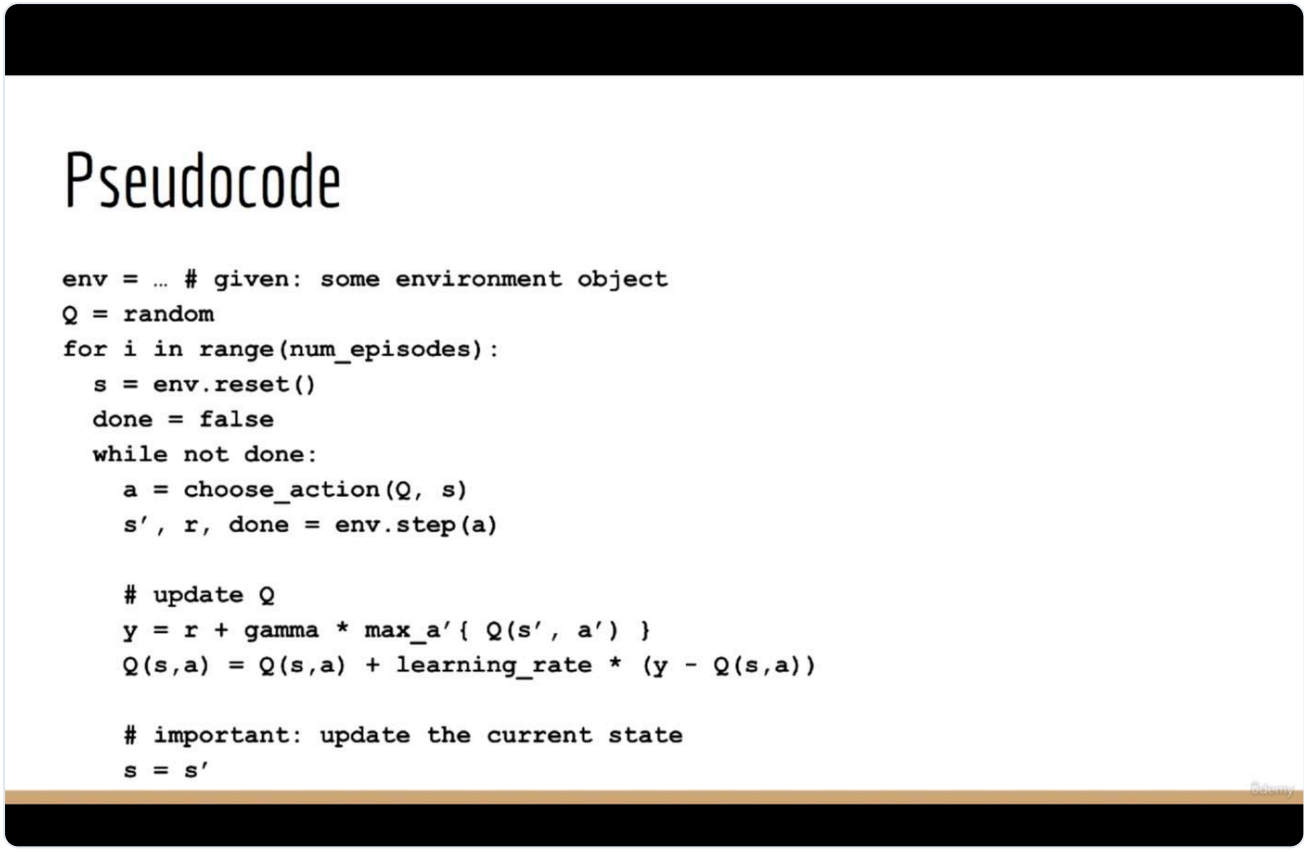

FIGURE 5.20

Lastly we update the current state s to be the next state s prime. Finally when we exit the loop we have found the optimal policy let's summarize what we just did. Since that was pretty long first we started by noting that Monte Carlo will not work for infinitely long episodes and that it's problematic in that we have to wait until an episode is over in order to do any learning. Instead we make use of the fact that the return can be defined recursively. This allows us to approximate the return using only the next reward and the value at the next state. We also learn that reinforcement learning ultimately starts to look like supervised learning where our target is not a real target but rather an estimation made by our own model.

We are essentially doing gradient descent on a Q table using this approach.

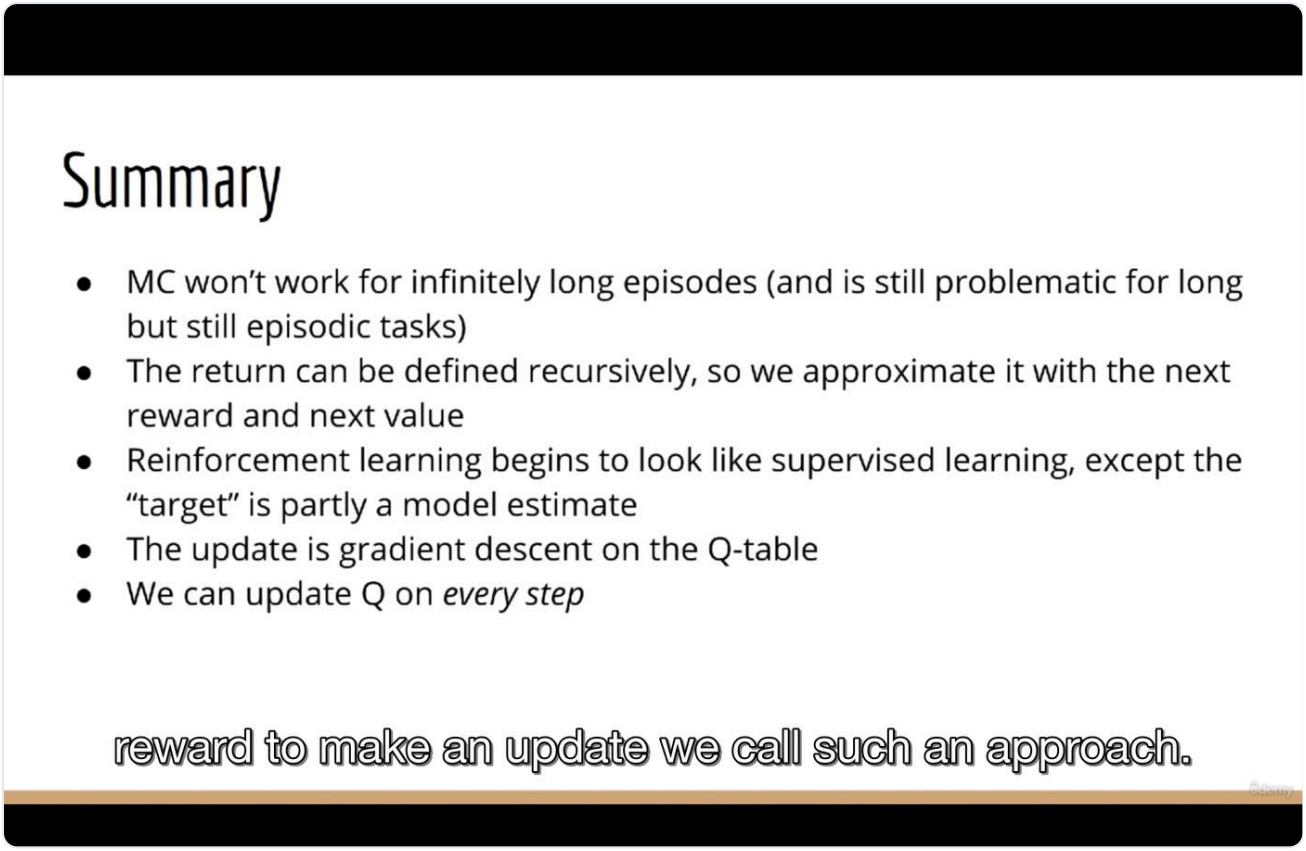

FIGURE 5.21

It allows us to update our Q table on every step because we only have to wait until we receive a single reward to make an update we call such an approach.

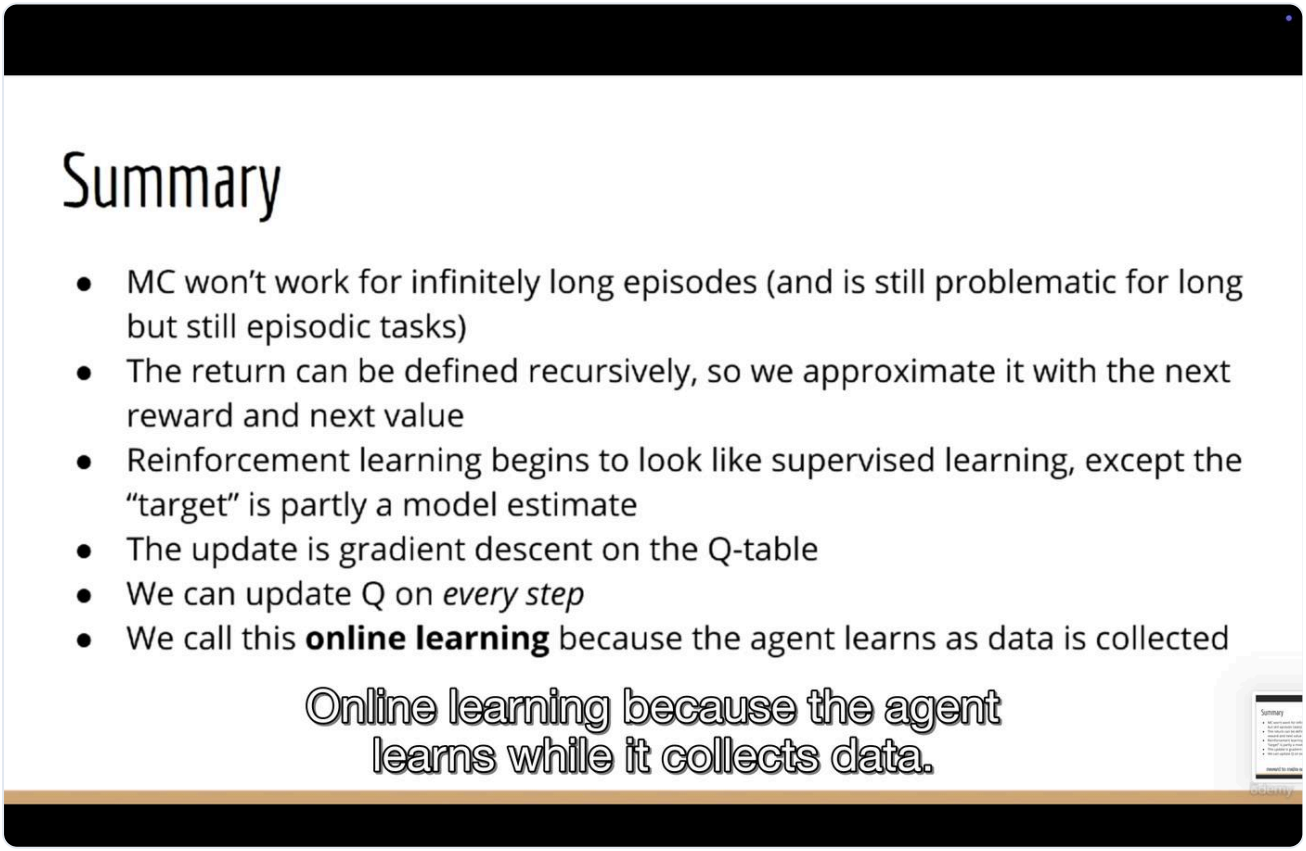

FIGURE 5.22

Online learning because the agent learns while it collects data.

LECTURE 99

Using V(s) for Policy Improvement

In this lecture we are going to extend our knowledge of basic Q learning to deep Q learning that is Q learning that uses neural networks to understand the motivation behind this. Consider what we have been doing so far. Q.

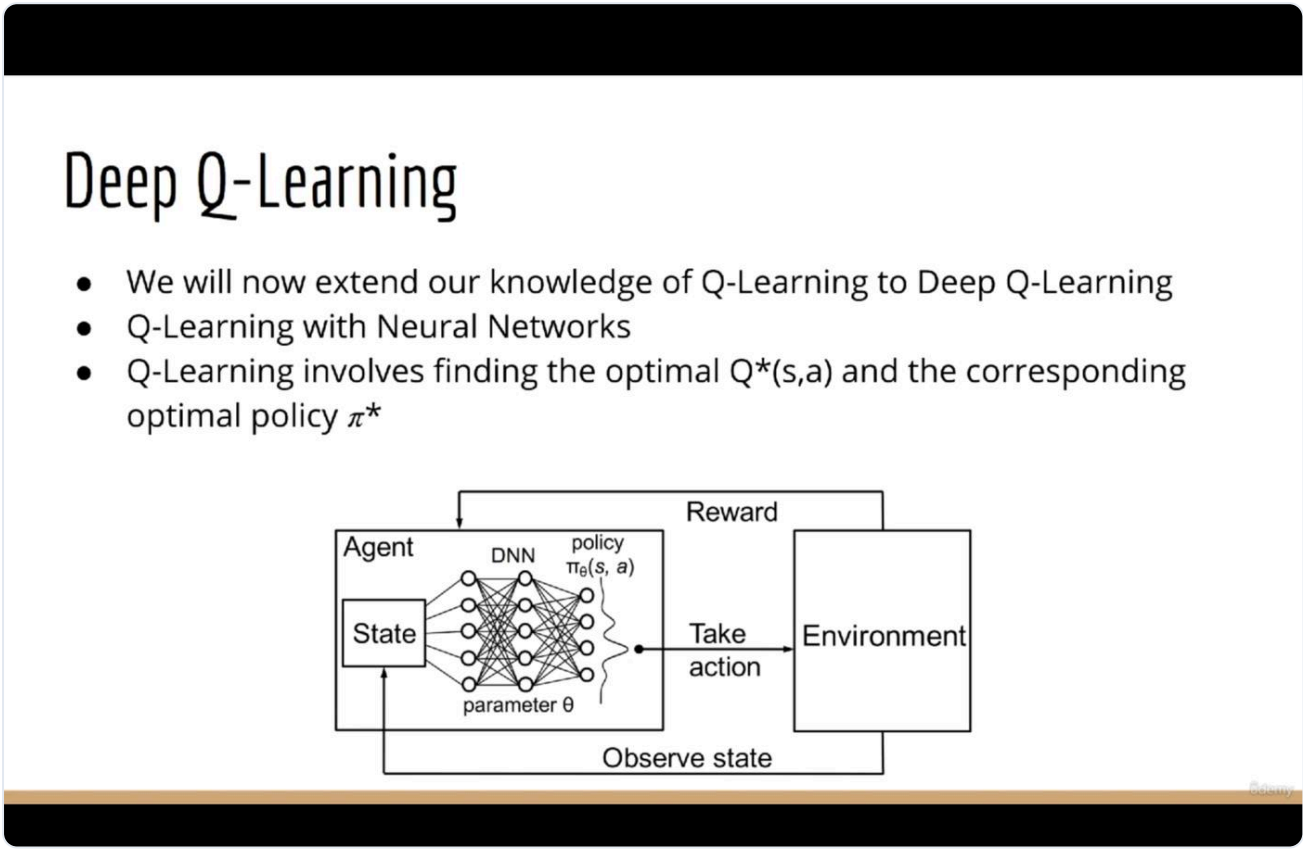

FIGURE 6.1 | Q-Learning with Neural Networks

Learning involves finding the optimal Q of S and a and the corresponding optimal policy. As you recall just like with supervised learning targets we can think of the states as categories and the actions as categories in a computer. We would represent them with integers starting from zero. For this reason we call Q of s a a Q table it's a table as in something that looks like a data frame or a matrix.

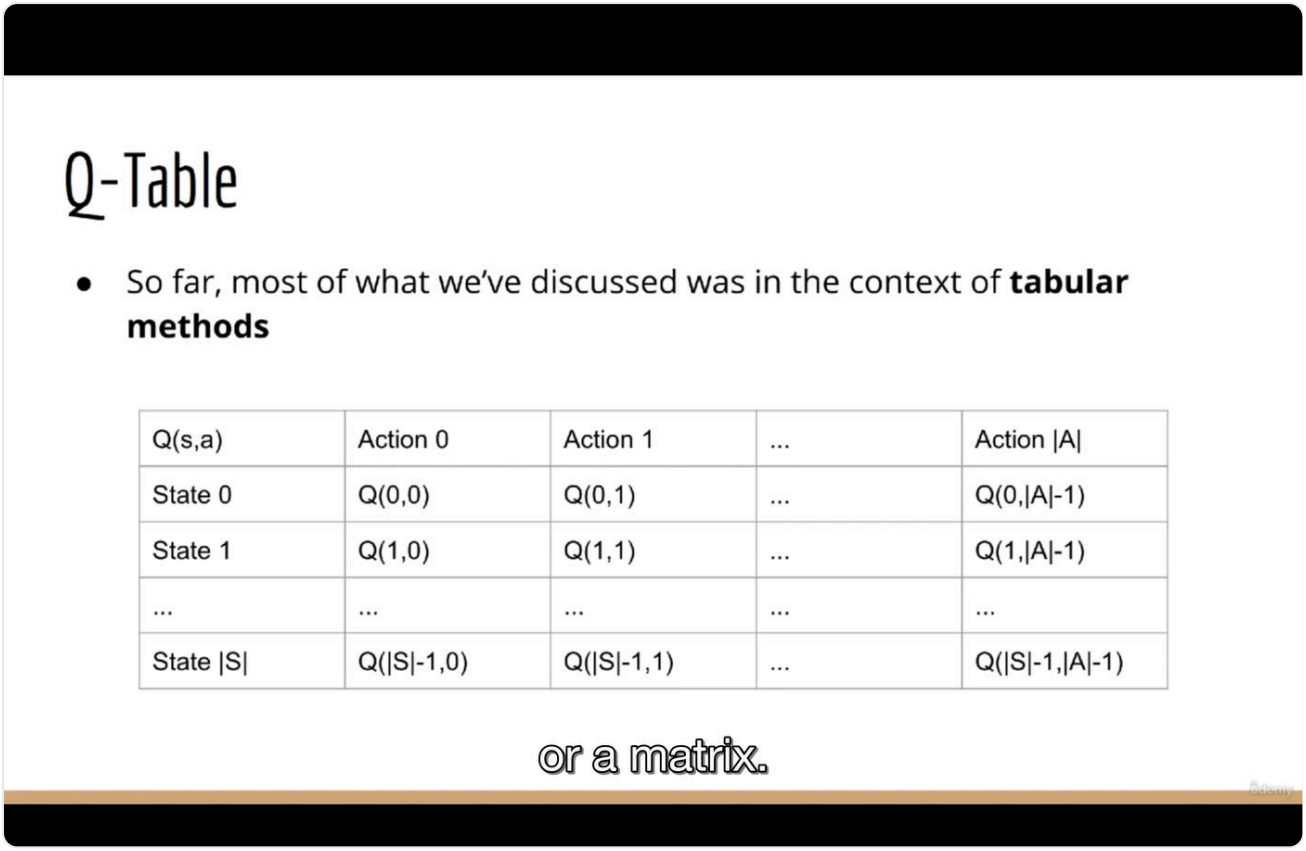

FIGURE 6.2

Q zero zero means the action value for state zero and action zero. This is nothing but a two dimensional array methods which use Q tables are called tabular methods for obvious reasons. In this section we haven't spent too much time explaining in detail how to implement tabular methods because my goal was to get you to deep Q learning as quickly as possible.

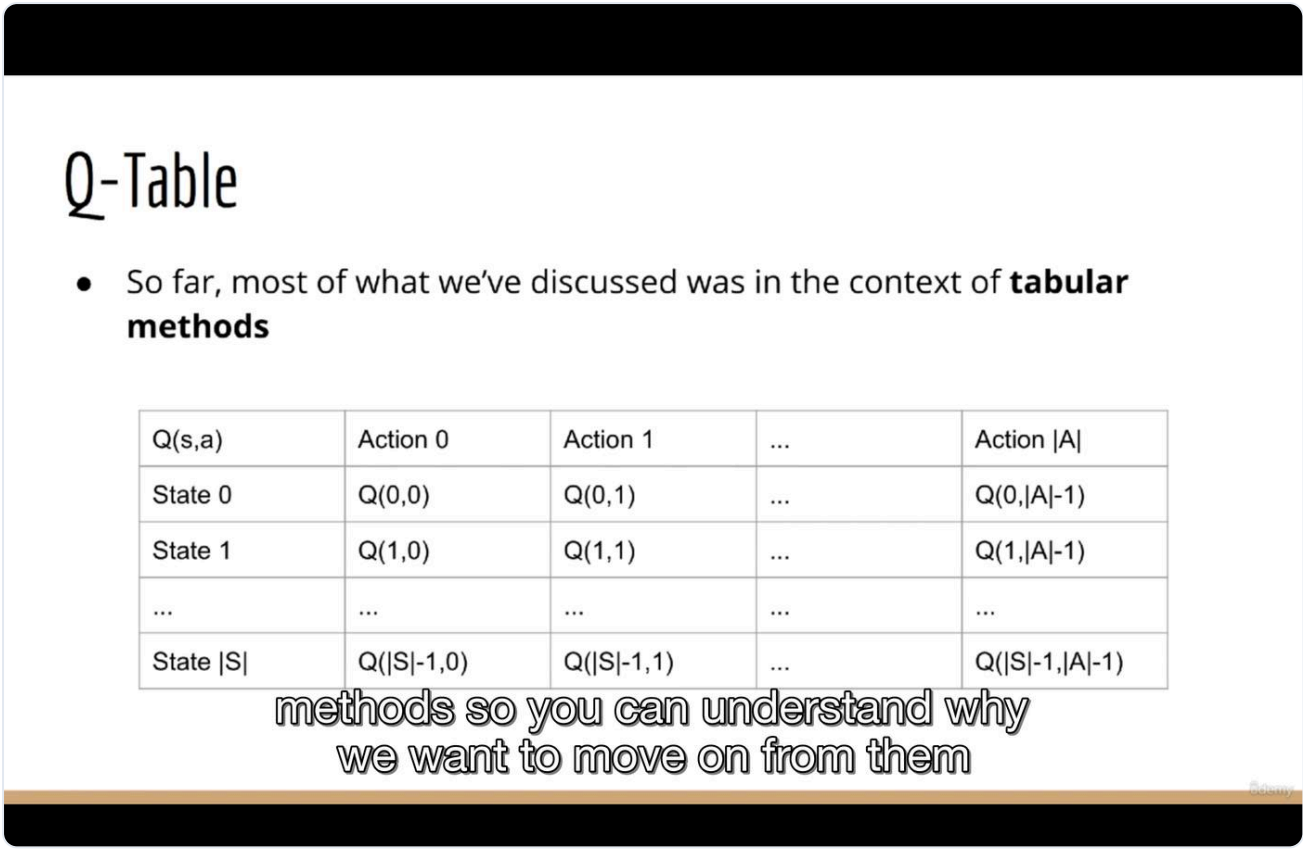

FIGURE 6.3

However it's important to realize that what we have been discussing so far was in the form of tabular methods so you can understand why we want to move on from them consider the scenario where your state space or your action space is continuous or infinite. It should be clear that storing a Q table is no longer a viable option. We cannot have an infinitely sized table. One option which would be relatively simple to implement is to use the binning approach to force your state space or action space to be discrete.

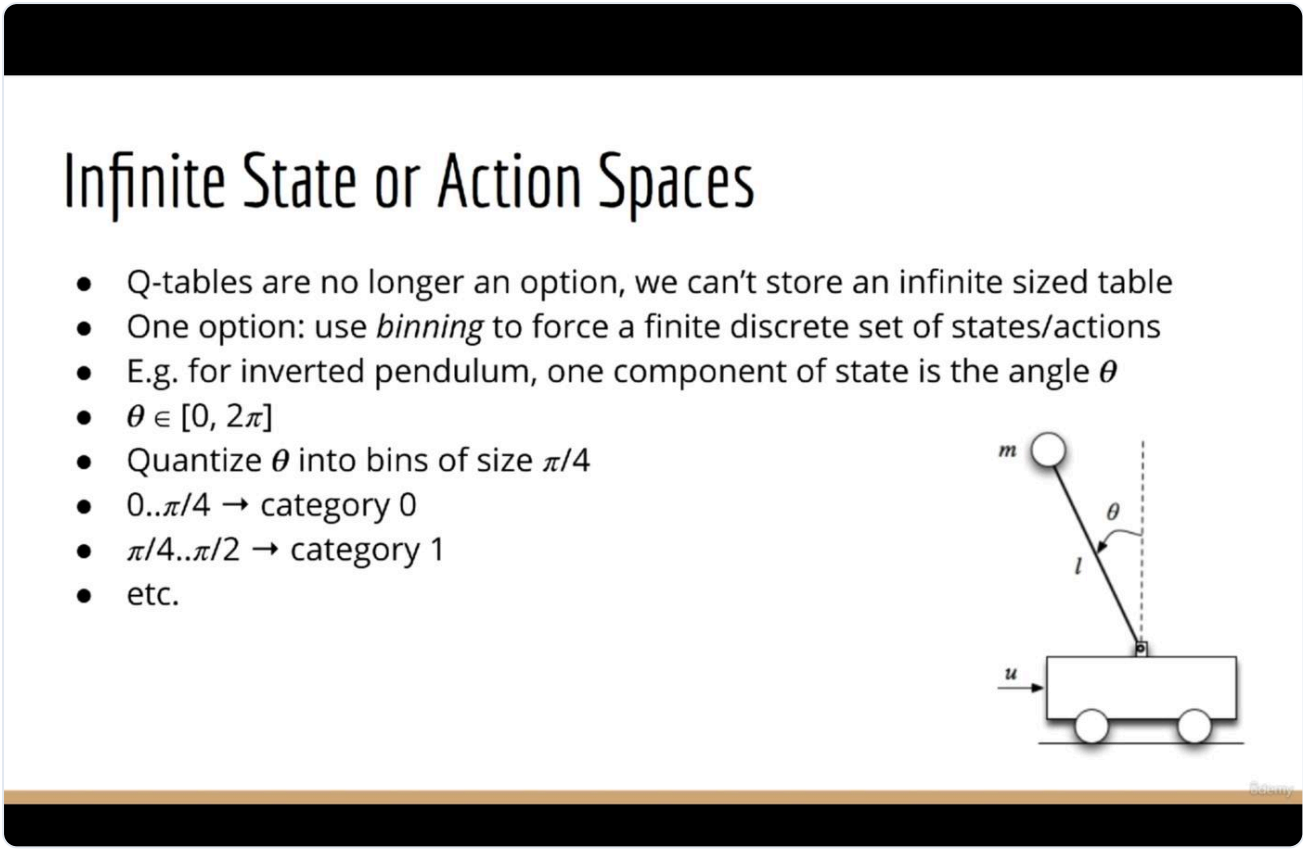

FIGURE 6.4 | Infinite State or Action Spaces

For example suppose you're playing the inverted pendulum game in one of your states is the angle of the pendulum the angle of course must be between zero and 2 pi. Thus we can do something like break this up into equal sized bins of size pi over four so any angle between a zero and pi over four would be considered category 0 pi over four to pi over 2 would be considered Category 1 and so on. This is a fine approach but it can only take you so far instead. If we want to do something more flexible we have to make use of machine learning.

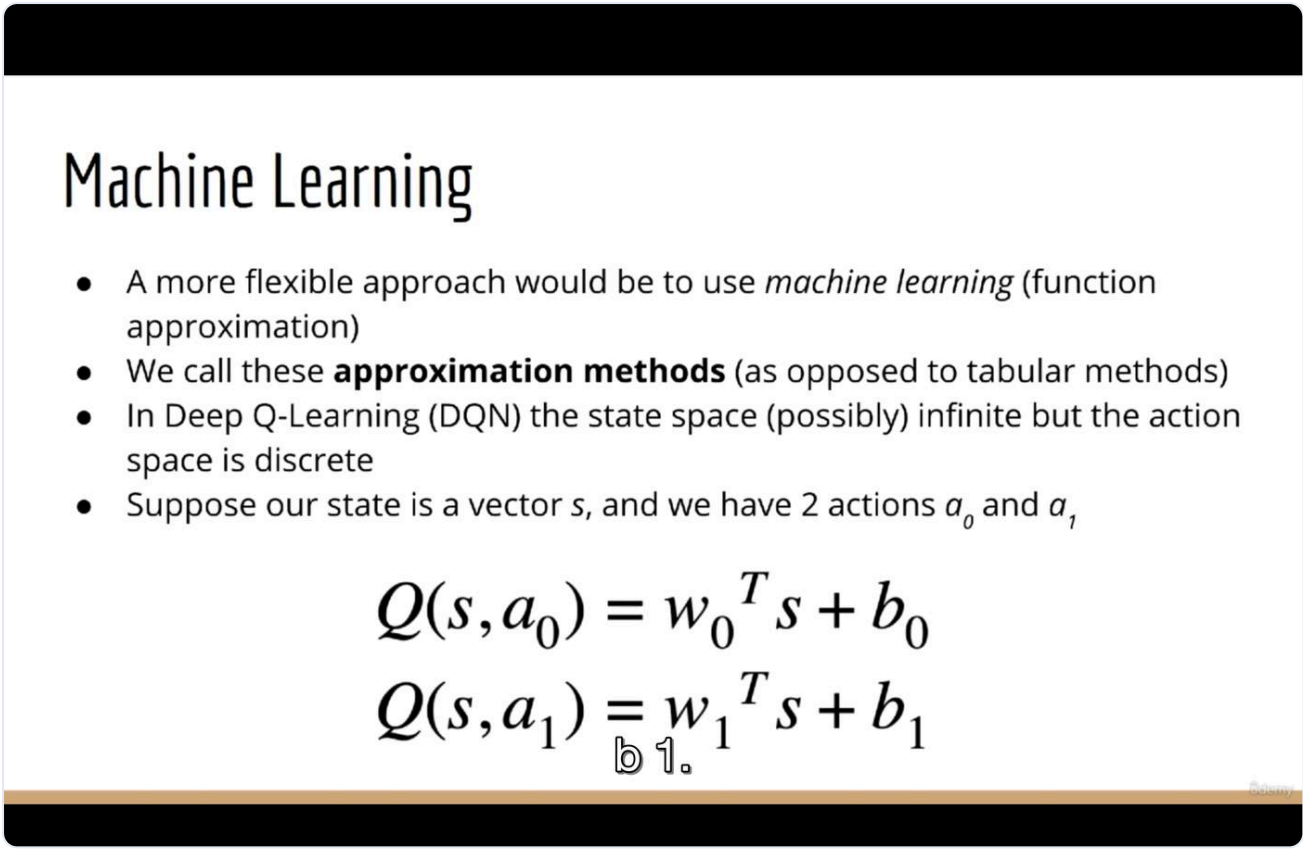

FIGURE 6.5 | Machine Learning

We call these approximation methods because we are using machine learning models as function approximators for deep Q learning. We will consider the case where the state is continuous but the action is discrete because the state is continuous. We can't use a Q table. Instead let's suppose our state is represented by a continuous vector s and let's say for simplicity's sake that we have two actions a zero and a one in this case we can represent Q of s a zero as the dot product between W 0 and s plus B zero we can similarly represent Q of S and a 1 as w 1 transpose s plus b 1. In other words we can approximate the Q value for both actions as linear regressions.

Of course there is no need to represent the weights separately. Instead we can combine all the weights into a weight matrix and combine all the bias terms into one bias vector. If the state is a continuous vector of size D and we have k different actions then the weight matrix will be of size D by K and the bias vector will be of size k.

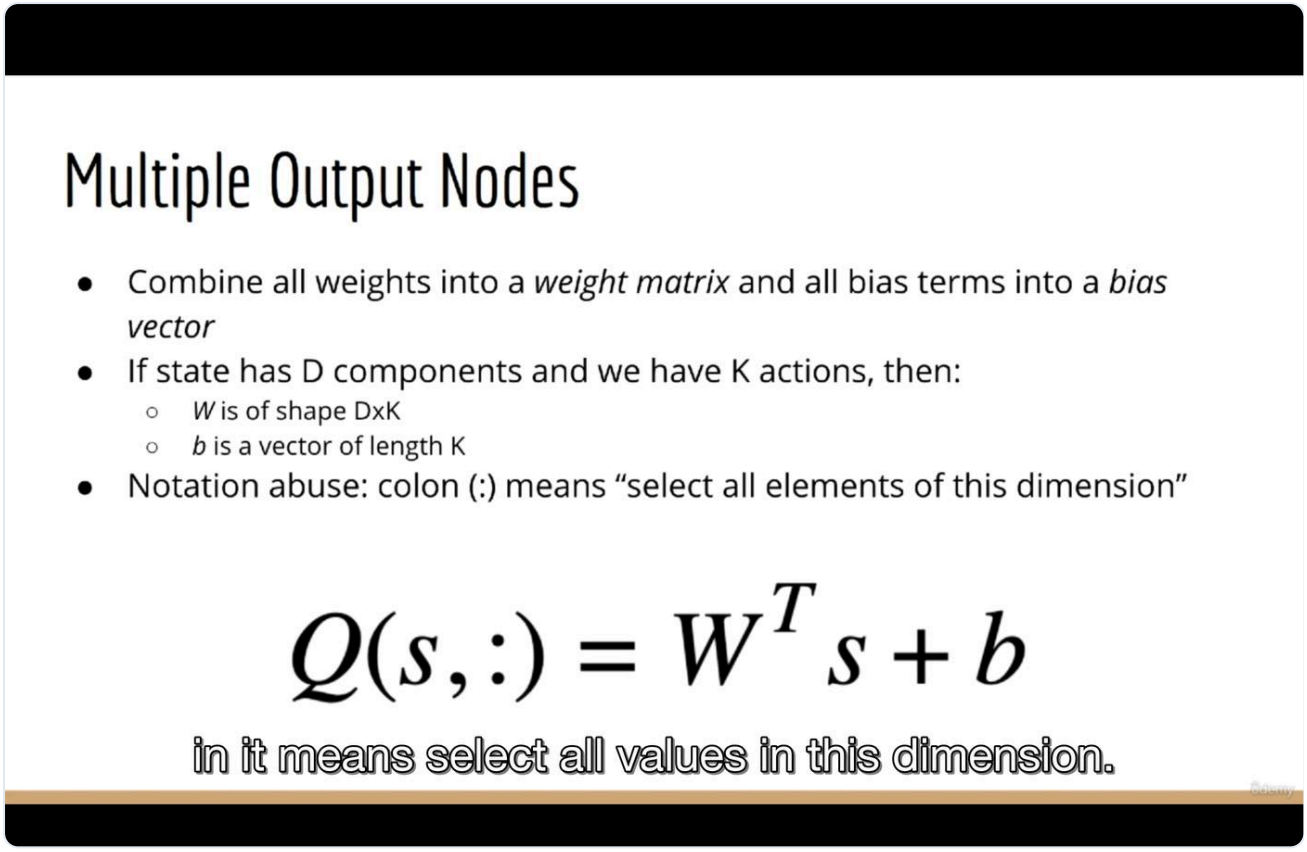

FIGURE 6.6

Notice how I'm abusing notation here a little bit by mixing in NumPy notation into our math the colon means select all values in this dimension. So this is the Q value given the state s over all actions one example of this is the inverted pendulum task in this task. Your state consists of four components the position of the car the velocity of the car the angle of the pendulum and the angular velocity of the pendulum. We can write this using the notation x x Di theta theta. The action in this environment is the amount of force to apply to the car.

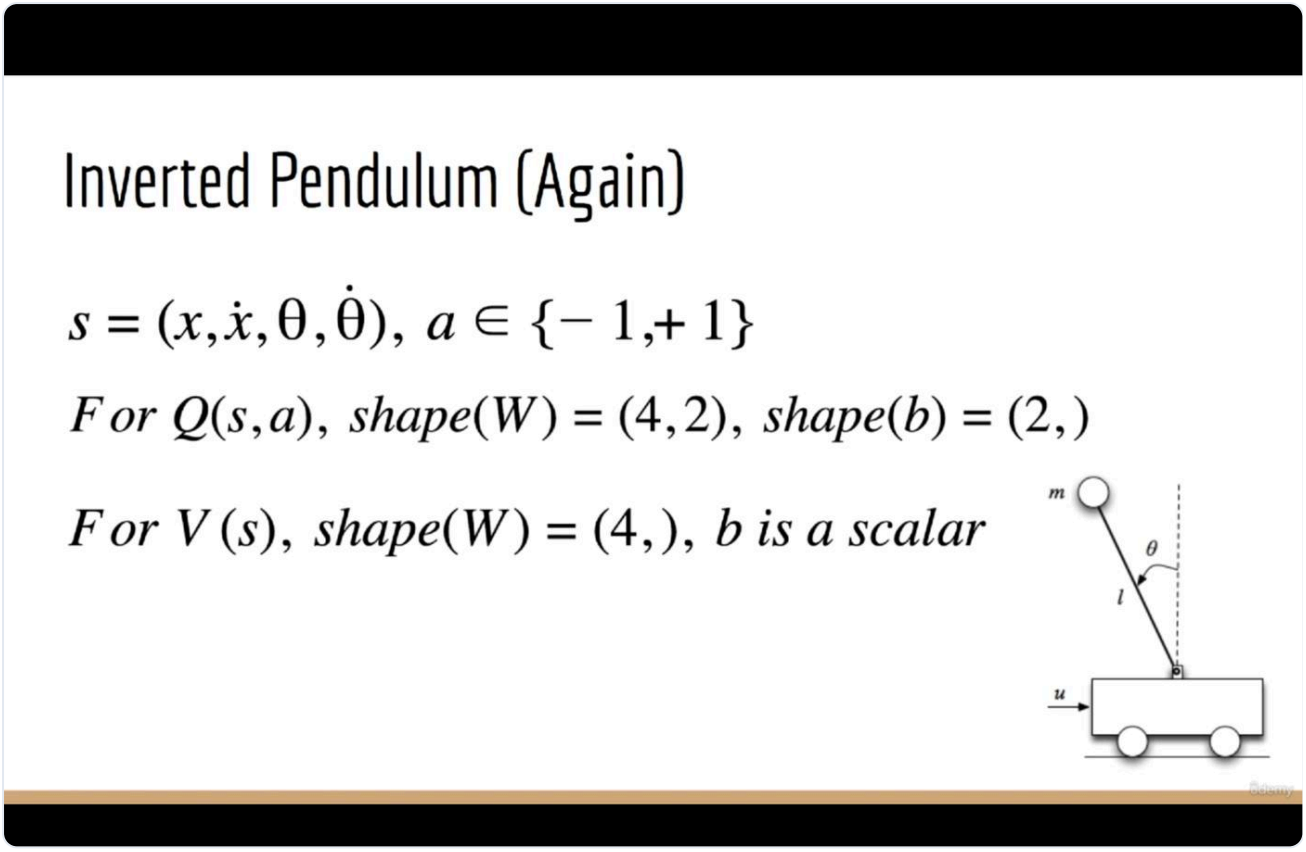

FIGURE 6.7 | Inverted Pendulum (Again)

Although this could be continuous since force is a continuous variable environments like CartPole in OpenAI Gym usually discretize this force. So you can only apply a force of either plus 1 or minus 1 in this case for the action value. The weight matrix would be a size 4 by 2 and the bias vector would be a size 2. If we're predicting the state value V of S then the output is a scalar. In this case the weight would be a vector of size 4 and the bias would be a scalar previously when we discussed tabular temporal difference learning we would update V of s and Q of s and a directly because those were just entries in a table now.

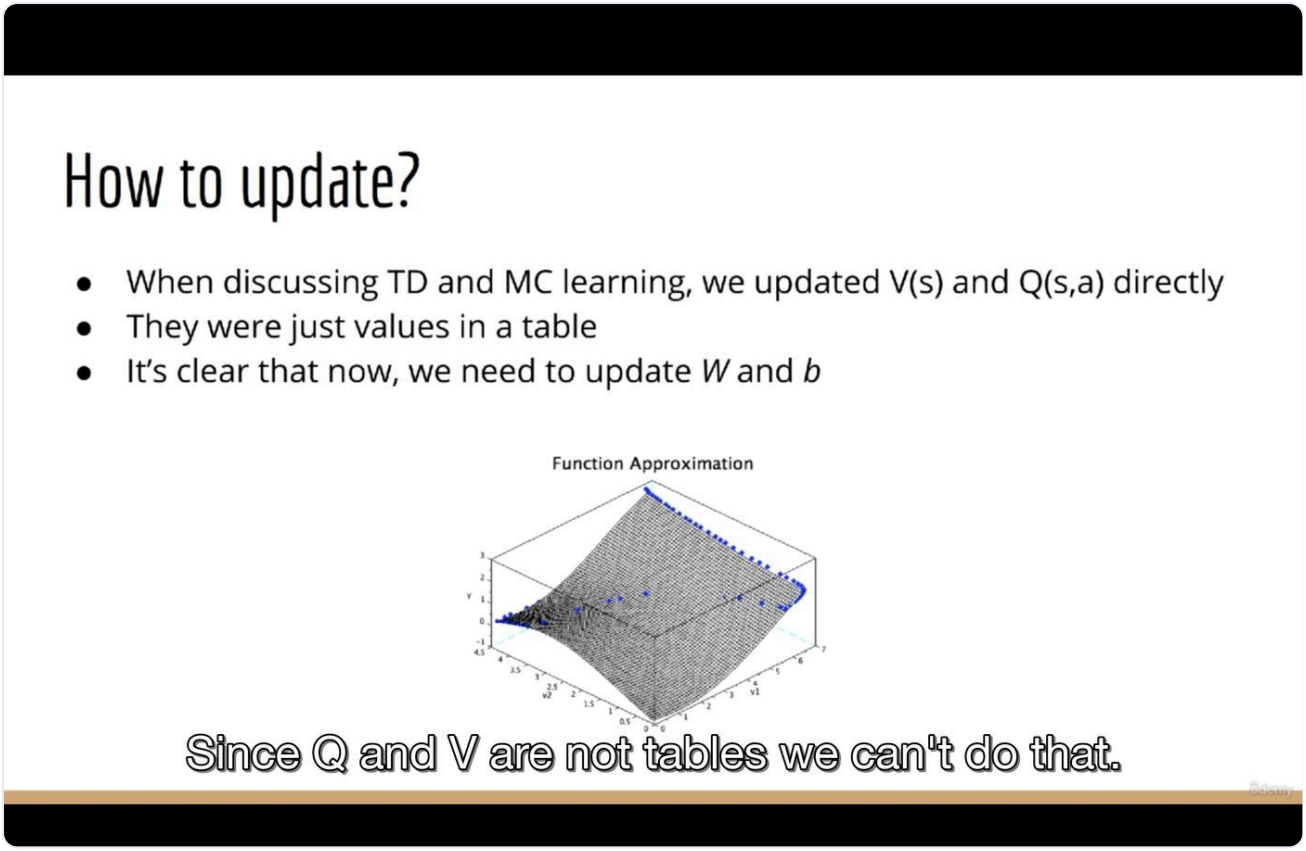

FIGURE 6.8 | How to Update?

Since Q and V are not tables we can't do that. So what do we do instead. Well it's clear that what has to be updated are the weight matrix and biased vector let's start with the prediction problem finding V of s here is where treating the problem as a supervised learning problem becomes important. What is the target. What is the prediction and what are we trying to update. Let's list these out the target is the return or in the case of Q learning. It's the estimation of the return r plus gamma times V of s prime. One exception to this is if s prime is a terminal state.

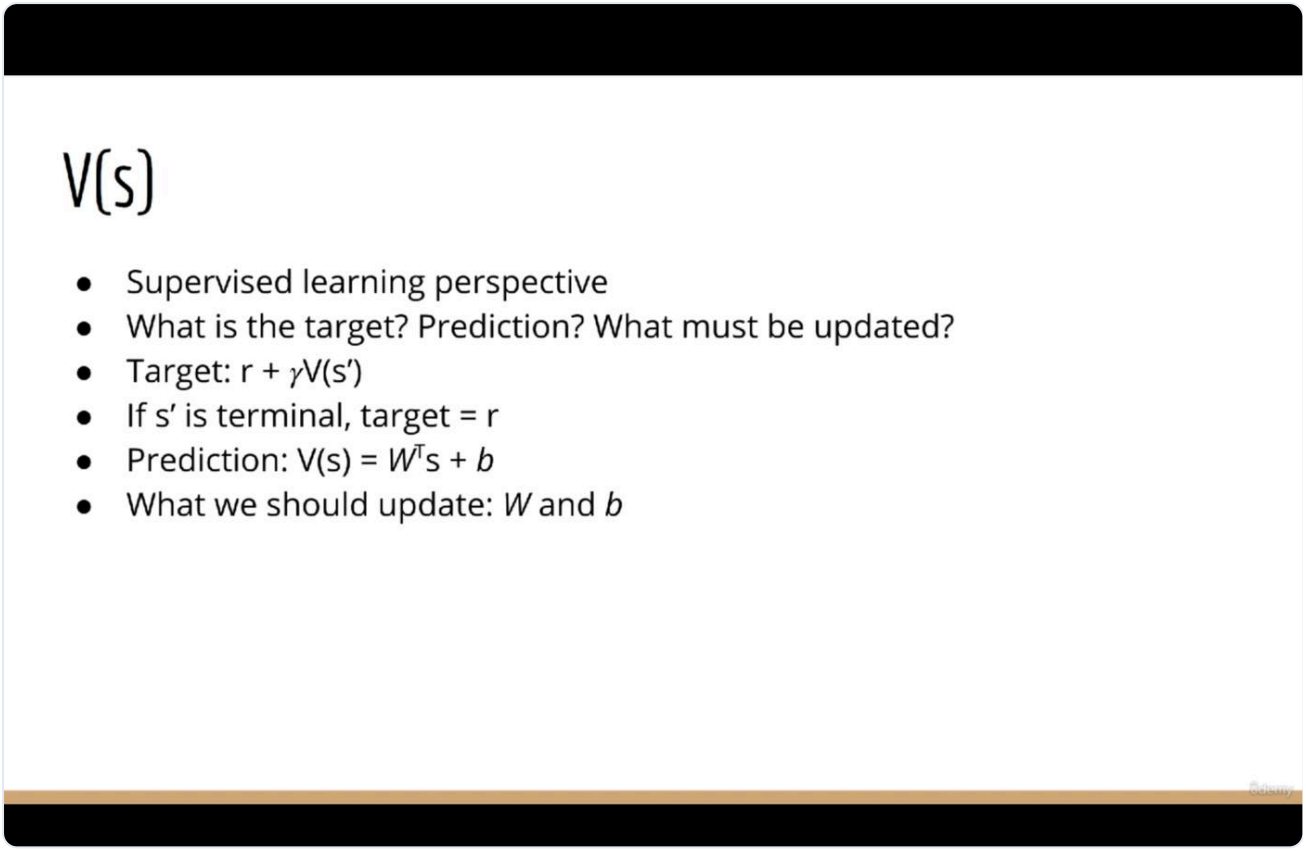

FIGURE 6.9 | What Is the Target? Prediction? What Must Be Updated?

We know that a terminal state represents the end of an episode. So there are no further states and hence no further rewards. Therefore if s prime is terminal the target is simply the reward. R next what is the prediction. The prediction is V of s and what do we want to update. We want to update the linear regression parameters W and b since this is a regression. We know that we should use the squared error and do gradient descent to update the parameters.

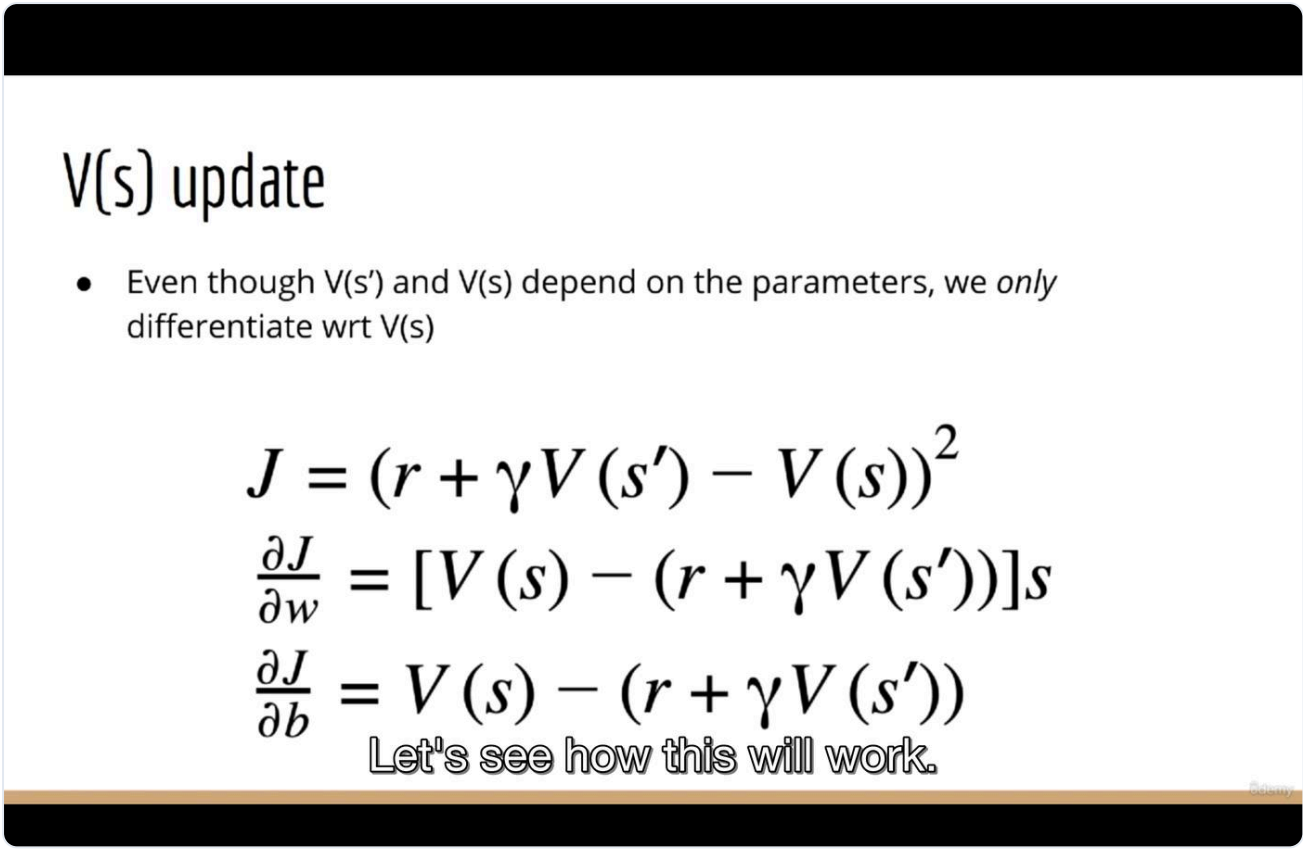

FIGURE 6.10

Let's see how this will work. First we set up our loss J equals R Plus gamma times V of s prime minus V of s all squared. Next we find the gradient of J.

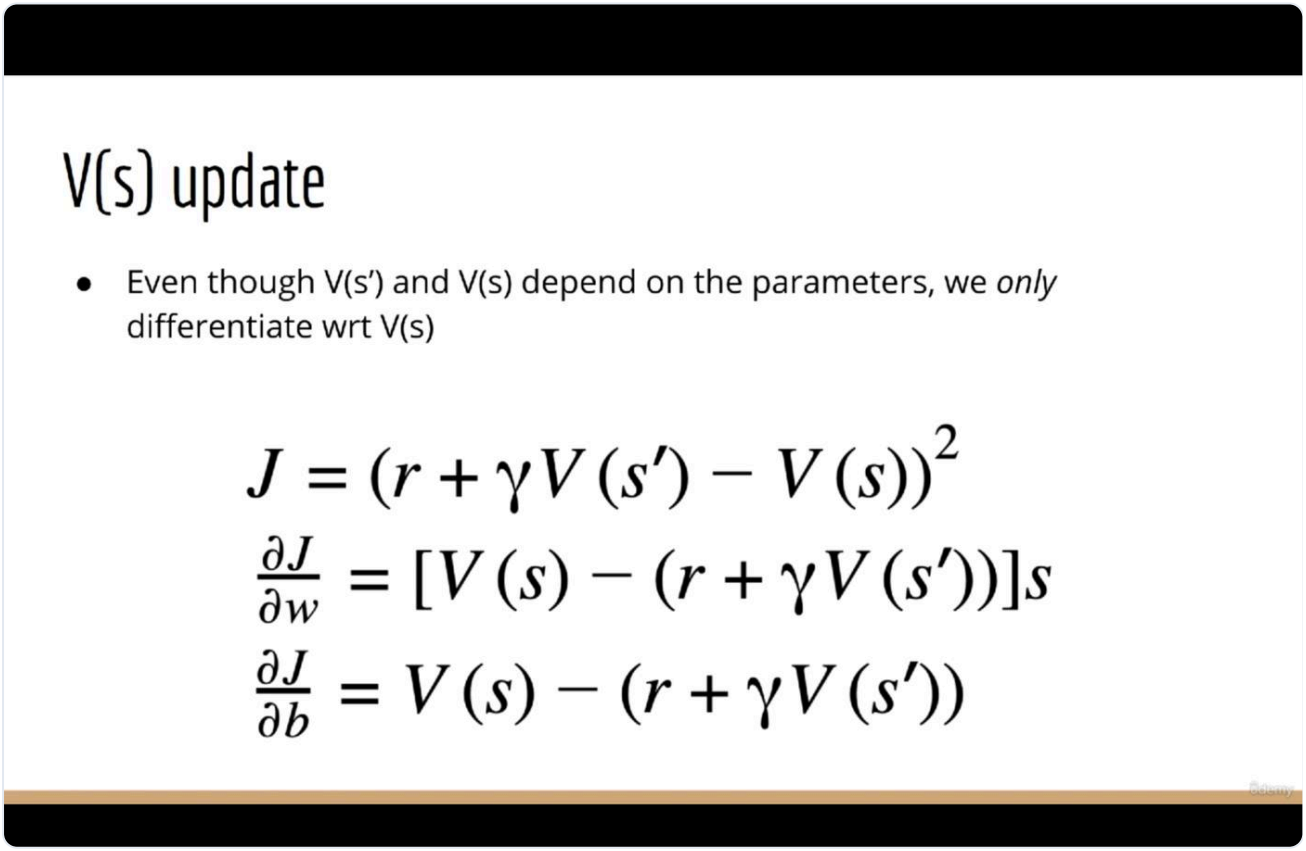

FIGURE 6.11

With respect to the parameters one important thing to realize here is that even though both V of s and V of s prime depend on the model parameters we only differentiate V of s with respect to W and b we pretend that V of s prime is constant. Since it's part of the target and we do not differentiate with respect to the target so let's see what temporal difference learning for the prediction problem would look like in code. Basically this loop is exactly the same as what we had before except for the fact that instead of updating v directly we are updating W and B. So first assume we are given an environment and a policy assume that the policy is a function which accepts as input a state and returns an action next we initialize the weight and bias to be random.

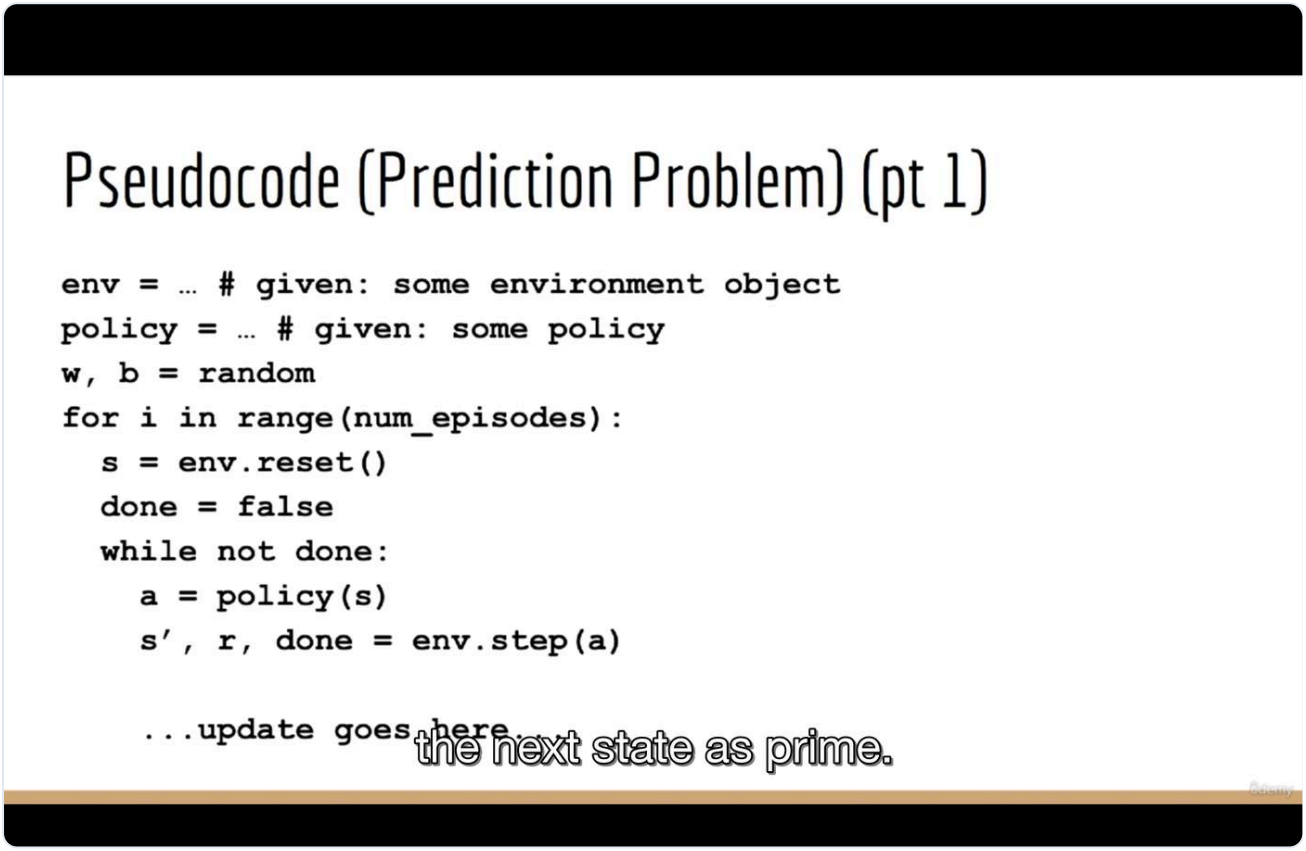

FIGURE 6.12

Next we enter a loop for a predetermined number of episodes inside the loop. We play one episode we call env dot reset to get the first state s which is now a continuous vector. As usual we set the done flag to false and enter a loop which exits when done becomes true inside the loop. We use our given policy function to get the action a we perform this action in the environment to get the next state s prime.

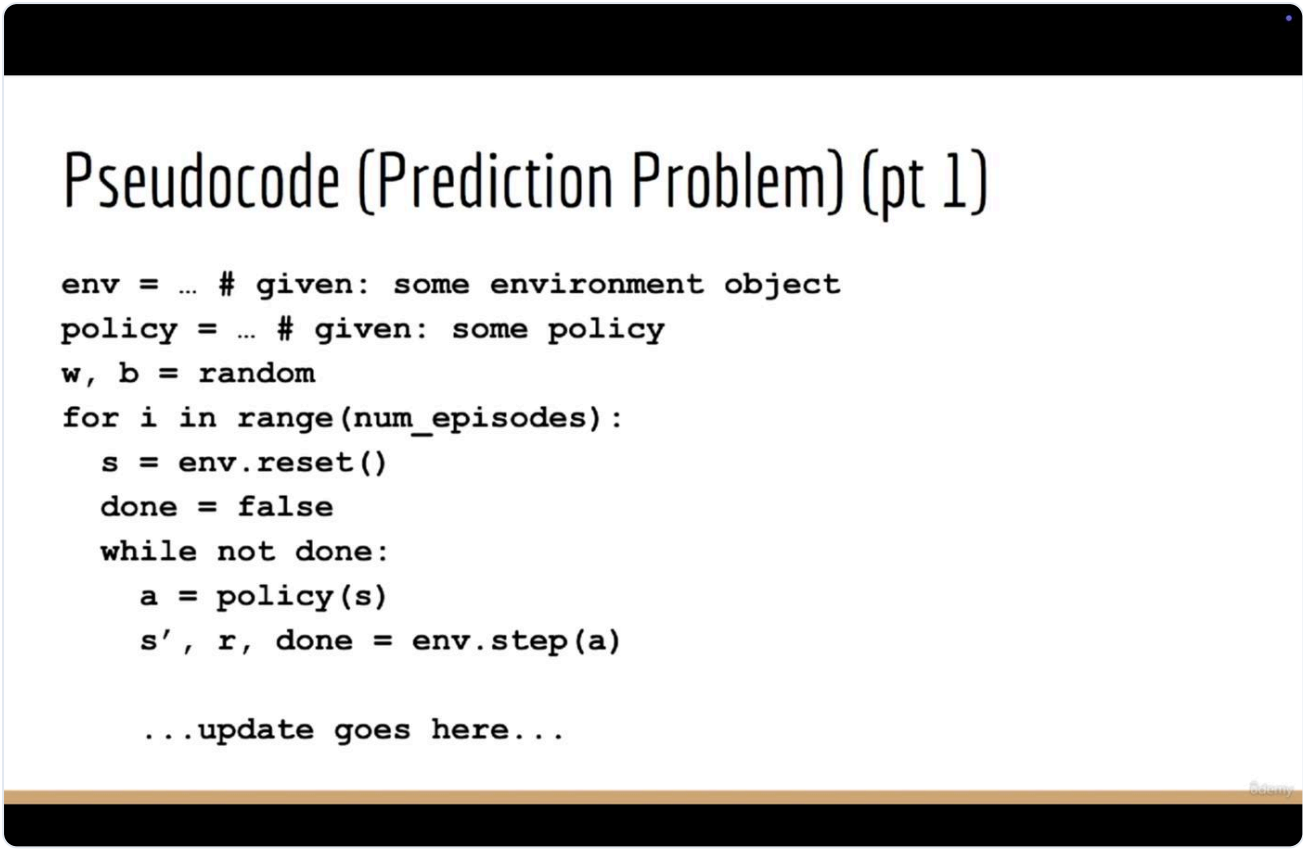

FIGURE 6.13

The reward R and the done flag. Then it's time for our big update in this update. We'll need the values of both V of s and V of s prime which we can calculate using our linear regression model then we run our gradient descent step using the gradients we derived earlier.

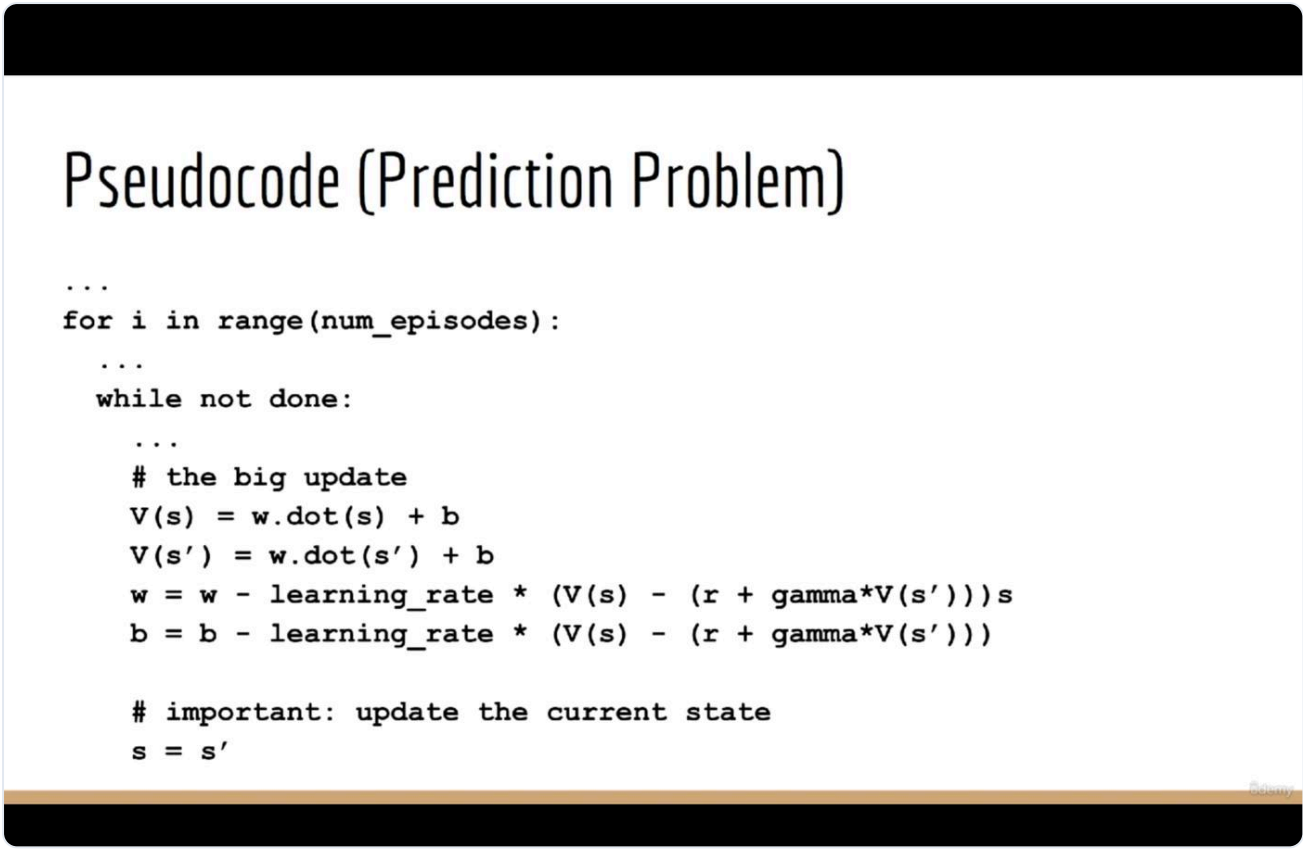

FIGURE 6.14

Lastly we update s to be s prime when the loop is done w and B will have converged as you'll see very shortly approximating V of s is not that useful for the prediction problem. We would be given the policy. But what if we are trying to use the value function to help us choose the action then what is the policy. Well normally it would involve taking the argmax over Q of s and a over all actions but if we have only V of s this is not indexed by the action a.

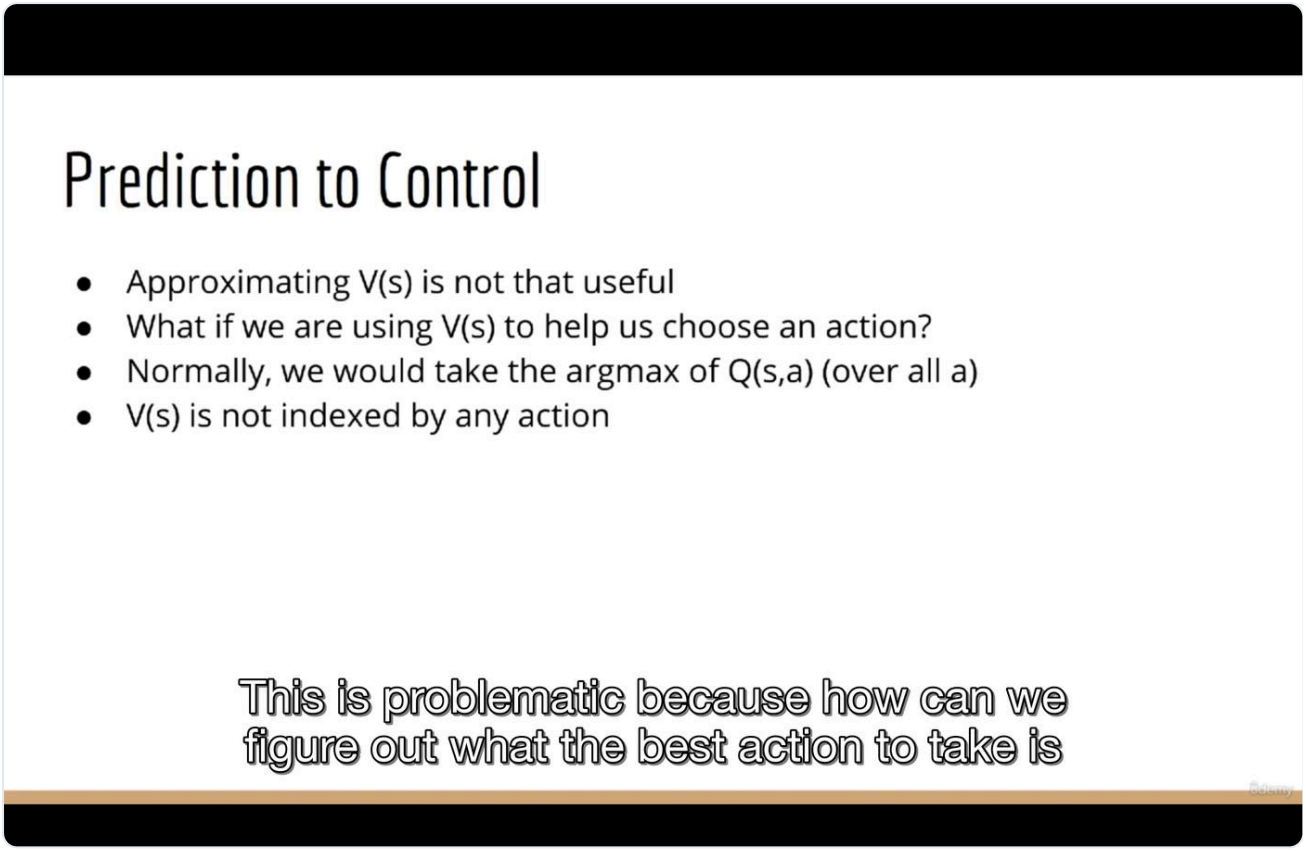

FIGURE 6.15 | Approximating V(s) Is Not That Useful

This is problematic because how can we figure out what the best action to take is here's one way we could do it which is not optimal. Imagine you are in the state s and you want to know what is the best action to perform.

FIGURE 6.16 | Using V(s) for Policy Improvement

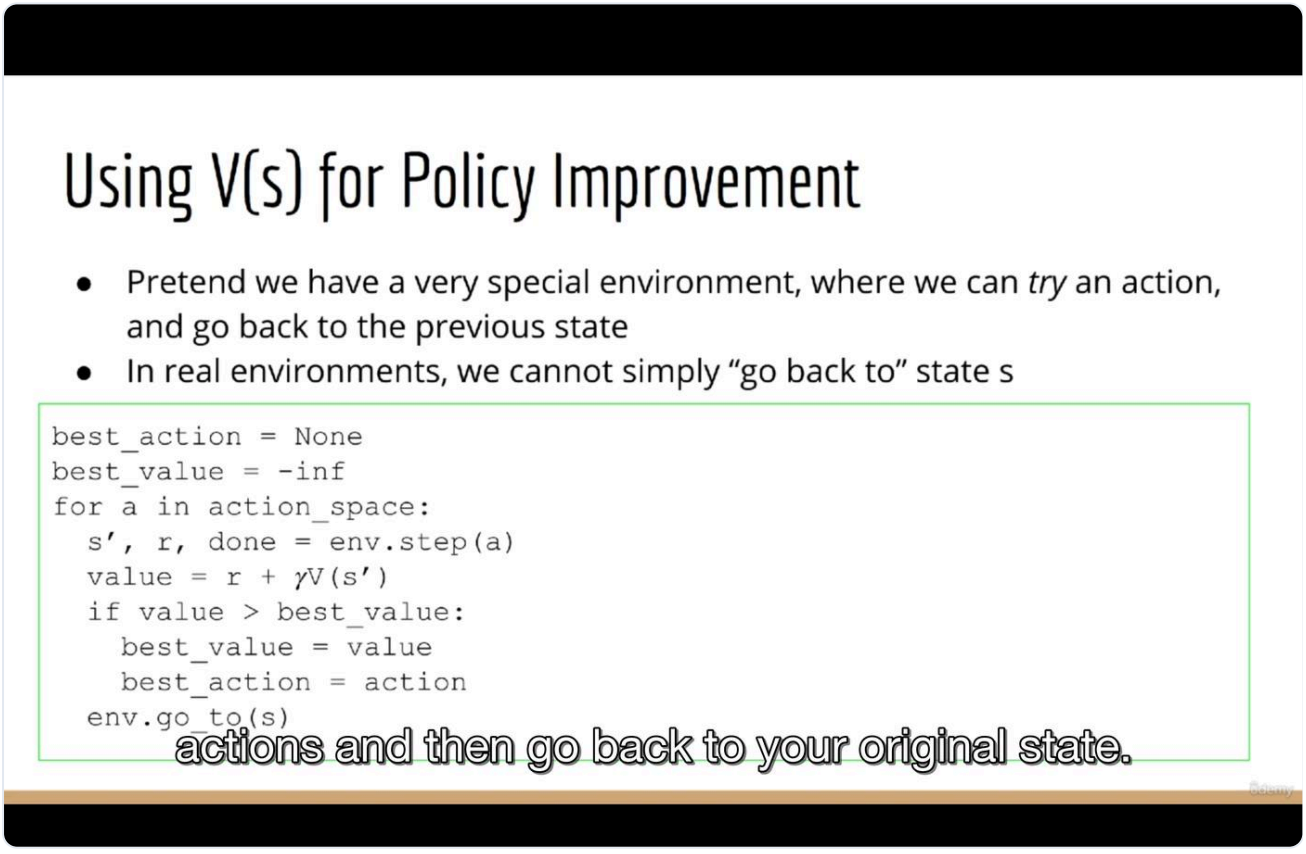

FIGURE 6.17 | Using V(s) for Policy Improvement

Imagine that you are in a very special environment where you can try an action and then go back one step to put yourself back into the state s. In this case what you can do is try every single action so you arrive at all the possible next states as prime and receive the reward R then out of all of those actions you choose the one that gives you the maximum value of r plus gamma times V of s prime. Of course this approach is not at all ideal because in any realistic environment you cannot simply try actions and then go back to your original state. It would certainly be nice if such functionality existed in the real world. So what do we do instead. Of course the answer is to use the action value Q In this case.

What is the target as before if s prime is terminal then the return is just r the reward.

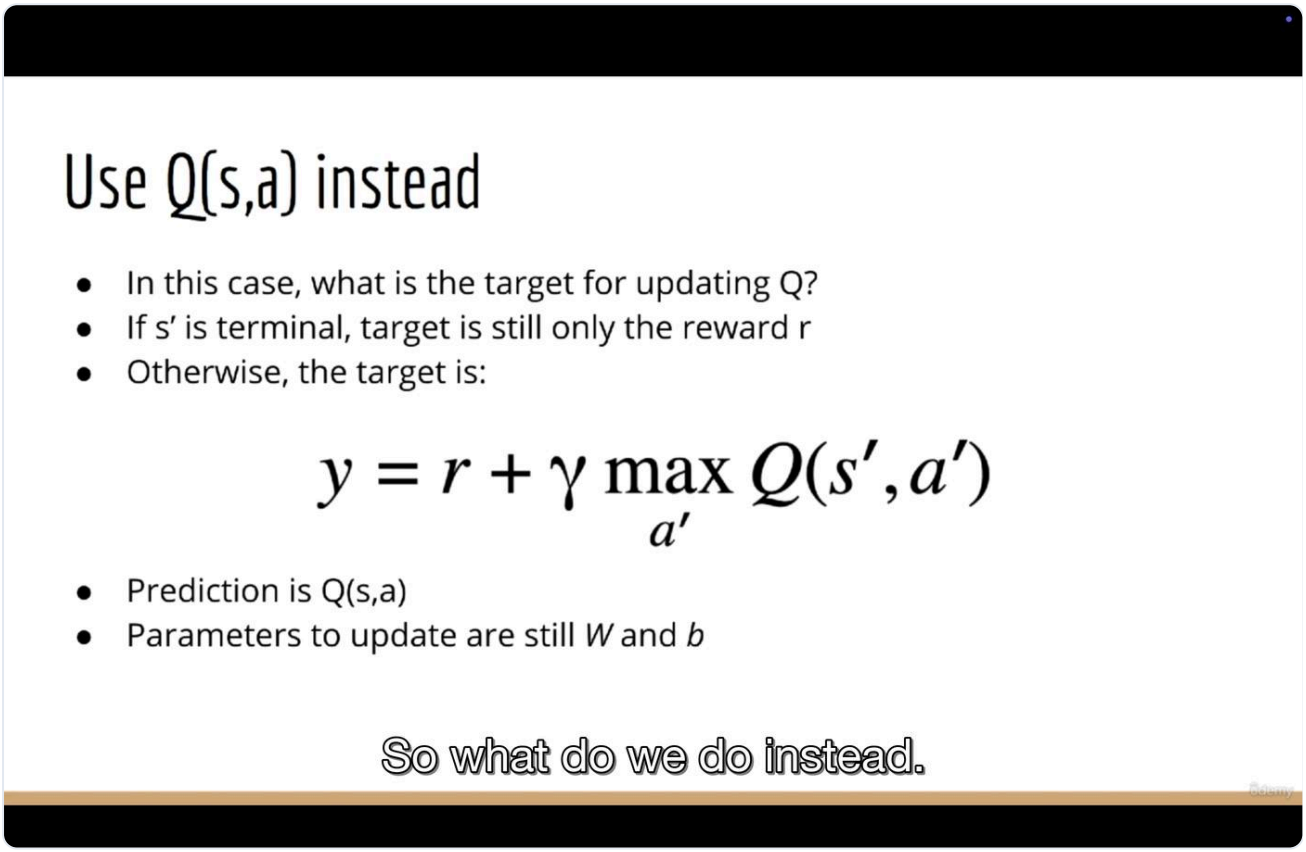

FIGURE 6.18

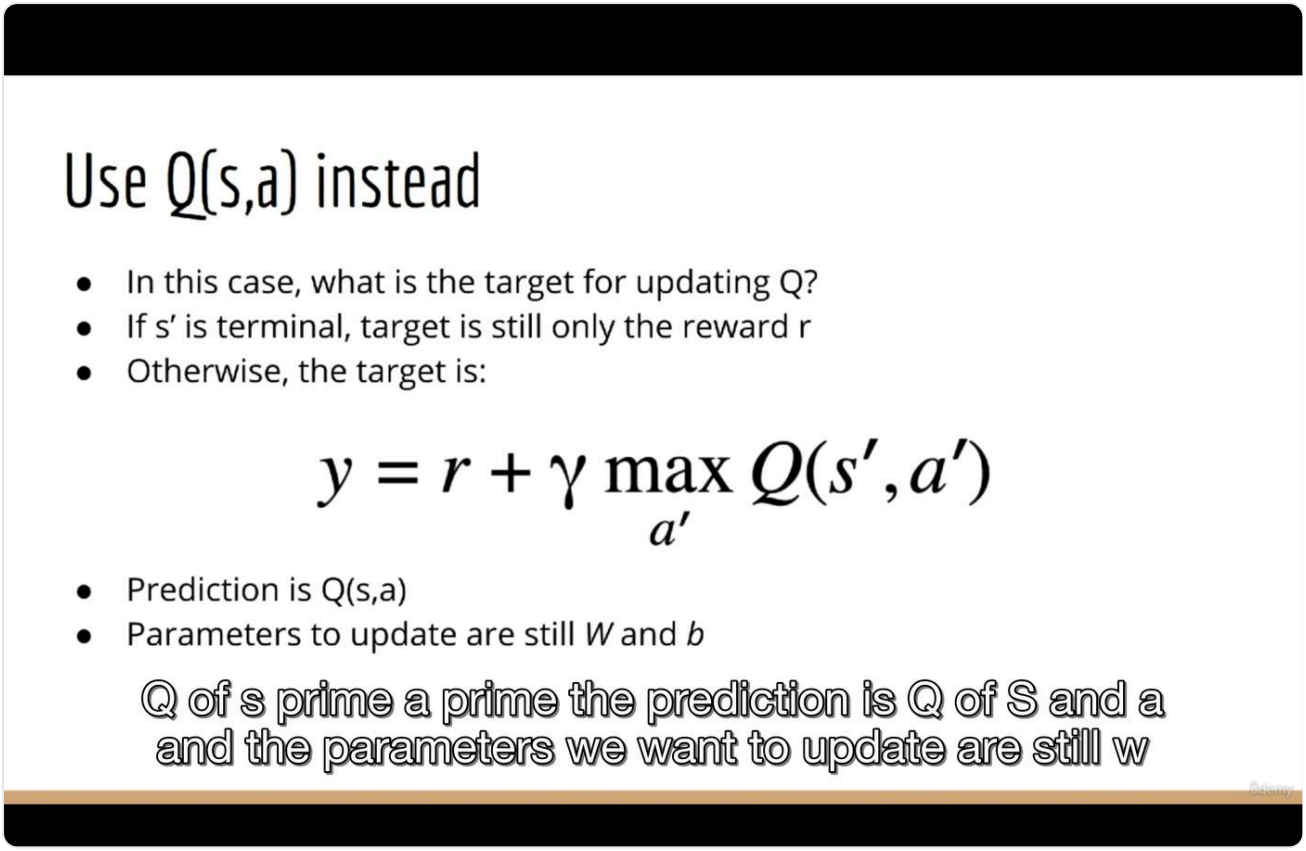

FIGURE 6.19

If s prime is not terminal then the target is R Plus gamma times the max over all actions a prime of Q of s prime a prime the prediction is Q of s and a and the parameters we want to update are still w and b. So here's what the update looks like in code. First the squared error loss is r plus gamma times the maximum over a prime of Q of s prime a prime minus Q of s and a all squared the gradient is Q of S and a minus the target times S. However an important caveat is that this gradient only applies to the action a which was the action actually taken since that's where the prediction came from.

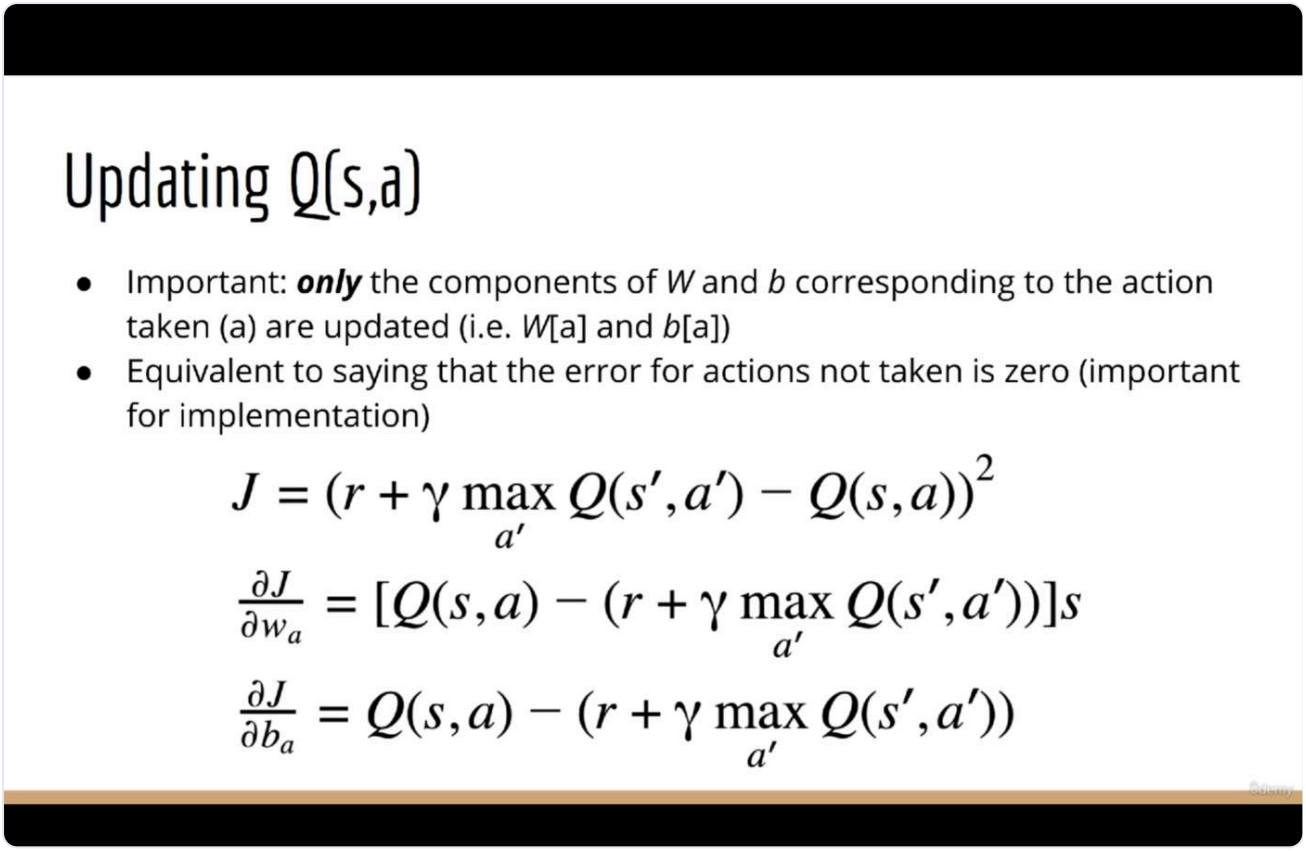

FIGURE 6.20

The weights for the other actions didn't make predictions. So they are not updated. This is of course equivalent to saying that their error is zero which will come into play during the implementation.

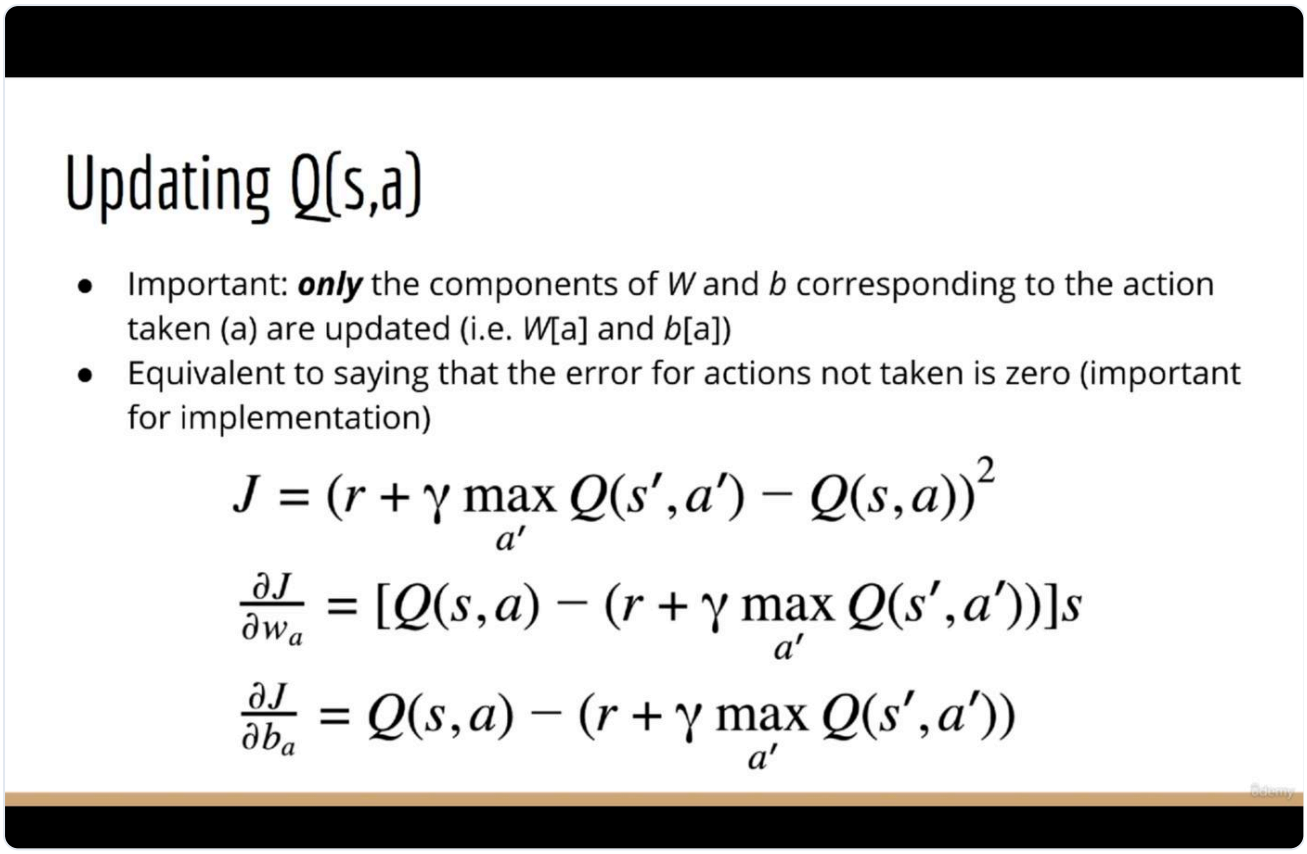

FIGURE 6.21

We do the same thing for the bias term only the bias term which influences the prediction for the action a is updated in other words we index both w and b by the action a finally here's what Q learning looks like when we use approximation methods. Let's start again with what the choose action function would look like. Again we use epsilon greedy for exploration First we generate a random number between 0 and 1. If this number is less than epsilon we perform a random action.

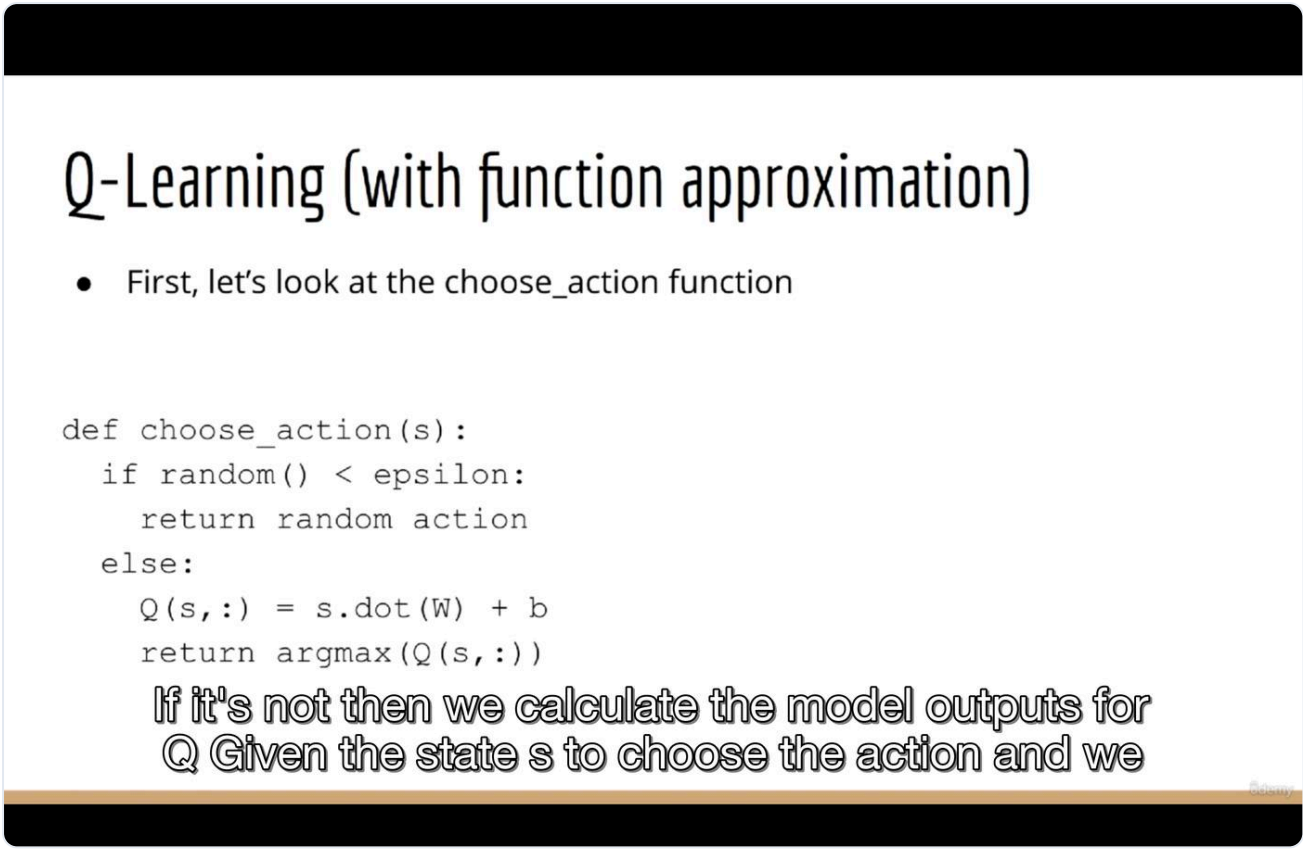

FIGURE 6.22 | Q-Learning (with Function Approximation)

If it's not then we calculate the model outputs for Q Given the state s to choose the action and we take the argmax over Q at the state s. Again this works because the actions are encoded as integers starting from 0 next.

Q-Learning (with function approximation)

- First, let's look at the choose_action function

```
def choose_action(s):
  if random() < epsilon:
    return random action
  else:
    Q(s,:) = s.dot(W) + b
    return argmax(Q(s,:))
```

FIGURE 6.23 | Q-Learning (with Function Approximation)

Q-Learning Pseudocode

```
env = … # given: some environment object
W, b = random
for i in range(num_episodes):
  s = env.reset()
  done = false
  while not done:
    a = choose_action(s)
    s', r, done = env.step(a)

    # update Q
    y = r + gamma * max_a'{ Q(s', a') }
    W[a] = W[a] - learning_rate * (y - Q(s,a))s
    b[a] = b[a] - learning_rate * (y - Q(s,a))

    s = s' # important: update the current state
```

FIGURE 6.24

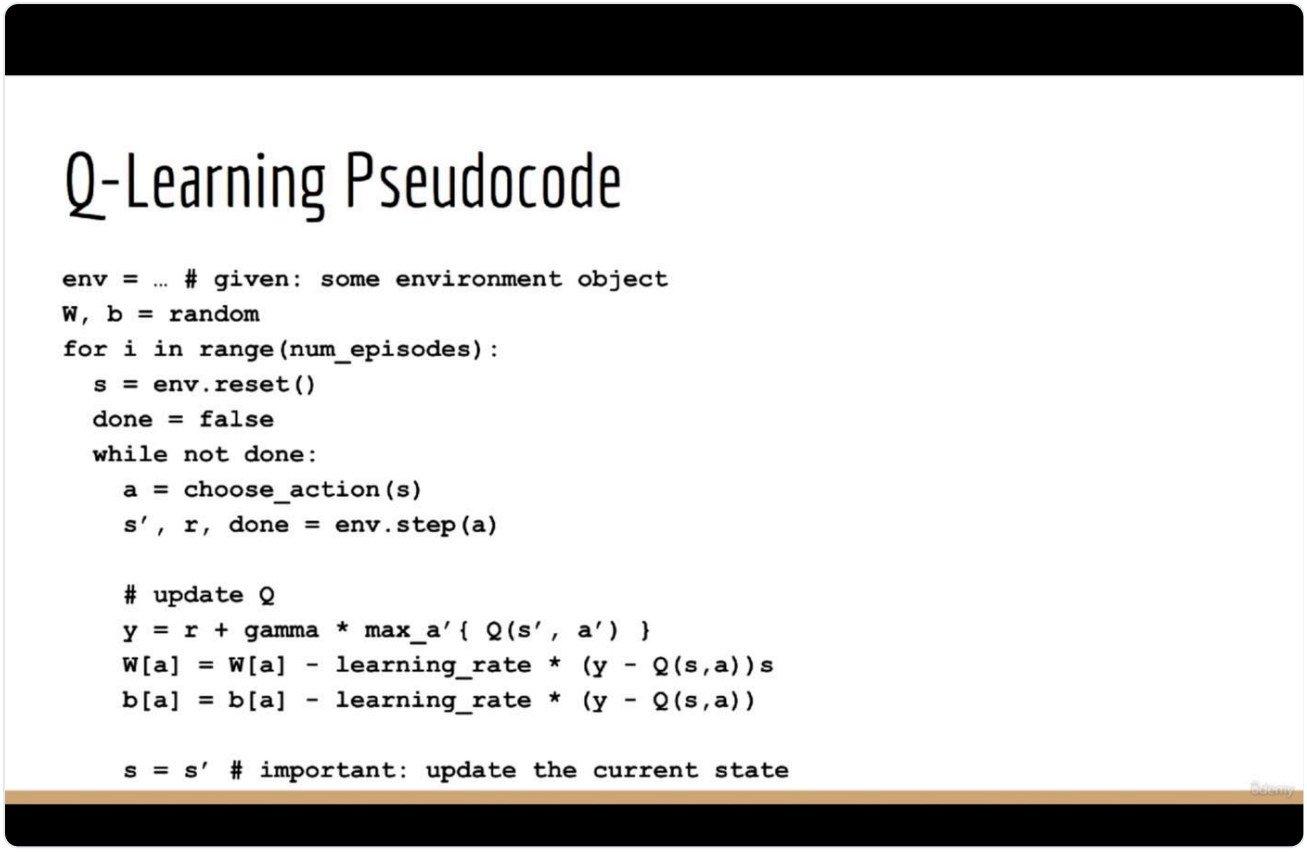

FIGURE 6.25

Let's look at the Q learning loop as before we start by initializing W and b to be random. Then we enter a loop and play one episode per iteration we reset the environment and set the done flag to false inside the second loop. We choose an action using the function we defined earlier. Then we take a step in the environment to arrive in the next state and receive our reward R. Next is our big update. It helps the first calculate the target since it looks pretty complicated so we'll call that y it's equal to r plus gamma times the max over all actions a prime of Q of s prime a prime. And remember Q itself is a model prediction. So anytime you see Q here that means we're calling model dot predict next.

We run our gradient descent updates for w and b but importantly we only update the components of W and B which correspond to the chosen action a lastly we update the state s to become s prime.

LECTURE 100

Training Neural Networks Is Inherently Unstable

In this lecture we are going to continue our discussion of deep Q learning and approximation methods. You'll notice that in the last lecture we didn't invoke the use of deep neural networks at all. Well first it hopes to do something simple.

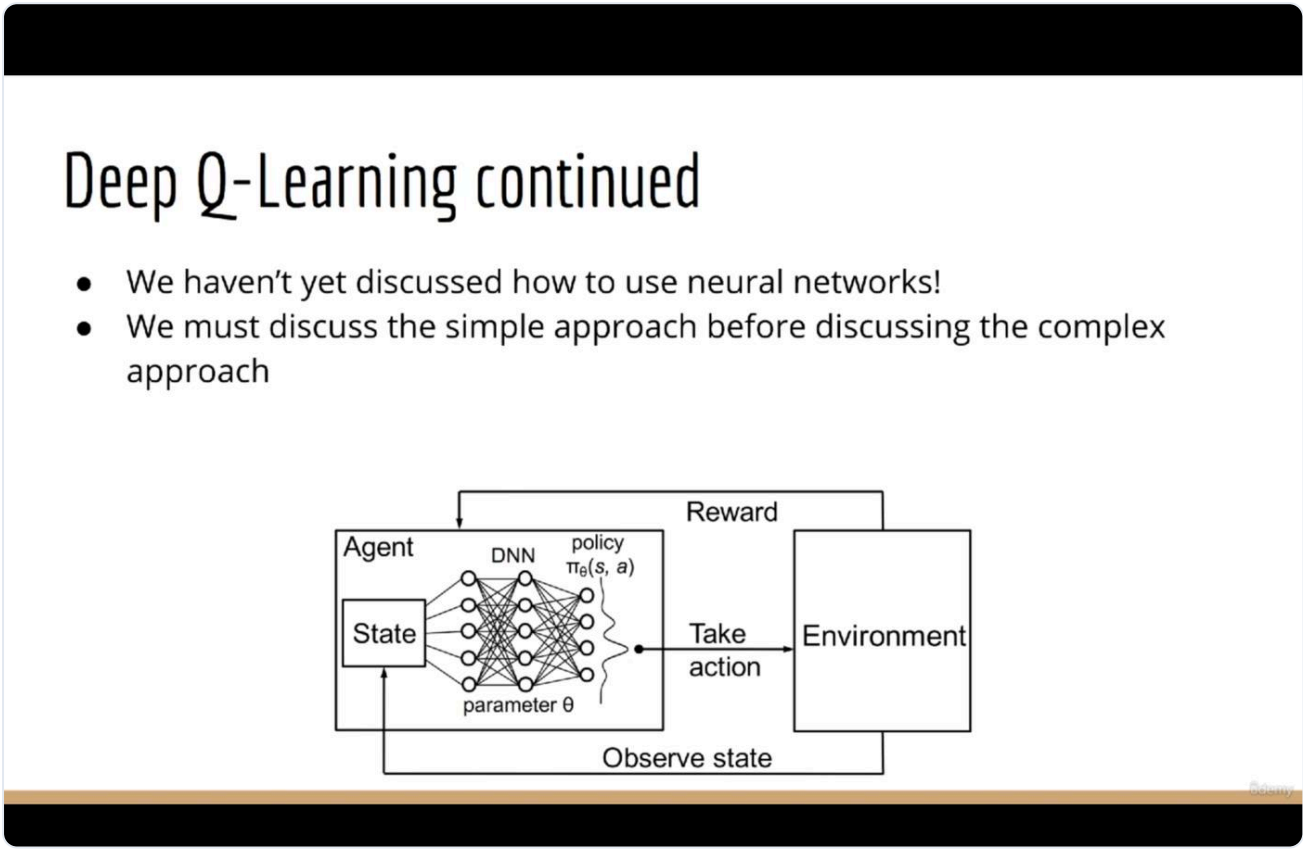

FIGURE 7.1

Before we do something complicated lay down the foundation before you start adding all the bells and whistles. Now you might be wondering is it not trivial to use neural networks for Q learning. Is it not the case that all we have to do is replace the linear model with a neural network regression model.

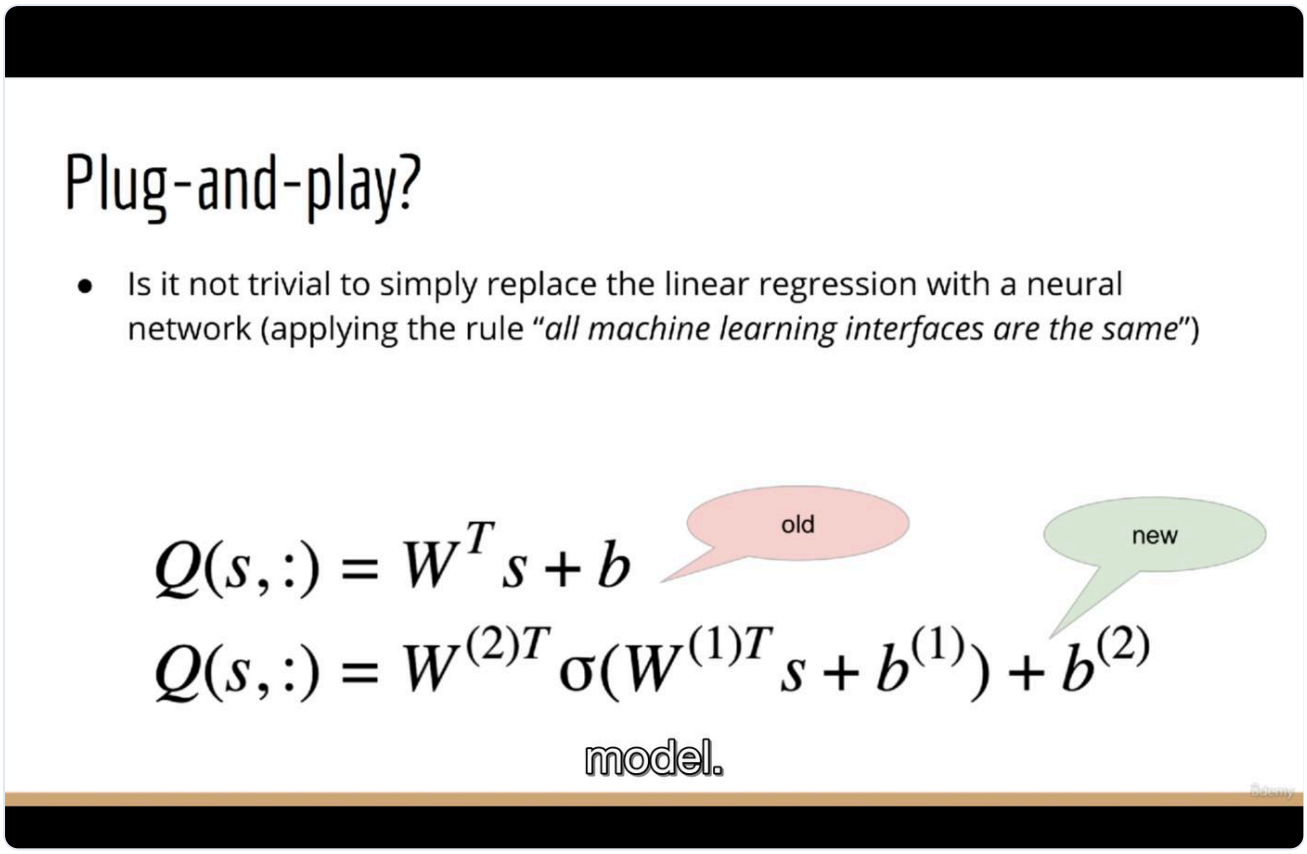

FIGURE 7.2

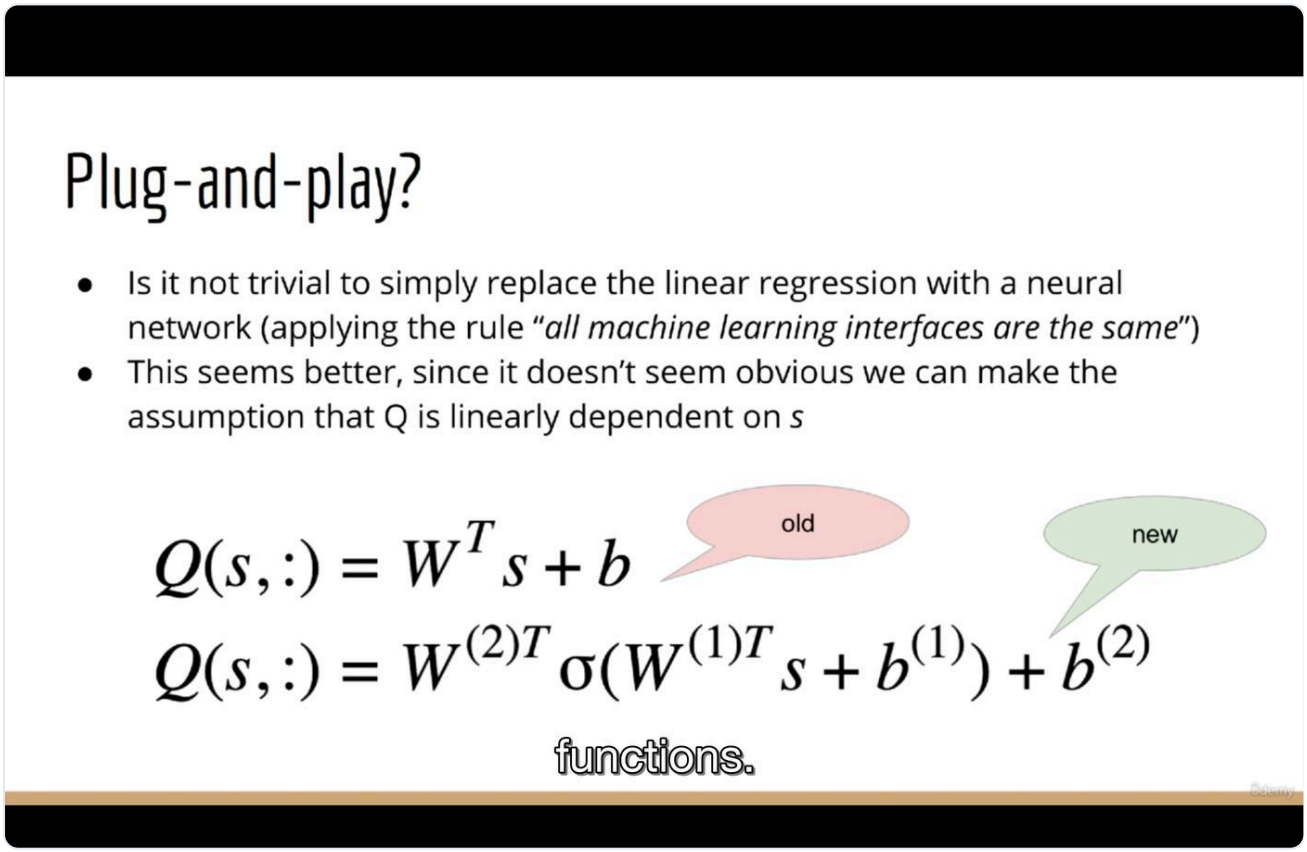

FIGURE 7.3

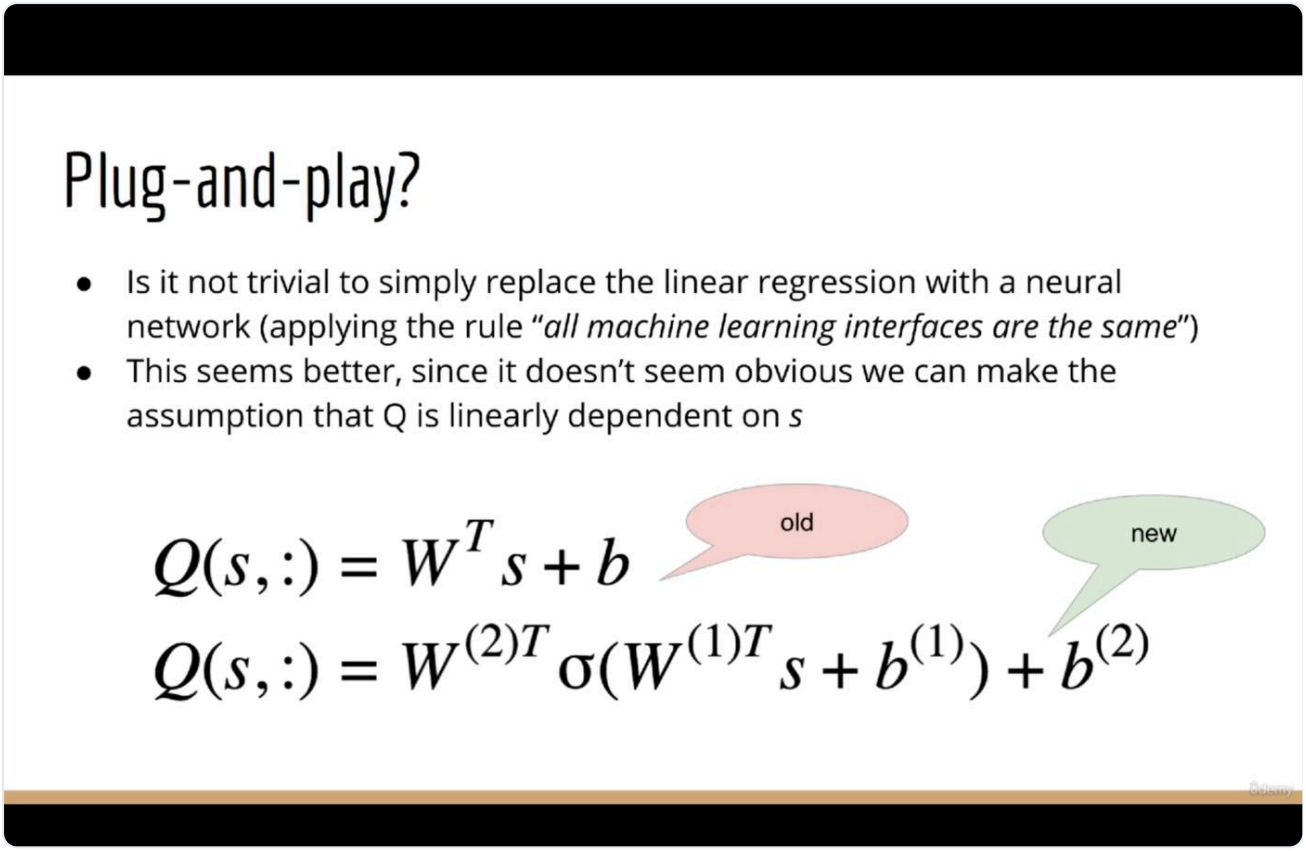

FIGURE 7.4

This is exactly the rule all machine learning interfaces are the same. So instead of W transpose x plus B we could have a two layer neural network. As I've written down here this seems perfectly reasonable. In fact it seems like it would be better than a linear model a linear model can only approximate linear functions. How do we know that Q is a linear function of the state s. In fact it's not clear at all that we can make this assumption. It doesn't even seem like a good assumption. Neural networks are more flexible and they can handle all kinds of inputs such as images text time series and so forth. So here's what it would look like. Conceptually our loss is still the same.

It's just the squared error.

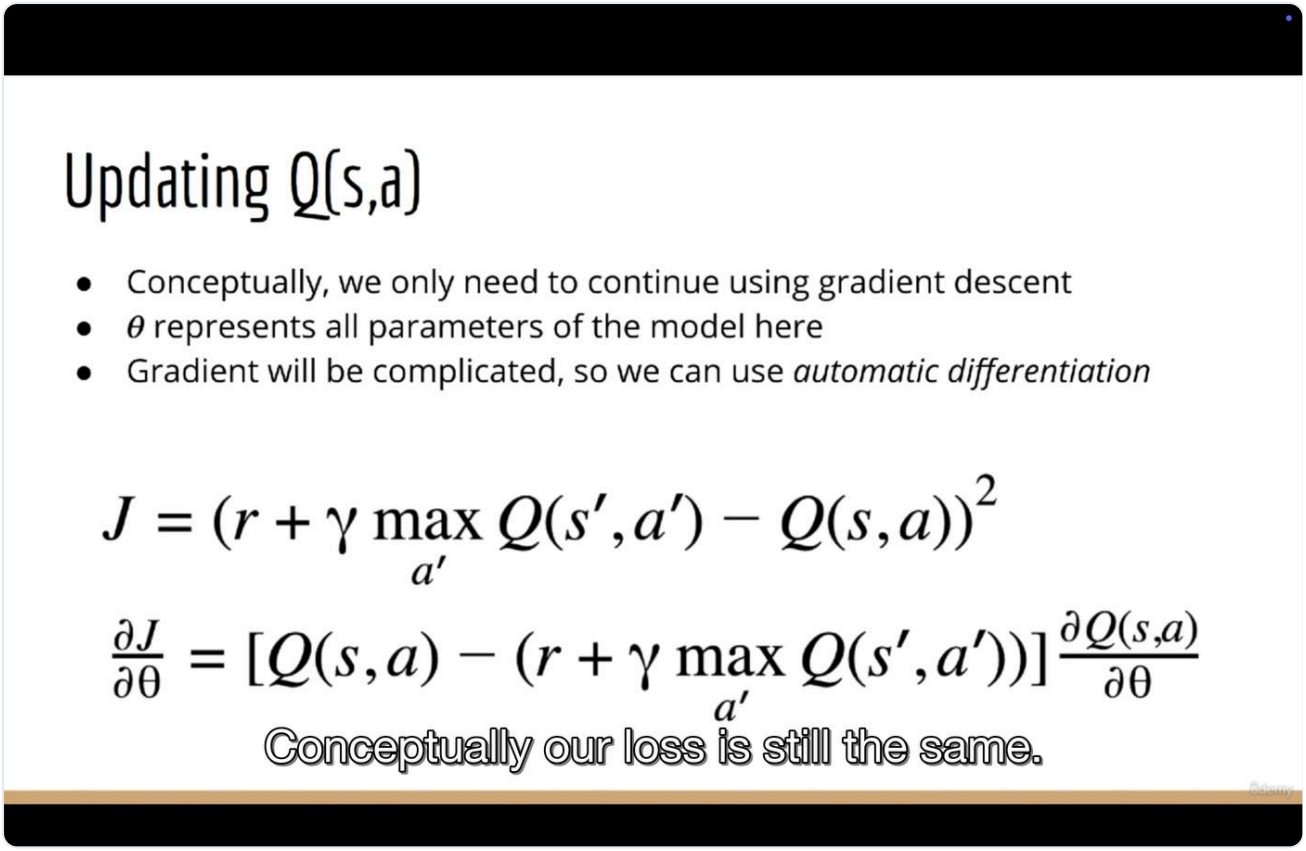

FIGURE 7.5

Any parameters of the model would be updated using gradient descent. The only difference now is that the gradient will be different since in this case the gradients are more complicated. We don't want to derive them ourselves. Instead we would like to make use of automatic differentiation which is included in all modern Deep Learning libraries. In this case I've written the gradient in terms of a generic parameter theta but you can imagine that it applies to all the weights and biases of whatever neural network we decide to use. The problem is training neural networks is inherently unstable. You probably haven't noticed this yourself.

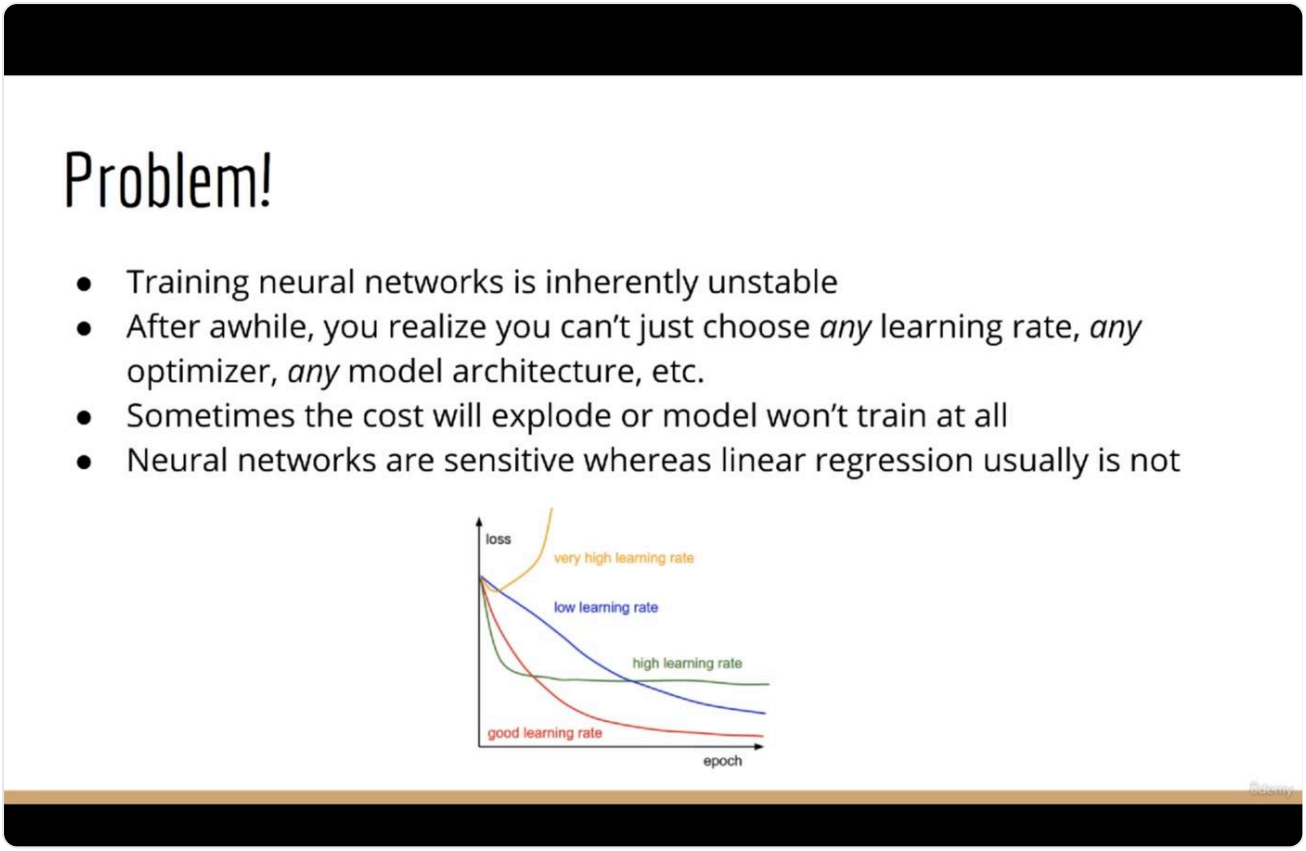

FIGURE 7.6 | Training Neural Networks Is Inherently Unstable

If you only just started studying deep learning but after a while you'll realize you can't just choose any learning rate and any optimizer in any model architecture. Sometimes your cost will explode. Sometimes your model will not train at all. Neural networks are pretty sensitive to these choices whereas linear regression usually isn't. Now if you're a beginner and you're just taking a course on deep learning then the instructor has figured out all this stuff for you so you don't really recognize the hard work that goes into this on top of this we have another problem. Temporal Difference learning itself is also inherently unstable. Technically we are doing gradient descent but remember that the target is not a real target.

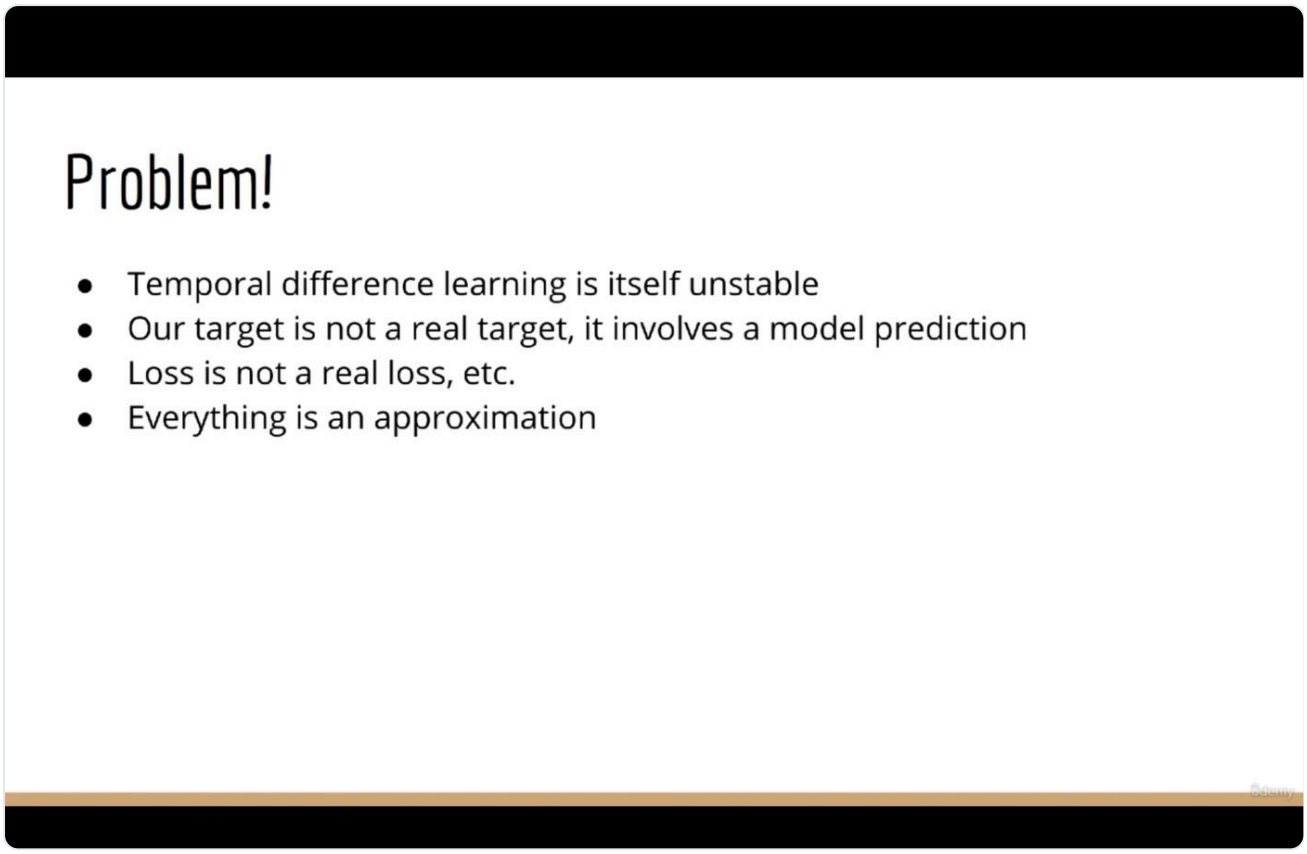

FIGURE 7.7 | Temporal Difference Learning Is Itself Unstable

Therefore our loss is not a real loss and our gradient is not a real gradient of a real loss with a real target. It's approximations all the way through what happens when you put together these two inherently unstable algorithms. Well you get something that barely works if it works at all.

Too much instability

- What happens when you combine the instability of neural networks with the instability of TD learning?
- It barely works!
- That's not to say there won't be instances where you *can* just plug in a neural network and make it work
- But it won't be as easy as linear regression

FIGURE 7.8

Deep Q-Learning

- There have been a number of approaches to Deep Q-Learning (DQN) that have been developed over the years - I will introduce you to **one**
- The experience replay buffer / experience replay memory
- Previously, we used *stochastic* gradient descent
  - One sample at a time
- *Batch* gradient descent is more stable
  - Multiple samples at a time

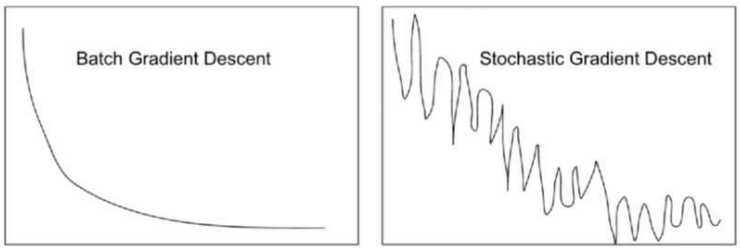

FIGURE 7.9

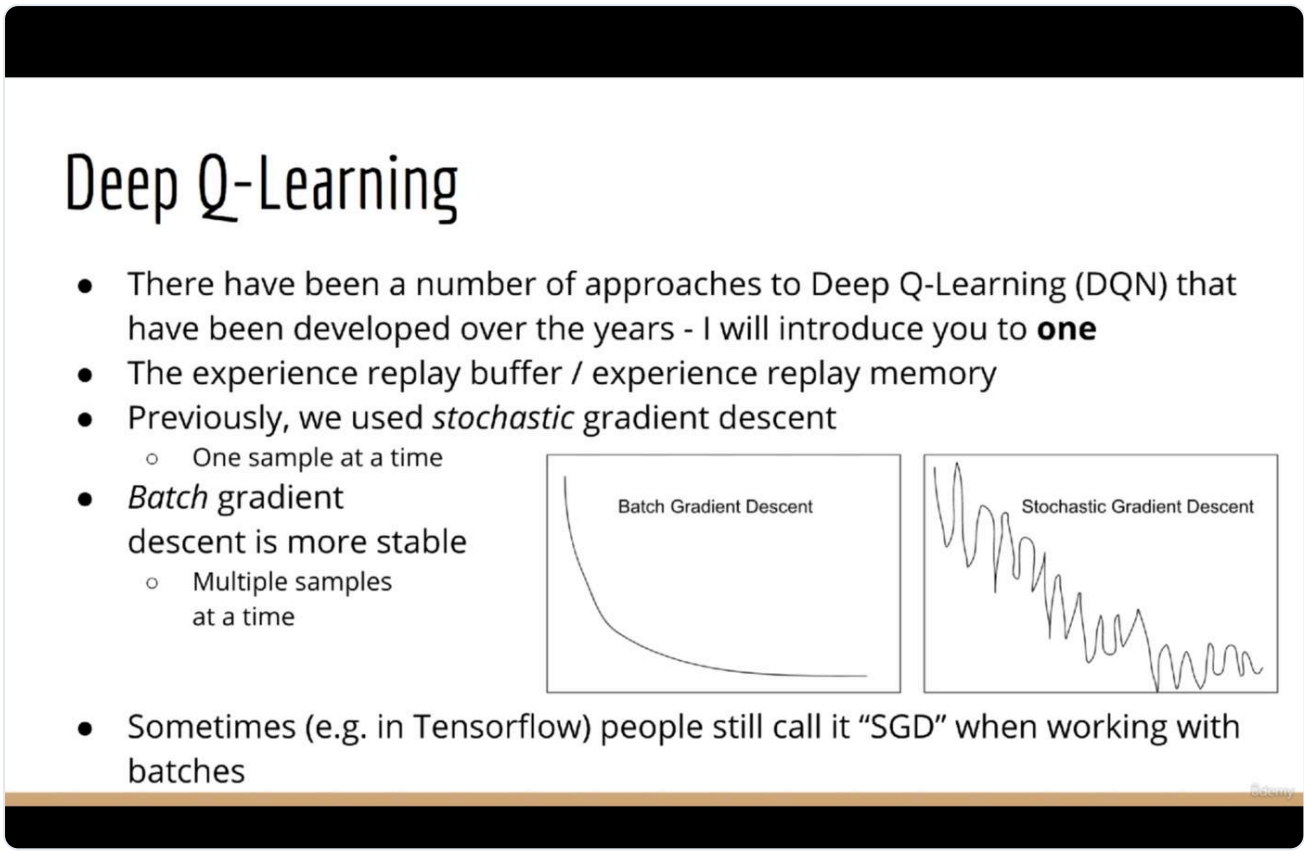

FIGURE 7.10 | The Experience Replay Buffer / Experience Replay Memory

Now that's not to say there won't be instances where you can simply plug in a neural network into your Q learning script and make it work but generally speaking this is not going to work as nicely as say linear regression for a normal supervised learning dataset so there are actually a number of approaches called deep Q learning that have been developed over the years. But in this lecture my goal is to introduce you to one of these approaches. The idea is we're going to use something called an experience replay buffer or an experience replay memory. You will notice that in our previous updates we were doing stochastic gradient descent.

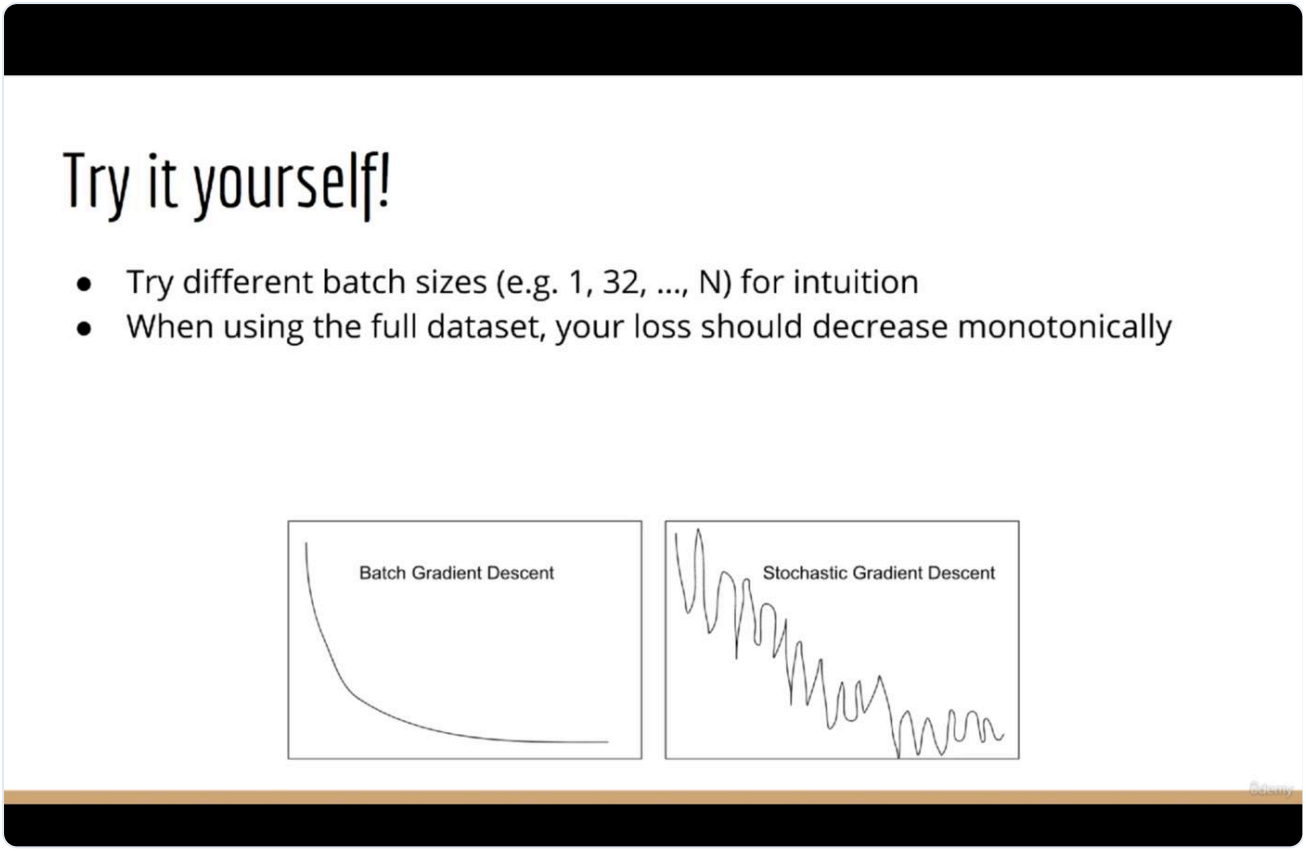

FIGURE 7.11

In other words looking at one sample at a time what we want to do in order to make this process a little more stable is Batch gradient descent which looks at multiple samples at a time. In this way we hope to stabilize the gradient as a side note in libraries like TensorFlow it's called stochastic gradient descent or SGD. Even when you use batches however I like to make the distinction that batch gradient descent refers to looking at multiple samples whereas stochastic gradient descent refers to looking at one sample you're encouraged to try this yourself to gain some intuition grab any supervised learning dataset and compare the learning curve you get when you look at one sample at a time versus when you look at say thirty two samples at a time.

You can also compare an entire range of samples from one up to the size of the dataset.

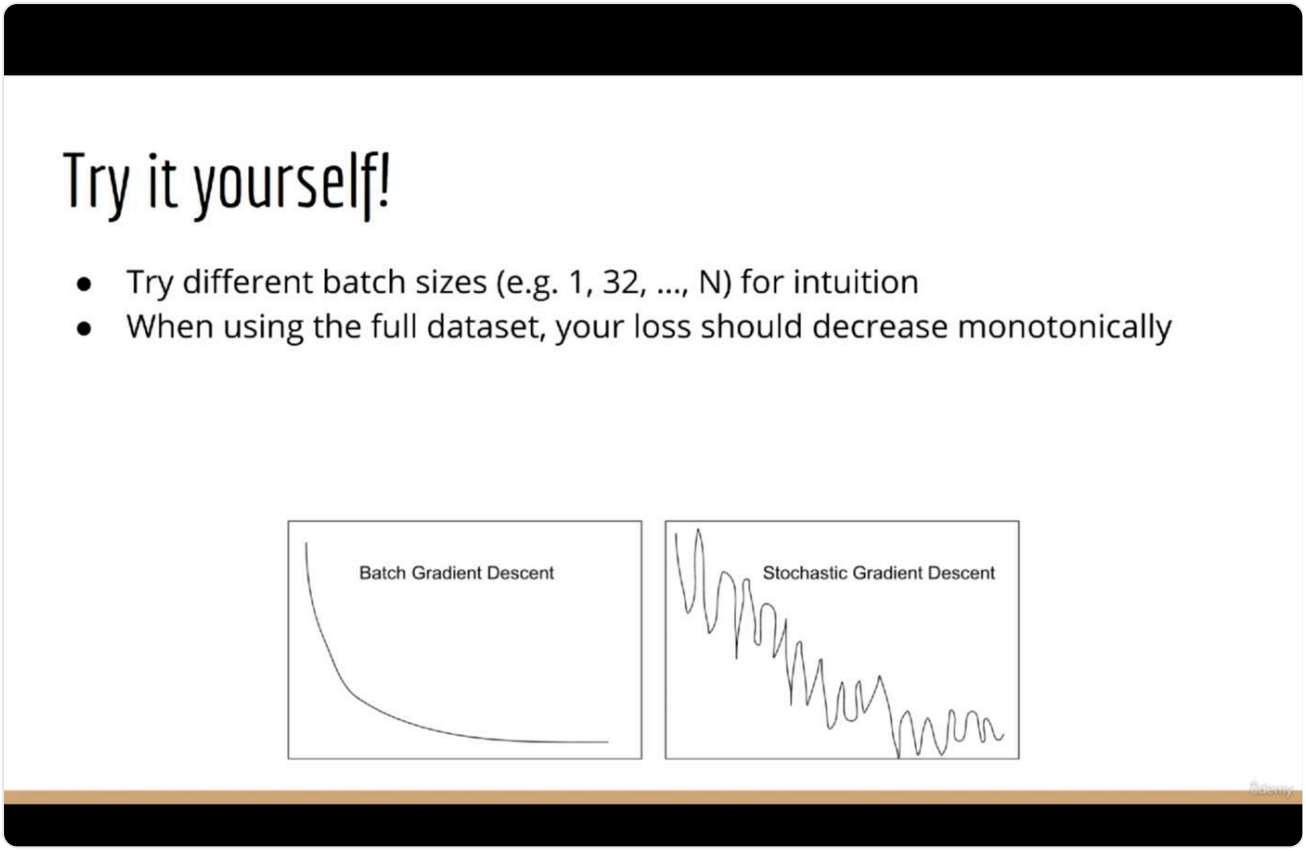

FIGURE 7.12

You should find that the loss per iteration is much more stable for batch gradient descent than stochastic gradient descent. The other extreme is that you can use the full data set on each round of gradient descent in which case your loss function should decrease monotonically another point is that it's not good to have each sample be very correlated to the previous samples when you are doing gradient descent. This is why we like to shuffle the data on each epoch when we are doing gradient descent and reinforcement learning. The data is very correlated because you're going from one state to the next in a Markov process so you can almost even think of it as like a time series.

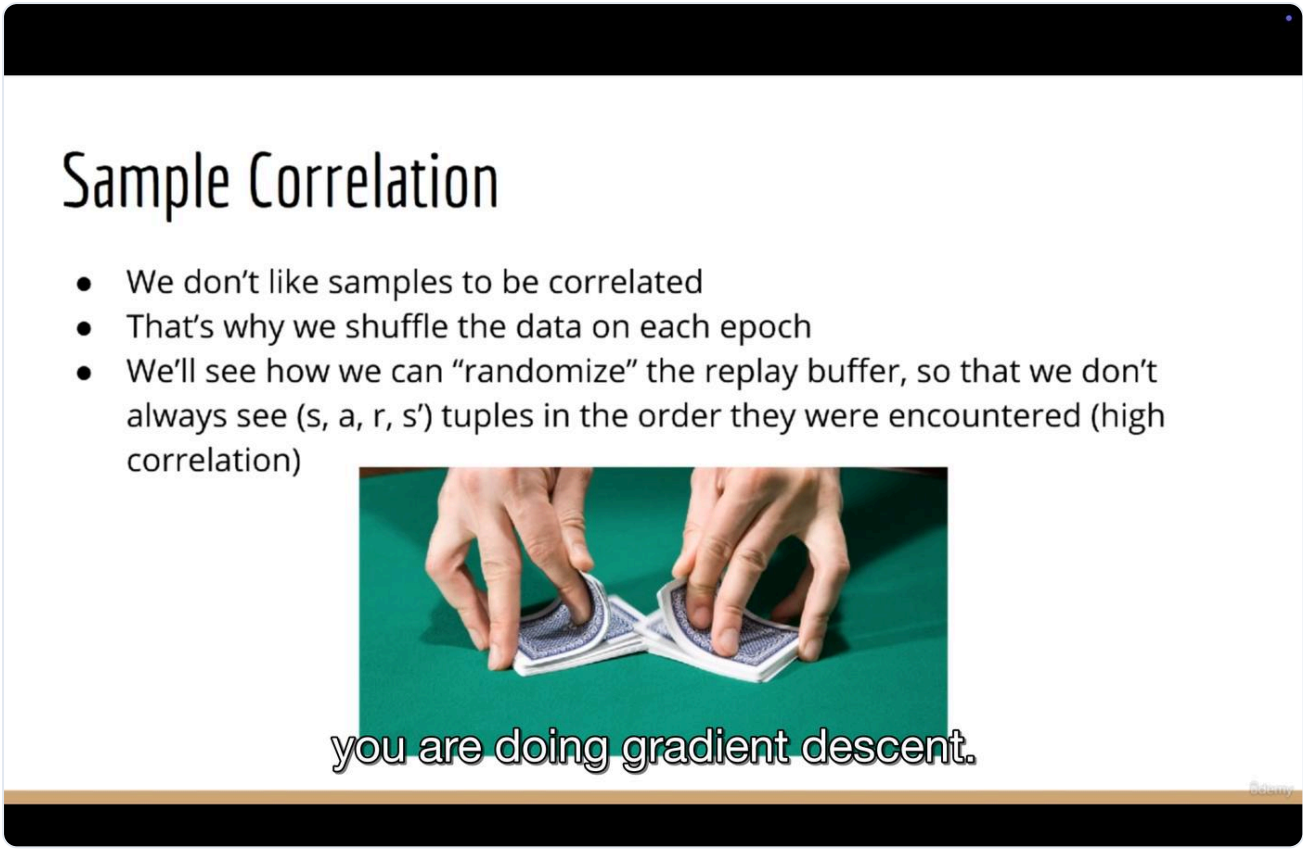

FIGURE 7.13

Every sample is correlated with the previous sample we'll see how we can randomize the data from the replay buffer later on next. Let's look at how a replay buffer would work. Conceptually you can imagine it like a python list that stores tuples of the form s a r s prime and done as we encounter them. We can call these tuples transitions on this slide.

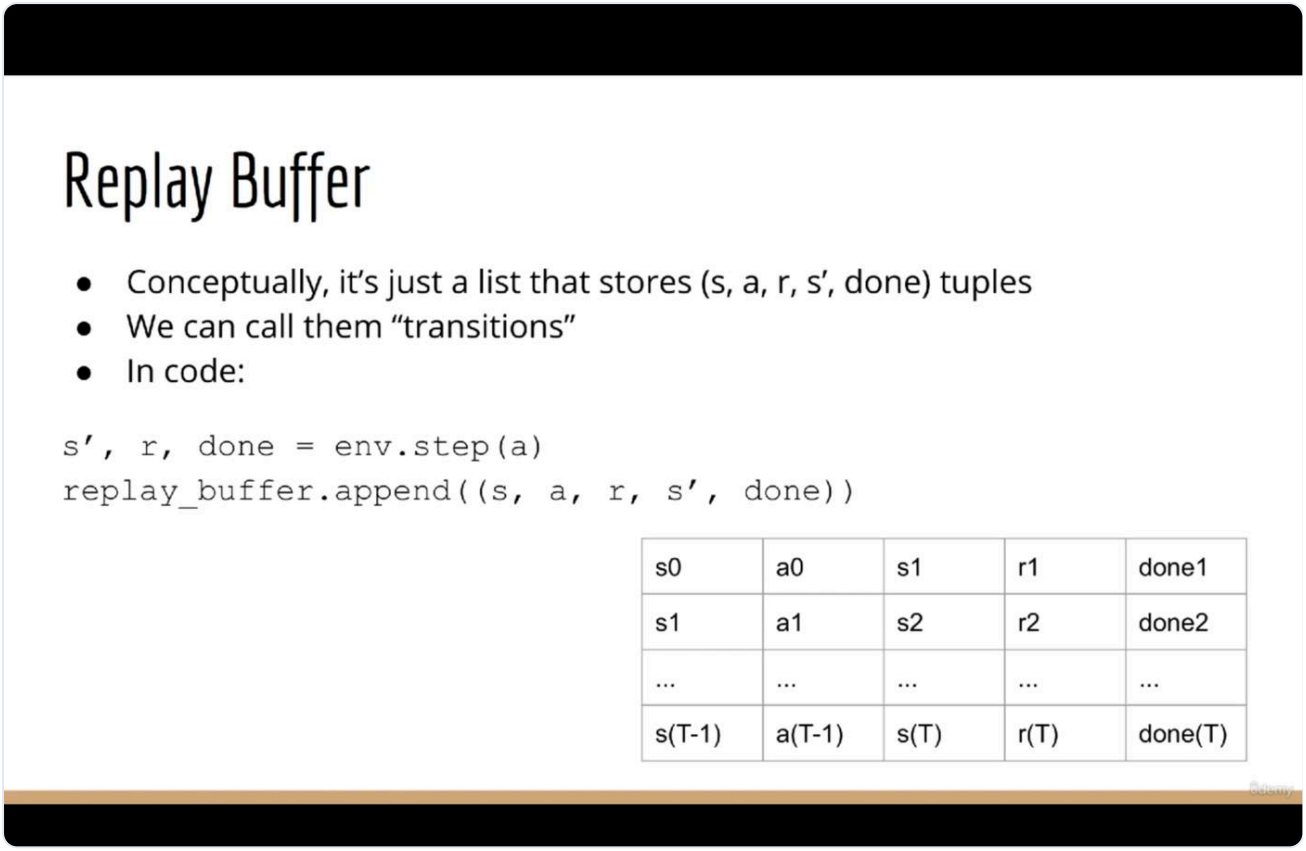

FIGURE 7.14 | Replay Buffer

I visualize what such a list might look like in memory and how we might update it in code. Basically it's just a one line addition in our loop every time we take a step in the environment. We just add the latest SARS prime done to our replay list one important characteristic of the replay buffer is that it has a fixed size at some point.

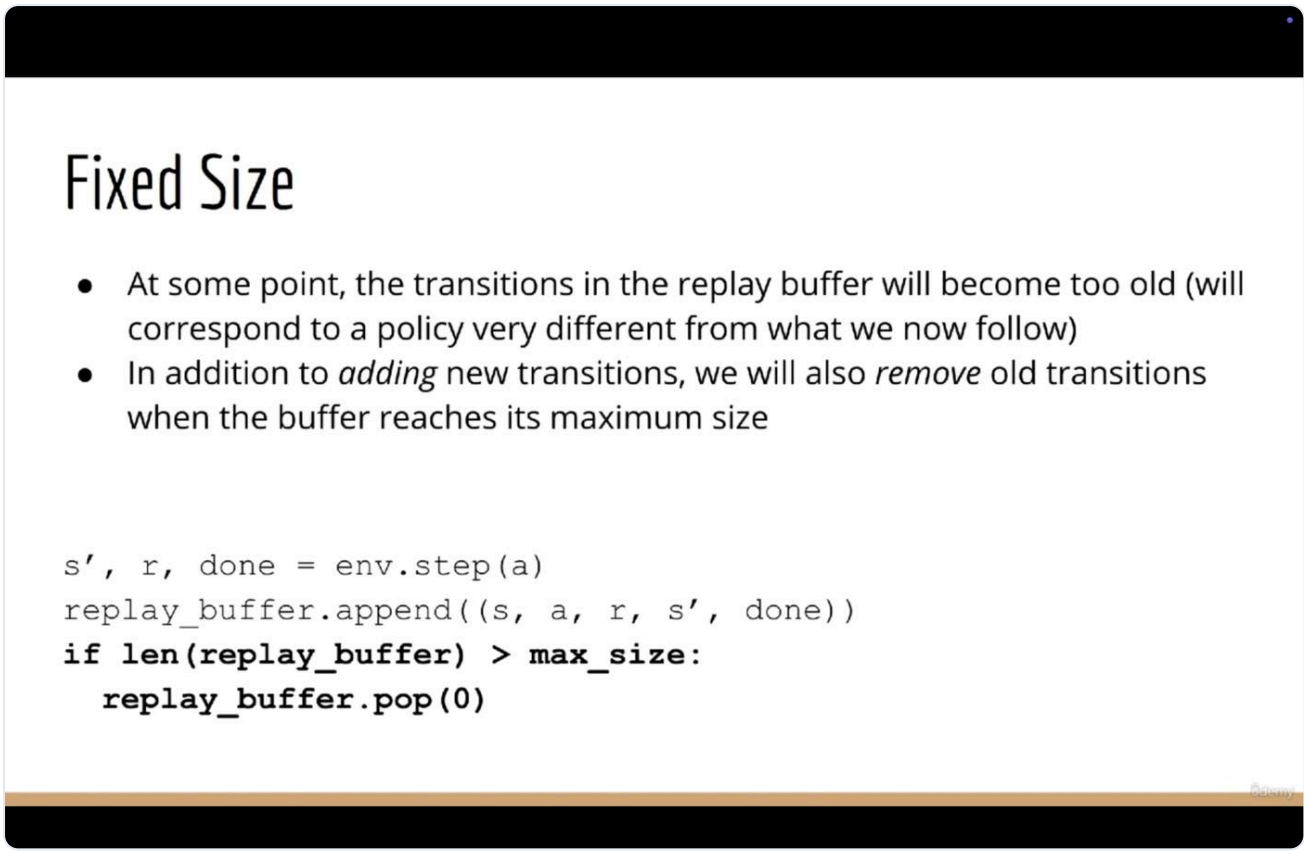

FIGURE 7.15 | Fixed Size

Some of the transitions in the replay buffer will be old and they will correspond to a policy that we are no longer following. In this case we may not want to use these to form batches of data when we perform gradient descent. So here's a new addition we want to add. Instead of only appending new transitions to the replay buffer we are also going to remove transitions when the replay buffer reaches its maximum size. Since this is a list where we are always appending to the end the transition we want to remove will always be located at the beginning. The oldest transition the last thing we have to consider is what happens when we actually want to perform the update.

In this slide I will describe conceptually what we want to do although in code we will make this a little more efficient. So let's suppose we have some function called train in this function. We first to sample a batch of transitions from our replay buffer. This is now a list of tuples where each tuple Contains SARS prime done transitions. Next we initialize our inputs and targets as empty lists. Then we loop through each transition if done is true. That means as prime is terminal. So the return is just R. Otherwise the target is our plus gamma times the max over Q of s prime a prime after we've created the target we can append that to our targets list then we add the input the state s to our inputs list and then once we have a batch of inputs and targets.

This is nothing more than gradient descent. So all we need to do is the usual gradient descent update parameter minus learning rate times gradient. So here's what the pseudocode for deep Q learning would look like. It's very similar to Q learning with approximation methods except now we are going to make use of our replay buffer. In fact this code is a little simpler because we al-

ready describe the functions to update the replay buffer and train the model. So to start we initialize our replay buffer to an empty list. We also instantiate a model which of course comes with weights which are randomly initialized. You can obviously picture it as a neural network.

Next we enter a loop that runs for a predetermined number of episodes inside the loop. We start a new episode by calling env.reset and setting done to false. Then we loop through each episode first we choose an action using Epsilon greedy. Next we take that action in the environment to get s prime R and done. Then we add the latest transition s a r s prime done to our replay buffer this function also removes the oldest transition if the buffer has reached its maximum size.

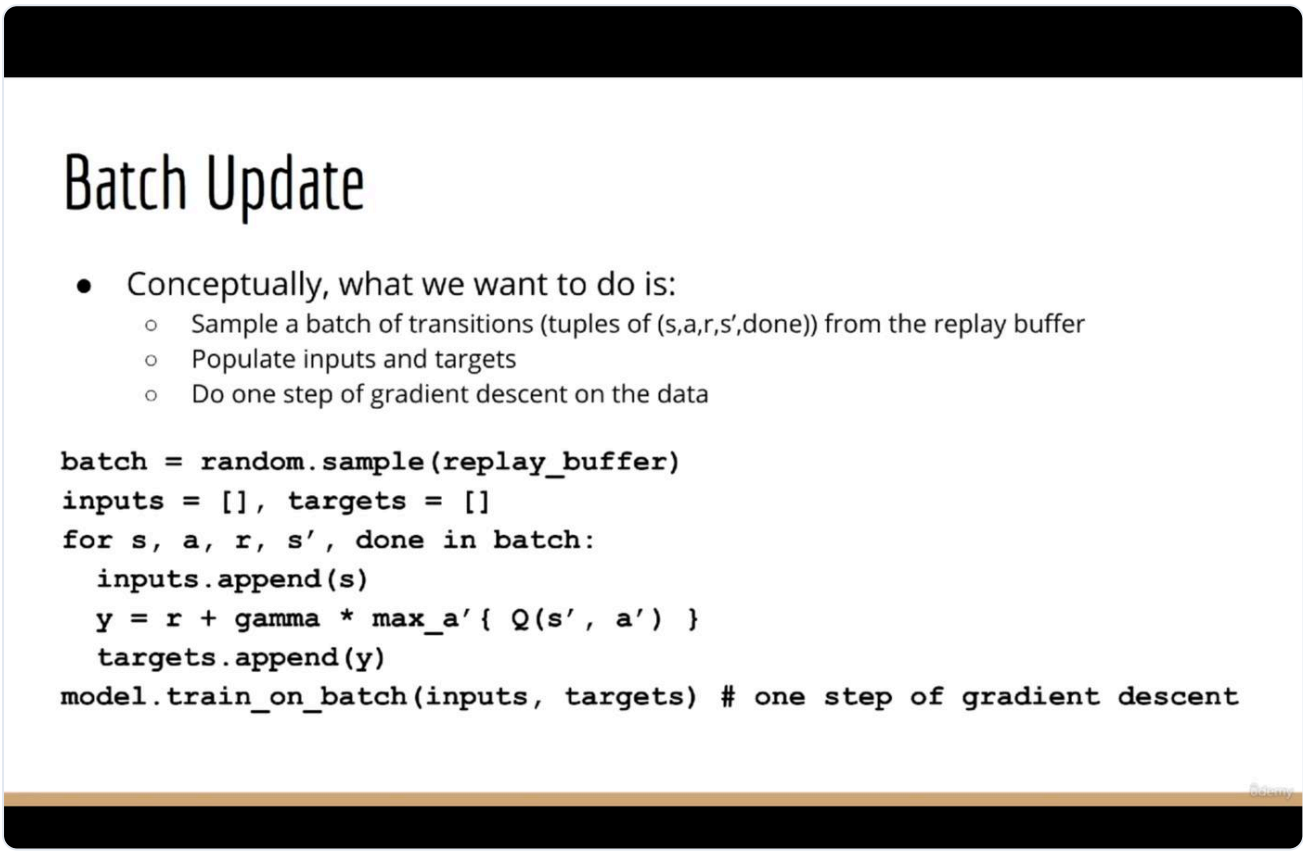

FIGURE 7.16 | Batch Update

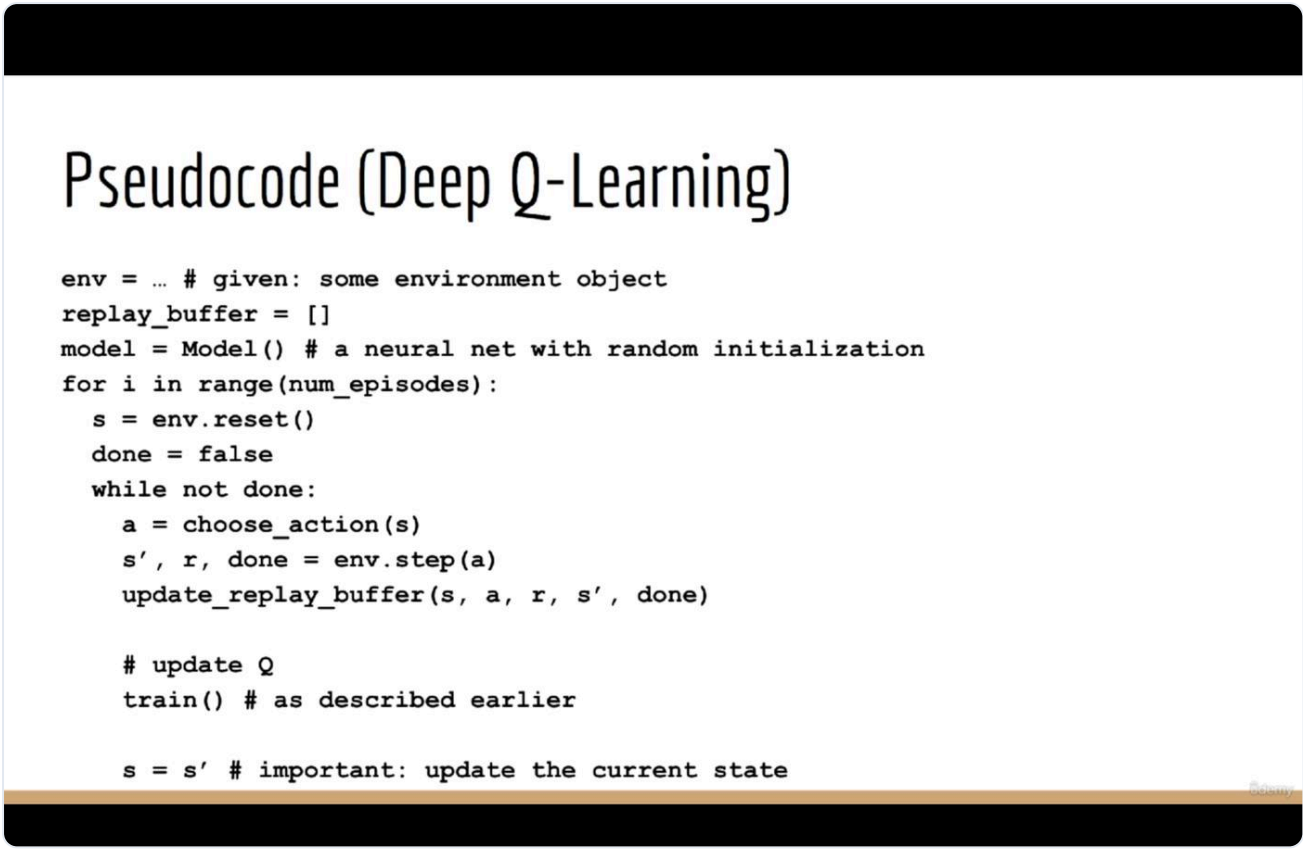

FIGURE 7.17

Next we call the train function which samples a batch of data from the replay buffer calculates the corresponding targets and runs one step of gradient descent. Lastly we set the state s to the next state as prime for the next iteration of the loop.

LECTURE 101

Reinforcement Learning

To end and summarize this section I want to discuss at a high level what it takes to learn it reinforcement learning for real. Personally I think it would be quite difficult to grasp reinforcement learning and understand it on an abstract level from just a single section of a single course let's start by recognizing that reinforcement learning is very different from supervised and unsupervised learning. Nowadays when you're studying supervised and unsupervised learning many beginner courses take the approach of avoiding how to implement the models. This is not a good approach because it doesn't prepare you for understanding real machine learning.

Let's suppose the structure of the courses like this First you discuss the intuition behind how the model works. Just some pictures and some descriptive ideas.

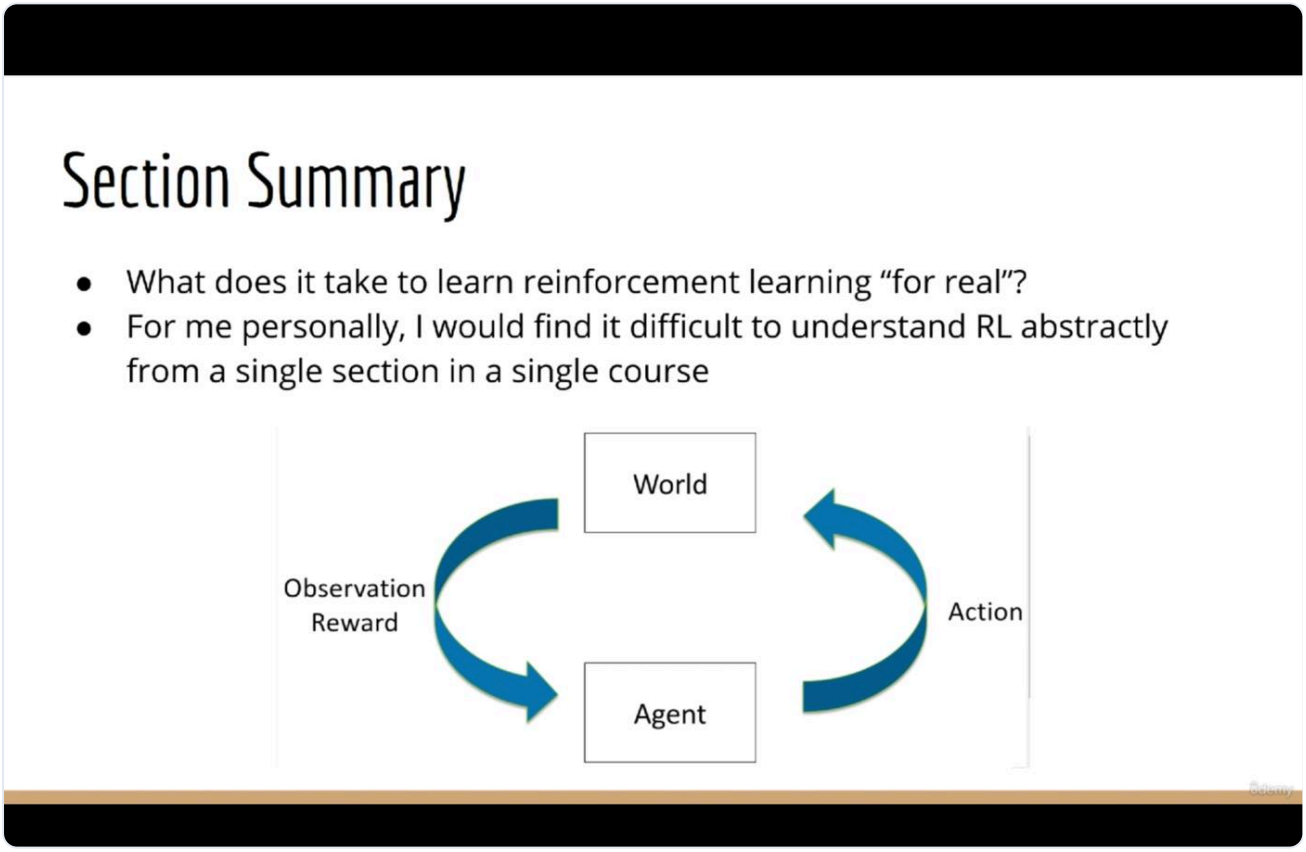

FIGURE 8.1

Then you learn how to use that model in scikit-learn not including loading in the data or looking at the results. So what is the problem with this approach. Well it allows you to not only make many mistakes and have serious misunderstandings about the content but perhaps the biggest problem with this approach is that you will never know what those misunderstandings are. In other words you are in a place where you don't know what you don't know. You don't know your mistakes and hence you can't fix them.

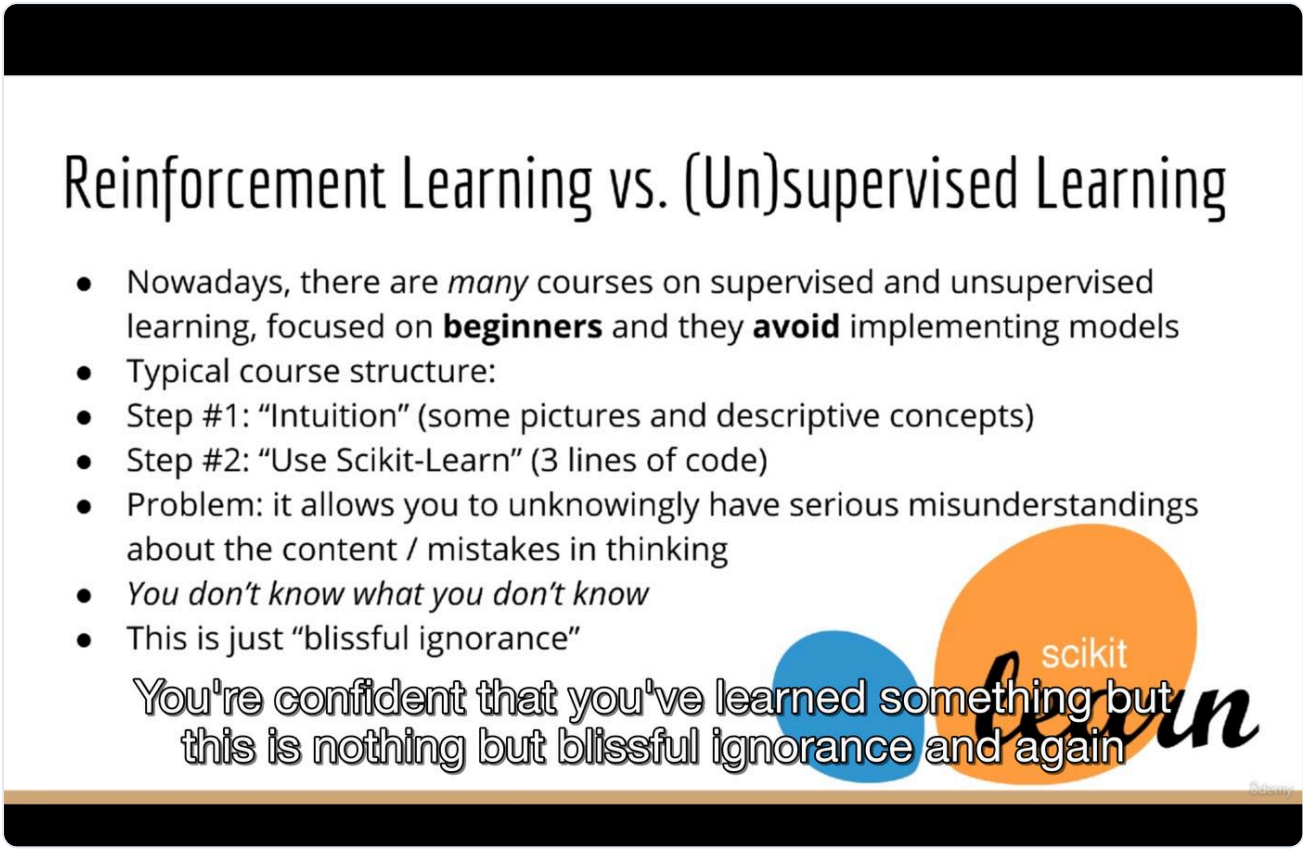

FIGURE 8.2 | Nowadays, There Are Many Courses on Supervised and Unsupervised

You're confident that you've learned something but this is nothing but blissful ignorance and again these are just my observations of other students. So don't take it as a personal judgement of you. This is not so much a problem.

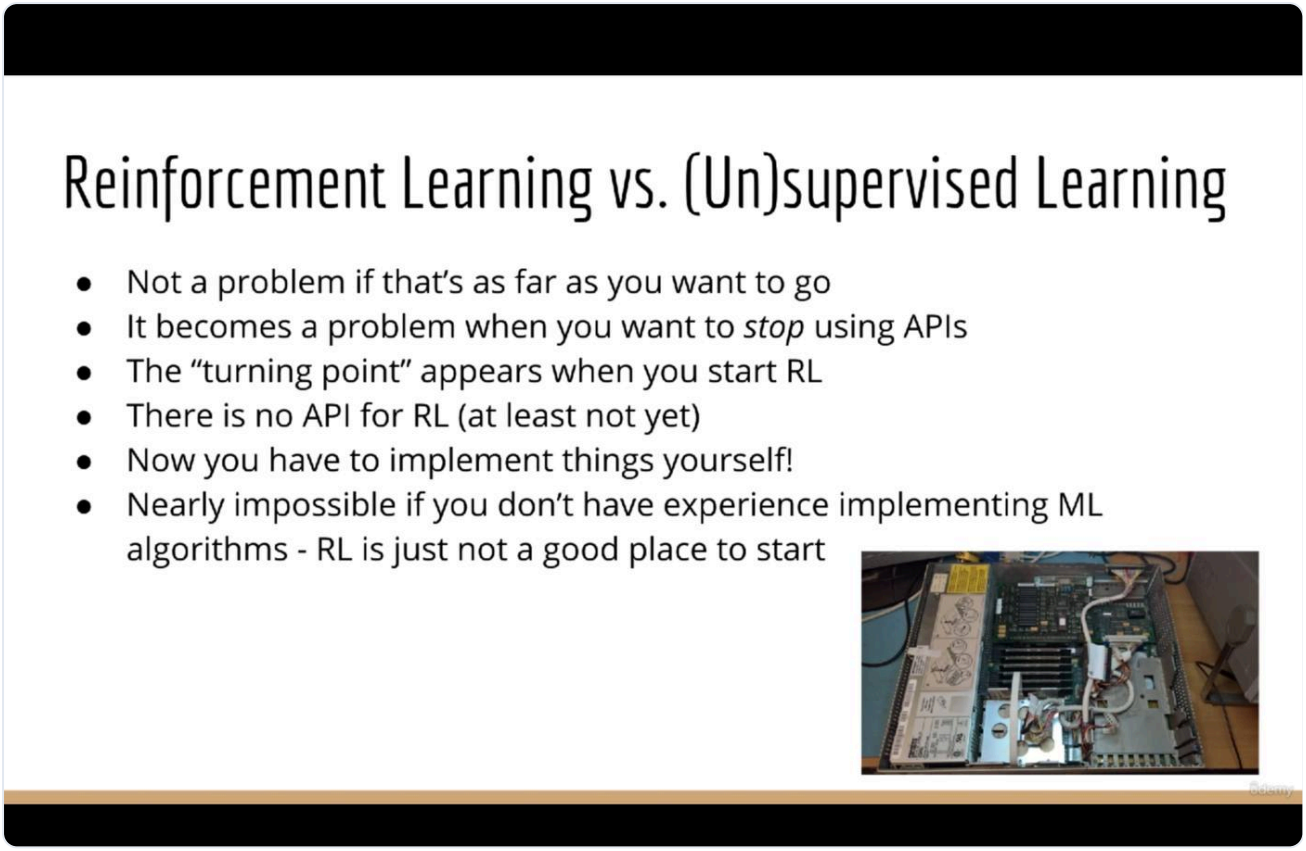

FIGURE 8.3

If that's how far you want to go if you only ever plan on using APIs and you'll never want to become an actual professional that's totally fine and nobody is judging you but it becomes a problem when it's time to stop using APIs. This turning point appears when you want to learn how to do reinforcement learning. There is no API for reinforcement learning at least not yet. So what is one to do. Well you have to implement those algorithms yourself. The problem is if you have no experience implementing machine learning algorithms reinforcement learning is not a good place to start. I want to also comment about the many blogs tutorials and so forth that exist out there attempting to teach you about reinforcement learning.

Consider how long it took us in this section to go from nothing to deep Q learning at a pace that I would consider as fast as possible. Compare that to the length of a typical blog tutorial.

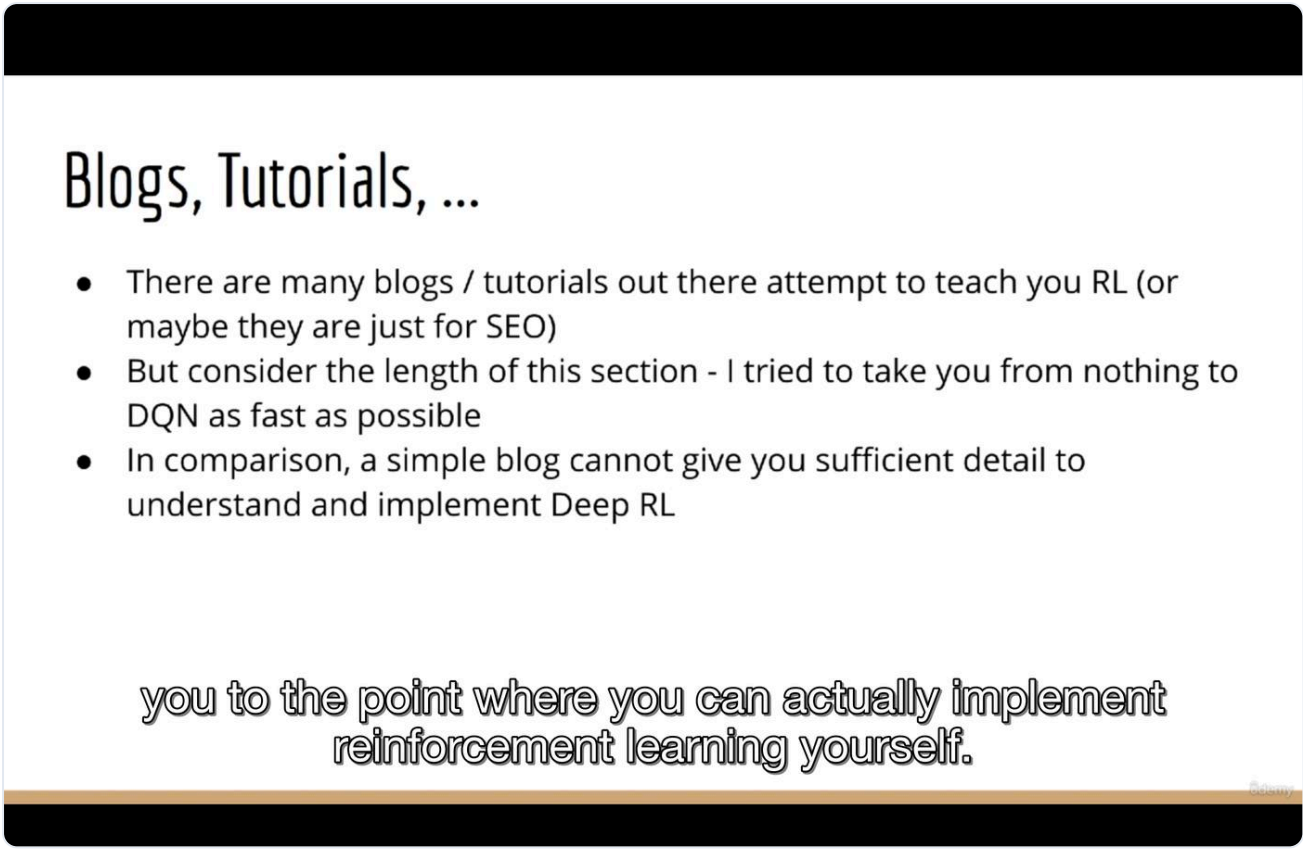

FIGURE 8.4 | Blogs, Tutorials

It should be clear that it would be impossible for a short blog post to include enough detail to get you to the point where you can actually implement reinforcement learning yourself. The best advice I can give you if you want to learn reinforcement learning for real is this. I would recommend taking a full course or multiple courses about reinforcement learning. Learn about basic tabular reinforcement learning before you move on to approximation methods that will make understanding the concepts much easier in this section we spent most of our time discussing the concepts under the assumption that we were using a Q table and not a neural network.

You want to learn about the basic approaches to reinforcement learning which can be categorized into number one dynamic programming methods. Number two Monte Carlo methods.

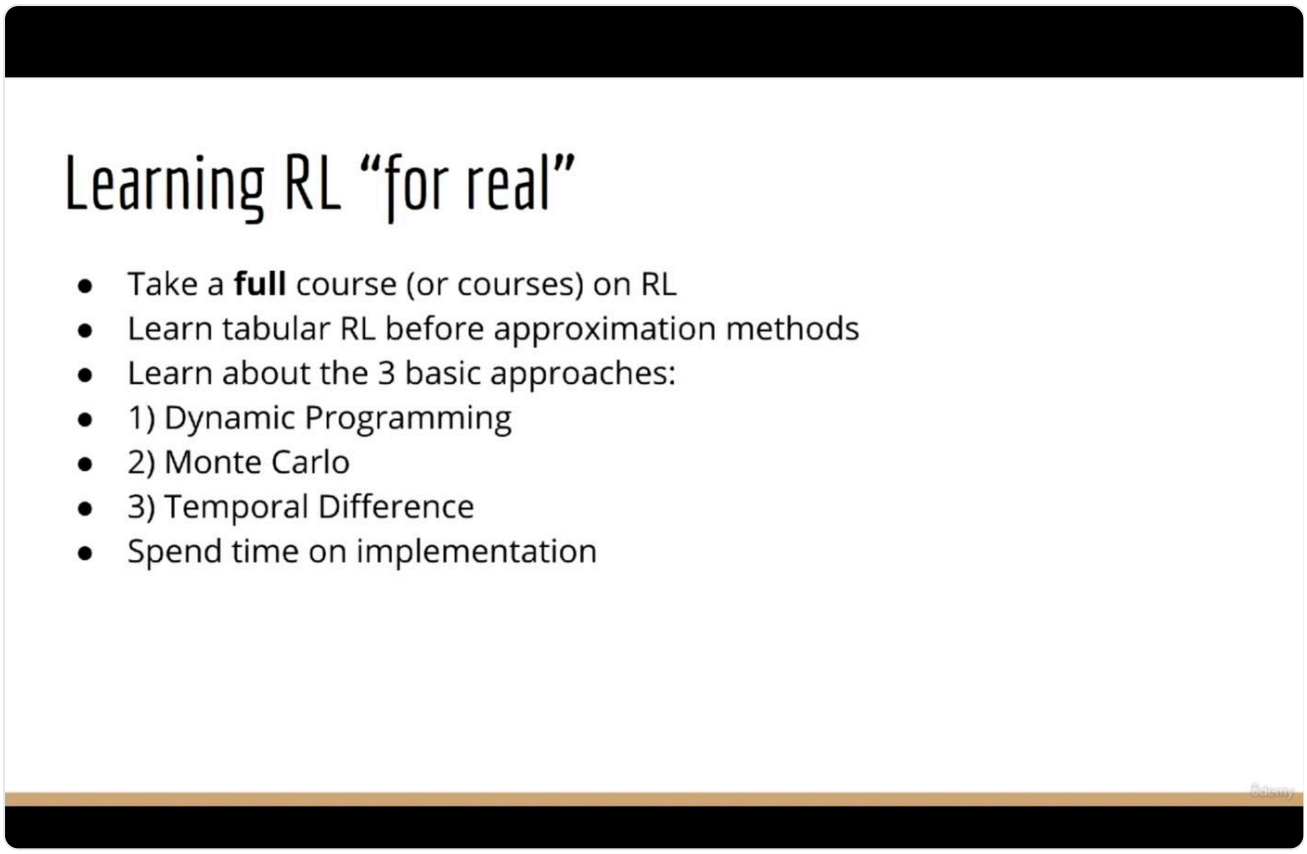

FIGURE 8.5 | Take a Full Course (or Courses) on RL

And number three. Temporal Difference methods. And make sure to spend some time on implementation after that. Move on to approximation methods with linear models. Once you've done that and you've implemented it reinforcement learning algorithms with linear models move on to using deep learning for function approximation at this point you should be very comfortable with the concepts but implementation is still another matter entirely.

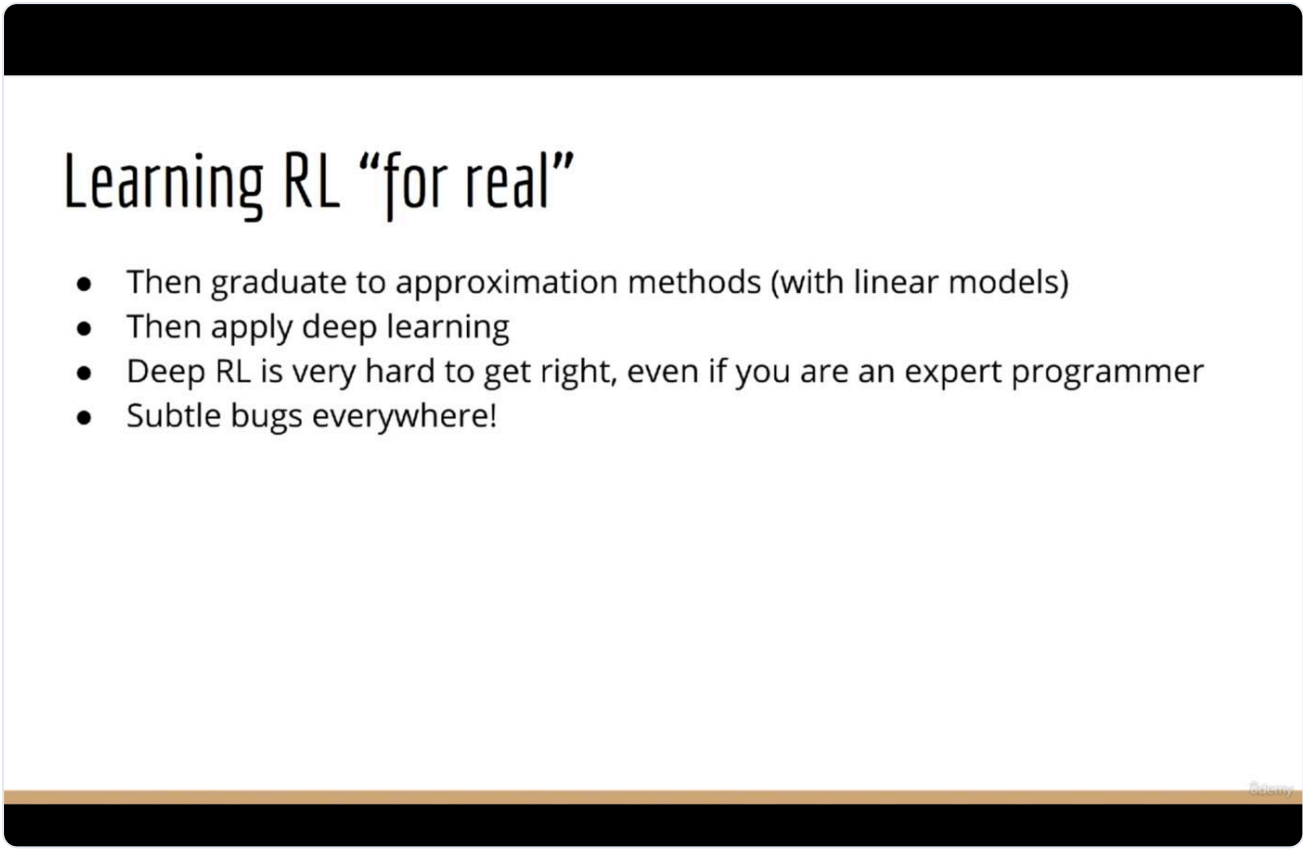

FIGURE 8.6 | Then Graduate to Approximation Methods (with Linear Models)

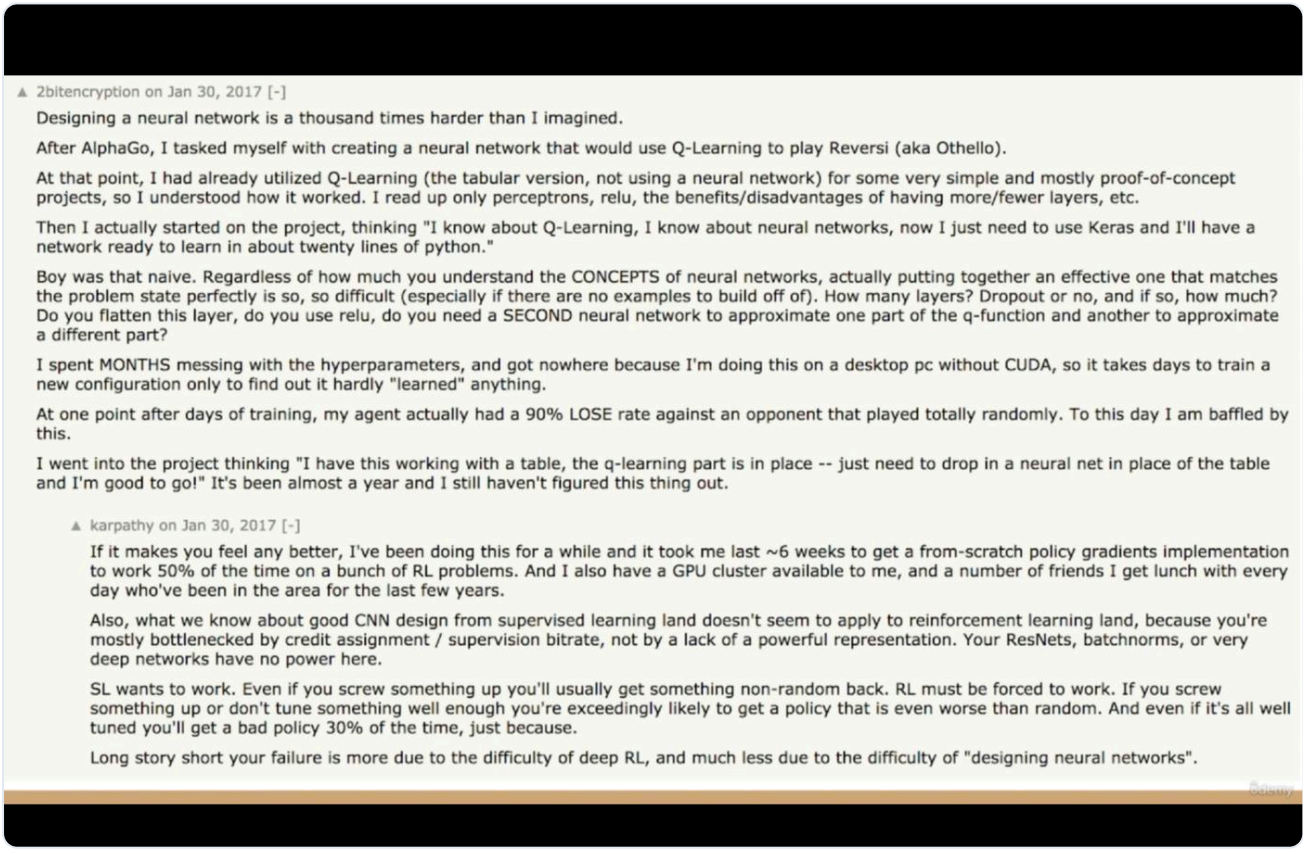

FIGURE 8.7

Unlike supervised and unsupervised learning reinforcement learning is very hard to get right even if you are an expert programmer. It is very easy to introduce subtle bugs that are difficult or nearly impossible to track down and of course you don't have to take my word for it. Here is a post by Andrej Karpathy probably one of the most famous reinforcement learning researchers talking about just how difficult it can be and this come in another user talks about how they spent an entire year trying to get Q learning to work with neural networks for a particular game. Many students who take my courses get frustrated when an exercise takes more than a few minutes.

Now imagine working on something for one year Andrej responds to this comment and says it took me six weeks to get policy gradients working and keep in mind he's one of the world's leading researchers in reinforcement learning. He has all the state of the art technology he needs on demand and he's surrounded by other researchers who are just as smart as he is so he can ask them for advice or to check his code at any time. So the key idea I want you to keep in mind is that as you learn reinforcement learning realize that implementation is very non-trivial even for experts. You should expect to spend a lot of time and effort trying to get things right.

LECTURE 102

A Matter of Perspective

In this lecture I'm going to introduce you to the next project of this course. Creating a reinforcement learning trading bot.

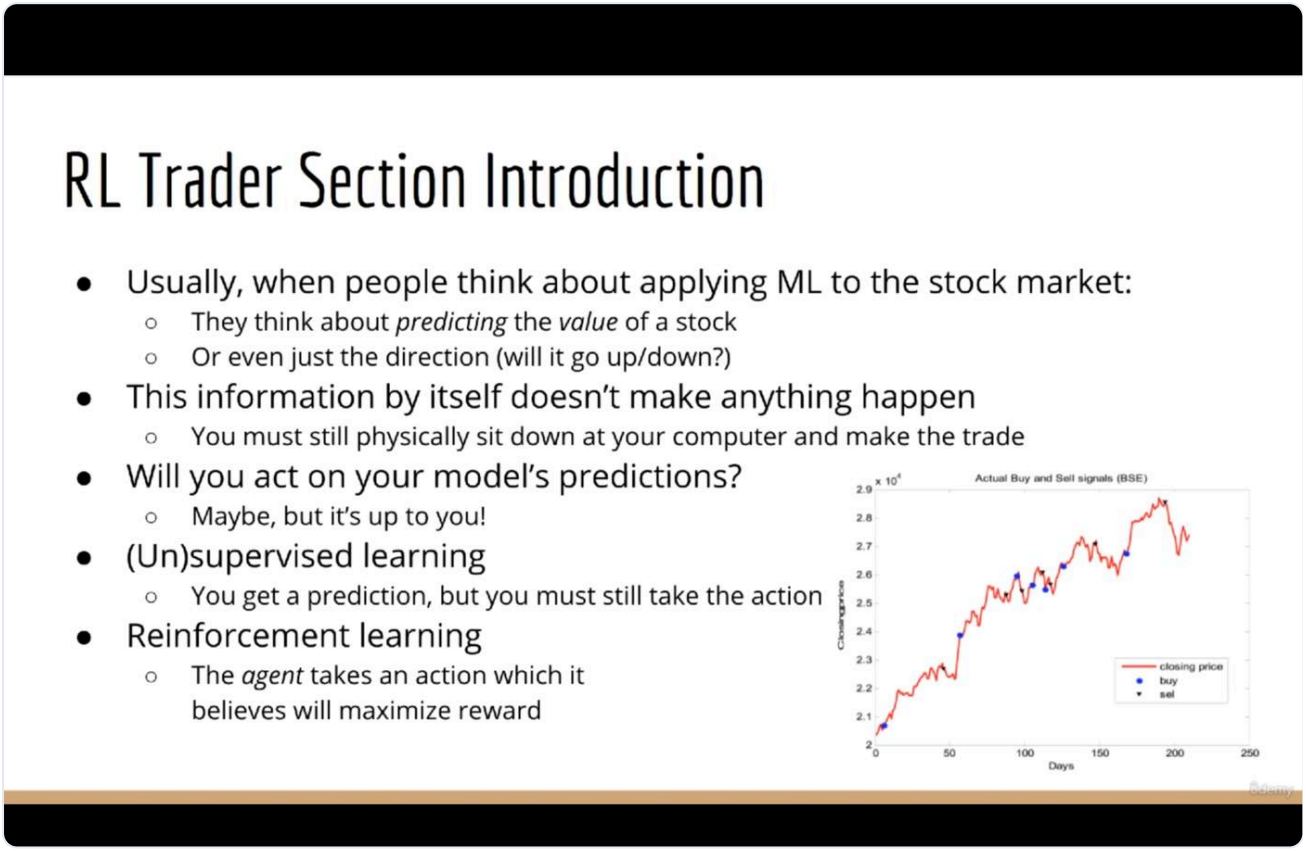

FIGURE 9.1 | They Think About Predicting the Value of a Stock

Usually when people think about applying machine learning to the stock market they usually think about it in terms of predicting the value of a stock which includes even just the direction like whether it will go up tomorrow or down tomorrow. Of course that information by itself doesn't do anything physically. You still need to sit down at your computer and make a trade. If we're talking about automated high frequency trading then that's a different story. Even so let's say your model predicts that the stocks you are looking at will go up tomorrow. Does that mean you'll make a trade. Maybe. But what if you're busy and you forget. Or what if you believe it's going to go up only slightly and then decrease rapidly then probably you don't want to buy that stock.

This is the key difference between a traditional supervised and unsupervised learning versus reinforcement learning supervised learning only makes a prediction. It doesn't actually take any action based on that prediction. I can predict tomorrow's stock price but I still have to choose whether I will act on that information or not. Reinforcement learning on the other hand not only makes predictions but also takes actions in the environment that you provide. So in this section

of the course we are going to study how a reinforcement learning algorithm might take such actions in a stock trading environment let's just go over a rough outline of how this is going to work.

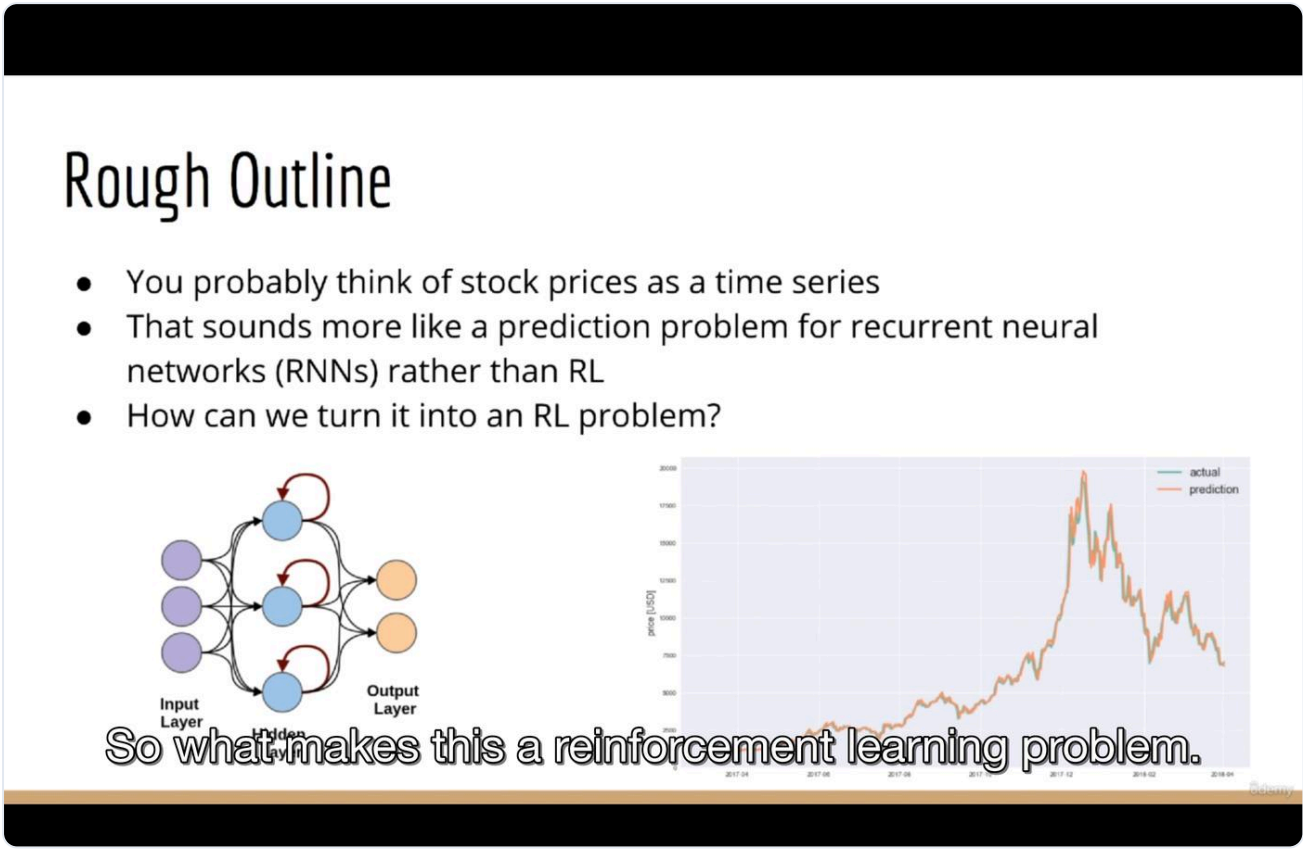

FIGURE 9.2

Currently you probably just think of stock prices as a simple time series data set at this time it has this value at the next time it has this other value and so on. That sounds more appropriate for a recurrent neural network rather than reinforcement learning. So what makes this a reinforcement learning problem. Well it's a matter of perspective. Imagine let's say you're hooked up to a stock trading API using this API you can call functions which do real world financial transactions. So if I call the buy function passing in the argument GOOG and 10 that means I'm going to buy 10 shares of Google stock.

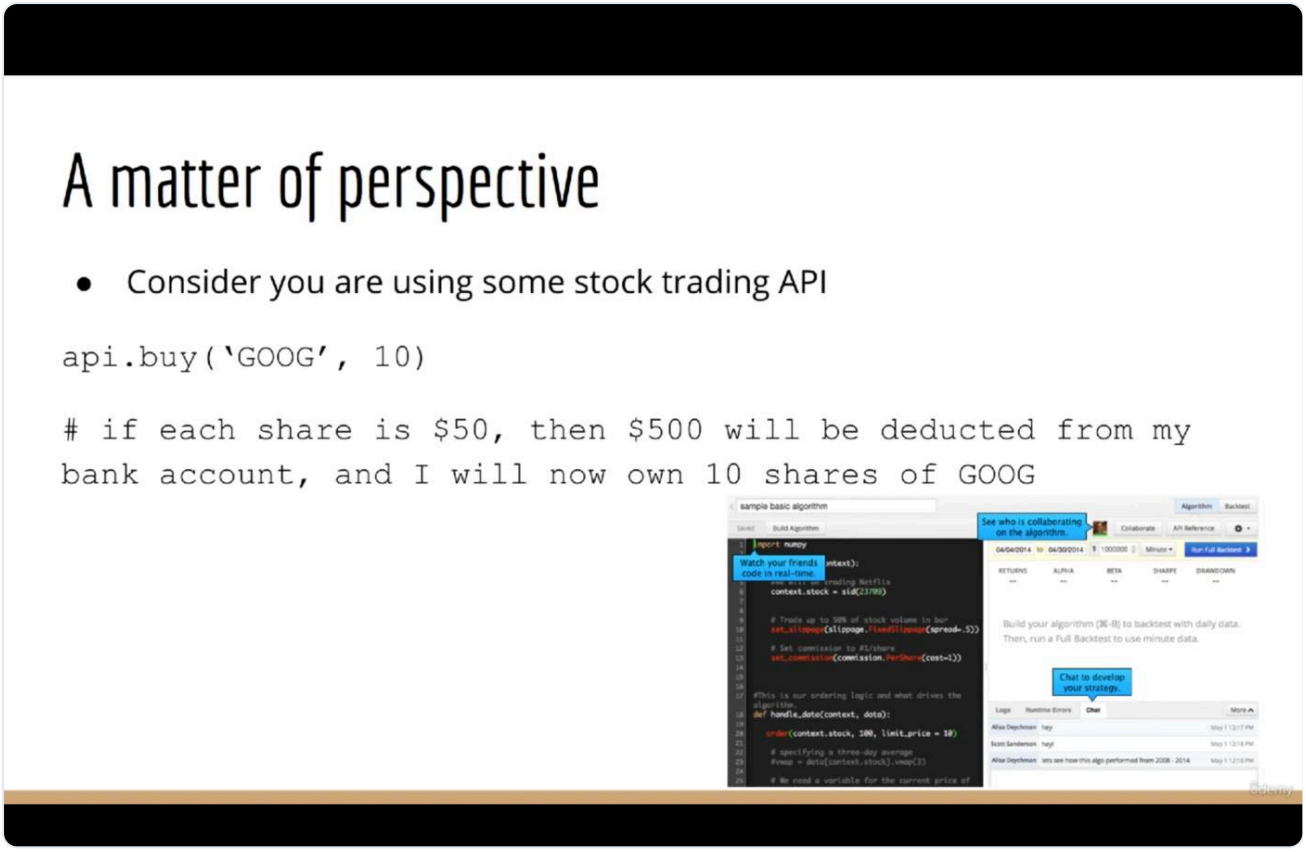

FIGURE 9.3 | A Matter of Perspective

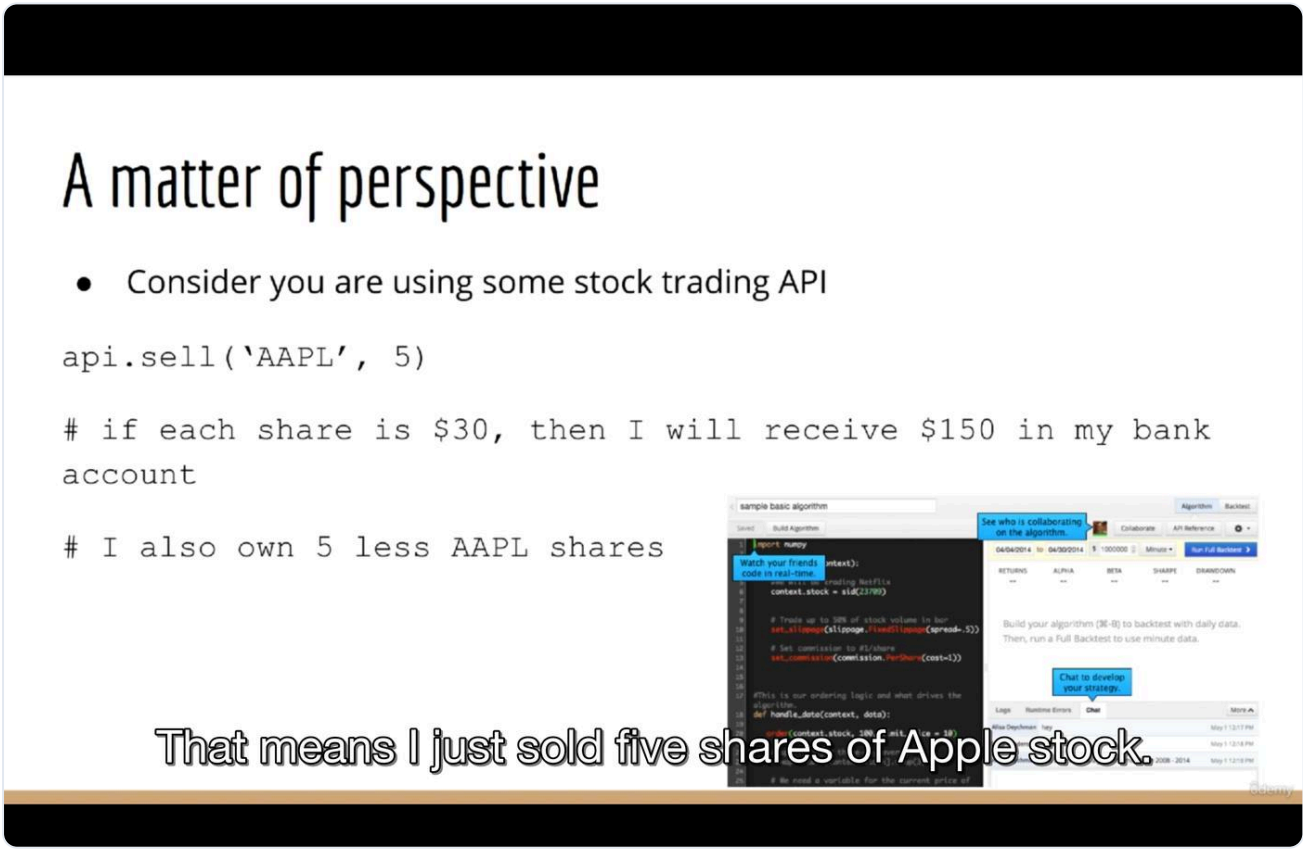

FIGURE 9.4 | A Matter of Perspective

If each share is worth fifty dollars that means five hundred dollars is going to be deducted from my bank account and instead I will now own it 10 shares of Google let's say I call the sell function and I pass in the argument AAPL with the number five. That means I just sold five shares of Apple stock. If one share of Apple is worth thirty dollars then I will now receive one hundred fifty dollars in my bank account and I will own five less shares of Apple stock than I did before. Importantly you can see that the act of calling these functions is an action you might think of your state as information such as recent stock prices. How much cash you have to buy my stocks.

How many shares of each stock you own the values of those stocks and so on.

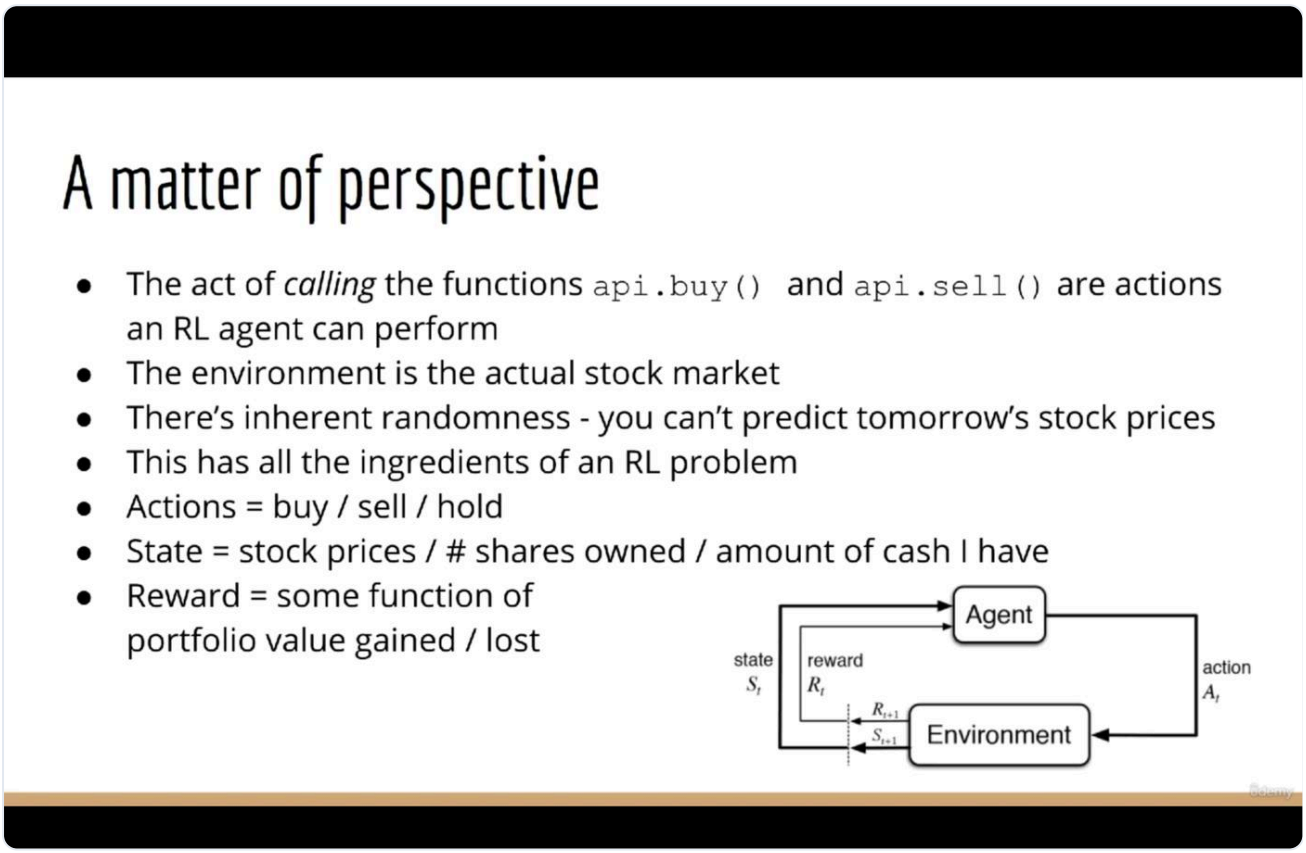

FIGURE 9.5 | A Matter of Perspective

The environment is the actual stock market. It's inherently random because you can't really predict what's going to happen to tomorrow's stock price but your actions affect the environment. In other words these are all the ingredients we need to specify our problem as a reinforcement learning problem. We can perform actions such as buy and sell in the environment and our state is made up of information about various stocks in our own portfolio and the environment is the stock market itself. The reward is some function of the money we made or lost something useful to try. Which is probably something many of you have done already is to think about how you yourself are a reinforcement learning agent.

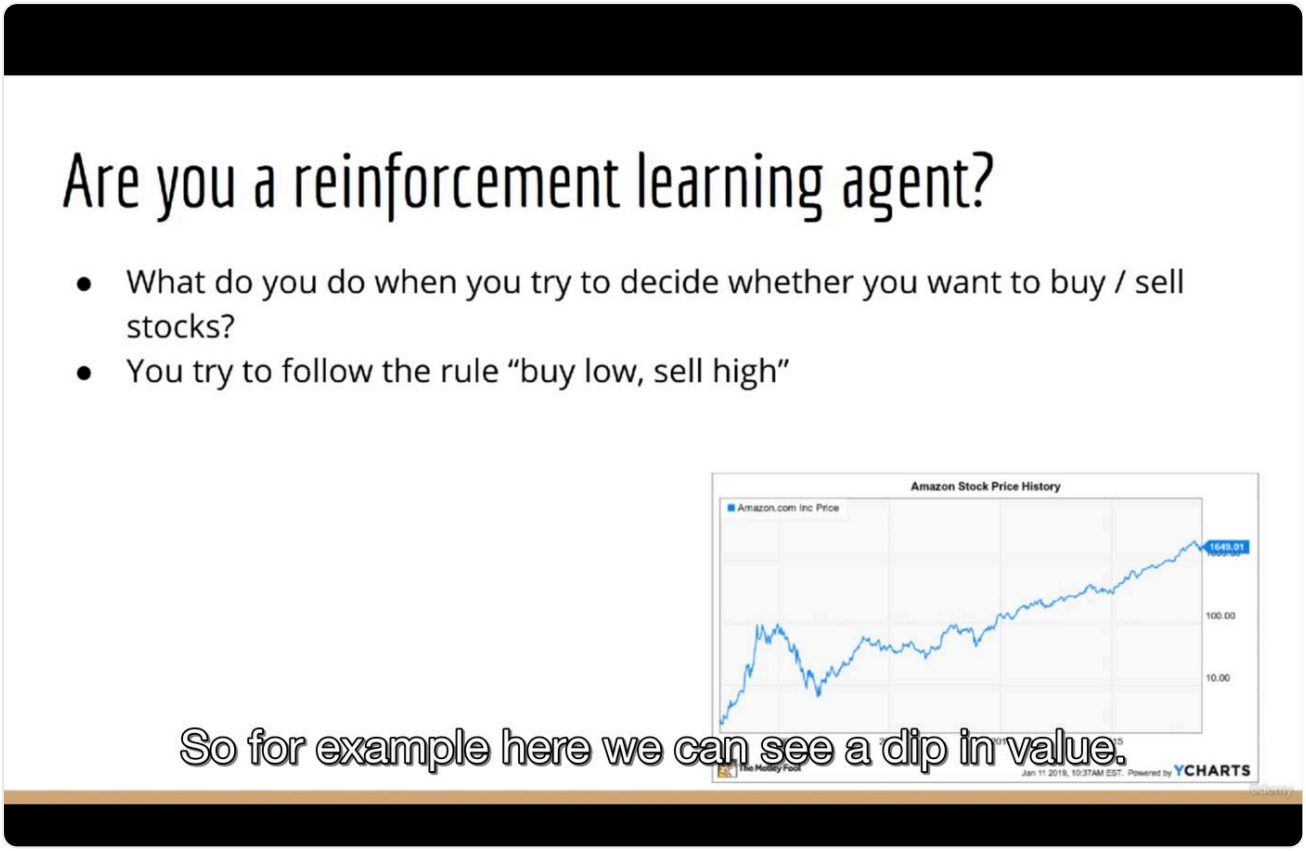

FIGURE 9.6 | Are You a Reinforcement Learning Agent?

FIGURE 9.7 | Are You a Reinforcement Learning Agent?

When you are looking at a stock and trying to decide whether or not to purchase some shares you generally want to follow the rule buy low sell high. So for example here we can see a dip in value. This would be a really good time to buy. And here we see a peak. This would be a really good time to sell well. Only if you need the money. Hopefully you are investing in something where the general trend is always going up. So if you don't need the money then the best thing to do is just let it sit and continue to increase in value. Of course the problem is that in the real world you are trying to make these decisions without knowledge of the future.

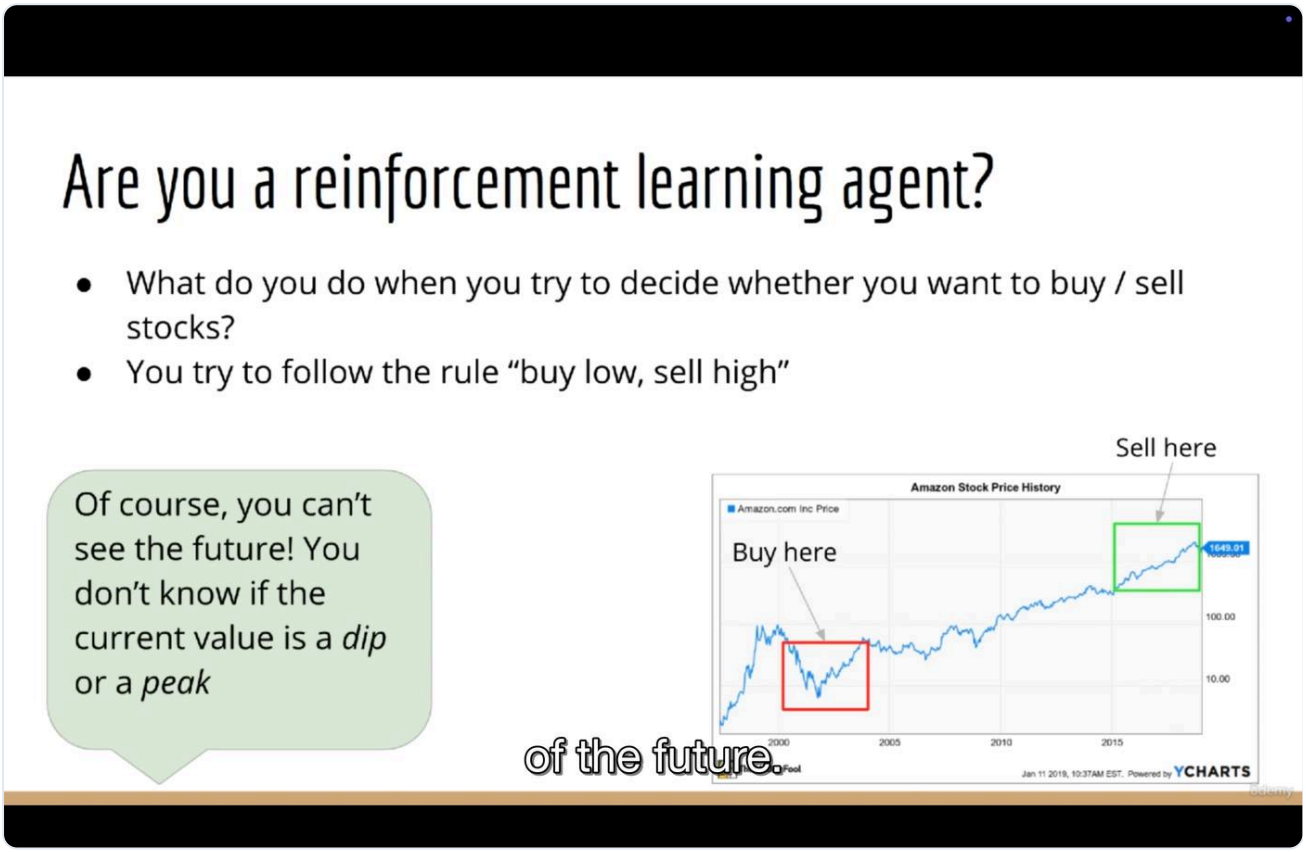

FIGURE 9.8 | Are You a Reinforcement Learning Agent?

How do you know if the most recent price is a dip or a peak. In fact we do not. And so perhaps this is a job best left for the machines.

LECTURE 103

Actions (Simplified)

In this lecture we are going to describe the environment we'll be working with in the following lectures.

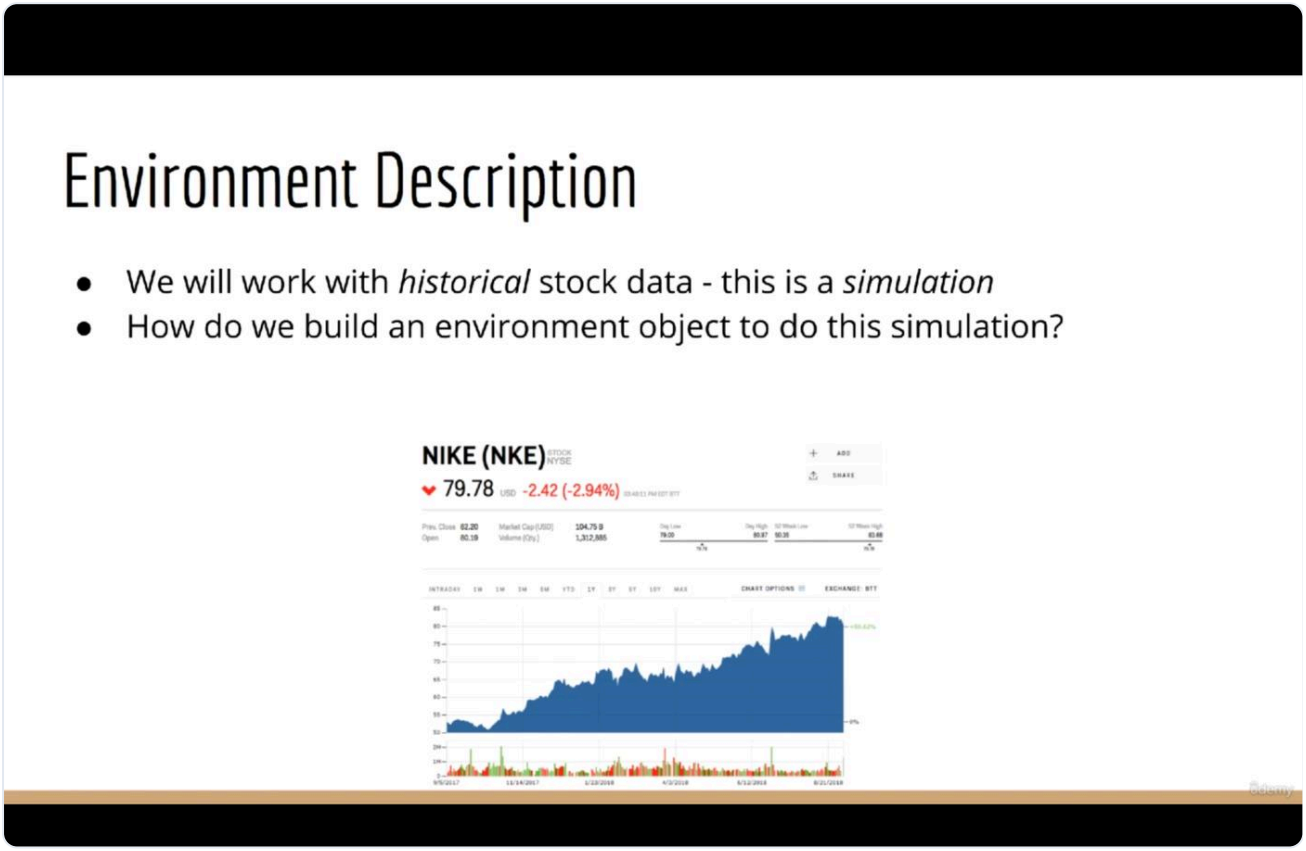

FIGURE 10.1

Firstly because we'll be working with historical stock data. This is a simulation. Of course you would not want to try such an experiment with real money. So really our job is to figure out how to build an environment object in code that simulates the stock market in general. Here's how we can think of the API for an environment by the way. If you're familiar with OpenAI Gym then this is probably just review for you. But it's good to go over this anyway.

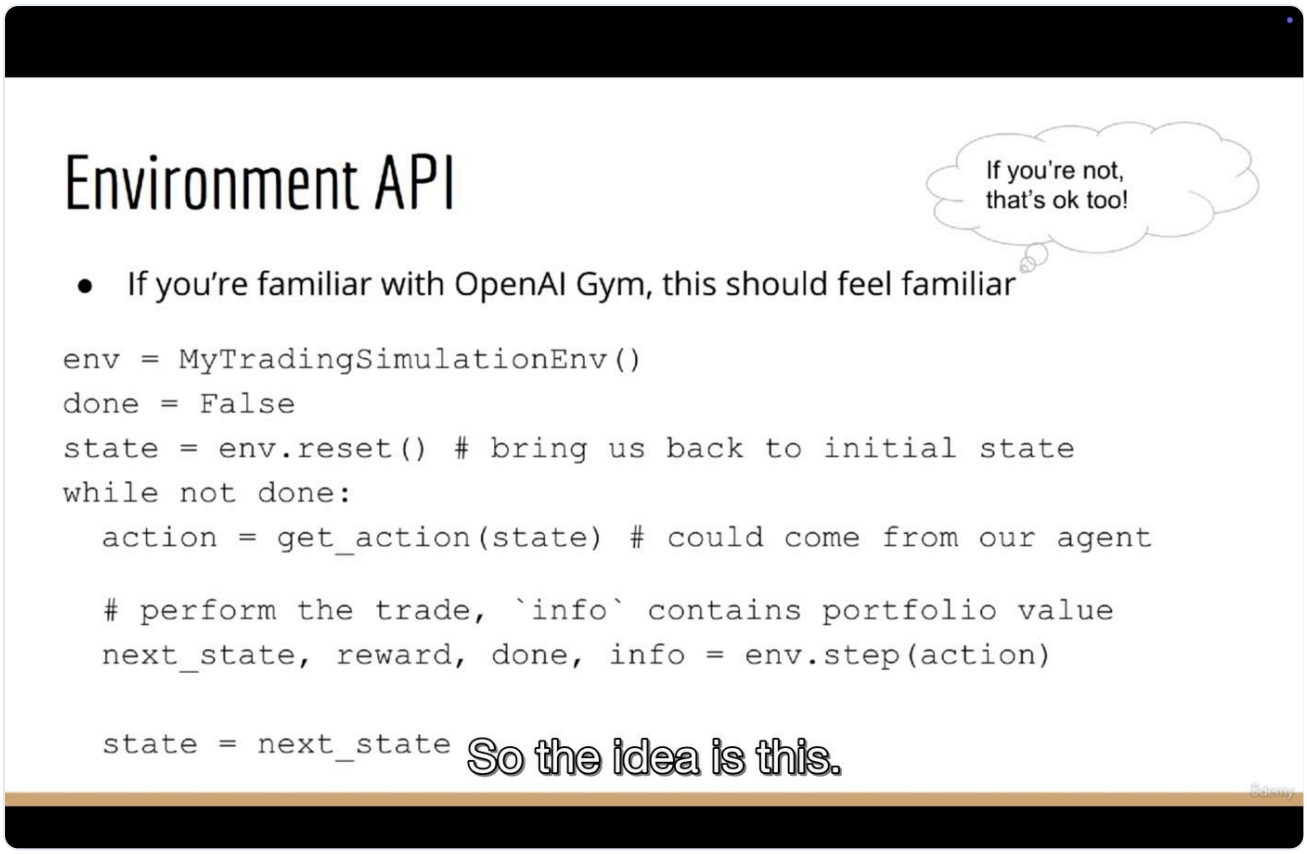

FIGURE 10.2

So the idea is this. First we are going to instantiate an environment object. Then we are going to initialize a boolean done flag equal a false. We are also going to call env.reset which puts us back into the starting position for this environment and returns the initial state. As a side note this state vector may not be at a good scale to pass into a neural network. As you recall remember that we like to normalize data before passing it into a neural network or linear regression so keep in mind that you can do this as an optional step then we are going to enter a loop which only quits when done becomes true inside the loop. We are going to choose an action to perform in the environment.

This might come from our agent but that's not a necessary detail at this stage because we are only thinking about the API for the environment you could just as easily choose a random action although most likely this will lead to a suboptimal reward. The next step is to actually perform the action in the environment. We do that by calling env.step and passing in the action as an argument this will return a few things. First it returns the next state. Second it returns the reward for arriving in the next state. Third it returns a done flag to tell us whether or not the episode is over. Finally it returns an info dictionary which can tell us additional information about the environment.

This one is not strictly necessary and in fact it's empty for many environments but for us we actually going to populate the info dictionary to tell us the current value of our portfolio. This is not part of the state but can be calculated from the state variables. Thus it's easier to simply calculate it inside the environment and return it along with everything else. Finally we assign the next state variable to the state variable in the case where on the next step the agent needs to use the state to choose an action so that's pretty simple and you'll find that in general no matter what en-

vironment you are looking at it's going to have an API. Just like what we saw the questions we really want to answer now are what should the state be and what should the action be and what should the reward be.

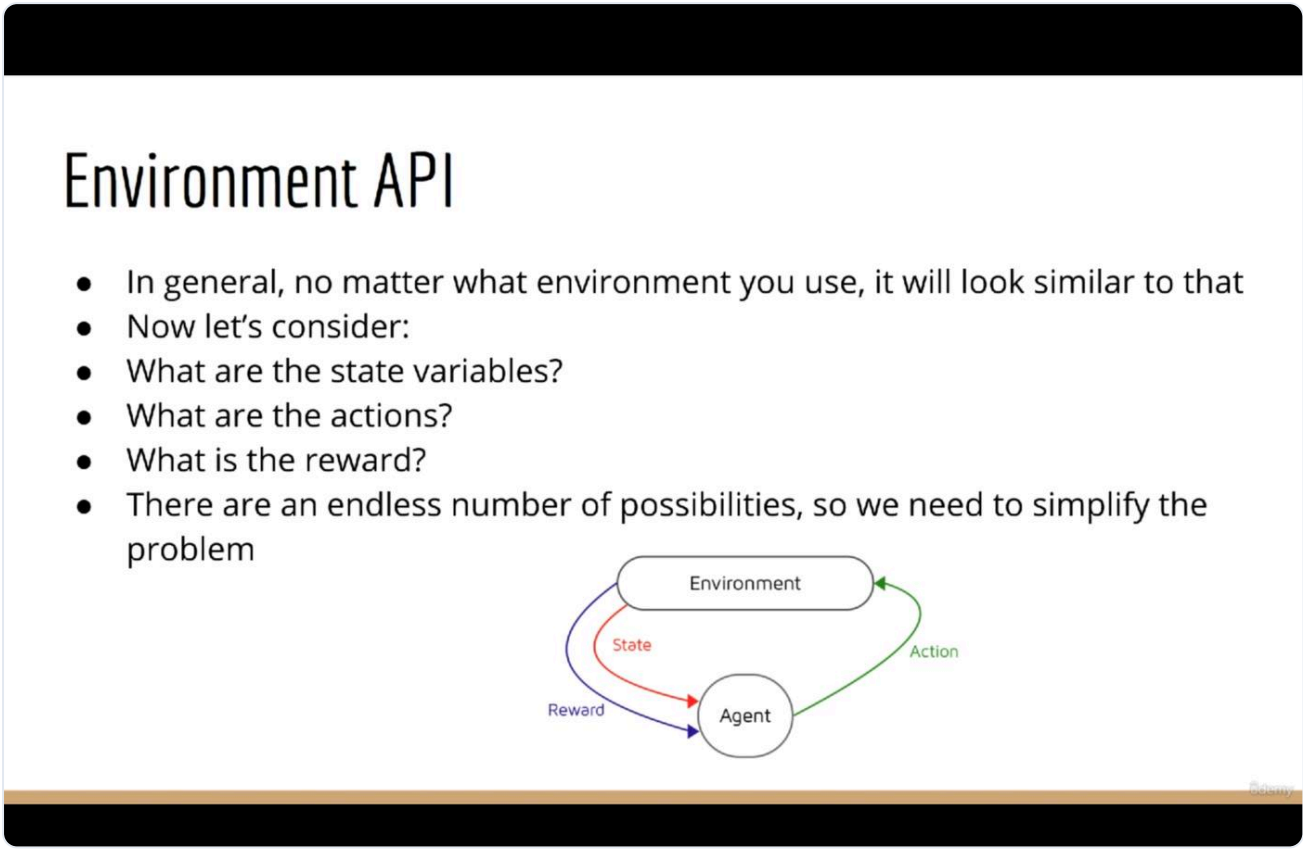

FIGURE 10.3 | What Are the State Variables?

The reason we need to discuss these is because there are an endless number of possibilities and complications. We aren't necessarily going to have to simplify the problem a little bit but first let me explain to you why this simplification is necessary let's start with the state. There are many things you could consider here. First you can think of it exactly like a time series problem. Look at the pattern of stock movements in the past and from that make a decision. That's probably the first thing you and I would do when we decide if we're going to buy or sell a stock. However there are other things to consider.

FIGURE 10.4 | Many Options

We also have to ask do we own enough cash to buy the stocks we want to buy and given the prices of existing shares I own. Is it worth it to sell them in order to get more cash to buy a different stock. So in fact this can become a complex decision problem. We are going to borrow some ideas from a paper called Practical deep reinforcement learning approach for stock trading. This approach used a more advanced reinforcement learning technique known as DDPG but we can apply a few of the ideas they proposed. So here's how we're going to represent our state. It will consist of three parts First we're going to record how many shares of each stock we own. So for example if I'm looking at Apple Motorola and Starbucks this means I own three shares of Apple five shares of Motorola and seven Shares of Starbucks second.

We're going to list out the current share price of each stock. So this means Apple is trading at fifty dollars per share. Motorola is trading at twenty dollars per share. And Starbucks is trading at thirty dollars per share. Finally the last value of the state is how much pure cash we have.

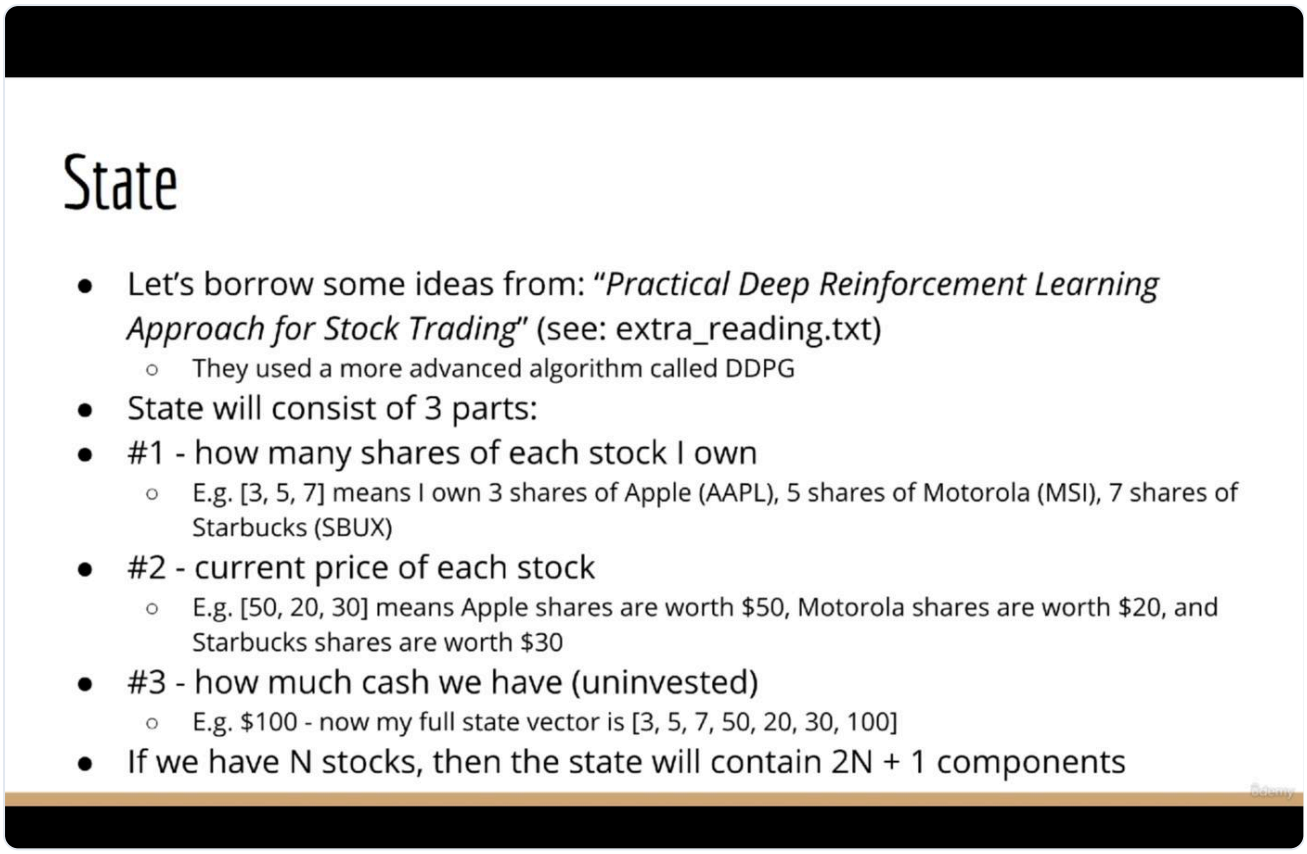

FIGURE 10.5 | They Used a More Advanced Algorithm Called DDPG

That's cash that's not invested in any stock which just sits there and doesn't gain any interest so let's say we have one hundred dollars in cash then our total state vector will be 3 5 7 50 20 30 and 100 you should be able to confirm that if we have N stocks then the size of our state vector will be 2N plus one next. Let's consider the actions again if we consider the sheer amount of possibilities the action space would be extremely large for any given stock. I have three possible options. I can sell I can buy or I can hold. Which means do nothing. Now you might think three is not bad. But now remember we have three stocks to consider for each of these.

I can exercise any of the three options above. So that gives me three to the power three possible actions or twenty seven actions.

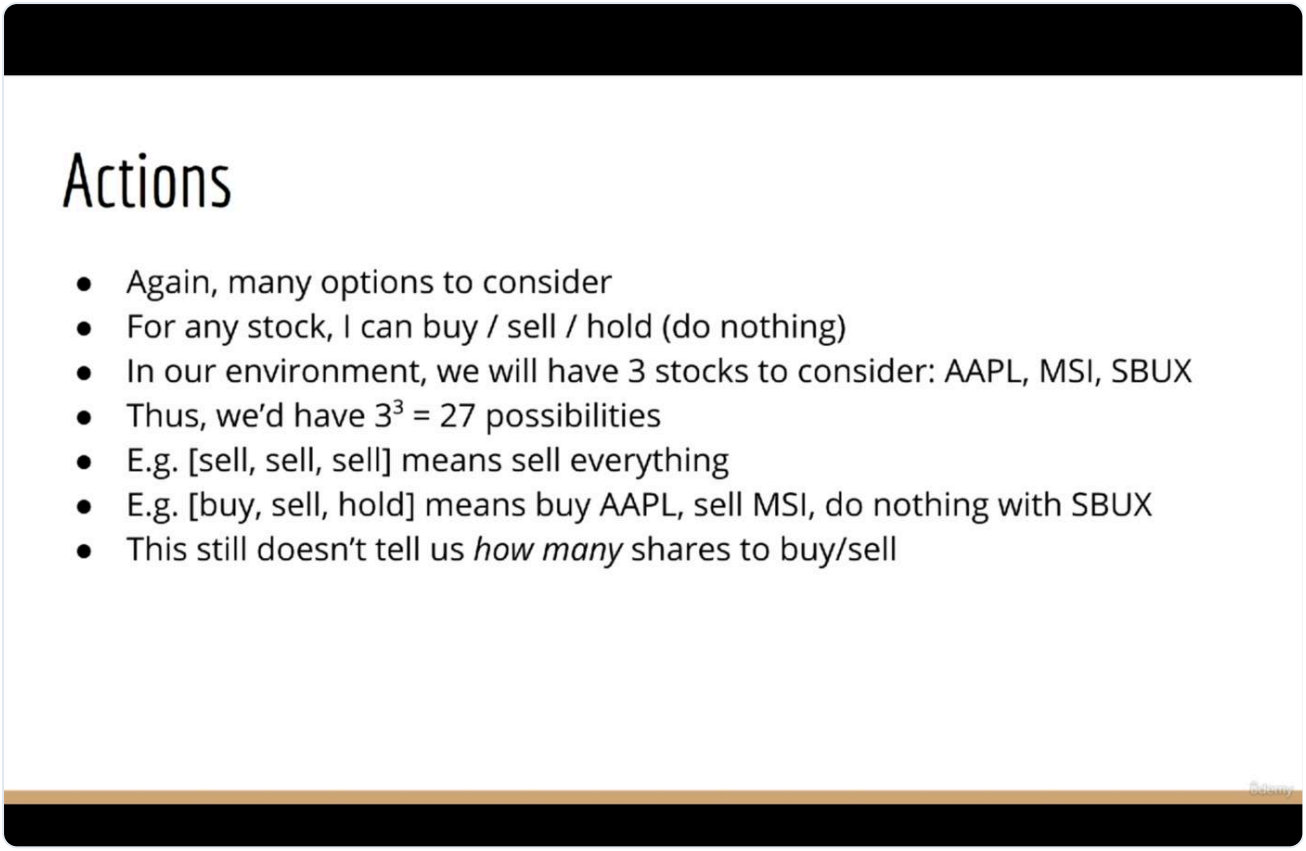

FIGURE 10.6 | Again, Many Options to Consider

For example my action vector maybe sell sell sell which means sell my Apple shares sell my Motorola shares and sell my Starbucks shares or it might be buy sell hold which means buy Apple shares sell Motorola shares and do nothing with my Starbucks shares however this is still not the end of the story because this doesn't say anything about how many shares to buy or sell. If I own ten shares of a stock I can sell anywhere from zero to 10 of those shares. Luckily we are going to simplify this problem a little bit. So here's how we're going to treat actions in our example. It's going to be extremely simplified compared to how things work in the real world but it's a decent start.

First we're not going to consider any transaction costs. For example if you buy shares using your bank's investing platform usually that would cost you ten dollars or so. For us it will be zero next when we sell. We will always sell all of our shares for that stock. So let's say we own 10 shares of Apple stock and we decide to sell. That means we sell all 10 shares secondly when we buy we are going to buy as many shares as possible for the stock we choose to buy. Now you might wonder if I choose multiple stocks to buy and I want to buy as many as possible. How can I do that. Well it's kind of ambiguous. You might think you want to choose the stocks in such a way that leads to using up as much cash as possible.

But in fact this is actually a hard problem known as the knapsack problem.

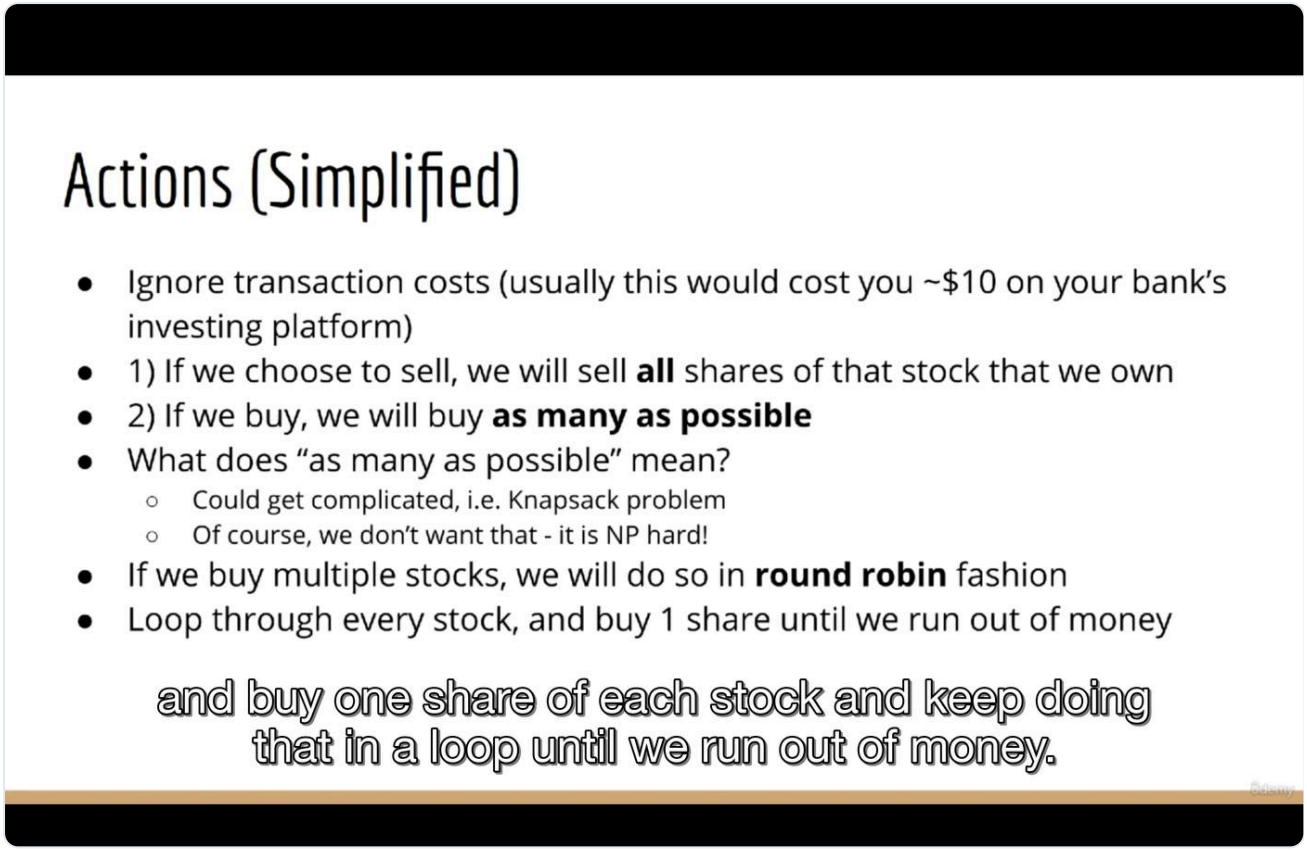

FIGURE 10.7 | Actions (Simplified)

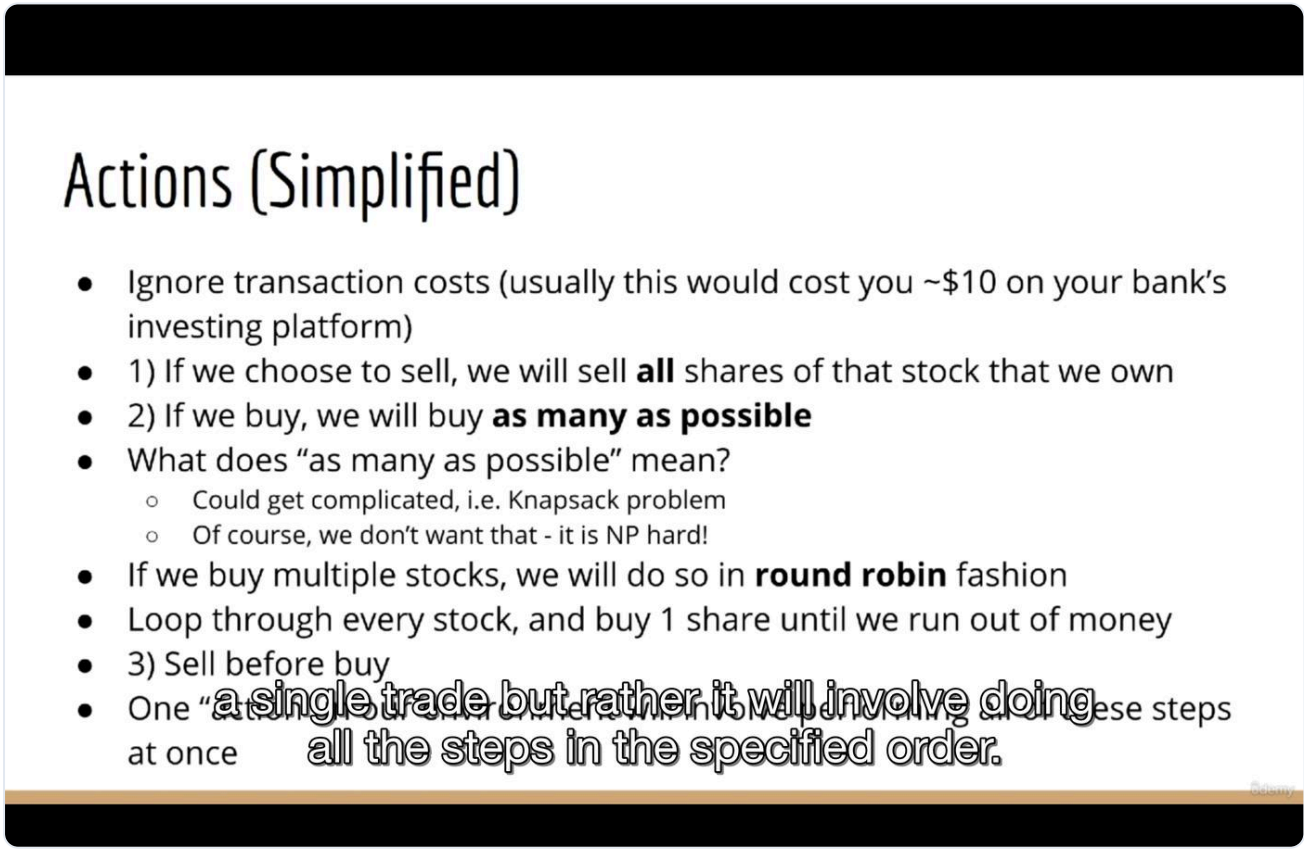

FIGURE 10.8 | Actions (Simplified)

So what we're going to do is we're going to take a simple greedy approach lived through every stock and buy one share of each stock and keep doing that in a loop until we run out of money. Third we will also sell the stocks we want to sell before we buy anything that will leave us with more cash that we can use to buy new stocks this may seem like a very cautious approach but in fact this already leaves us with 27 possible actions which means our neural network will have to approximate 27 different values which is pretty large already and so an action in this environment is not just making a single trade but rather it will involve doing all the steps in the specified order. Finally we have the reward.

This one is simple. The reward will just be the change in the value of our portfolio.

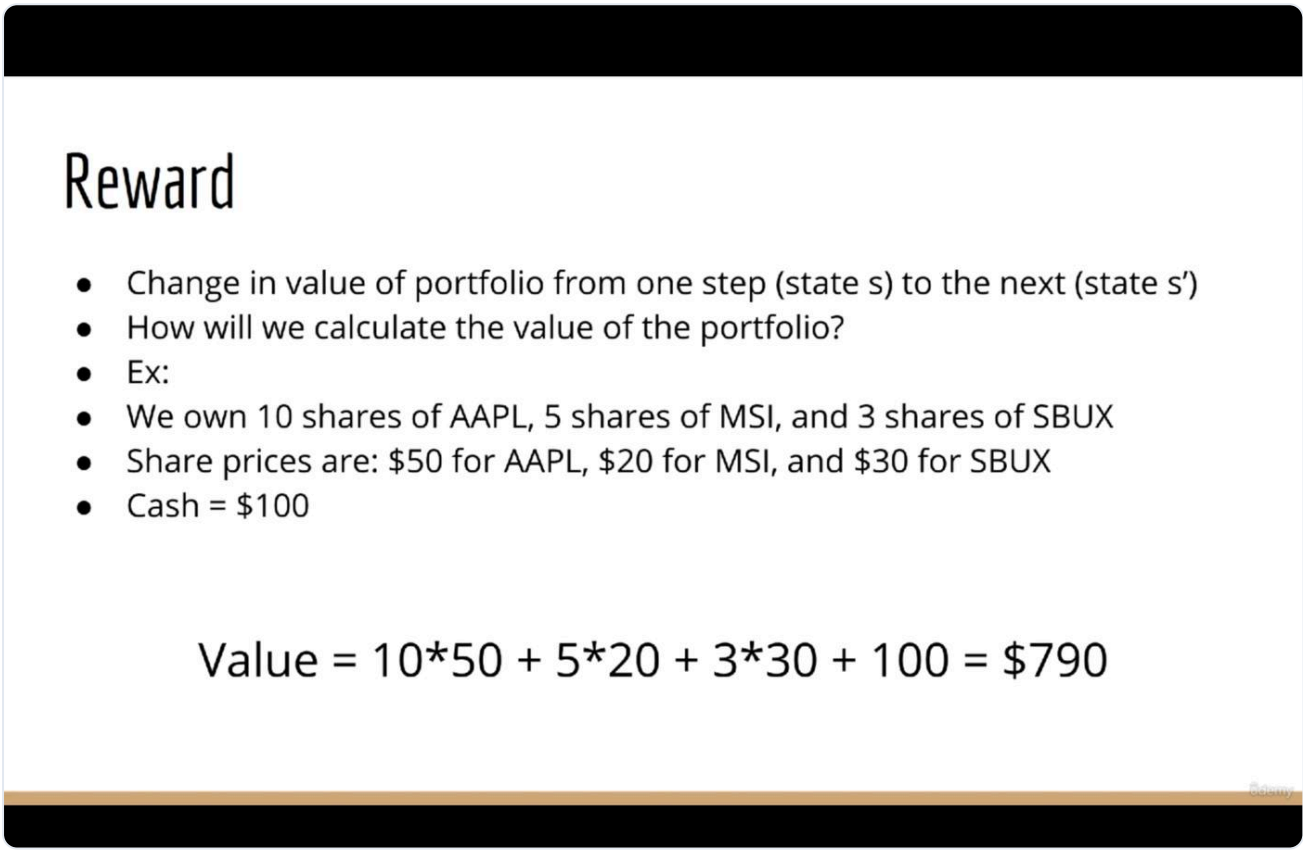

FIGURE 10.9 | How Will We Calculate the Value of the Portfolio?

Now it's We're thinking about how well we calculate the value of our portfolio as an example suppose we own 10 shares of Apple 5 shares a Motorola and 3 Shares of Starbucks. The corresponding share prices are 50 dollars for Apple twenty dollars for Motorola and thirty dollars for Starbucks. Let's also suppose we have one hundred dollars in cash not invested in any stock. Then the total value of our portfolio will be ten times 50 plus five times 20 plus three times 30 plus 100. That's equal to seven hundred ninety dollars in general. If we store the shares we own in a vector called S and we store the corresponding share prices in an array called P and we store the amount of cash we have in a variable called C then the total value of our portfolio can be calculated as follows.

It is equal to the dot product of S and P plus C the reward then we'll just be the difference between these two. Comparing the most recent timestamp and the previous timestamp to summarize this lecture let's recap the important points about the environment and its implementation. First our

environment will be an object that mimics the OpenAI Gym API so it will have functions like reset and step which returns all the information we need to implement our reinforcement learning program. Next for our environment we'll be considering three stocks Apple Motorola and Starbucks next. Our state is a vector with three pieces of information. First it includes the number of shares of each stock that we want to consider.

Second it includes the share price for each of those stocks. Third it includes the amount of cash we have that's not invested in any stock. Next our actions are a simplified subset of the large number of actions we can perform in the real world. We also assume there are no transaction costs. Simply put we have three options for each stock buy sell or hold. We'll take an all or nothing approach where if we buy we're going to buy as many shares as possible and if we sell we're going to sell all of the shares we own. And these actions can be applied in any combination for each stock we own. So if we're considering three stocks then we'll have three to the power three possible actions you'll notice that even with just three stocks and a much simplified action space we still have quite a large number of actions so if we have N stocks we would have three to the power N possible actions.

It grows exponentially with the number of stocks we own. So encoding the actions in this way will not scale.

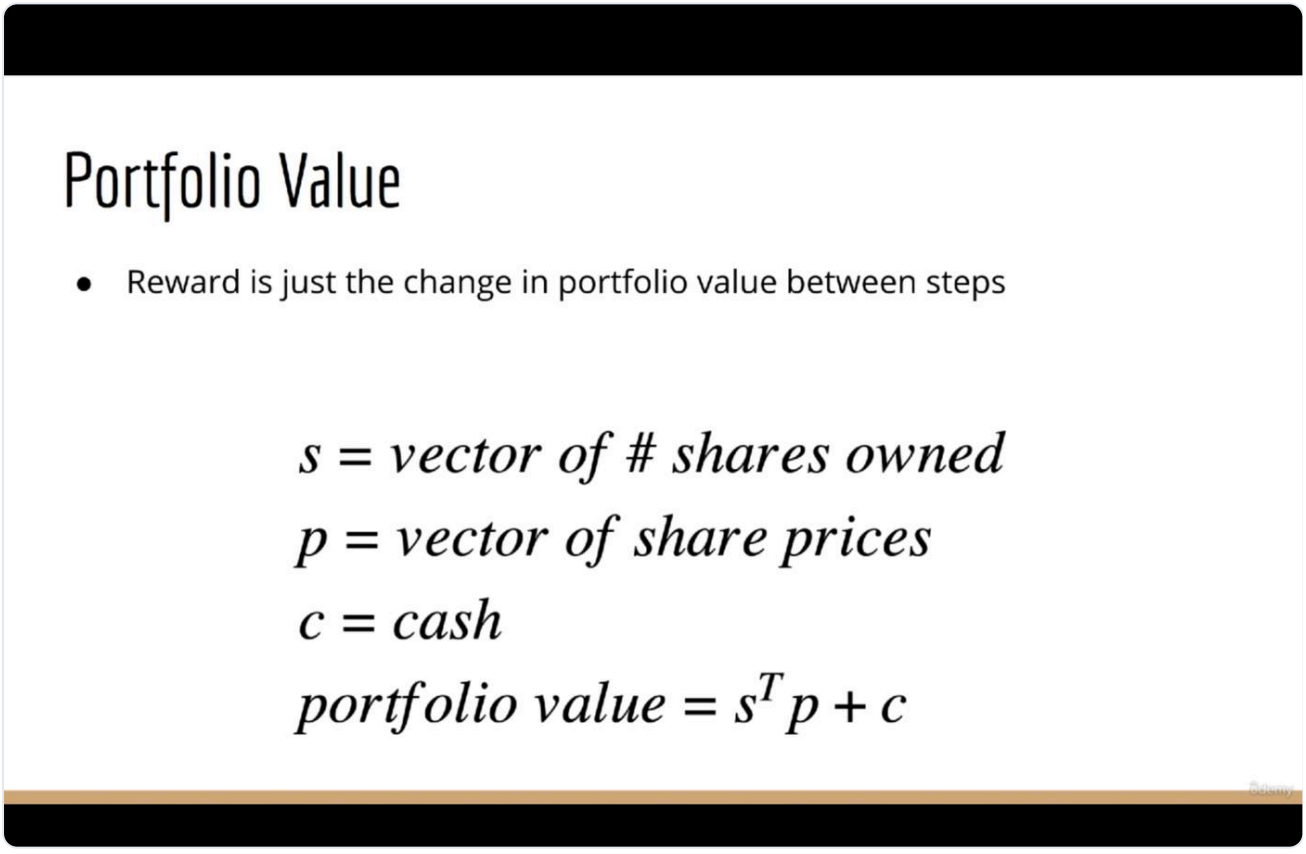

FIGURE 10.10

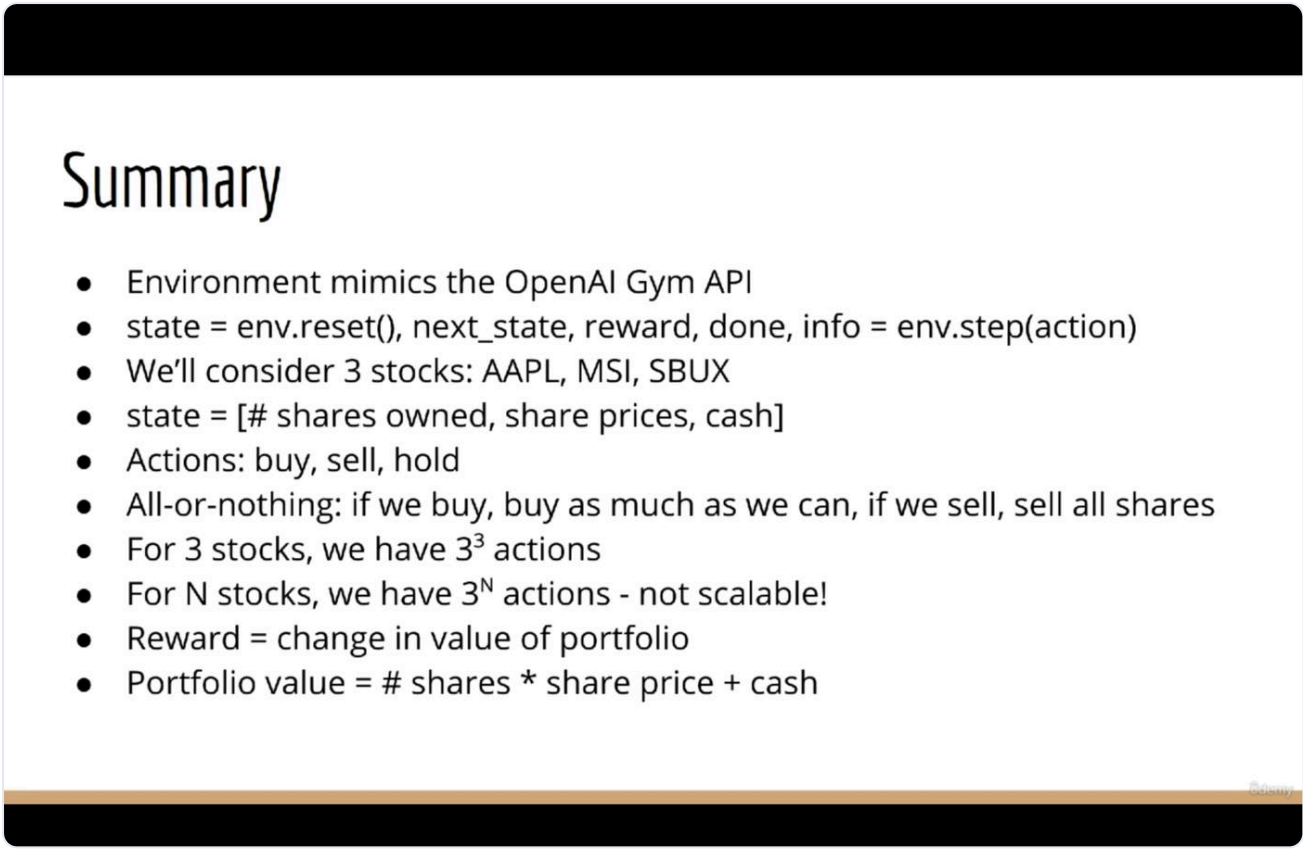

FIGURE 10.11 | Environment Mimics the OpenAI Gym API

If we have many stocks to consider finally the reward is just the change in value of our portfolio from the previous steps to the current step. The value of our portfolio is just the price of each stock we own. Times the number of shares we own plus any uninvested cash we have.

LECTURE 104

Using the Buffer (During Training)

In this lecture we are going to discuss how to implement a replay memory or replay buffer efficiently to understand why we would want to do this. Let's first consider how we might implement a replay buffer using a naive programming approach.

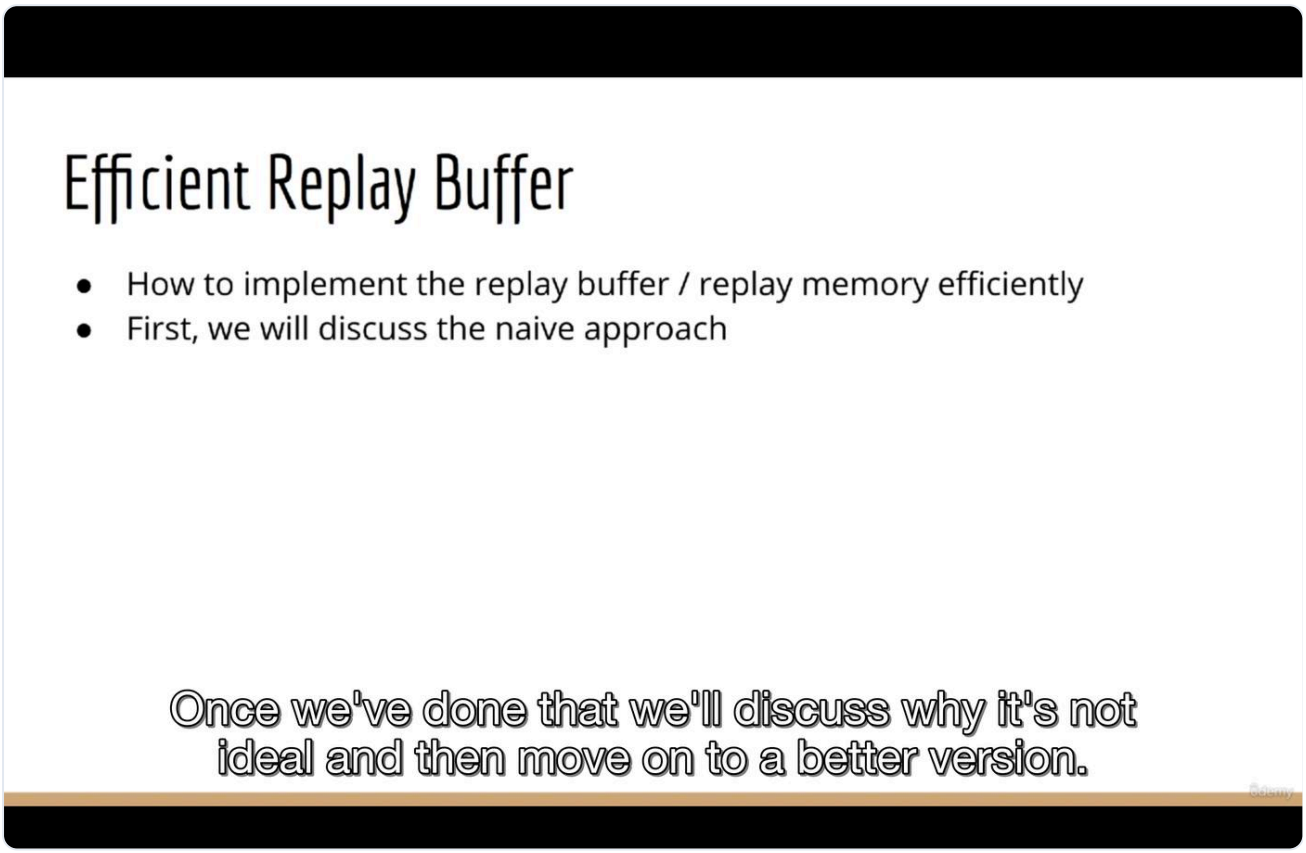

FIGURE 11.1

Once we've done that we'll discuss why it's not ideal and then move on to a better version. In essence a replay buffer is simply a python list. Inside this list we are going to store tuples with five elements each s a r s prime and the done flag. So as our agent takes a step and encounters a new tuple it will just store this tuple in our list. The buffer of course has a maximum size. So once we've hit the size limit we're going to throw out the oldest value.

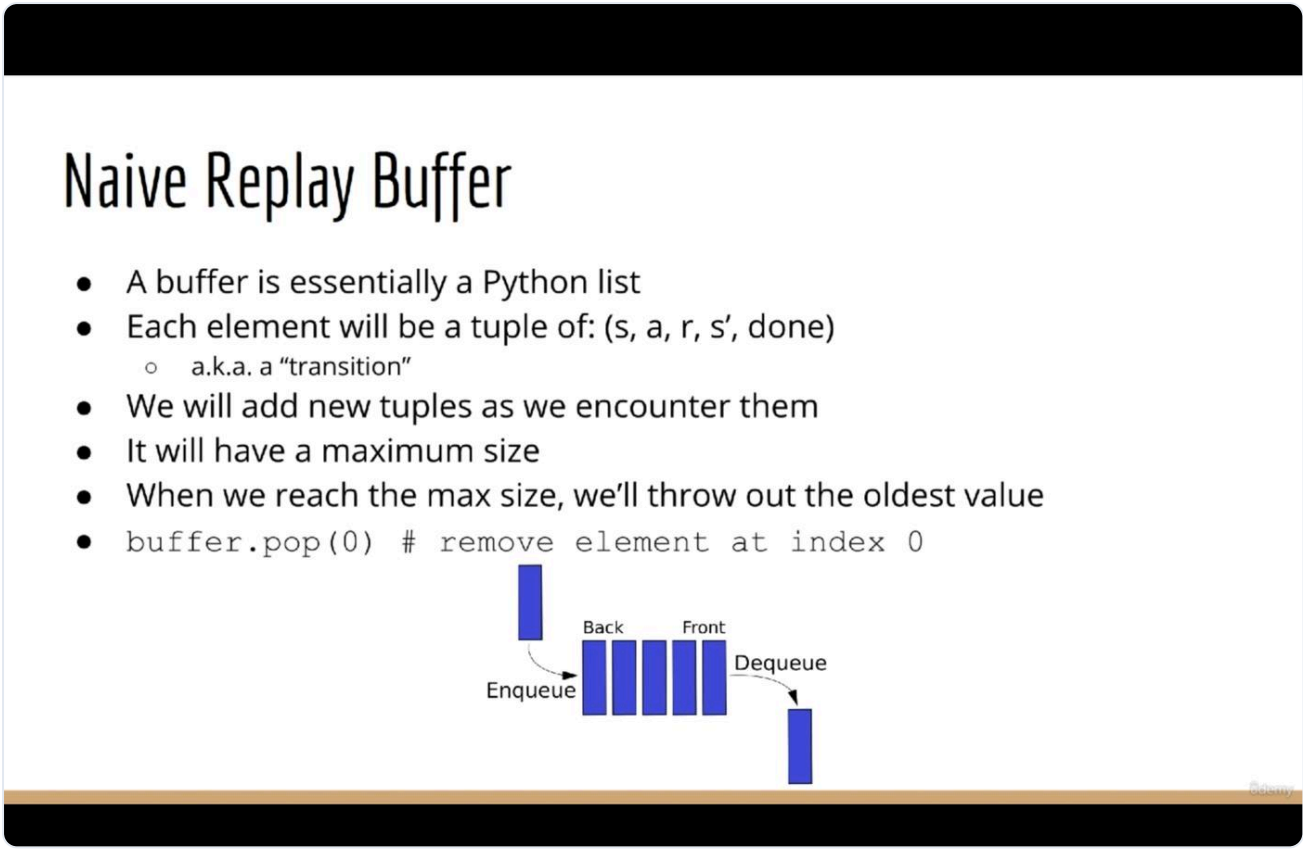

FIGURE 11.2 | Naive Replay Buffer

How do we do that. Well we can use the python function pop and passing the argument 0. This means remove the element at index 0 next. Let's consider how we will use the buffer during training. What we would like to do is grab a batch of random samples from our buffer. From this we can calculate input target pairs and then do one step of gradient descent. Remember that the target will be calculated as our plus gamma times the max over a prime of Q of s prime a prime and this only applies if as prime is not a terminal state.

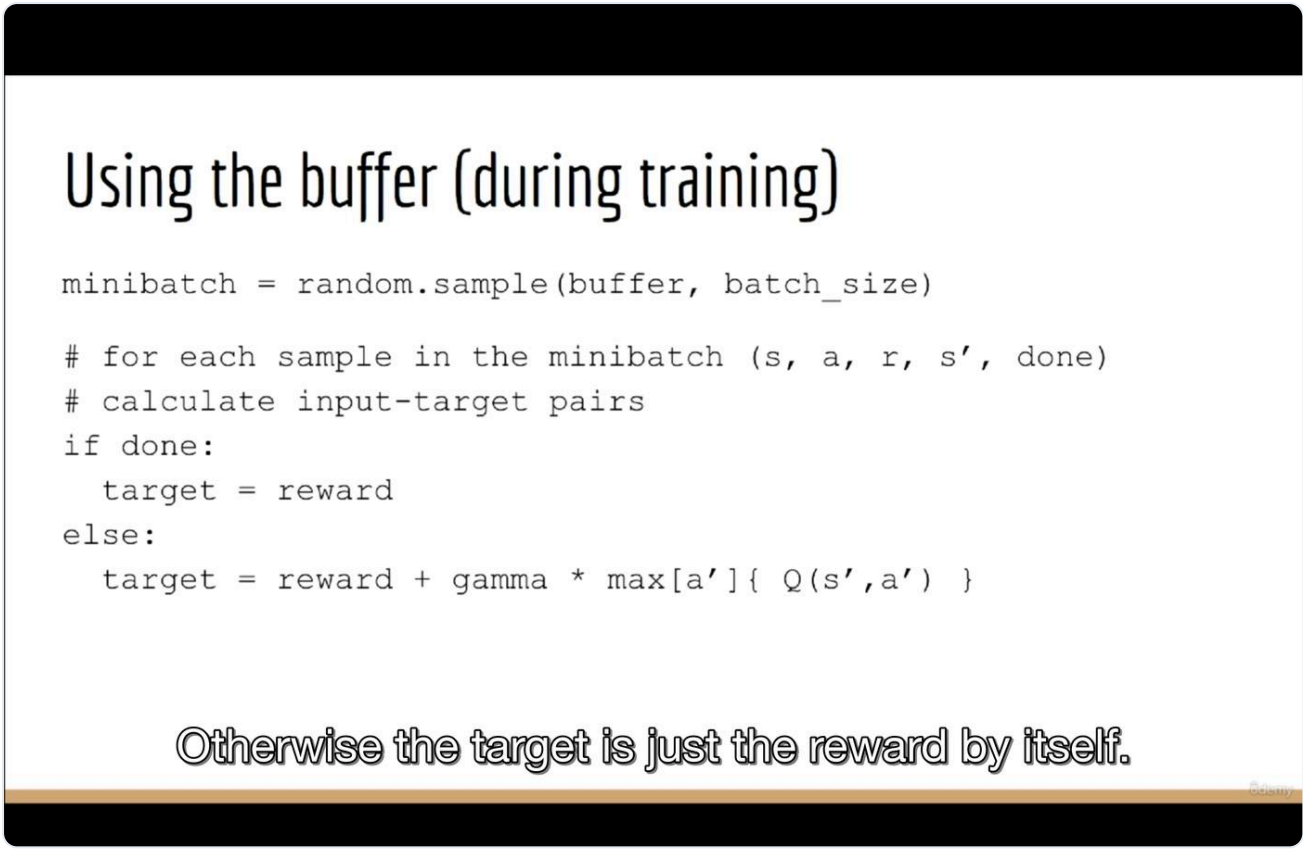

FIGURE 11.3 | Using the Buffer (During Training)

Otherwise the target is just the reward by itself. In other words we're going to do some kind of calculation over the batch of data we just sampled.

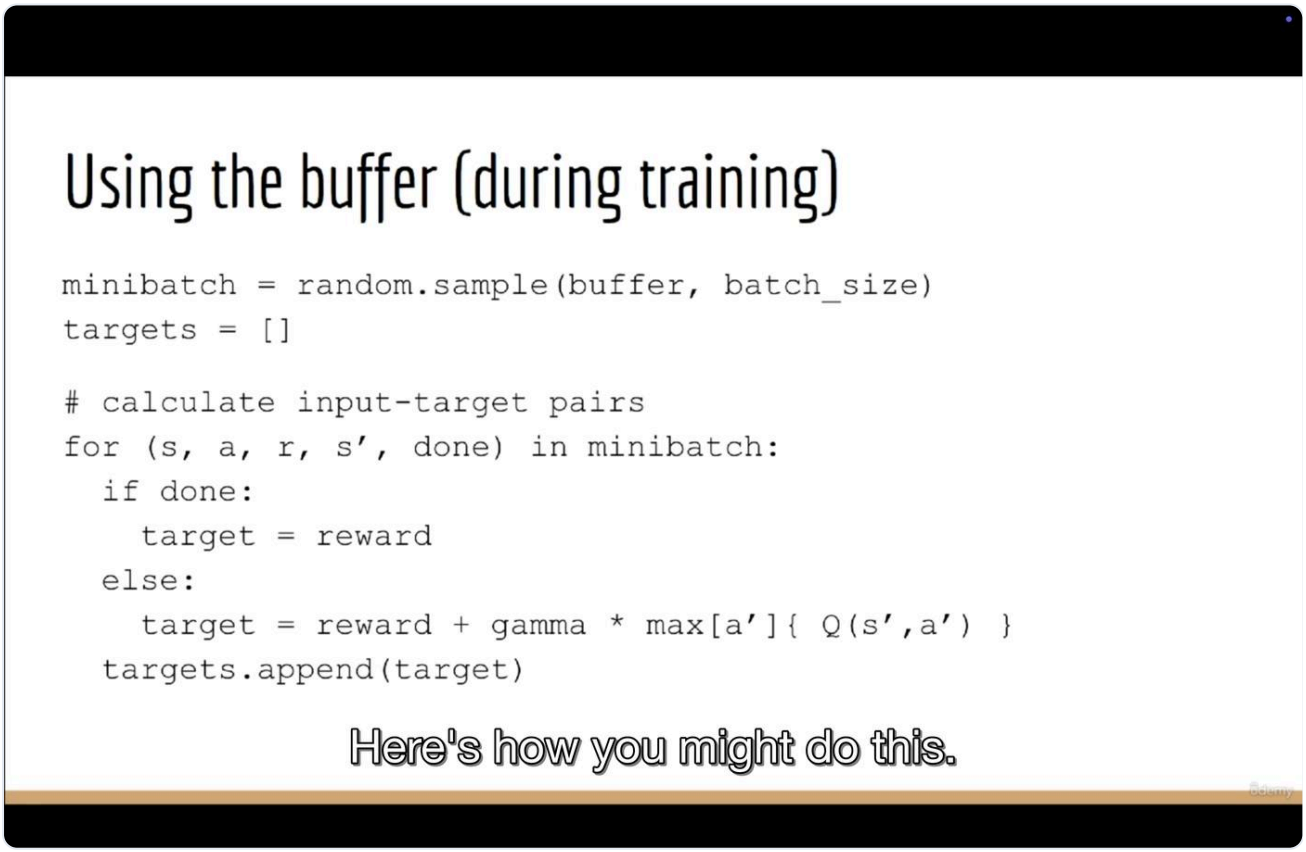

FIGURE 11.4 | Using the Buffer (During Training)

Here's how you might do this. Start with an empty array of targets. Then loop through each of the tuples you sampled for each iteration check whether the done flag is true or not. If it's true then the target is just R. Otherwise it's just the formula we saw previously. Of course as you may know we don't like doing for loops in Python. If we can avoid for loops we'll be slower than doing batch operations in NumPy. It's true that you could gather the batches together and cast them into a NumPy array but that means you still had to store a bunch of lists not ideal here's another problem with continually appending a new tuples and popping old ones. It may seem like this will take up only a constant amount of memory since the memory storing the old removed values should be freed by the garbage collector.

Unfortunately python will not do this and you will end up with a memory leak and a very slow computer.

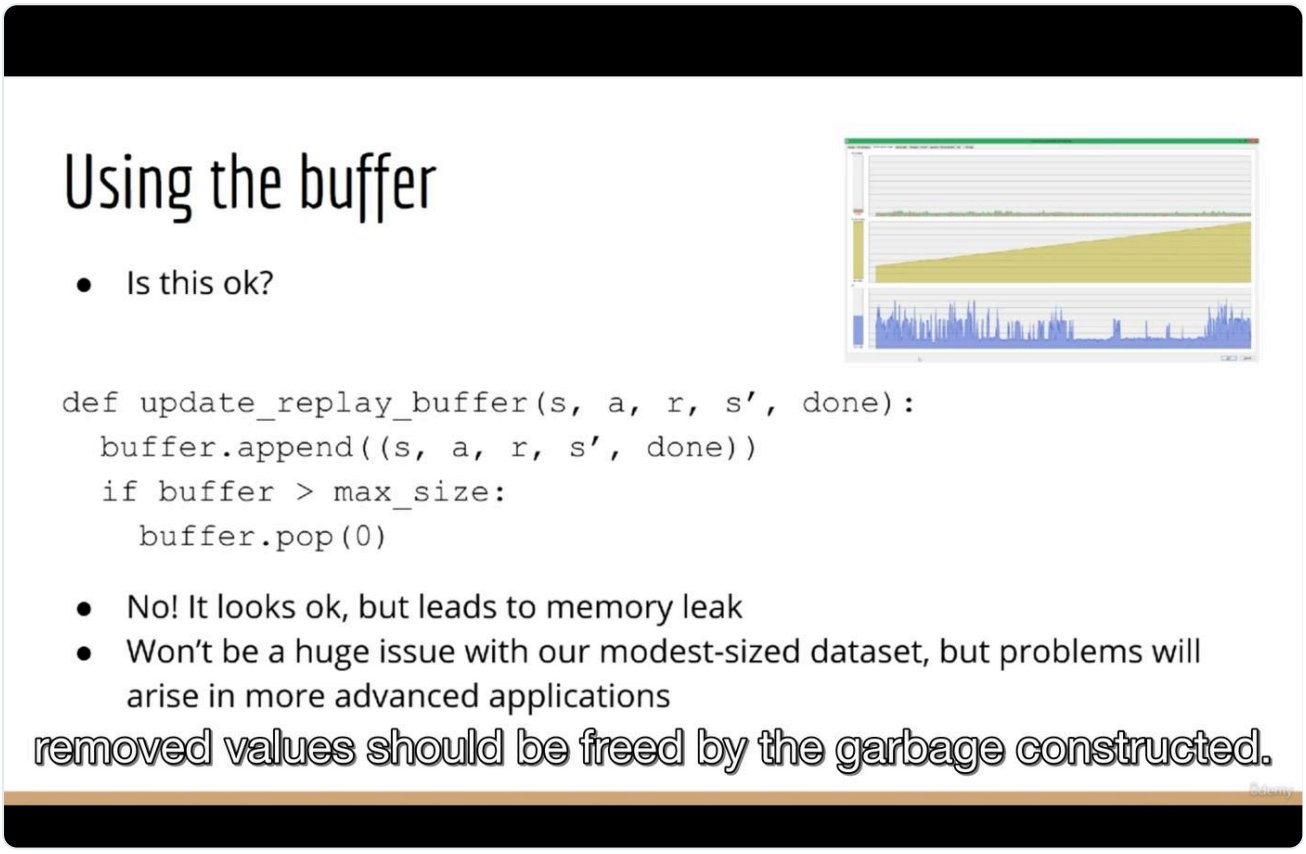

FIGURE 11.5 | Using the Buffer

This probably won't be an issue with a modestly sized dataset like the one we have but it's good to be aware of the kinds of problems that may arise when you want to graduate to more advanced applications.

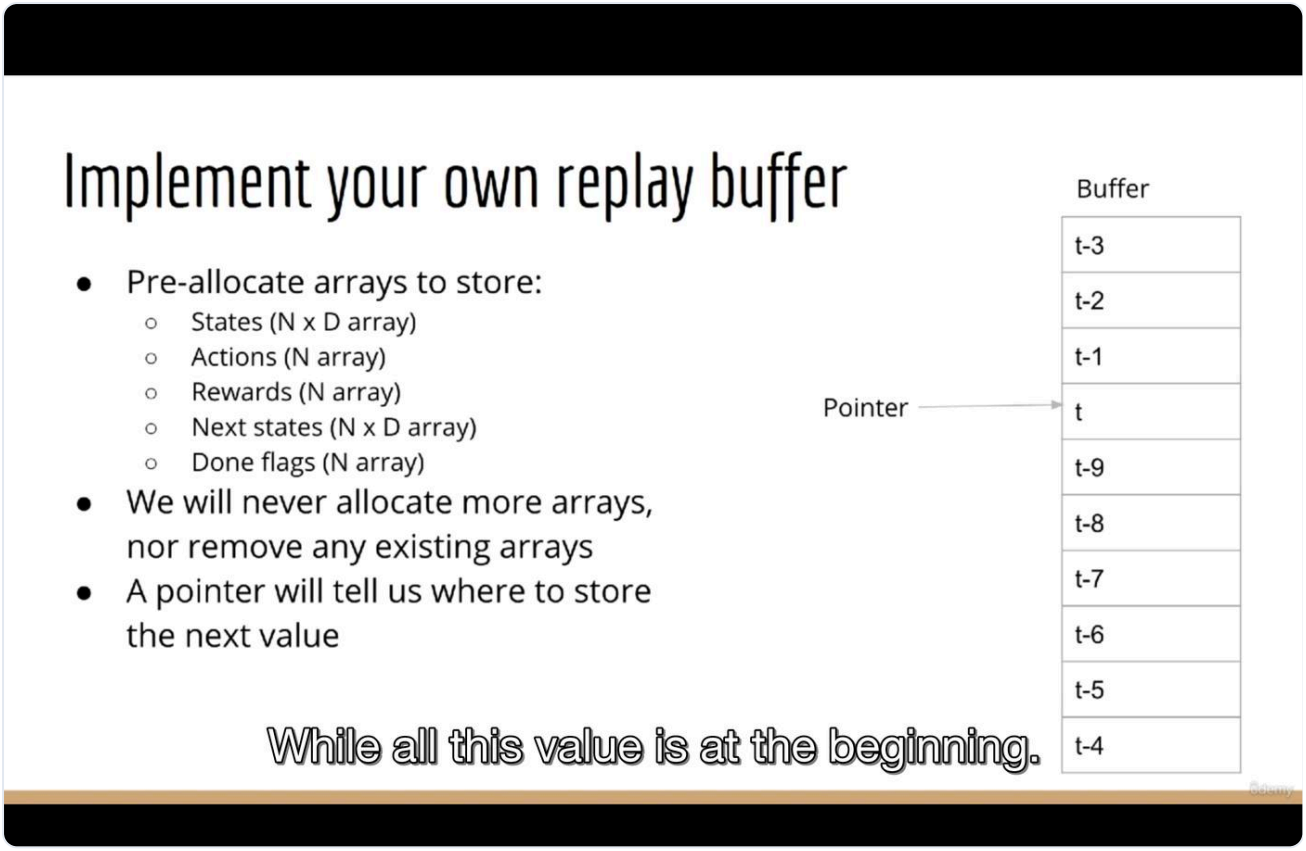

FIGURE 11.6 | Implement Your Own Replay Buffer Bute

So what's the solution the solution is to implement your own replay buffer. Here's a high level view of how it'll work. First we're going to pre allocate the arrays we will use to store the states actions rewards and so on. So this will really take up a constant amount of RAM. We will never allocate new arrays nor try to remove any arrays. All we will do is populate our pre allocated arrays. Second we'll use a pointer which will tell us the current insertion location of our transition tuples. So every time we encounter a new tuple we'll store it in the next available location in the array. Now what happens when the array is full. Well the oldest value is at the beginning.

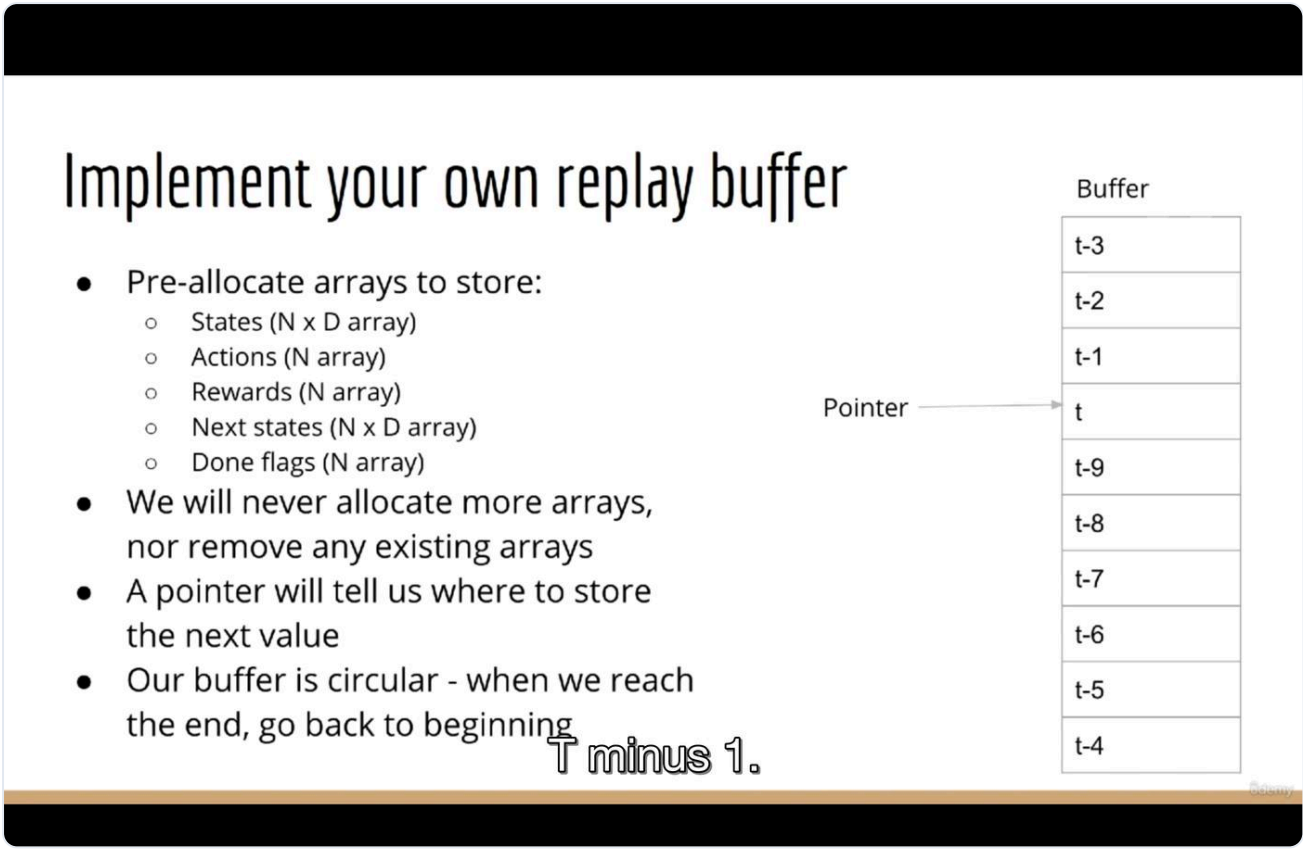

FIGURE 11.7 | Implement Your Own Replay Buffer Outer

So that means we can simply overwrite it in this way our buffer is circular. When we get to the end we just go back to the beginning. In this example you see here we can see that the pointer shows us the value at the current time t. But before that we have the value at the previous time. T minus 1. This goes back to T minus 2 T minus 3. But whereas T minus 4. In fact it's at the end of the array because when we store the value for T minus 4 we were at the end. So the next value T minus 3 would have been stored at the beginning since we had to circle back to replace the oldest value on the next step. We would replace the current oldest value which is at T minus nine one small detail.

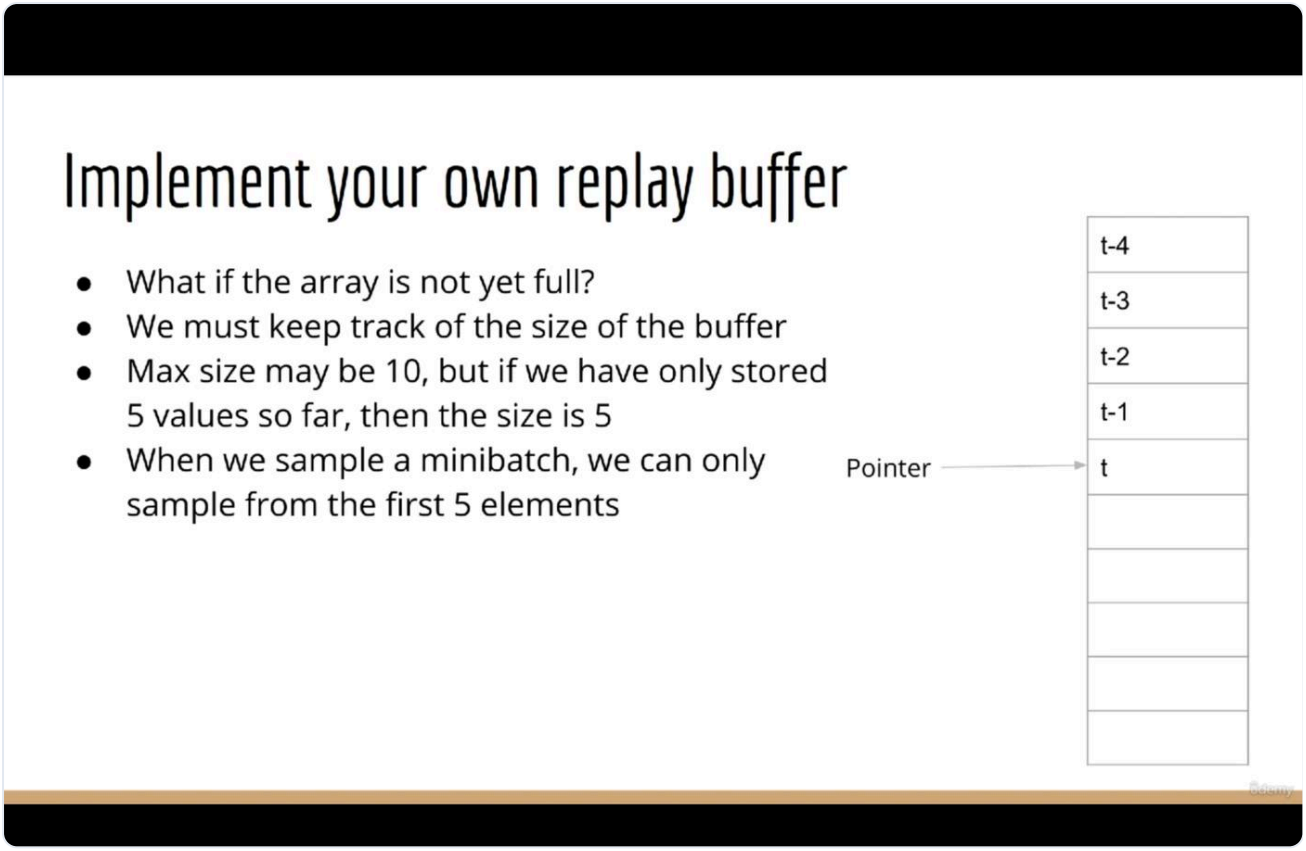

FIGURE 11.8 | Implement Your Own Replay Buffer

We have to think about is what happens when the array is not yet full. In this case we will still need to keep track of the current size of our buffer. For example the pre allocated array may be of size 10 but we will only have store 5 values than the effective size of the buffer will be 5.

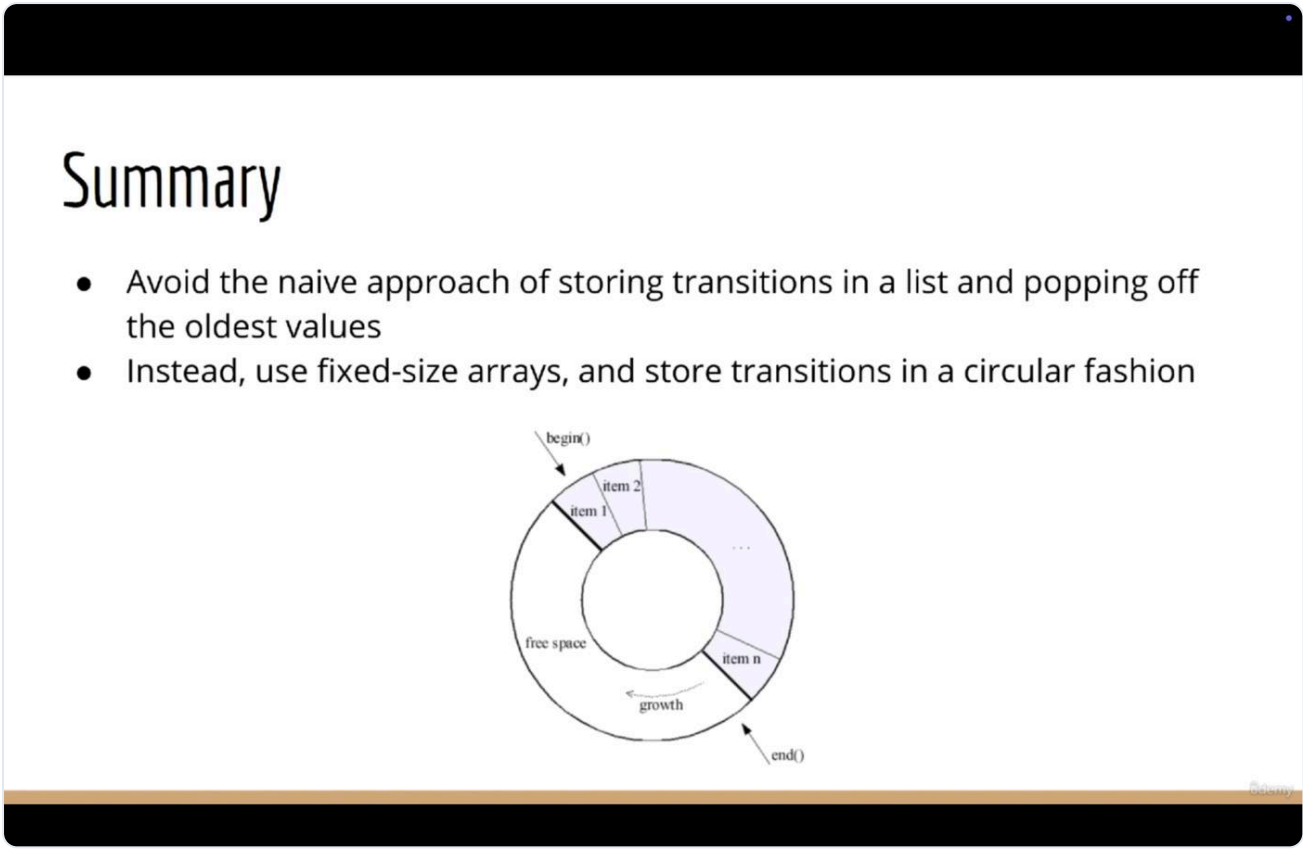

FIGURE 11.9

When we sample a batch from this buffer we can only sample values from zero up to 5 so that's it for this lecture instead of using the naive approach of storing our transitions in a list then sampling from the list to get yet another list. We will instead use a fixed size array and store transitions in this fixed size array in a circular fashion.

LECTURE 105

Using the Environment

In this lecture we are going to discuss the layout and design of our reinforcement learning trading bot. First at a very high level we are going to have two modes of operation. Train and test. As usual we want all of our training data to be in the past and all of our test data to be stock prices that came after the training data.

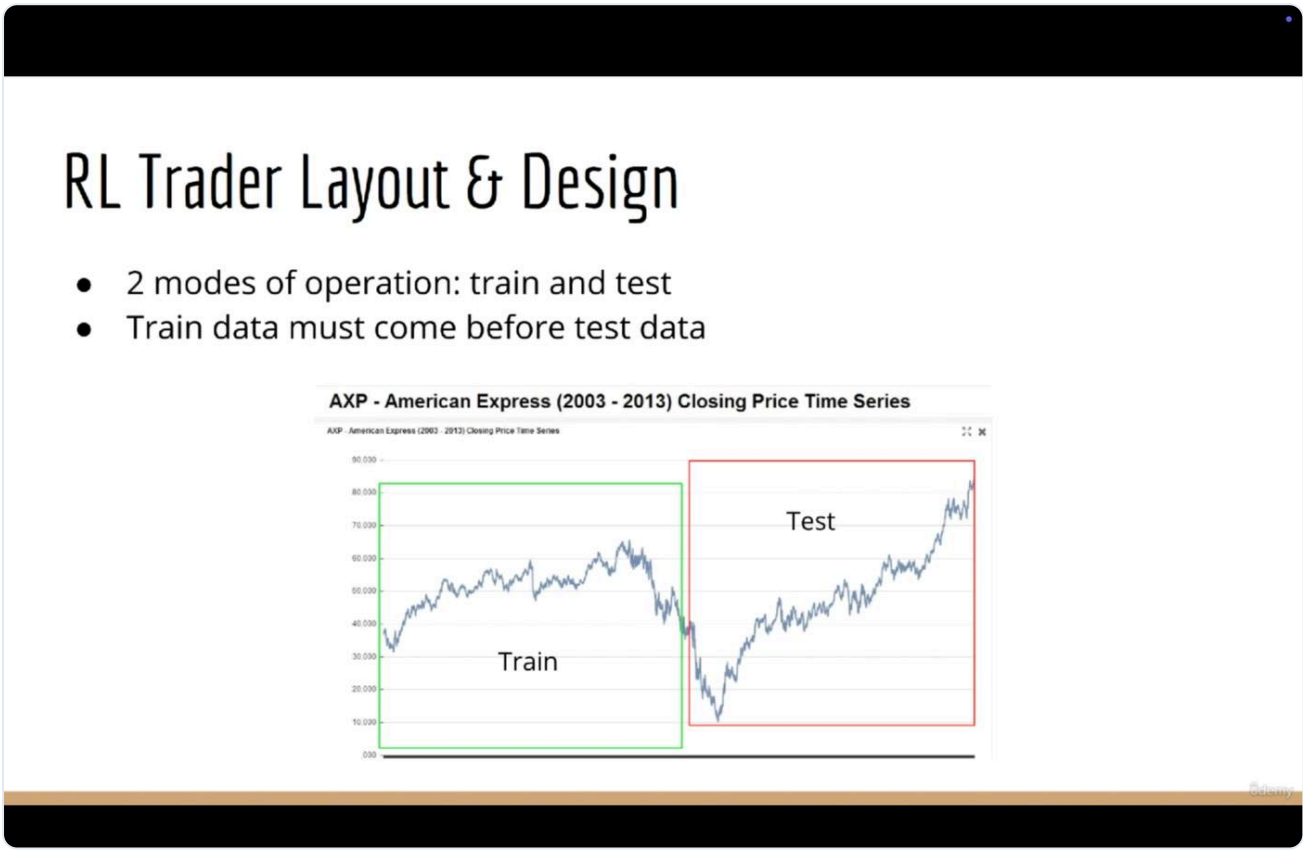

FIGURE 12.1 | RL Trader Layout & Design

So we are going to train our agent to maximize its reward over an episode using only the training data. Then we are going to use this trained agent on the test data to see what the value of our portfolio is. By the end of the test period to start let's think about how this would work if we had access to all the objects we needed.

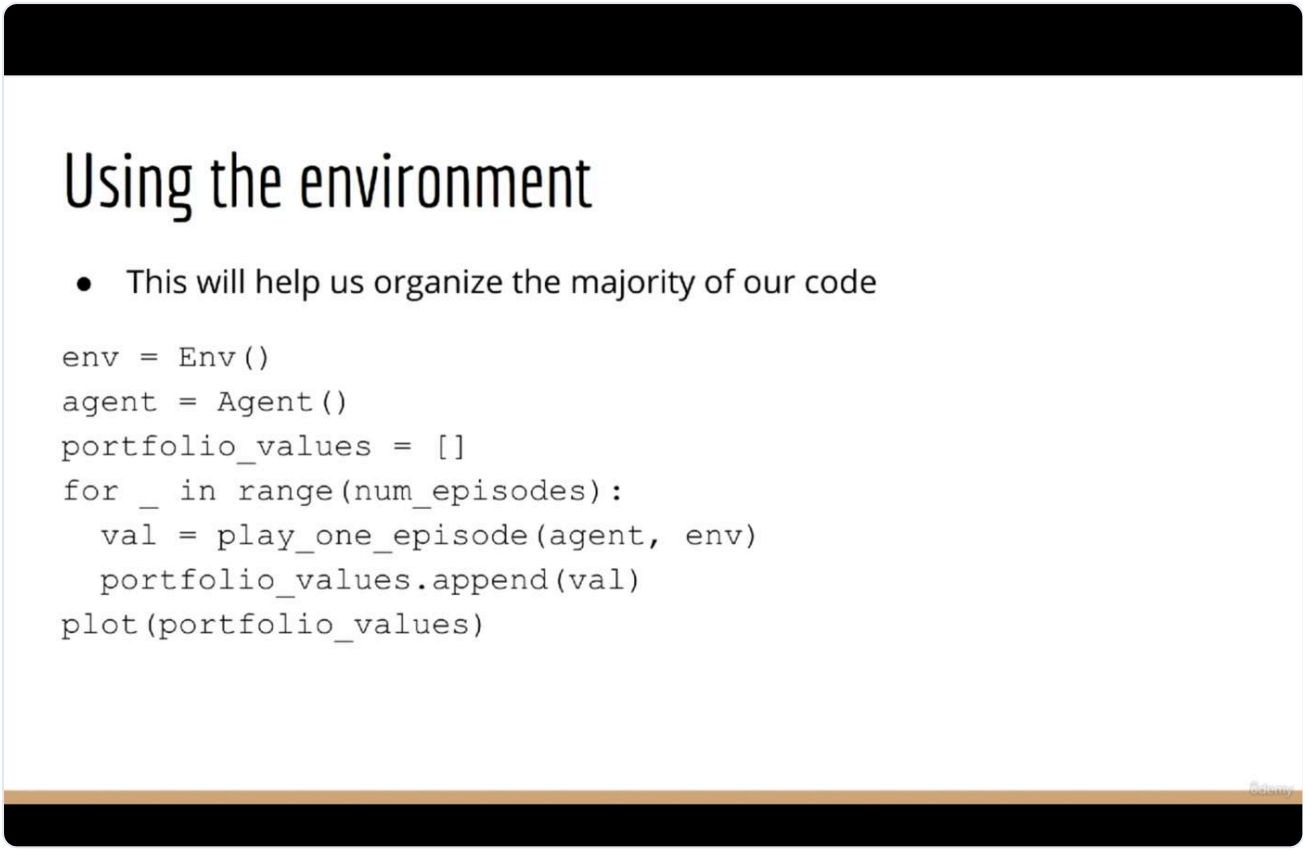

FIGURE 12.2 | Using the Environment

Even without the agent this will help us organize the majority of our code. The main part of the code will look something like this. First create an instance of the environment next create an instance of the agent. Don't worry about what this does yet then in a loop we're going to have a function called play one episode which accepts the environment and the agent and returns the value of the portfolio at the end of the episode. When our loop is done we're going to save the portfolio values for later so that we can plot them and analyze them. So this is pretty simple but now we have to figure out what should go in the function play one episode. So here's what the play 1 episode function might look like.

play_one_episode

```
def play_one_episode(agent, env):
  s = env.reset()
  done = False
  while not done:
    a = agent.get_action(s)
    s', r, done, info = env.step(a)
    if train_mode:
      agent.update_replay_memory(s, a, r, s', done)
      agent.replay() # sample batch and do gradient descent
    s = s'
  return info['portfolio_val']
```

FIGURE 12.3

As always we start by resetting the environment to get back to the initial state. Next we initialize our done flag to false and enter a loop that only quits when done becomes true inside the loop. We choose an action. Now at this point you know that this action is coming from our agent but will defer how the agent works until later next. We're going to call the env.step function to perform the action and get back the next state reward. And so on next we're going to check if our script is in train mode if it is then we have to train our agent. We want to update our replay buffer by adding the most recent SARS prime and done tuple to the replay buffer then we'll call Agent replay to grab a sample from our replay buffer and run one step of gradient descent finally we'll set the current state to be the next state for the next iteration of this loop when we're done we'll return the value of our portfolio one additional detail to keep in mind is that our data is not yet normalized.

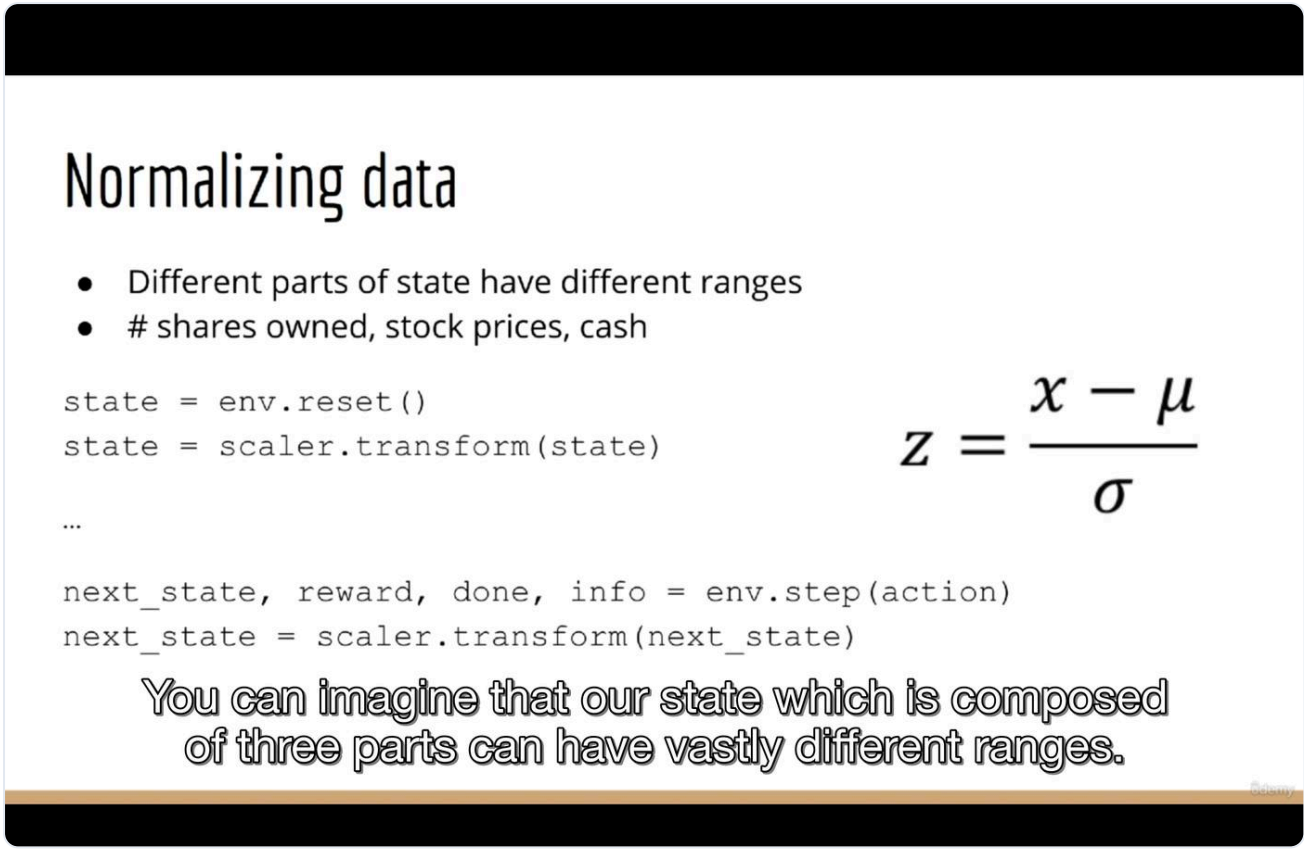

FIGURE 12.4

You can imagine that our state which is composed of three parts can have vastly different ranges. The first part consists of the number of shares we own. The second part consists of share prices in the third part consists of how much cash we have sitting uninvested. So we'll want to normalize this data. We can do this very simply whenever we get a new state. We'll have a scalar object from scikit-learn which will take our state and standardize it to have zero mean and unit variance. So not a huge addition to our previous code next. Let's imagine what our environment objects will actually look like. First it's going to accept a time series of stock prices as input into its constructor we'll also have a pointer to tell us what day it is.

So we know the current stock prices while also want to know how much cash we initially start with our initial investment from this we can do everything we need to do our reset function will bring our pointer back to the beginning of the time series and recalculate our state which of course should be all cash and no investment our step function will take taken an action and then buy and sell the stocks specified by the action then it'll set our pointer to the next day's stock prices. We'll also calculate the next state and the portfolio value. From this we can calculate the reward. The done flag will simply be set to true. If we reach the end of our time series so that's basically it for the environment we have the Constructor a reset function and a step function Finally let's consider our agent object which is complicated but no more complicated than the environment the essential parts of the agent are.

The replay buffer which is what we use to store our transitions in the environment and a neural network or some other kind of model which is what we will use to approximate our Q values. So what are the essential functions of the agent as we've seen. We'll need a one function called update replay memory which takes in a state action reward next state and done flag and stores this

in our replay buffer object. Next we'll need a get action function which accepts as input a state and decides what action to perform in the environment. And because this is Q learning it's going to use the Q learning rule or some variant of it like Epsilon greedy finally we'll need a replay function which does the following.

First it grabs a random sample from the replay buffer next. It uses this to calculate a supervised learning dataset which consists of input and target pairs which is what we need to train our model once we have our data set. We can call model that train to run one iteration of gradient descent. All right so that's it for this lecture. There are certainly more details we haven't yet discussed since this was designed to just give you an overview at this point. You understand that our script will have a train mode and a test mode. In both cases there will be a main loop where we call play one episode again and again playing one episode involves basically just going back and forth between the agent and the environment.

The environment produces states and rewards the agent takes in states and returns actions to perform in the environment during train mode. The agent will store the states actions and rewards and perform. Q learning updates in order to train the Q function approximator.

LECTURE 106

Max Size and Set of States

Check the notebook `24.rl_trader.ipynb` for the code.

In this lecture we are going to begin discussing the code for our reinforcement learning. Stock trader you will notice that we are not using Google Colab for this which I found performs very slowly compared to running it locally. For this reason I'm running the scripts locally. But as usual if you prefer to run this code in Google Colab all you would need to do is paste the code and modify the data source the relevant file for this lecture is rl_trader.py which can be found in the course repository in this lecture. We're going to describe just a few initial bits and pieces of our program. The first function we have is get data. Luckily I've already included the data in the course repository.

So all we have to do is import it. The data is stored in a CSV which we can load in a pandas data frame. But of course what we are interested in are the values so we can call dot values to get a NumPy array. Note that in order to simplify this problem we're going to use the close price only. You're welcome to look at the CSV for yourself if you want to know what's in it. As usual this returns a T by D sequence where t is the number of times steps and d the feature dimensionality and in this case that's the number of stocks three here are the three stocks we'll be looking at in case you're interested. First we have Apple that's AAPL next we have Motorola that's MSI and finally we have Starbucks that's SBUX note that each row corresponds to the same date.

So for each stock we have data from about February 2013 to February 2018. Next we have the replay buffer. This has three functions. The constructor store and sample batch in the constructor. We initialize our array buffers and pointers. First we have obs1 buf and obs2 buf which store the states and next states respectively. Next we have acts buf which stores our actions. These are represented by integers from zero up to 26 inclusive so we can use uint8 to represent them. Next we have the rewards buffer which stores our rewards and we have the done buffer which stores the done flag. This can only be 0 or 1 So again it can be a uint8. Lastly we have a pointer which starts at zero the current size of the buffer is zero and the max size of the buffer is specified as the size argument.

Next we have the store function. This store is the state action reward. Next state and done flag in their respective buffers at the index self dot pointer. After this we increment the pointer so that the next time we call the store function all the values will be stored in the next position. And remember this is circular. So we make use of the modulo operation when incrementing the pointer would go up to max size it's set back down to zero. Lastly we set the current size of our memory which is equal to the minimum of the previous size plus one. Since we just added one more value or max size. In the case the buffer is full finally we have the sample batch function.

This chooses random indices from 0 up to the size of the buffer. Then we return a dictionary containing the state's actions rewards and so forth indexed by those indices the next thing we're going to look at is the get scalar function this takes in an environment object since that will be used

in fitting our scalar. The idea is in order to get the right parameters for a scalar. We must have some data in order to get this data. It's easiest to just play an episode randomly and store each of the states we encounter. There is no need to have an agent because such an agent wouldn't be trained anyway. So you can see that when we choose an action to perform the only thing we need to do is sample a value from the action space.

When we're done we create a standard scalar object and fit it to the states we encountered. Now one thing you could do to make this more accurate is run it for multiple episodes. Next we have a function called maybe make Dir. This is more just a utility function. It checks if a particular directory exists and if it doesn't then it creates the directory. The reason we need this is because we're going to store our trained model and the rewards we encounter to separate files so that we can plot them afterwards. Next we have the MLP class which is going to represent our Q function approximator. As you can see. This is just the standard ANN with an arbitrary number of hidden layers.

So as input to the constructor we take in the number of inputs the number of actions the number of hidden layers and the number of hidden units. Next we create a variable M initialized to be the number of inputs and a list to store our layers. Next we enter a loop so we can add each of our layers. So in hidden layers times I'm going to create a linear layer with size M by hidden dim I'm going to update m to be the size of the next layer as output. I'm going to append to the layer to my list of layers and then I'm going to add a ReLU non linearity outside the loop. I'm going to add one final layer with output size equal to N action and then I'm going to pass all these layers into a sequential object and then store that in the layers attribute.

Next we have the forward function which simply passes the data through our previously created sequential object. Next we have some special functions that are not normally part of a PyTorch model for saving and loading for saving. I just called torch.save pass in the state dict belonging to the current object and then the path to save the file to for loading. I call the load state dict function. Belonging to this class and then passing the output from torch dot load path next. We have a few convenience functions to interact with the rest of the script. I like the rest of the script to be library agnostic. What I mean by that is I don't want the rest of the scripts to have to know anything about PyTorch at all.

It should work no matter how I implemented my neural network whether that's with PyTorch TensorFlow JAX or any other library. So to that end we basically need two kinds of functions that scikit-learn would normally provide a training function and a prediction function in the only interface. These should be NumPy arrays or other basic Python types. There should be no PyTorch specific inputs or outputs. So for the predict function we take in a model and a set of states. We can assume that the set of states has the typical N by D data shape inside the function we use torch.no_grad since this is just a prediction. First we convert the states to float 32 and then convert it into a torch tensor.

Then we pass that into the model and retrieve the output once we have the output we call dot numpy to return the prediction as a NumPy array next we have the train one step function. I call it this because we're going to do one step of our optimizer for every step we take in the environment. As input we take in the model the criterion the optimizer and a set of inputs and targets.

Now you might be like Wait a minute I thought you said you don't want any PyTorch specific inputs. So I lied a little bit. Well you'll see later that the agent object will have no PyTorch specific arguments. So that's kind of the true interface. Alternatively you could create the criterion and the optimizer as global variables otherwise we'd have to make this function part of the model including the criterion and the optimizer and so forth which would be very unconventional PyTorch.

So we don't want to do that. So anyway the first thing we do is convert the inputs and targets into float32 and turn them into torch tensors. Next we zero the gradients. Next we pass the inputs through the model to get the outputs. And next we calculate the loss. And as usual we call loss.backward and optimizer.step. And this does one step of gradient descent.

LECTURE 107

Stock

Check the notebook `24.rl_trader.ipynb` for the code.

In this lecture we are going to define the environment class we're going to call it multi stock env at the top here. We just have some comments describing both the state and action since these are non-trivial. So as you recall the state consists of the number of shares of each stock that we own. The price of each stock and the cash that we own uninvested. The actions can be different combinations of sell hold or buy for the three different stocks. Next we have the Constructor it takes in two arguments a two dimensional data array containing a time series of stock prices and also the initial investment amount with a default value of twenty thousand dollars.

Next we set the attribute stock price history to the data we passed in. Next we set the attributes n step and n stock which we obtained from the shape of the data array. Next we set the attribute initial investment to the initial investment we passed in. Next we initialize a few attributes to none. Although we'll be setting these shortly next we set the attribute action space which is equal to the integers 0 up to 3 to the power n stock next for convenience will want to have a corresponding action list. Since we have twenty seven possible actions this will be a list of length twenty seven. So the action space only tells us the integers from zero up to twenty six whereas for this thing it actually gives us some examples of what we can have.

So for example 0 0 0 means sell all your stock 0 0 1 means sell the first 2 stocks but do nothing for the third stock. Now this code may look complicated but you can try it in your terminal or in a notebook to confirm that this is the result. So you're gonna get an array consisting of values just like these. Next we set the state dim which will tell us the size of the state vector. As mentioned earlier this is the number of stocks. Times two plus one. Finally we call the reset function which initialize is a few more attributes and returns the initial state so let's look at the reset function. Here we set the attribute current step equal to zero. This means that we point to the first day of stock prices in our dataset.

Next we set the attribute stock owned to an array of all zeros. This array will tell us how many shares of each stock we own. So when we start we're not going to own any stock. Next we have the stock price which tells us the current price of each stock on the current day. This is of course just the time series index by the current step. Next we have the cash in hand attribute which to begin with is our initial investment. Since we've not yet bought any stocks. Finally we want to return the state vector which is done by using the function get obs. We'll see what's inside this function shortly next we have the step function which performs an action in the environment and returns the next state and reward among other things.

First we check that the action that we passed in exists in the action space. Next we get the current value of our portfolio and assign this to the variable prev val. Next we increment the current step pointer and also the current stock prices attribute so the day goes up the next day and the price goes up to the price for the next day.

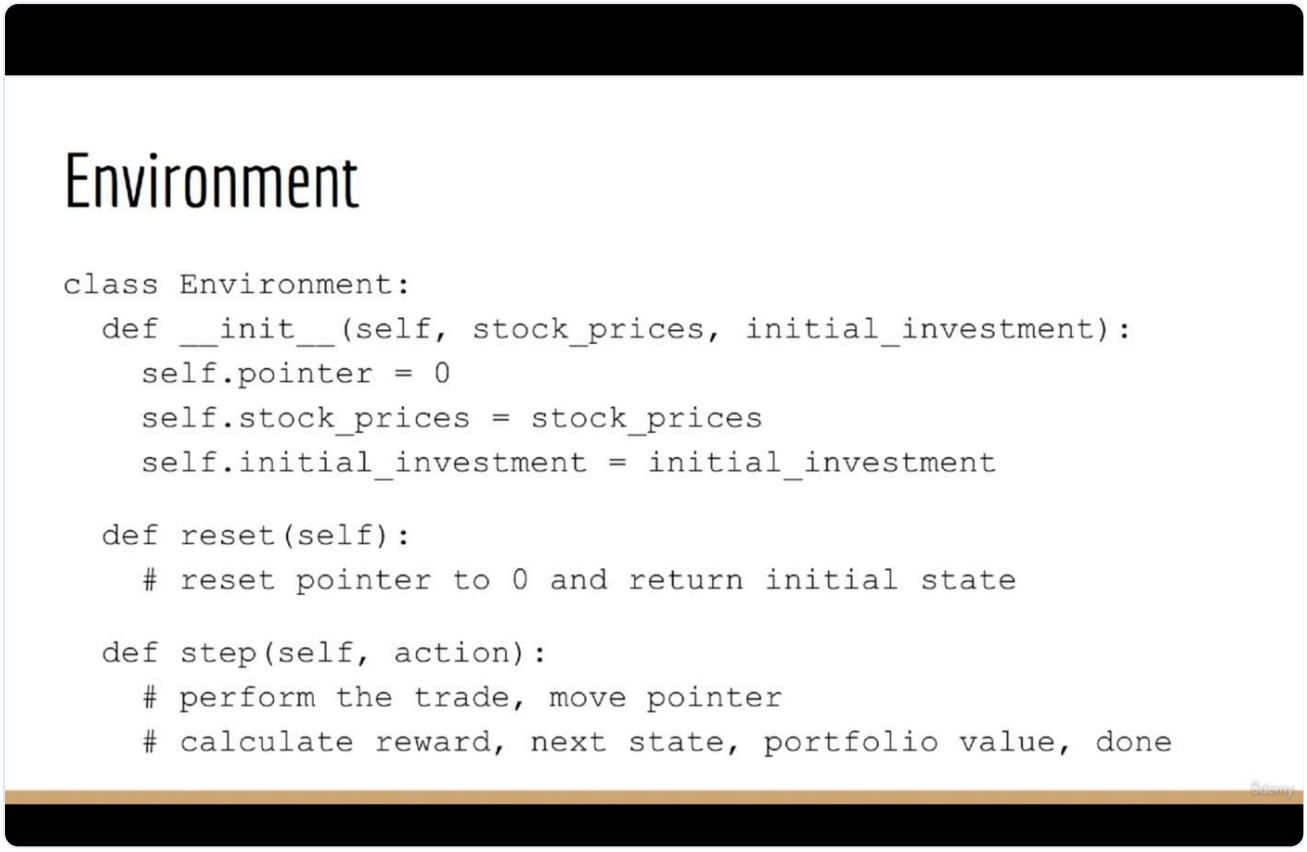

FIGURE 14.1

Next we perform the trade. This is another function we'll be discussing later. But you can assume it involves some combination of buying selling or holding of the stocks we are considering. Next we get the value again and assign this to the cur val variable. Next we can calculate the reward which is just the difference between the current value of our portfolio and the value of our portfolio. Before making the trade next we set the done flag which is true only if we've reached the end of the time series next we set the info dictionary which allows us to return at the current value of our portfolio since this is part of neither the state nor the reward.

Lastly we returned the next state the reward done flag an info dictionary just like the OpenAI Gym API so that's it for the public functions of this class. Next let's look at the functions we made use of earlier. First we have get OBS This returns the state by the way sometimes like right now we are going to use the terms state and observation interchangeably. There are cases where we might not want to do this such as when the state is some transformation of the observation or it could even be multiple past observations stacked up together. For this script however we'll assume that they are one in the same as mentioned earlier. This is going to be a vector with three components.

The first component is the number of shares of each stock that we own. So that's stock owned. This should be a list of size three. The second component is the value of the stock. This should also be a list of size three. Finally we add the cash in hand attribute to our observation at the last index and then we return our observation. Next we have the get Val function. This returns the current value of our portfolio. This is equal to the number of shares we own for each stock multi-

plied by the value of each stock. Added to the cash we have lastly we have the trade function which is probably the most complex function in this class to start. We need to grab the action vector from our action list which is index by the action argument which is just an integer.

So this is going to return us vector of size 3 which will tell us for each of the three stocks whether to sell buy or hold. So let's just do a simple example to explain this more clearly. First that the action will be an integer from 0 up to twenty six and we're going to use that to tell us which actions we actually need to perform for each stock. So for example if we retrieve the vector 2 1 0. That means buy the first stock hold the second stock and sell the third stock. Next since we want to sell everything we want to sell before we buy anything we have to figure out what needs to be bought and what needs to be sold. So we are going to create these two lists sell index and buy index.

These will contain the indices for the stocks that we want to buy or sell to populate these lists. We're going to loop through action vec then we check if value is equal to zero. If it's zero. That means we want to sell this stock and if it's 2 That means we want to buy this stock. Next we're going to sell all the stocks we want to sell. As mentioned this problem is greatly simplified because it's all or nothing for each stock. We want to sell. We're going to sell every share we own. So we loop through each index in the sell index list. Then we increment our cash in hand to be the total value of the shares that we previously owned so that would be stock price times the number of stocks sold and then we set the number of shares we own to zero.

Next we have our buy loop. Again this is extremely simplified. We're just going to loop through each stock one at a time and buy one share of that stock until we run out of cash. This is of course a great simplification since let's say we have nine dollars but the share we want to buy next is ten dollars. We can't buy it but maybe the next stock is only five dollars so we could have afforded one share but we would never see this. Since we set the can buy flag to false and exit the loop. Of course this only happens on the boundary of the stock. So you're looking at stock 3 and going back to stock 1. In any case inside the loop first we check if we have enough cash to afford a one share of the current stock so cash in hand must be greater than the stock price.

Then we increment the stock owned by one and subtract the value of the stock from the cash in hand.

LECTURE 108

Batch Size

Check the notebook `24.rl_trader.ipynb` for the code.

In this lecture we are going to describe the agent the agent is our artificial intelligence. It's responsible for taking past experiences learning from them and taking actions such that they will maximize future rewards. First we have our constructor it takes into two arguments the size of a state vector and the number of actions. These correspond to the number of inputs and outputs of our neural network respectively. Next we initialize an instance of the replay buffer object with the memory size of 500. Next we have a few hyperparameters such as gamma the discount rate the initial value of epsilon the final value of epsilon and the factor to change Epsilon by on each round.

Lastly we create an instance of our model by calling the MLP constructor. We also create the associated loss and optimizer and make these attributes of the agent. Next we have the update. Replay memory function. This takes in a state action reward. Next state and done flag and stores it in the replay buffer. Next we have the act function. This takes in a state and uses Epsilon greedy to choose an action based on that state. First we generate a random number between 0 and 1 and check if it's less than epsilon. If it is we perform a random action by calling np.random.choice. Otherwise we perform a greedy action by grabbing all the Q values for the input state then the action to perform is the action which leads to the maximum Q value.

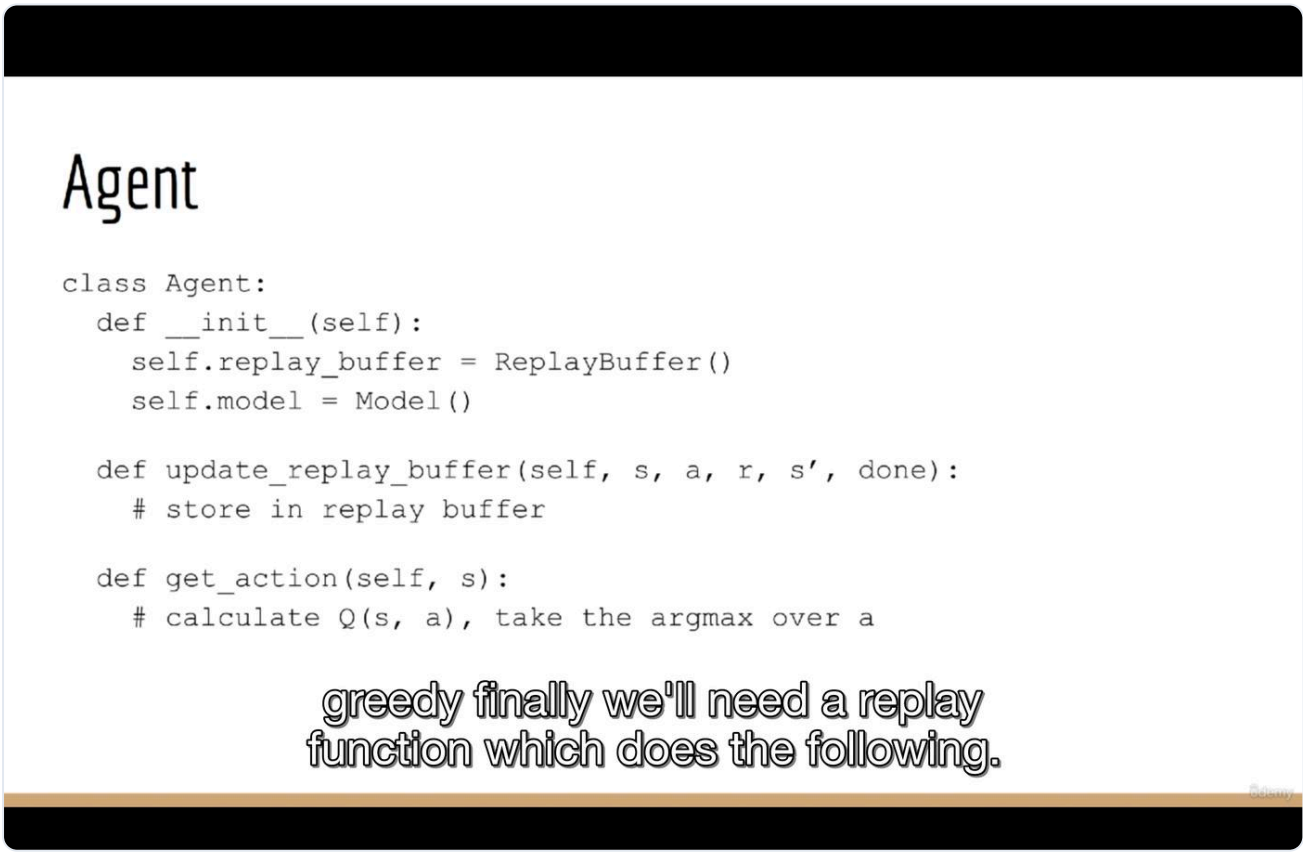

FIGURE 15.1

Agent

```
class Agent:
  def __init__(self):
    self.replay_buffer = ReplayBuffer()
    self.model = Model()

  def update_replay_buffer(self, s, a, r, s', done):
    # store in replay buffer

  def get_action(self, s):
    # calculate Q(s, a), take the argmax over a

  def replay(self):
    # sample from replay buffer, make input-target pairs
    # model.train_on_batch(inputs, targets)
```

FIGURE 15.2

Summary

- Train mode and test mode
- Play one episode
- Environment - takes in actions, produces states and rewards
- Agent - takes in state, produces action
- During training, store states / actions / rewards, update Q(s,a)

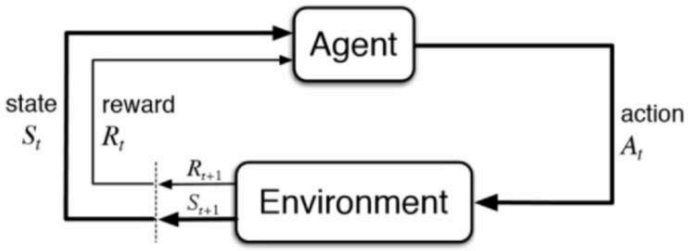

FIGURE 15.3

So we take the argmax over the model predictions remember that the output of a PyTorch model is batch size by number of outputs. So we have to index the return value by zero before taking the argmax. The batch size is just one. So we're looking at the 0th index. Next we have the replay function which is the most important function for this class. This is the function that does the learning. It accepts one argument the batch size which tells us how many samples to grab from the replay memory. So to start we first check if the size of the replay buffer is greater than the batch size. If it's not we can't grab a full batch. So we just return. Otherwise we continue.

Next we call self.memory.sample batch with the batch size argument. Remember this returns a dictionary so we grab the states actions rewards and so on using the corresponding keys next we calculate the targets for each state for reference. Here is the equation again. Mathematically it's a little different here because we are now working with a batch of data rather than just one sample. Also we have to remember that by definition the value of a terminal state is zero. Therefore if the next state is a terminal state then Y should just be the reward R. To that end we can multiply by one minus done since done equals one if it's a terminal state then you would get one minus one which is zero and anything times zero zero.

So effectively the target would just be R the reward now. Interestingly this may or may not be correct for the scenario. Realistically the stock market never ends and so you don't really have a proper terminal state. This is only where our data ends. So one might theorize that it may be better not to do this saying that this is the end of the episode is sort of pedantic. It's not really the end of the episode it's just the end of our data. The stock market will effectively go on forever. Of course you can just test it out and see if it has any effect now at this point even though we've calculated the target. We are technically not done currently. The targets are just a 1D array of length batch size but the model predictions are a 2D array of batch size by number of actions so they don't match.

The problem is the targets and the predictions don't have the same shape as always. I'll remind you how important it is to think about shapes. Now one option to solve this problem would be to just use a custom loss function but if you would like to use the default mean squared error then the targets and the predictions have to be the same shape. In other words for each sample we must have a target for each action. Even if that action was not the one taken by the agent in order to do this we're going to use our model to make a prediction for each state and action so we'll call that targets full. So this will be batch size by number of actions. Now the key is we only want to change this array for actions in which we really have targets.

So the targets we calculated above to do this. We can use double indexing where the first index which is np.arange of batch size. And this represents which row to choose the second index which is actions tells us which column to choose. And we can set these to the targets we calculated previously. The trick is for all other actions the target will be equal to the prediction because the target actually is the prediction therefore the error of those values will be 0 and they will not have any influence on the gradient descent step. Next we run one step of gradient descent using the function model train on batch passing in the inputs and targets we just calculated finally we update Epsilon to reduce the amount of exploration over time.

If Epsilon is still bigger than epsilon min then we decrease Epsilon by a factor of epsilon decay after this we have two simple functions to save and load the model weights. These will be useful since the script takes some time to train. So what we can do is train the script in one run save the weights and then test it later with different configurations.

LECTURE 109

Plot RI Rewards and Portfolio Value

Check the notebook `24.rl_trader.ipynb` for the code.

In this lecture, we are going to finish looking at the rest of this script and take a look at the results. First, we have the play one episode function, which is pretty much exactly the same as what we discussed in the theory outside the loop. We reset the environment, grab the initial state and transform it. Next, we initialize done to false. Next, we enter a loop which exits only when done becomes true. Next, we use the agent to determine the next action. Then we perform the action in the environment. We get back the next state. Reward done flag and info dictionary. Next, we scale the next state. Next, we check if we are in train mode, if we are, then we add the latest transition to our replay buffer and we call the replay function to run one step of gradient descent.

Next, we set the state variable to next state for the next iteration of the Loop. Lastly, when we exit the loop, we return the current value of the portfolio. Next, we have the main section first, there are a few configuration variables, so Models folder tells us where we will save our models. Rewards folder will tell us where we will store our rewards from both the training and testing phases. The number of episodes to run is 2000. The batch size for sampling from the replay memory is thirty two and our initial investment is twenty thousand. Next, we create an argument parser object, so that we can run the script with command line arguments. We'll have one argument, the mode where we can pass in a train or test.

Next, we create the model directory and rewards directory in the case where they do not yet exist. Next, we call the getData function to get our Time series. We call the shape attribute to get the number of time steps and the number of stocks. Next, we split the data into train and test, the first half is train and the second half is test. Next, we create an instance of our environment objects with the training data. After that, we get the dimensionality of the state and the dimensionality of the action space. We pass these into the agent constructor to get the agent. We also called get Scaler to get the scalar objects. Next, we initialize an empty list, which will store the portfolio values for each episode we play.

Next, we have an important if statement, this is going to overwrite some of the things we just created in the case where we are in test mode. So if we're in test mode, we want to use the scalar that we had during training so that the neural network corresponds with the same scalar and we don't accidentally use a different scalar. Next, we recreate the environment with the test data rather than the training data. Also, we have to make sure to set Epsilon to a small value. Otherwise its default value is one which is just pure exploration. Note that if you set it to zero, there is no point in running multiple episodes because both the data and our agent will be deterministic.

So you'll get the same result each time. Finally, we have to make sure we load up the weights that we saved during trainings, we call agent load and pass in those weights. Next, we enter a loop to play our episodes, num episodes times. Inside the loop. We grab the current time since we would like to know the duration of each loop iteration. Next, we play one episode and receive the port-

folio value at the end of the episode. Then we get the current time and subtract the start time to get the duration of the episode. Next, we print out all this information, the episode number, the portfolio value and the episode duration. Next, we append to the portfolio value to our list of portfolio values.

Next, we check if the mode is train, at this point, we are finished playing all of our episodes, so if the mode is train, then we'll save our neural network to a file called dqn.h5 will also save a scalar to scaler.pkl. Lastly, for both train and test will save the rewards we got so the train data will be saved in train.npy and the test data will be saved in test.npy. All right, so now we're going to look at another script, which is just for convenience, for plotting the rewards we saved, just so you don't have to write this yourself. This can be found in plot_rl_rewards.py. Basically, it loads up the rewards, which you can switch to train or test depending on what you want to plot.

And then it prints the average reward, the min and the max. It also plots a histogram of the reward. So we know the distribution of rewards over each episode. Next, I'm going to show you how to run this, so when you want to run this in train mode and plot the train results, you're going to run python rl_trader.py -m train and then you can do python plot_rl_rewards.py -m train. When you want to test, you just switch the mode to test. So you do python rl_trader.py -m test and python plot_rl_rewards.py -m test. So here are the results for training. The rewards are pretty good, we can even get up to a five X increase in our original investment, which is extremely high.

For testing, they are also pretty good, our original investment usually always increases by a significant amount. By the way, remember that our data set is from daily close prices in the span of five years since the test period is half of the data set. That means this increase is over a span of about 2.5 years. So you can take this profit and calculate the effective yearly return to get a better idea of how this agent performs compared to, say, the S&P 500.

LECTURE 110

Stock Prices and Trained Agent

In this lecture we are going to discuss some additional results from my trading bot.

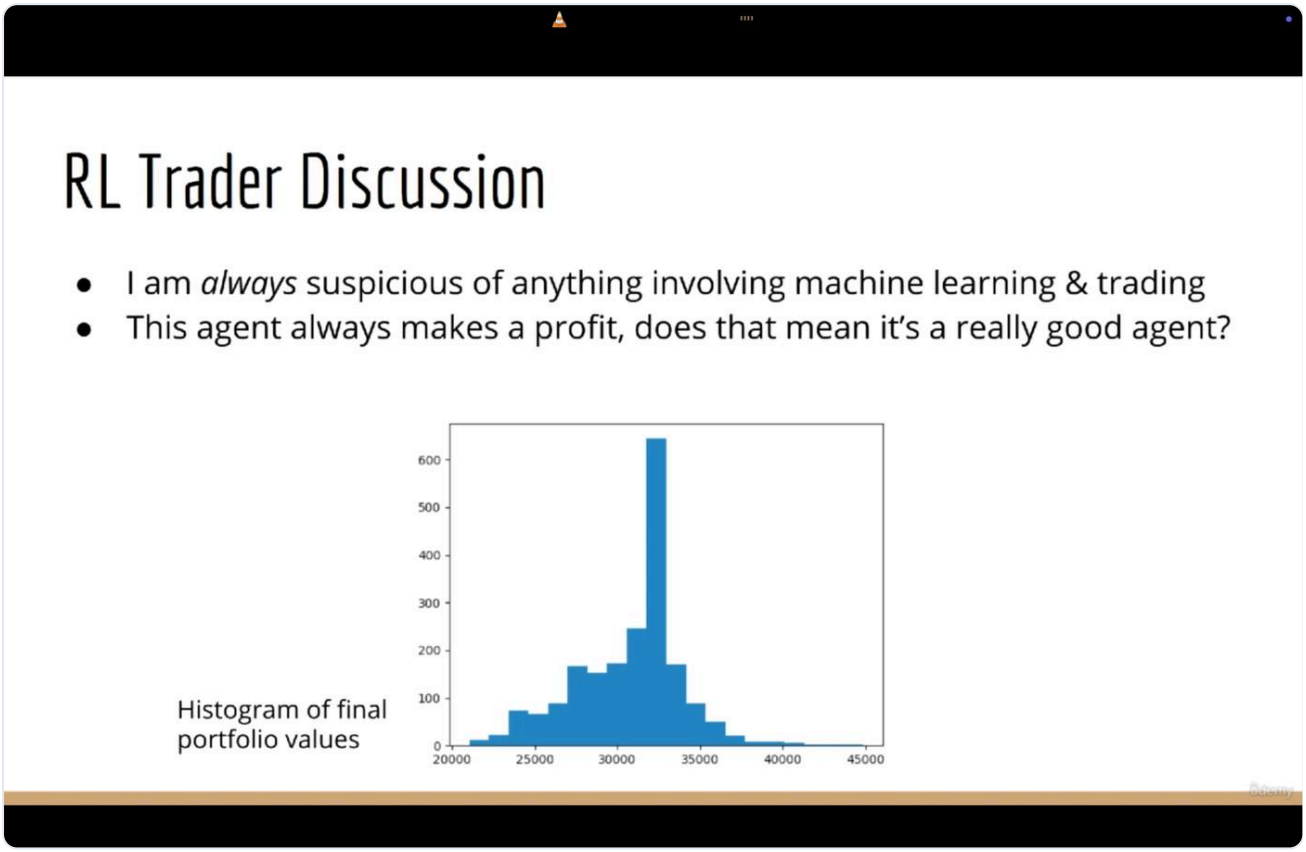

FIGURE 17.1

Personally I am always suspicious of anything involving machine learning and trading. You might think because this agent almost always makes a profit. It must be a really good agent. However there are things we have not yet considered first in order to really evaluate the agent. We have to have something to compare it to a reasonable benchmark I think is to see how a completely random agent performs that's equivalent to running a script with Epsilon equals one or to not have an agent at all.

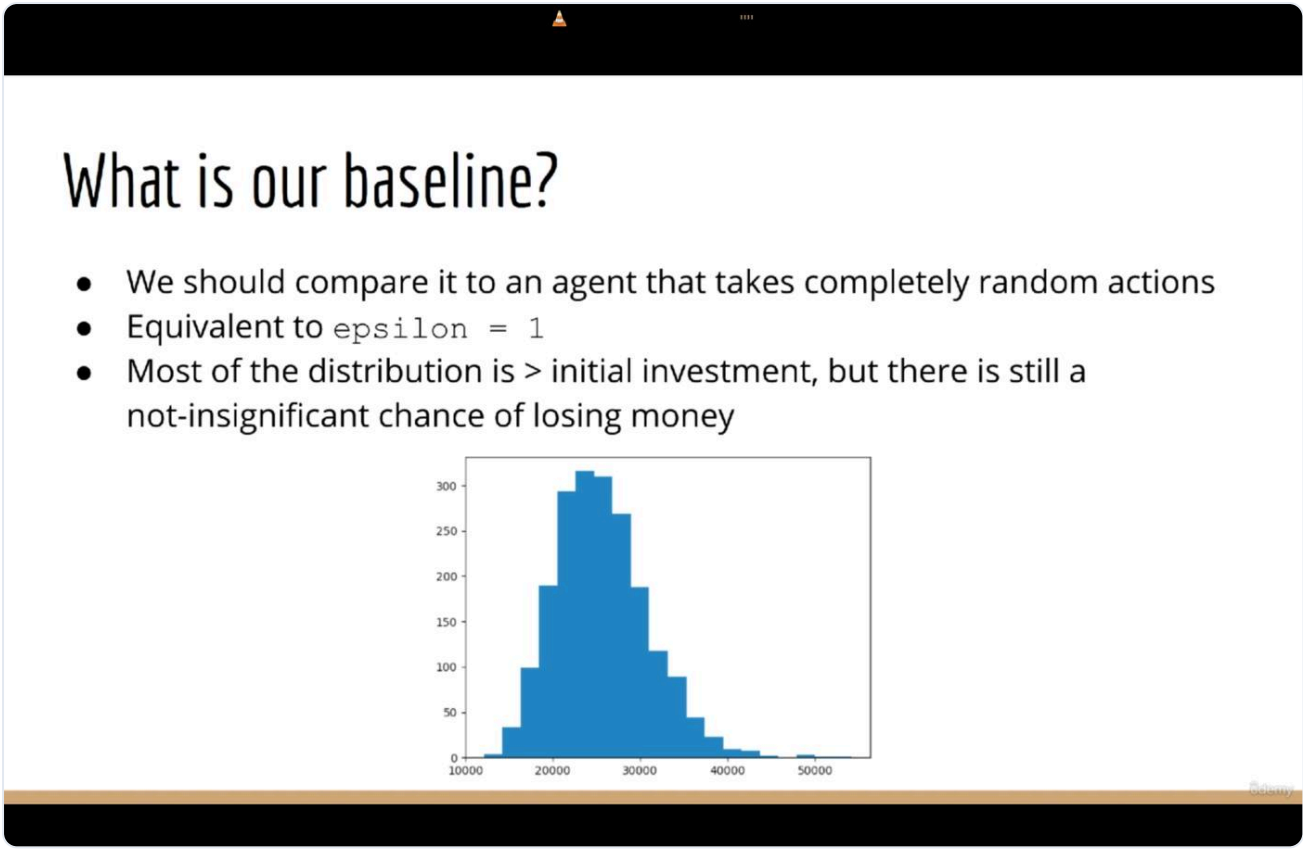

FIGURE 17.2 | What Is Our Baseline?

When we do that we get a histogram like this as you can see most of the weight of the distribution is above our initial investment which means even with a totally random strategy we would probably make money but also there are some pretty low values here too where we have lost money. The probability of that happening is not insignificant. On average however the profit is still not bad compared to our trained agent. Our trained agent performs much better the probability of losing money is much less and the floor is only just under the initial investment.

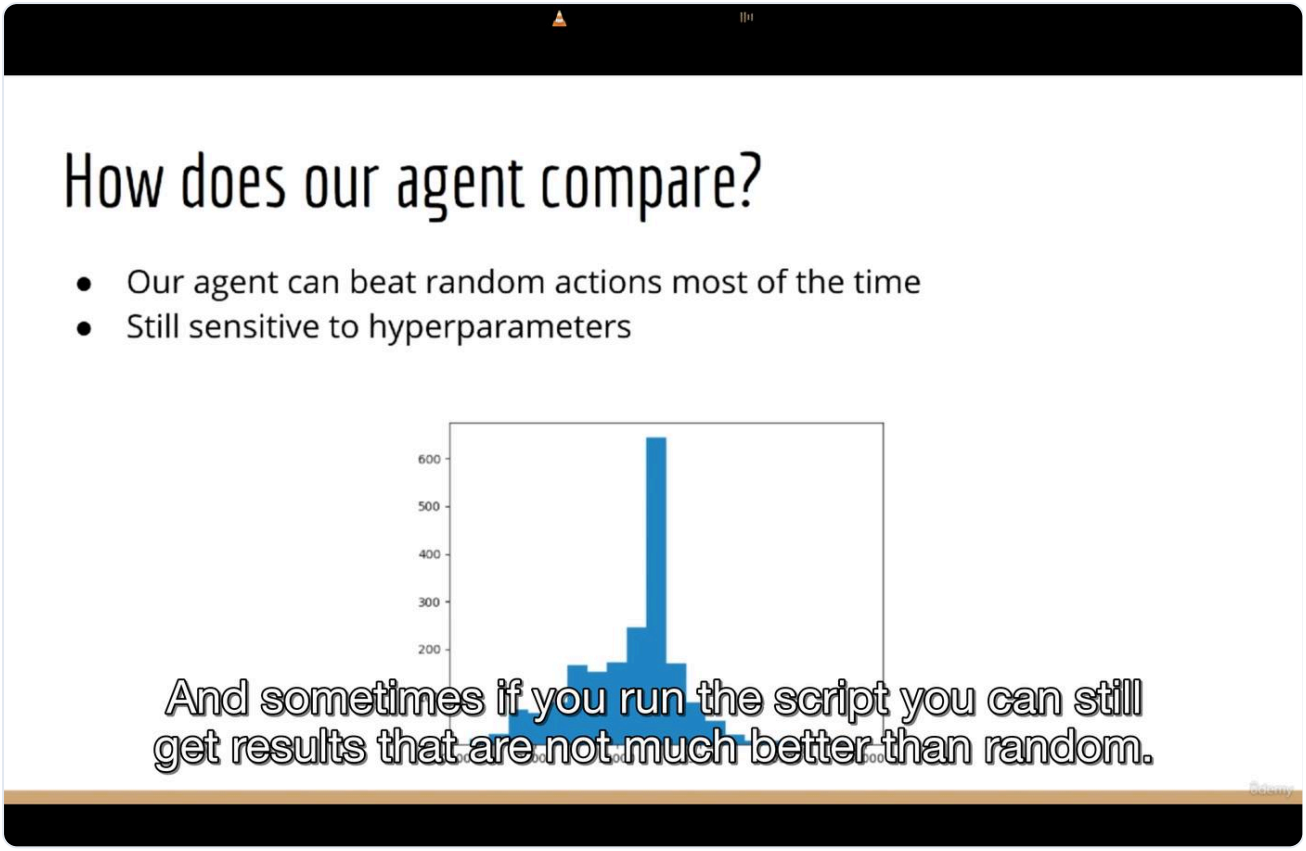

FIGURE 17.3 | How Does Our Agent Compare?

Of course this really depends on the hyperparameters you choose and the final values of the weights. And sometimes if you run the script you can still get results that are not much better than random. Now you're probably wondering how is it that even if we take purely random actions we can still make a decent profit. Well it has to do with our data dataset in general for the three stocks we chose. The value is increasing.

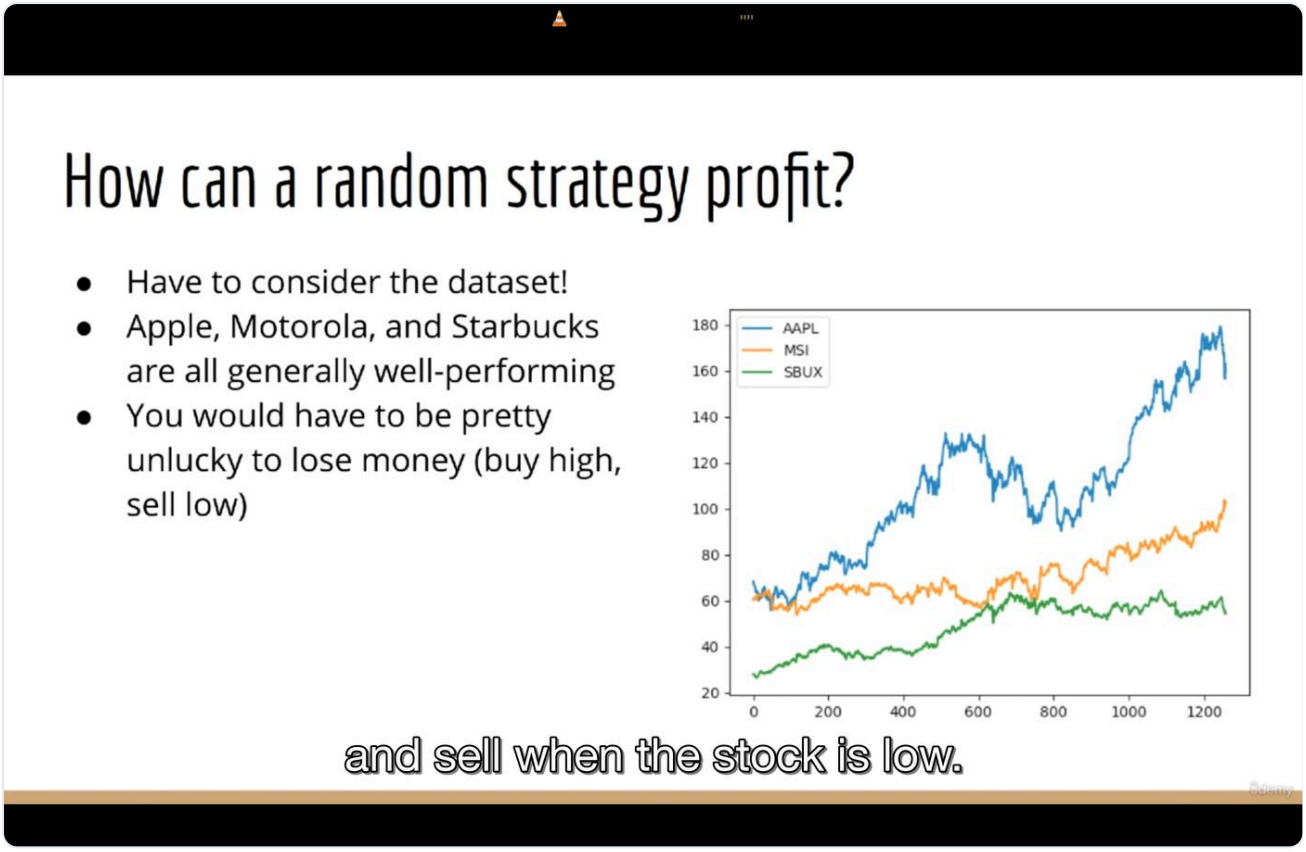

FIGURE 17.4 | How Can a Random Strategy Profit?

Therefore it doesn't really matter what we do unless we always unluckily buy when the stock is high and sell when the stock is low. But if the stock is mostly going up then there's less of a chance of that happening. So one thing you can do if you really want to test this out is to test it on a set of stocks which go up and down a synthetic data set would be a great choice here.

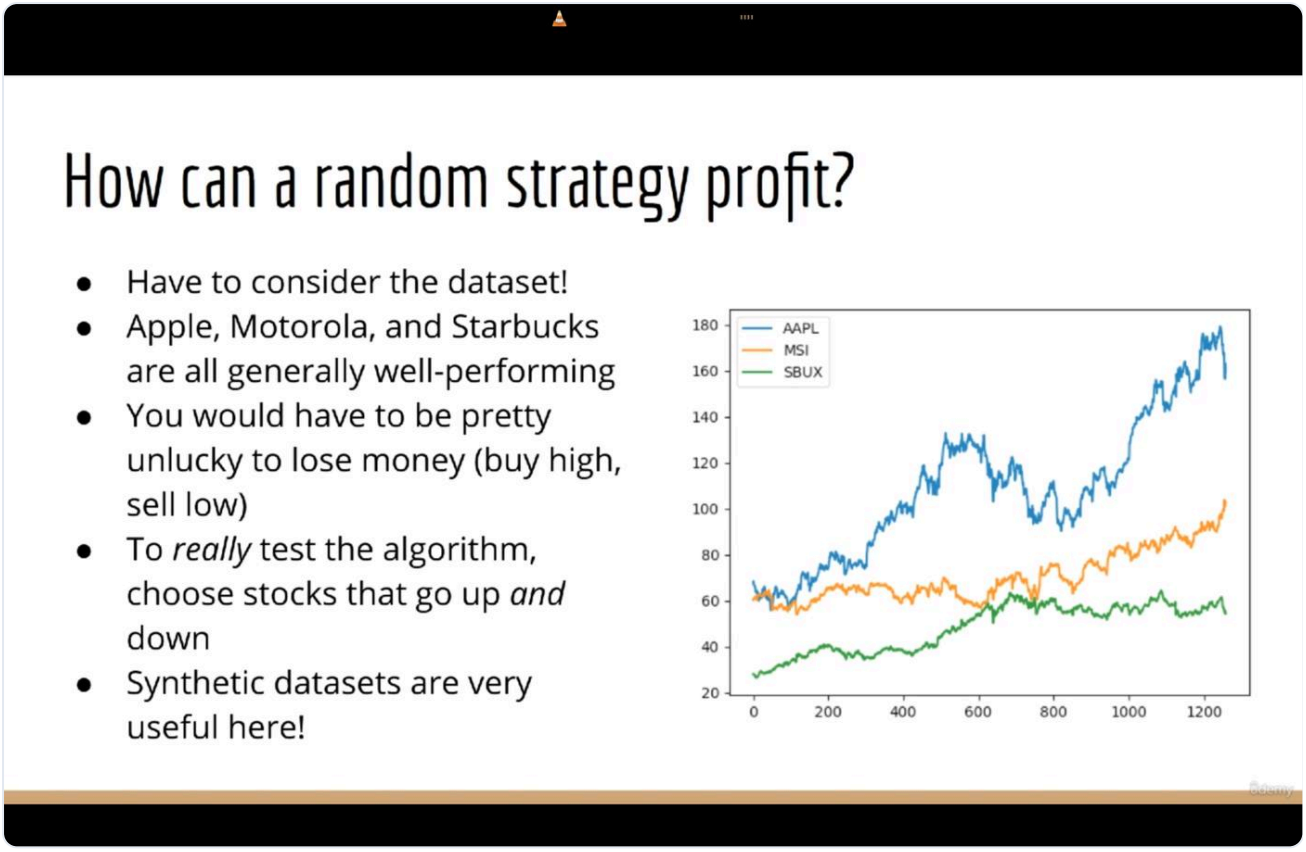

FIGURE 17.5 | How Can a Random Strategy Profit?

Here's some other ways you could extend this project. One that you could incorporate metadata also such as the sentiment of news articles or Twitter feeds. Of course this now becomes somewhat of an engineering problem because you have to gather all this data for every day in the span of five years to another thing you could do is incorporate past values into the state.

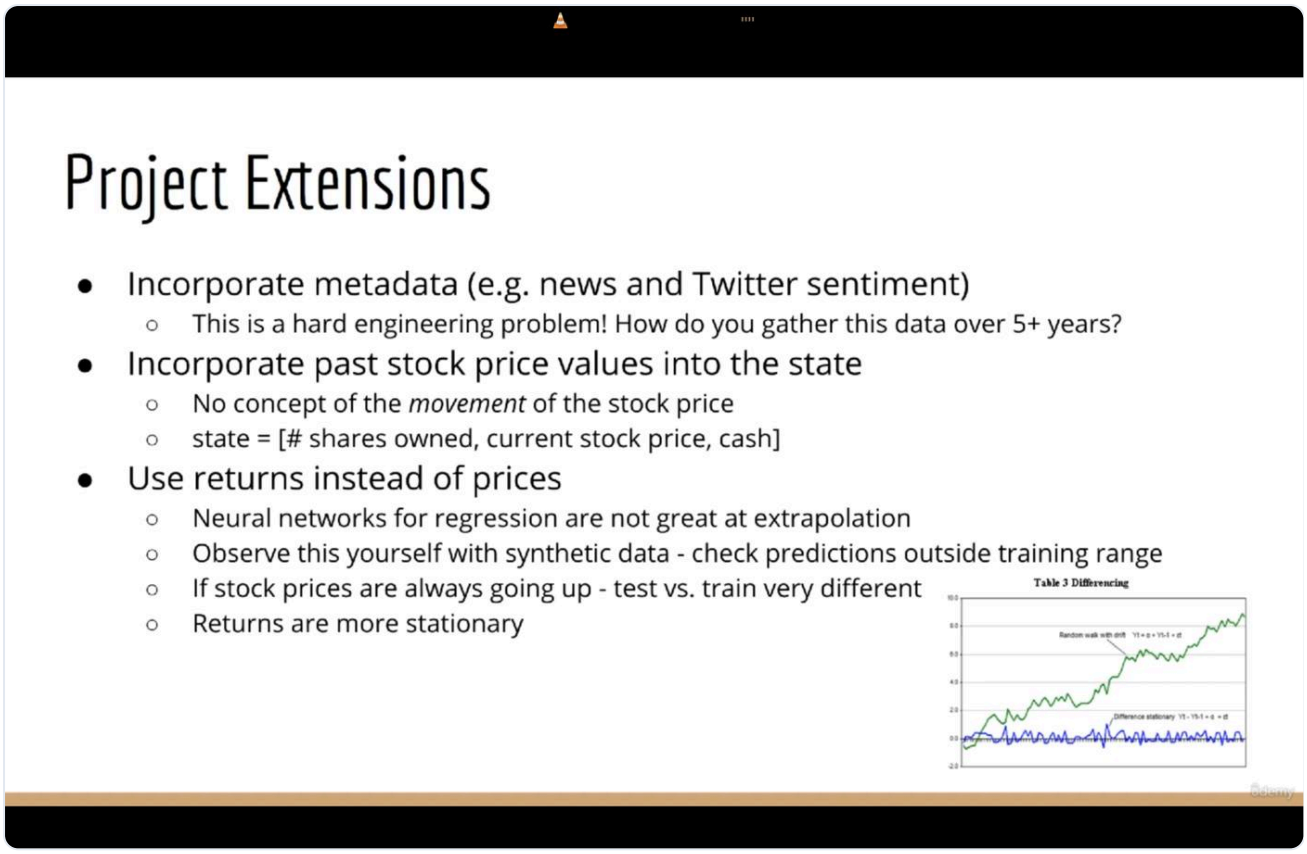

FIGURE 17.6

Currently the model doesn't have any concept of the movement of stock prices. It only knows the current stock prices. The number of shares and the uninvested cash. 3 You could try using returns instead of actual prices. If you recall neural networks are not that great at extrapolating for regression. We can observe this pattern by creating a synthetic dataset and looking at what a neural network does when it sees an input outside the range. It was trained on with these stock prices which are always going up that's bound to happen. Returns however should be more stationary.

LECTURE 111

Review: Maximum Likelihood Estimation (MLE)

In this lecture we are going to look at a slightly more advanced topic in PyTorch which is creating custom loss functions. In addition we're going to apply this new knowledge to build a new kind of neural network one that not only estimates a function but also estimates the uncertainty in the function prediction previously. We spent the entire course only looking at neural networks that act as function approximators for each input. We get some output prediction but you may have noticed something curious. There is in fact a discrepancy between a regression and classification with regression. We only get one answer. The model's output is our best guess for the target given that input.

But with classification we get a probability distribution.

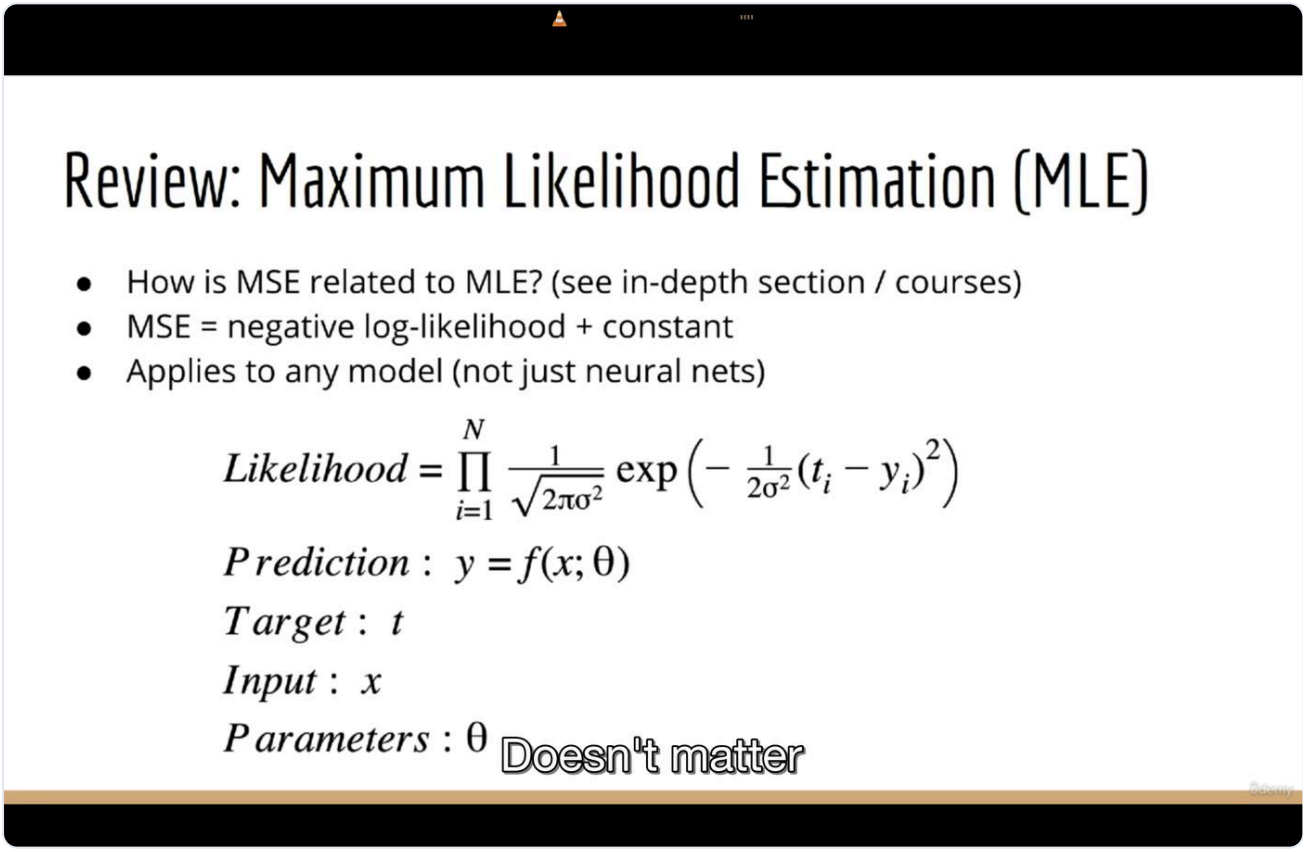

FIGURE 18.1 | Review: Maximum Likelihood Estimation (MLE)

This probability distribution gives us some sense of the confidence in that prediction. For example if we're doing binary classification and the output is zero point nine nine then we might conclude that our model is very confident in that prediction. If the model outputs zero point five five then it would still make the same prediction. But intuitively we know that we are less confident in that prediction. You can apply the same reasoning to softmax outputs. So given this knowledge you might wonder why don't we output probability distributions when we're doing a regression

so if you've studied regression and the mean squared error more in-depth then you know that doing regression with the squared error is nothing more than maximum likelihood estimation with maximum likelihood estimation.

We start with the likelihood.

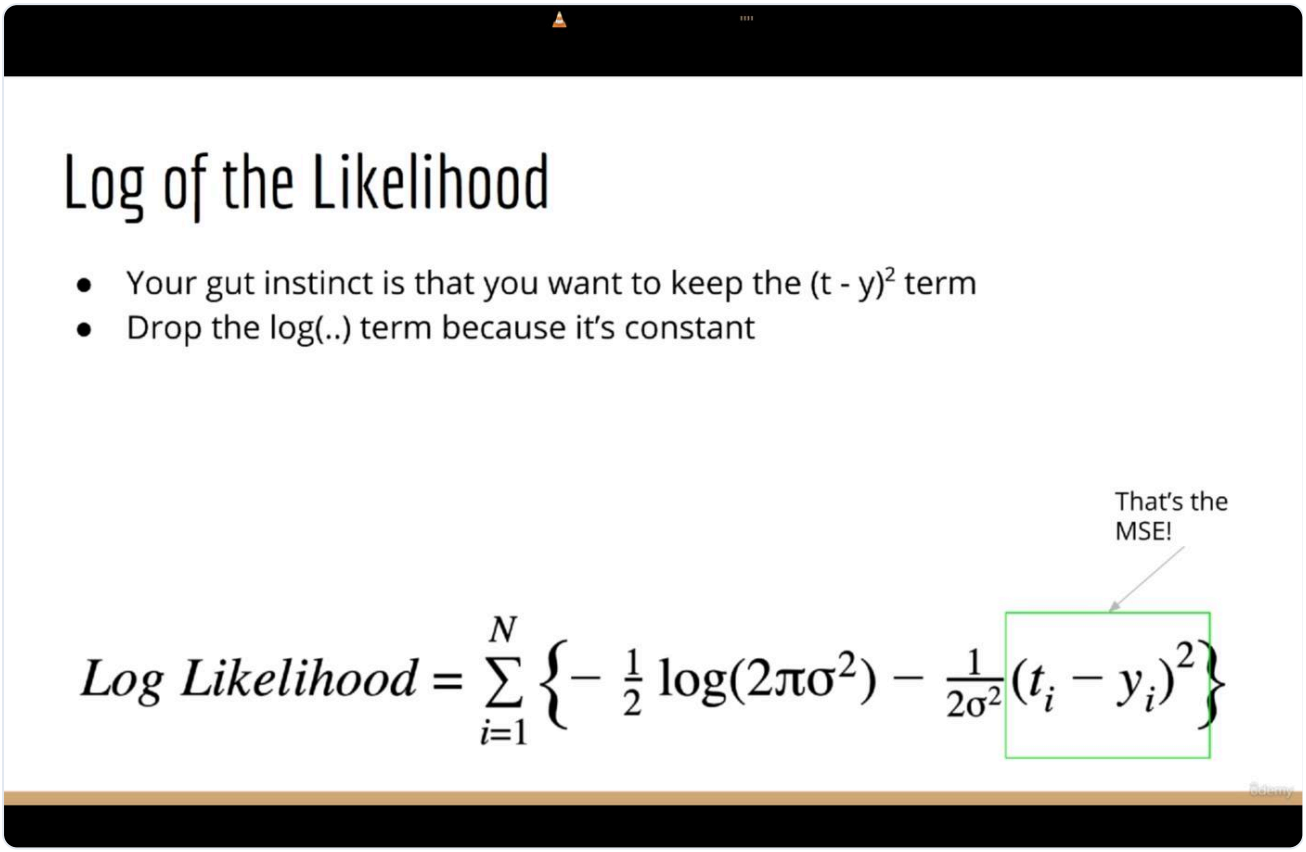

FIGURE 18.2 | Log of the Likelihood

Take the log of the likelihood and then negate the likelihood to make it a loss to minimize rather than a utility to maximize in this form. We say that T which is the target is the random variable which we assume comes from a Gaussian distribution. Y is the model prediction which is the mean of the distribution and X is the input to the model we call theta. The collection of model parameters. So this can represent a neural network linear regression or any other regression model. Doesn't matter as you've seen the usual first step is to take the log of this function at this point your gut instinct should be this. We know that we want to end up with the T minus Y squared term and all the other terms should drop off.

That's what gives us the sum of squared error or the mean square error and we know that we can do this because the log of 2 pi sigma squared is constant constants don't affect the gradient of the loss.

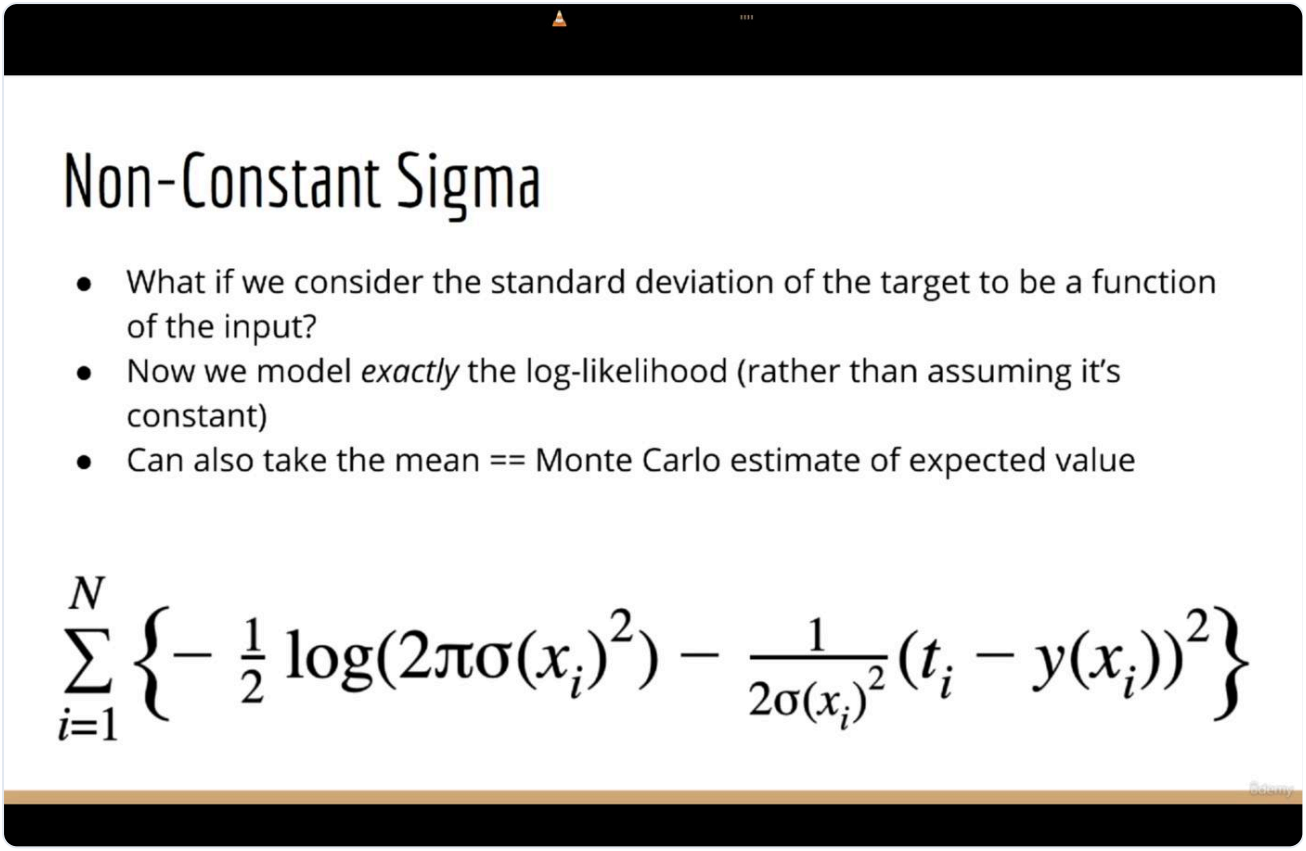

FIGURE 18.3

So whatever model parameters maximize this log likelihood would also maximize the log likelihood minus or plus any constants but here's the major difference in this lecture in this lecture. We are not going to make Sigma a constant. Instead we can ask what if sigma is also a function of x. So now this term can't be dropped from the objective function because it is not constant. What this means is that in this scenario this whole thing will be our loss function instead of just the squared error. And as usual we can take the mean instead of the sum which is equivalent to having a Monte Carlo estimate of the expected value. The next question you should have is OK now we're trying to predict two things the target and its corresponding variance.

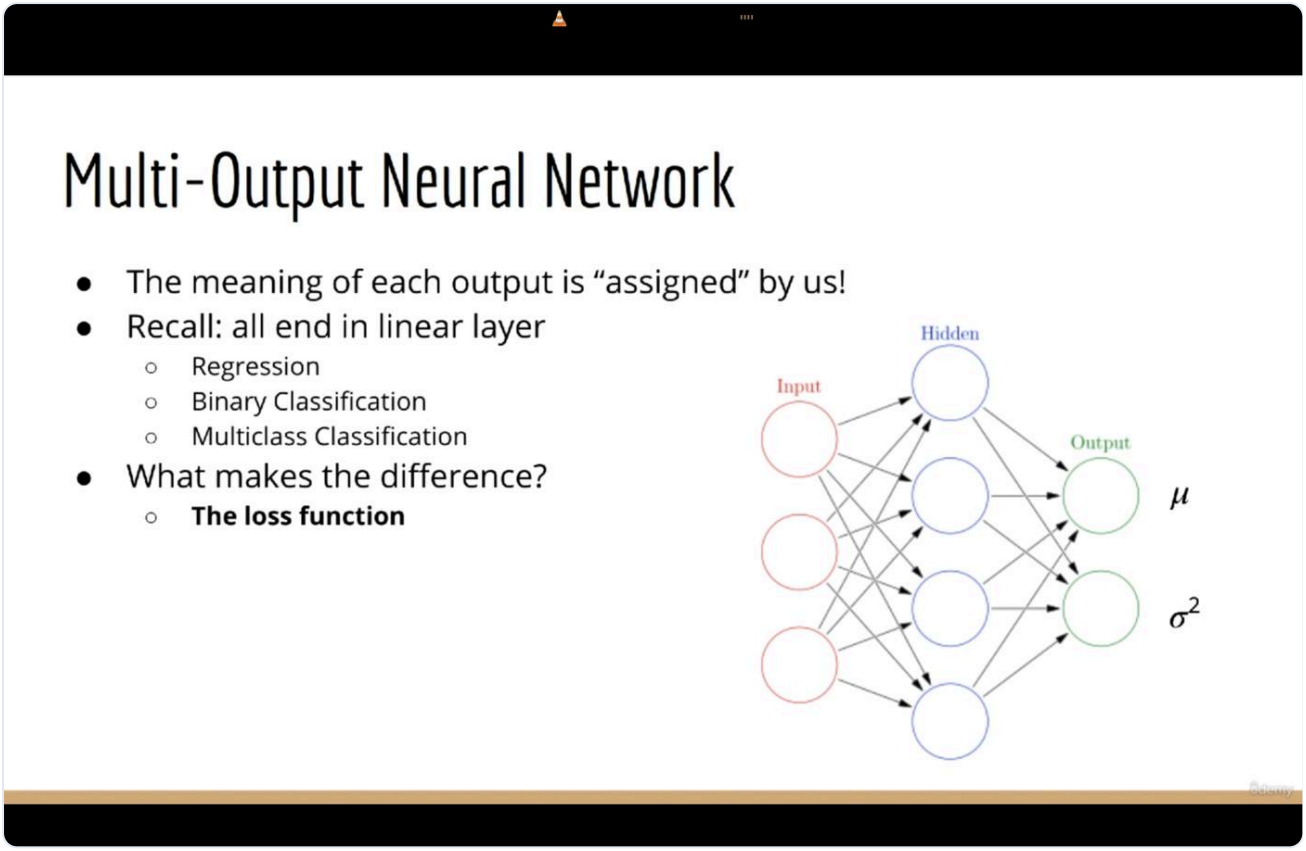

FIGURE 18.4

How can we do something like that. Well why not just have a neural network with two output nodes. Let's remember that it doesn't matter what the meaning of these two output nodes is. We've seen that a neural network can end with a linear layer with any number of output nodes. That's the case whether we're doing regression binary classification or multi class classification. So then what's the difference between these three cases. The difference is that the meaning is assigned by us the designer of the program and we assign that meaning through the loss function. In other words it's up to us to come up with the appropriate loss function such that the meaning of these two outputs is the Gaussian mean and variance but you should be suspicious at this point.

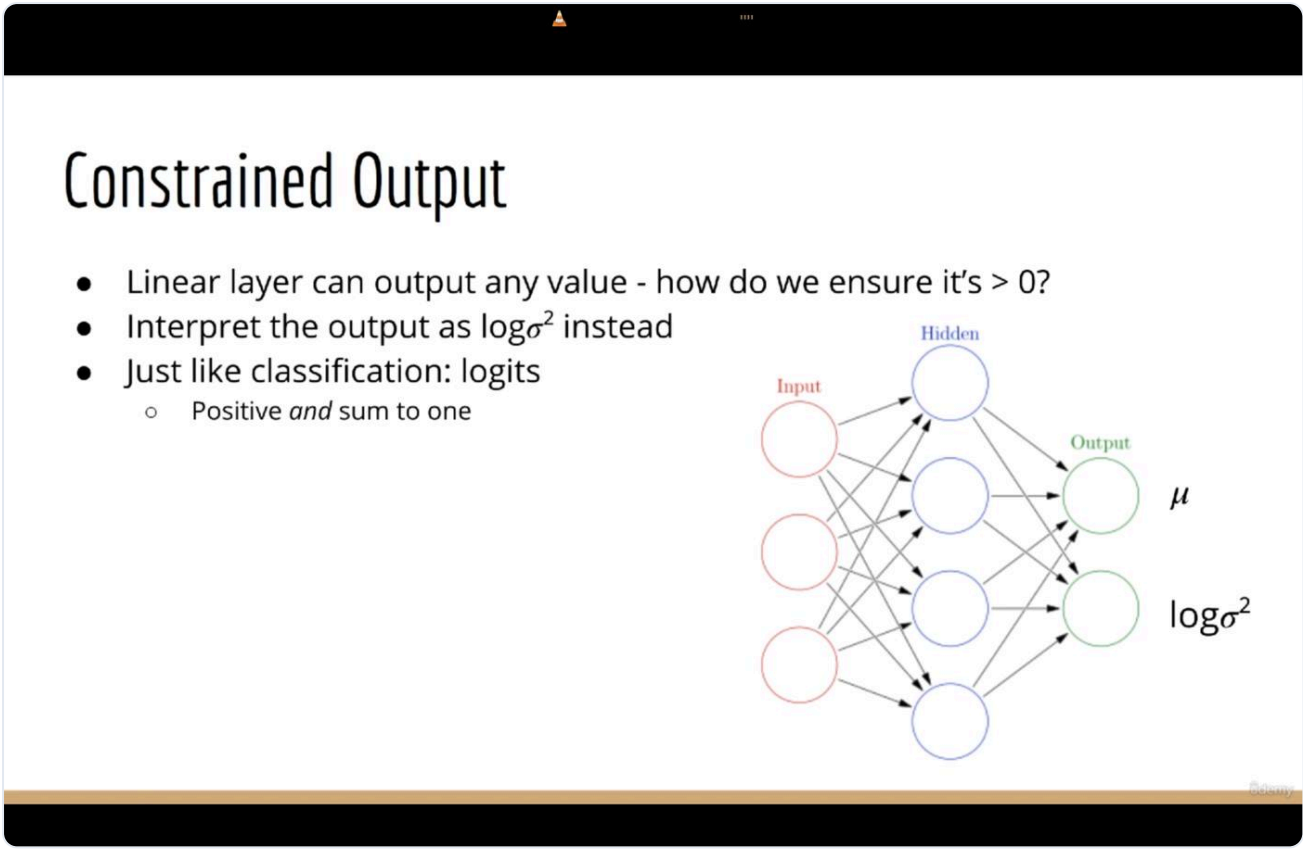

FIGURE 18.5

A neural network layer can output any value but don't we have a constraint on the value of a variance. In fact we require that a variance is positive in order to enforce the variance to be positive. We can make the assumption that the neural network doesn't output the variance directly but rather the log of the variance if you think hard this is just like the classification case. The neural network doesn't output probabilities directly.

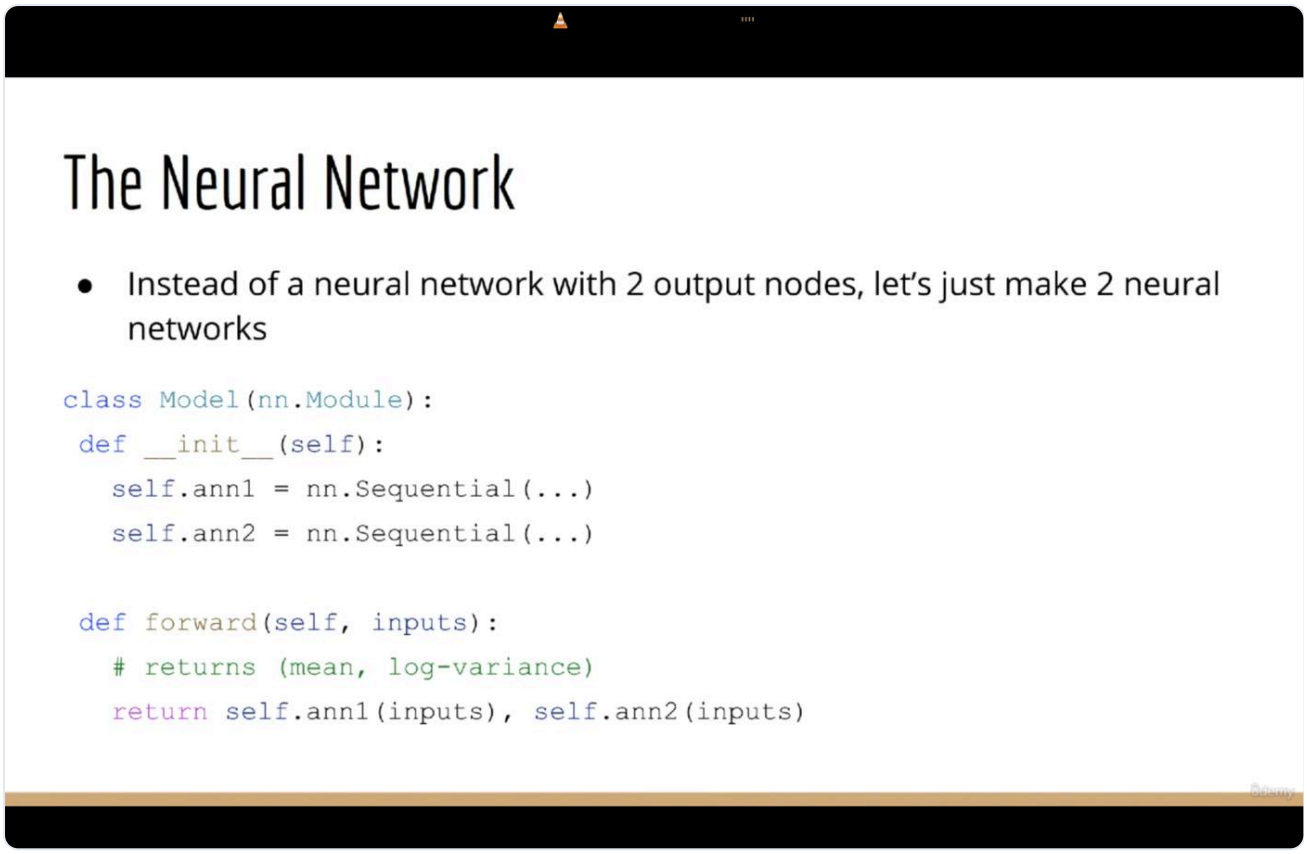

FIGURE 18.6 | The Neural Network

Instead it outputs logits which are related to log probabilities. In fact that has more constraints because not only do probability distributions have to have positive values they also have to sum to one using this information we can determine what the loss function should look like with respect to the neural network's outputs. First let's assume that instead of having a neural network with two output nodes we simply have two neural networks in our model and then the forward function returns the output from these individual neural networks then the output of our model from the forward function will be a tuple with the first element is the mean predictions and the second element is the log variance predictions.

Now we get back to our original question which was how do we create a custom loss function in PyTorch. As always it's easiest to start with what we're familiar with the mean squared error.

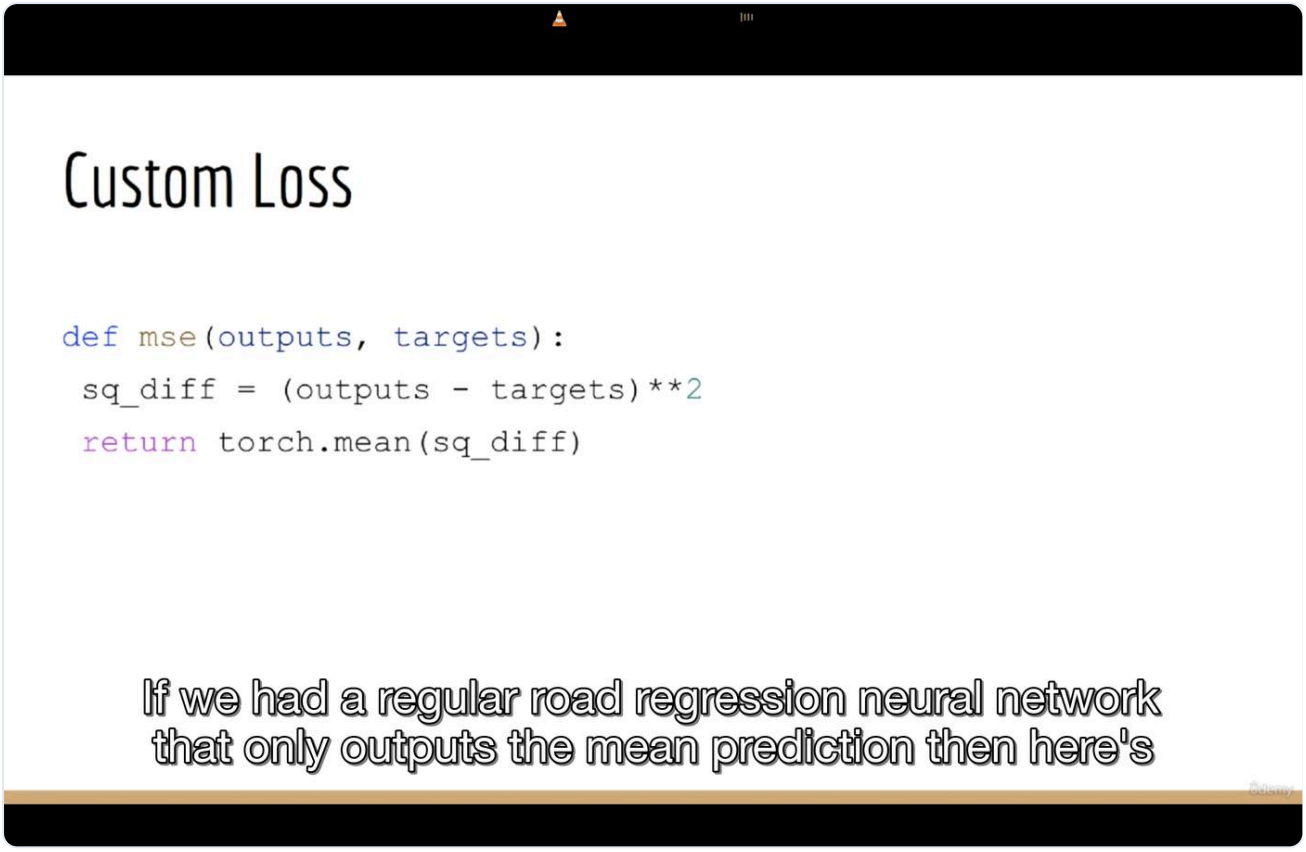

FIGURE 18.7 | Custom Loss

If we had a regular regression neural network that only outputs the mean prediction then here's how you would do it.

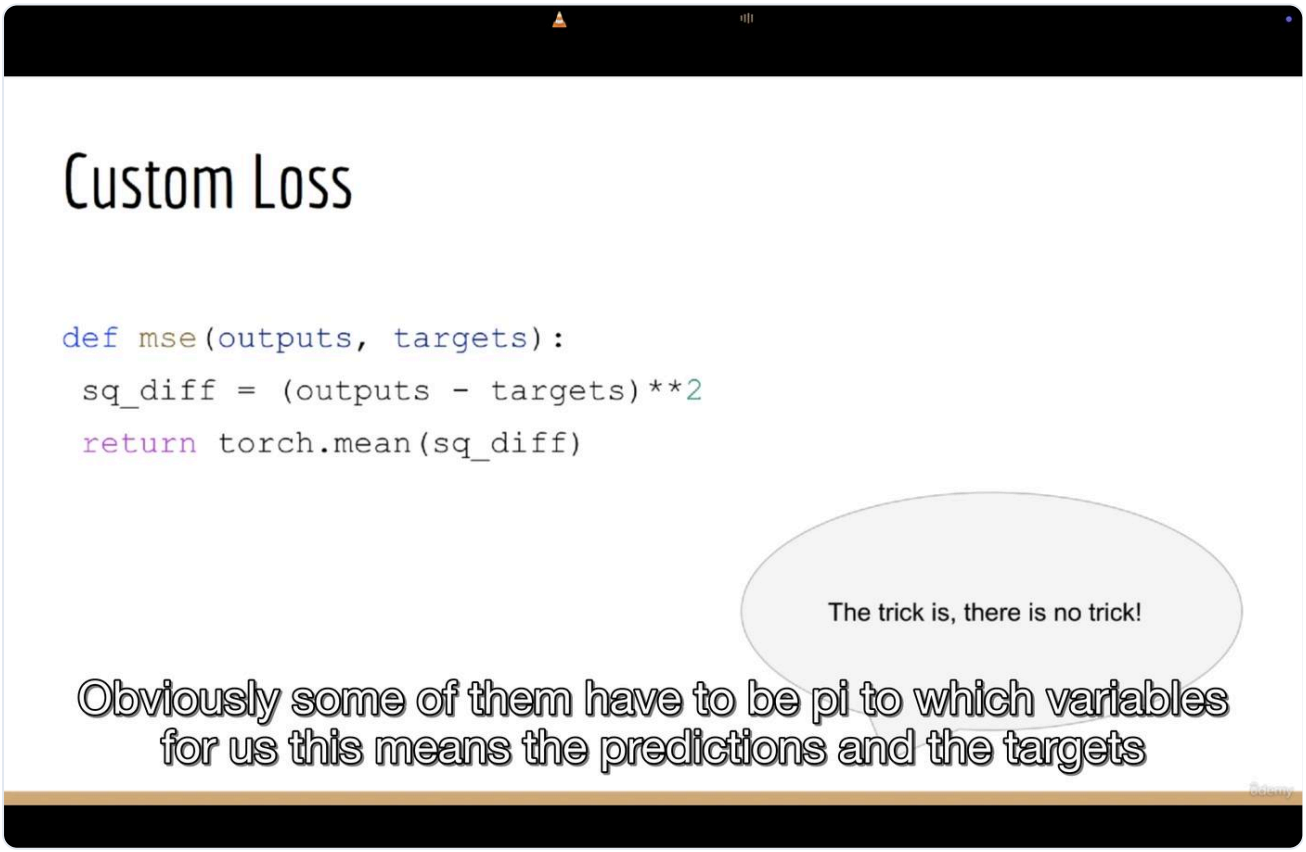

FIGURE 18.8 | Custom Loss

The trick is there is no trick. The lost function is merely a regular Python function that takes in any arguments you want. Obviously some of them have to be PyTorch variables for us this means the predictions and the targets then inside this function we find the element wise difference between the predictions and the targets square them and then take the mean. Pretty straightforward so a custom loss function is just a regular function that does mathematical operations using PyTorch. So how do we do this. When it comes to the full negative log likelihood First we still take in the same arguments the predictions and the targets.

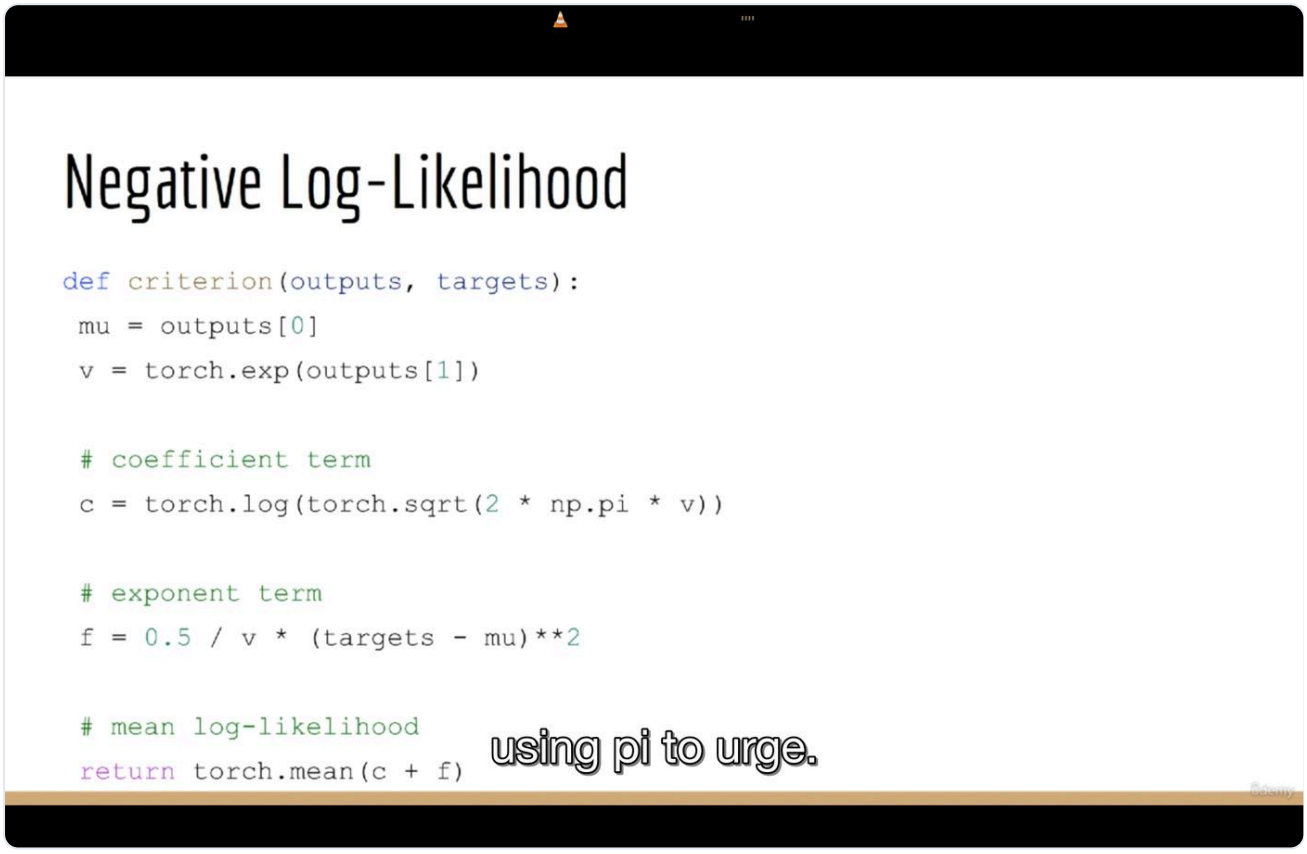

FIGURE 18.9

This time the predictions are tuples so we can split them up. The first element becomes the mean which I'll call mu and the second element becomes the log variance. Since the exp function is the inverse of the log function I can take the exp of the log to get the variance itself which I'll call V then we simply need to plug and chug into our formula for the negative log likelihood. This consists of the coefficient term and the exponent term since we're in log space. We can just add these two together and this gives us the element wise negative log likelihood after which we can take the mean so at this point you may be wondering how is a model like this useful.

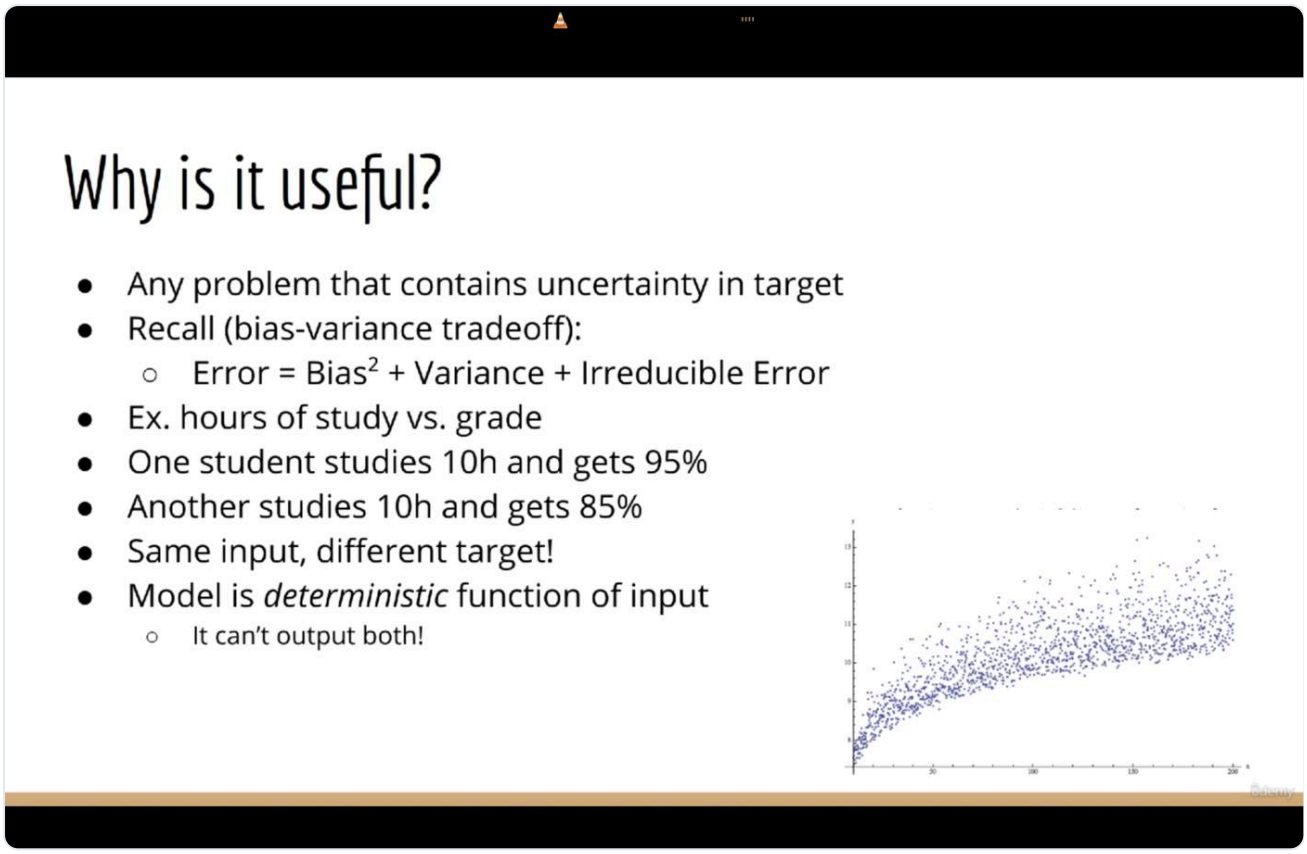

FIGURE 18.10 | Why Is It Useful?

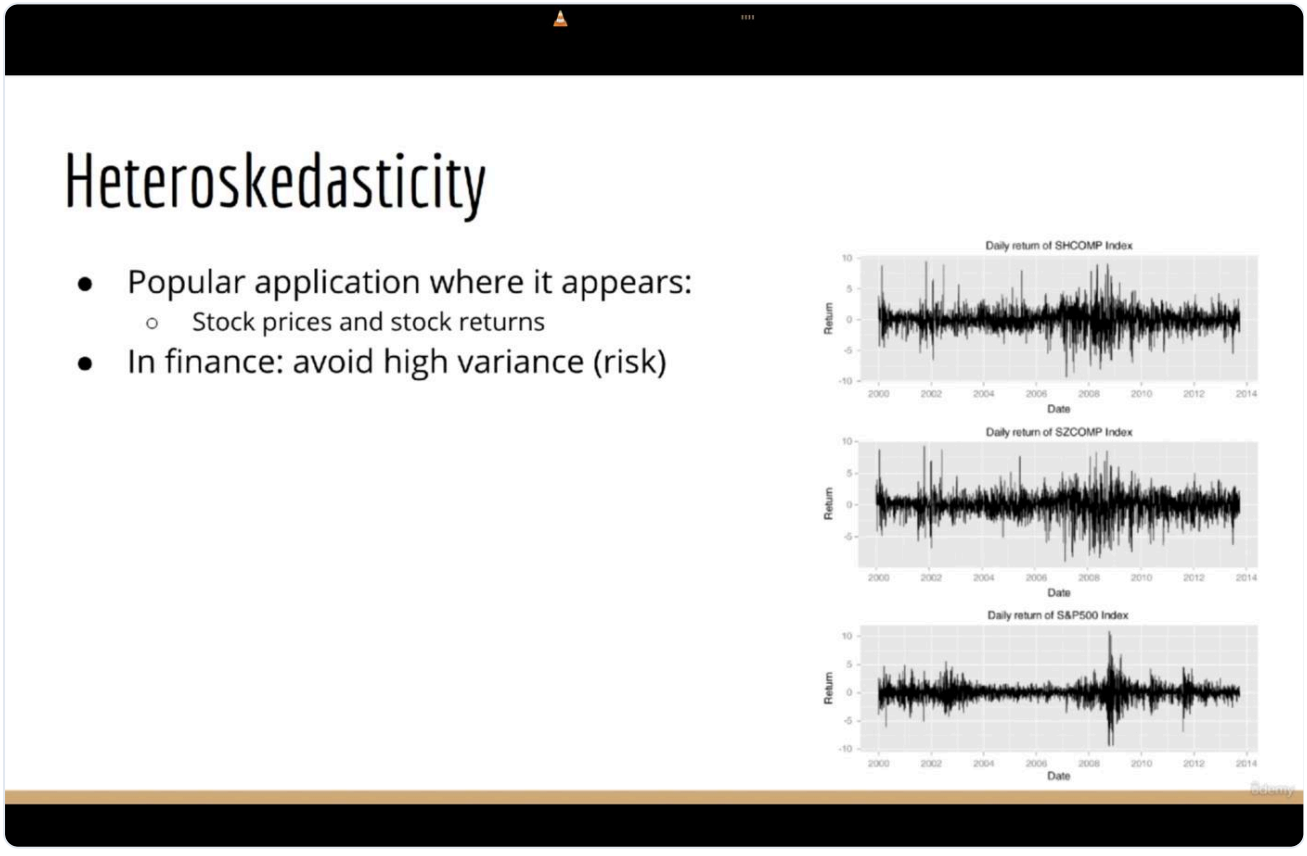

FIGURE 18.11

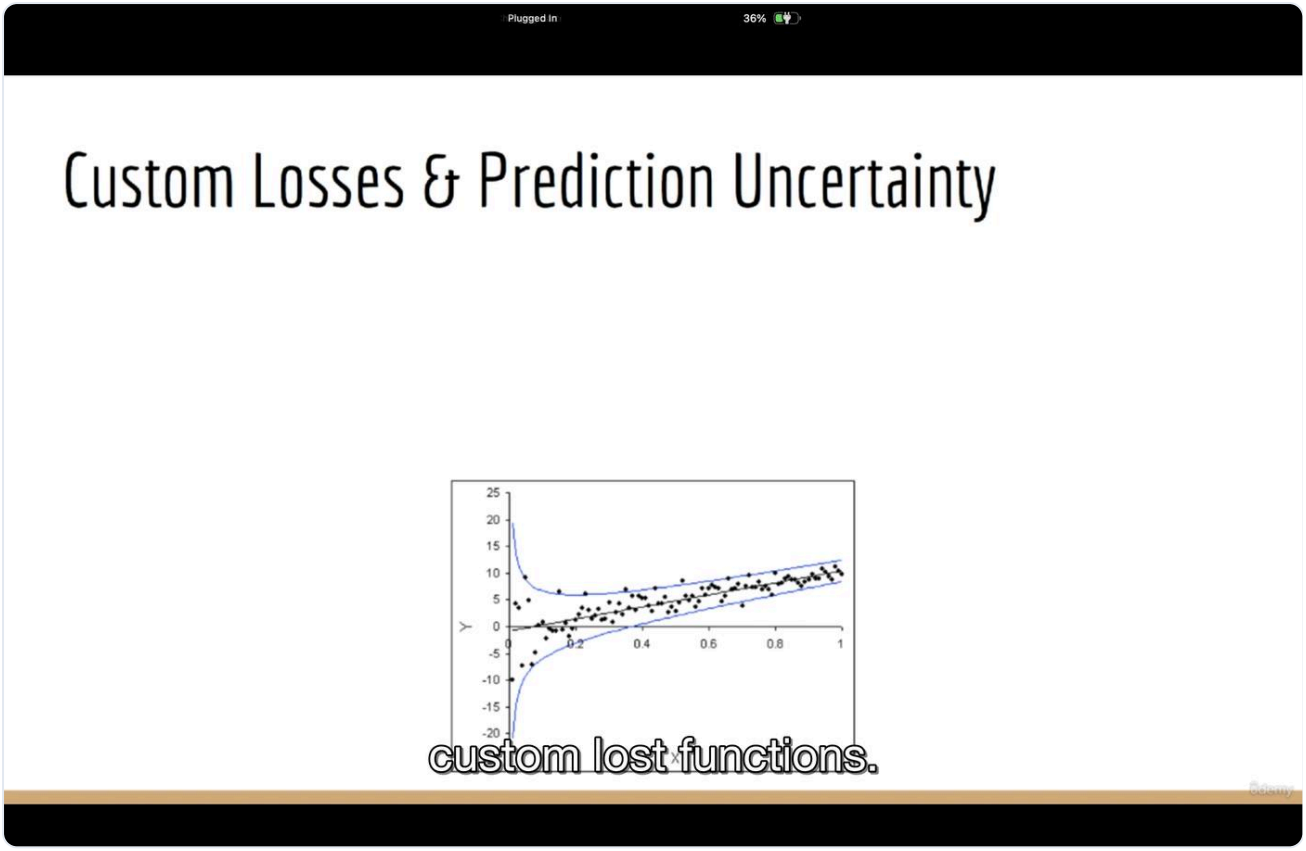

FIGURE 18.12 | Custom Losses & Prediction Uncertainty

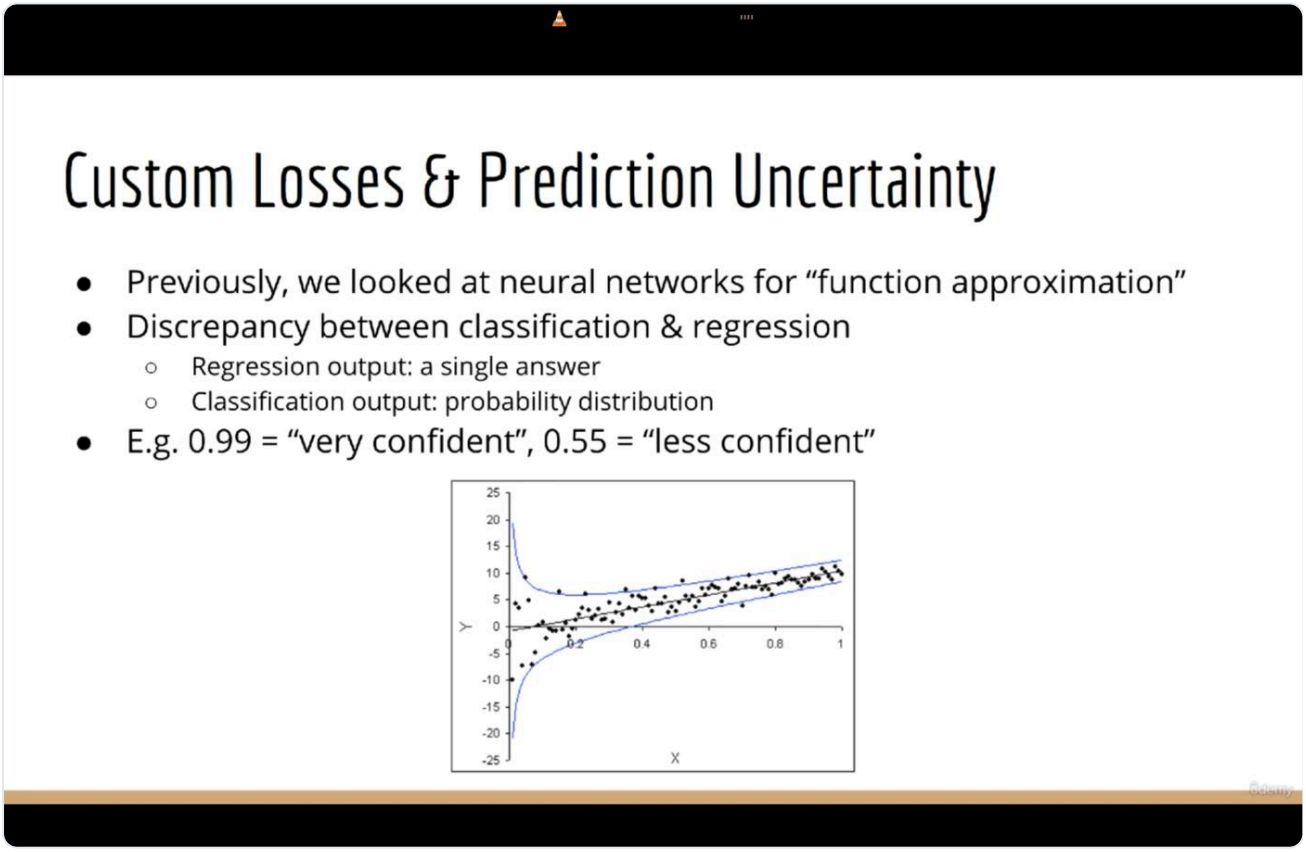

FIGURE 18.13 | Custom Losses & Prediction Uncertainty

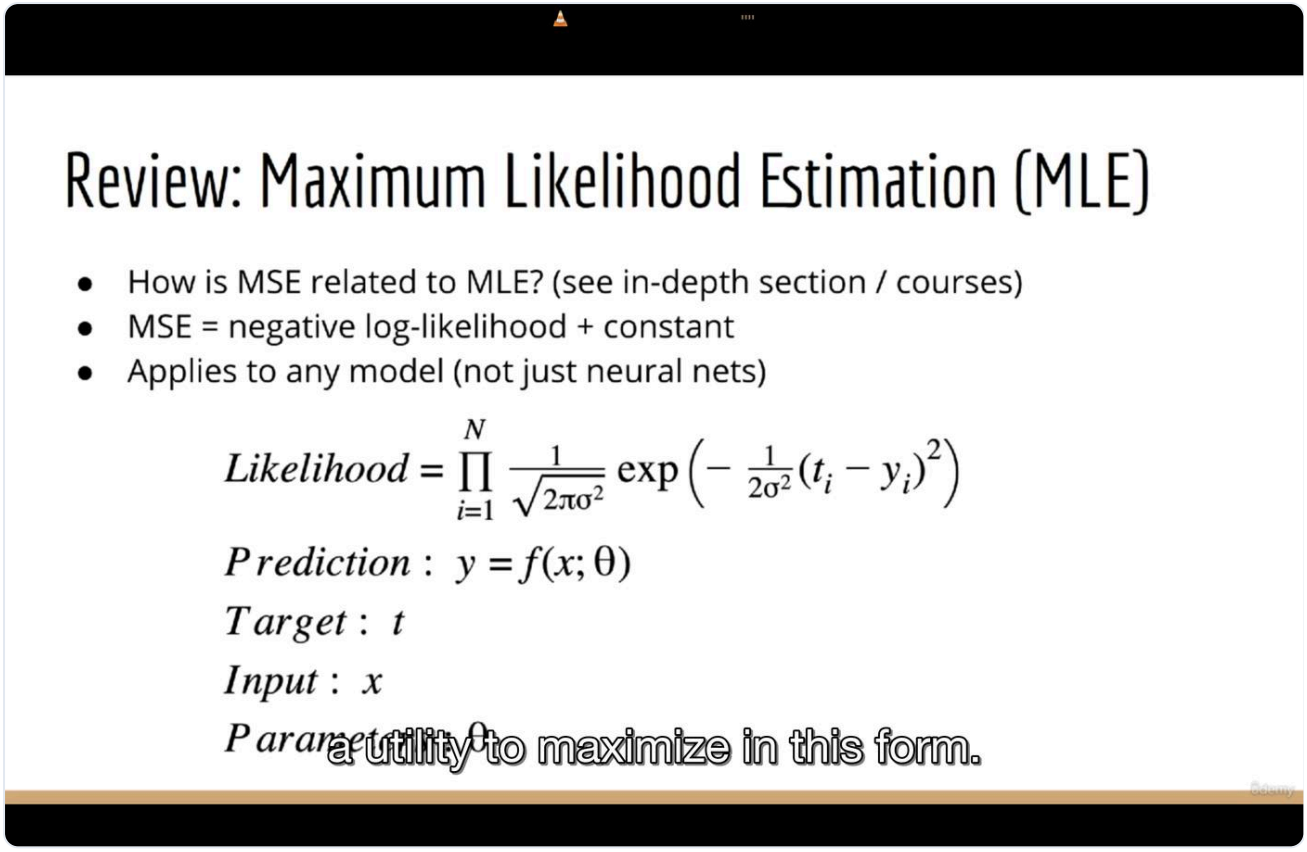

FIGURE 18.14 | Review: Maximum Likelihood Estimation (MLE)

And naturally you can apply this technique to any problem that contains uncertainty in the target function if we recall the bias variance tradeoff. This is the statement that the error of your model is the bias squared plus the variance plus an irreducible error the irreducible error is called that because it's a characteristic of the data itself and your model can't make a perfect prediction because of it. This arises naturally. For example suppose you're measuring the relationship between how long students study on a test and the grade let's say you have one student who study 10 hours and got a ninety five percent. And you have another student who study 10 hours and got an 85 percent.

In this case the input is the same but the target is different. And since your model is only a function of the inputs it can't make a perfect prediction. If there are two different possible target values for that single input as you can imagine this kind of noise can appear everywhere. So instead of ending with a generic statement like my model has a low R squared you can quantify the uncertainty of your model instead when the variability of the target. With respect to the input is not constant. We say that it's heteroskedastic we can think of other situations where heteroskedasticity can arise. One popular application is with stock returns and stock prices.

These are well known to be heteroskedastic in finance. We often want to avoid high variance because we think of variance as risk. And so in order to minimize risk we should minimize variance. And in order to minimize variance it would be useful to model not just the prices of the returns but the variance itself.

LECTURE 112

Standard Deviation

Check the notebook `25.PyTorch-Uncertainty-Estimation.ipynb` for the code.

In this lecture we are going to look at the code to demonstrate uncertainty estimation using PyTorch. This lecture is going to walk you through a prepared Colab notebook. Although a very good exercise which I always recommend is once you know how this is done to try and recreate it yourself with as few references as possible as usual you can look at the title of the notebook to determine what notebook we are currently looking at. So let's begin by writing a function that generates data from a known synthetic distribution. This might seem weird but in fact this is exactly what we do when we run an experiment or do data collection. It's like we have some function called generate batch and that function is a black box which involves whatever data generating process happens to result in the data you're measuring.

Of course the difference is that you don't ever know the true distribution of your data nor the underlying data generating process. And that's why we need machine learning. But for this example we do know the true distribution and the data generating process and hopefully that will give you some insight on what machine learning is trying to do. So what does this function look like. First we generate a bunch of random points for the input x from the uniform distribution between minus five and plus five. Next we write an expression for the standard deviation as a function of x. As you can see it's a linear function of x. So when x is small the standard deviation is small and when X is large the standard deviation is large.

Next we get the target we assigned the target to be the cosine of X minus some linear function of x plus Gaussian noise center at zero with a standard deviation of SD. Then we return x and y if we plot the data you should be able to recognize each of these elements. First we can see the periodic component due to the cosine. That's why this thing appears like a wave. Second we can see that a trend downward. Thanks to the linear component. Finally we see heteroskedasticity. The noise seems to increase as X increases. In fact we know that the noise standard deviation increases linearly with X. Importantly we can see here that given the same X it's possible to get many different values for the target.

Next we create our model. This is a pretty basic custom model which contains two simple ANNs. The first ANN will be used to predict the mean of the output and the second ANN will be used to predict the log variance in the forward function. You can see how this is kind of the opposite of a recommender system. A recommender system takes in two inputs a user and an item and returns one output a predicted rating on the other hand. This takes in one input and produces two outputs. Next we instantiate the model and then we create our custom loss. So in this function we'll stick with the convention of calling the first argument outputs and the second argument targets inside the function.

We know that outputs is a tuple with two components. So we take the first component and assign that to a variable called mu. We take the second component exponentiate it and assign that to a variable called V. Next we calculate the coefficient term of the Gaussian distribution. That's the log of the square root of two pi times V. Next we calculate the exponent term of the Gaussian distribution. That's zero point five divided by V times the square difference between the targets and mu. Lastly we add the coefficient term and the exponent term together and take the mean. Next we have our optimizer and our training function. So this is nothing really new. All right.

So here's our loss per iteration. There should seem a little strange since what we're used to when we look at a loss per iteration is that the loss usually has a steady decrease downward. This on the other hand seems to be very noisy. So what is the cause of all this noise. Well remember that our data itself has noise and the loss function is the negative log likelihood. Remember that the loss doesn't only contain just the squared error anymore. So we shouldn't expect the loss to go down to zero since that would mean the variance is zero which we know is not true. Next we plot the model predictions in order to do this. We're going to generate a large batch of 1000 twenty four points after doing a scatter plot.

We convert these points into torch tensors reshape them and pass them through our model. The model returns two things our y hats and the log variance. Since our model outputs the log of the variance we can exponentiate it to get the variance but that's not actually what we want. If you recall the variance doesn't have the same units as the original random variable so it doesn't make sense to plot them together. Rather we would like to plot the standard deviation which is the square root of the variance equivalently. We can just divide by 2 before taking the exponent since the square root is the same thing as a power of 1/2. Next we plot our prediction along with the corresponding uncertainty on top of our scatter plot of the data in order for a plot to look like it should we have to sort the data points first.

This is because matplotlib joins the points you pass in by a line in whatever order you pass them in we can get the sorted index by using the argsort function. Next we plot X versus Y. This should follow the center of the data points for each X. Next we use the fill between function to draw a transparent band with width equal to the standard deviation so that's why we have y hat minus SD and then y hat plus SD. So here's the plot and as you can see the band increases as we go from left to right and our model accurately predicts that the variance should increase as X increases.

LECTURE 113

Section Outline

In this lecture I'm going to introduce you to the next section of the course which is on facial recognition using Siamese networks. Facial recognition has been a very controversial topic in the recent years as governments and basically everyone who is not the government have competing interests. So it's important for everyone to understand facial recognition its strengths and weaknesses and what it can and cannot do so that you can be knowledgeable about how this technology will impact the way we live. Let's outline everything we will do in this section before moving on to the meat of the content.

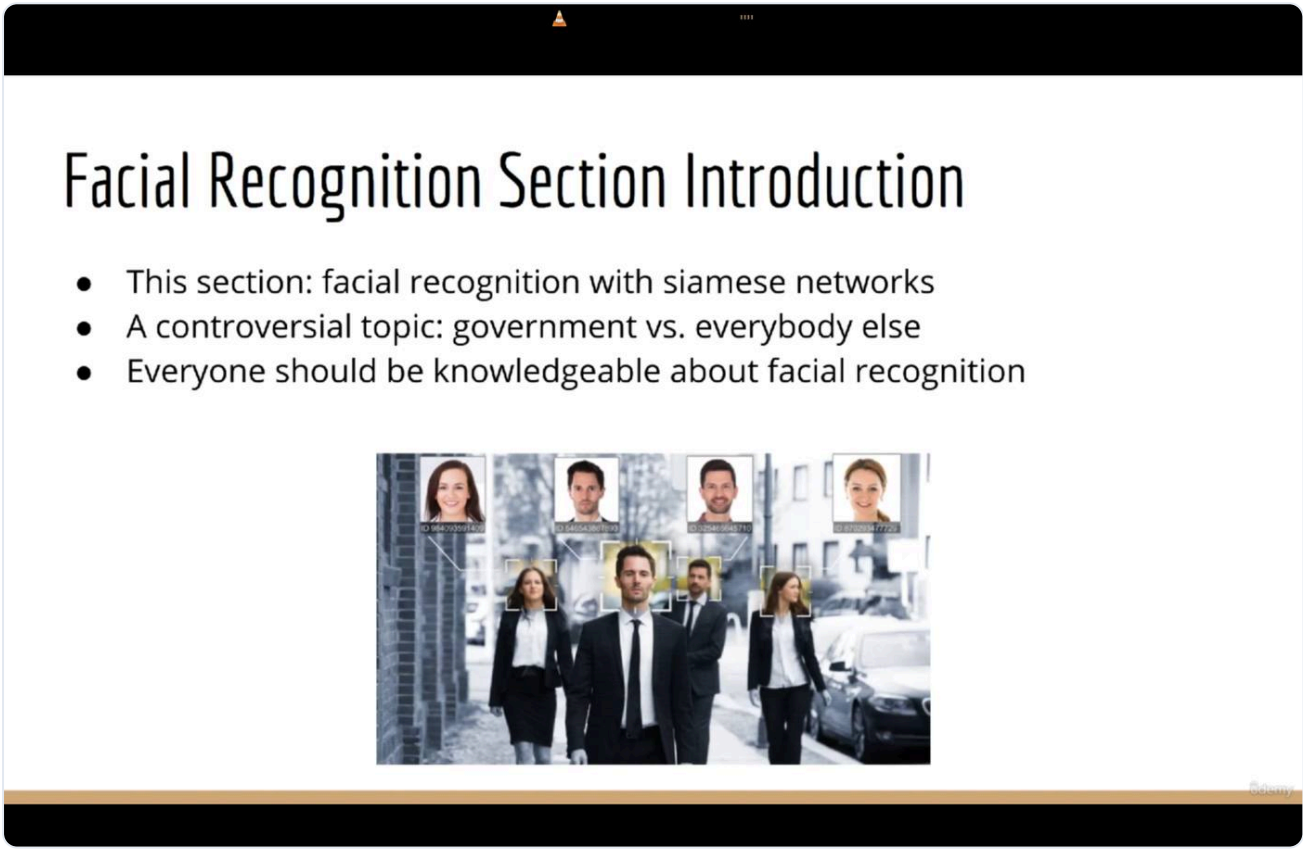

FIGURE 20.1 | Facial Recognition Section Introduction

I think what you'll find is the theory behind facial recognition for the algorithm we are about to discuss is quite intuitive and simple. You can probably arrive at a decent understanding of it in just five minutes. The real challenge is going to be the little details that go into the implementation specifically a lot of these little details have to do with data processing. It's not so straightforward as here's a data set of inputs and targets.

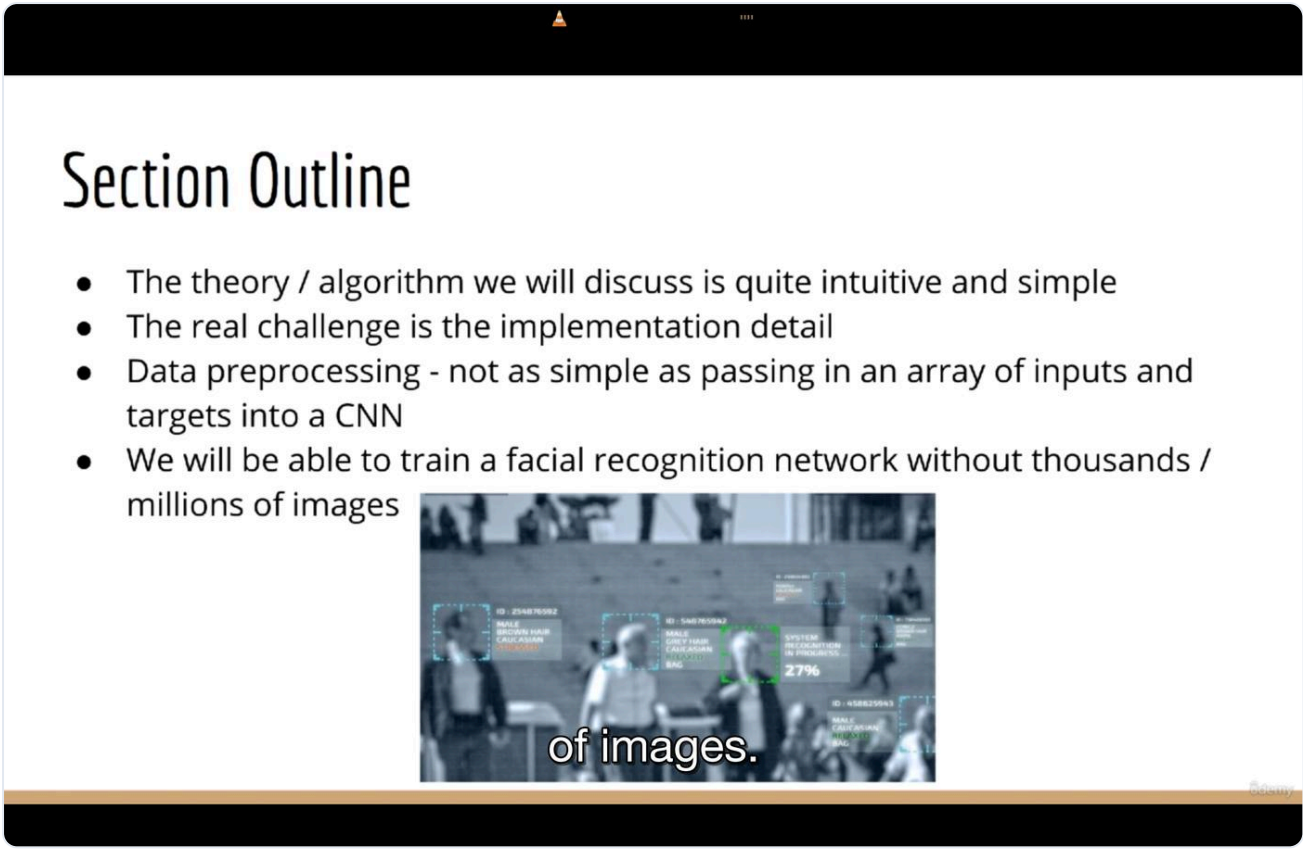

FIGURE 20.2 | Section Outline

Now throw them into a CNN usually that would require a very large set of images which technically we have thanks to surveillance and people giving up their data to companies like Facebook and Instagram but rather as we will learn we can train a facial recognition network without thousands or millions of images. So how can this be. Think of how a facial recognition system might work. Suppose I have a picture of you like say your student I.D.

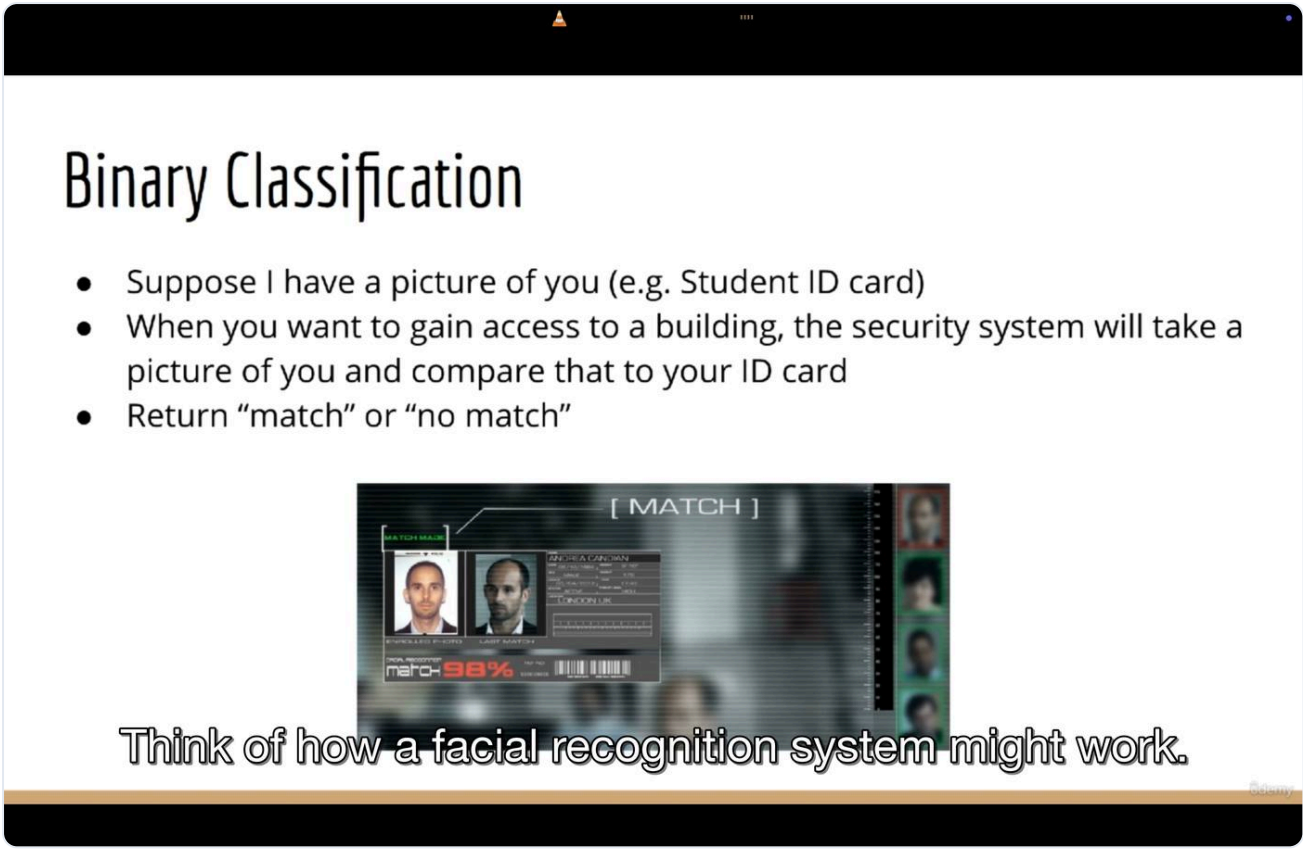

FIGURE 20.3 | Binary Classification

Card and when you walk into a building a camera will take a picture of you and compare that picture to your I.D. Card. So this is like a binary classification problem. We want to know is it a match or not. If we're given two pictures of the same person it should return a match. If we're given two pictures of different people it should say it's not a match all right. So why does this approach require less data. I don't want to get into too much detail right now but it all has to do with how we always want to consider pairs. Imagine we have three pictures a B and C.. How many possible pairs are there. We have a b aC and B.C.

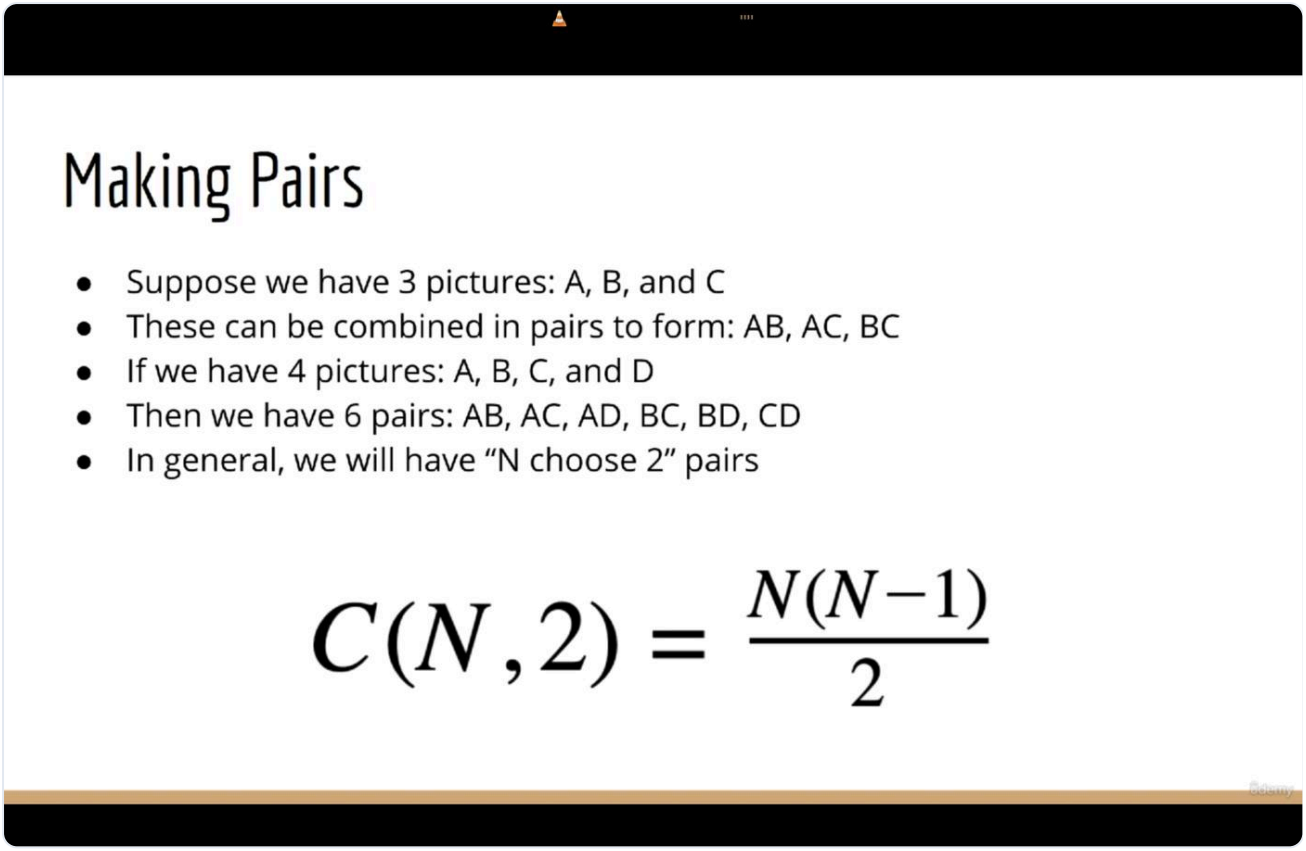

FIGURE 20.4 | Suppose We Have 3 Pictures: A, B, and C

If we have four pictures A B C and D. How many possible pairs are there. Now we have a b aC a d B.C. BD and CD that's 6. So what's the pattern. In fact this is what we call a counting problem. If we have N pictures then there are N choose two possible ways to combine different pictures.

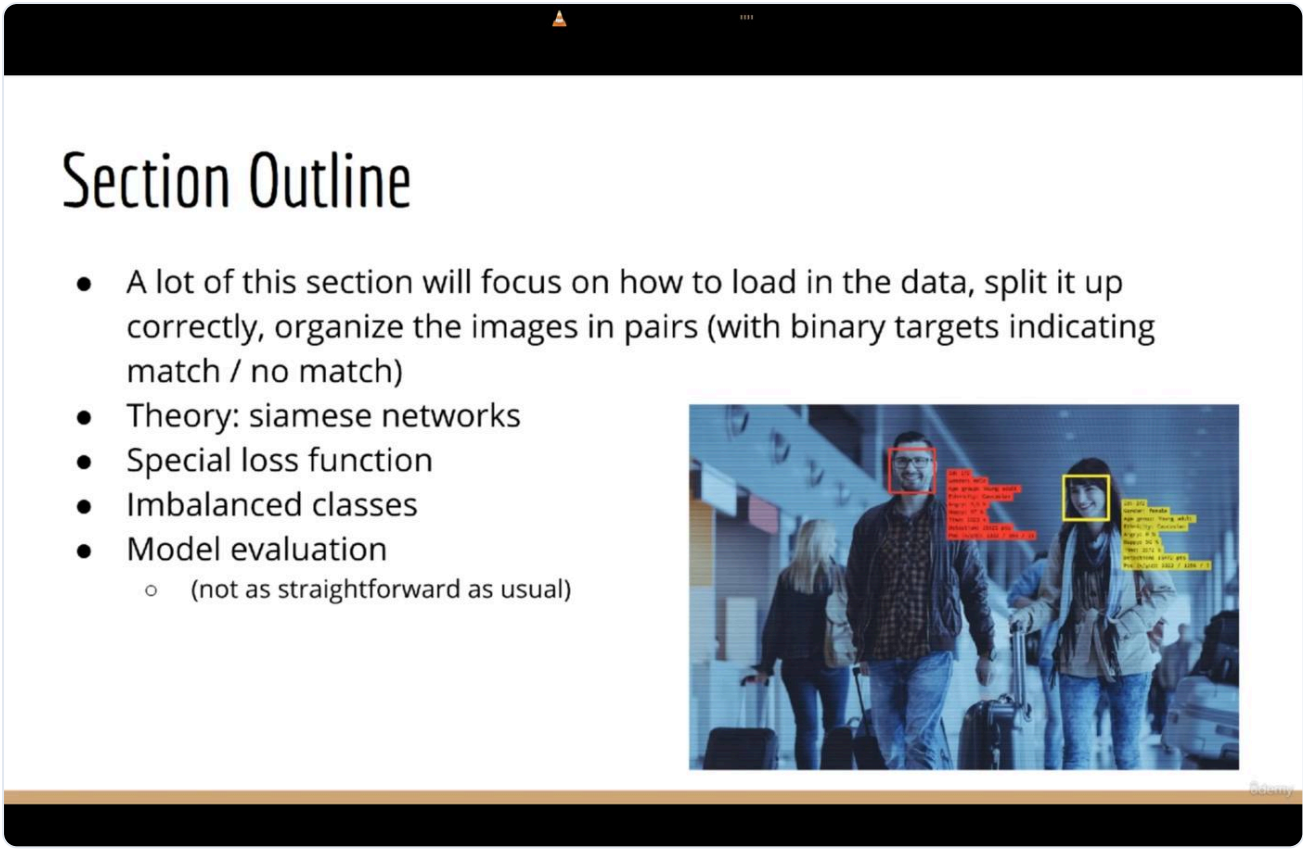

FIGURE 20.5 | Section Outline

This is equal to N times N minus one divided by two so a lot of this section is just going to be figuring out how to load in the data split it up appropriately and feed it into our facial recognition network. Of course we'll still need to discuss the theory behind the algorithm and the special loss function that we use in this scenario. You'll also learn why this approach leads to imbalanced classes and how to evaluate the model which is not as straightforward as usual. Thanks for listening and I'll see you in the next lecture.

LECTURE 114

Contrastive Loss

In this lecture we are going to discuss the theory behind the facial recognition approach. We are going to take in this section at this point. I hope you have given some thought as to how you might approach facial recognition if you had to create your own implementation. I think this is a worthwhile exercise especially if you've never seen any facial recognition approaches before. This is kind of a one time opportunity because once you do learn about how facial recognition is currently done it will taint your thinking.

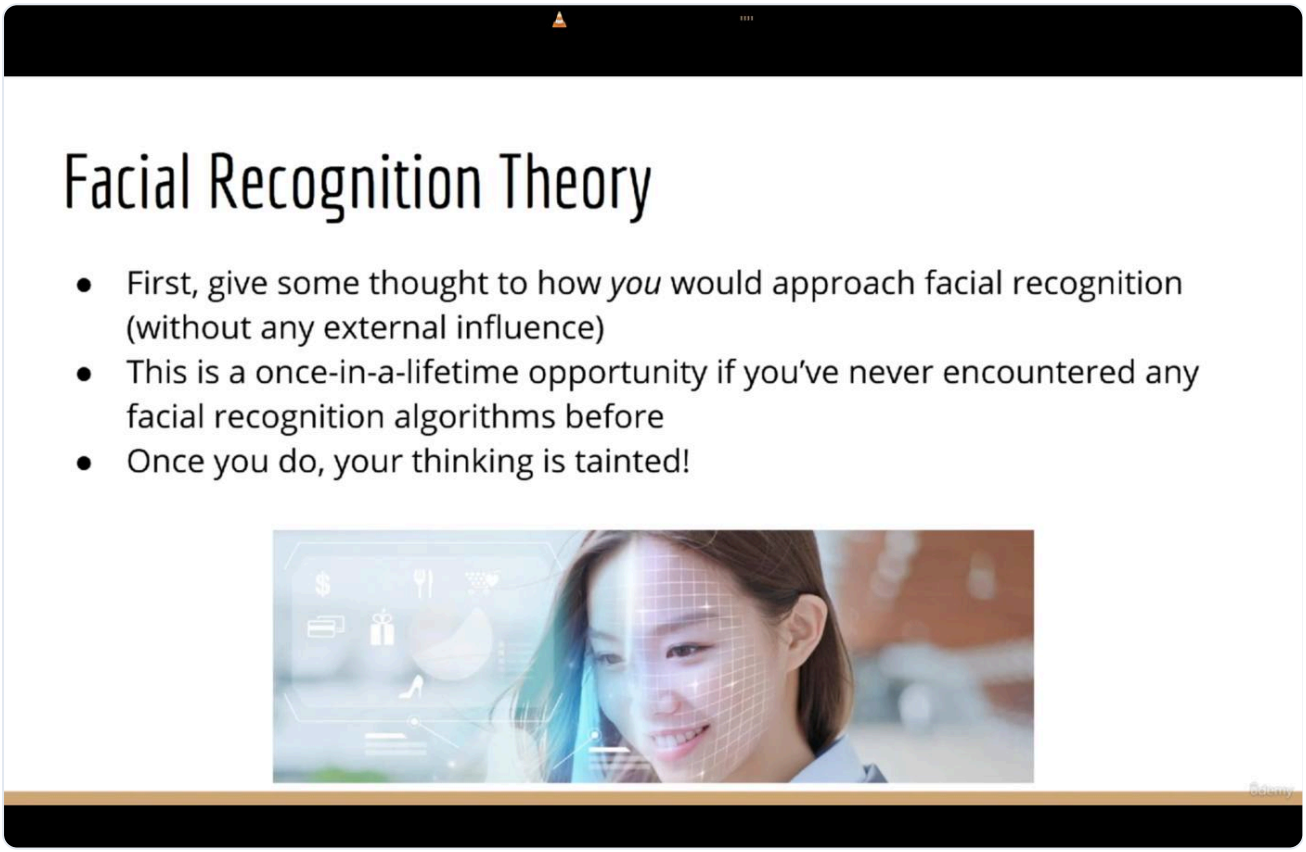

FIGURE 21.1 | Facial Recognition Theory

So take this time to think about how you would approach facial recognition without any external influences OK. So now that you've had the chance to think about how you might approach facial recognition let me describe to you what most might consider a good first crack at this problem. Facial recognition seems like nothing but image classification. Suppose for example our school has 100 students and given an image of a face.

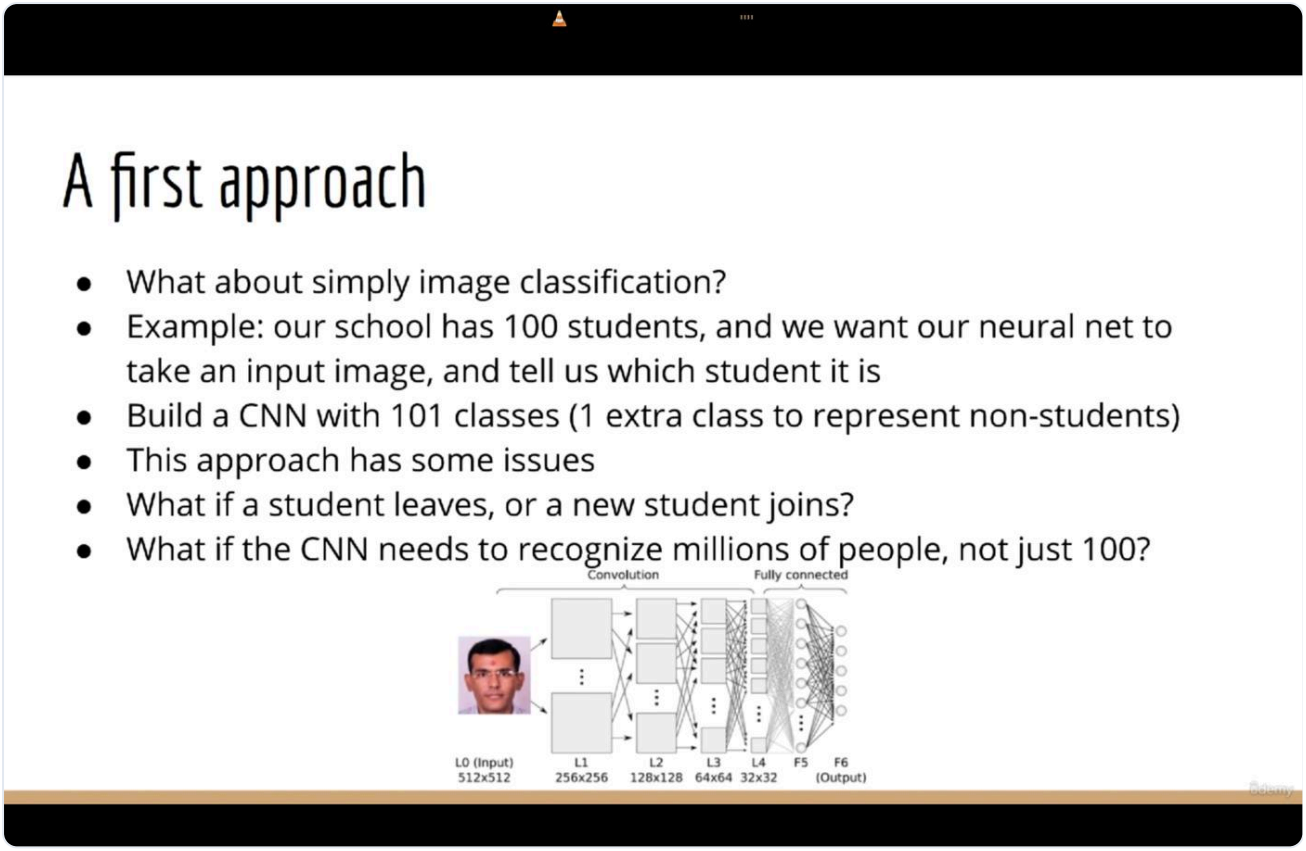

FIGURE 21.2 | A First Approach

Let's suppose we want our neural network to tell us which student that is so we might build a CNN with 101 classes where the extra class would be for images of faces which are not one of the 100 students. Now this is not necessarily a bad approach but there are some issues. For example what if one student leaves the school or another student joins the school.

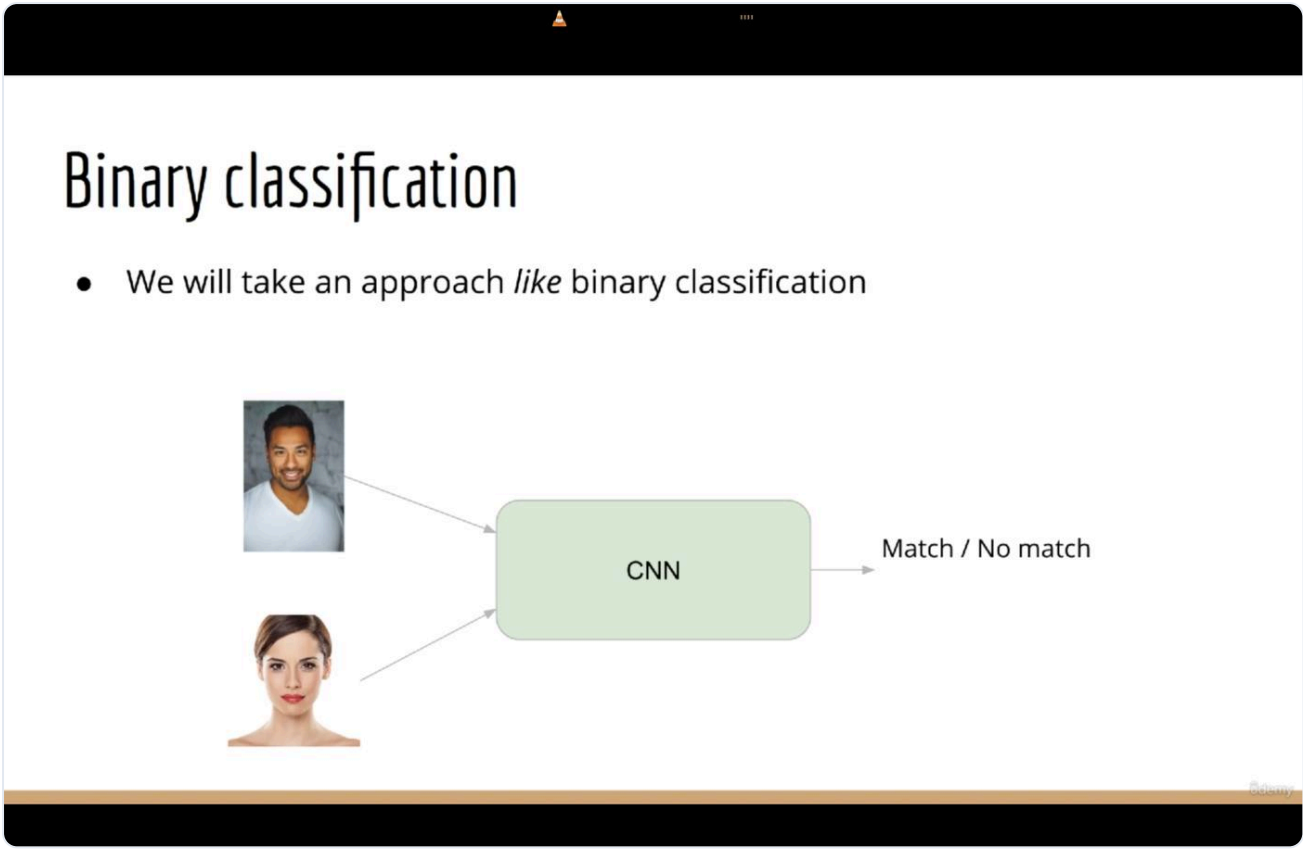

FIGURE 21.3 | Binary Classification

Now we would need to build a new classifier or what if we need our neural network to recognize millions of people having a neural network with millions of output classes is generally not a good idea. In this section we are going to take an approach similar to binary classification. Suppose we have some neural network where we can feed in two images the output of the neural network is just a binary decision.

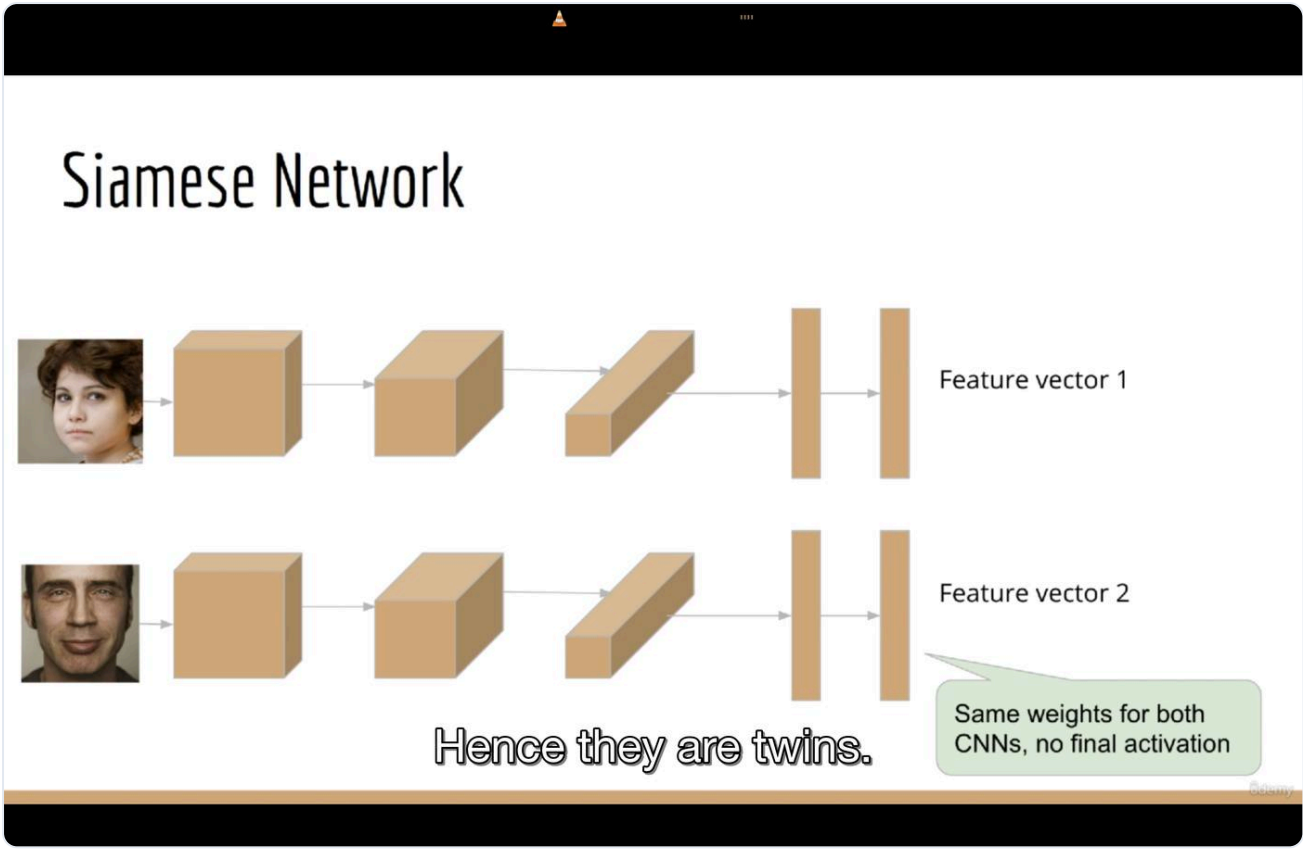

FIGURE 21.4 | Siamese Network

Are these two images of the same person or not. The architecture we will use for this approach is called a Siamese network. This is of course named after the famous Siamese twins.

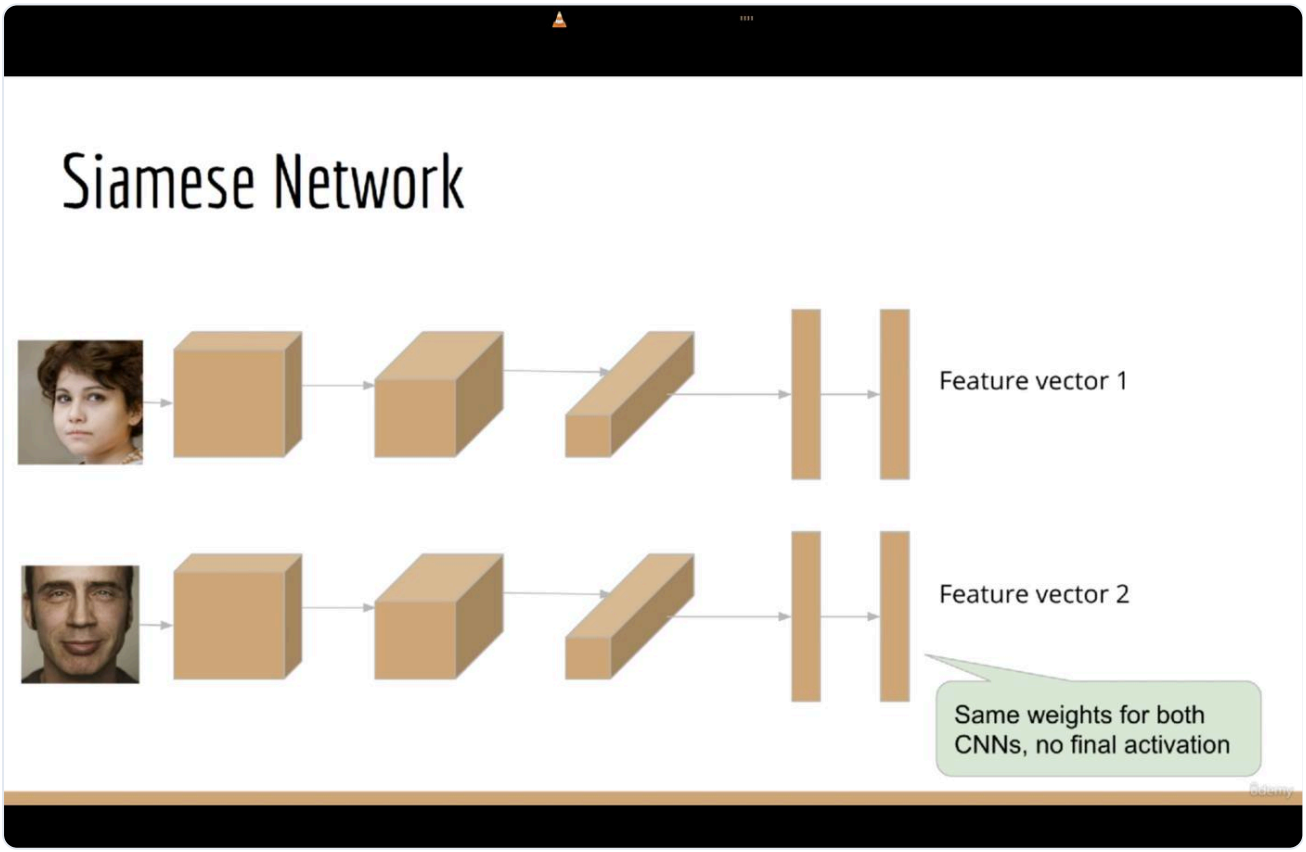

FIGURE 21.5 | Siamese Network

The idea is this we are going to have two CNNs with the exact same shared weights. Hence they are twins. Both of these CNNs are not classifiers so they don't have any classification head and they don't end in a softmax.

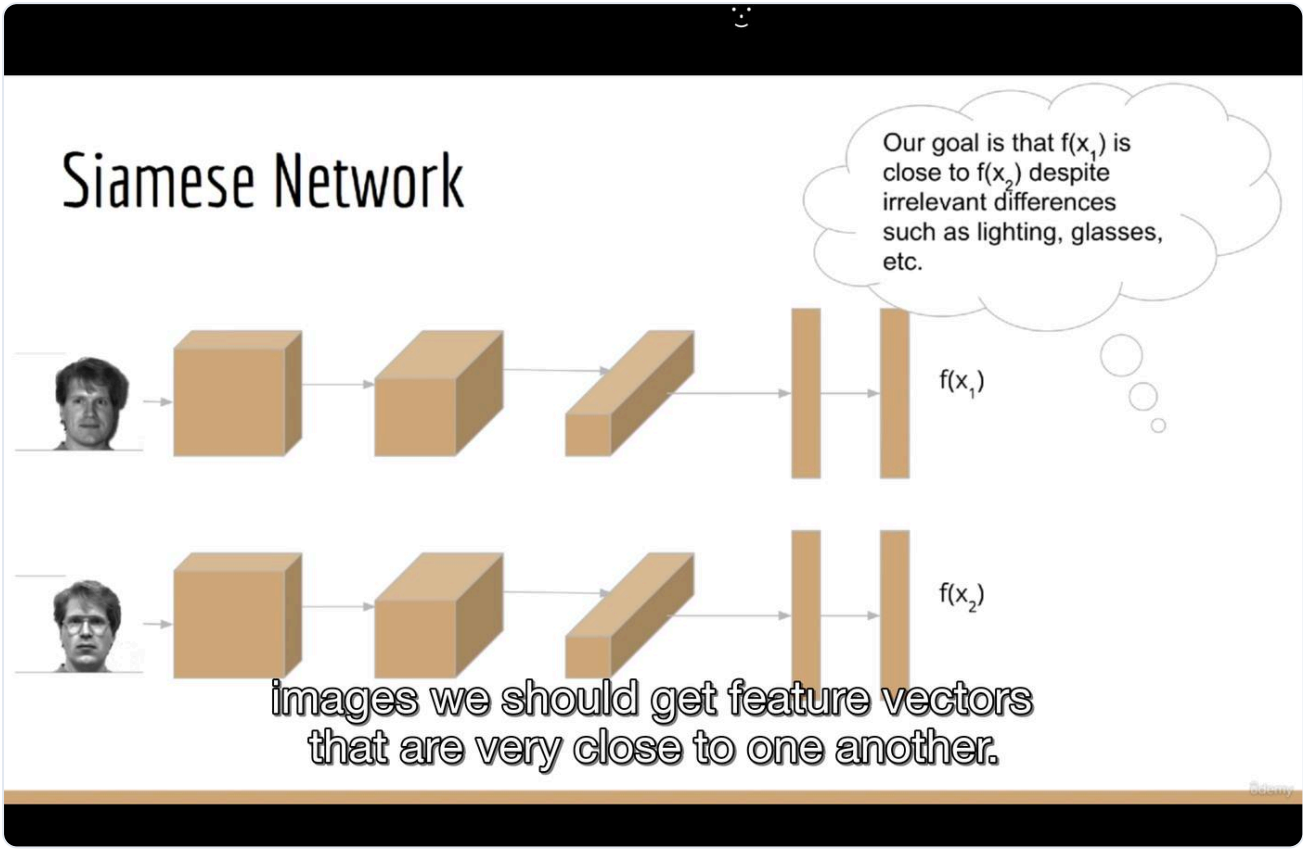

FIGURE 21.6 | Our Goal Is That (X,) Is

Instead the final layer is just a linear regression with no activation which just outputs some feature vector here's what we'd like these CNNs to do if we input two images of the same person even if they are different images we should get feature vectors that are very close to one another. So the CNN would have to be trained to ignore things like lighting conditions different angles different face sizes. Maybe one day you are wearing glasses and one day you are not.

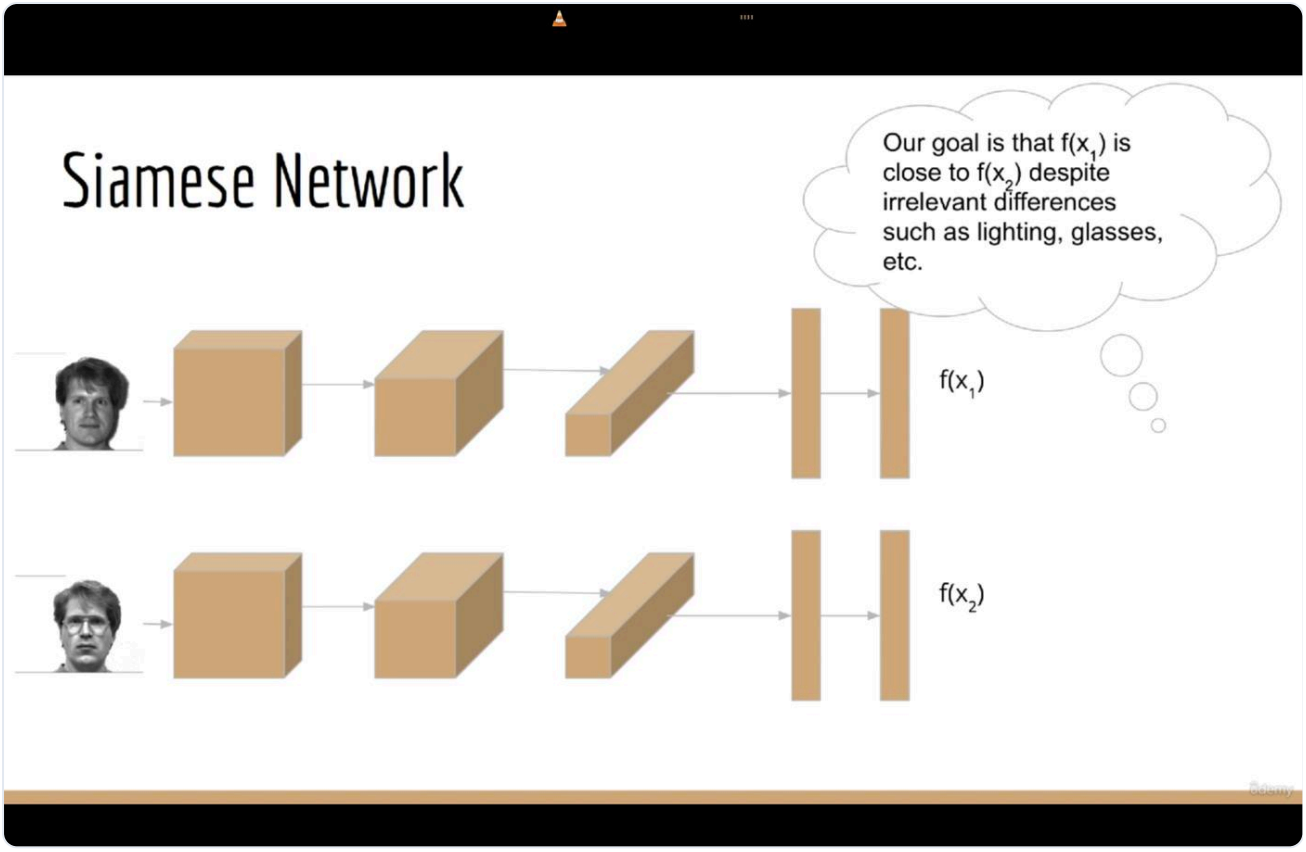

FIGURE 21.7 | Our Goal Is That F(x,) Is

And so forth. In other words the CNN has to learn the features that are actually distinctive to your face not other differences in images on the other hand if we input two images of different people the feature vectors should be far apart.

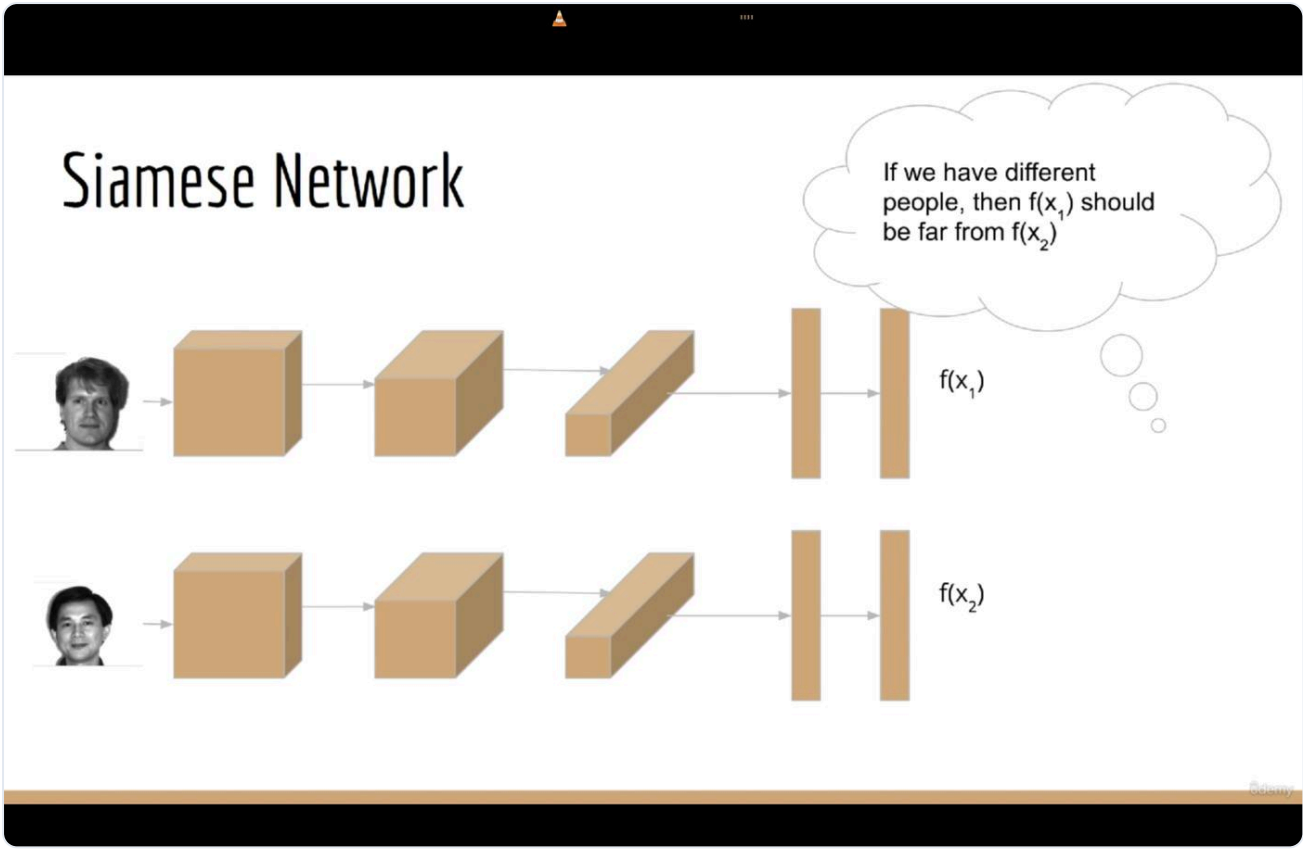

FIGURE 21.8

So even if the two images have the same lighting conditions are at the same angle have the same size and so on the CNN should learn how to distinguish those two faces using the actual features of those faces. You can think of this in terms of word embedding. We know that after training an algorithm like word2vec or GloVe we can map words into a vector space.

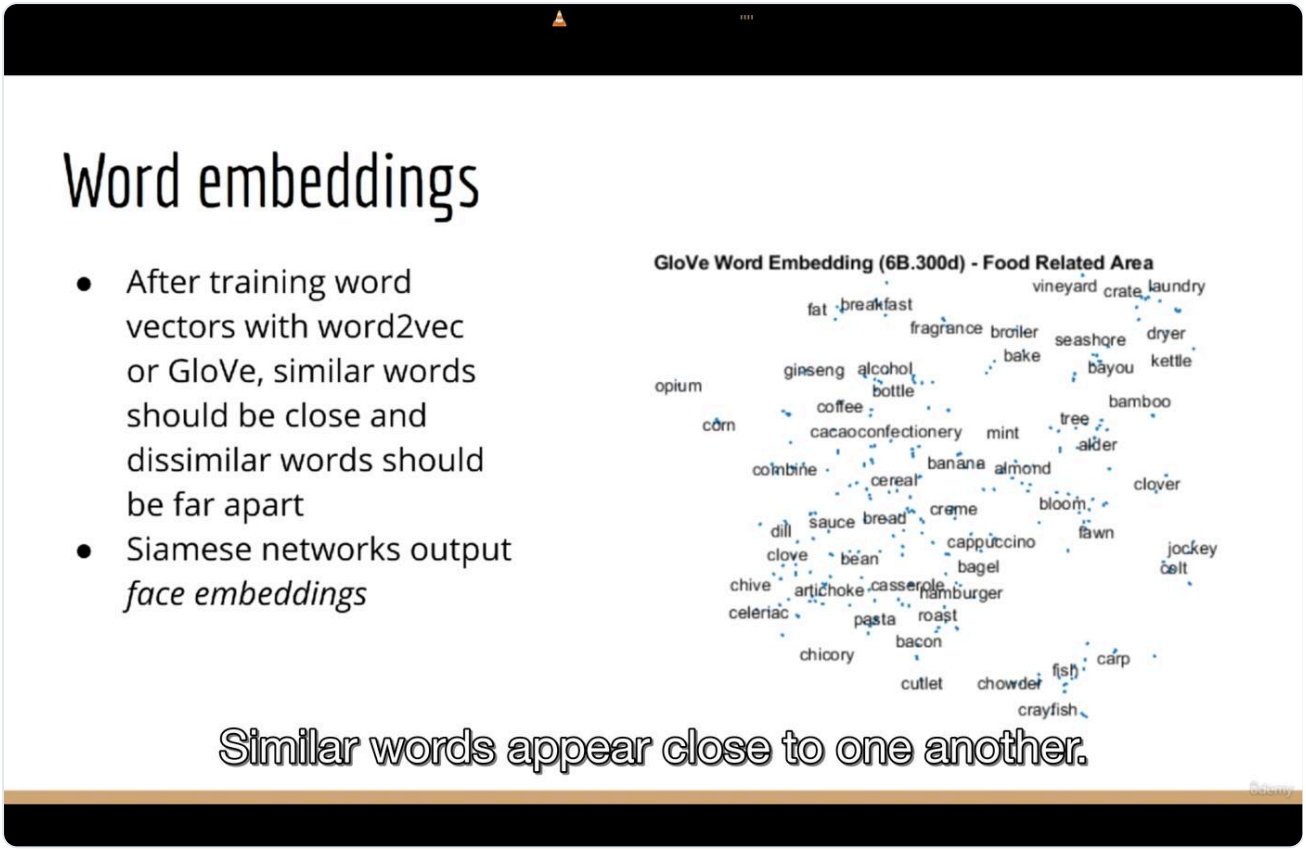

FIGURE 21.9 | Word Embeddings

Similar words appear close to one another. While dissimilar words appear far apart. For example fish should be close to carp and laundry should be close to dryer whereas fish should be far away from laundry in the same way the Siamese network outputs face embeddings. These are simply a mapping from an image of a face to a feature vector at this point. You may be wondering once we've converted both input faces into feature vectors. What do we do with them. I said earlier that we would do something like binary classification. Well in fact there's nothing stopping us from doing exactly binary classification. So let's say we take these two feature vectors and then we calculate the absolute difference between them to give us a new vector.

Then we simply pass this new vector z through a logistic regression. In this way we are doing exactly binary classification. Now you might wonder why should we take the absolute difference. Well this is because if we input image one into the first neural network and image 2 into the second neural network it should give us the same feature vector. If we reversed the images so we input image 2 into the first neural network and we input image 1 into the second neural network.

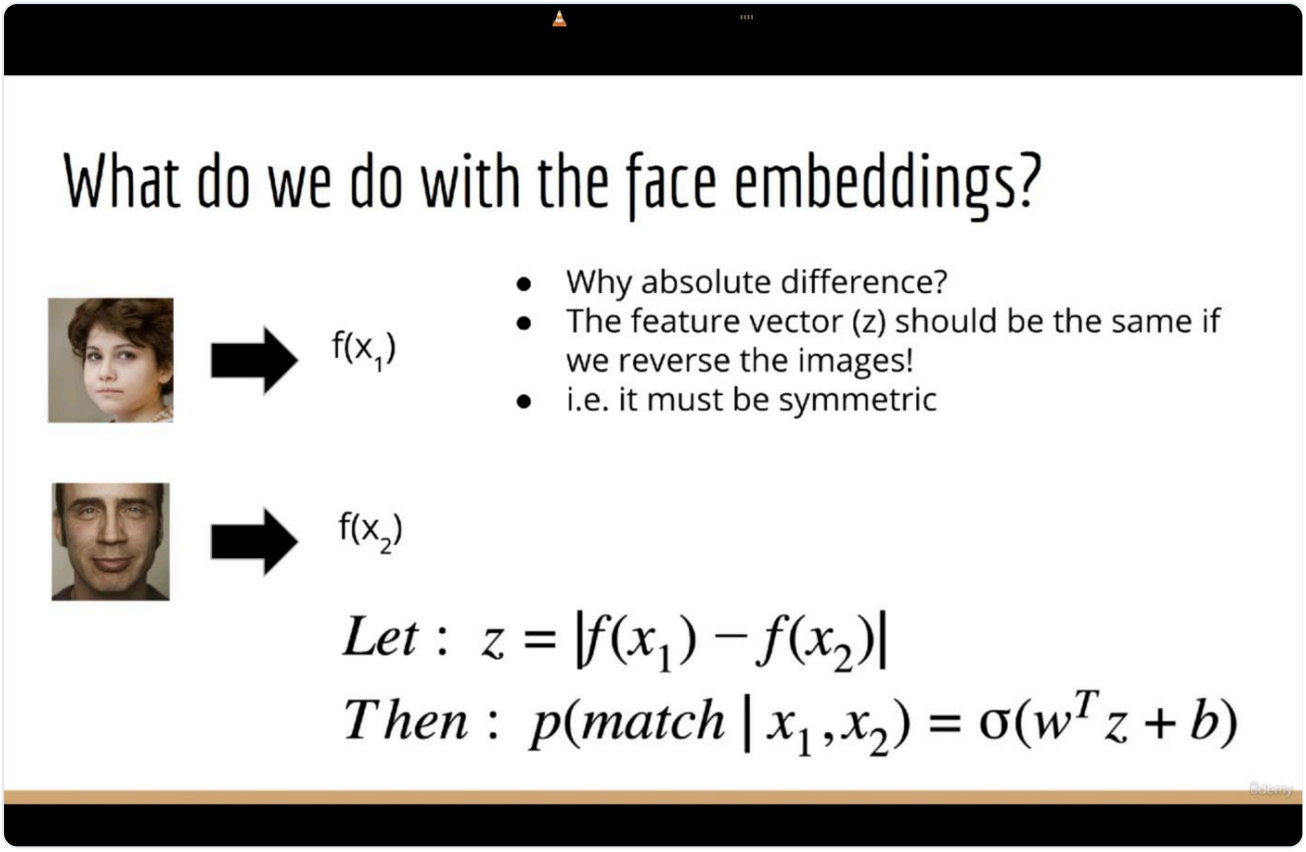

FIGURE 21.10 | What Do We Do with the Face Embeddings?

The difference between these two images should be the same in both cases and thus the feature vector must be symmetric another option here is to build the feature vector using something called the chi square similarity which uses the formula you see here.

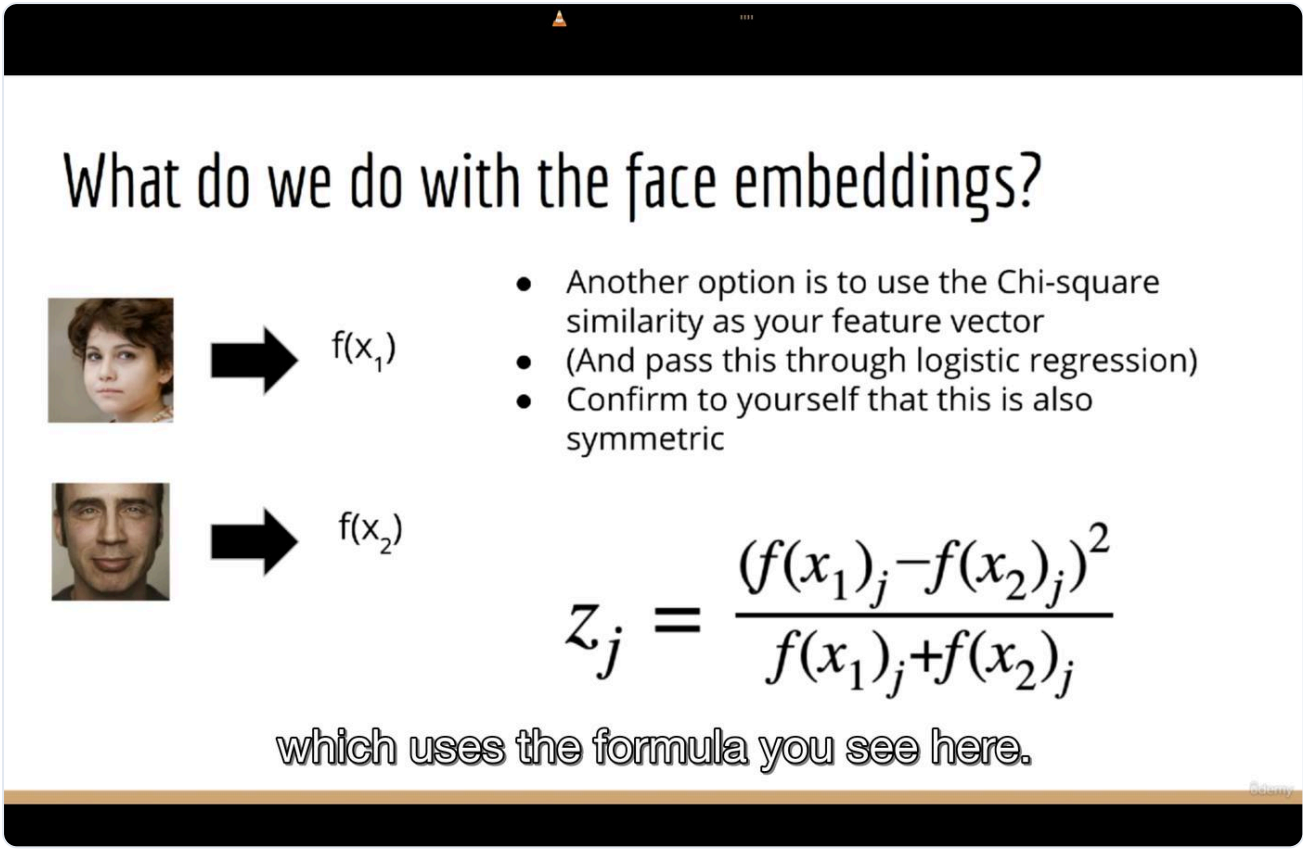

FIGURE 21.11 | What Do We Do with the Face Embeddings?

Basically it's just a different way to calculate the features you should confirm to yourself that the chi square similarity is also symmetric but for our coding exercises I would actually like to show you a different approach. This uses what is called the contrastive loss and it accomplishes exactly what our intuition wants us to do. Remember the idea is that images of the same person should have feature vectors which are close together whereas images of different people should have features which are far apart. So how can we push the feature vectors closer together or further apart. The answer is the contrastive loss.

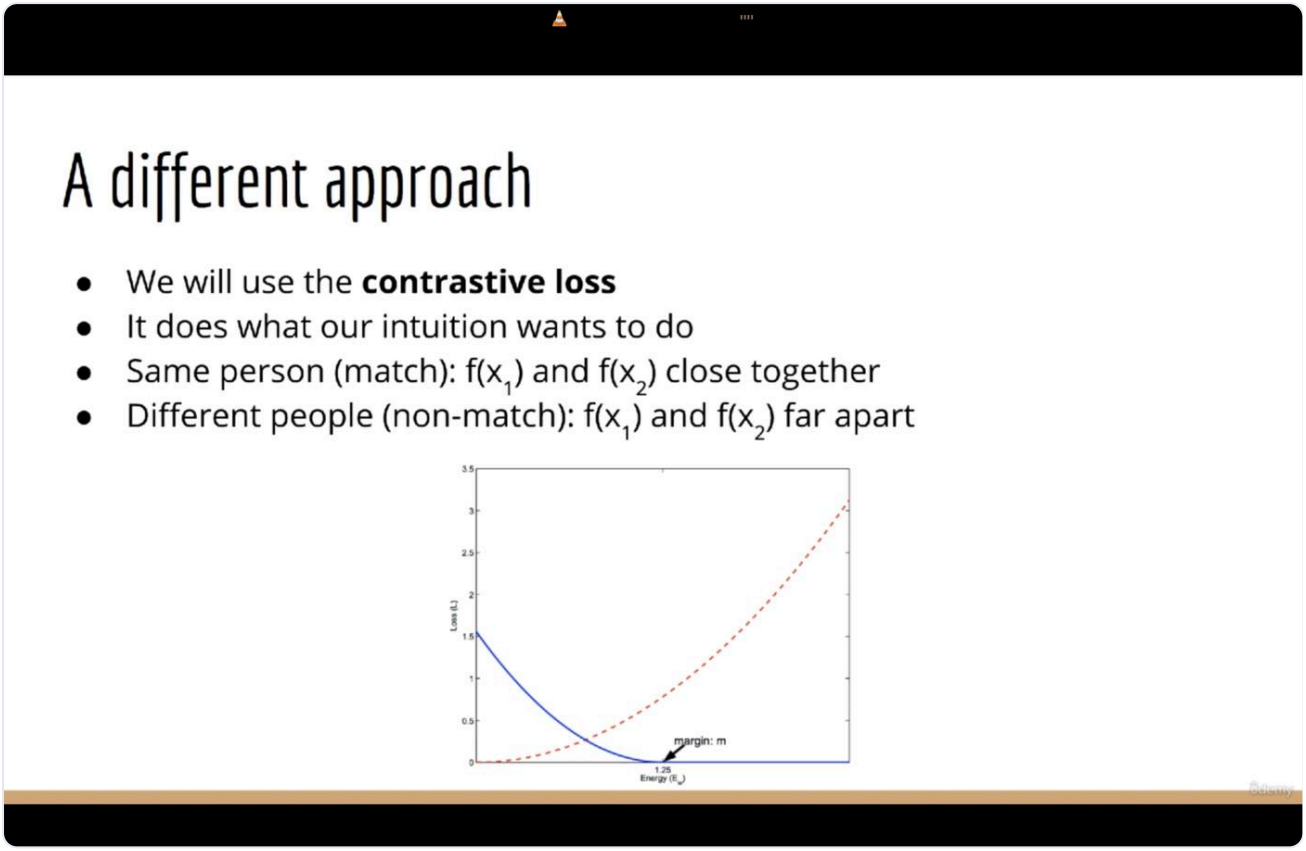

FIGURE 21.12 | A Different Approach

So how does the contrastive loss work. Well it helps to look at the formula. Basically it looks like what you would get if you combine the squared error with the binary cross entropy which kind of makes sense because this is a binary problem but we are also working with continuous feature vectors. First let's talk about how it works and then we'll talk about why it works. So let's discuss all the variables.

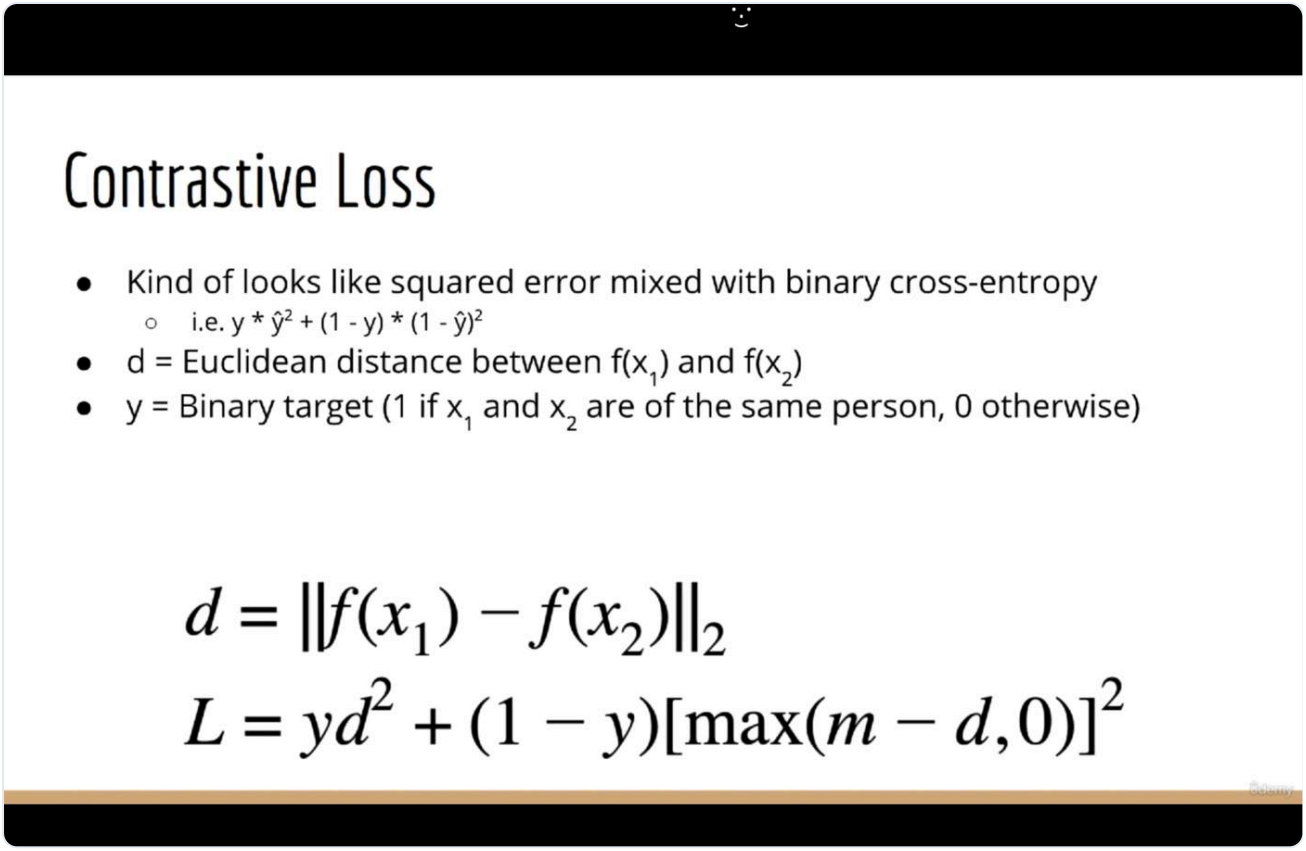

FIGURE 21.13 | Contrastive Loss

What does D that's the Euclidean distance between the two feature vectors or the two embeddings. You can also think of this as the L2 norm of f of x 1 minus f of x 2. Next what is y y is the target it's 1 if x 1 and x 2 are images of the same person and 0 otherwise next what is m m is what we refer to as the margin. This is the key ingredient of this loss. We'll discuss this more when we talk about why this loss works finally. Note that this loss is specified for a single training sample which consists of a pair of images x 1 and x 2.

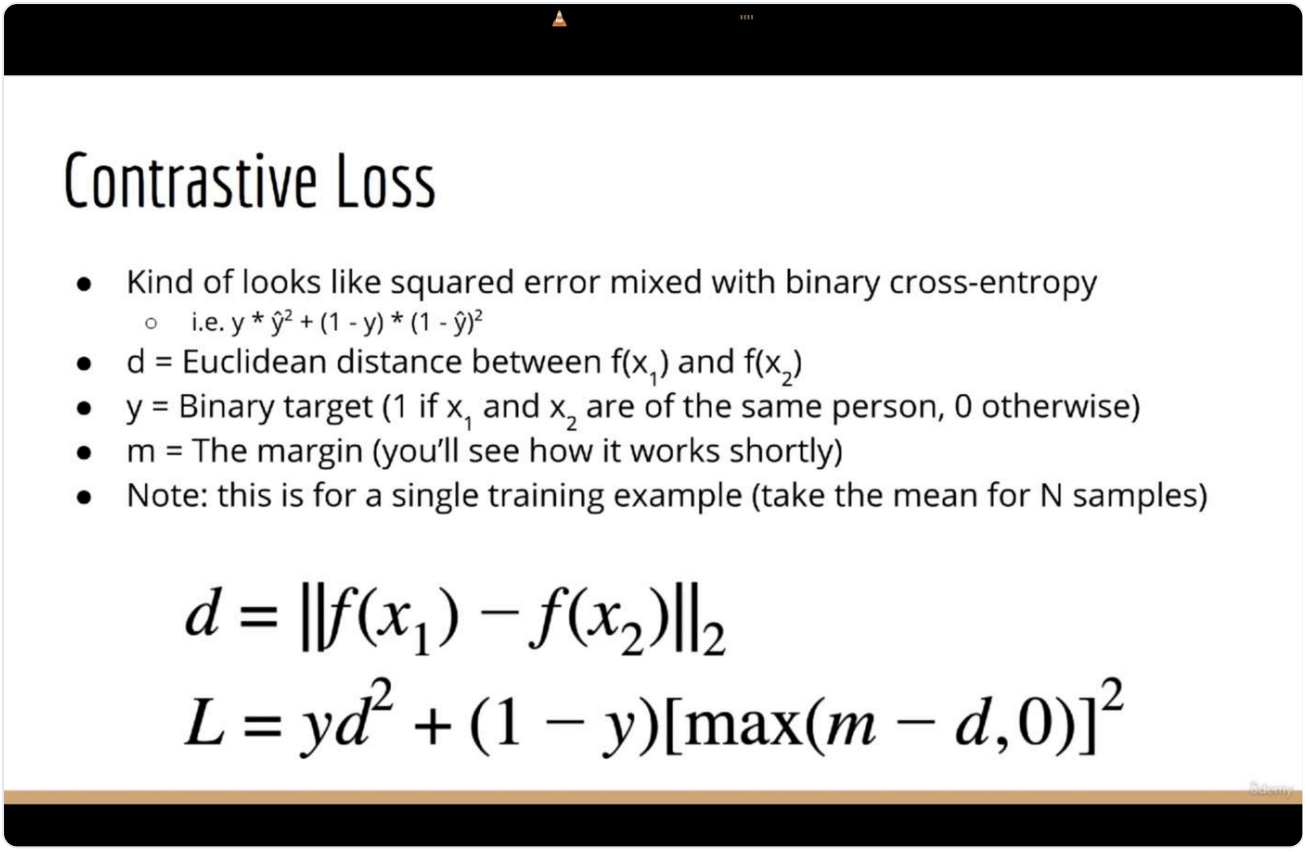

FIGURE 21.14 | Contrastive Loss

So if we have multiple training samples then we would just take the mean of this okay. So now that you know what each of the variables in the loss means why does it actually work. Let's look at the first term since it's a little simpler. It's y times D squared. Remember that y can only be one or zero it's one when the two images are of the same person.

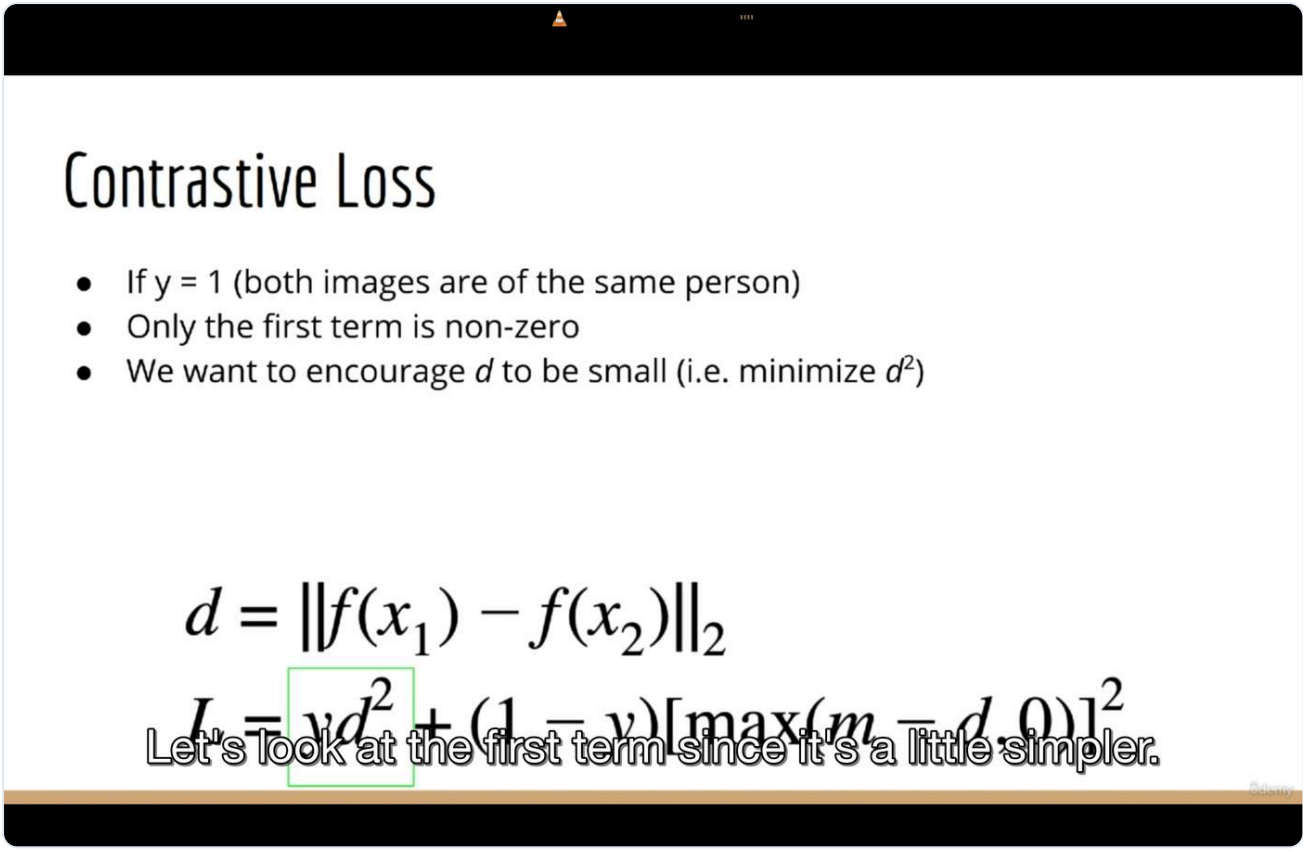

FIGURE 21.15 | Contrastive Loss

Therefore when both images are of the same person only the first term matters and we want D squared to be small. We want to minimize D squared if both images are of the same person. The other scenario is when we have images of different people when y is 0. Only the second term matters. Notice how we are taking the max of m minus D and 0 basically it say if D is bigger than M then m minus D will be negative. So just take the loss to be zero.

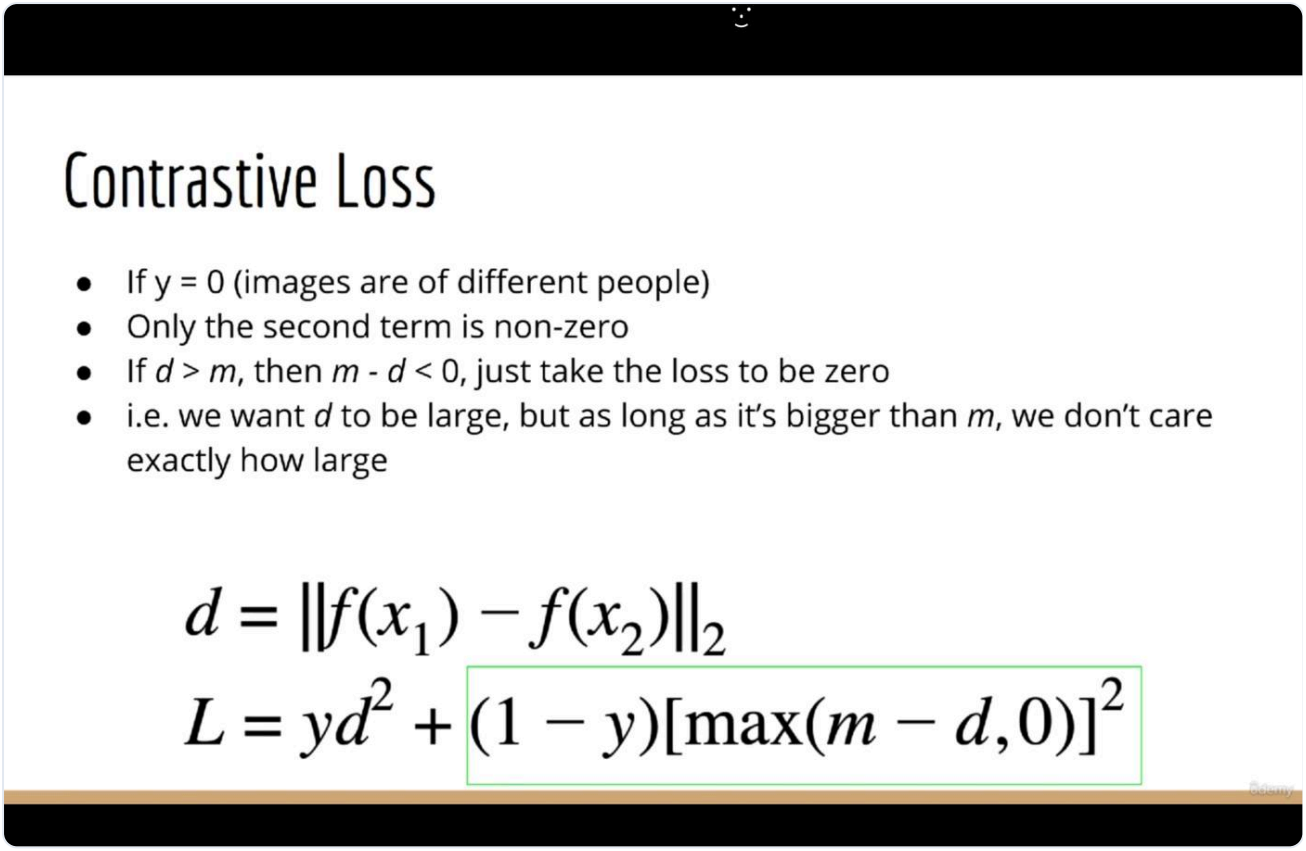

FIGURE 21.16 | Contrastive Loss

Thus it saying if we have two images of different people we want D to be large but as long as it's greater than or equal to M we don't really care how large exactly it is. So we don't want to push D to infinity just bigger than M is good enough. What is the result of this loss.

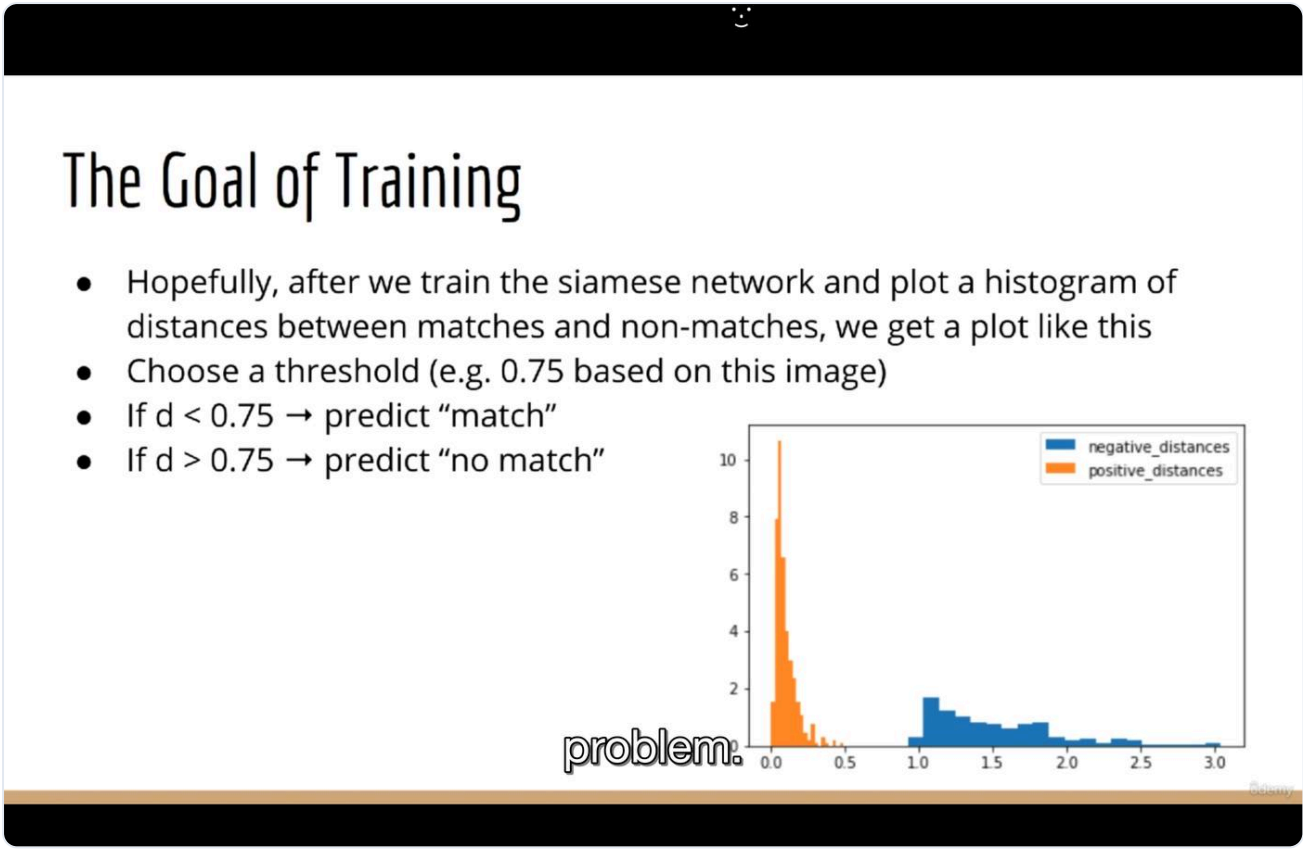

FIGURE 21.17 | The Goal of Training

Well hopefully if we draw a histogram of the distances we get when we feed in images of matches and non matches it will look like this. You can see that we get two distributions. The distribution for the matches should hopefully be very close to zero and the distribution for the non matches should hopefully all be equal to or greater than M. The idea is once we can see a plot like this we know how to classify whether or not two images are a match. We can choose a threshold for example with this plot. That might be zero point seven five. Then when we plug in two images into our Siamese network and we get a distance less than zero point seventy five we can predict that they are a match.

If it's bigger than zero point seven five we can predict that it's not a match. As a side note you can see that we are going to set m to 1 which seems to work well enough for this problem. You are of course encouraged to try different hyperparameters on your own.

LECTURE 115

Siamese Networks Code Outline

As I mentioned in the introduction to this section the theory behind Siamese networks is not all that difficult.

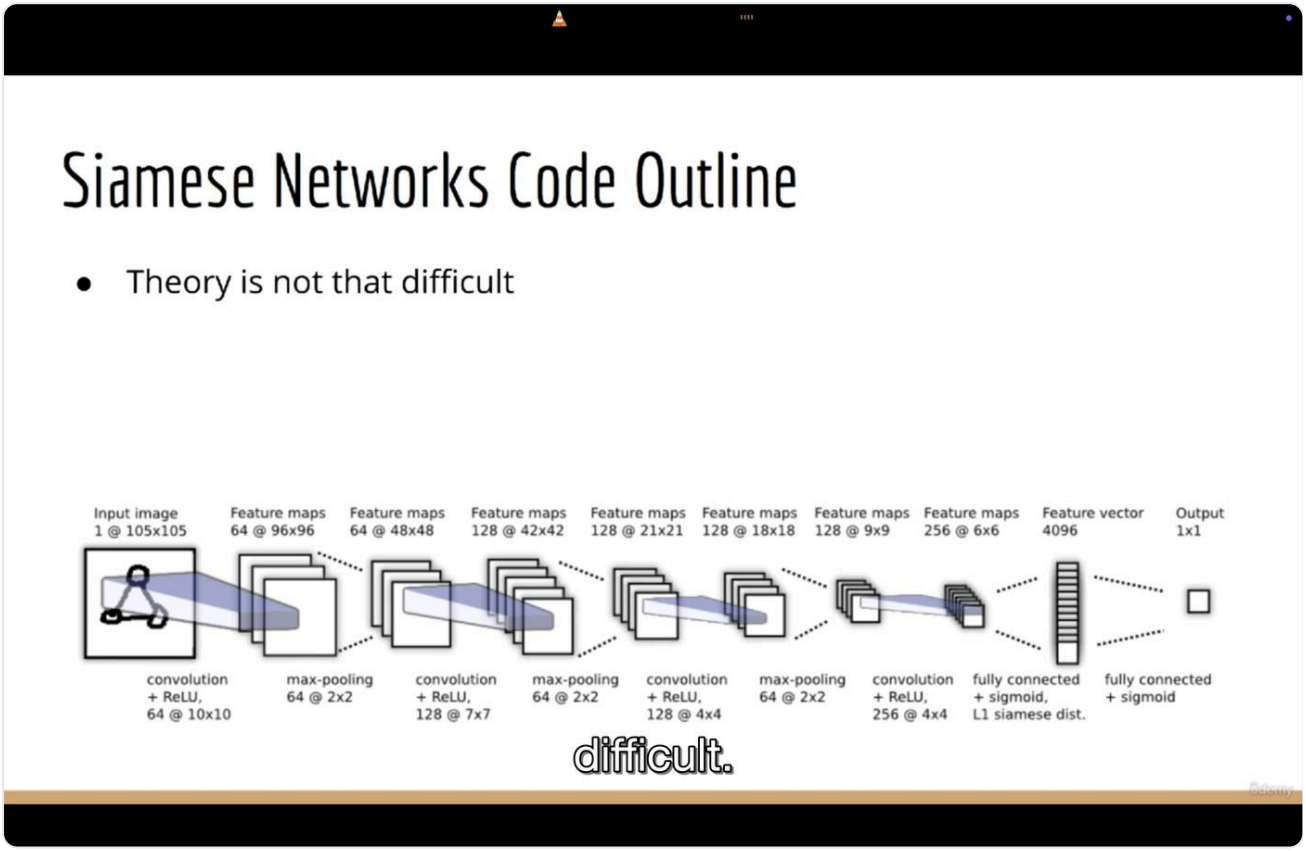

FIGURE 22.1 | Siamese Networks Code Outline

It's more complicated than a CNN you would use for image classification but it still follows the same steps. We have a system of CNNs we set up a loss and then we minimize that loss.

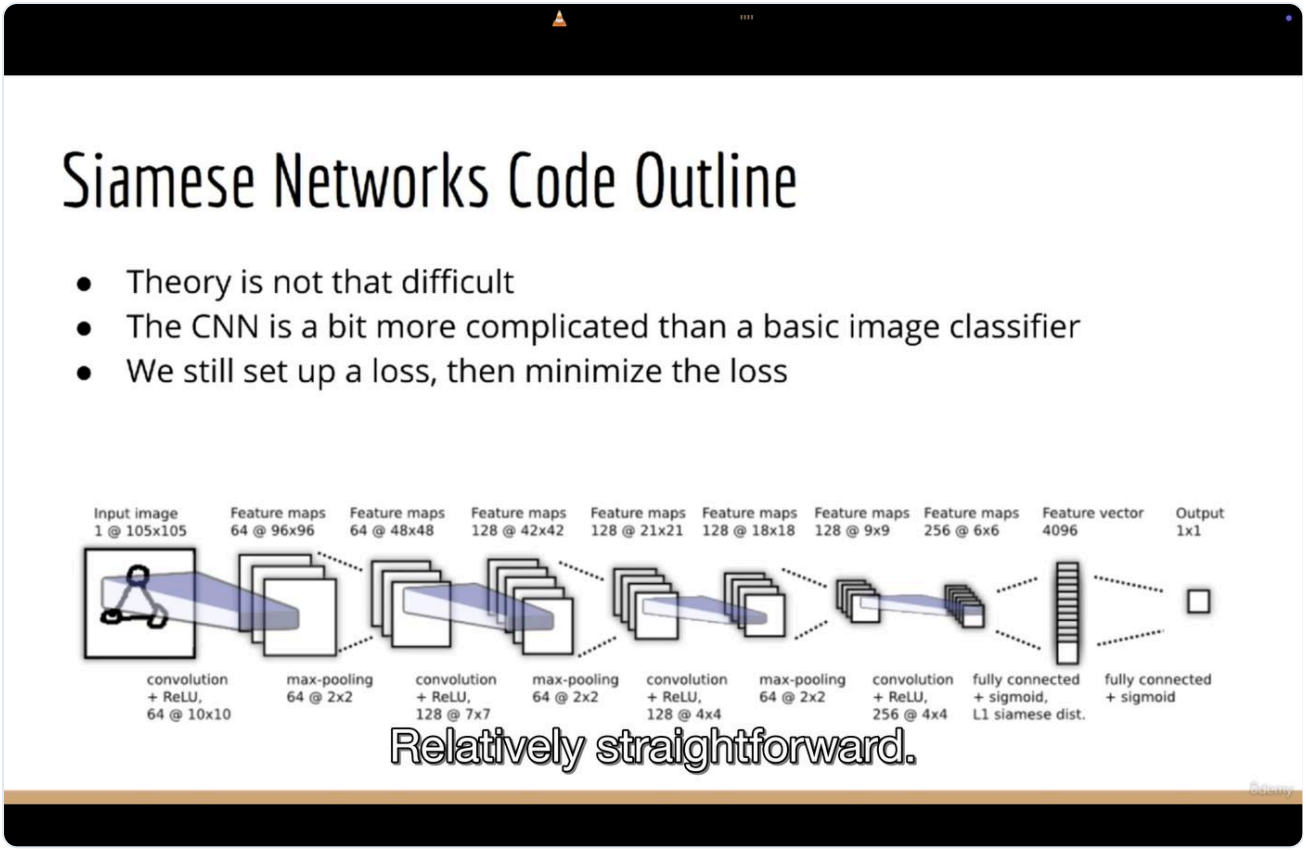

FIGURE 22.2 | Siamese Networks Code Outline

Relatively straightforward.

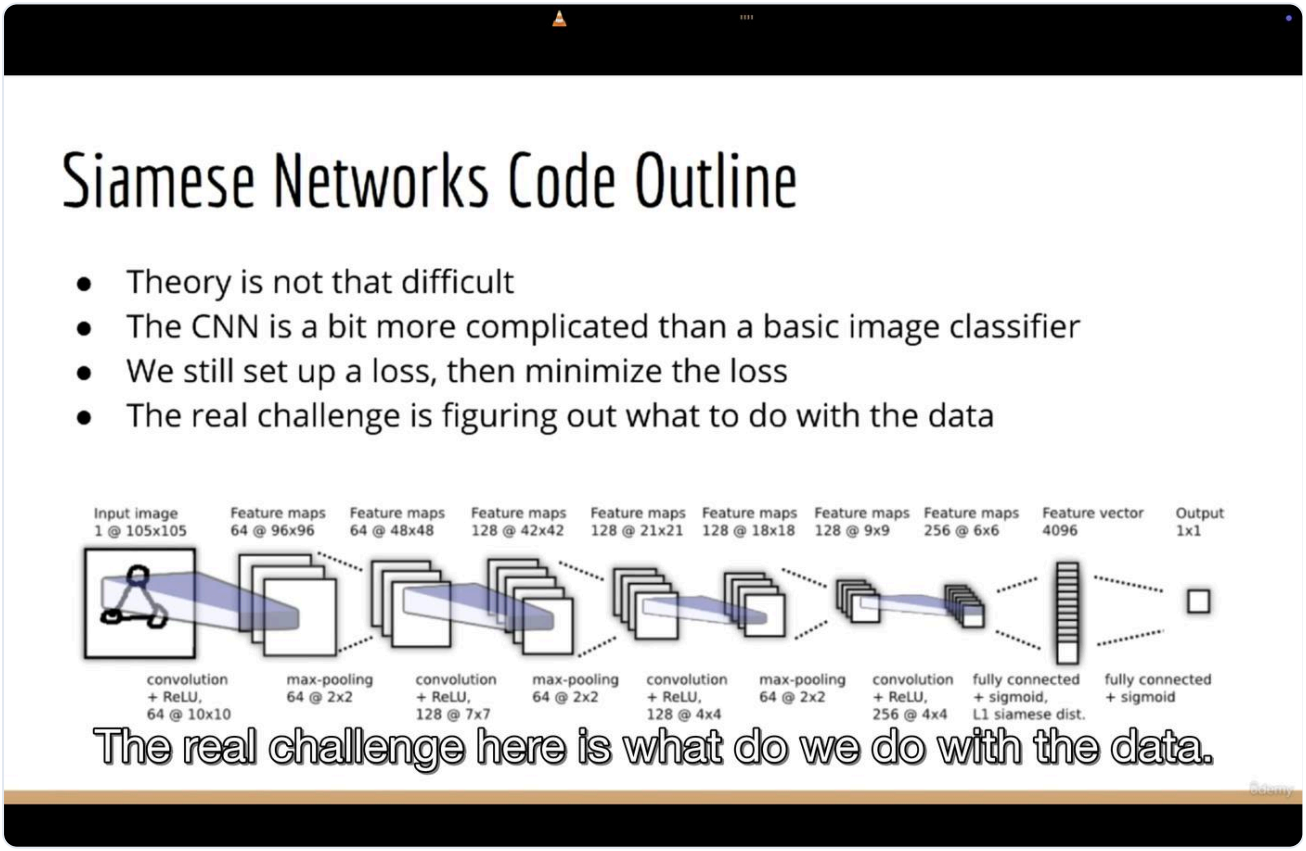

FIGURE 22.3 | Siamese Networks Code Outline

The real challenge here is what do we do with the data.

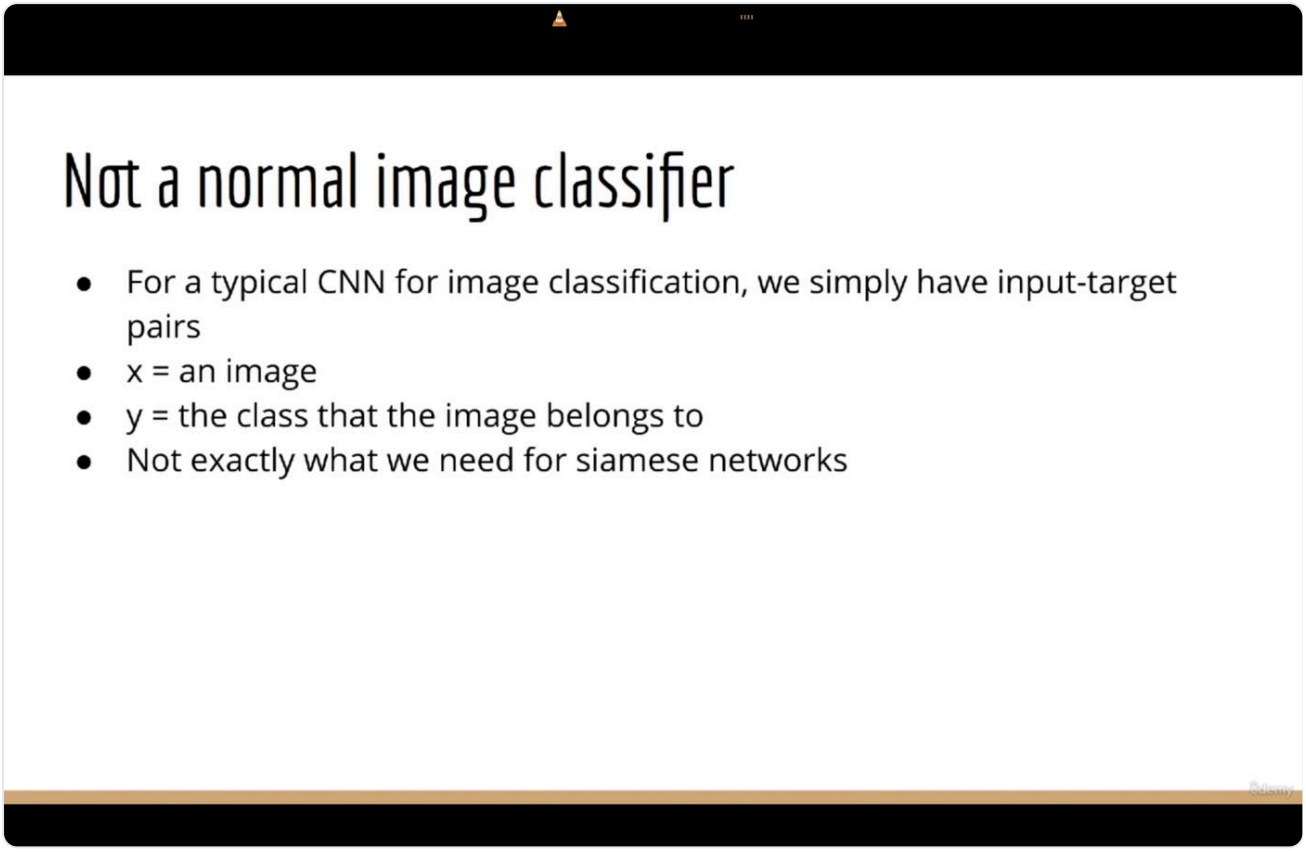

FIGURE 22.4

Normally when we train a CNN for image classification we have a simple dataset of input target pairs for x. We have a single image and for y we have the class which the image belongs to. But what happens with a Siamese network. We don't have any label for a specific image necessarily or at least not the labels we will use directly here's what will be given. We're still going to get a list of images for each image. We will be told their identity. For example the first image belongs to subject one.

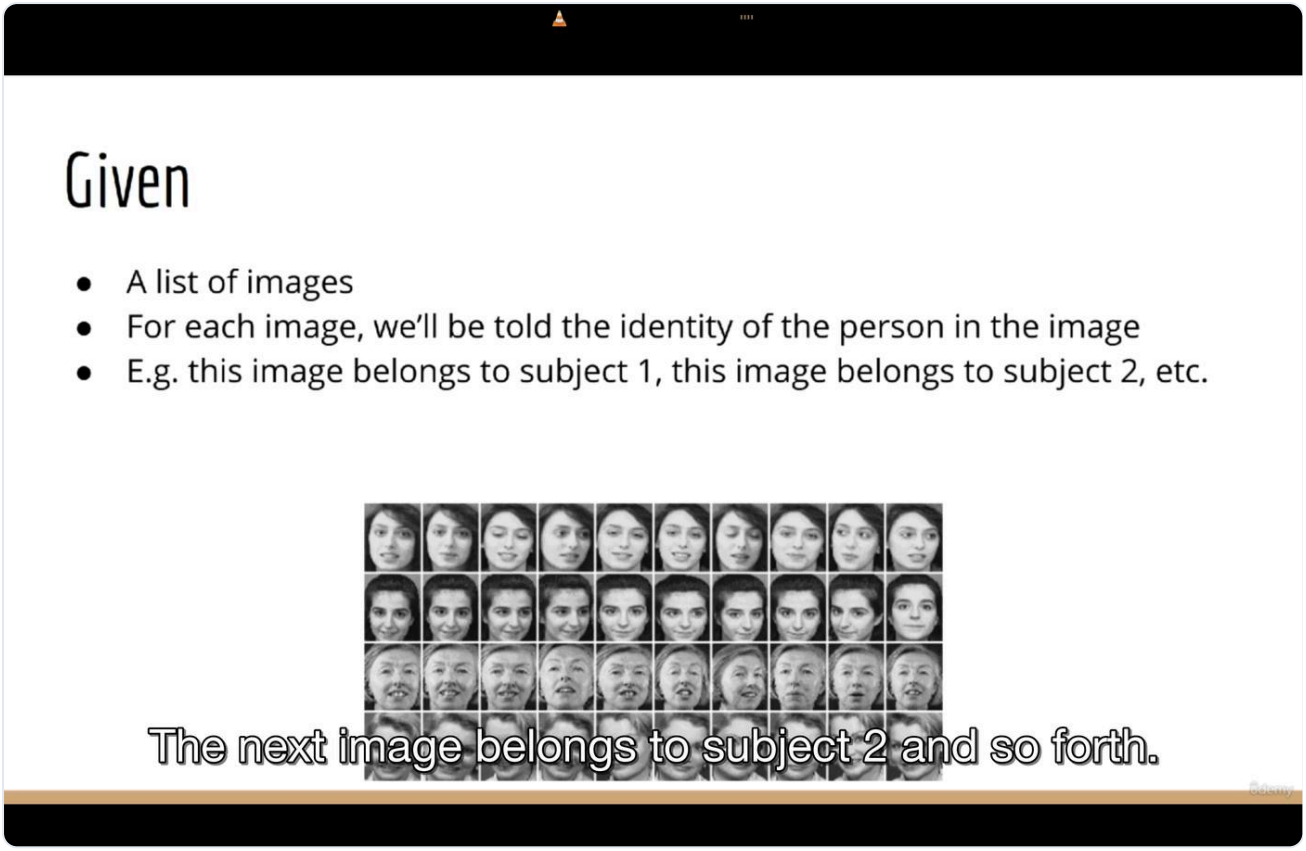

FIGURE 22.5

The next image belongs to subject 2 and so forth.

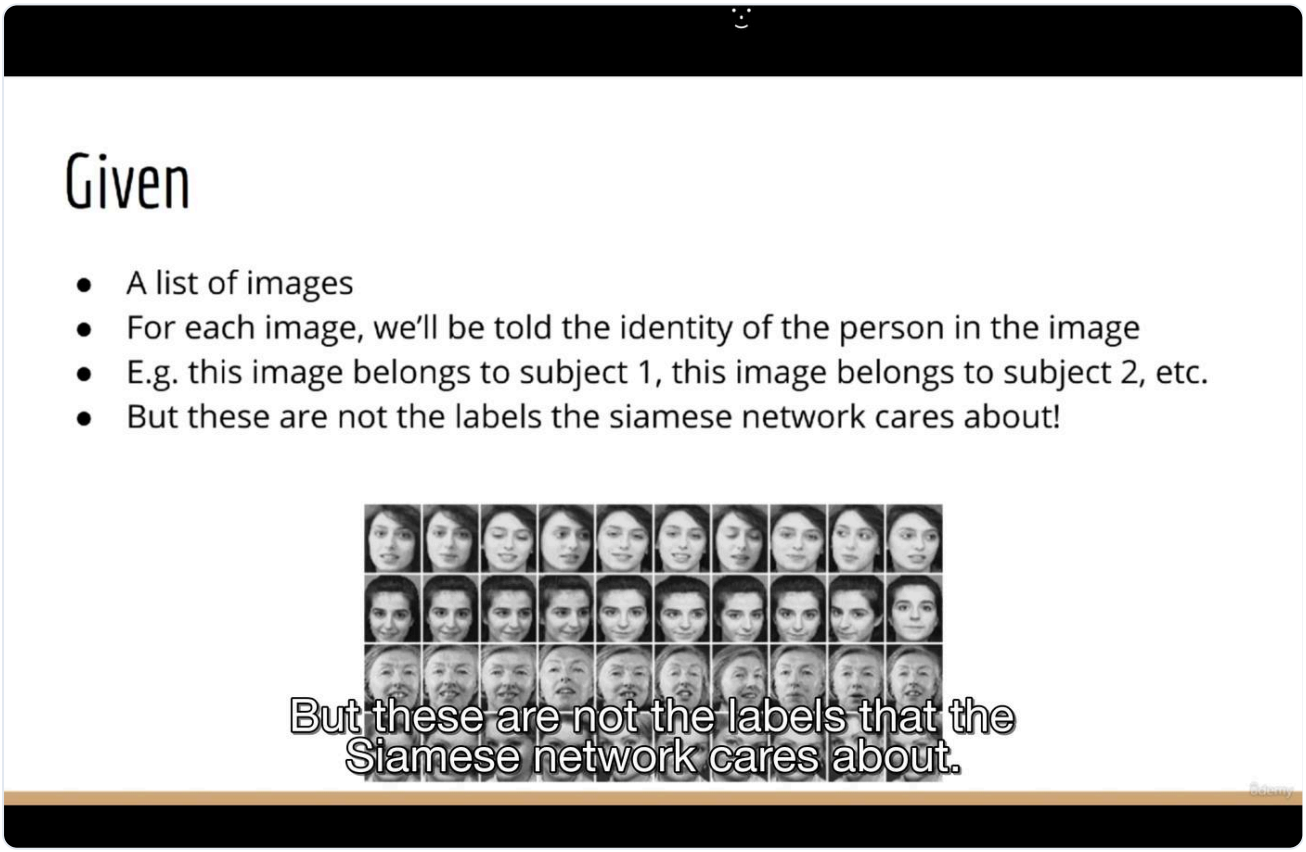

FIGURE 22.6 | A List of Images

But these are not the labels that the Siamese network cares about.

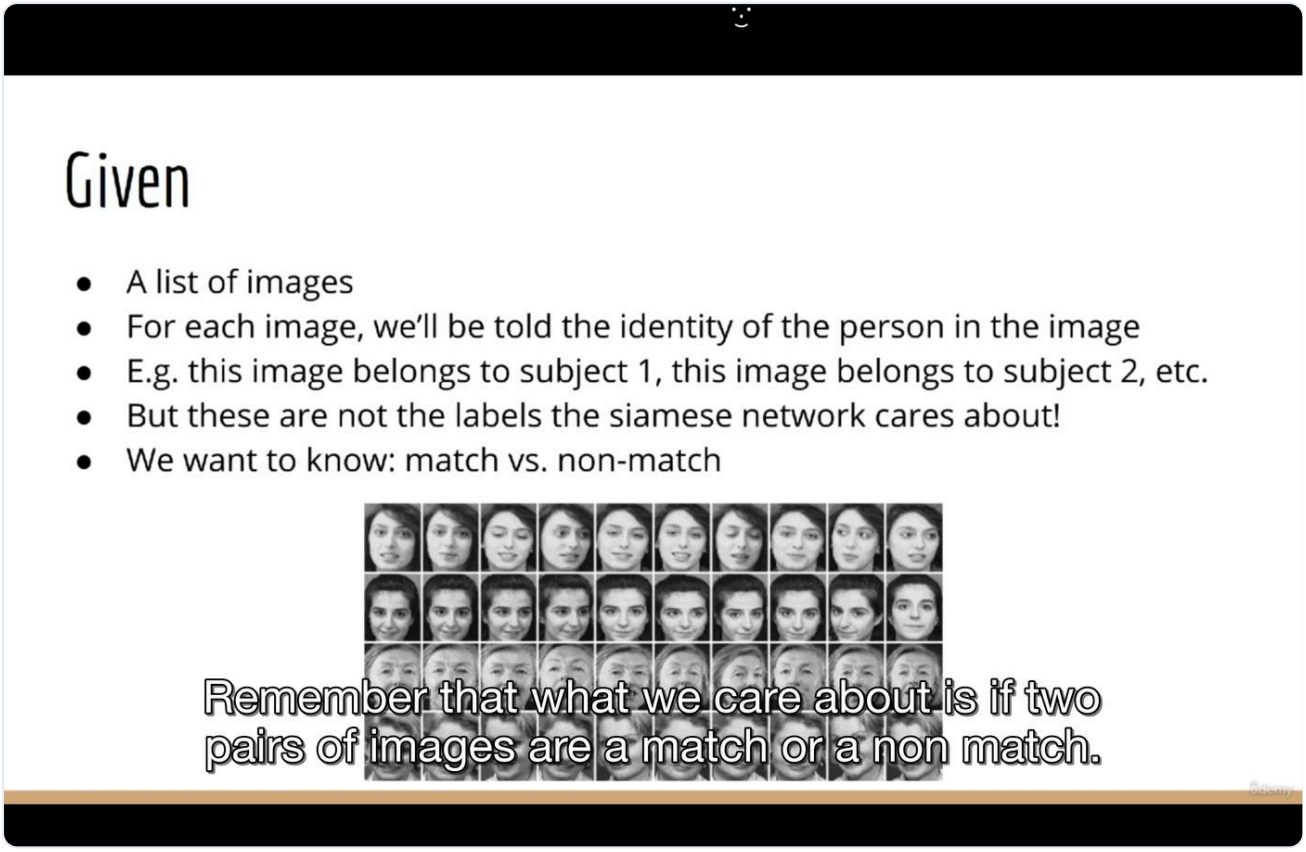

FIGURE 22.7 | A List of Images

Remember that what we care about is if two pairs of images are a match or a non match. So how do we do this what we have to make our own dataset to keep things simple let's say we have three subjects and we have two images for each subject.

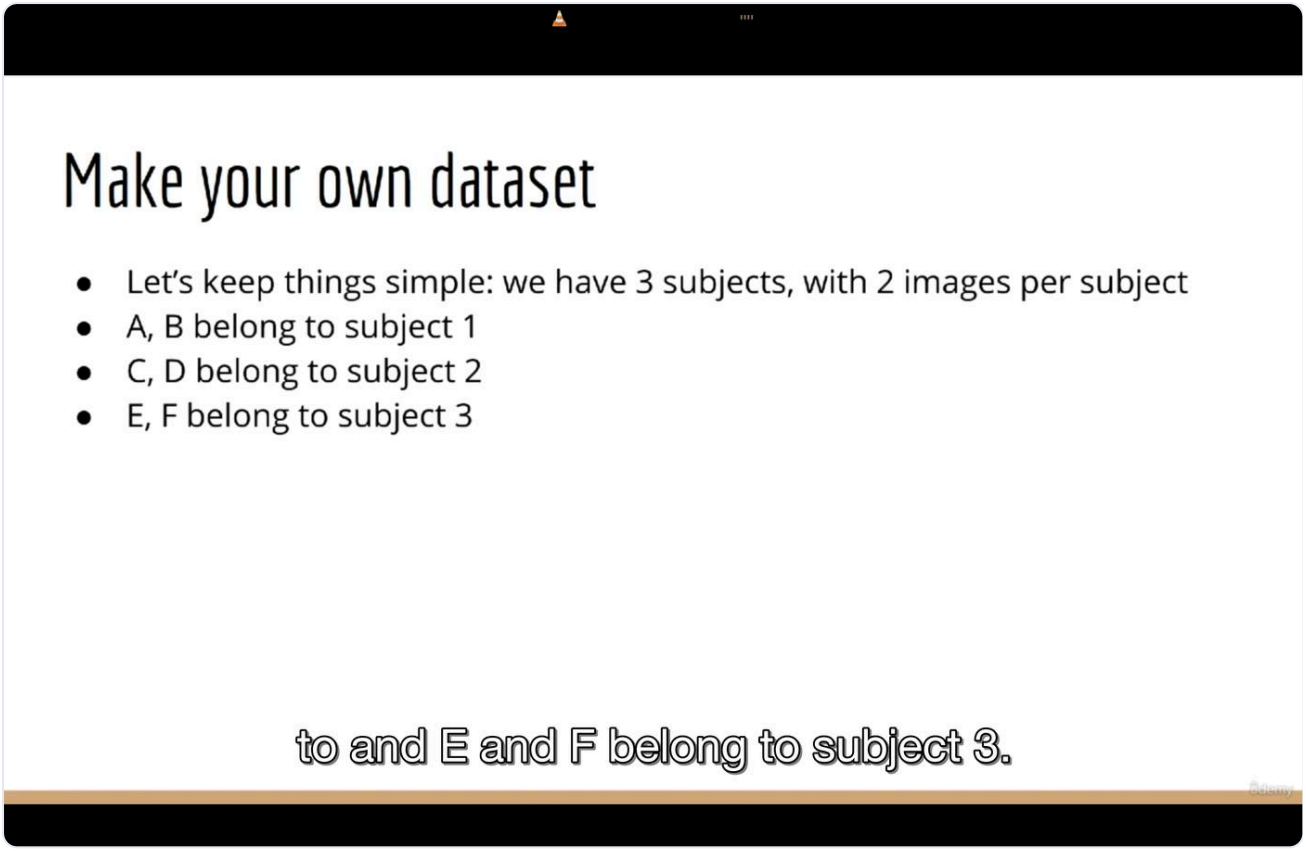

FIGURE 22.8 | Make Your Own Dataset

Let's call these images a b c d e and f so A and B belong to subject one C and D belong to subject 2 and E and F belong to subject 3. Now this becomes a counting problem. What are all the possible ways we can combine these images into pairs if you recall when we have N images. We should end up with N choose two pairs. So let's see what we get.

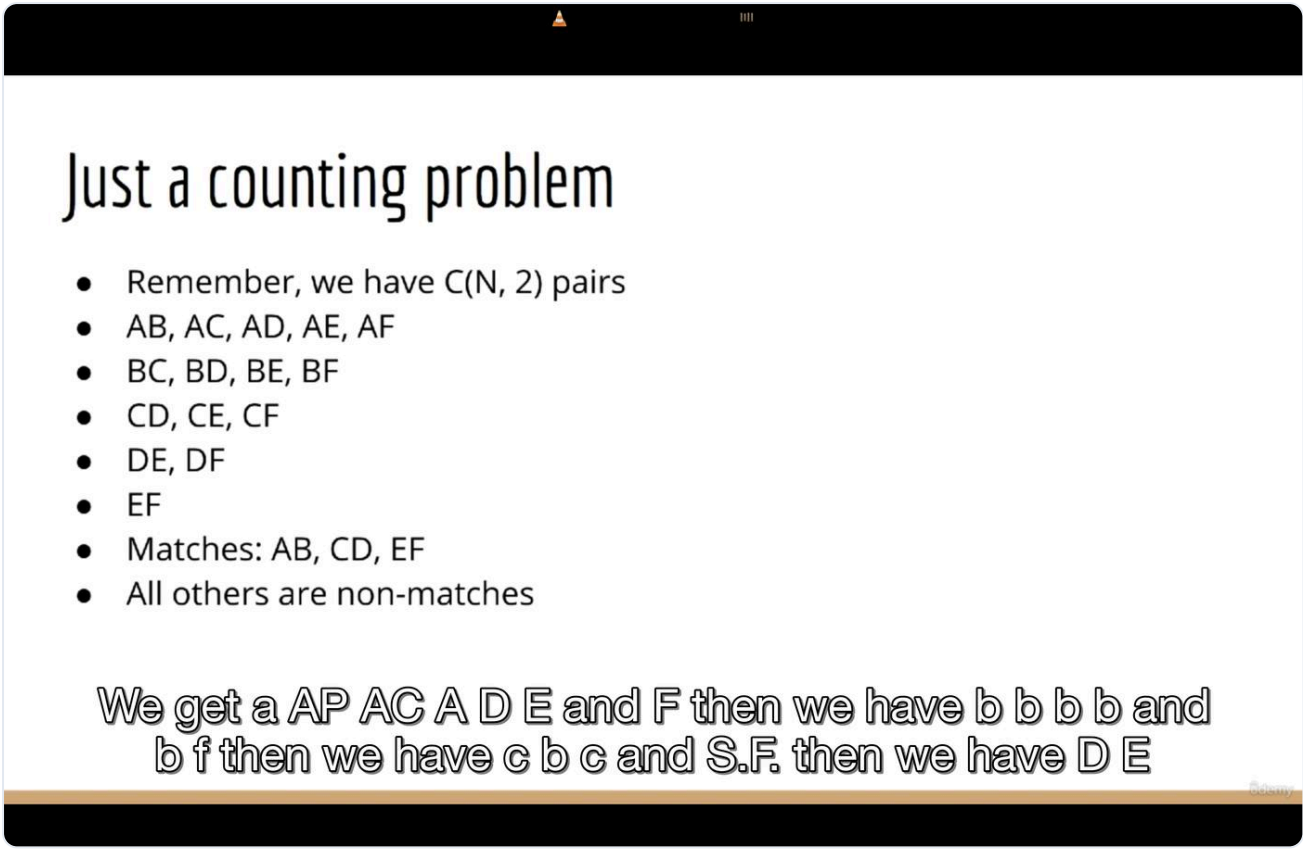

FIGURE 22.9 | Just a Counting Problem

We get a AP AC A D E and F then we have b b b b and b f then we have c b c and S.F. Then we have D E and F and then we have just e f. And of course we have the corresponding labels. So a matches with B C matches with D and E matches with f all other samples are non matches. You will notice a problem which is inherent in the way that this data is created.

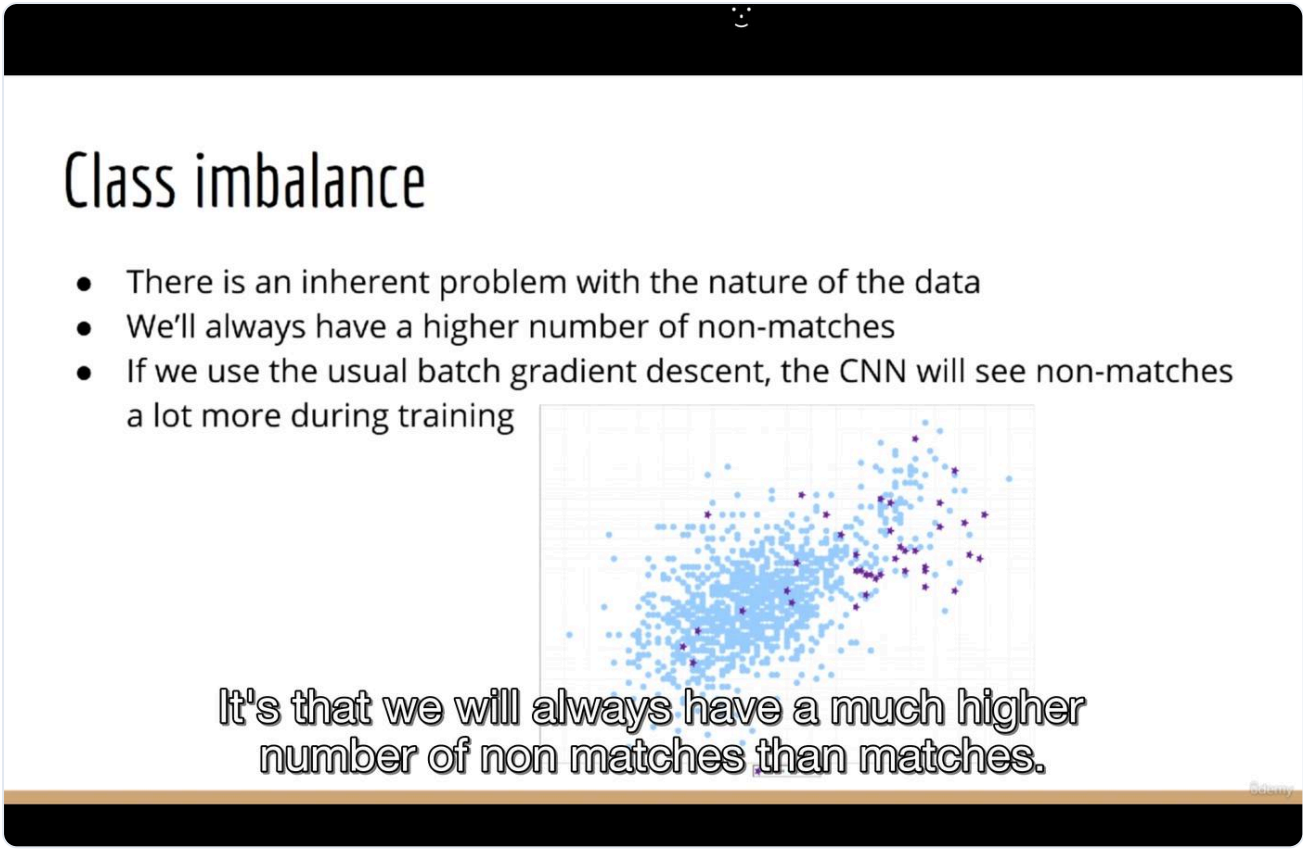

FIGURE 22.10

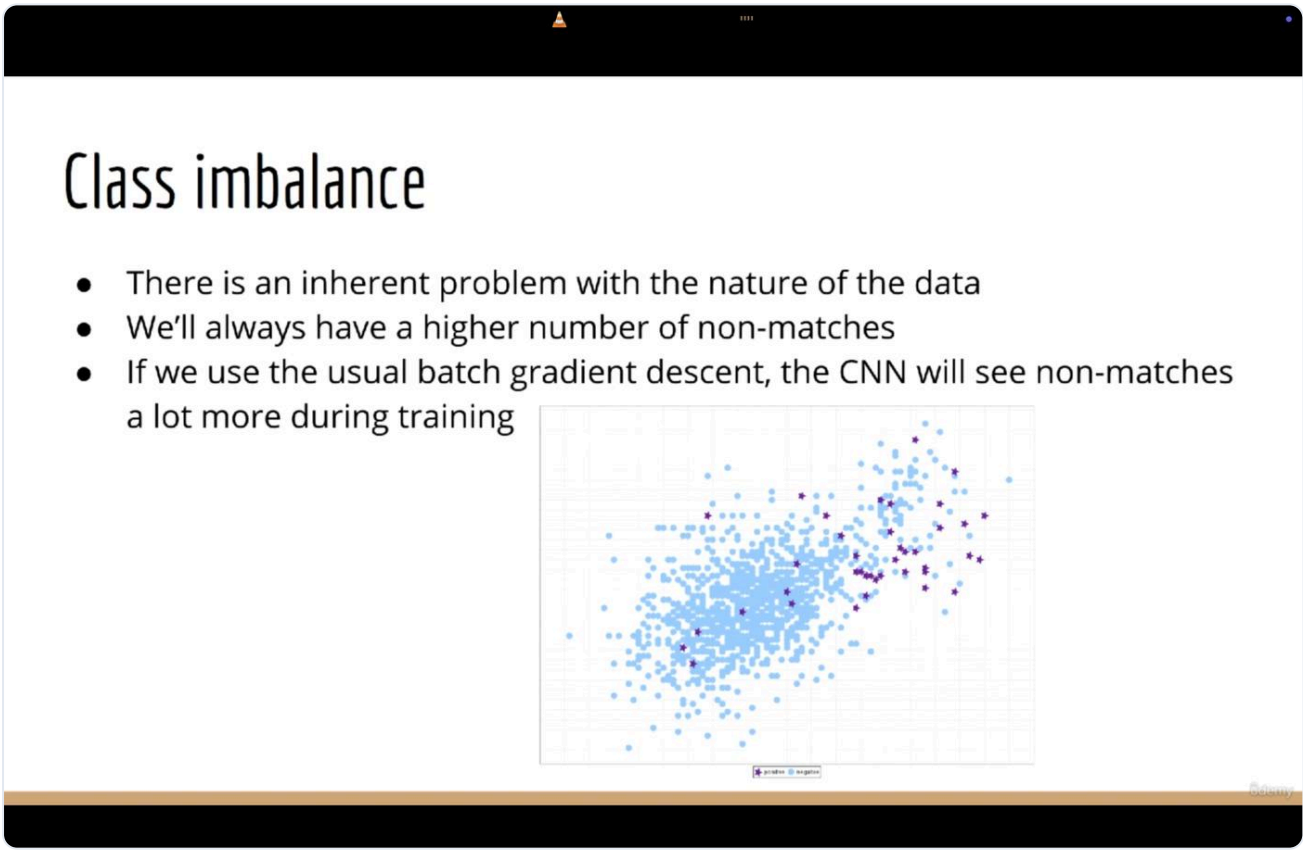

FIGURE 22.11 | We'll Always Have a Higher Number of Non-matches

It's that we will always have a much higher number of non matches than matches. In other words we will always have imbalanced classes. So if we tried to train the Siamese network with the usual batch gradient descent we are going to see a lot more of the non match class than of the match class to solve this problem. We are going to do two things first because we always have N choose two combinations of images. There will always be many more training samples than actual images. We don't want to store all those pairs of images in memory because that would be redundant. We would just be storing the same images over and over again. Thus we want to create our own generator and loop over that inside our training function.

This will allow us to generate the data on the fly and not have to store it in some gigantic matrix. Second by running our own generator. This offers us some flexibility.

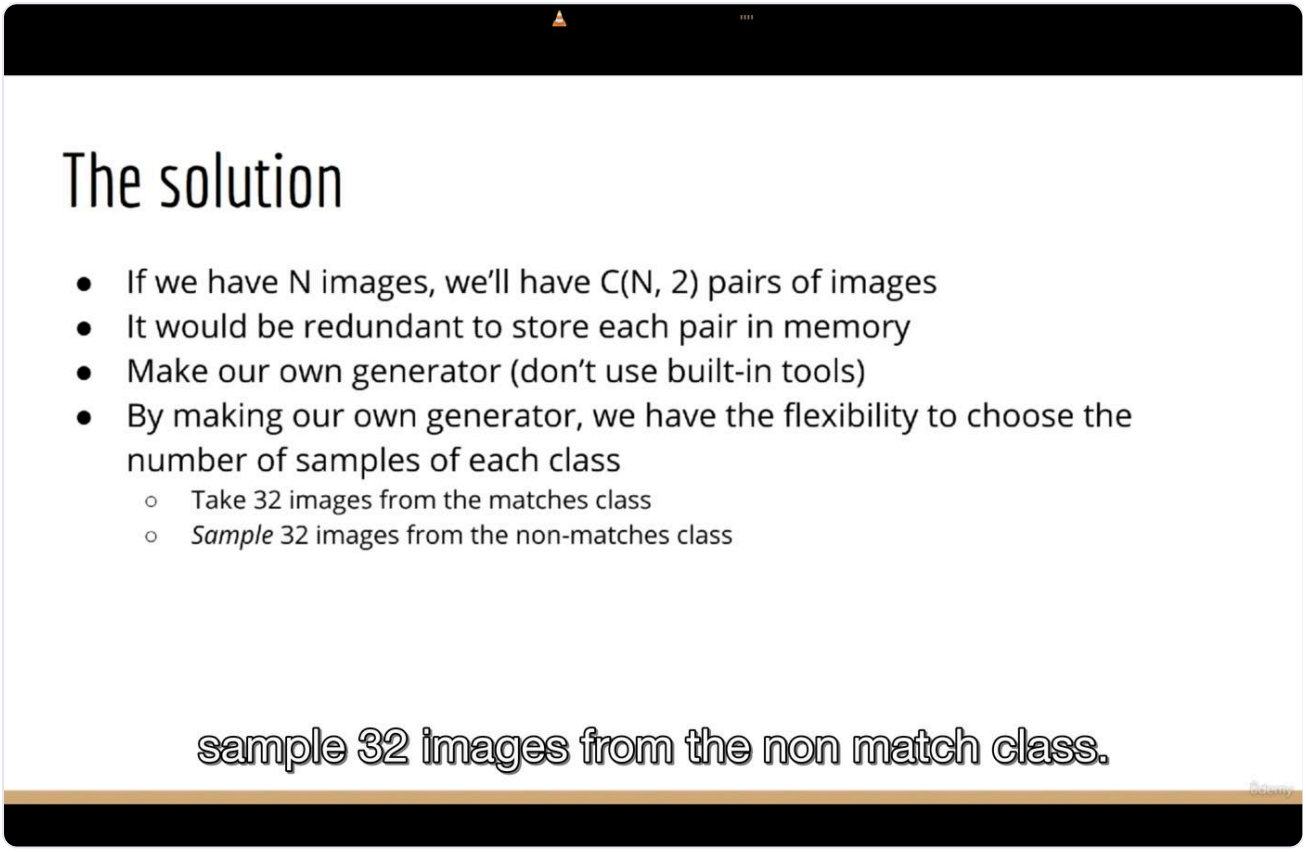

FIGURE 22.12

What we're going to do is for each batch we'll take say 32 images from the match class and then we will sample 32 images from the non match class. In other words for every epoch we'll be looking at all the matches but only some of the non matches so far a simple example earlier where we had the images A B C D E and F here's what we would do. Only a b c d and e f are matches all the other training examples are non matches. Now let's say we do a gradient descent with a batch size of six then three training examples from the batch would be the matches a b c d and e f the other three training examples would be chosen randomly from the non match class. So let's say we choose AC DC and b f.

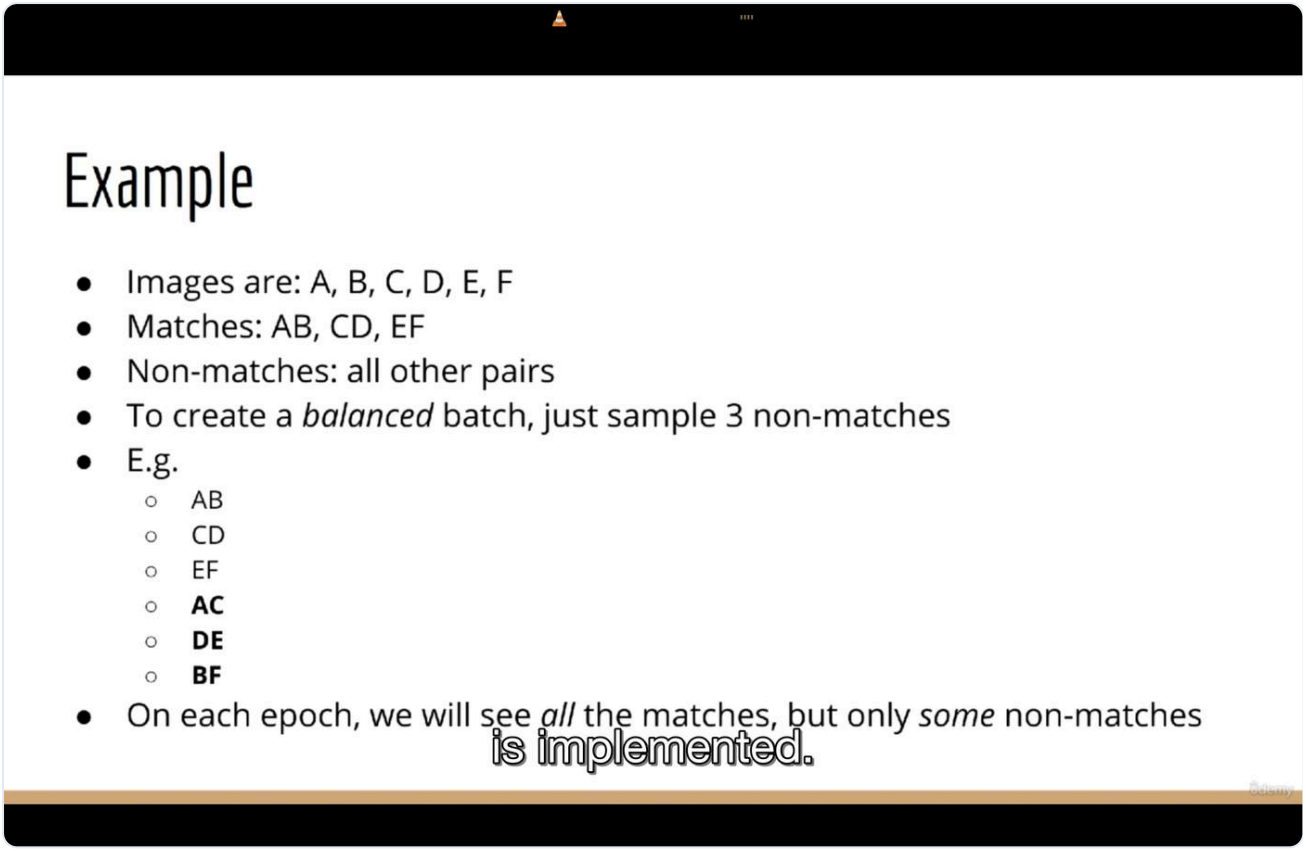

FIGURE 22.13

Of course this means that on each epoch we will always see all of the matches but only some of the non matches in the following lectures we will look at the actual code to see how the process we just discussed is implemented.

LECTURE 116

File Paths

See the notebook `26.PyTorch-Siamese-Networks.ipynb` for the code.

In this lecture we are going to begin looking at our facial recognition code with Siamese networks. As usual you can look at the title of the notebook to determine what notebook we are currently looking at. Since this is a large piece of code it's going to span over several lectures with each lecture focused on a single theme. The theme of this lecture will be loading in the data. As I mentioned previously the actual theory and model behind this project are pretty simple. And most of the work will go into loading in the data and processing it correctly so the first thing you'll notice in this script is the strange imports. This course is about PyTorch.

But I'm importing TensorFlow and Keras. Why am I doing that. Well first let's start by recognizing that these are both just Python libraries. There's no reason you can't use multiple Python libraries simultaneously. As we often do for example we use NumPy and matplotlib side by side. So just because PyTorch and TensorFlow overlap in what they do in the sense that they're both ways of building neural networks it's totally acceptable to use a different library that has some useful functionality even if the neural network parts of these libraries are incompatible so first let's download our dataset. We'll be using the Yale face data set. It's just a bunch of different faces that have different emotions like happy sad and so forth next.

Since this is a zip file we're going to unzip it using the unzip command next we're going to get the file paths of all the images using the glob function you'll notice that while all the images are GIFs they don't end in the file extension .gif they have file names like subjects 0 1 one happy subject. Oh one not sad and so forth. So we are looking for any file that starts with the string subject. Next we are going to shuffle the file paths which will be important later when we create our train and test sets. Next we're going to set the number of files to a variable called n so we have one hundred sixty six images. Next we have a function that loads in the data from a file path and returns a NumPy array.

There are multiple things going on here. So let's break it down. First we call image.load_img to load in the file as an image object. This also allows us to resize the image to 60 by 80. Which I found leaves a smaller memory footprint and doesn't prevent us from getting good results. The original image size was 243 by 320 which was unnecessarily large. Next we call img_to_array to convert the image object into a NumPy array. And finally we convert it to a uint8 which takes up only eight bits per pixel rather than 32 or 64 for a float or double next. As I always like to do I'm going to plot a random image from the data. Just for fun. This always helps me get into the flow of things when I'm coding if we took the shape of our image.

You might expect to see 60 by 80 since well that's what we specify. We want the size to be but something weird is happening here if you look closely the actual image shape is 60 by 80 by 3. That just seem weird to you because if we look at these images they are clearly grayscale images yet they are stored as color images with three color channels as an exercise. I would recommend

verifying that each of these channels actually have the same values and so they are redundant. Therefore we lose nothing by converting these technically color images into grayscale. We can do that by taking the mean along the last axis next we're going to load in the images into a single NumPy array so it's N by whatever the image shape is 60 by 80.

So all we do here is we loop through each of the image file paths we call the load img function which we just define and then we store the image in our big array of images. Next we're going to create an array of labels of length n these are not our binary labels. These are just labels telling us which subjects the corresponding images of. Basically this is just a bunch of manual string parsing since I don't like to play with regex. So first we split out the file name by splitting on the last forward slash. Next we split out the subject's part of the file name by splitting on the first dot. So now we have a string that's like subject a one a subject or two subjects or three and so forth.

Finally we just remove the subject string so we're left with 0 1 0 2 or 3 and so on. And since the ID starts from one I just subtract one so that it starts from zero.

LECTURE 117

Test Images

See the notebook `26.PyTorch-Siamese-Networks.ipynb` for the code.

In this lecture we are going to continue looking at siamese.py. This lecture will focus on splitting the data into train and test sets. Previously what we did was loading in all of our images into a big NumPy array and we also have another array to tell us which subject those images contain. To start I would like to know how many images of each subject we have to do this. I'm going to use the Counter object and pass in the labels. Now when we print this out we can see that most of the subjects have 11 images except the first subjects which has 12 images. The reason why we want to do this is so that we can select an appropriate number of test images next we're going to grab a set of unique labels.

Next we're going to get the number of subjects which is just the length of the set we just created now. Here is one key part of this. We're going to make it so that for each subject three images will be used for the test set. Importantly what you do not want to do is make your data set first where you have a bunch of pairs of images and then do a train test split. The reason you don't want to do that is because when you create your pairs of images the images will be repeated multiple times. So if you try to do a train test split after pairing then you will have the same images appearing in both the train and test sets. So don't make that mistake instead.

We are going to decide first which images will be used for train and test then we'll create our pairs out of those train and test images. So since we chose three as the number of images to take from each subject for the test set that means n test will be three times n subjects then n train will just be N minus n test. Next we're going to initialize the arrays that will hold the train and test images. So we create train images train labels test images and test labels using the n train and n test we just defined importantly realize that we haven't yet created our pairs of images. This is all just pre work that we need to do. Next we're going to fill up the arrays we just initialize.

So currently they are all just zero. And the reason we need to do it this way is because all the images are currently randomly ordered. So we're going to loop through each image in the data set and for each subject we'll take the first three images we encounter and assign that to the test set then any image thereafter we will assign it to the train set so we start with a dictionary called count so far this will count how many images for each subject we've encountered so far in our loop. So the key to this dictionary is the subject ID and the value is the count. We also keep track of train index and test index so we know where to place the image we encounter inside the loop.

The first thing we do is increment the count for the current label. So we index label and then we add one and then we use get here so that the default value is 0. If there is nothing there already next we check whether the count for the subject is bigger than three if it is. That means we've stored all the test images already. So that means we should store this image as a train image so we

store that image variable in the train images array at the current index train index then we store the label in train labels then we increment train index then in the else block we do the same thing except for the test images and test labels.

LECTURE 118

Subject and Set of Indices

See the notebook `26.PyTorch-Siamese-Networks.ipynb` for the code.

Before we move on it helps to think about what exactly we are trying to build.

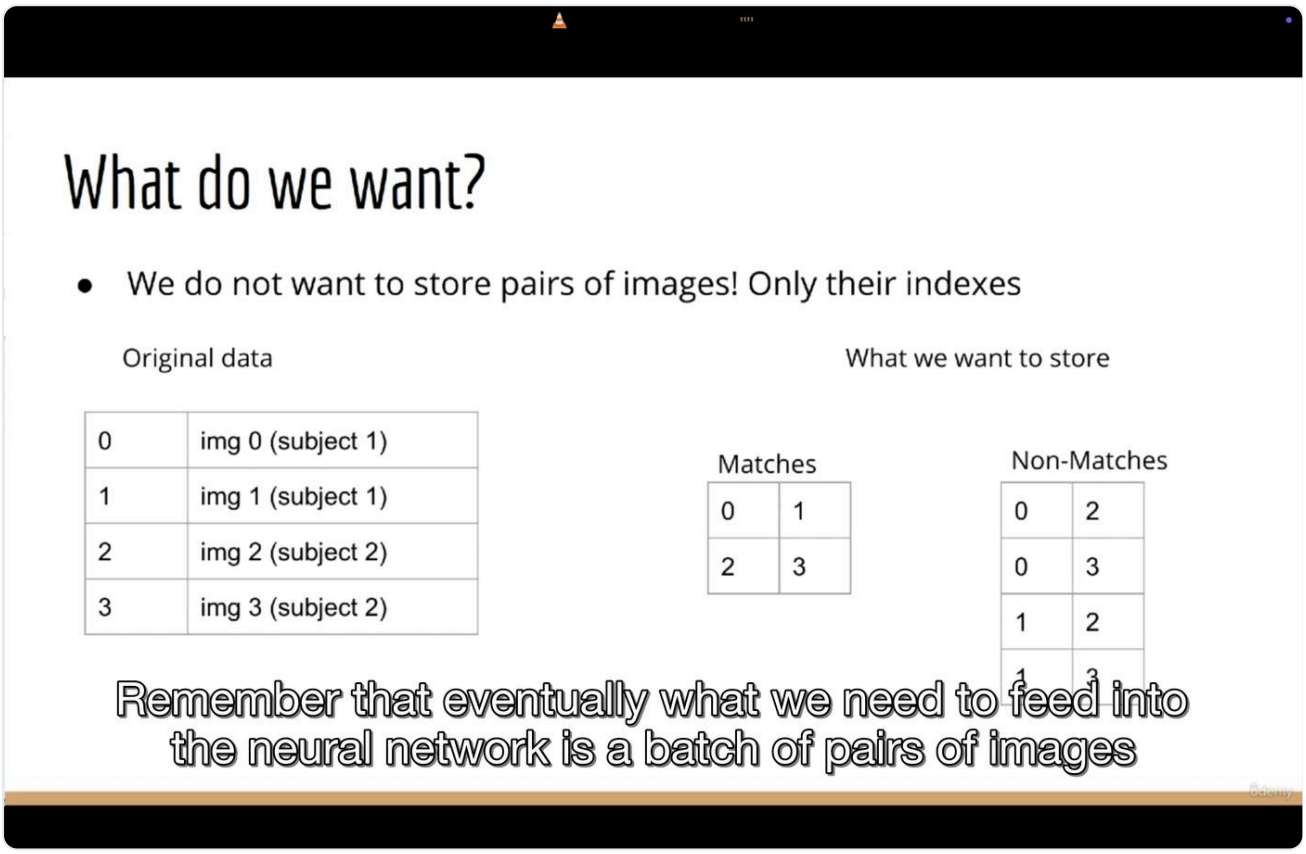

FIGURE 25.1 | What Do We Want?

Remember that eventually what we need to feed into the neural network is a batch of pairs of images along with the target telling us whether each of those pairs is a match or not however. Keep in mind that we do not actually want to store these batches of images because that would mean we would repeat the same images in memory more times than we have to. Instead it would be better if we could just generate these batches on the fly to that end. What we actually want is just a list of pairs of indexes. We'll have one array to carry the matching pairs and one array to carry the non matching pairs in both arrays.

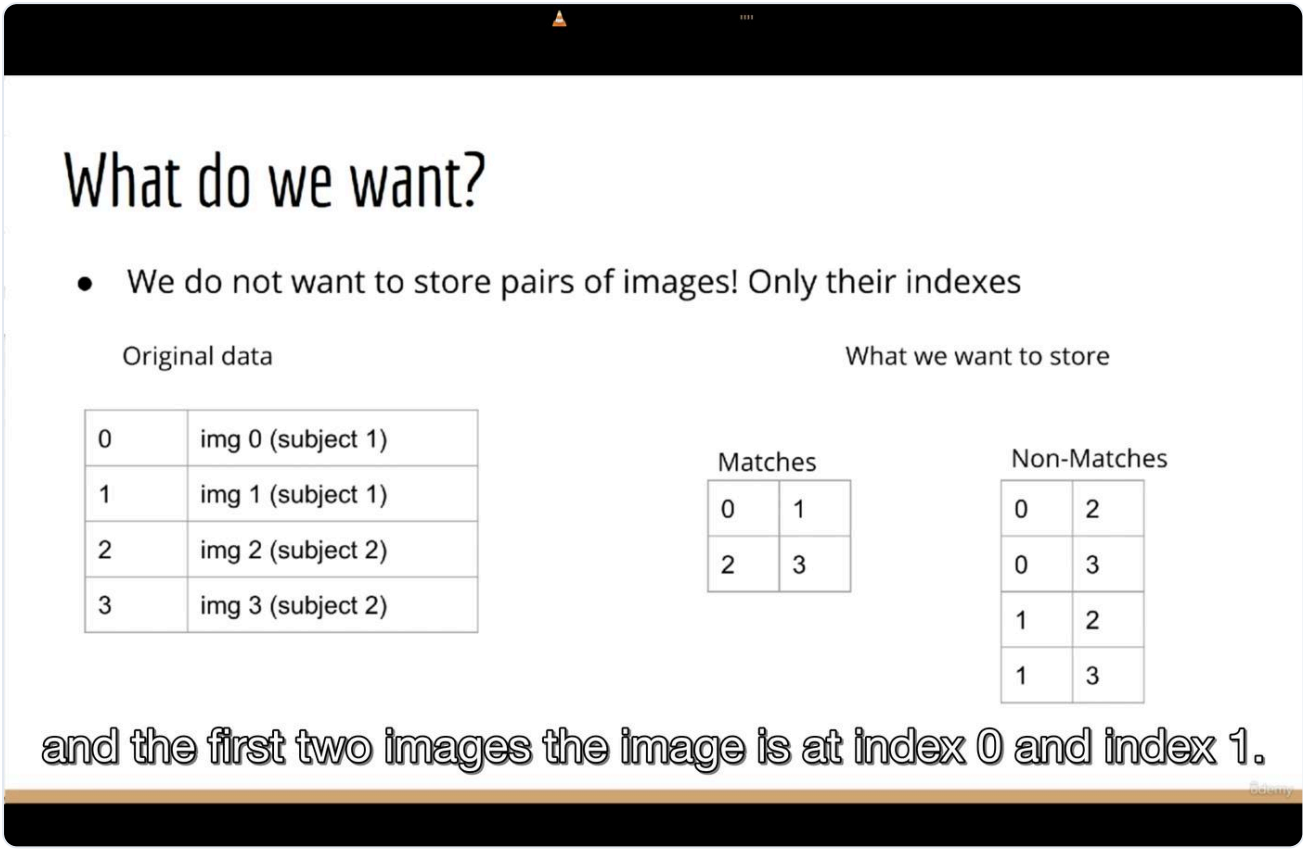

FIGURE 25.2 | What Do We Want?

All we will store is simply a series of tuples of indexes so you can imagine if I have this big array and the first two images the image is at index 0 and index 1. If their matches I don't want to store those images together I'm just going to store 0 and 1 the tuple and then grab those images later. Now in order to create such a data structure it helps to first have a dictionary that maps from subject to the indices where that subjects images are stored. So that's what I'm creating here in the next block. Train label to idx and test label to idx for both of these. The key will be the subject ID and the value will be a list containing all the indexes where that subjects images are stored.

You should be able to see that this is what these two loops are doing. I'm just looping through each of the train labels then I'm using the label as the key and adding the index to a list which is stored as the corresponding value and then I do the same thing for the test labels in the next block is where I create my list of tuples for both train and test. I'm going to have two lists one to store the matches which I'm calling the positives and one to store the non matches which I'm calling the negatives. It helps again to think of just a simple example. So imagine we have just four images A and B which belong to subject 1 and C and D which belong to subject 2.

For this example we will just have 2 positive samples A B and C D so that would be stored in our positives list for the negatives list we would have A C A D B C and B D in total that's 6 pairs which is equal to 4 choose two so here's how we're going to build these lists in code. We're going to start by looping over those labels to index mappings we just created. Remember that the key is the label or subject ID. Whereas the value is a list of indices where that subject's images are stored. So the first thing I do in the loop is figure out all the indices that do not belong to the subject. So I call that other indices so that's easy. I just take the set difference between the full set of indices from 0 up to n train and then subtract the set of indices that belong to this subject.

And remember that this is not numerical subtraction but set difference in then I'm going to loop through each index for each index that belongs to this user for each index. I'm going to loop through every index after it in the same list. So I loop through indices and then I loop through indices again but only after the index i and then I append all these indexes to the train positives list then I loop through all the other indices and add all these pairs to the train negatives as a side note if you find this complicated I always recommend simply not following my code. Instead write it yourself and you'll encounter all the obstacles and have to solve the same problems on your own.

This will give you a much better understanding of what's going on rather than trying to follow me and so after this we just do the same thing over the test set. So it's the exact same structure except instead of a train label to index we have test label to index and we're populating test positives and test negatives rather than train positives and train negatives.

LECTURE 119

Batch and Positive Samples

See the notebook `26.PyTorch-Siamese-Networks.ipynb` for the code.

In this lecture we are going to continue looking at our Siamese network code specifically to take what we just created and write some data generator functions. These will be the functions that we iterate over inside our training loop. Later on the basic idea is we're going to pre allocate some arrays and then fill them up with data from the pairs of indices that we just created. So we're going to start out by looking at the train generator function as you can see. I've also specified the batch size here which will be 64 for the script inside the function. I'm going to set and batches to be the number of batches it's essentially the number of positive samples divided by the batch size but rounded up since we round up.

That means the last batch will always have less samples than the rest. Next we're going to enter an infinite loop inside the loop. We're going to shuffle the positives so that's this one. This is so that each epoch we'll encounter each of the positive samples in a different order. There's no need to shuffle the negative samples since we are already going to be choosing those at random next we're going to pre allocate some arrays. First we set and samples to be batch size times too. So in actuality the batch size will be 128 rather than 64. And this is because the first half of the dataset will be the positive samples and the second half will be the negative samples.

Next we create X batch 1 x batch 2 and Y batch. Currently they are all just zeros and we're going to fill them up with an X for loop so inside this loop we're going to do this n batches times. First we get the indices from the positive pairs for this batch. So that's from i times batch size up to i plus 1 times batch size. Next we set the index J to 0. So J will be used to index our batches as we add samples to the pre allocated arrays. Next we loop through the pairs of positive samples for this batch so we get the idx1 and idx2 then inside the loop we grab the train images at idx1 and idx2 and add these to x batch 1 and x batch 2 then we set y batch to 1.

Since these are all matches then we increment J. So that on the next round we store the image in the next spot. Next we enter another loop this time for the negative samples but before we enter this loop we choose a random set of samples using np.random.choice. The difference inside this loop is we set y batch to zero. Since these are all non matches. Finally the true data for this batch only goes up to the index J. So we index x batch one x batch two and y batch only up to J. Now we have our batch of data but there are a few small issues. First our images are currently of size n by h by W which is not an appropriate input shape for a CNN. A CNN wants its input to be four dimensional and in PyTorch that's N by color by height by width.

So we need to add a superfluous one dimension in between the batch index and the height index. Additionally although we could do this in the training loop itself we can save some space by converting our data into torch tensors now. As a side note even though Y is technically a binary integer label it's more useful to have it as a float because we'll be multiplying it in our cost function so from that we get x1 x 2 and Y and then we yield x1 x 2 and Y. Note that we must enclose x1 and

x2 with square brackets to make it a list. So this will not work if you return a tuple and it will also not work. If you try to just return x 1 x 2 and Y without any brackets this is because these two variables will be the input into the model and Y will be the target next we have the test generator.

Now this is essentially the exact same thing except that we don't need to shuffle the data and it's using the test variables rather than the train variables.

LECTURE 120

Non Matching Pairs and Test Generator

In this lecture we are going to build our model. The loss function and train our Siamese network. To start we're going to define our network which actually itself contains an entire neural network. As usual you can feel free to play around with these hyperparameters and choose your own architecture. This CNN is just what I happened to copy from the fashion MNIST example earlier in the course notice that instead of a final softmax we end with a dense layer that has no activation. Since we are not doing classification in the forward function it's very simple. First as input we accept two batches of images. im1 and im2. We pass both of these through the same CNN to get feat 1 and feat 2 the embedding for both sets of images.

Finally we calculate the Euclidean distance between feat one and feat two for each row. So if we have N input pairs we'll have N output distances. Next we create an instance of our model and set the feature dimension to be 50. Next we move the model to the GPU. Next we create our custom loss function. The contrastive loss the first argument is the prediction of Y and the second argument is the target t inside this function. We start by calculating the squared distance for the non matching pairs. This is that funny calculation you saw in the theory lecture where we subtract y from the margin and so forth. So here you see we're using a margin value of 1.

This means that for non matching pairs we want the distance to be at least 1. But as long as it's bigger than that we don't really care how big it is. Now from the formula you know that we want to take the max between the margin minus Y and 0 but you'll recognize that this is exactly the ReLU function. So we can just use the built in ReLU in PyTorch. Next we pass this calculation into the rest of the loss which is relatively simple and take the mean over each sample next we create our optimizer. Usually I say loss and optimizer but that is not the case in the script. Since our loss isn't one of the built in losses next we have our training function.

So what's different about this. First you'll notice that in addition to the train generator and the test generator we also pass in the values for the number of steps per epoch. This is essential when you use a generator because if you don't know when an epoch is finished you'll just end up in an infinite loop. Recall that our generators have infinite while loops inside of them. This carries over from TensorFlow and Keras. So if you've ever used these libraries it should seem familiar inside the loop. It's mostly the usual stuff we loop over the train generator and on each iteration we get x1 x2 and the targets since that's what is yielded by our generator.

We move that to the GPU and so on and so forth. One minor difference is that I have this variable called steps that counts the number of steps I've completed so far. So at the end of our inner loop I increment steps by 1. And if it's greater than or equal to the steps per epoch I break out of the inner loop so it's the same basic idea for the test generator except no gradient descent. Okay so next we call a training function. As you can see this train is quite fast. Pretty good for a CNN. Next we plot the loss per iteration which looks OK and that's it for now in the next lecture we'll focus on how to evaluate our trained Siamese network.

LECTURE 121

Distances

So interestingly although we are doing something like binary classification our model does not actually output a binary decision. It only outputs distances between matches and non matches. Thus it's actually up to us to choose a threshold at which we consider two images to be a match or not. Now you might think since we push the matches toward zero and the non matches to at least one then the threshold should be in the middle like zero point five. But in fact the threshold we choose is a hyperparameter. It could be less than zero point five. Or it could be more. One thing you'll see is that we are going to get really good separation on the train set but not as good on the test set.

What this tells us is that it may be worthwhile to have three data sets train validation and test. And this way we could use the validation set to choose a threshold which is often what we use to evaluate all other hyperparameters as well. Then when we've chosen our hyperparameters we can test our model on the true test set before we start. Let's remember that when we're working with PyTorch we need to carry our arrays from NumPy land to PyTorch land before making predictions and vice versa. And so here's a convenience function called predict that does all that for us. It takes in two NumPy arrays of images x1 and x2. Then inside the function it converts x1 and x2 to torch tensors and moves them to the GPU.

Next we pass x1 and x2 into our model to get a prediction. Bring it back to the CPU and then bring it back to NumPy land. Finally we call the flatten function since the model returns an N by one output. But we just want a one dimensional array so let's look at this code to calculate the accuracy of our model now. You recall that with this particular dataset one inherent property of it is that we have imbalanced classes. There are many more non matches than matches. Therefore accuracy is not a reliable indicator of how good the model is. Instead we'll report the true positive rate and the true negative rate also known as the sensitivity and specificity.

So that's one of the things we're going to do in this function get train accuracy as you can see it accepts one argument the threshold which will be used to assign a classification decision. We're also going to make a histogram plot of both the matches and non matches so that we can appropriately choose a threshold. We'll store the distances we find for the matches in a list called positive distances and we'll store the distances we find for the non matches in a list called negative distances we initialize TP TN FP and FN to zero and we'll accumulate these as we loop through the data so first we're going to initialize two arrays x batch 1 and x batch 2.

Note that in this case we want to loop through every sample we have not just sample from the non match class like we did in the generator. So that's why we have a custom loop here and we're not using the generators we defined earlier inside the loop. We grab a batch of indices so that's pours back in is easier and you'll notice that we have two loops so there's one for the positive list and there's one for the negatives list so the tuples we encounter right now are positive matches. Next we fill up X batch 1 and x batch 2 so that's pretty simple it's the same as before. Next we use our predict function to get the distances for each pair of images so that distances.

And then we convert these into a list and store it in our positive distances list. Finally we check how many of these distances are less than the threshold. That's how many true positives we have. Then we check how many distances are greater than the threshold. And that's how many false negatives we have. Next we have the same loop but this time over the train negatives list this loop is largely the same. We gather the batch of data we pass it to model dot predicts to get the distances but this time we add the distances to the negative distances list any distances which are less than the threshold are now false positives and the distances greater than the threshold are true negatives once we are outside the loop we are done counting.

So we can calculate the true positive rate and the true negative rate. And finally we plot a histogram of the negative distances and positive distances now because there will be many more values in the negative distances list. Technically speaking this histogram should be much taller than the other one but we can pass in the argument density equals true. So that matplotlib will normalize the height of both histograms essentially making them into probability distributions. Next we have a function. Get test accuracy which does the exact same thing but over the test set and next we have the results. Note that this takes some time because we have to loop over all the data points and generate the data on the fly.

So I've printed out the batch number on each iteration of the loop once we're done. You can see that we get very good separation between the positive distances and the negative distances. All the positive distances are close to zero and all the negative distances are close to one or bigger than one as they should be. Theoretically we could choose a threshold that would give us 100 percent accuracy on the train set. But as I said earlier you probably want to do that with the validation set instead. Let's see why. So now we're going to look at the function get test accuracy and I've pre computed this threshold of zero point eight okay. So you can see that we don't get as good separation between the positive distances and the negative distances.

There is a bit of an overlap. Nonetheless we still get above 90 percent for both the true positive rate and the true negative rate. Keep in mind that the results here will be slightly different each time you run the script due to various factors. First it depends on the images that we chose for the train set and the test set. It also depends on the neural networks. Initial parameters hyperparameters and how long it was trained for. So I've gotten results better than this but I've also gotten results worse than this. So test it out on your own and see what you can get.

LECTURE 122

Facial Recognition Section Summary

In this lecture we are going to summarize everything we learned in this section of the course. This section was all about facial recognition with Siamese networks. One thing you saw in this section was that while the theory was relatively simple the implementation was complex not because of the model itself but because of all the data processing Siamese networks by themselves are quite simple. We passed two images through the same CNN and this gives us a face embedding for both images.

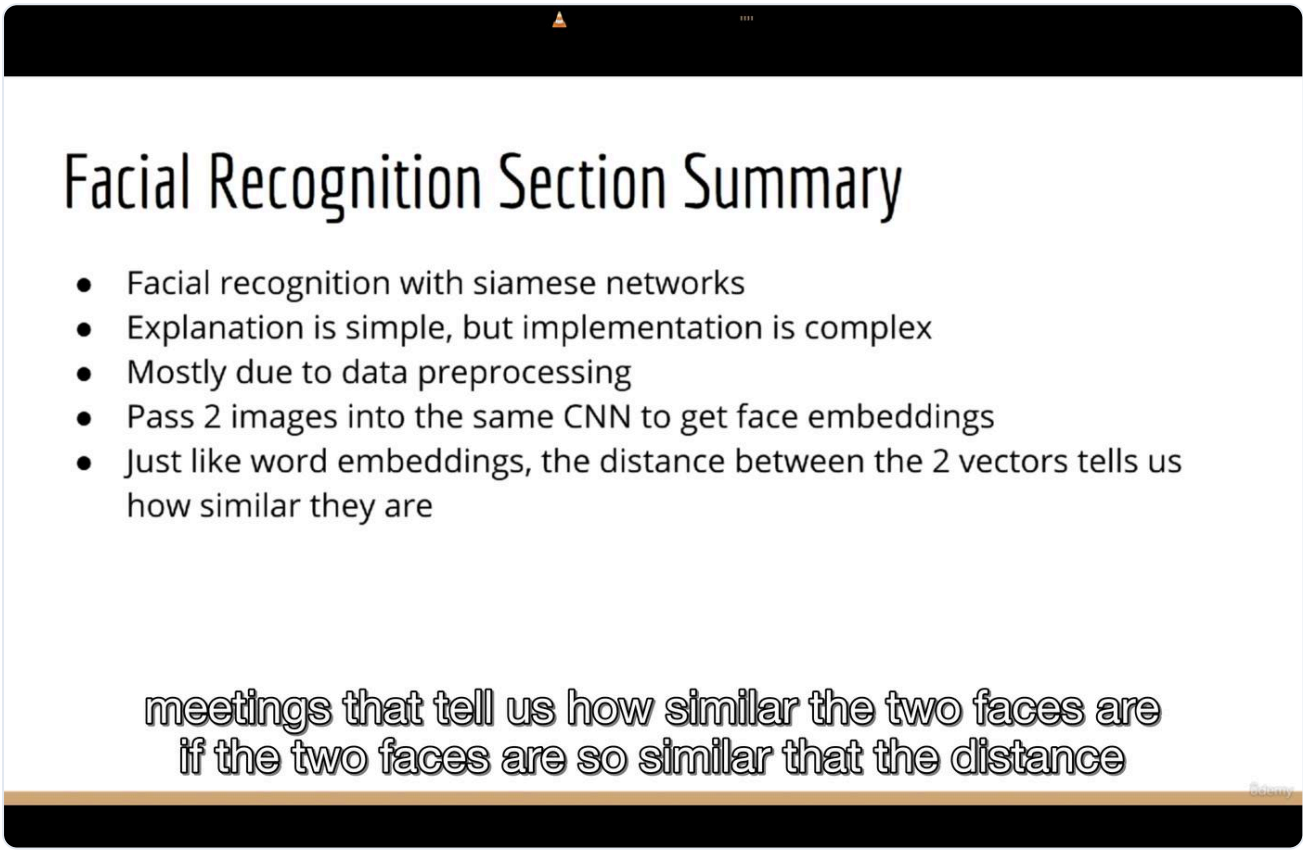

FIGURE 29.1 | Facial Recognition Section Summary

Then we would like to know the distance between these two embeddings just like we do with word embeddings that tell us how similar the two faces are if the two faces are so similar that the distance between the embeddings is less than some threshold and we predict that they are the same person.

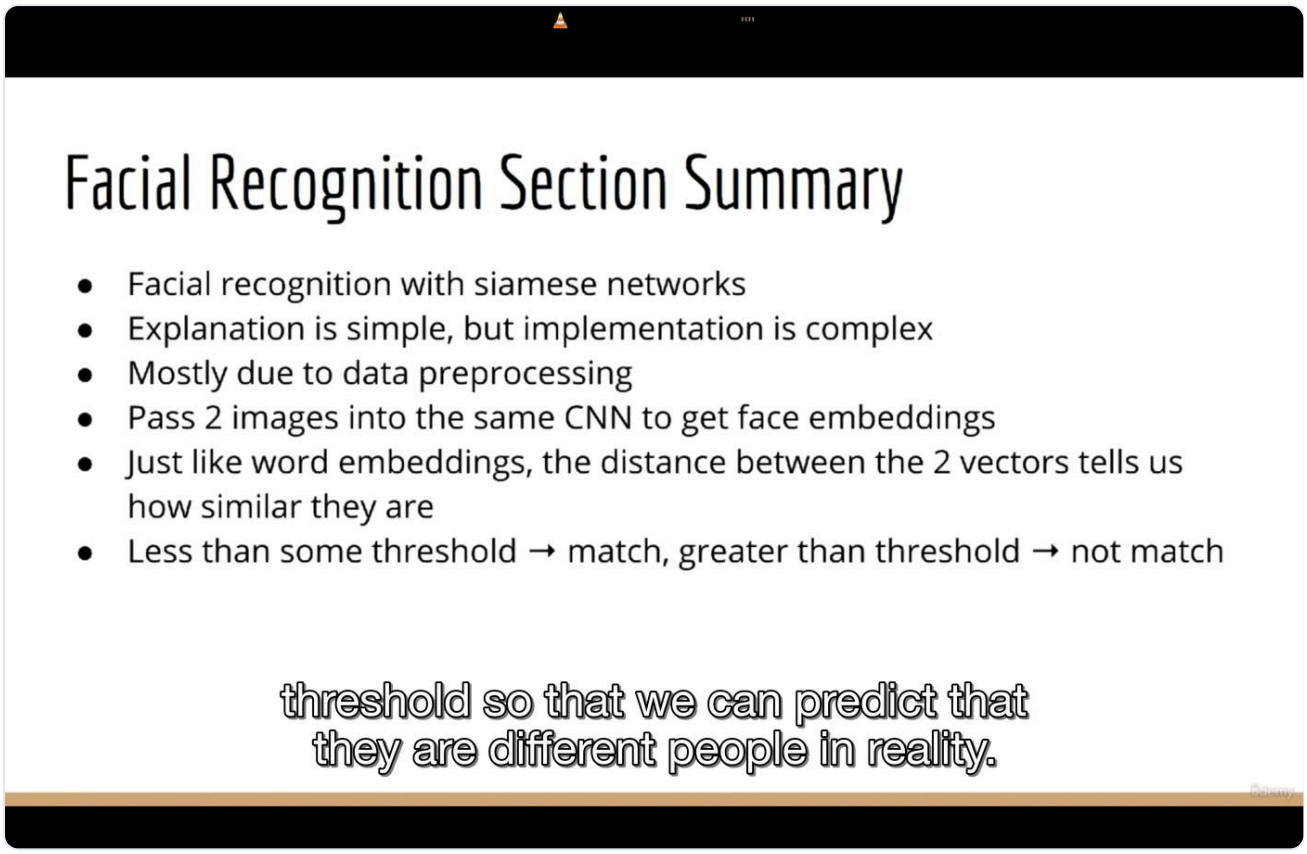

FIGURE 29.2 | Facial Recognition Section Summary

If the faces are from different people the distance between the embeddings should be greater than the threshold so that we can predict that they are different people in reality. Suppose you are running some kind of facial recognition database at your company instead of always passing in two faces at a time something simpler you could do is just store the embeddings themselves in a database.

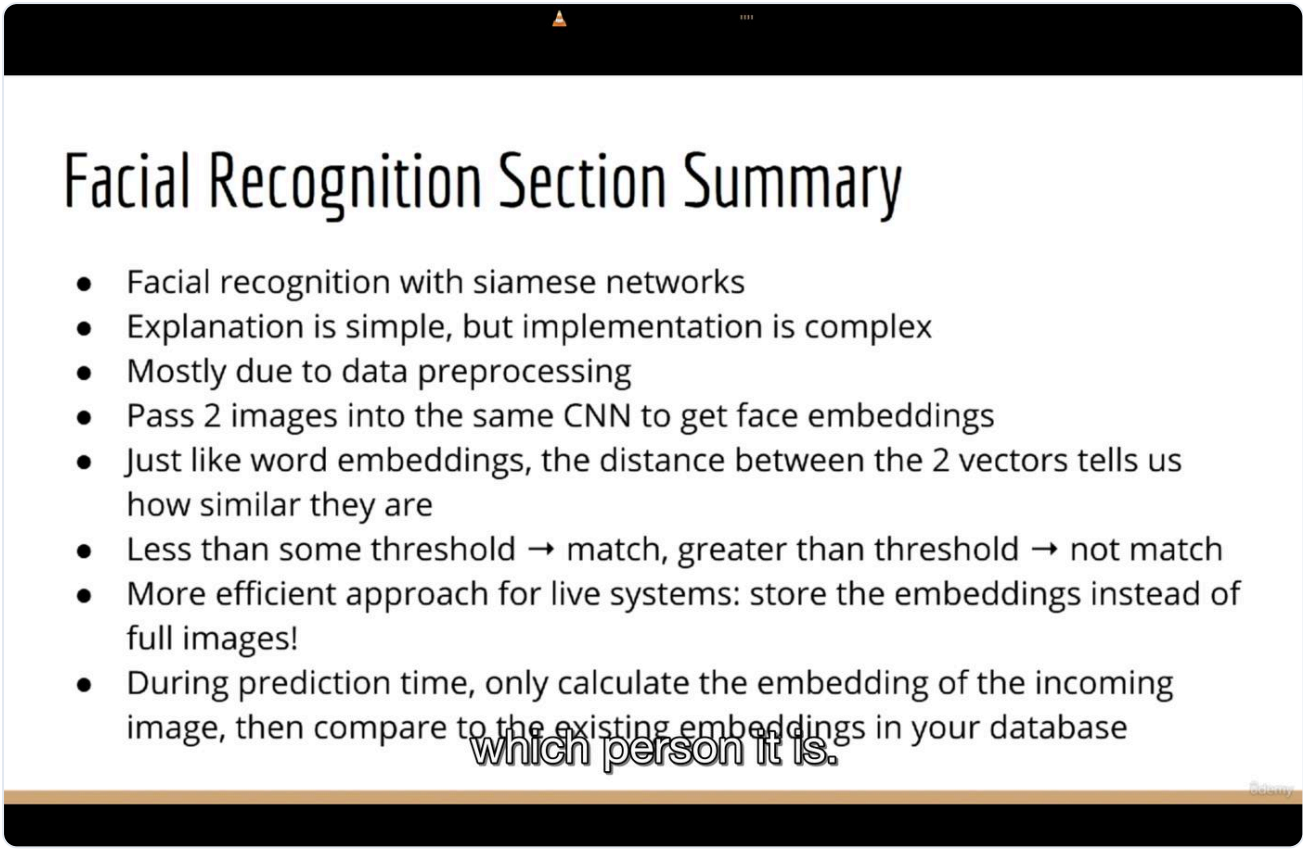

FIGURE 29.3 | Facial Recognition Section Summary

So for all the employees of the company or all of the students in a school don't store their images just store their embeddings then when it's prediction time you only have to calculate the embedding for the incoming image and compare that to all the existing embeddings in your database to determine which person it is. Next we looked at all the data preprocessing steps of which there were many. First we loaded in the data and split the images up into train and test. It was important to realize that the data being fed into the neural network itself not be split up but rather we should split up the images to start with. This is so that none of the images in the train set will appear in the test set and this is because the actual data set we train the neural network on simply consists of the same images being paired up amongst themselves multiple times.

We don't start with a lot of images but we end up with a lot of training samples which grow quite dramatically with the number of images we have. Next we looked at how to convert the data into pairs. There are many ways of doing this and sometimes it can be difficult to try and understand what someone else's approach is. So as always the most important thing is to code it up yourself. This always leads to the best understanding next.

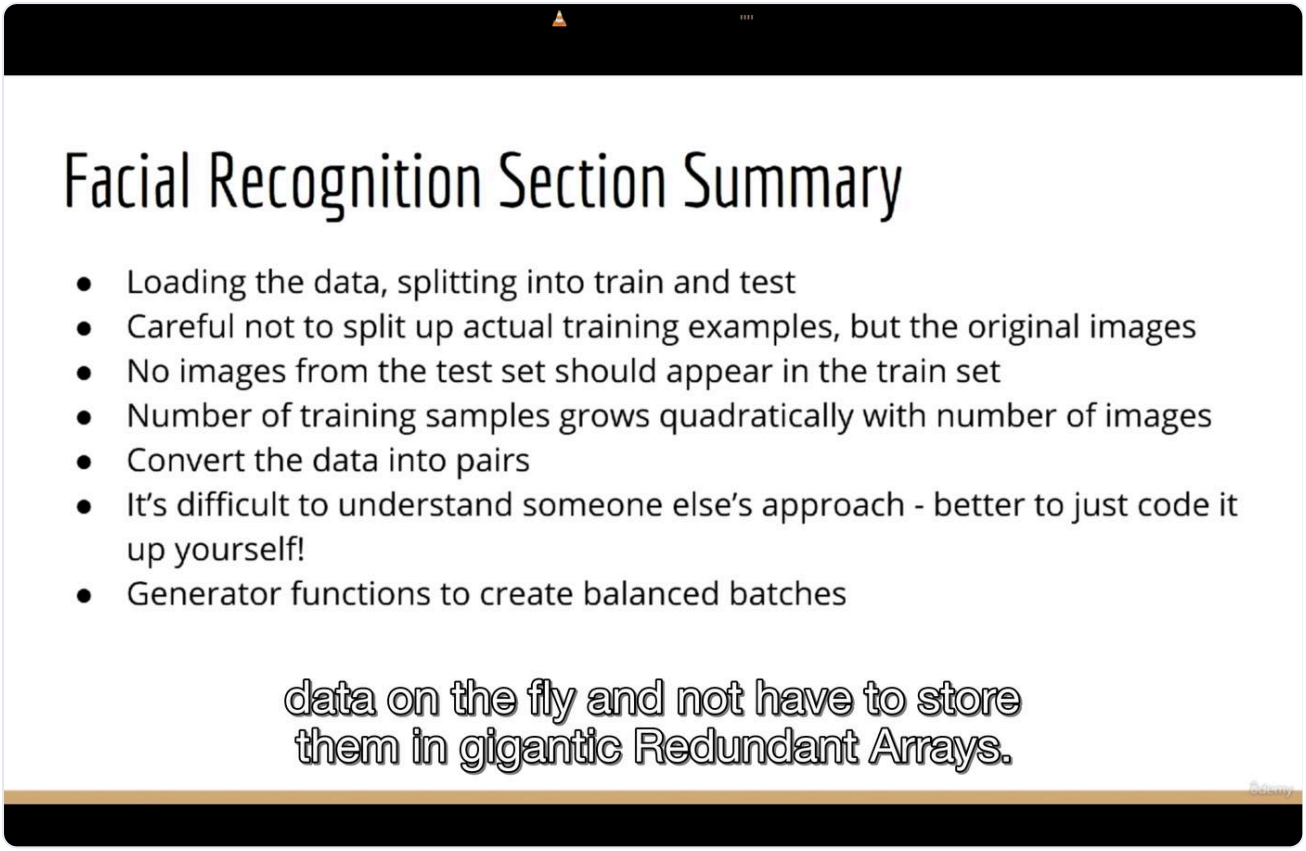

FIGURE 29.4 | Facial Recognition Section Summary

We wrote some data generator functions so that we could iterate over these in our training loop to generate data on the fly and not have to store them in gigantic Redundant Arrays. Finally we wrote a function to help us evaluate the Siamese networks. In particular it would draw a histogram of the match distances versus the non match distances and prints out the sensitivity and specificity. This is better than accuracy because we have imbalanced classes due to the nature of the data.

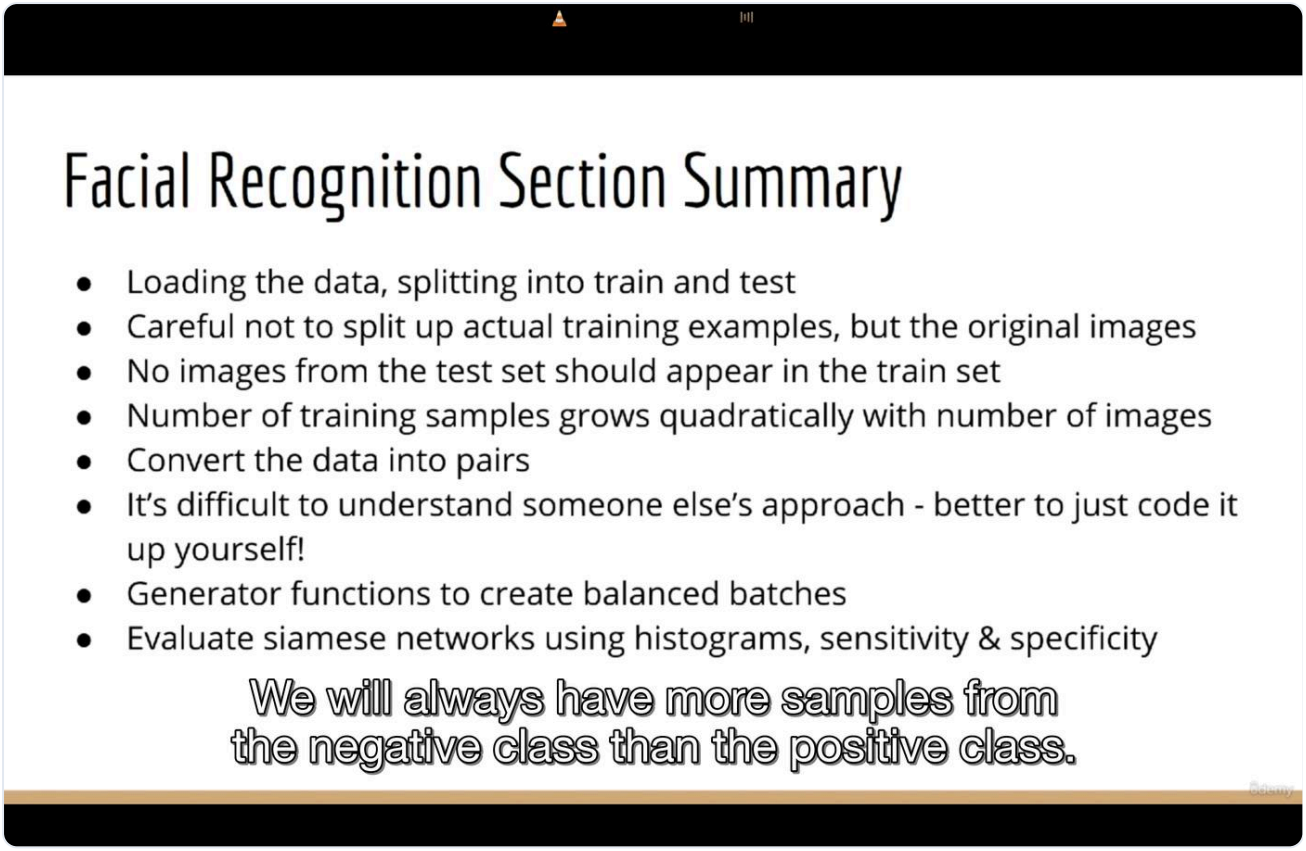

FIGURE 29.5 | Facial Recognition Section Summary

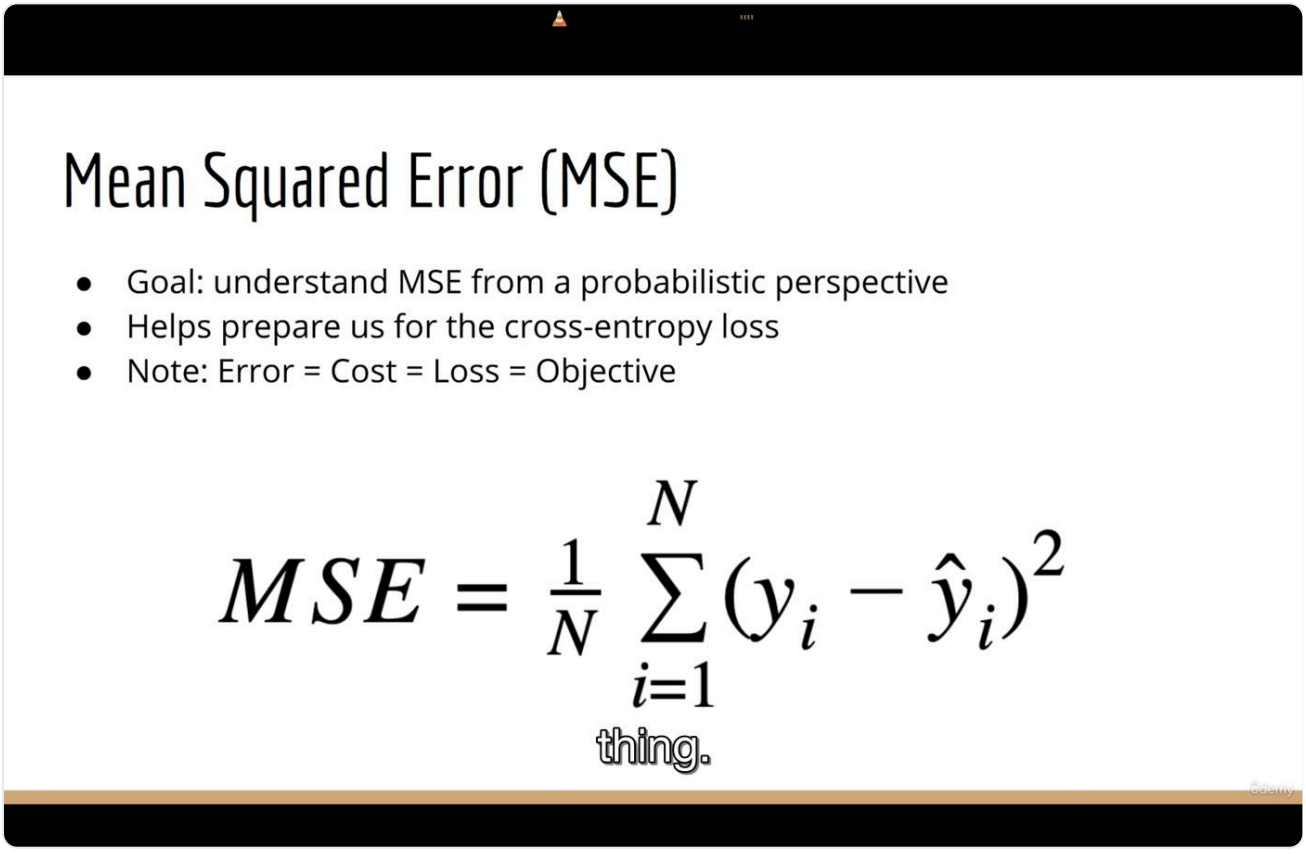

FIGURE 29.6

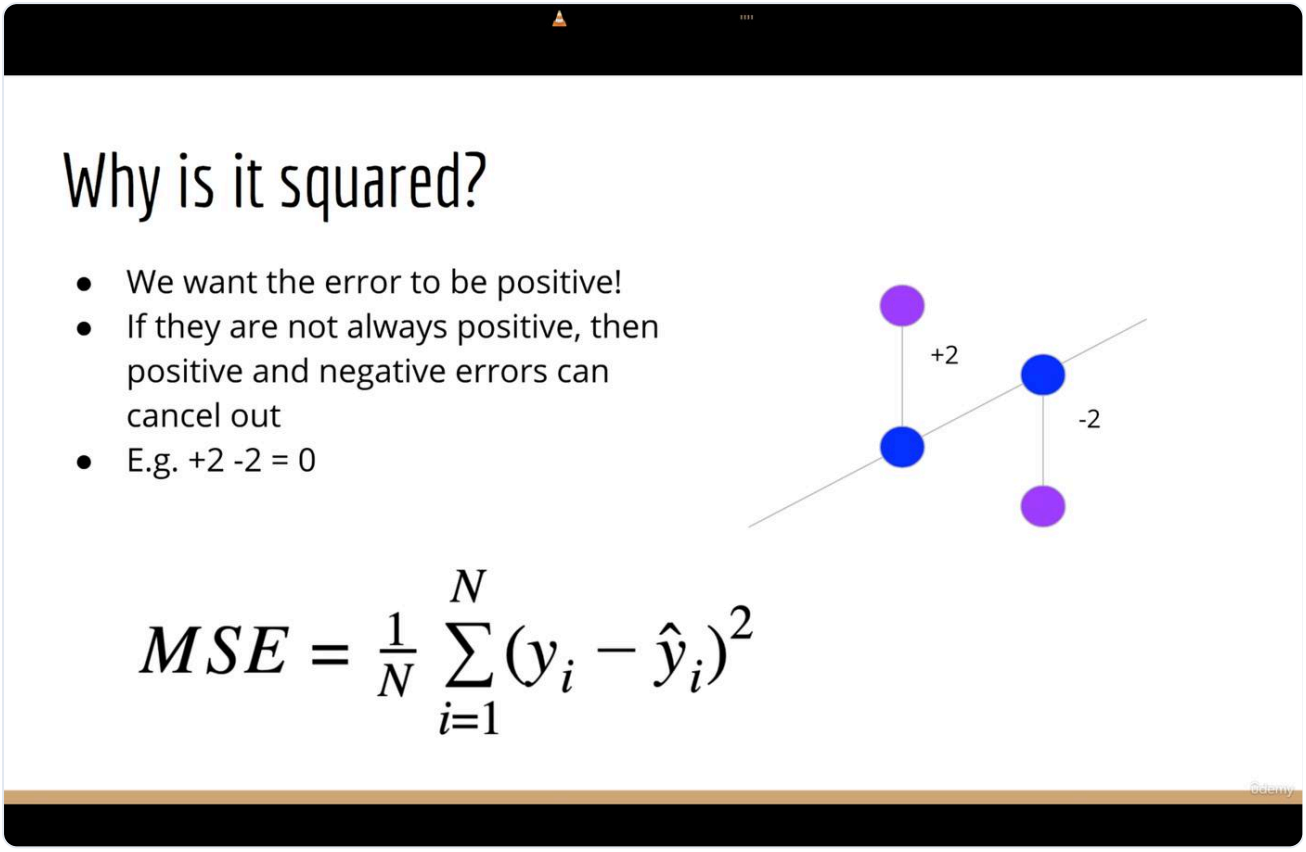

FIGURE 29.7 | Why Is It Squared?

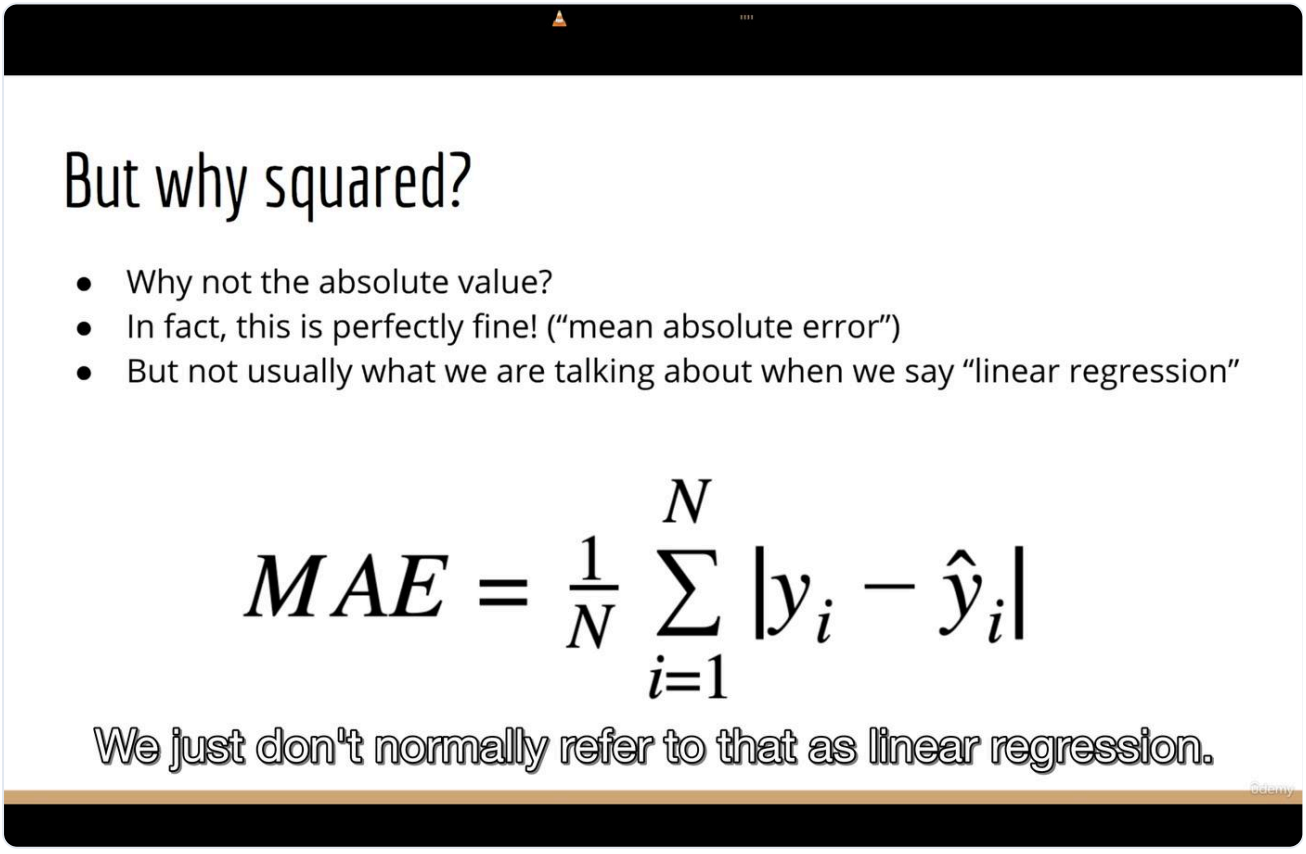

FIGURE 29.8

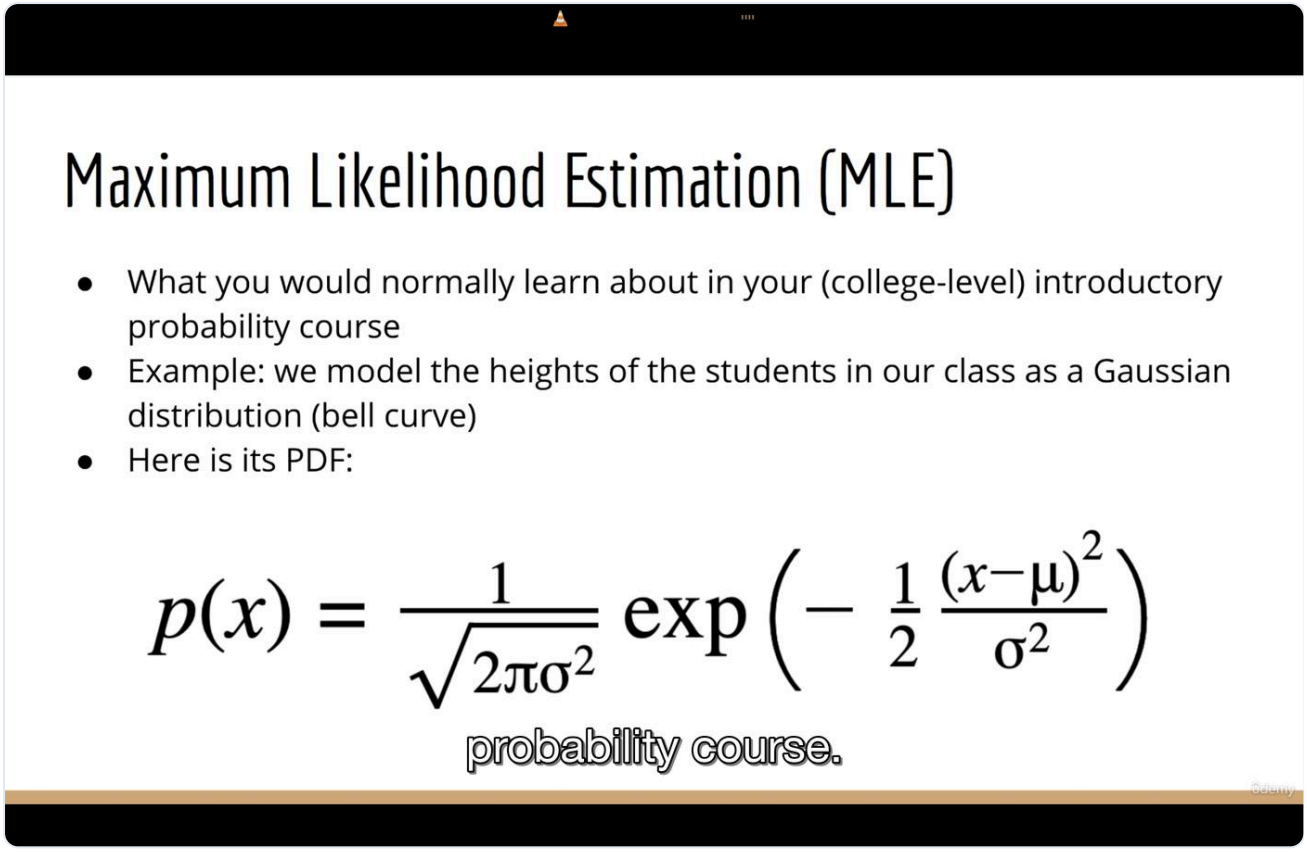

FIGURE 29.9

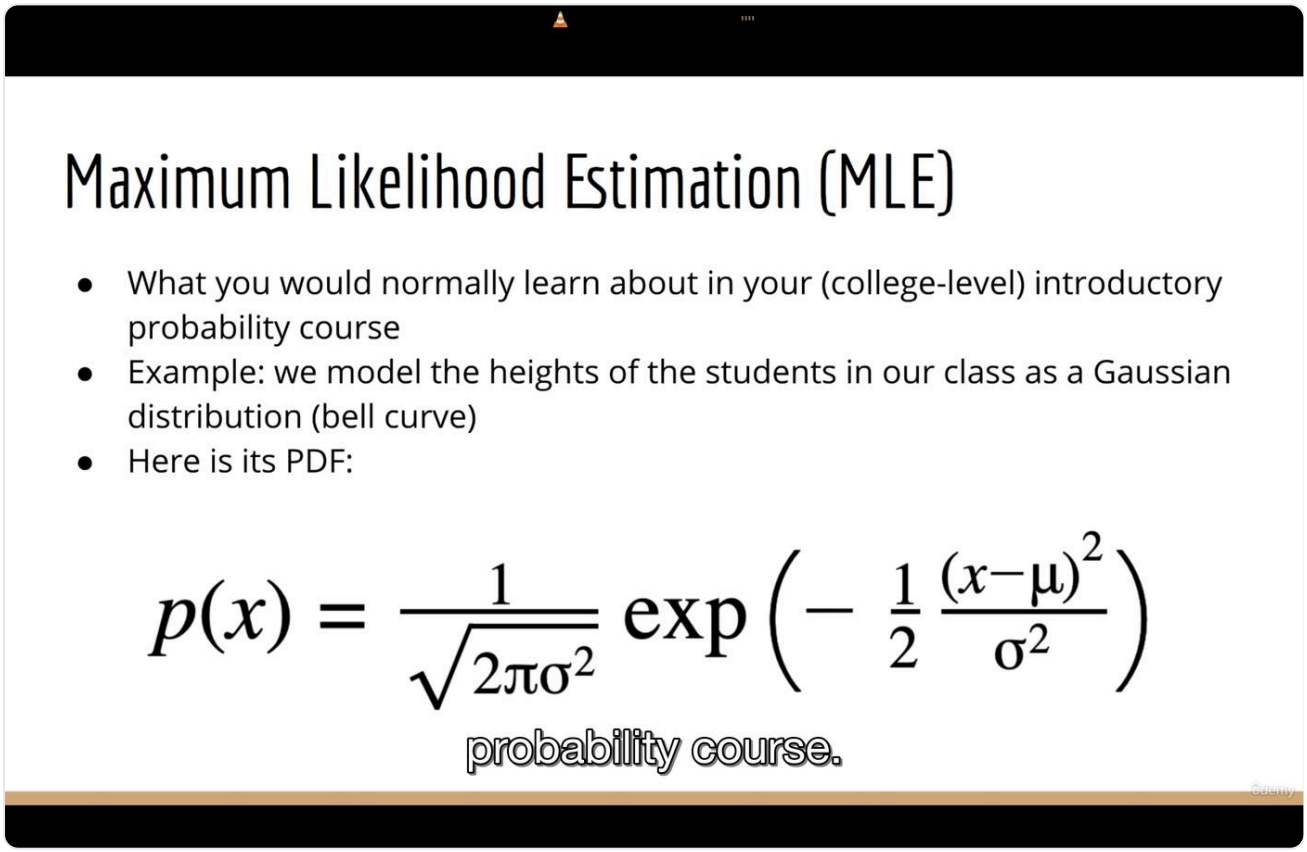

FIGURE 29.10

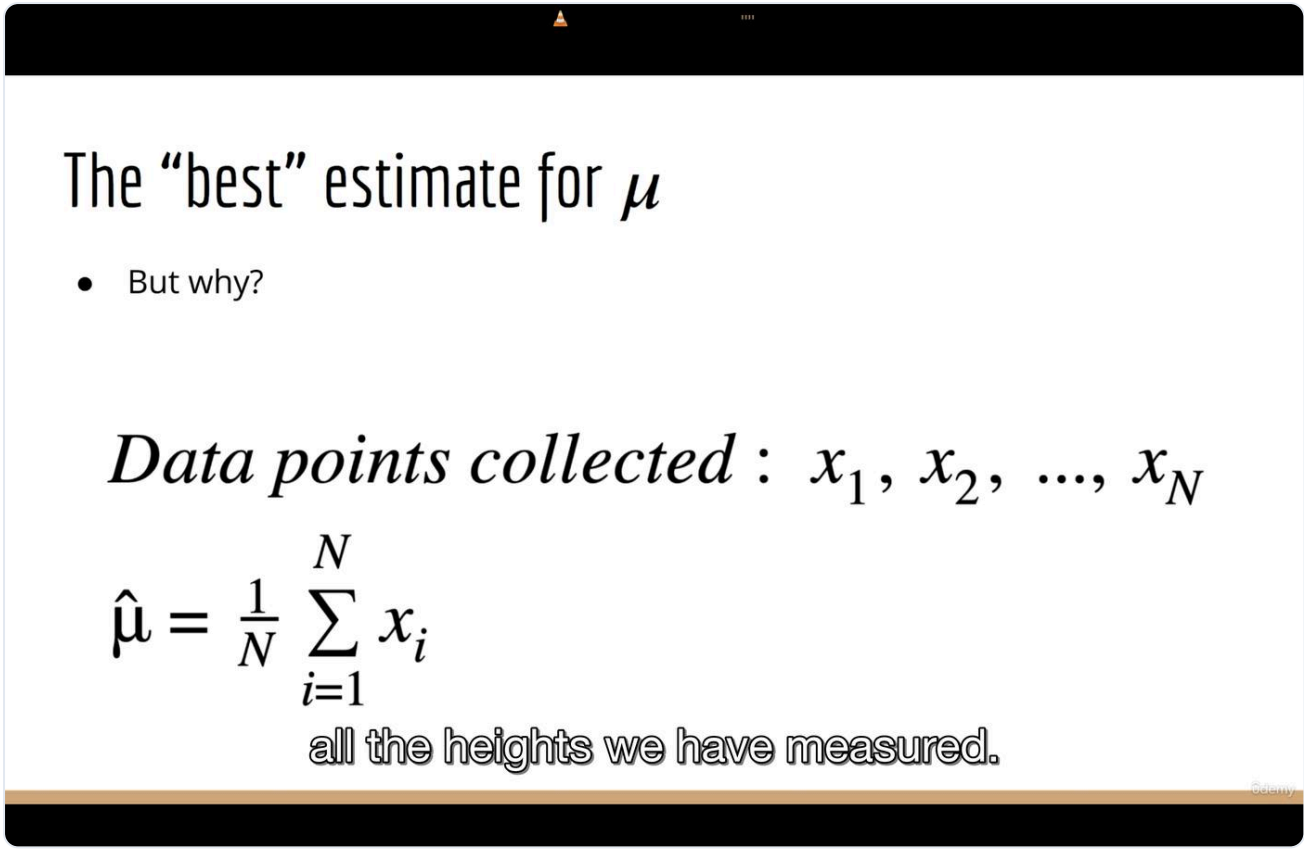

FIGURE 29.11

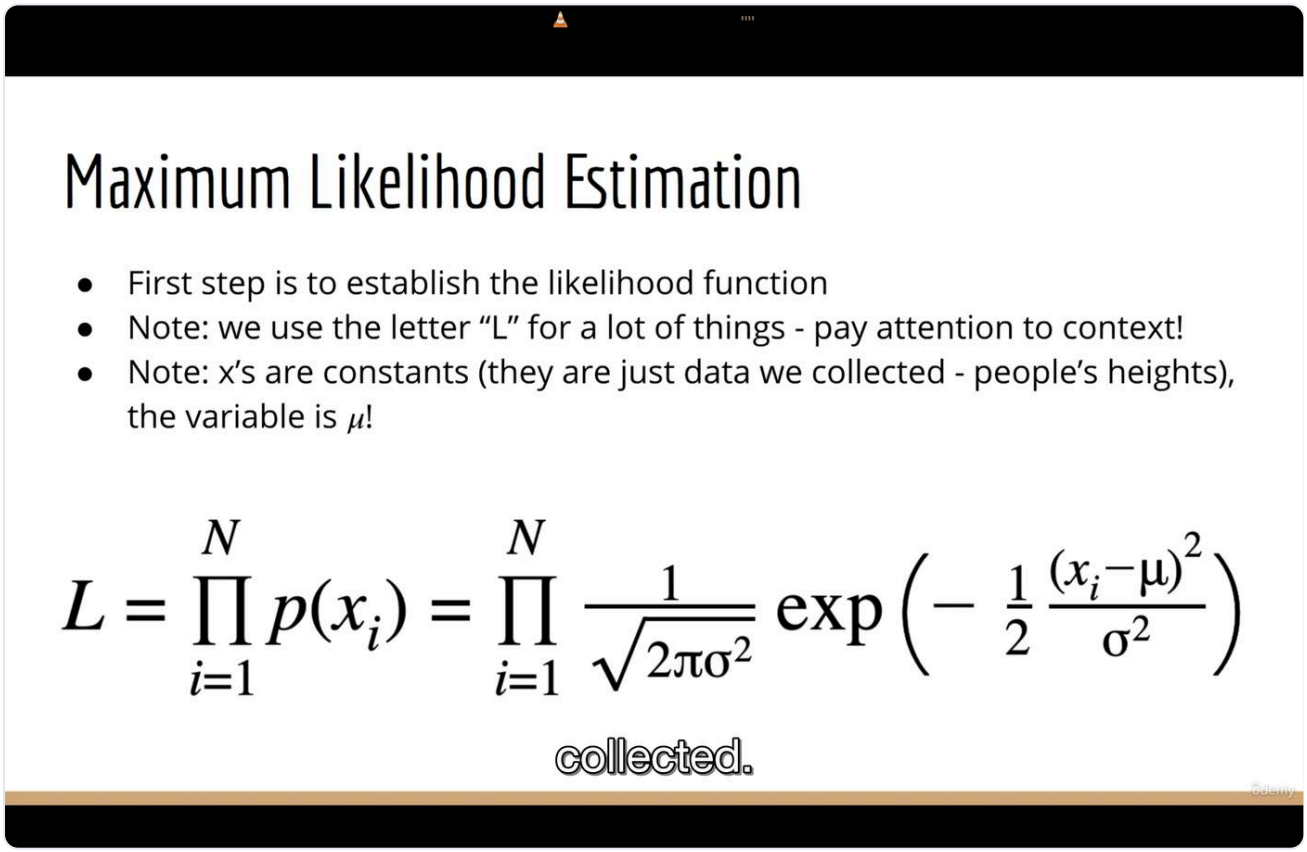

FIGURE 29.12

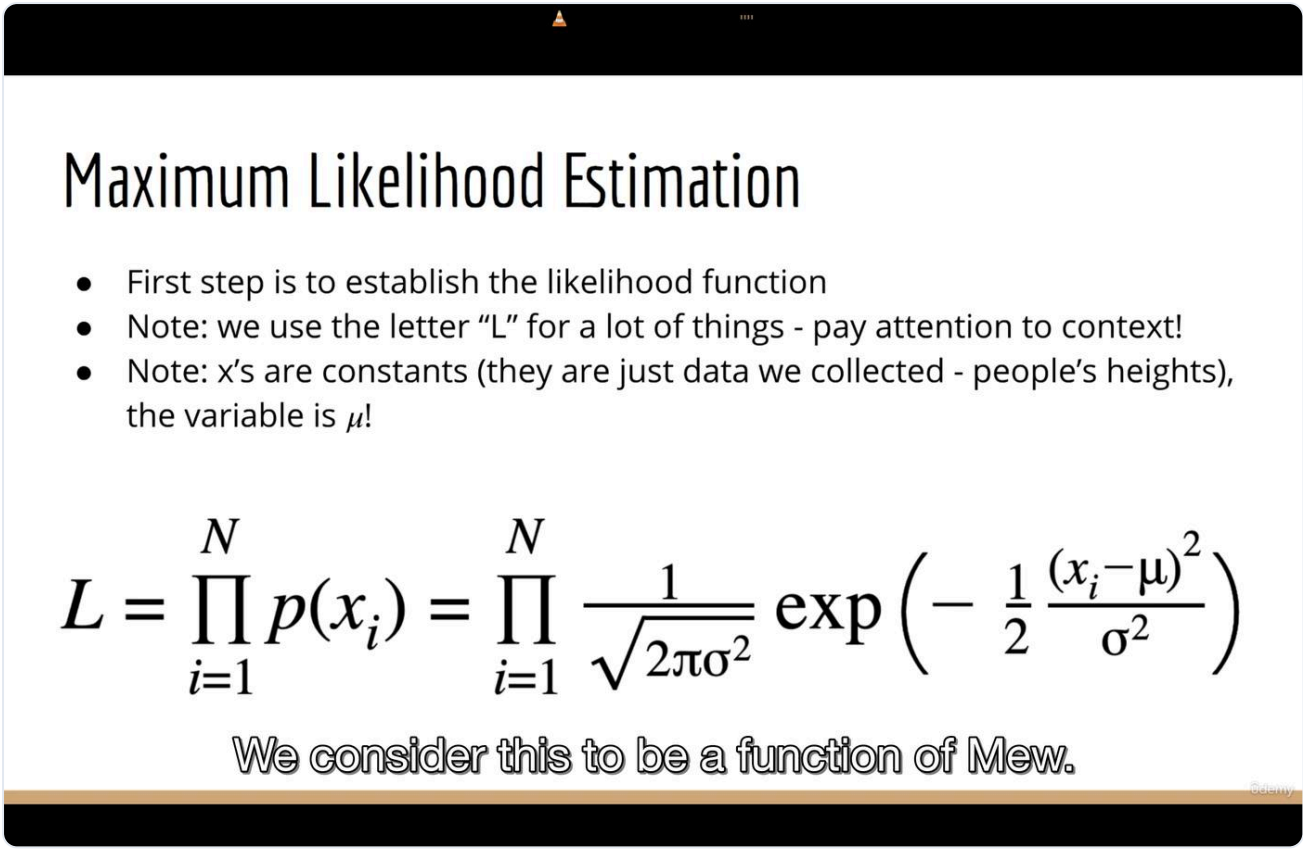

FIGURE 29.13

We will always have more samples from the negative class than the positive class.

LECTURE 123

Back to Regression

In this lecture we are going to discuss the mean squared error in depth the goal of this lecture is to ensure that you understand the mean squared error from a probabilistic perspective. This helps to give some theoretical backing for the mean squared error and it will help prepare us to discuss the cross entropy loss as well as a side note. I just want to remind you that there are multiple names that we use interchangeably. So if you hear error or loss or cost or objective. These are all generally referring to the same thing. So we might say cross entropy error but we also might say cross entropy loss they mean exactly the same thing. First let's just make sure the mean squared error makes sense in the first place.

Why is it squared. Well the idea is that we would like the error to always be positive. Imagine this scenario one of your predictions has error plus 2. The other prediction you've made has error minus 2. If you add these two errors together you would get plus two minus two which is yes zero. Of course this is not zero error because both of your predictions are off by 2. They should not cancel each other out. So squaring the error is a nice way to make it positive but this still doesn't really explain why we want the square rather than the absolute value. Why not some other method of making the difference positive. In fact it's perfectly OK to use the mean absolute error.

We just don't normally refer to that as linear regression. So when we say linear regression usually we are talking about a linear model using the mean squared error. OK so let's dig a little deeper on why we would want to square the error to understand this discussion. You have to be familiar with maximum likelihood estimation what you would normally learn in an introductory probability course. Let's say for example we would like to measure the heights of all the students in our class and we decide to model this as a Gaussian distribution. As you recall the Gaussian distribution is the typical bell curve. It has the probability density function that you see here which will become important momentarily.

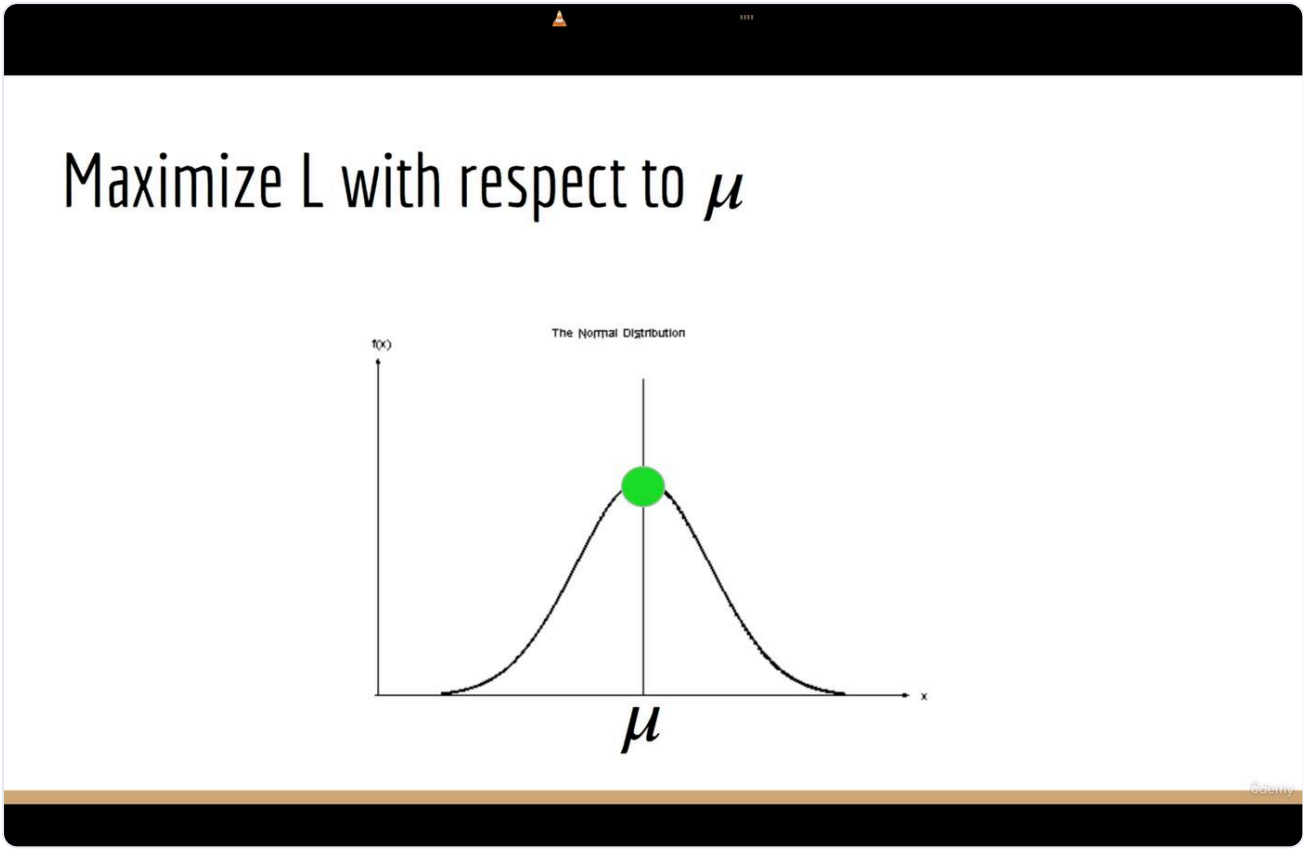

FIGURE 30.1 | Maximize L with Respect to Ys

We know intuitively that the best way to estimate mu let's call that mu hat is the sample mean of all the heights we have measured. The question is why is this so here's how we solve this problem. There is actually a method to find this mu. This method is called maximum likelihood estimation. As I mentioned earlier. So what are the steps. First we create the likelihood function let's call that L. Unfortunately we use the letter L for a lot of things and deep learning. So please don't confuse this L with anything else we've discussed. The likelihood is the multiplication of the probability density function at each of the x values we've collected. We consider this to be a function of mu.

So here MU is the variable and the X's are constants because they are just numbers. They are the heights of the students that we have measured in our class. The next thing we would like to do is maximize L with respect to mu. Let's think about why we would want to do this just for intuition. Imagine that you measure someone's height x. Now imagine mu is very far away from X in this case the probability of X is very tiny. We might think that X is not from this probability distribution because P of X yields a very small value but what if Mu is equal to x. In this case p of x yields a very large value. Thus we want p of x to be big if it is to be representative of the data we've collected so it's the same idea for L we want l to be big so that MU is close to all of the data points x that we have collected.

So how do we maximize L with respect to mu. Luckily we can just ask our old friend calculus. Normally the process would be taking the derivative of L with respect to MU and then set this derivative to zero and solve for mu. But for probabilities. Taking the log of L first usually leads to a more easily solvable derivative.

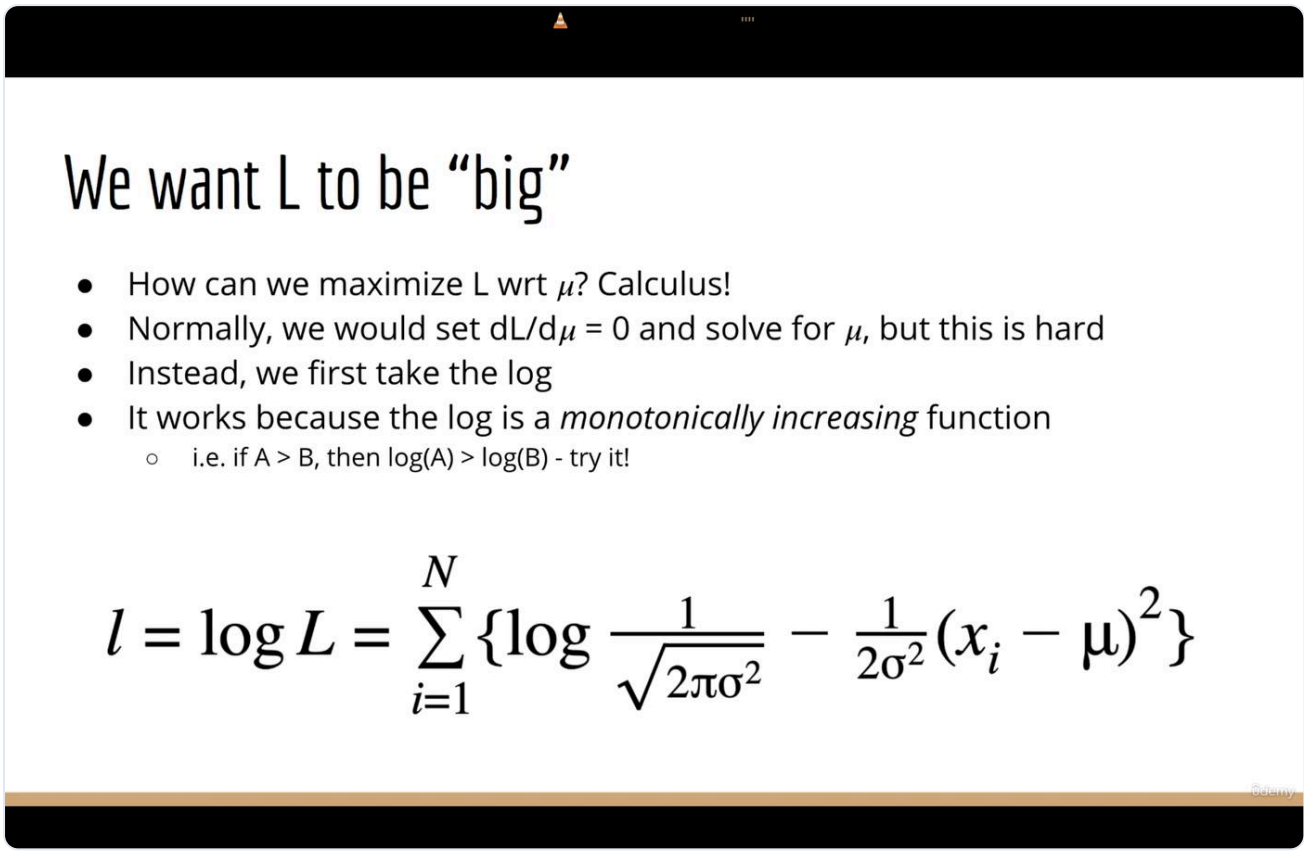

FIGURE 30.2 | Instead, We First Take the Log

The reason why this is allowed is because the log is a monotonically increasing function. Maximizing L is the same as maximizing the log of L. The value of Mu that maximizes L is the same as the value of Mu that maximizes log L. You can prove this to yourself intuitively by taking any two numbers a and b if a is bigger than B. It is also the case that log A is bigger than log B Okay so I'm not going to bore you with the calculation but as always you are welcome to do it yourself on paper using your trusty calculus skills. You should end up with the derivative we see here.

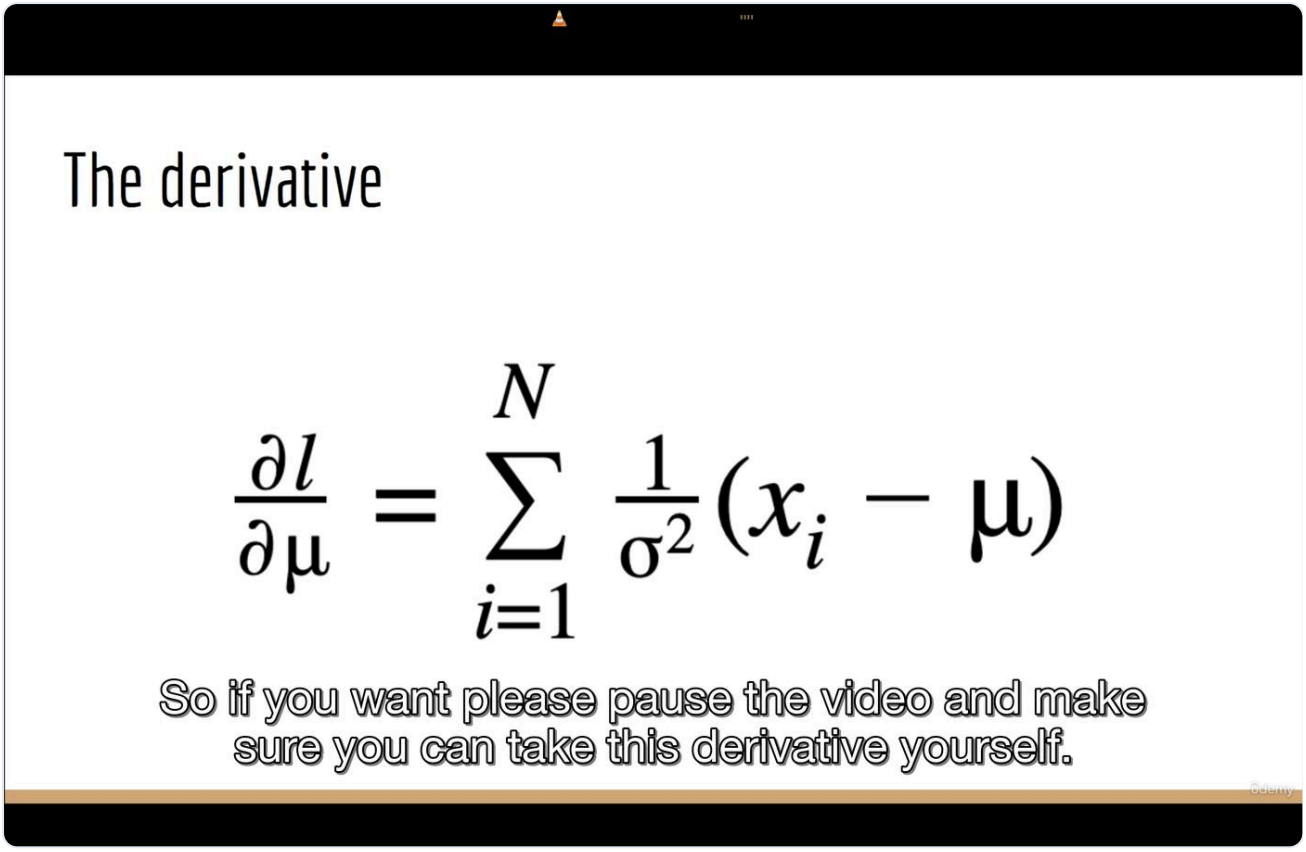

FIGURE 30.3 | The Derivative

So if you want please pause the video and make sure you can take this derivative yourself. OK so what do we get if we set this to zero and solve for mu.

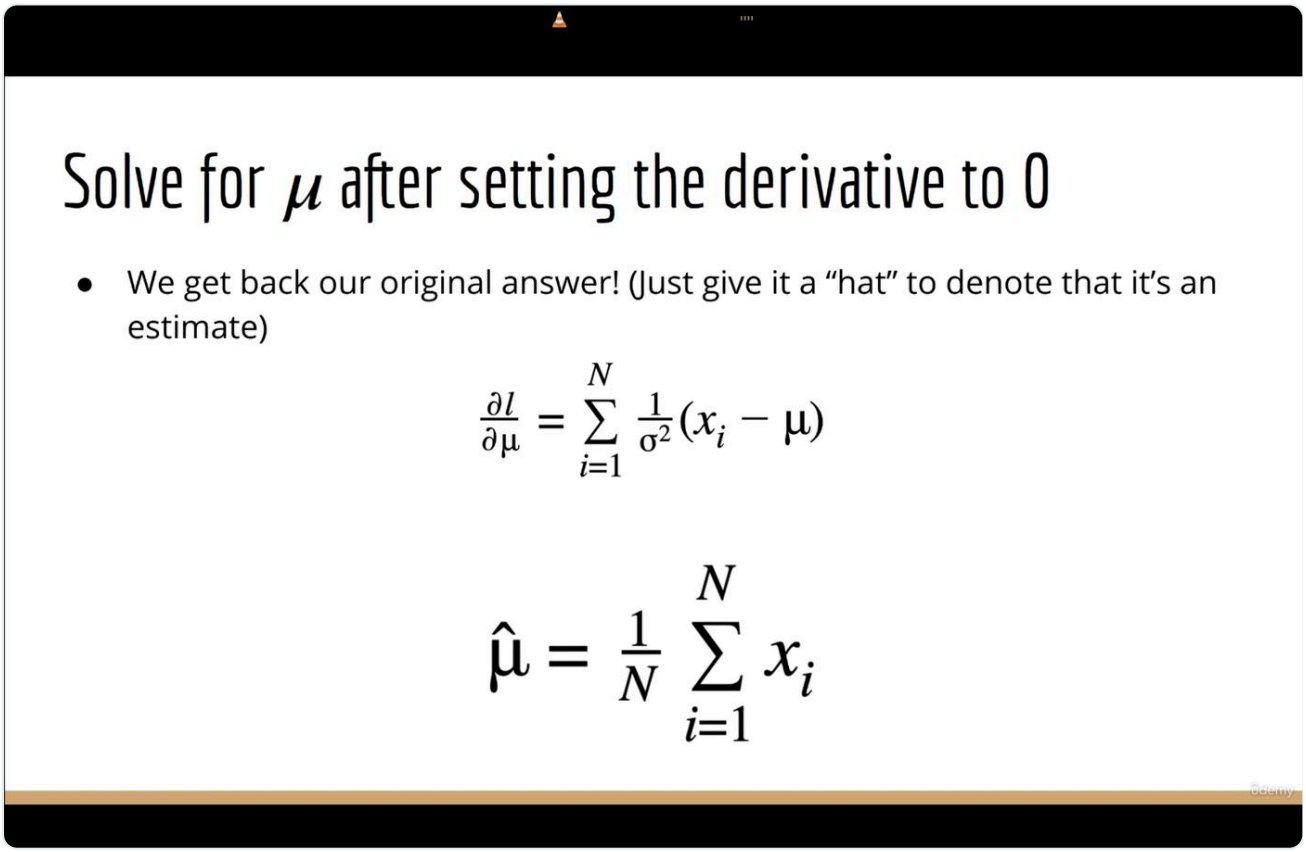

FIGURE 30.4

Well you can see that we get back our original answer that mu is the sample mean of X.. This is called the maximum likelihood estimate of Mu because in order to find it we maximize the likelihood here's what's important to notice about the log likelihood if you notice the log likelihood has the same form as this expression where I've replaced the constant terms with C1 and C2 you'll realize that it actually doesn't matter what values C1 and C2 have they don't affect the answer we get eventually that MU is the sample mean of X. Although we do recognize that C is positive C one is also positive given our old expression but it actually doesn't matter what it is at all.

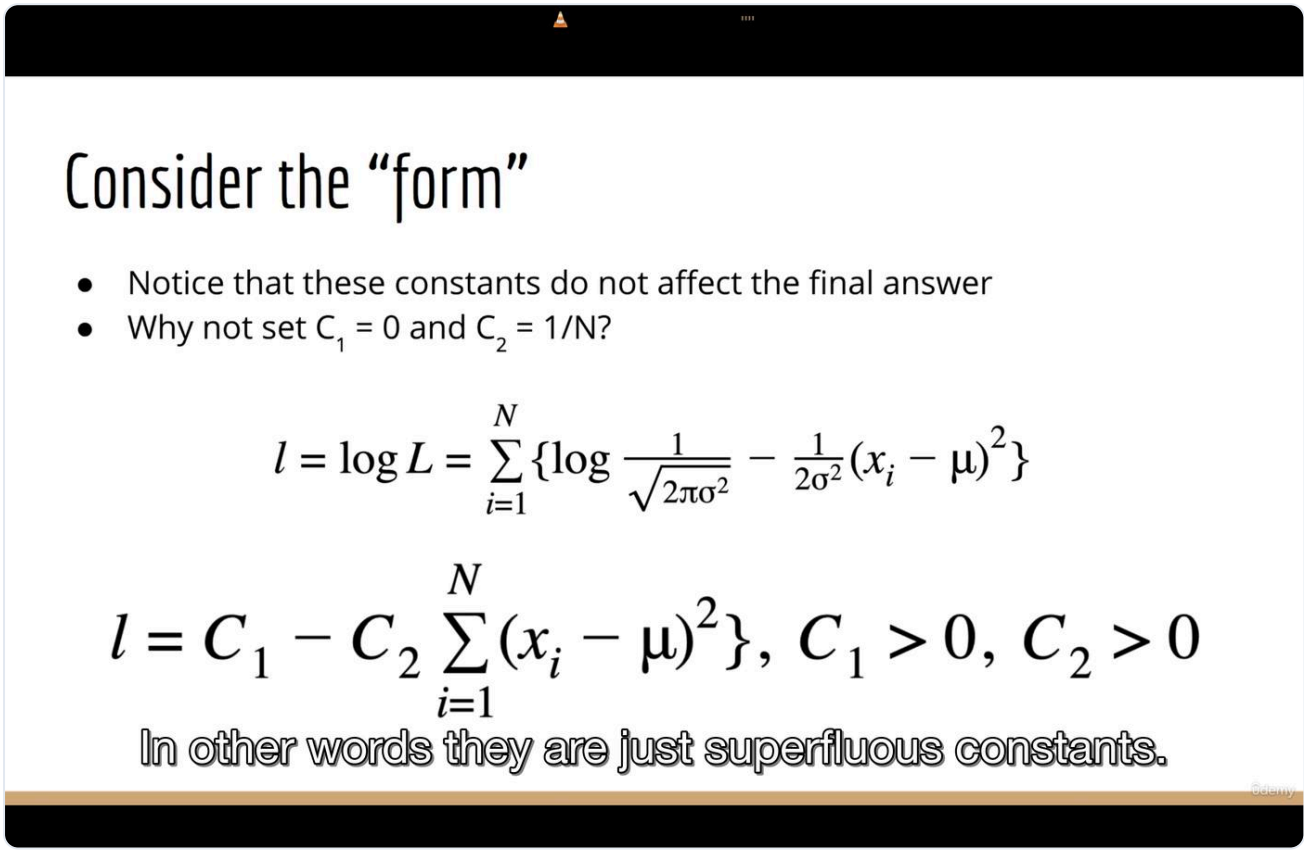

FIGURE 30.5

In other words they are just superfluous constants.

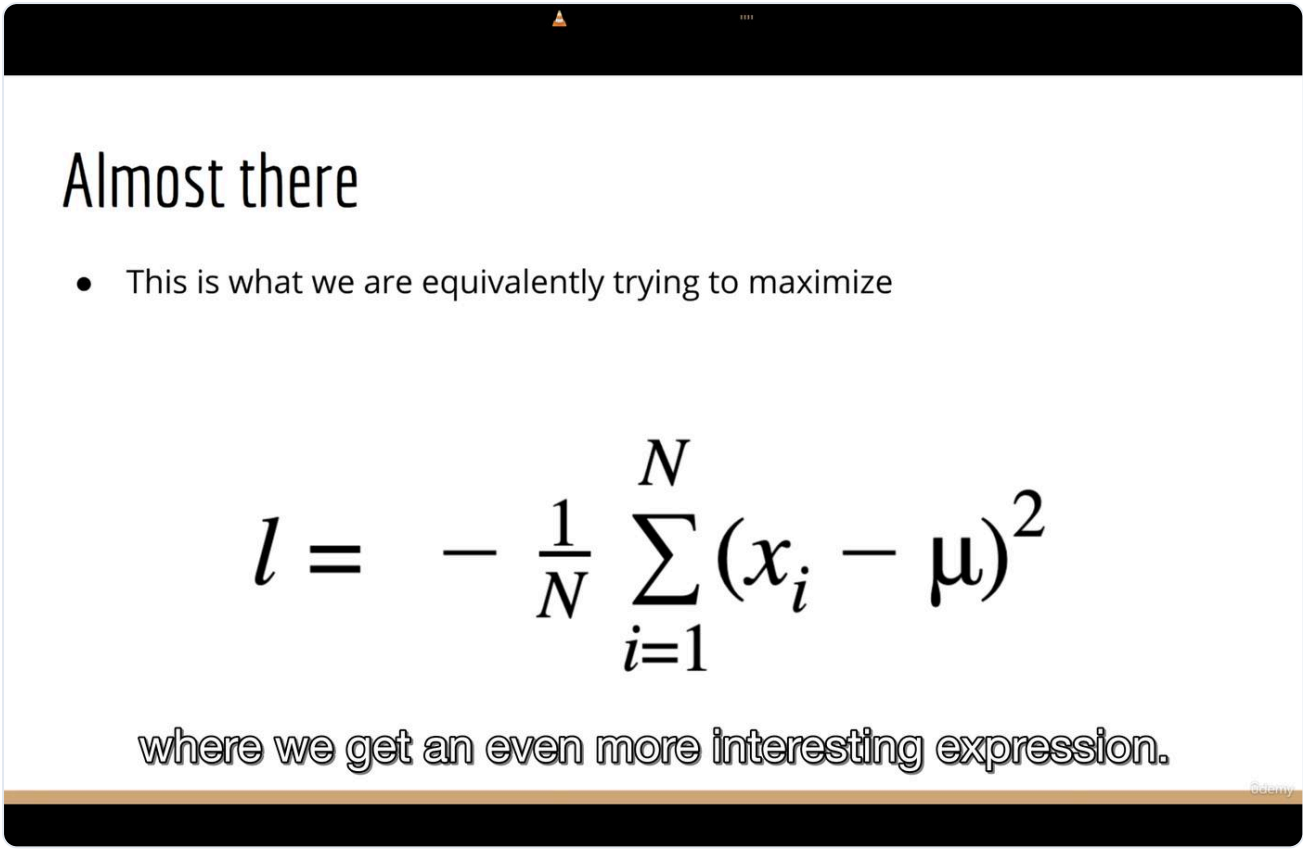

FIGURE 30.6

So what happens if we set these constants to convenient values such as 0 and 1 over n where we get an even more interesting expression. Little L.

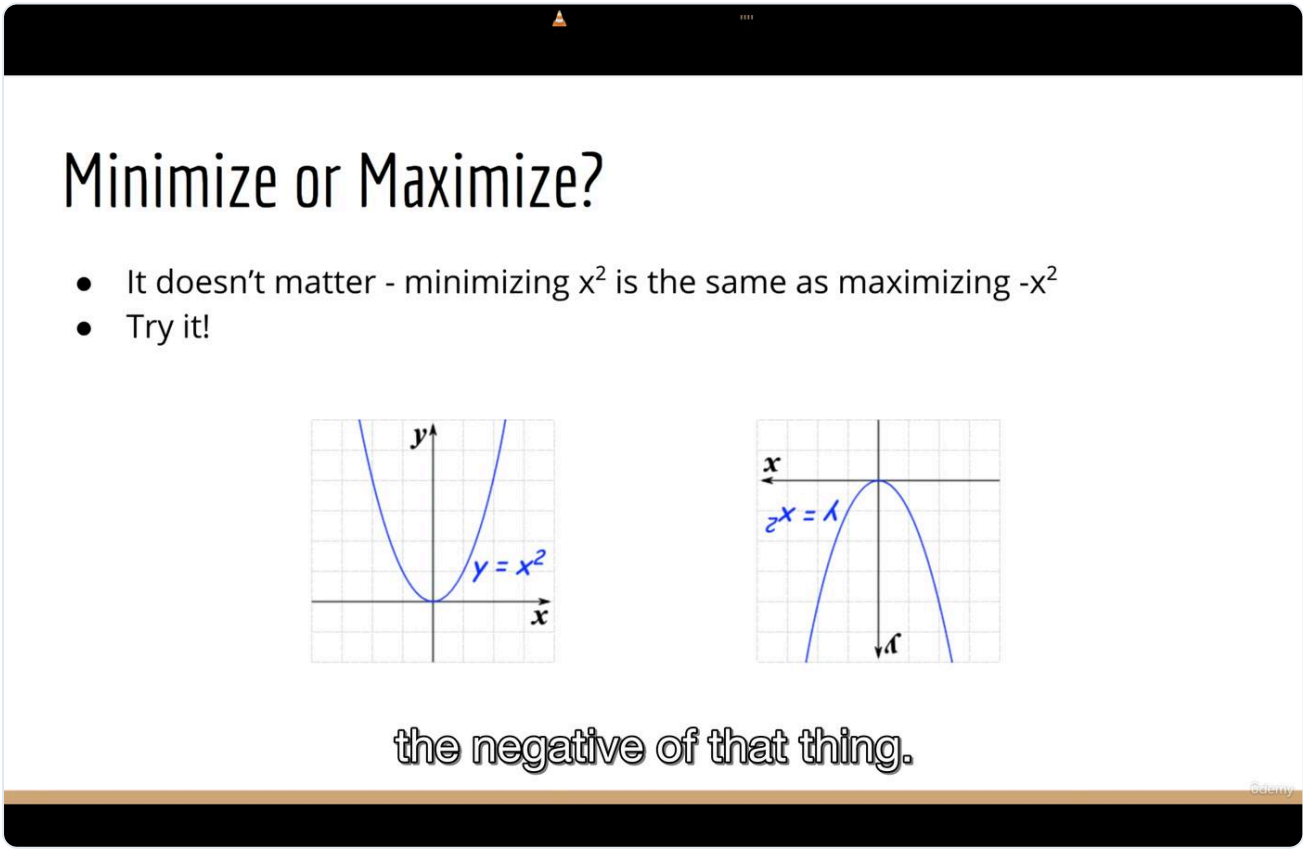

FIGURE 30.7

The thing we want to maximize is the negative sum of x i minus mu squared one thing you may recall from your study of calculus is that maximizing something is the same as minimizing the negative of that thing. In other words if I want to maximize negative x squared I should get the same answer as minimizing X squared the value x equals zero is the answer in both cases.

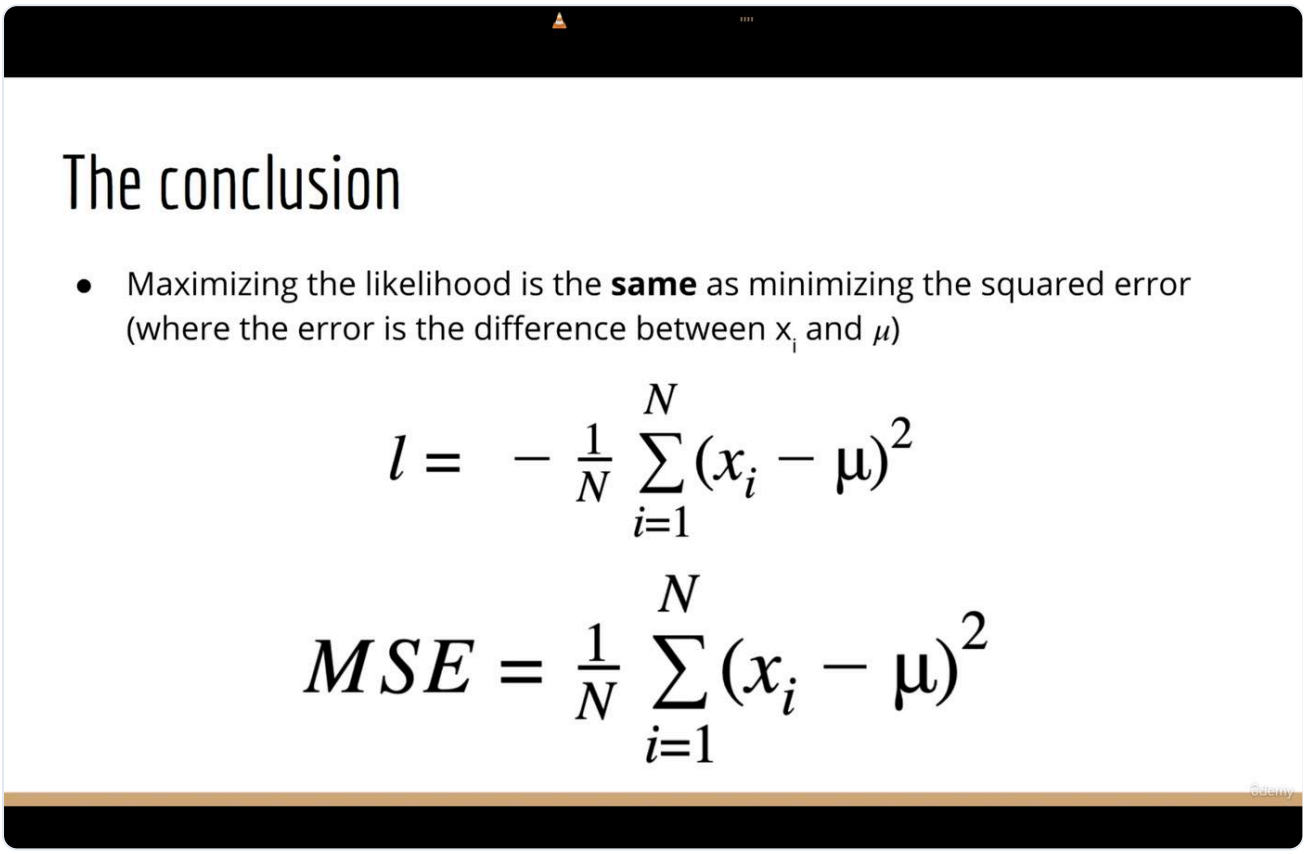

FIGURE 30.8

Hopefully you can see right away that this is obvious at this point you probably get the gist of what I'm trying to say maximizing the likelihood is the same as minimizing the mean squared error where the error is the difference between x i and mu so how can we relate this back to problems such as linear regression. Take a look at these two error functions.

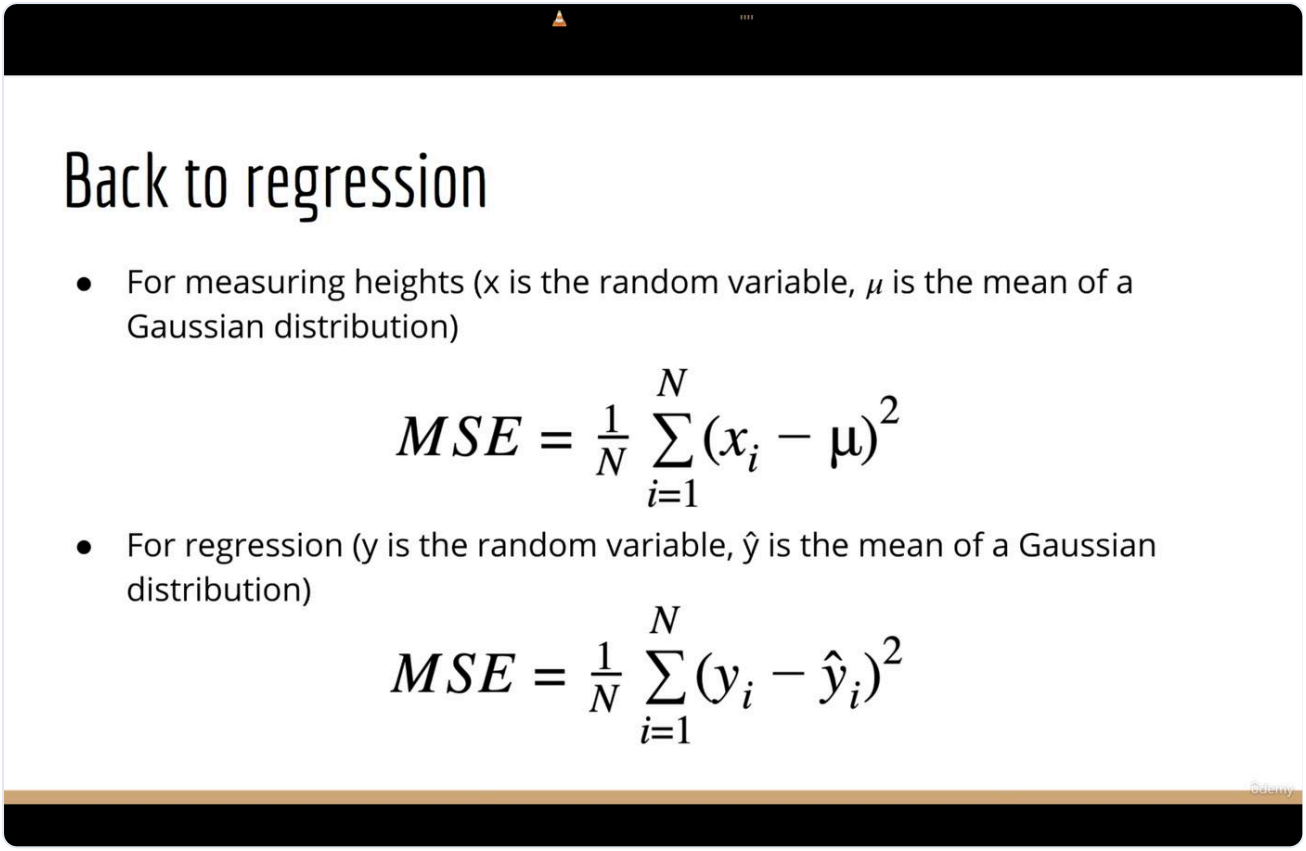

FIGURE 30.9 | Back to Regression

The first error function comes from our earlier problem finding the best mu to model the heights of the students in our class as a gaussian the second error comes from linear regression. Make the Y hats close to the Y's. How can we relate the second problem to the first problem.

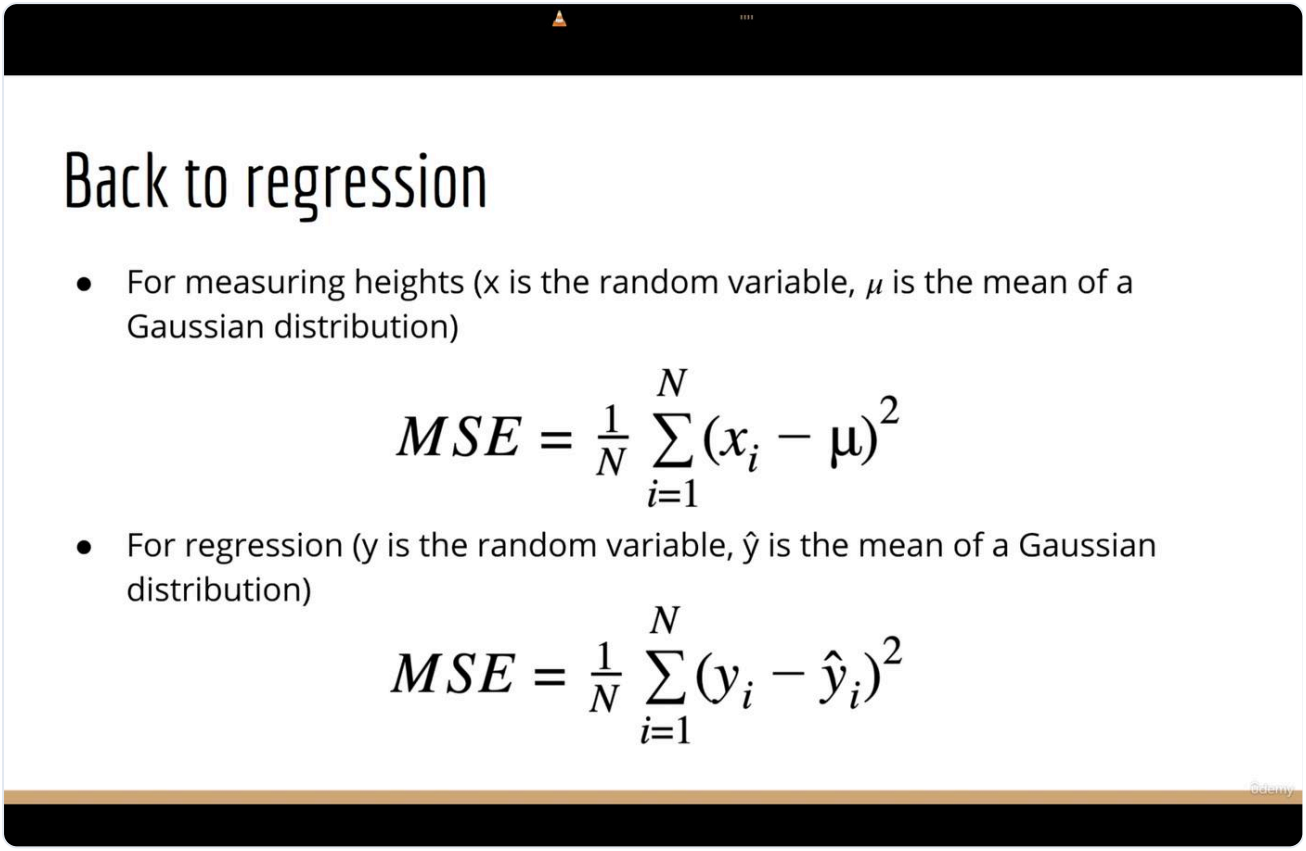

FIGURE 30.10 | Back to Regression

Well if we say that the y i's are samples drawn from a Gaussian distribution with a mean of y hat i then minimizing this mean squared error is the same as minimizing the negative log likelihood and that of course is the same as maximizing the log likelihood which is the same as maximizing the likelihood finally. One thing we can do is instead of saying that the mean of the distribution is y hat we can simply say that Y minus y hat is a Gaussian centered at zero equivalently.

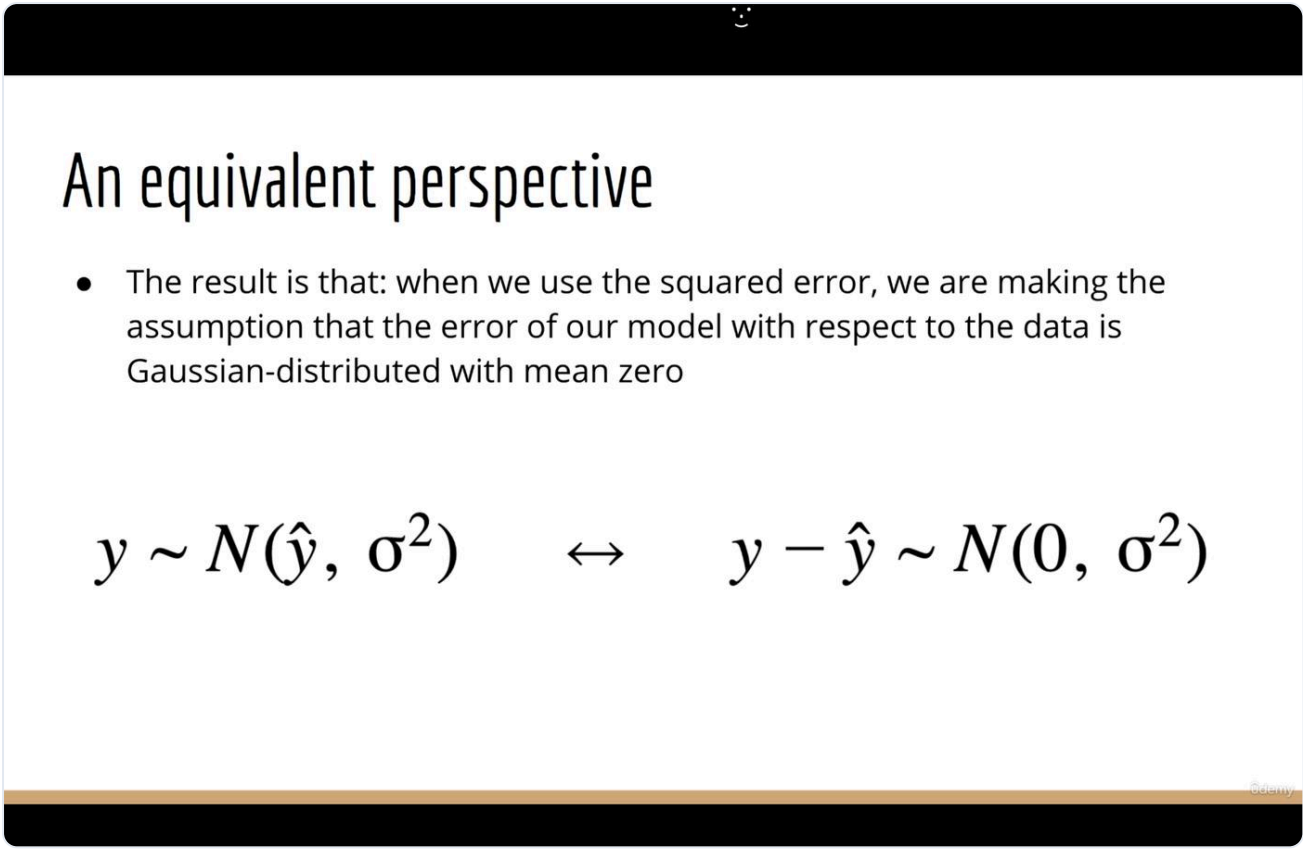

FIGURE 30.11 | Gaussian-distributed with Mean Zero

This is saying that when we use the squared error we are making the assumption that the error of our model with respect to the data is Gaussian distributed with mean zero. This formulation helps us to put a probabilistic backing on the error function and it will also help us make sense of the Cross entropy loss for classification.

LECTURE 124

The Bernoulli

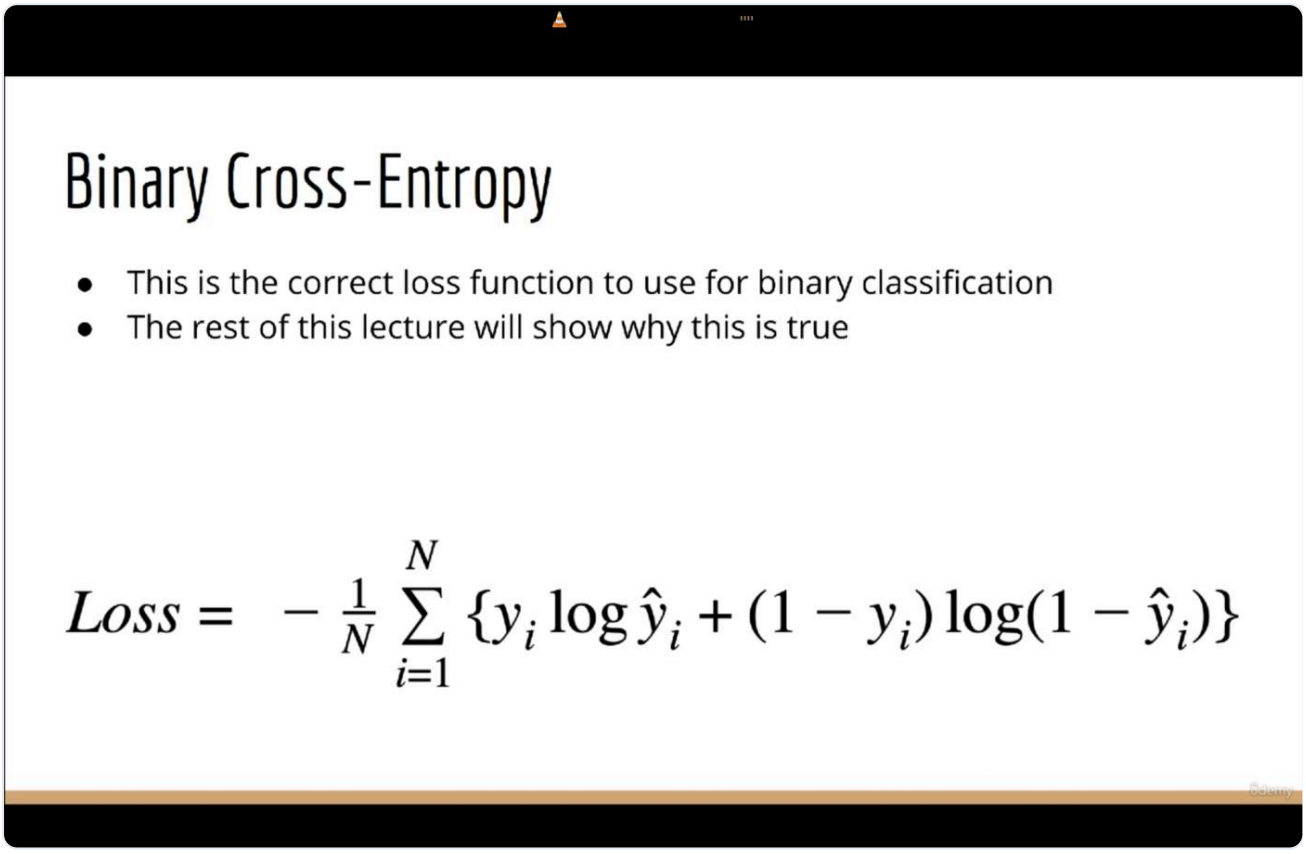

FIGURE 31.1

In this lecture we are going to discuss the binary cross entropy loss which is the correct loss function to use when we are doing binary classification. Let's recap the actual loss function and then we will use the rest of this lecture to determine how to arrive at this loss function. Since we know that this is going to be based in probability let's think about the associated probability problem. We know that the outcomes the Y's must be binary. That means the targets in a binary classification problem must be 0 or 1 spam or not spam fraud or not fraud etc..

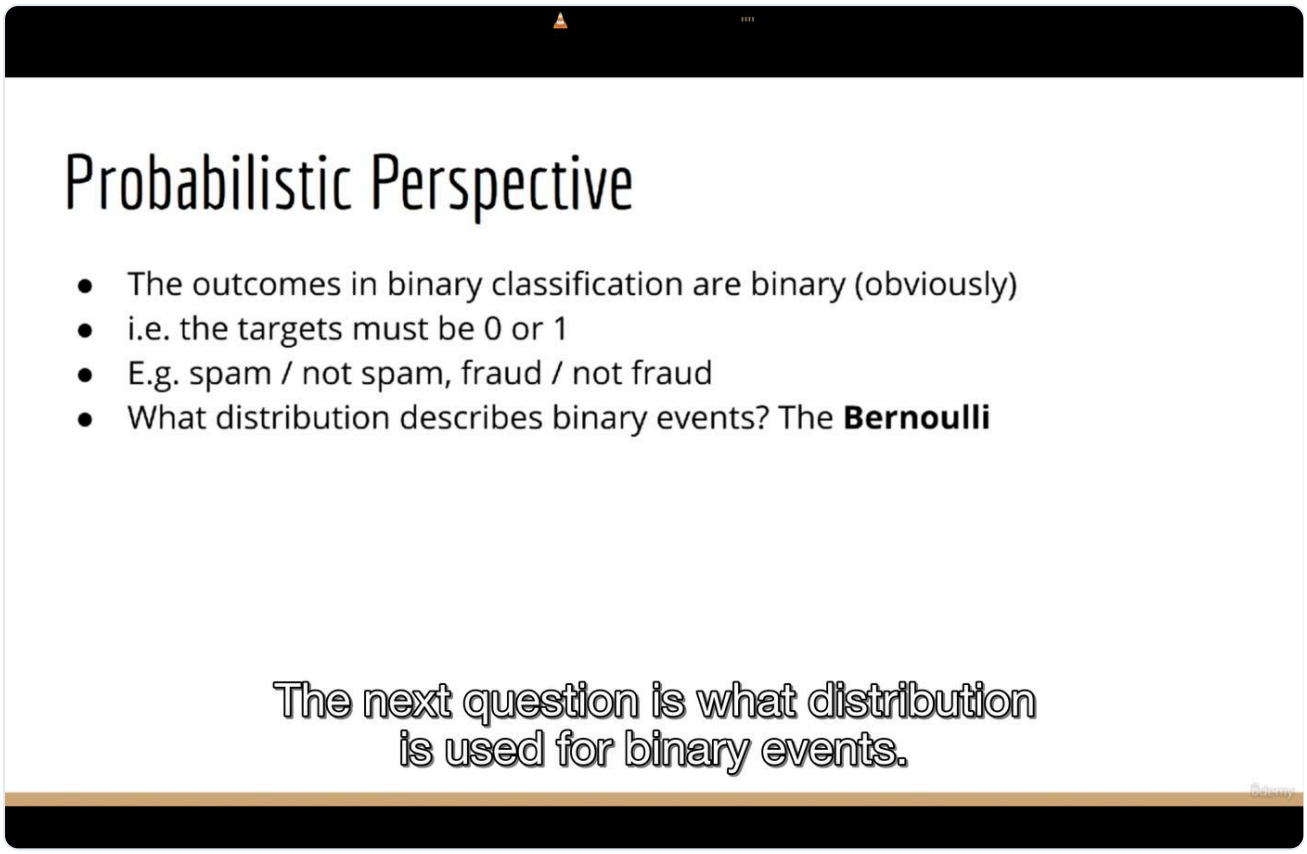

FIGURE 31.2 | The Outcomes in Binary Classification Are Binary (Obviously)

The next question is what distribution is used for binary events. The answer is the Bernoulli distribution probably the most typical example of the Bernoulli distribution is the coin toss. So imagine if you tossed a coin a bunch of times and you want to calculate the probability of heads. Call that mu. Intuitively we know that MU is the number of heads divided by the total number of coin tosses.

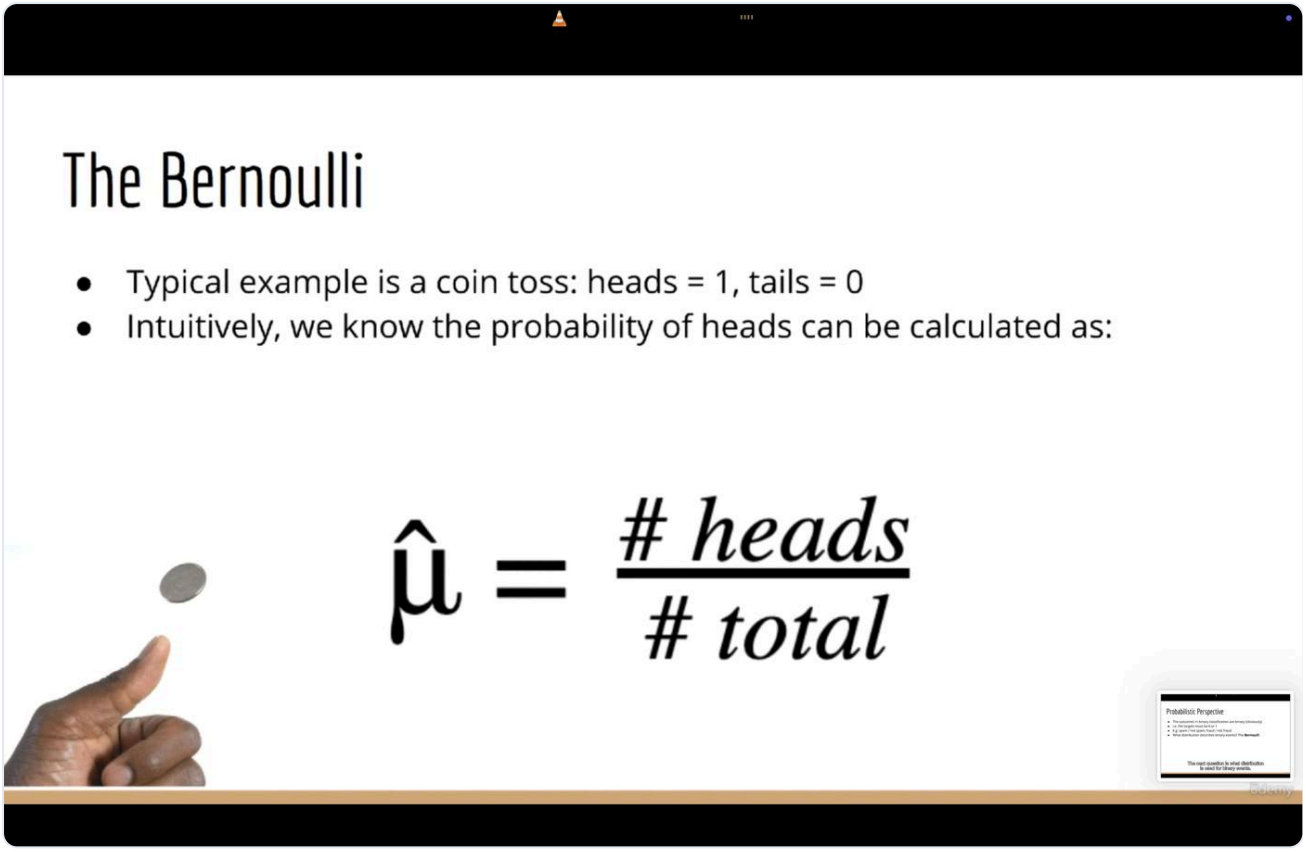

FIGURE 31.3 | The Bernoulli

Again we are going to go through the process of maximum likelihood estimation to see how we can arrive at this answer. All right. So let's say we've tossed our coin a bunch of times. Let's call the results x1 x 2 all the way up to X N. In this case the x i's can only take on 2 values 0 or 1.

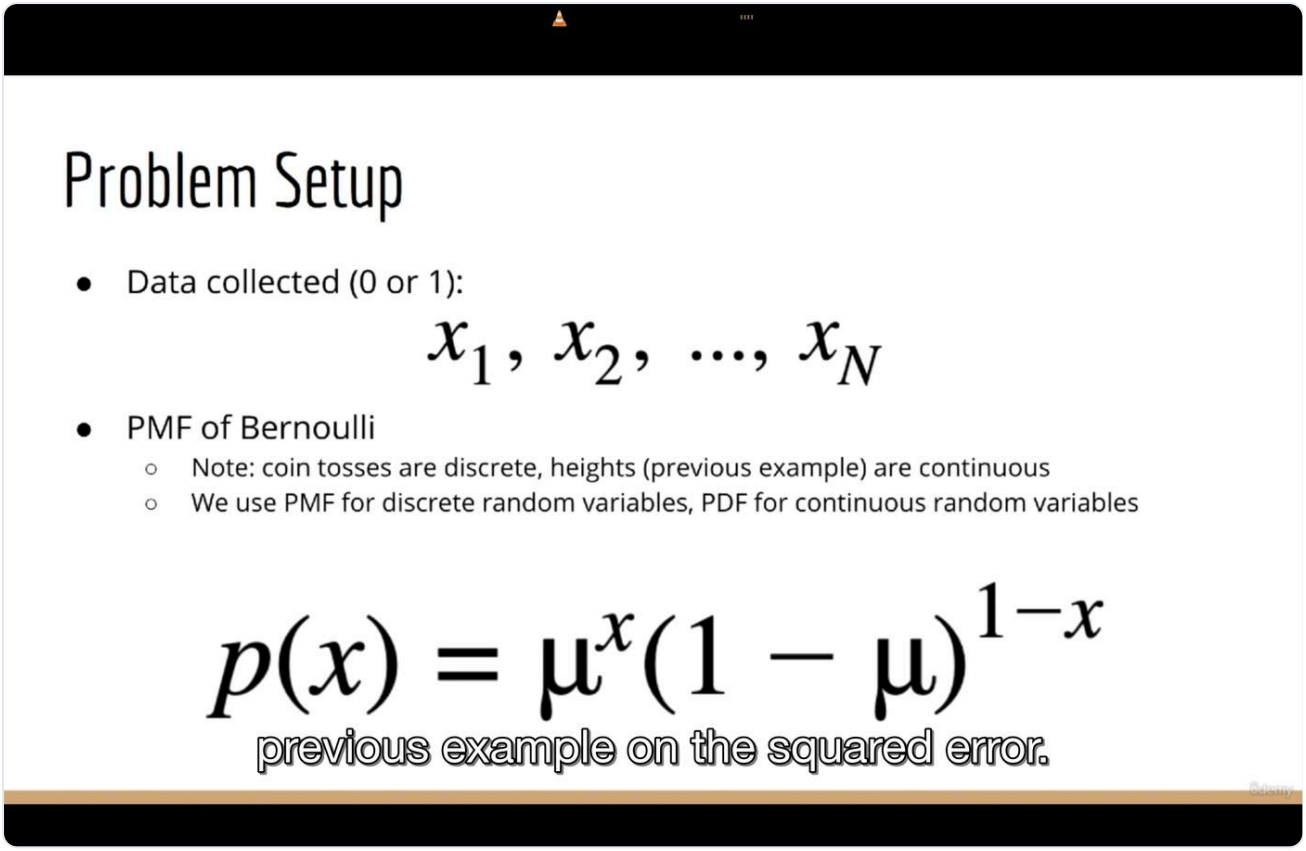

FIGURE 31.4

Let's say 1 means heads and zero means tails in order to calculate the likelihood we need the Bernoulli PMF which is the equation you see here now there's one small difference between this example in the previous example on the squared error. The difference is that for the previous example we were working with heights which are continuous valued. Now we are working with coin tosses which are discrete a coin toss can only give you 0 or 1. It can't give you say zero point five zero point nine. So we know from my study of probability that for continuous distributions we use the probability density function but for discrete distributions we use the probability mass function or PMF.

So this is a PMF and it returns a probability rather than a probability density since you already know the steps. Let's get right to it.

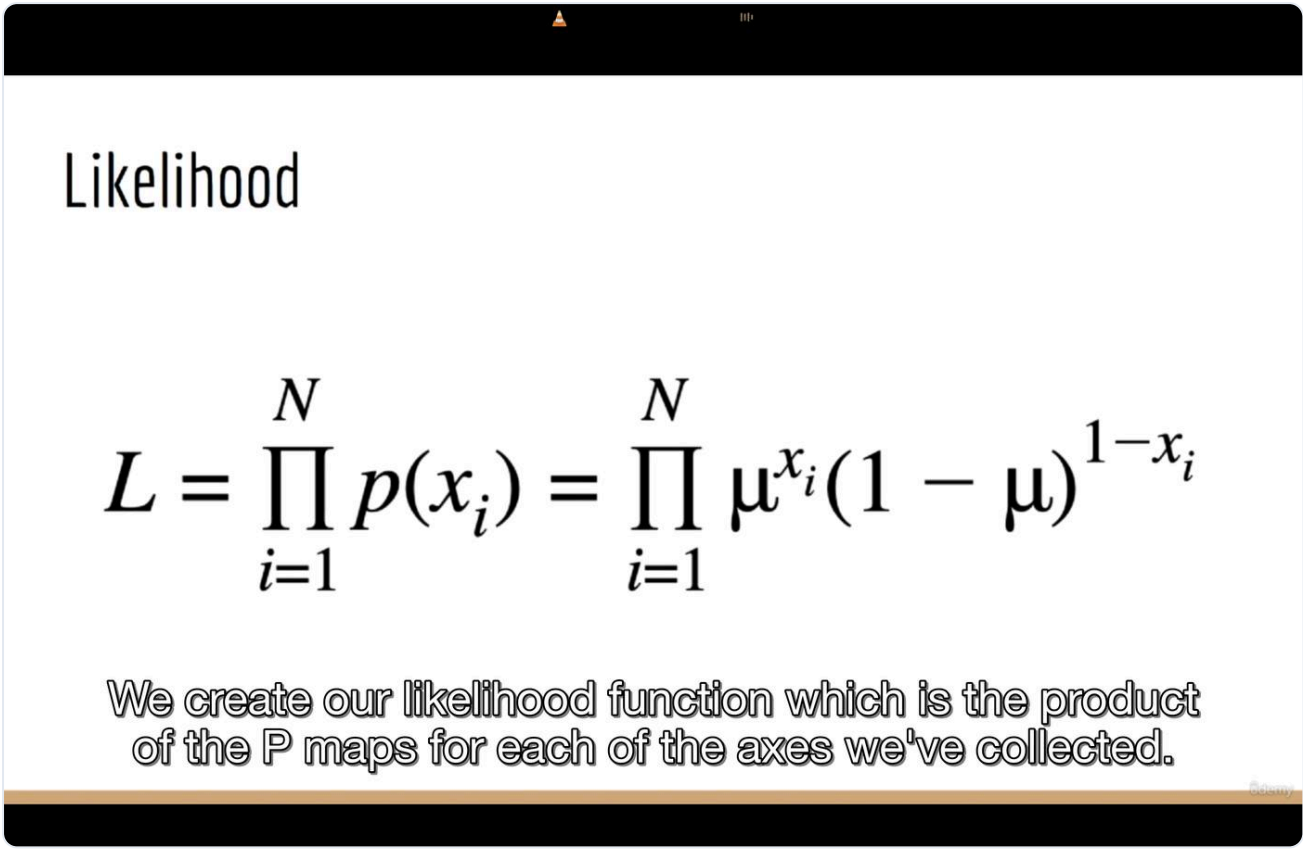

FIGURE 31.5

We create our likelihood function which is the product of the PMFs for each of the x's we've collected.

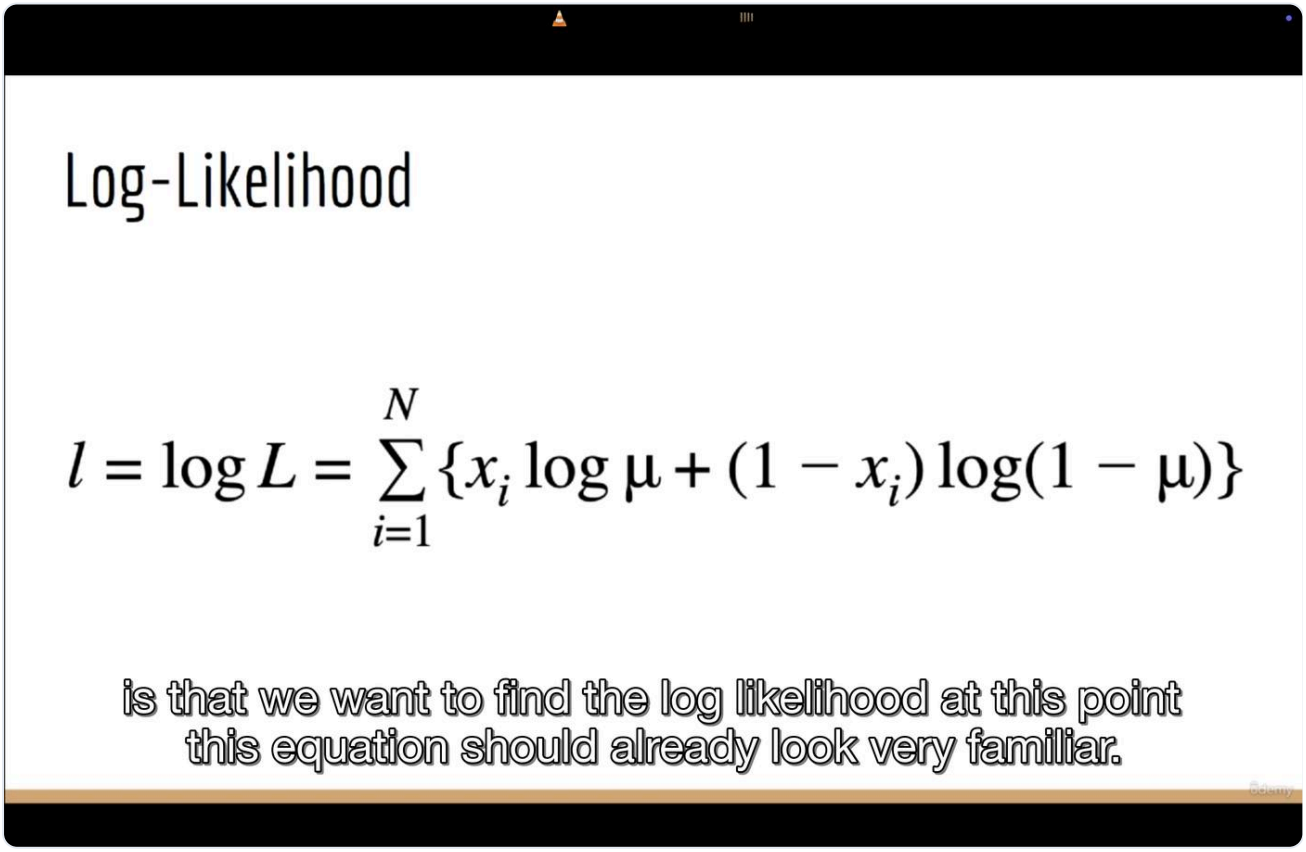

FIGURE 31.6

We know that we want to maximize the likelihood but the step we have to do before taking the derivative is that we want to find the log likelihood at this point this equation should already look very familiar. It's in the exact same form as the binary cross entropy except it negated as usual if you were to take the derivative of the log likelihood set it to zero and solve for mu. This is what you would get. Curiously we arrive at the same answer as the Gaussian case the sample mean of x. How can this be. Why isn't it.

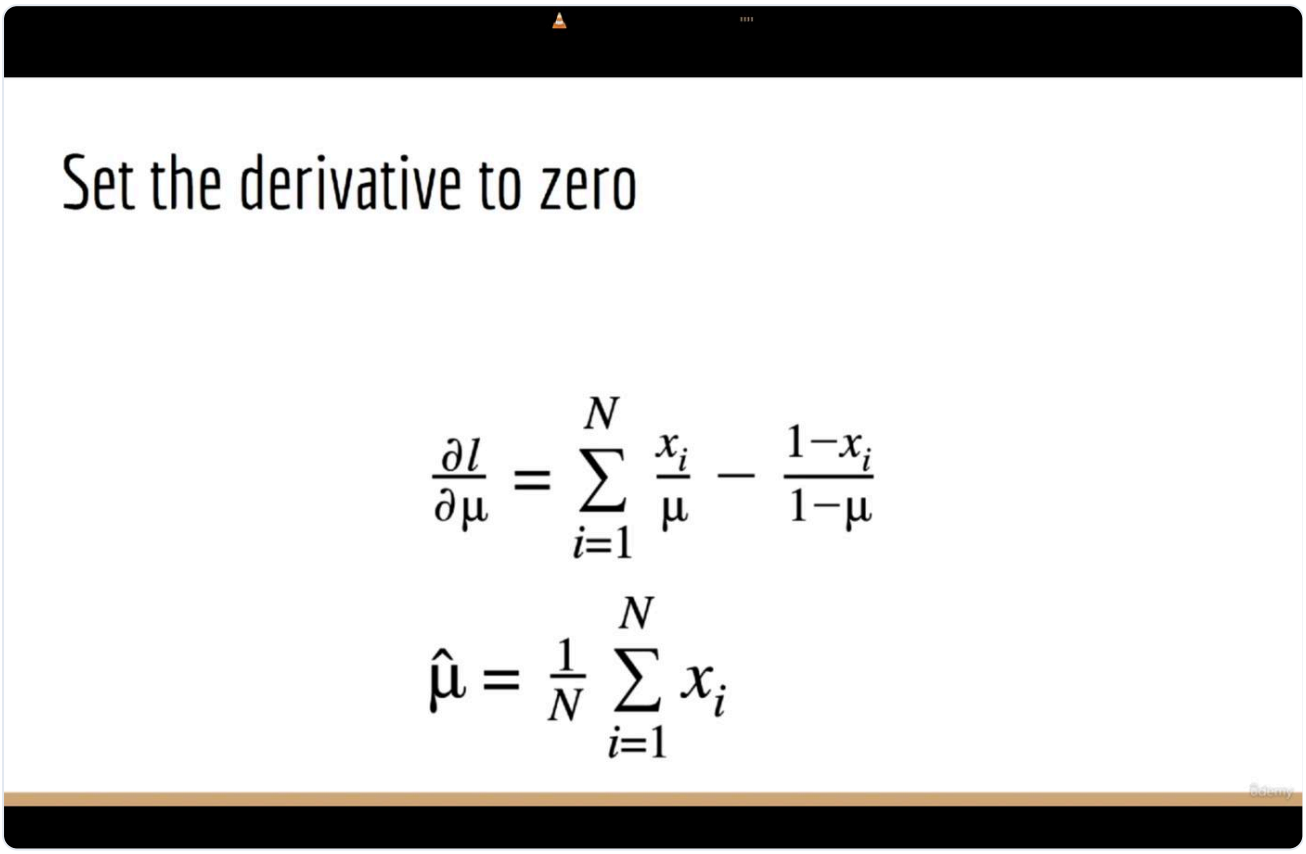

FIGURE 31.7 | Set the Derivative to Zero

Number of heads divided by n. In actuality it is. Remember that X can only take on the values 1 and 0. So when x is 0 that's tails. So they don't contribute anything to the sum of X the sum of X then is just the sum of a bunch of ones which is the number of times we flipped heads.

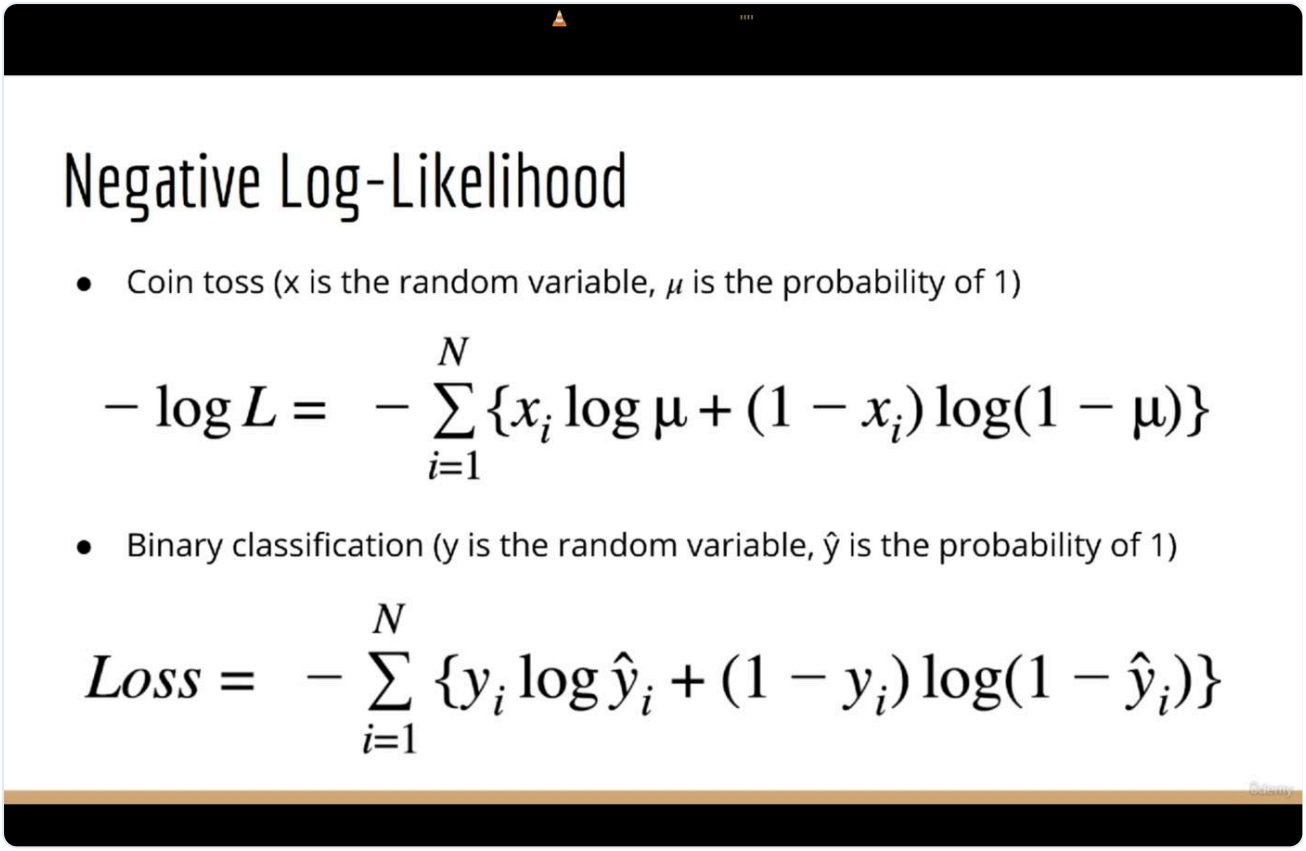

FIGURE 31.8

So in fact this is exactly the same as number of heads divided by n as before. If we take the negative log likelihood and we put it side by side with the binary cross entropy we can see the similarities. We can conclude that what the binary cross entropy is really saying is that y is the result of a coin toss with a probability of heads for that coin toss is given by y hat i. Analogously x i is the result of a coin toss where mu is the probability of heads for that coin toss as a side note. You want to keep in mind that we also usually divide the sum of the errors by N so that we get the average binary cross entropy per data point. This makes it just like the mean squared error in that the value is invariant to the number of samples we have.

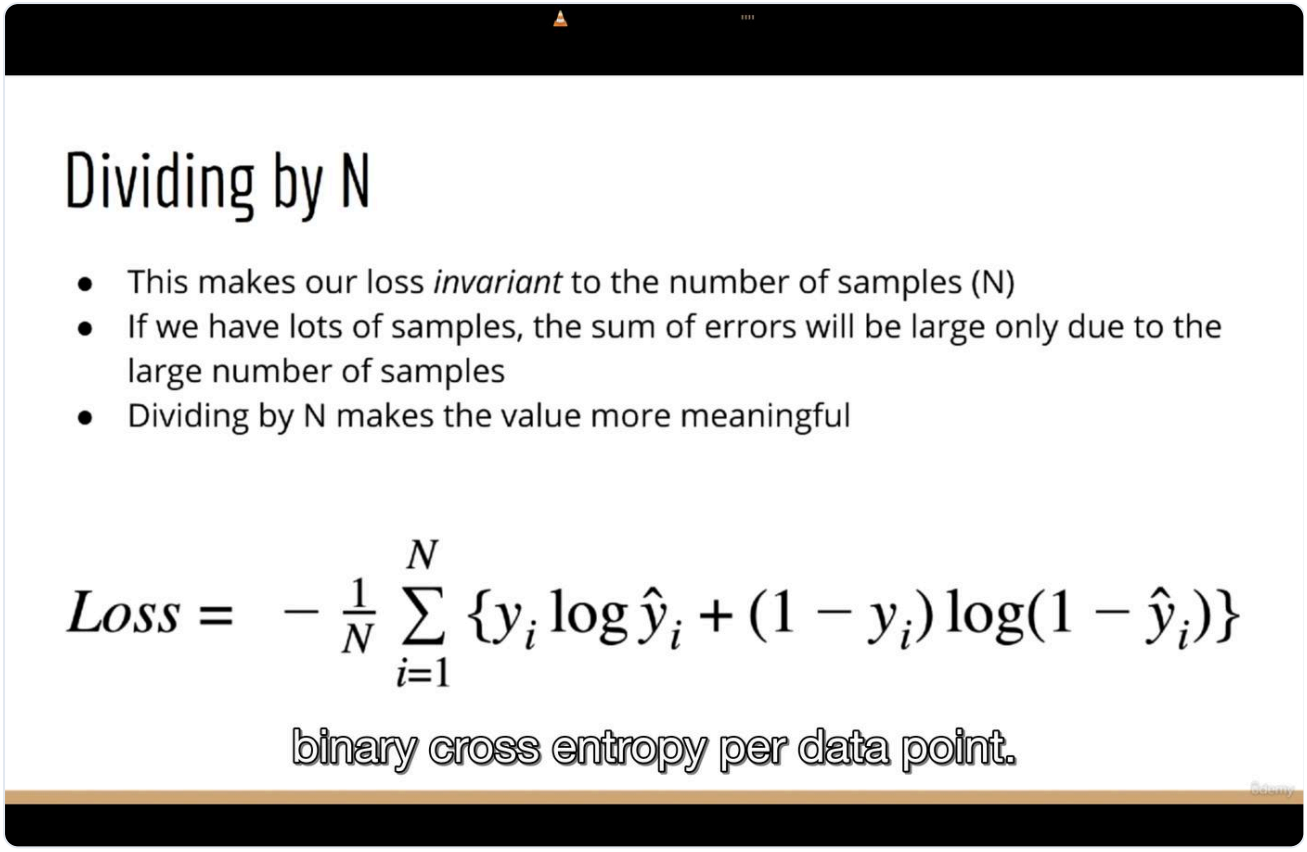

FIGURE 31.9

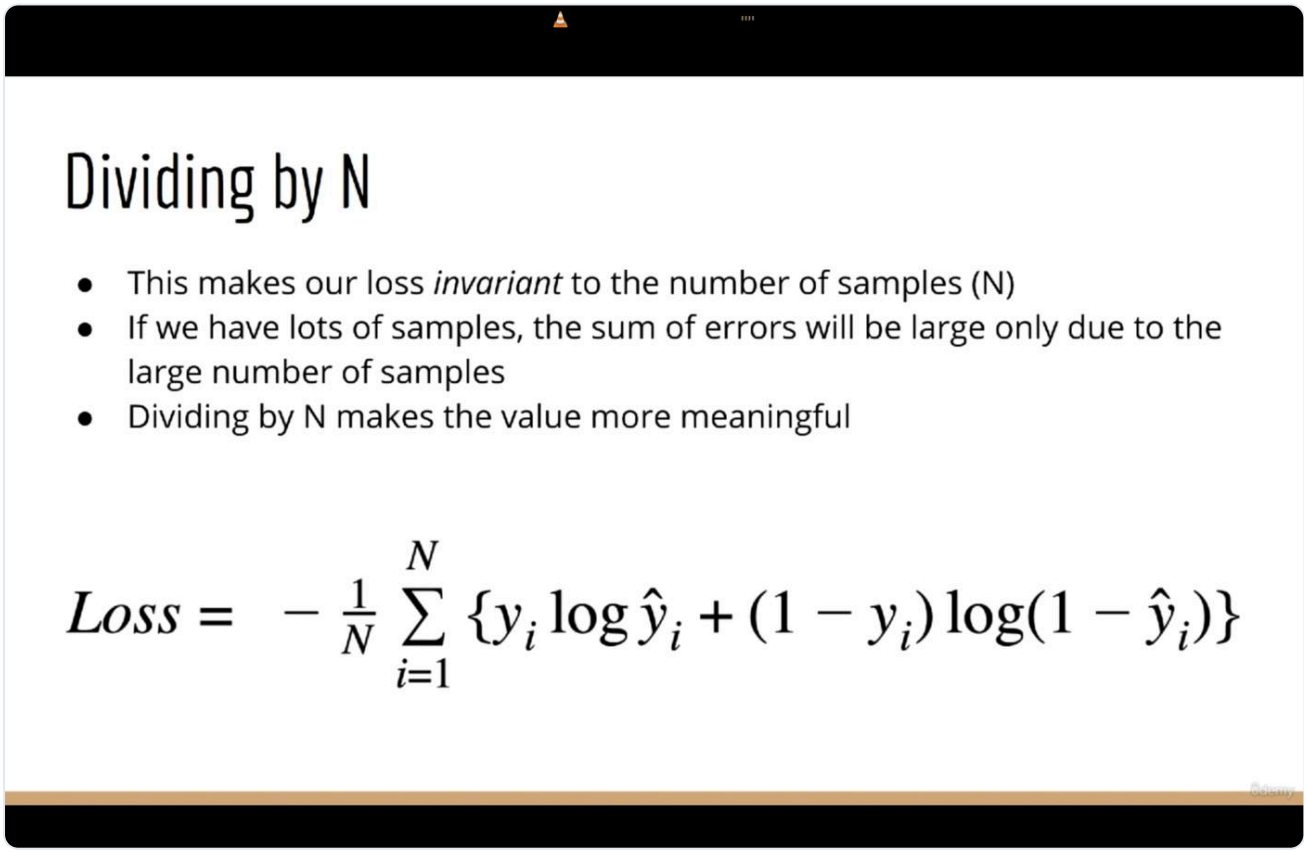

FIGURE 31.10

You can imagine that if we have lots of samples and we only use the sum the error would be very large only due to the fact that we have a large number of samples in order to make the value of the error more meaningful. We can take the average and that makes our error more interpretable So to conclude what have we done in this lecture. We have shown that just like the mean squared error the binary cross entropy loss function is based in probability. What it says is that for the regression case where we use the mean squared error it arises from the Gaussian distribution for the binary classification case where we use the binary cross entropy it arises from the Bernoulli distribution.

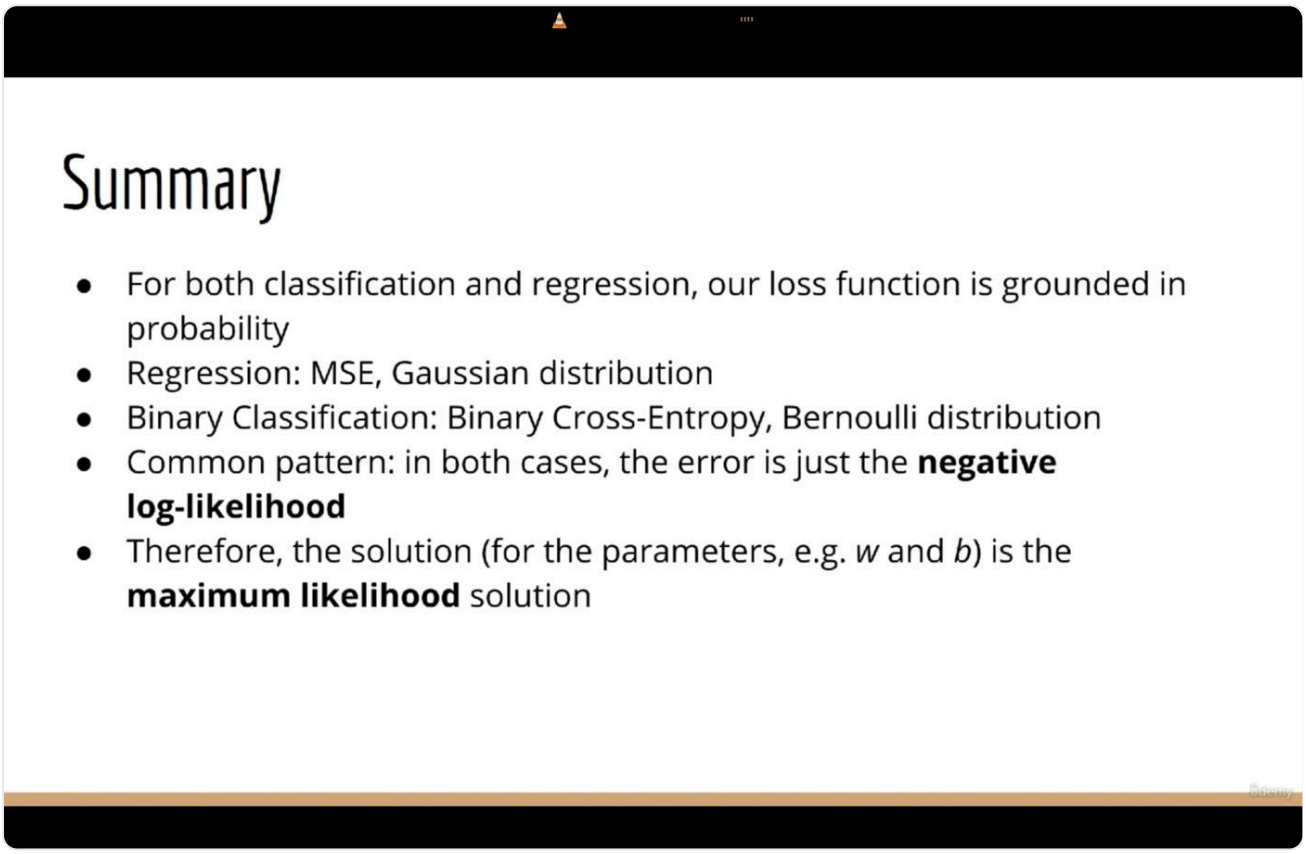

FIGURE 31.11 | Regression: MSE, Gaussian Distribution

Importantly the common pattern between the two is that in both cases the error function is really just the negative log likelihood and therefore the solution we end up finding in the end is called the maximum likelihood solution.

LECTURE 125

One-Hot Encoding Is Inefficient

In this lecture we are going to derive the categorical cross entropy loss function which is what we use when we are doing multi class classification.

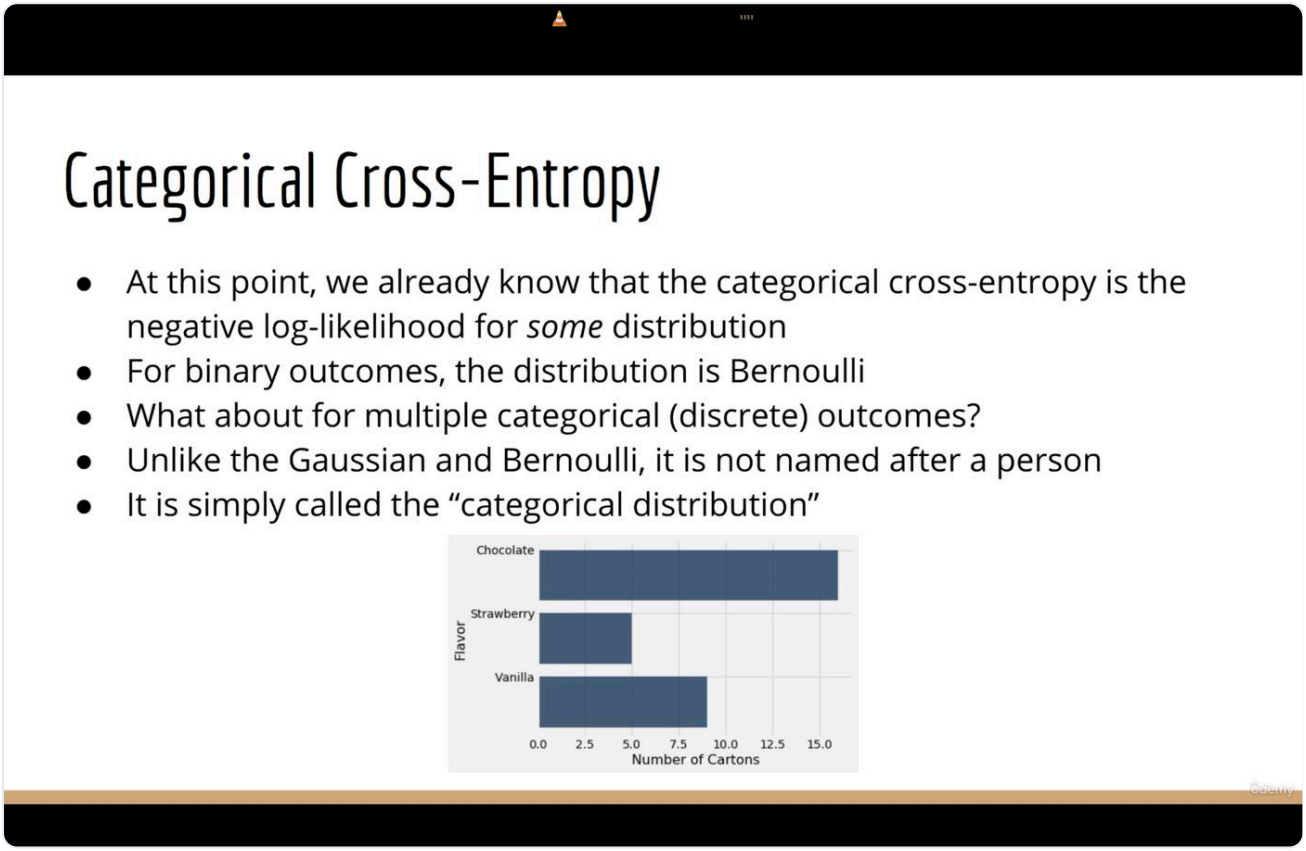

FIGURE 32.1 | For Binary Outcomes, the Distribution Is Bernoulli

At this point you probably already know that the categorical cross entropy is the negative log likelihood for some distribution. So let's jump right in and try to figure out what this distribution is. In the case where we only have binary outcomes the correct discrete distribution is the Bernoulli distribution. So what is the correct distribution for multiple categorical outcomes. Well unlike the Gaussian and Bernoulli distributions this distribution is not named after a person. It is simply called quite appropriately the categorical distribution the usual analogy for the categorical distribution is a die roll. So when you roll a die. There are six possible outcomes.

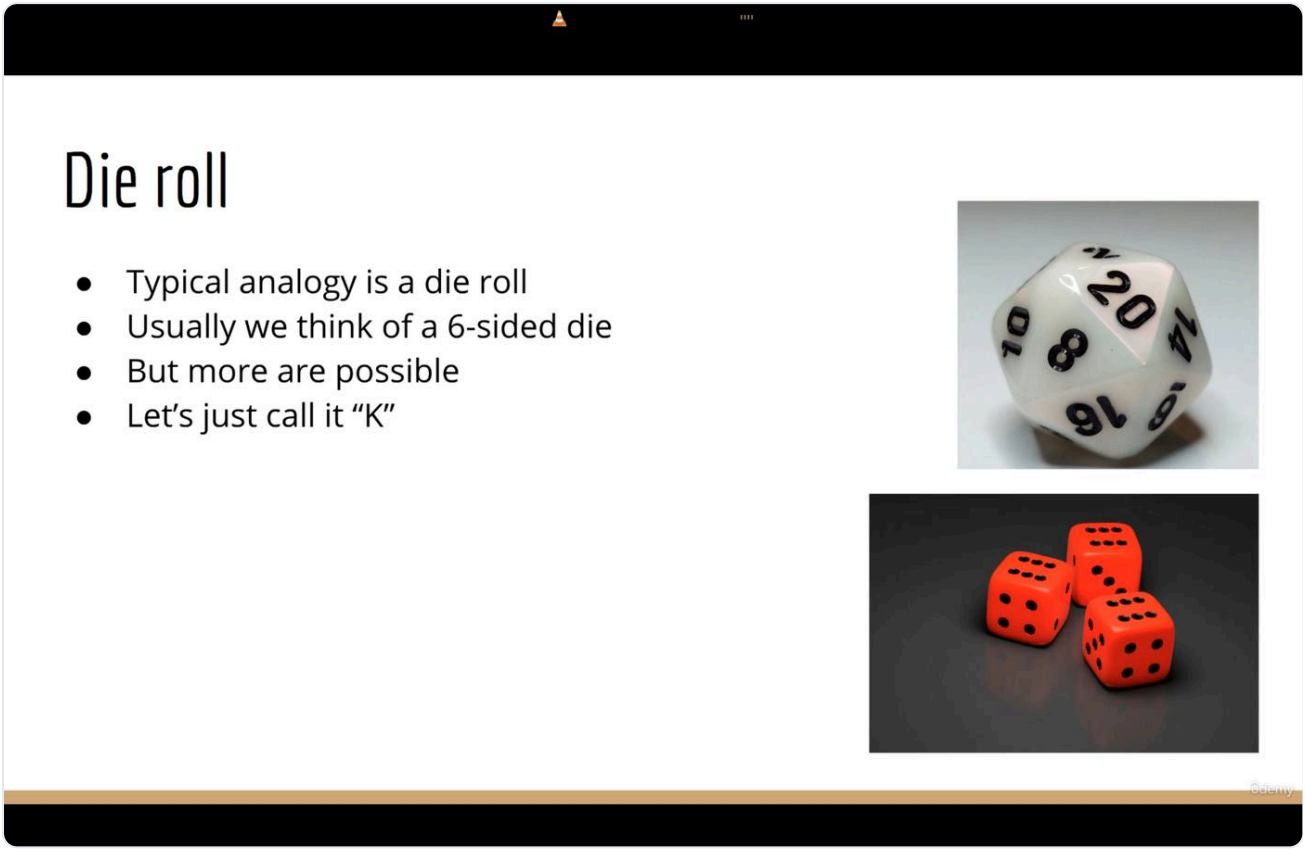

FIGURE 32.2 | Die Roll

Typically one two three all the way up to six. Of course there are different kinds of dice. Well you may have even more possible outcomes. In general we will refer to the number of possible outcomes as K. So what does the categorical distribution look like. Well it looks a little strange as you can see it is a product each of the mu k's is a parameter telling us the probability that the result of the dice roll was k. In other words mu k is equal to the probability that x is equal to k. So what does this mean about the one function in the exponent. This is what is called an indicator function.

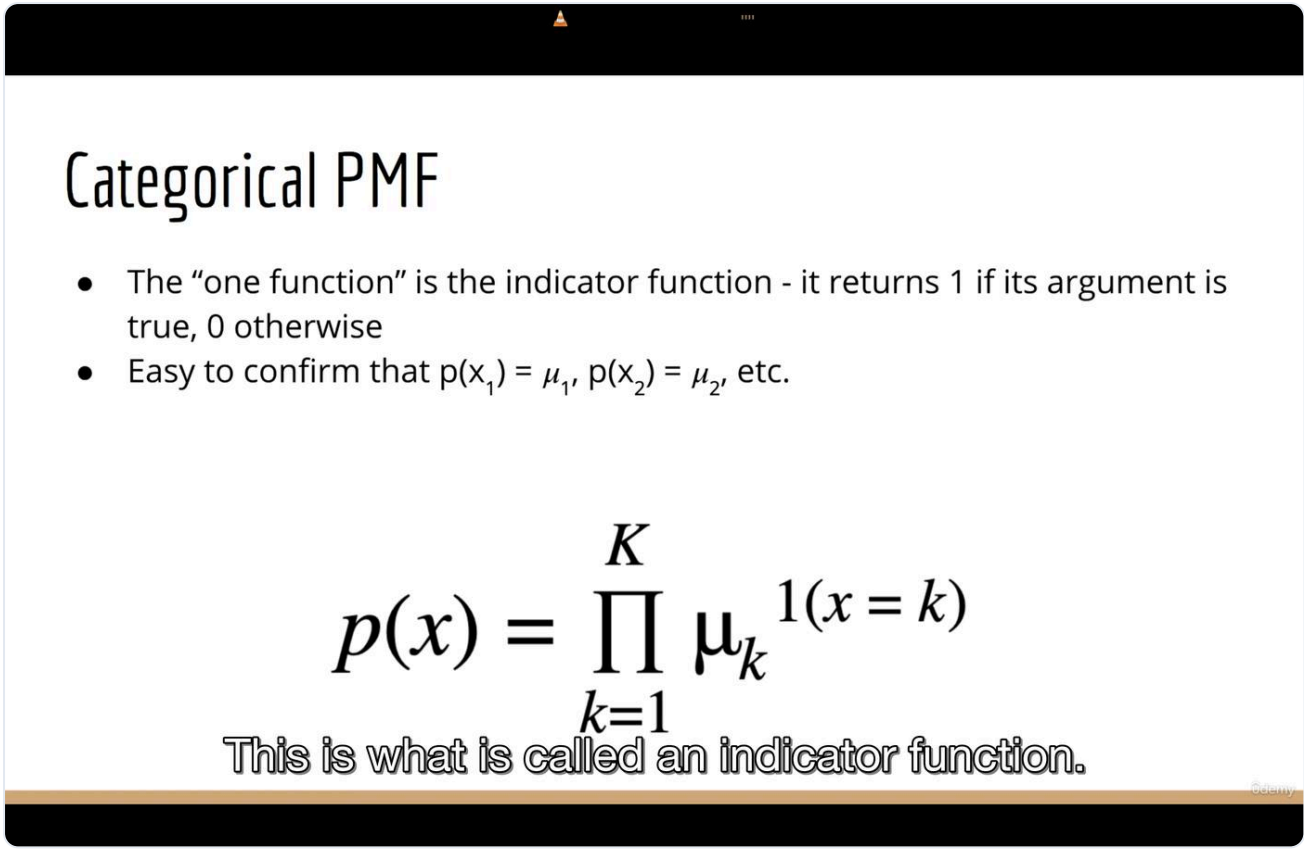

FIGURE 32.3

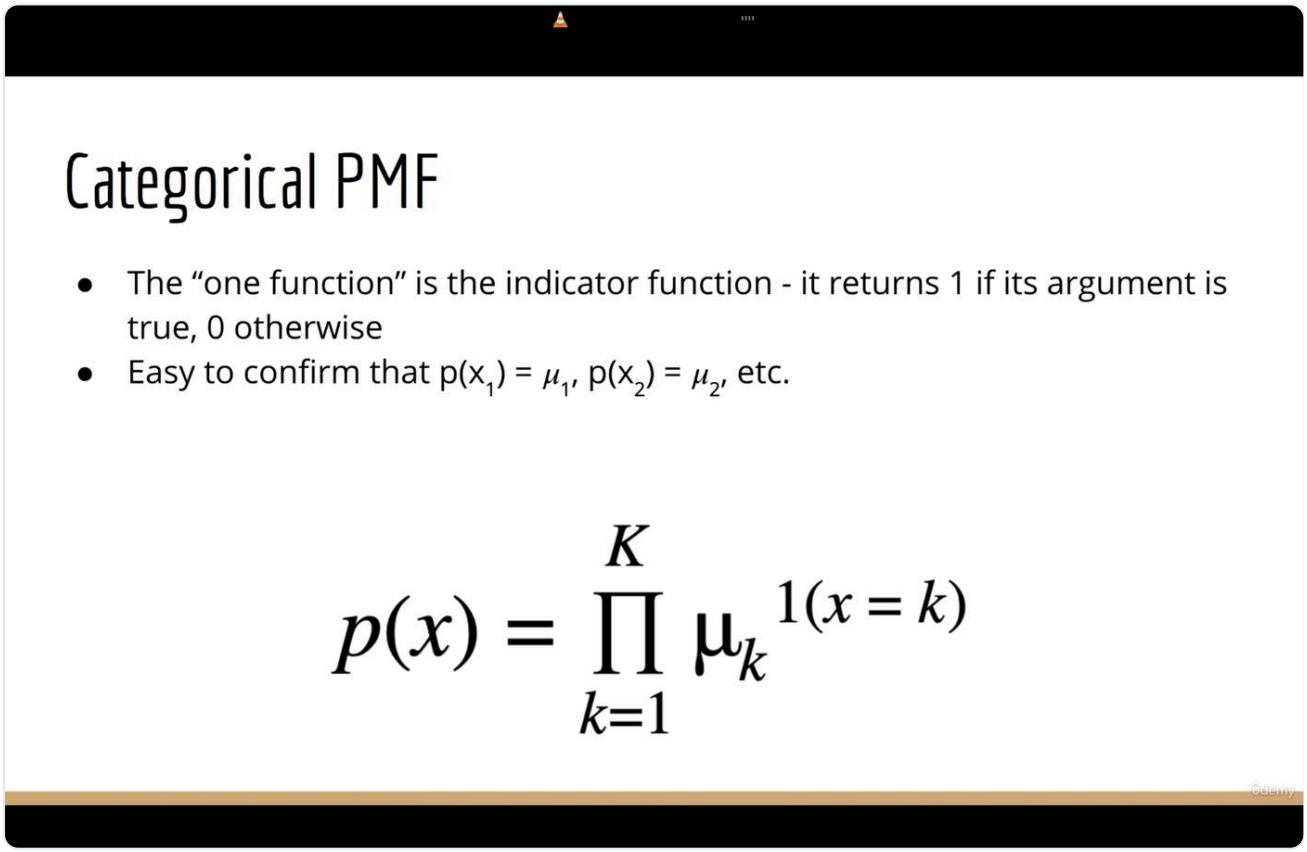

FIGURE 32.4

It returns one and when its argument is true and 0 otherwise Knowing this it's easy to confirm that if we plug in x equals 1 We should get that the probability that x is equal to 1 equals mu 1. The probability that x is equal to 2 is mu 2 and so forth. In other words we can confirm that this is indeed the PMF for the categorical distribution as usual we are going to pretend we've collected N data points and we are going to do maximum likelihood estimation to find all the mus. What is interesting about this likelihood is that because the likelihood is the product of PMFs and the PMF already has a product we end up with two products in the likelihood also notice that there are two indices in the exponent here.

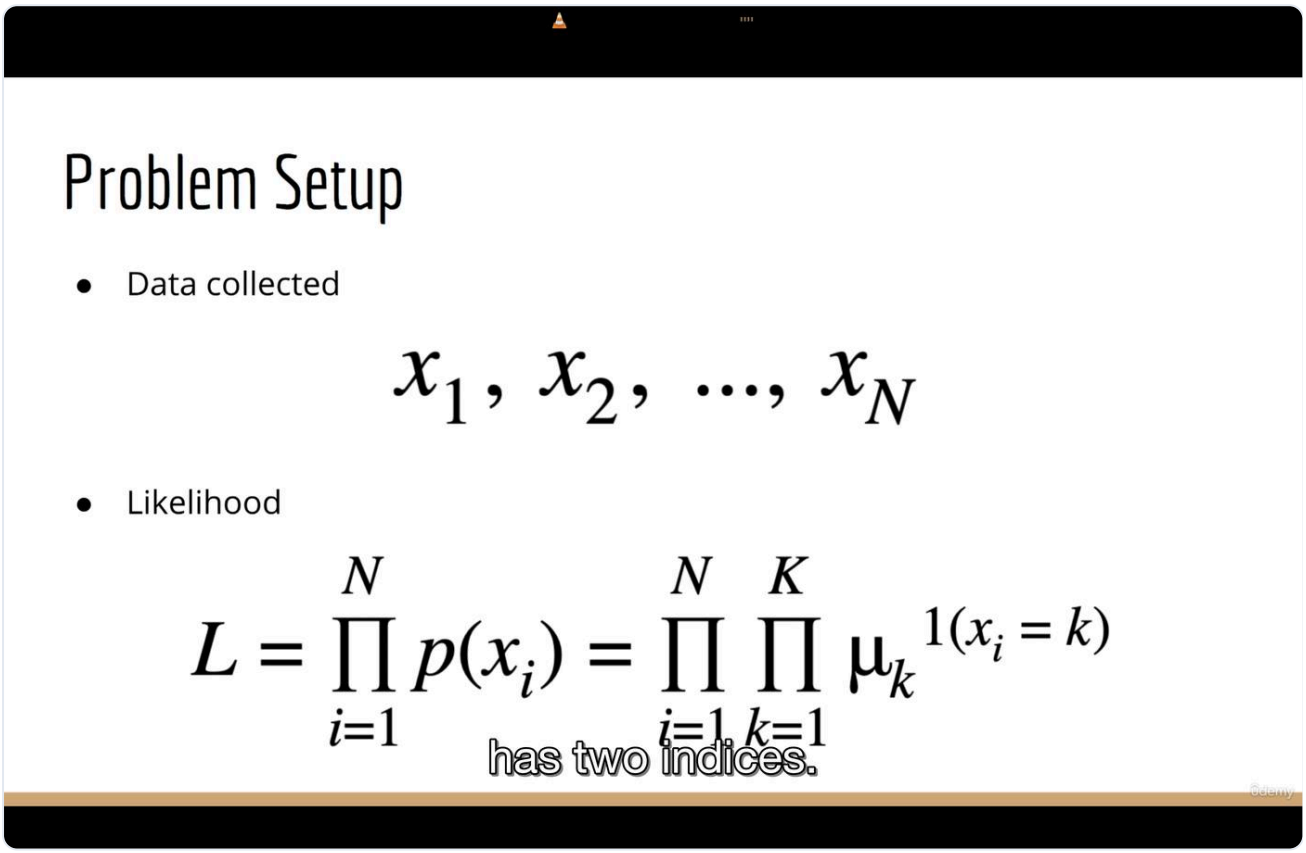

FIGURE 32.5

There is the index i and also the index K so that should remind you of a matrix because a matrix also has two indices. The row in the column after taking the log. We know that we get the log likelihood and we already know that the negative log likelihood is our categorical cross entropy.

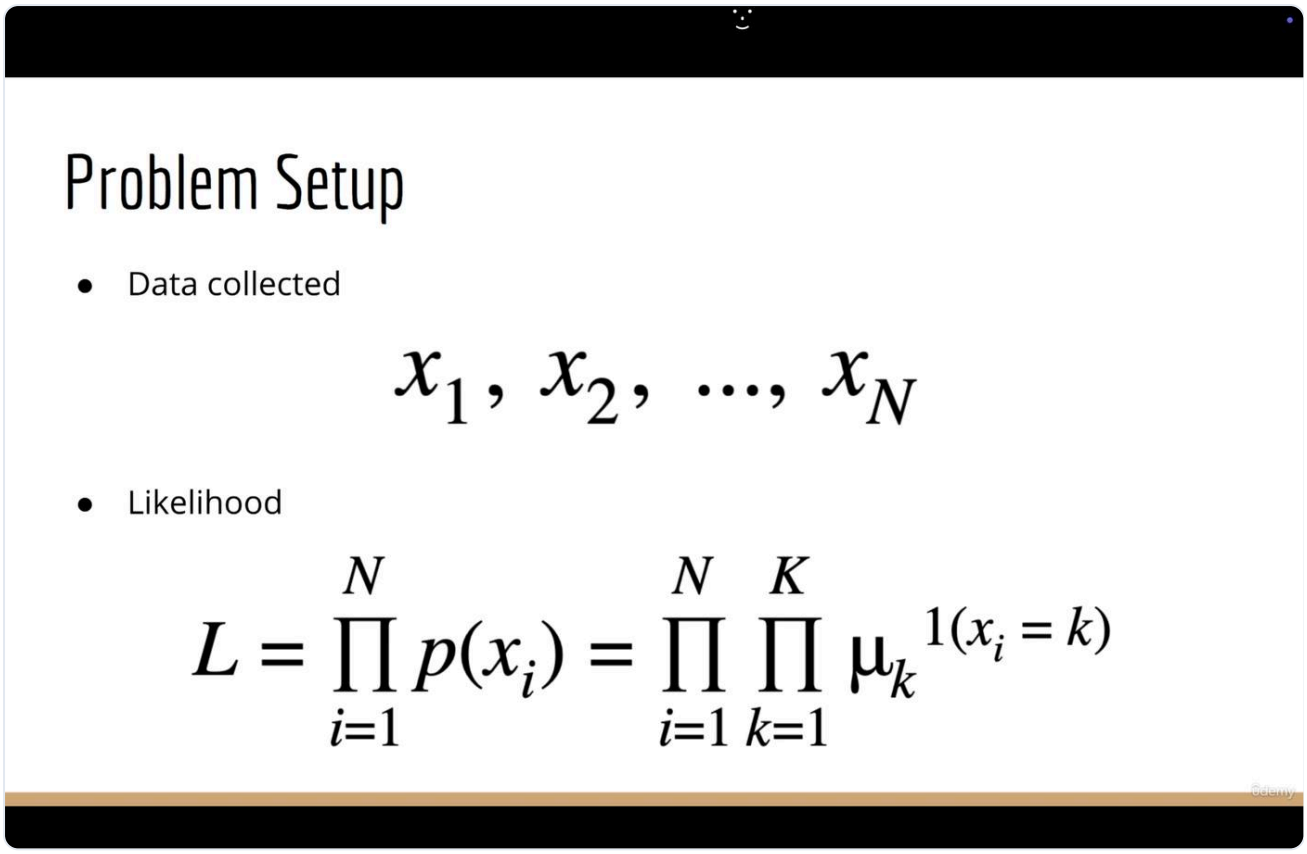

FIGURE 32.6

So if we take the negative of this expression we will get back something that looks exactly like the categorical cross entropy and thus we can relate this back to our categorical cross entropy loss function. Notice here that it's a tiny bit different from our log likelihood and that we don't have an indicator function anymore. This is because an equivalent way of writing this is double indexing. y i k is called a one hot encoded value because it can only take on the values 1 or 0 each row can only have one 1.

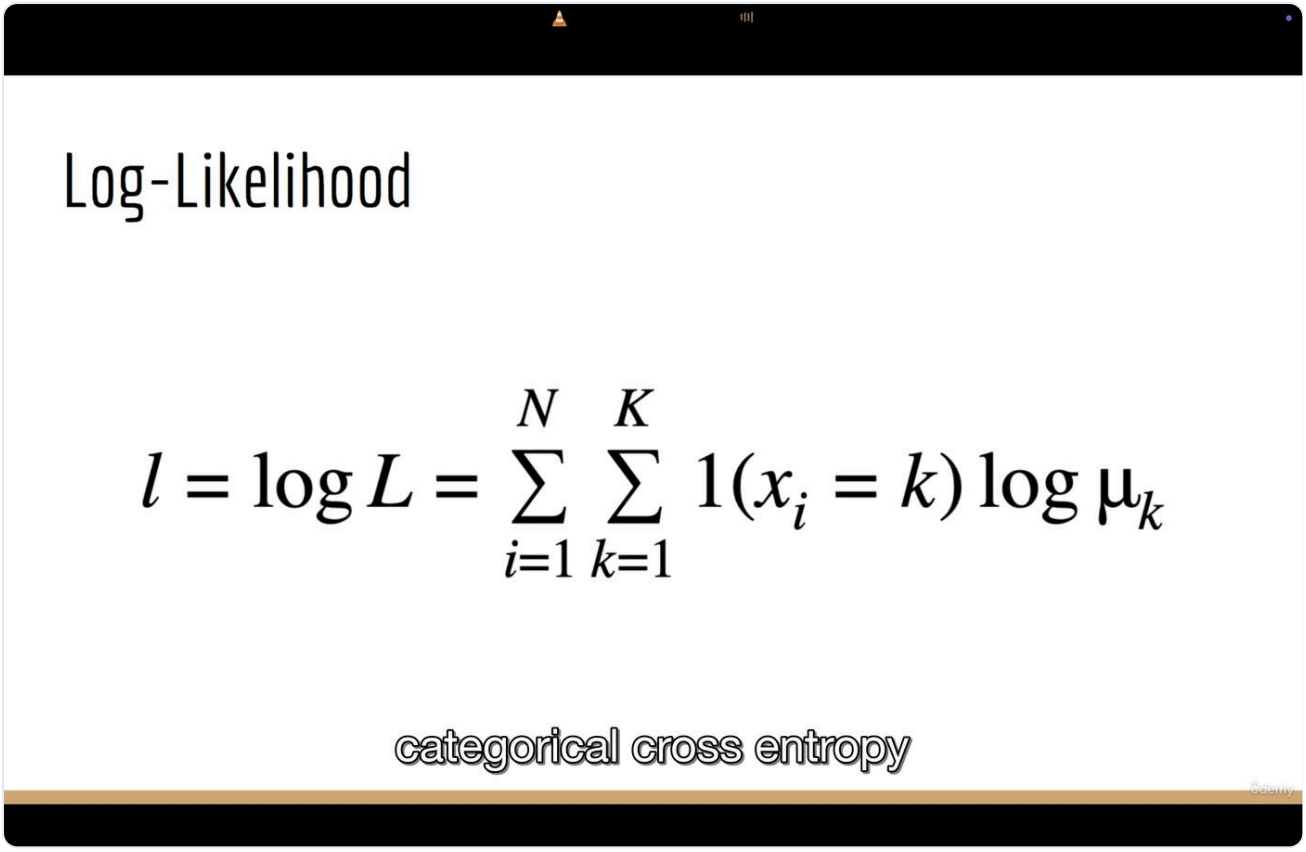

FIGURE 32.7

Remember that I represents the sample index and K represents the class label. The class label can only be one value. For example so for instance if I roll a die I can only get one number. It must be six or five or four three it can't be six and three at the same time so as another example if I have a picture and I'm classifying what type of animal it is it can only be one type of animal if it's a picture of a cat.

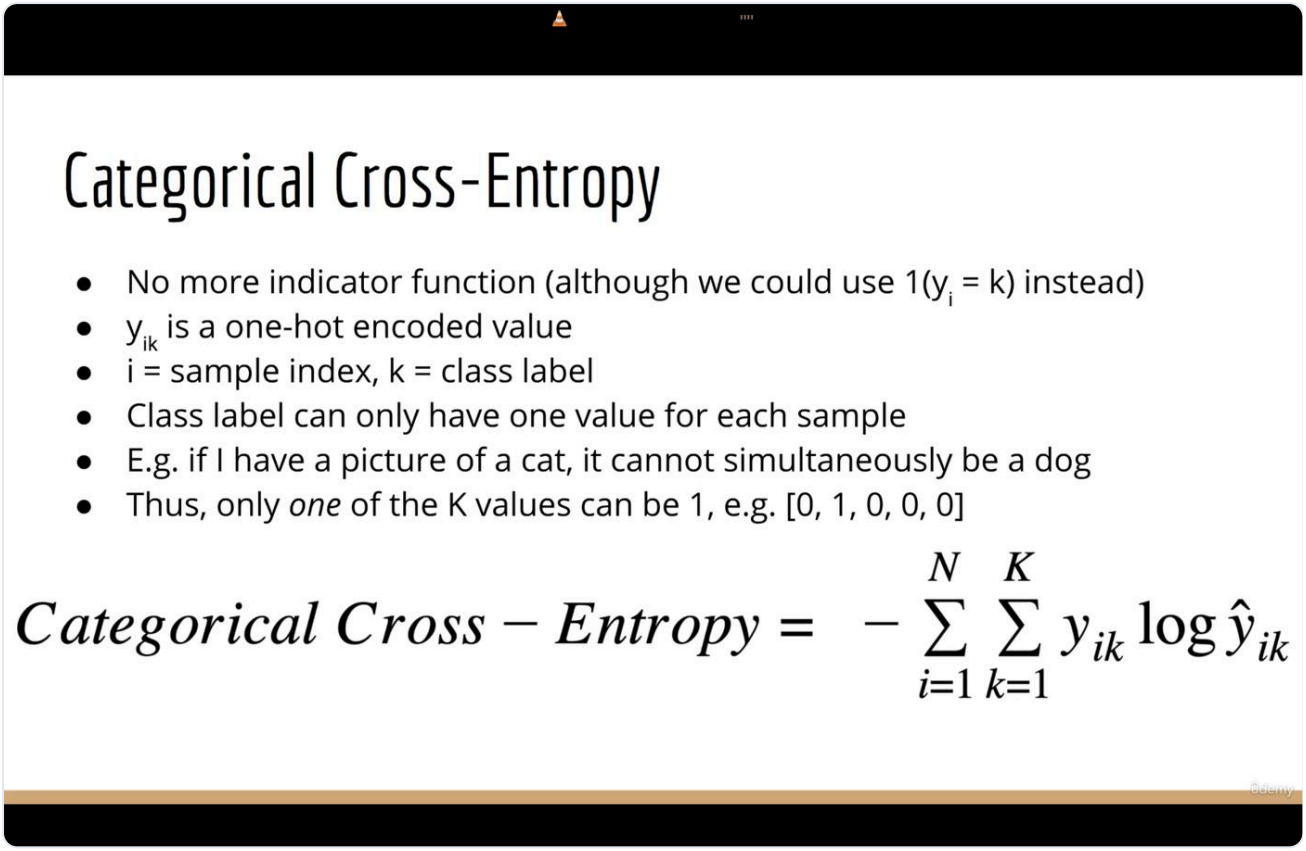

FIGURE 32.8

It cannot simultaneously be a picture of a dog therefore only one of the K values can have a one representing the target. The rest of the values must be 0 you'll notice that one hot encoding the targets like this can be quite inefficient. Let's do a simple example. Suppose we have four samples so N is four and three classes cat dog or bird. Suppose the targets are cat dog bird and cat. So numerically we can represent this with the vector one two three one. But in order to one hot encode this vector we must turn it into a four by three matrix or an N by K matrix containing only zeros and ones.

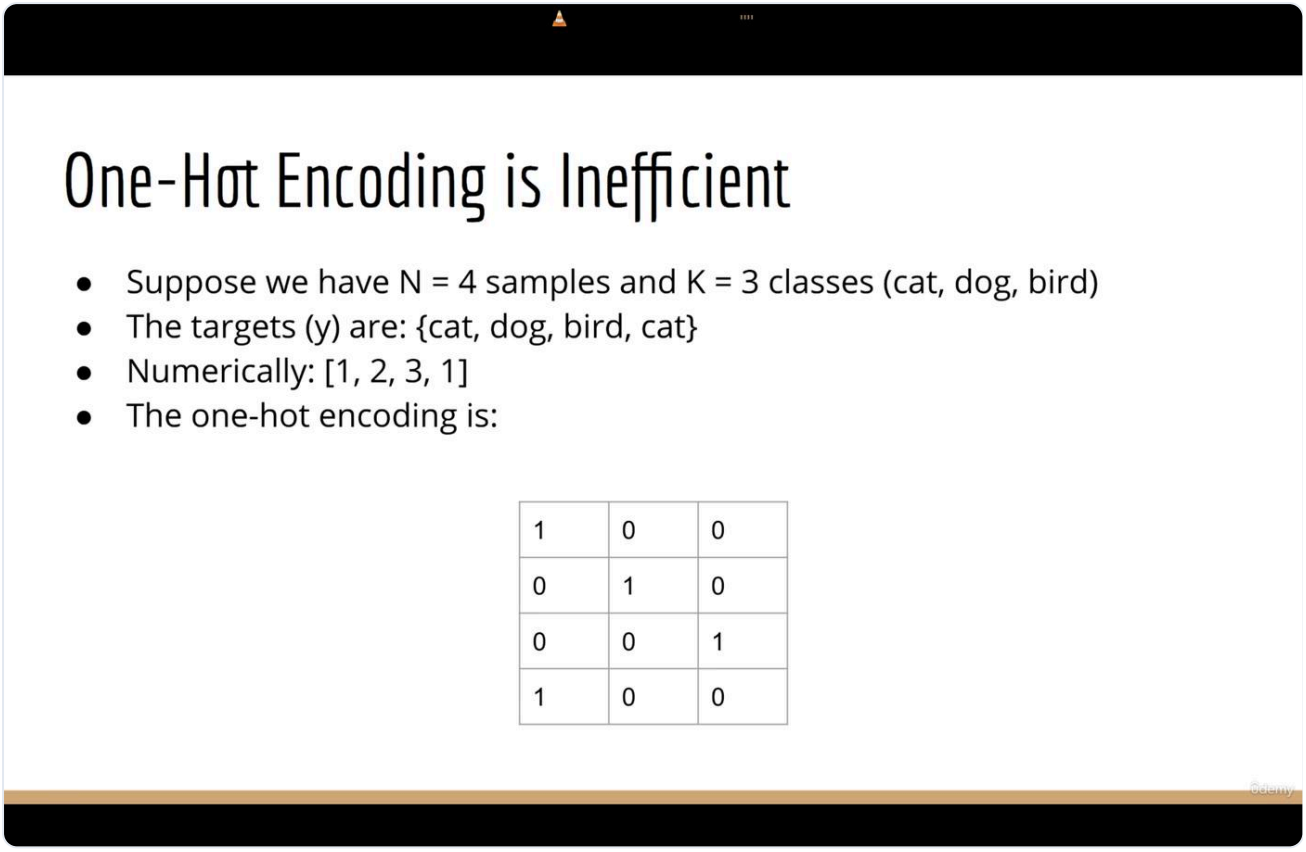

FIGURE 32.9 | One-Hot Encoding Is Inefficient

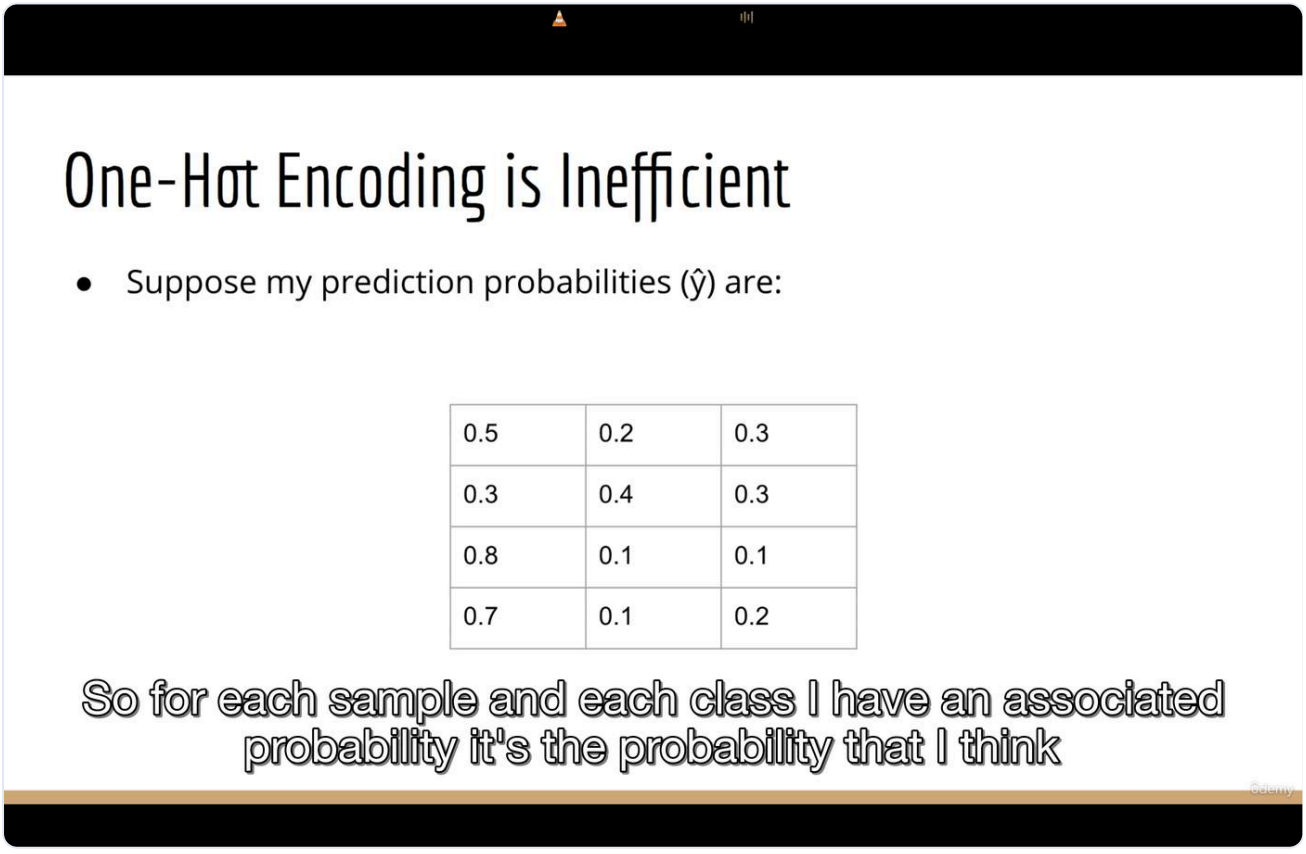

FIGURE 32.10 | One-Hot Encoding Is Inefficient

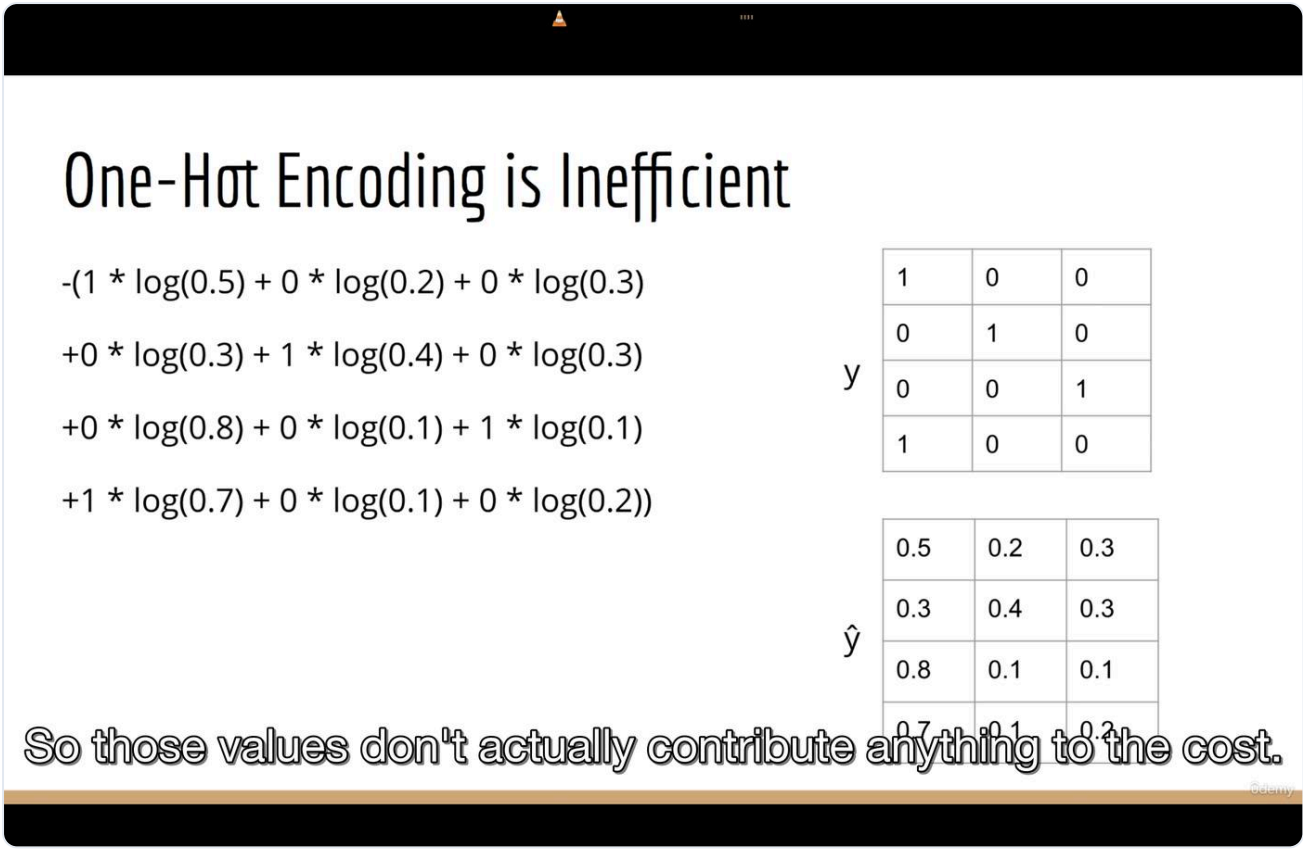

FIGURE 32.11 | One-Hot Encoding Is Inefficient

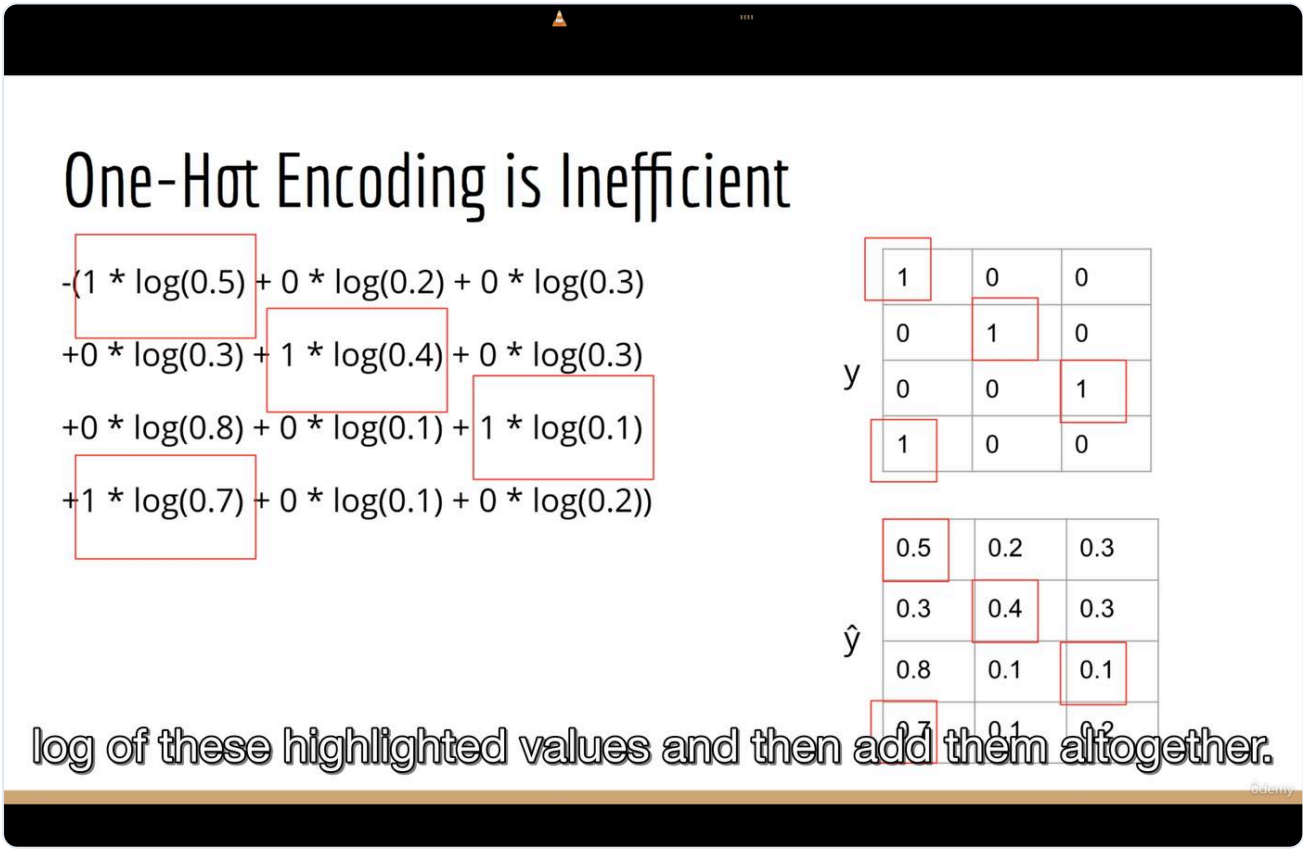

FIGURE 32.12 | One-Hot Encoding Is Inefficient

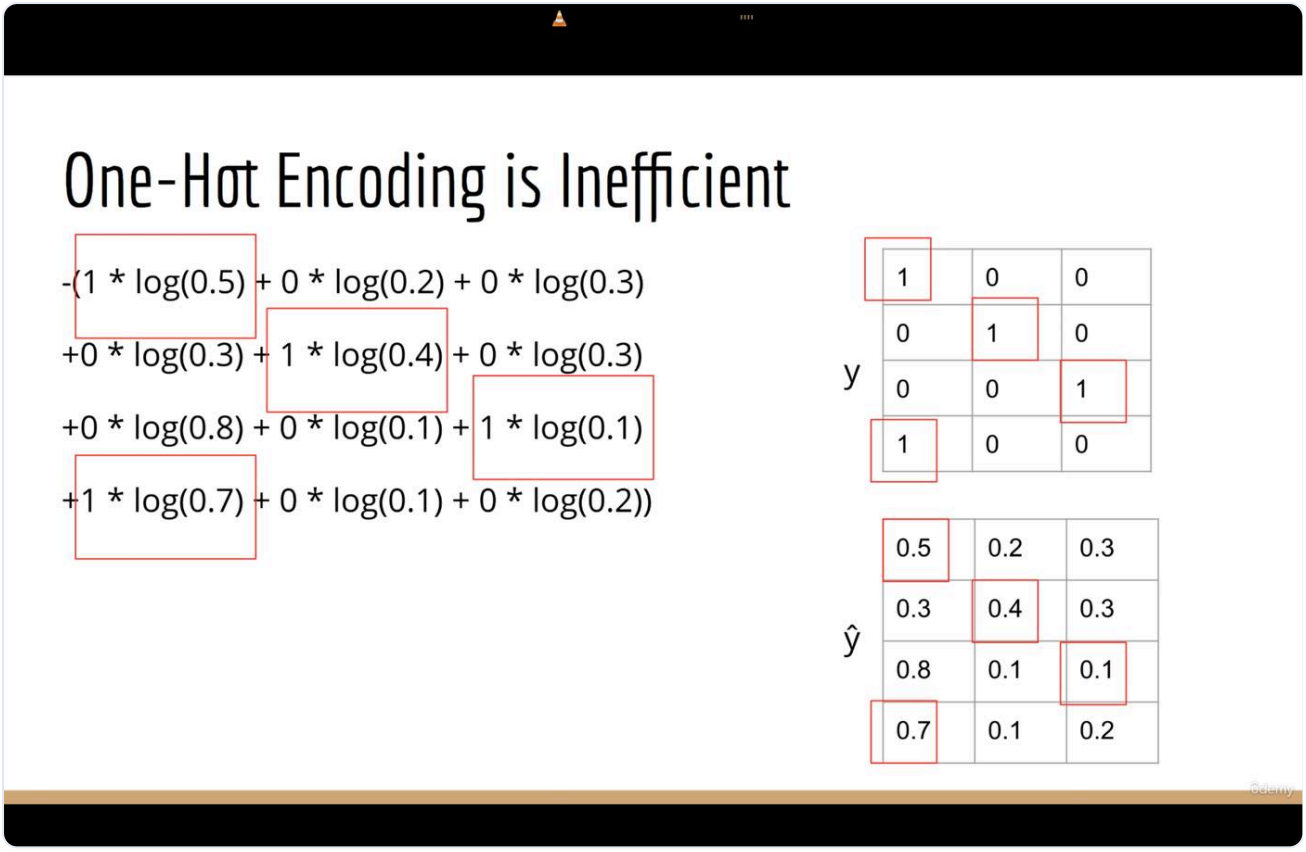

FIGURE 32.13 | One-Hot Encoding Is Inefficient

So the equivalent one hot encoding is 1 0 0 0 1 0 0 0 1 1 0 0. Now suppose my prediction probabilities also an n by K matrix are as follows. So for each sample and each class I have an associated probability it's the probability that I think the input belongs to that class after applying the categorical cross entropy formula. We get this value for the cost. Now you might be thinking that is a whole lot of multiplying by zero. We already know that anything multiplied by 0 0. So those values don't actually contribute anything to the cost. The only values that do matter are the ones which match the target. Remember the original target is the vector 1 2 3 1 What I would like to do ideally is to only take the log of these highlighted values and then add them altogether.

The question is can I do that without one hot encoding the target first. And indeed this is possible in NumPy. We can use the double indexing trick NumPy arrays allow you to index using things such as lists and other arrays instead of just single integers. So if I wanted to take my predictions and index only the relevant values here's what I would do. I would take y hat at i equals 1 and k equals 1 I would take y hat at i equals 2 and k equals 2. I would take y hat at i equals 3 and k equals 3 and I would take y hat at i equals 4 and k equals 1 using the NumPy double indexing method. I can actually index all of these values at once I simply pass in all the row indices and the corresponding column indices at the same time.

Importantly note that by doing this I never have to one hot encode the targets I can pass them directly into the square brackets as an exercise.

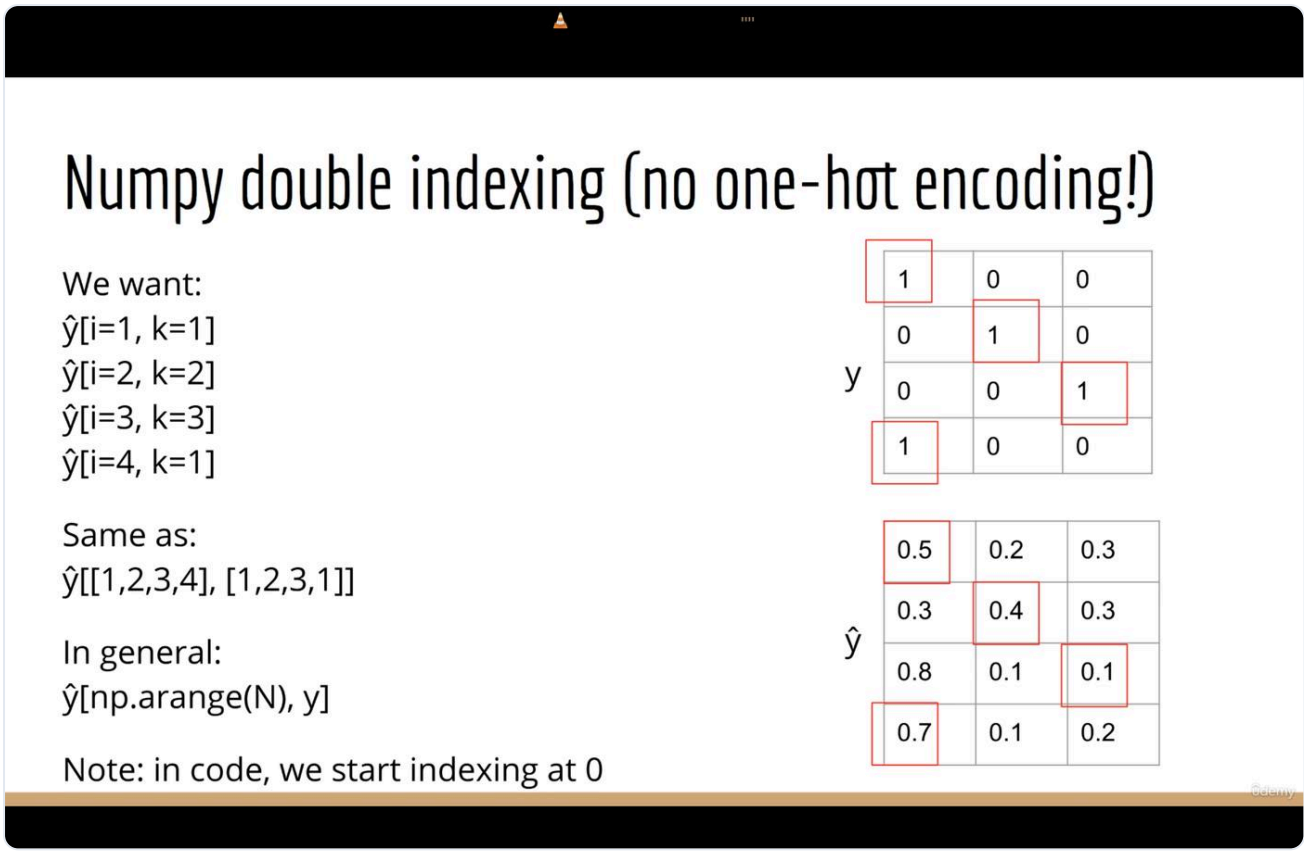

FIGURE 32.14

You should try this in actual NumPy code to verify that it really works just note that in code we index starting at 0 instead of 1. And so this is the idea behind the sparse categorical cross entropy.

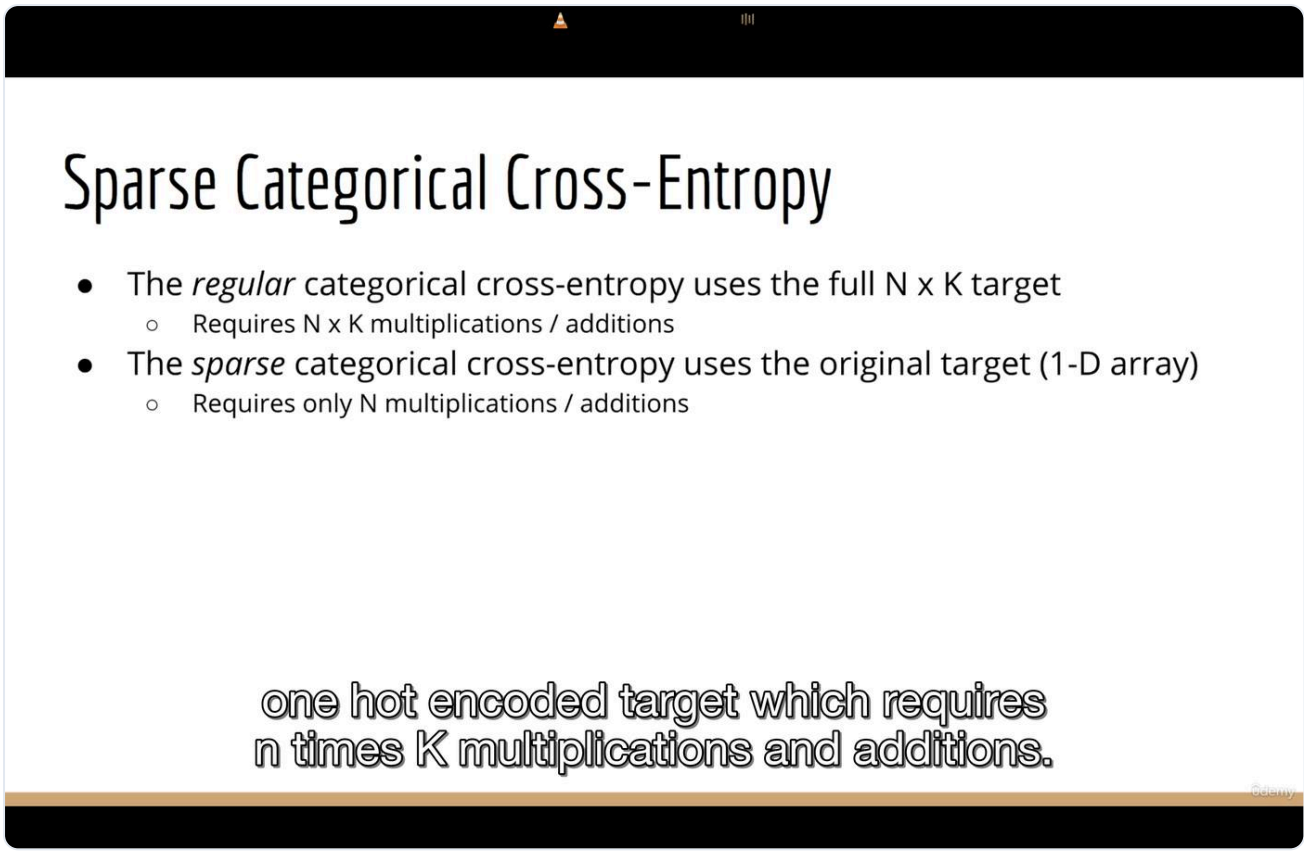

FIGURE 32.15

So when you are in TensorFlow 2.0 if you use the regular categorical cross entropy you are using the one hot encoded target which requires n times K multiplications and additions. If you use the sparse categorical cross entropy you were using the original target which requires only n multiplications and additions.

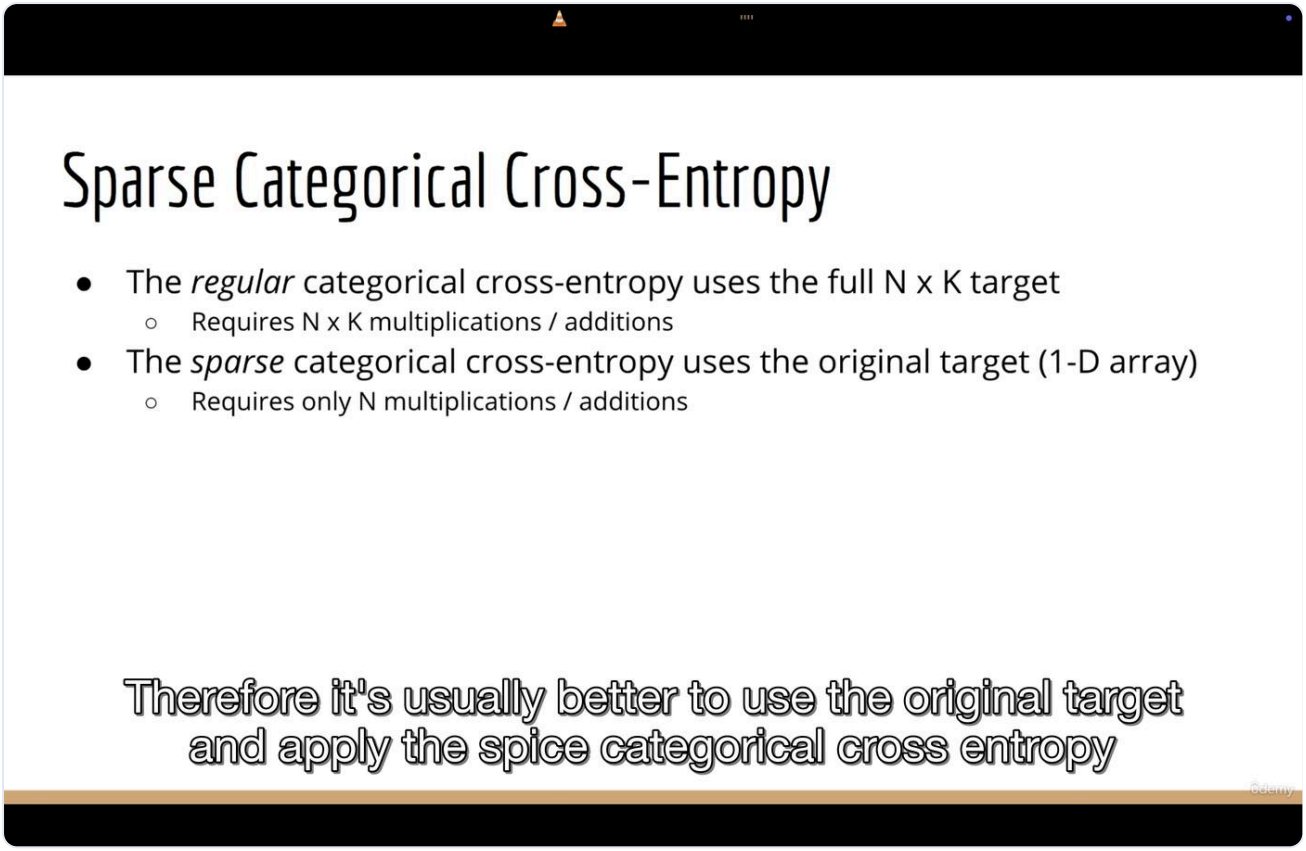

FIGURE 32.16

Therefore it's usually better to use the original target and apply the sparse categorical cross entropy loss.

LECTURE 126

Why Do We Need Gradient Descent?

In this lecture we are going to discuss gradient descent gradient descent is extremely important because it's the algorithm that we use to train all of our models from simple neurons to neural networks to convolutional neural networks and recurrent neural networks. This one basic idea is the backbone of deep learning and in fact it can be used for other machine learning models as well such as K Means clustering Hidden Markov models and Matrix Factorization.

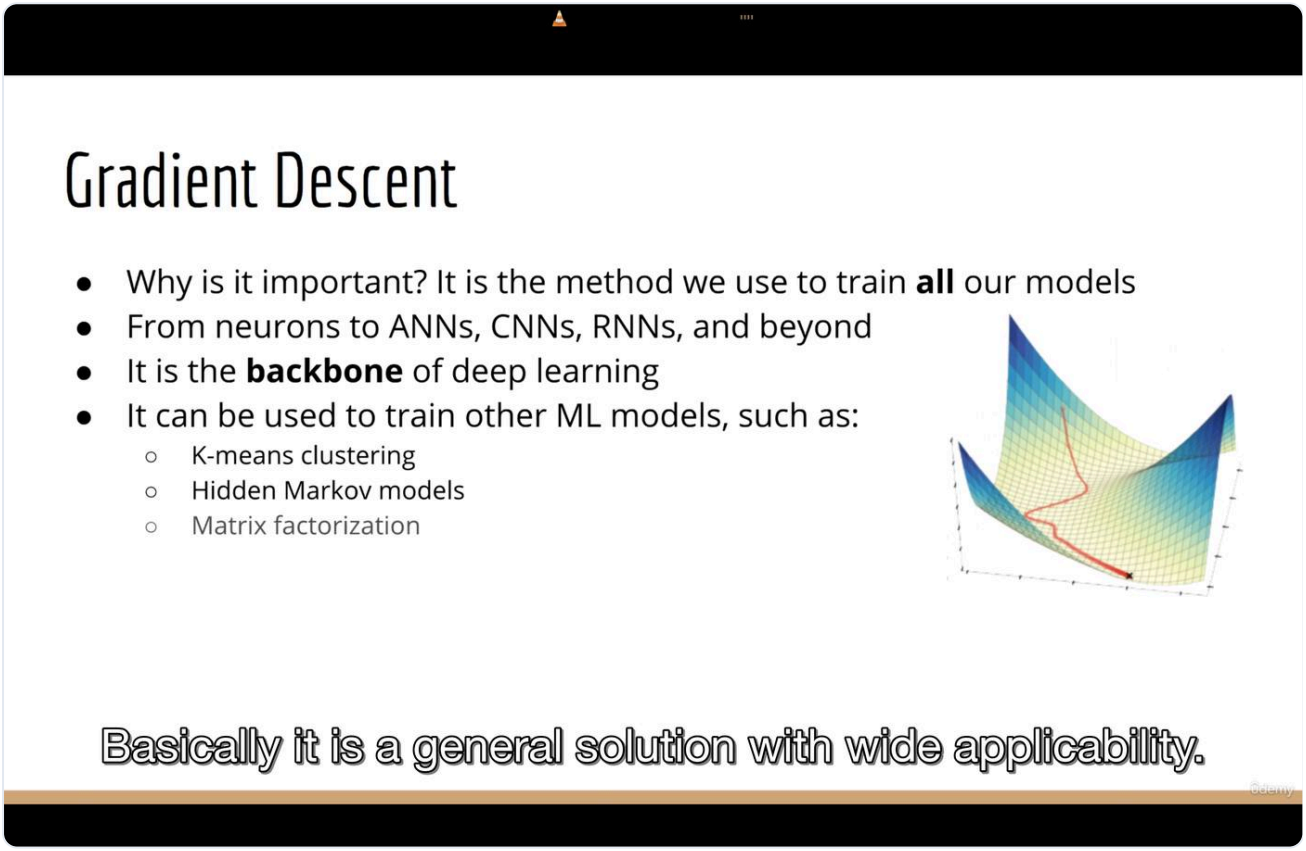

FIGURE 33.1 | Gradient Descent

Basically it is a general solution with wide applicability.

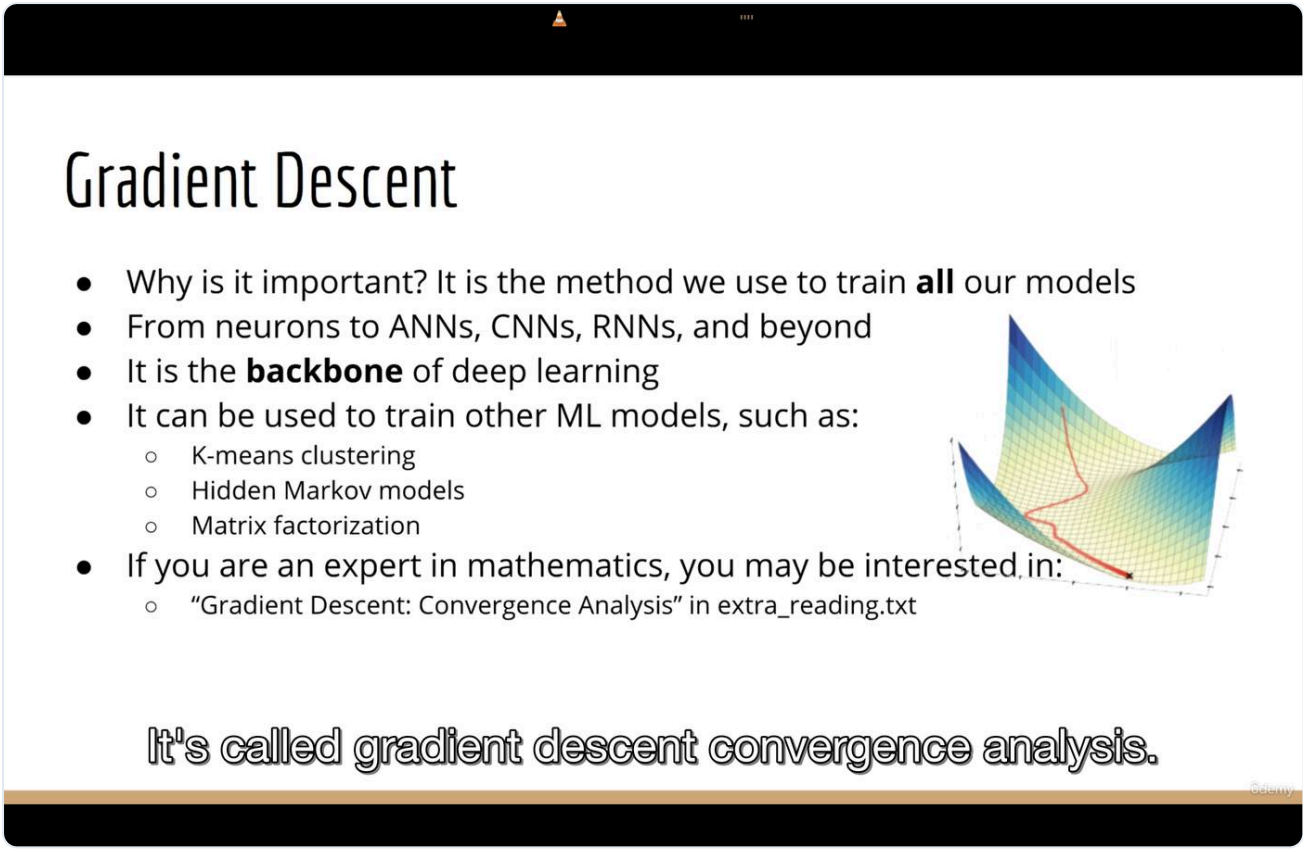

FIGURE 33.2 | It Is the Backbone of Deep Learning

This lecture will discuss the basic intuitions about gradient descent but if you are an expert in mathematics you might want to check out a rigorous proof that it works in a document I've attached in extra reading.txt. It's called gradient descent convergence analysis. First let's ask ourselves why do we need a gradient descent in the first place. What is the problem we are trying to solve.

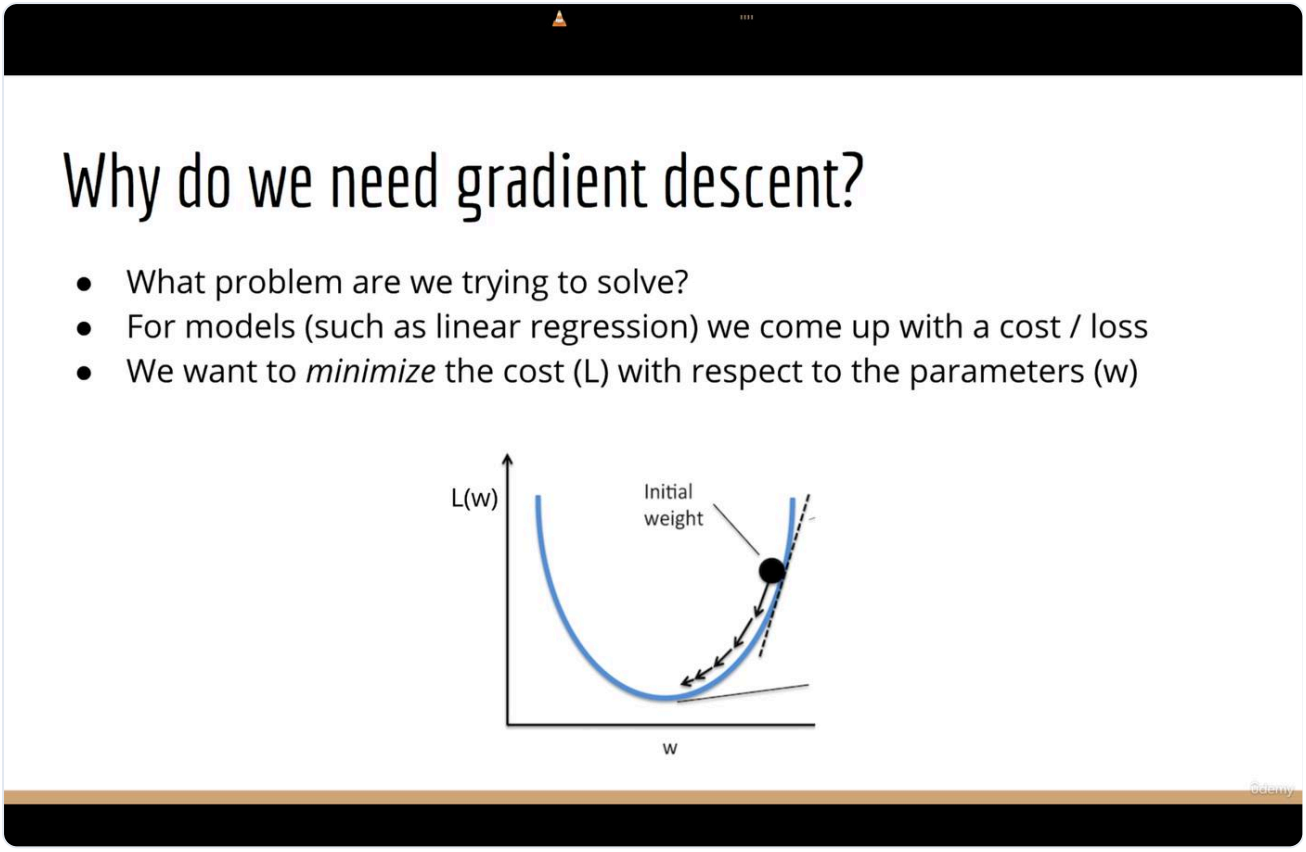

FIGURE 33.3 | Why Do We Need Gradient Descent?

Recall that for problems such as linear regression we are going to come up with an error function also known as a cost function or loss function. Our goal is to minimize this error with respect to the model parameters call that W.. So how do we minimize L with respect to W if you think back to your high school calculus studies you'll realize that you've already encountered this. Whenever we have a function and we want to minimize it or maximize it with respect to its arguments what do we do. We take its derivative and set it to zero. Then we solve for the parameter in question. So for example if you wanted to minimize 5 X squared plus three x plus one with respect to x you would find the derivative with respect to x set it to zero and solve for x.

The reason why this works is because the derivative or the slope at the minimum and maximum of a function is zero. In other words I can draw a line at this point. Tangent to the curve and it should be a horizontal line.

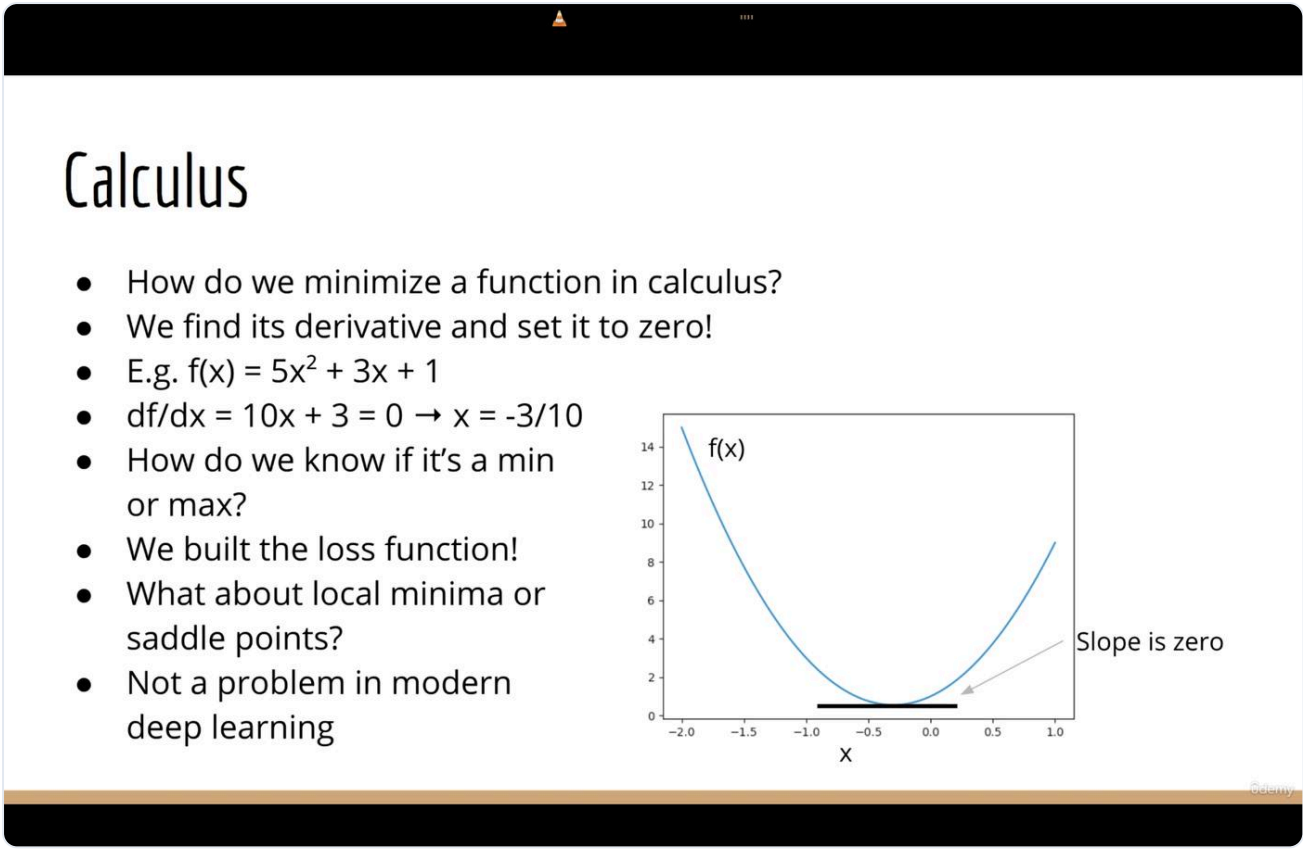

FIGURE 33.4 | How Do We Minimize a Function in Calculus?

Now you might ask how do we know if it's a maximum or minimum. There are ways to determine that but in machine learning we generally do not do that analysis. We pretty much already know that the loss functions we are working with have minimums and not maximums. Sometimes we may worry about saddle points or local minima but in general this is still not a problem for the latest deep learning methods the reason why is outside the scope of this course. But you are welcome to ask me about it on the Q and A Okay. This still doesn't answer the question why do we need a gradient descent. Well here's the problem sometimes we can take the derivative and try to set it to 0 and solve for the parameter.

But sometimes this equation is not solvable by the way I've maybe once or twice had someone asked me why some equations are not solvable while my answer is.

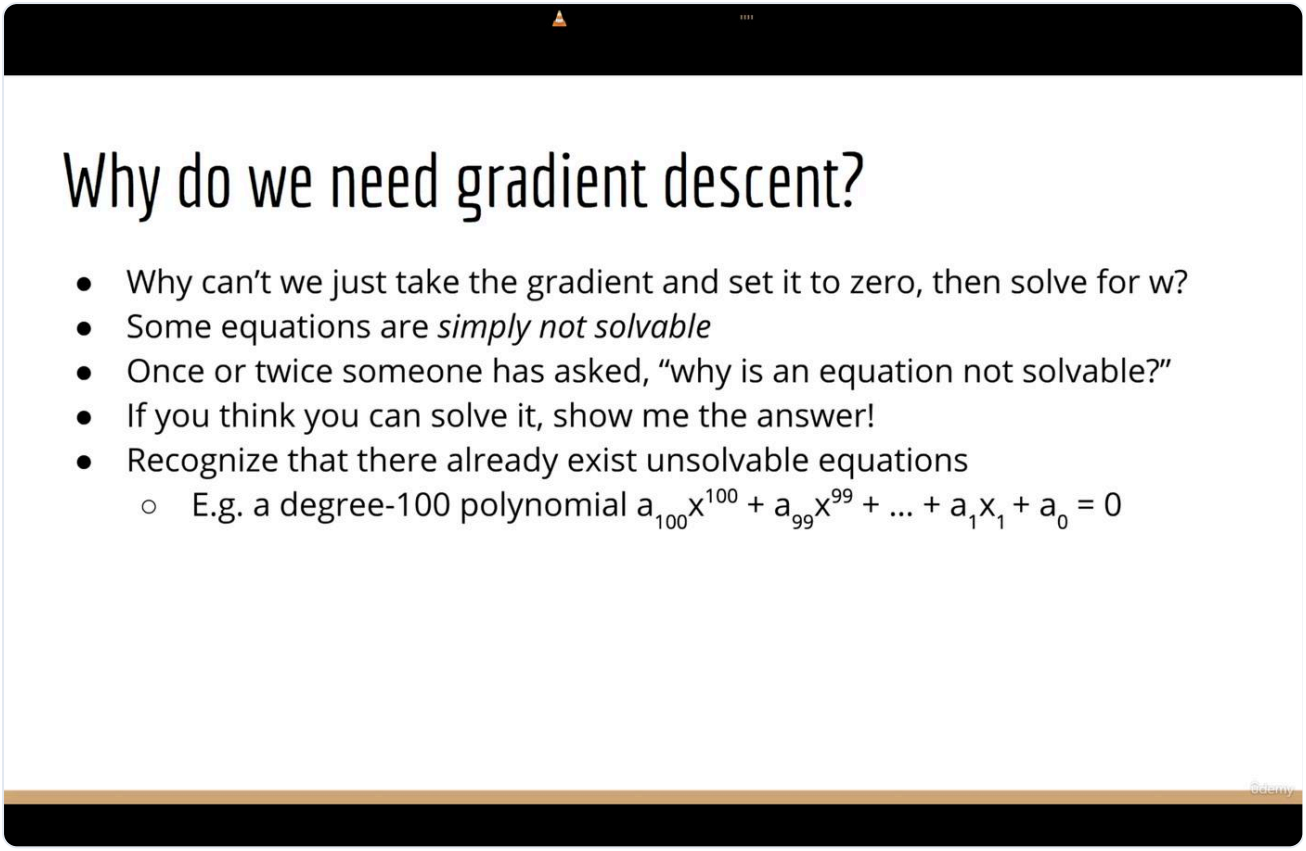

FIGURE 33.5 | Why Do We Need Gradient Descent?

If you think you can solve it then do it and show me the answer you can use your intuition about other equations we do not know how to solve. For example we can't solve. Let's say the roots of a degree 100 polynomial it just simply is impossible OK. So what is the general strategy we use in mathematics when we cannot solve an equation. The answer is we use numerical approximations. One such example is finding integrals. Sometimes you can't solve for an integral by hand. I think anyone who is taking calculus knows that solving some integrals can be quite challenging and for others literally impossible.

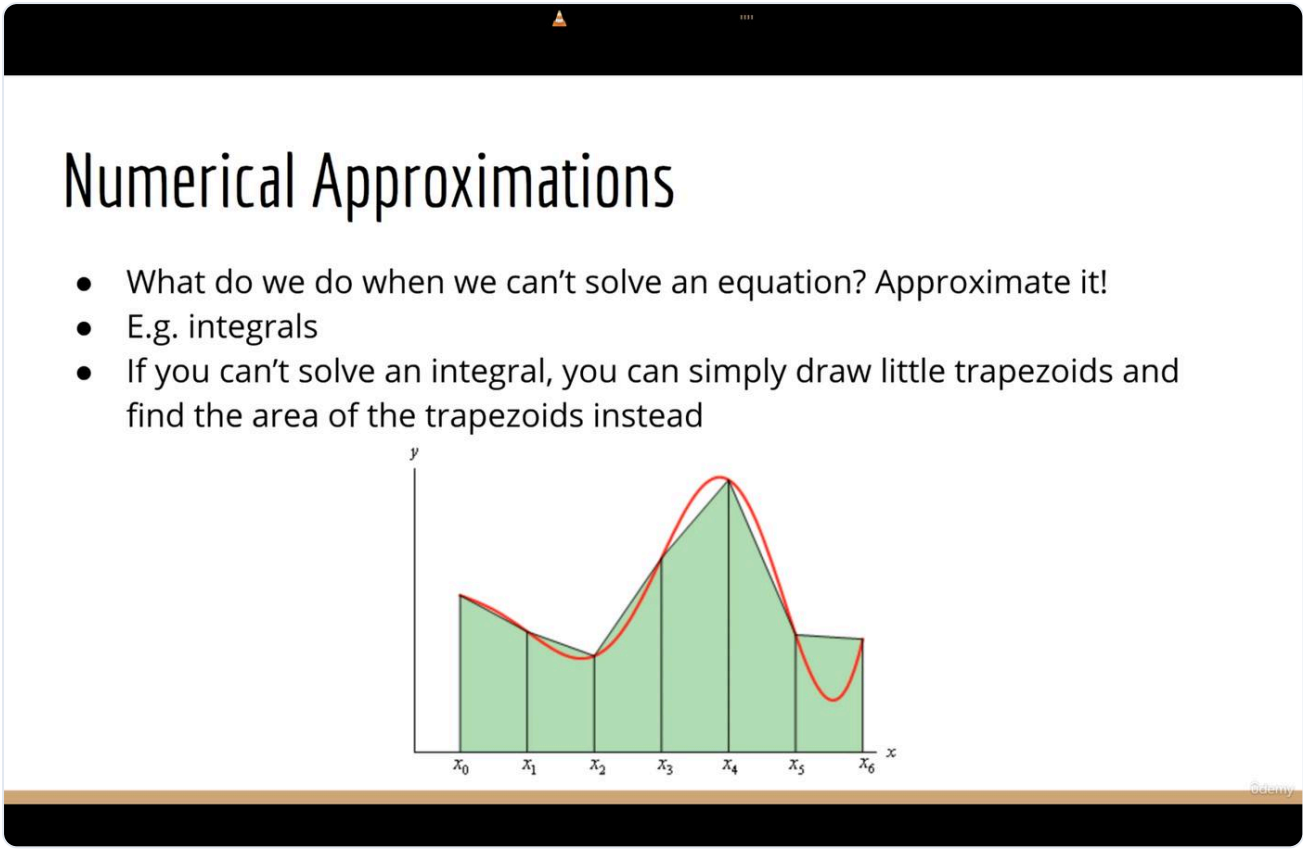

FIGURE 33.6 | Numerical Approximations

So what do we do. Well instead of trying to solve the equation by hand we just draw a little trapezoid underneath the curve and we find the area of those trapezoid. That will give us an approximation for the area under the curve. OK so that's the analogy. When we can't solve something analytically or by hand we resort to a numerical approximation methods. So what's the idea behind gradient descent. The idea is if I take the gradient of the function with respect to the parameter W and I take small steps in that direction the value of the function will decrease.

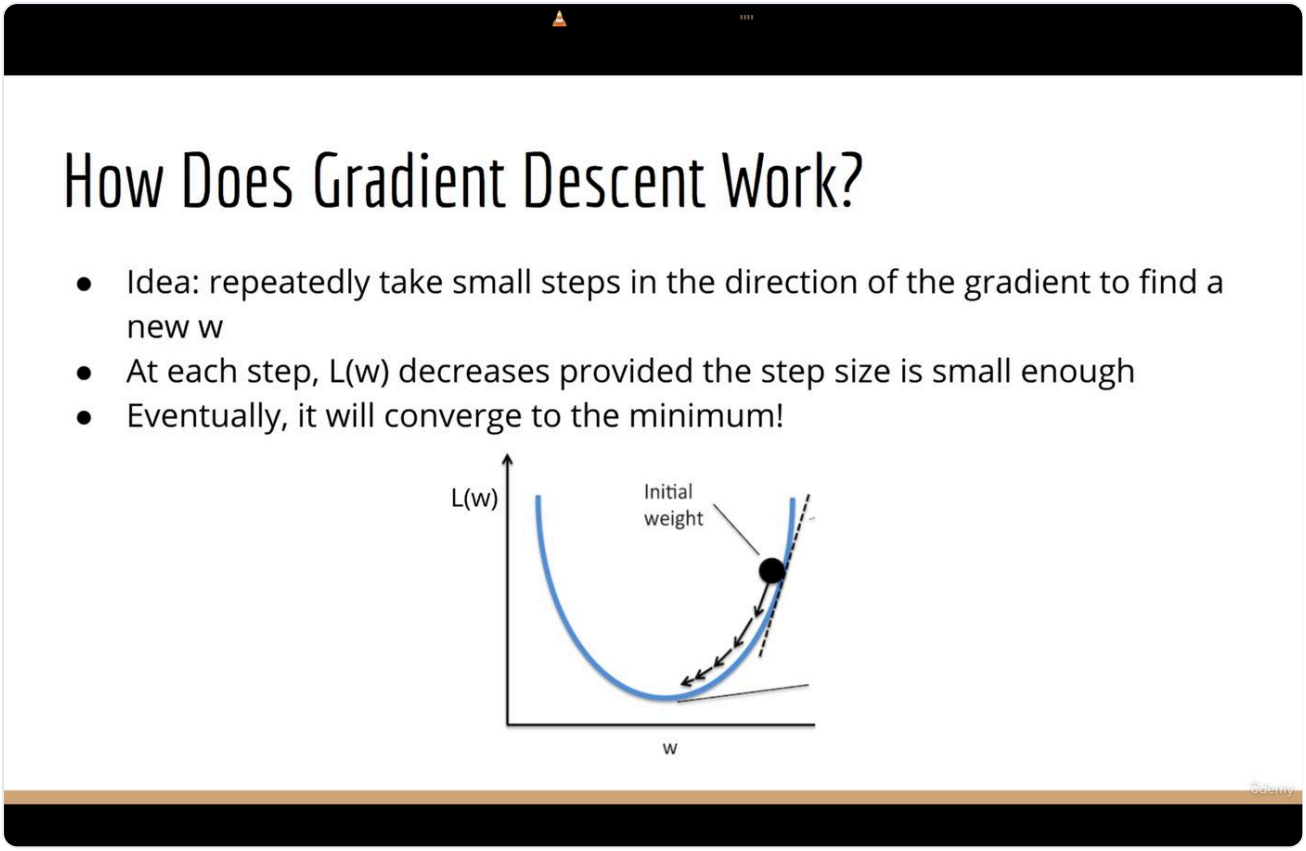

FIGURE 33.7 | How Does Gradient Descent Work?

Provided that my step size is small enough eventually as I keep taking these very small steps I'm going to get closer and closer to the bottom of the curve. And eventually I will converge to an answer by converge I mean that when you get near the bottom your steps will become so small that they will become insignificant. So w doesn't change as you take more steps at that point. You have found the value of W that minimizes your function.

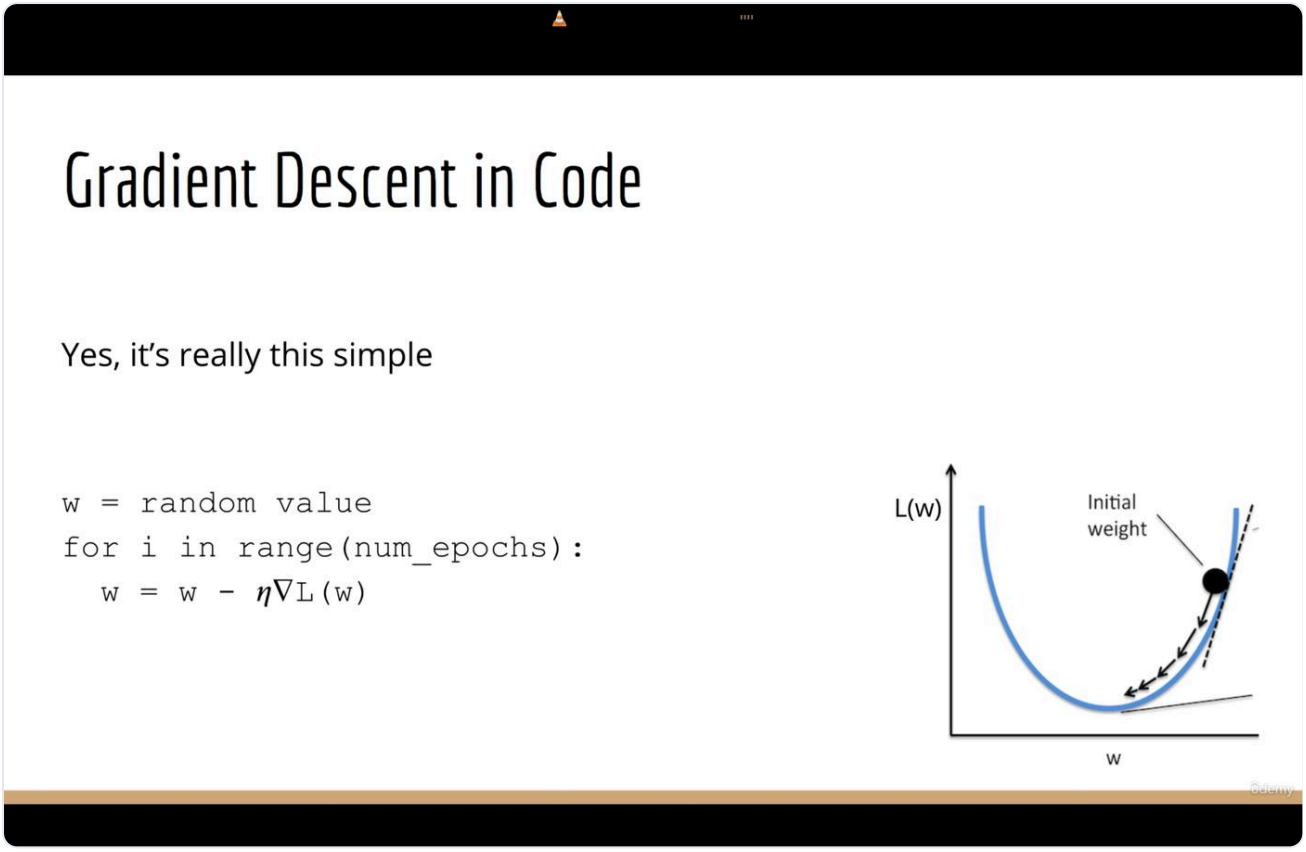

FIGURE 33.8 | Gradient Descent in Code

So what does this look like in code. Basically it's just a for loop before the loop we initialize W to some random value then inside the loop we continually update w using the gradient descent formula so here's an example of gradient descent at work. Let's say we want to minimize w squared. So that's our loss function. The derivative is 2 W.

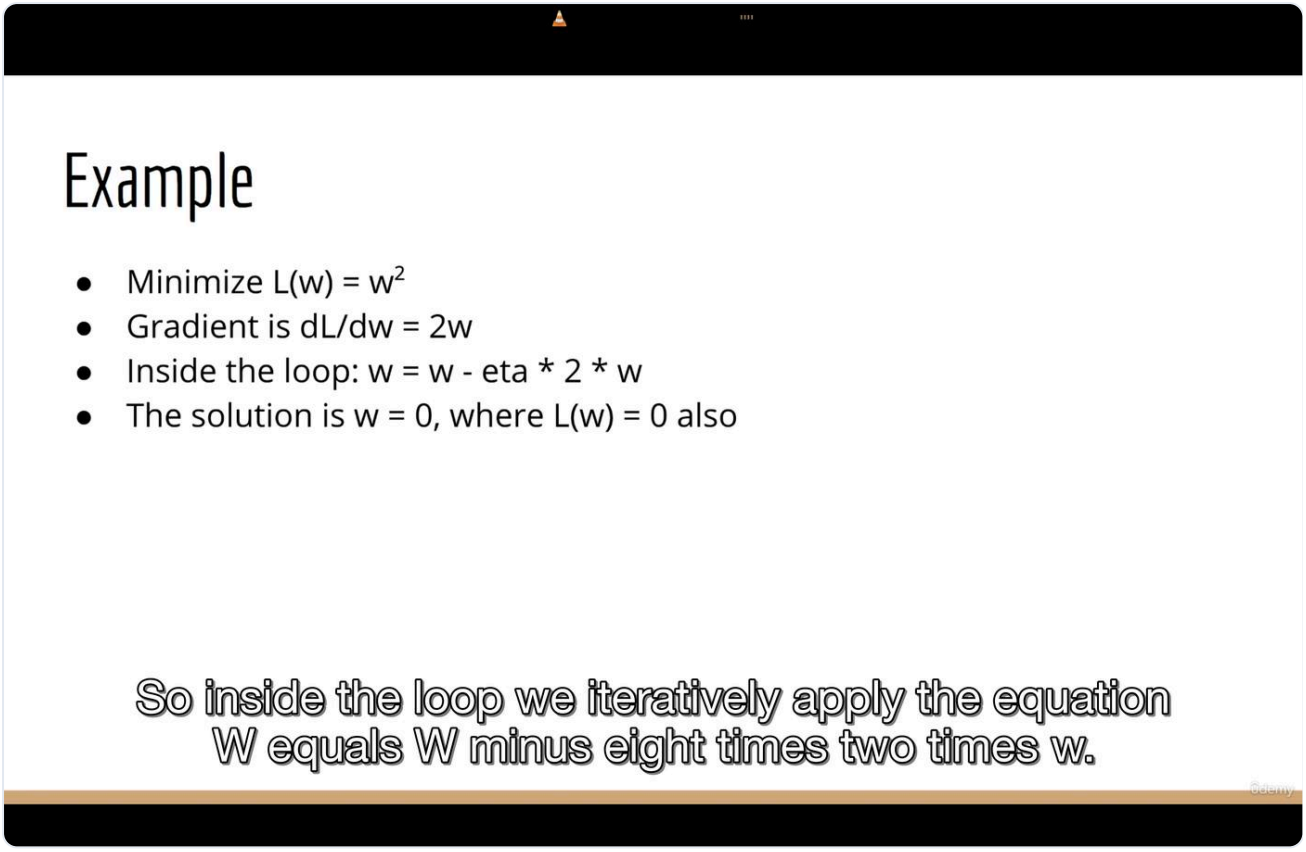

FIGURE 33.9

So inside the loop we iteratively apply the equation W equals W minus eta times two times w.

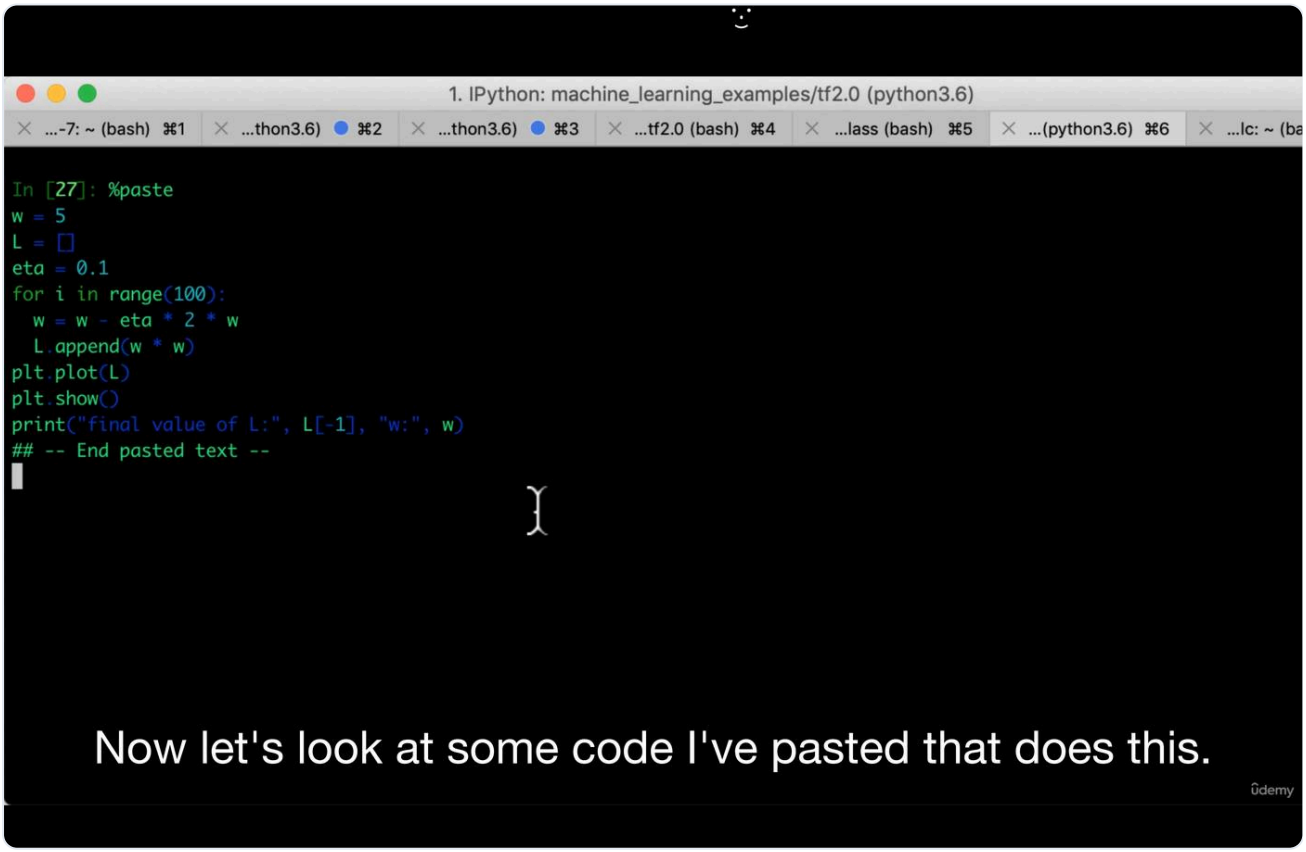

FIGURE 33.10

Now let's look at some code I've pasted that does this. As you can see I've initialized W to 5. I created an empty list called L where I'm going to store the loss at each iteration so that I can plot it later. Now in a loop I run my gradient descent update one hundred times and it each time I append w square to l finally I plot out we can see that eventually the value of the function goes down to zero. It's a minimum value. The value of W that we find at the end is also zero because this is the value of W that makes the original function zero. Now you will notice that there is some hyperparameters that can't quite be ignored in particular how many iterations should we run the loop for.

This is known as the number of epochs of course we must have a number of epochs high enough so that the cost function converges but it must not be too high. Otherwise we'll waste a lot of time where w simply doesn't change how small should our step size be.

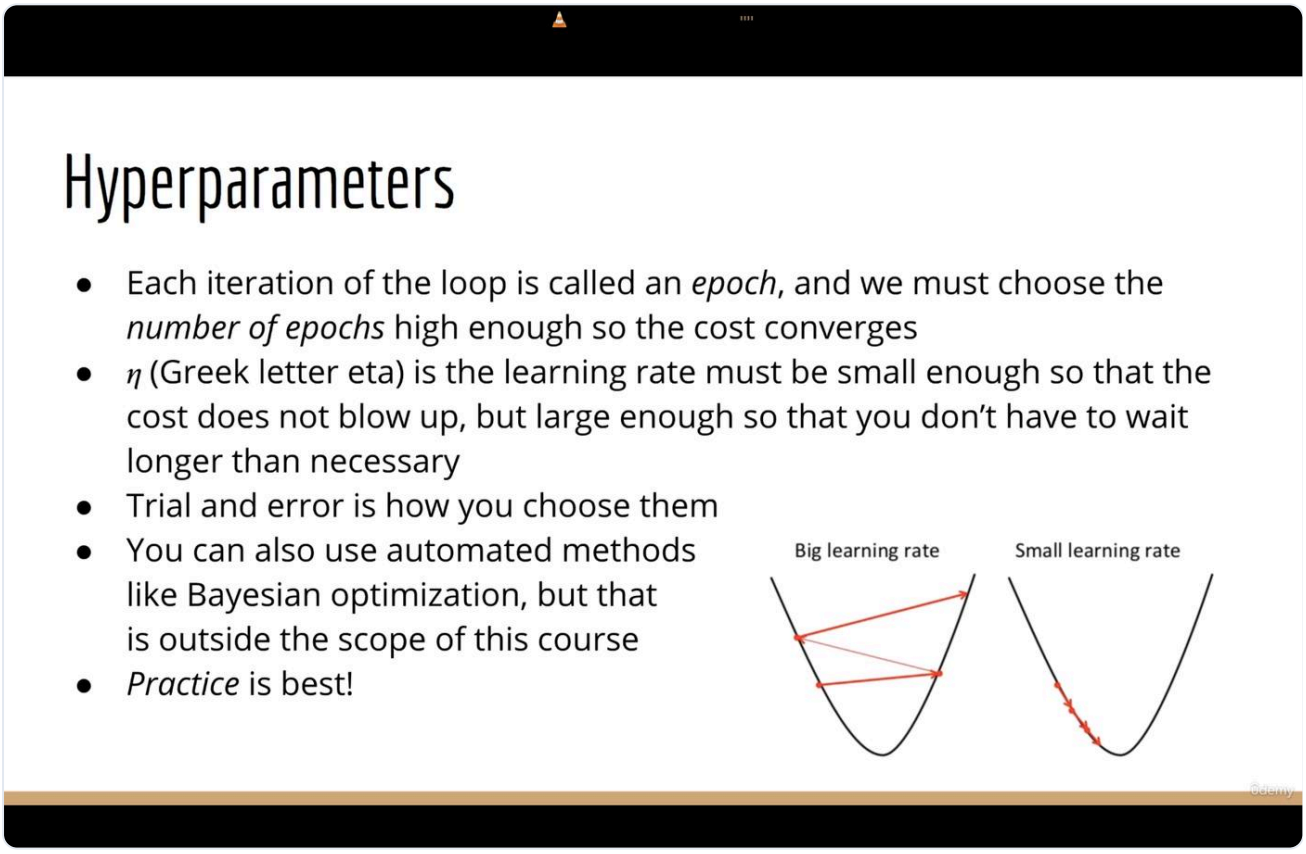

FIGURE 33.11

I've used the Greek symbol eta for this and we call it the learning rate the learning rate must be small enough so that the cost function doesn't explode but it also must be large enough so that the learning process doesn't take unnecessarily long. Both of these hyperparameters are values that we must choose using trial and error. There are a few more advanced methods such as Bayesian optimization that you could use also but they are outside the scope of this course. Normally for students of machine learning doing experiments manually and gaining intuition through practice is the best approach.

LECTURE 127

Stochastic Gradient Descent

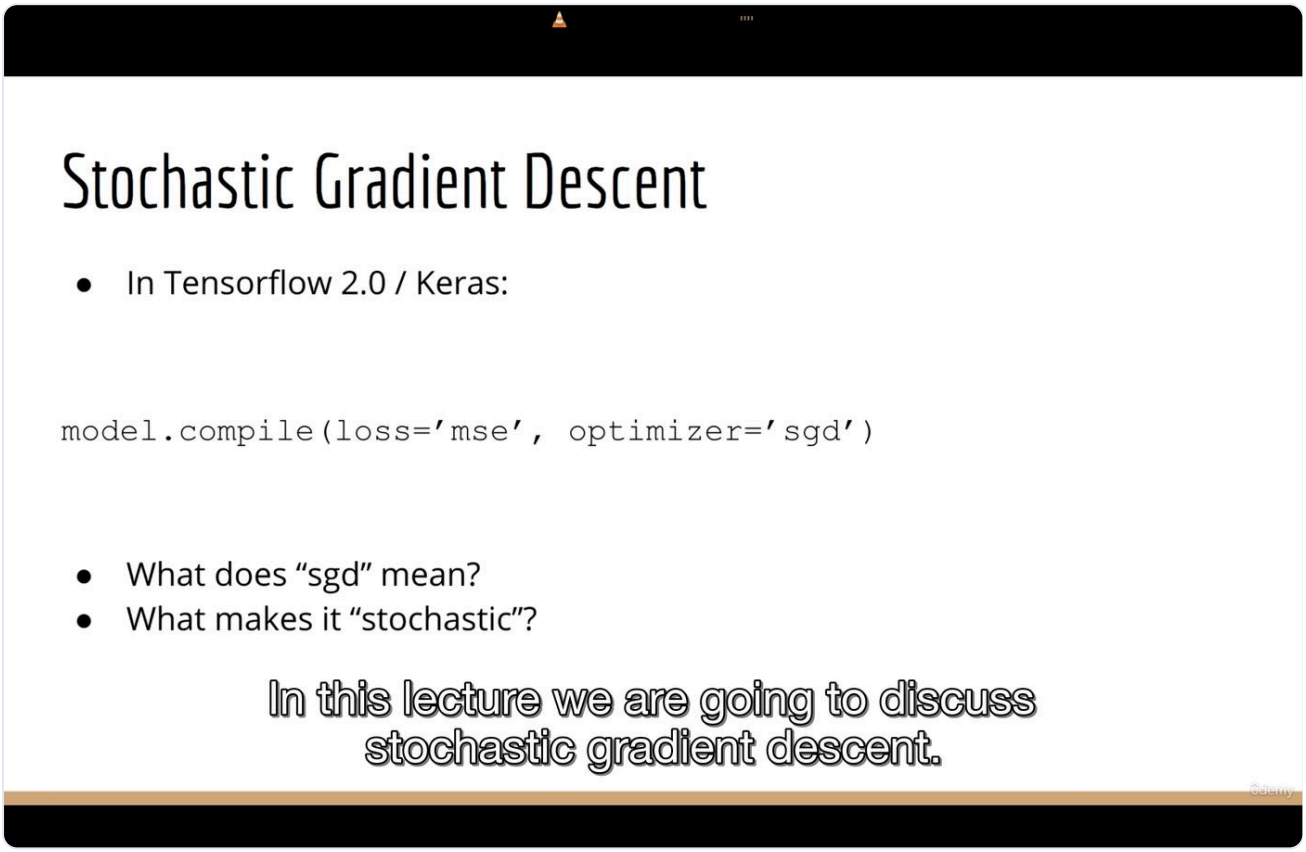

FIGURE 34.1 | Stochastic Gradient Descent

In this lecture we are going to discuss stochastic gradient descent. Previously we looked at regular gradient descent and how it can be used to optimize a function. But as you may know in TensorFlow 2.0 the gradient descent option is called SGD which is short for stochastic gradient descent. So what makes it stochastic and how is it different from non stochastic gradient descent. To understand this let's first think of a simpler problem. Let's suppose we want to measure the average height of all humans in the entire world. Now that would be quite a hefty task. There are 7 billion people in the world and it would take a very long time to ask each person individually.

But what if instead you took just one thousand random people and took the average height of these 1000 people would you expect the average height of these 1000 to be similar to the average height of the entire population. The answer is yes.

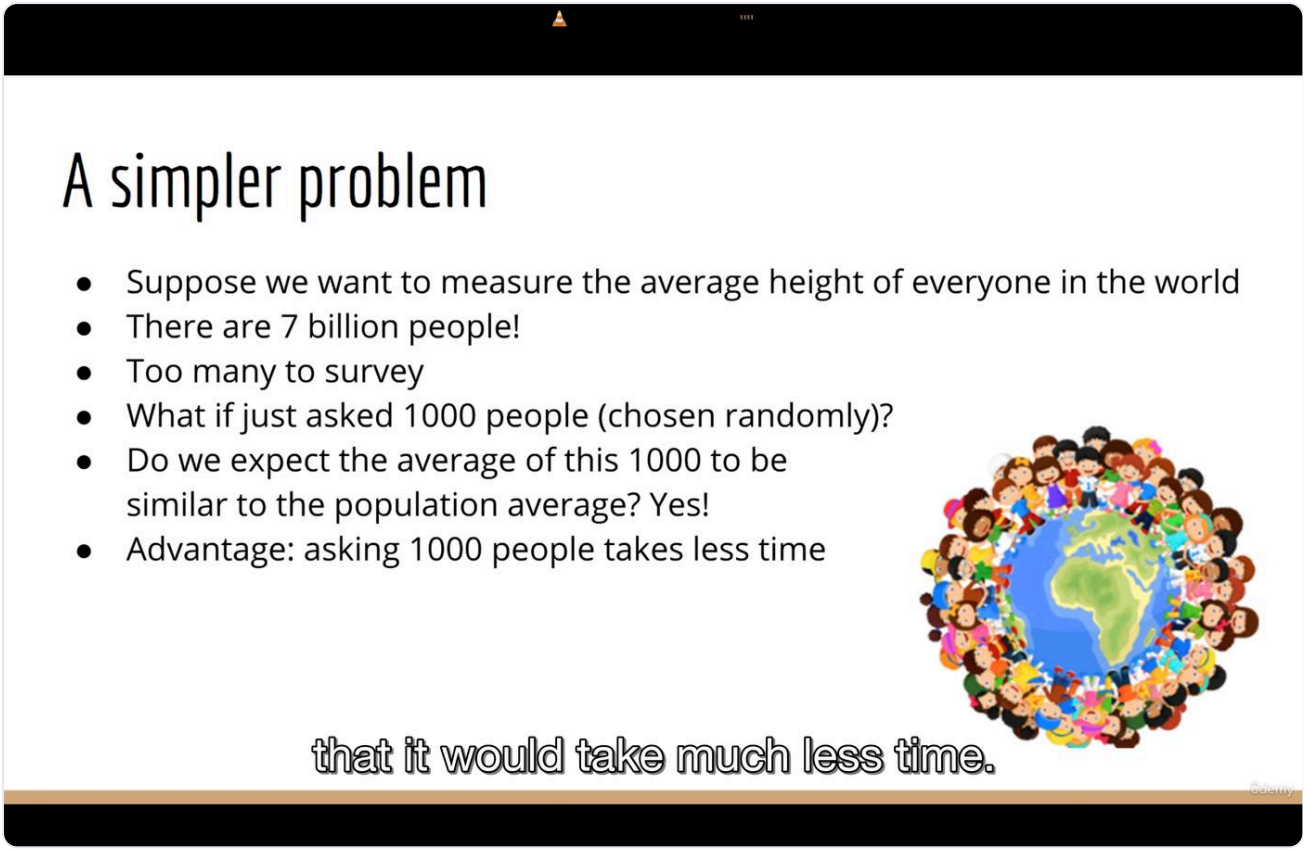

FIGURE 34.2 | A Simpler Problem

It should be clear that the main advantage of asking only 1000 people instead of 7 billion people is that it would take much less time. OK so how is this relevant for deep learning. If you recall in deep learning we have to calculate the cost. And from this cost we have to calculate its gradient so that we can do a gradient descent.

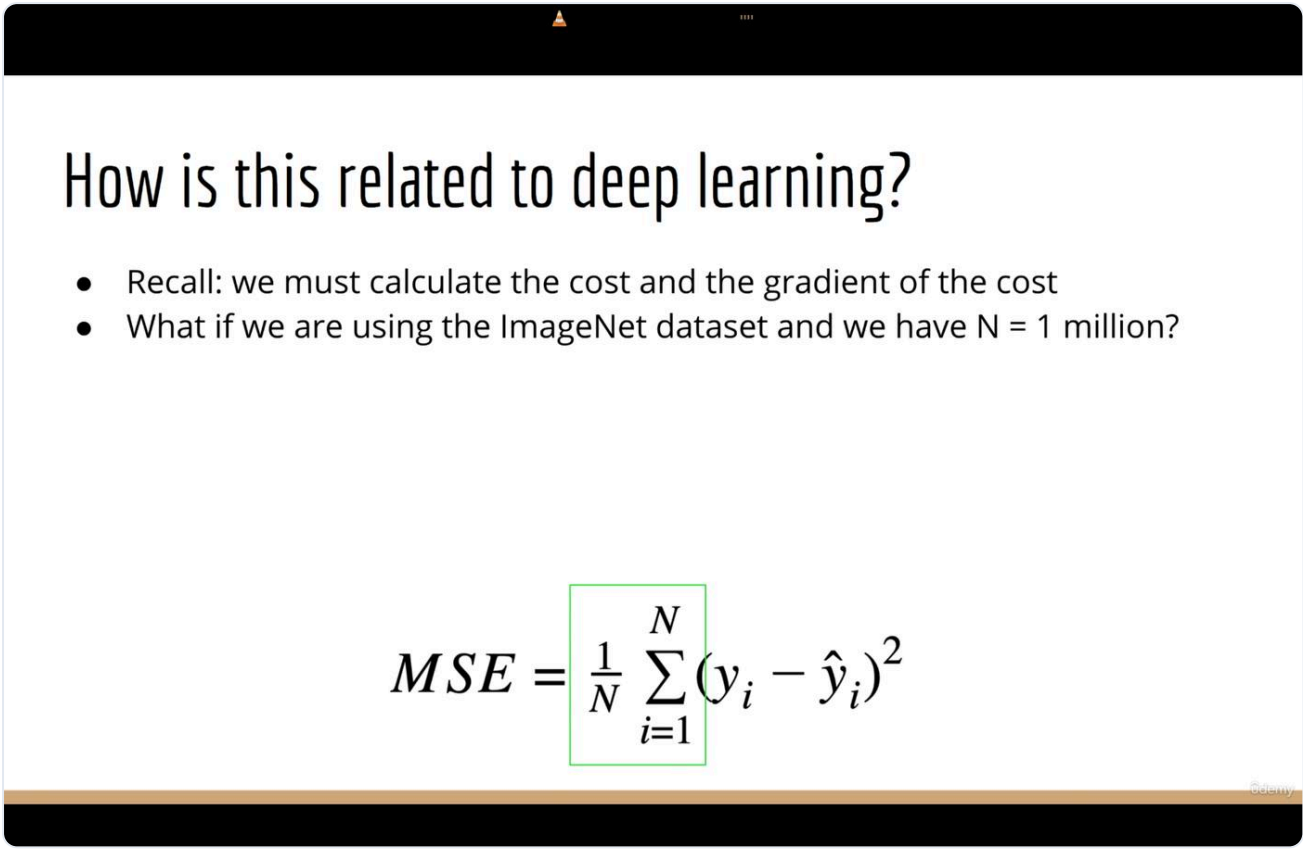

FIGURE 34.3 | How Is This Related to Deep Learning?

The problem is this cost depends on the number of training samples in our dataset. So imagine you are training on the famous ImageNet dataset which has about 1 million images. This means that for each iteration of your training loop you have to add up all the errors for all 1 million samples. That's going to take an extremely long time but what if instead I took the average error over just 1000 images. Of course they must be chosen randomly.

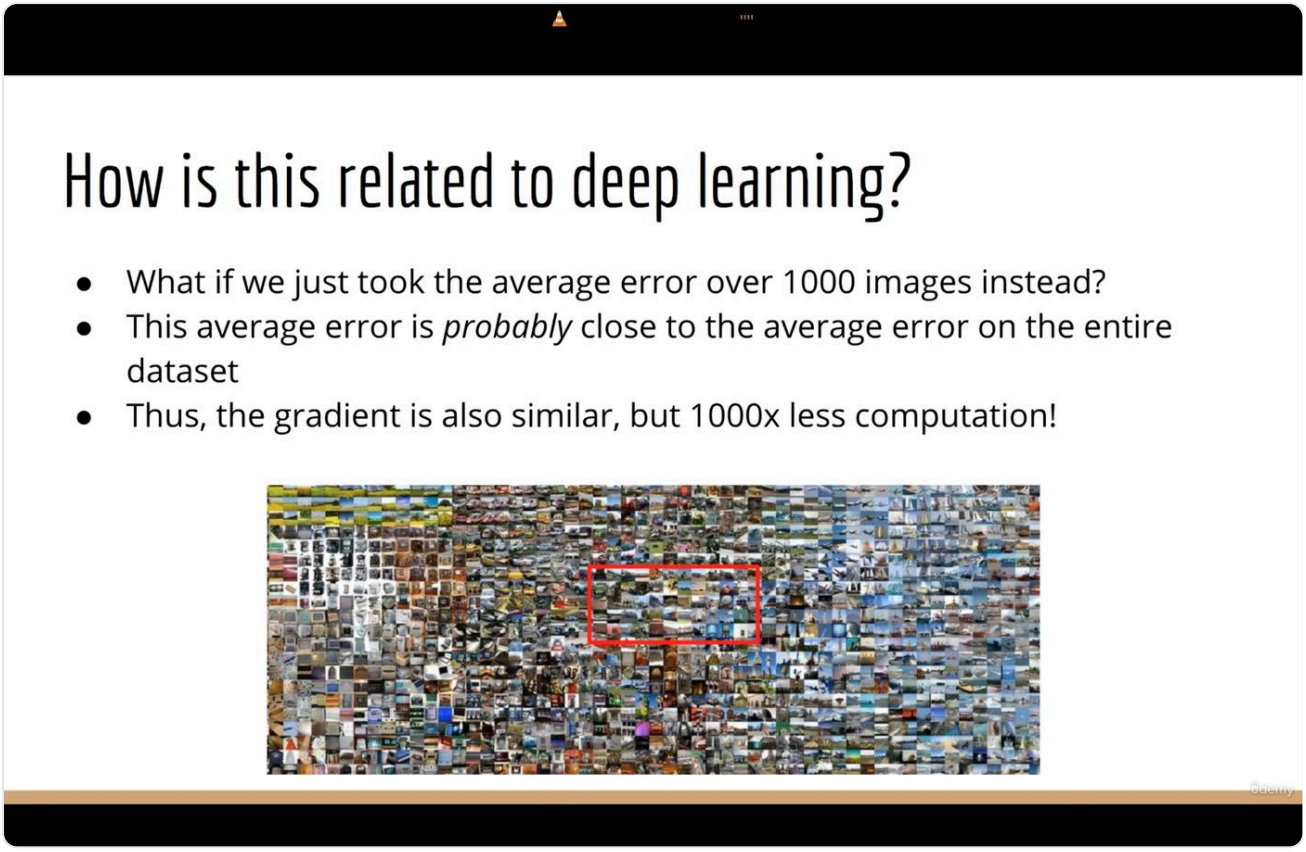

FIGURE 34.4 | How Is This Related to Deep Learning?

The same idea applies this average error is probably close to the average error over the entire dataset and so if we do gradient descent using only these 1000 images we will probably end up taking a very similar step.

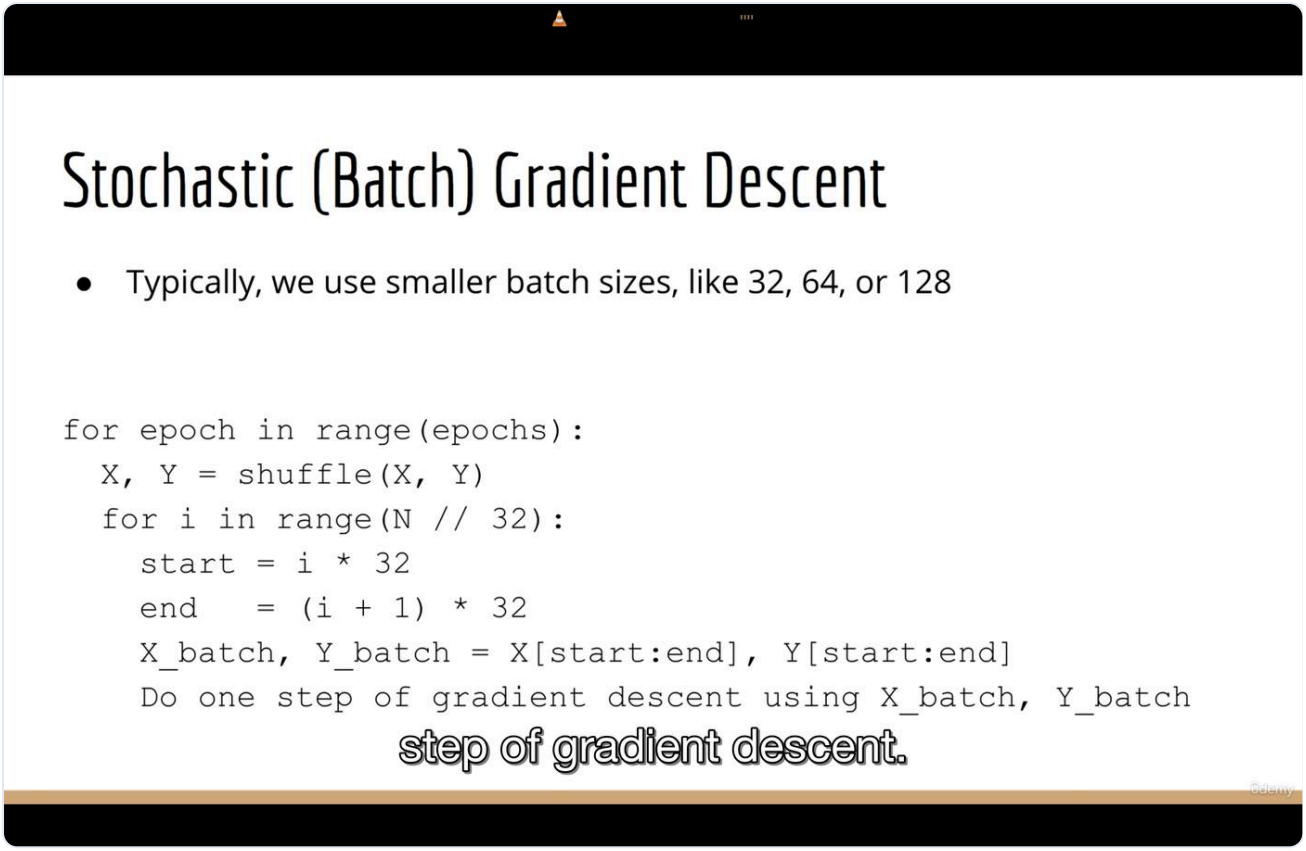

FIGURE 34.5

Just with 1000 times less computation in actuality we typically use smaller batch sizes like 32 64 or 128. The basic algorithm looks like this instead of just one loop over the epochs we have two nested loops one over the epochs and one over the training set. The idea is we are going to split up our data into batches and for each batch we are going to take one step of gradient descent. As you can see in this pseudocode we are using a batch size of 32 so the number of iterations of the inner loop is n divided by 32. We can specify the start index as I times thirty two and the end index as I plus 1 times 32. So this will count from zero to thirty to thirty two to sixty four sixty four ninety six and so forth on each epoch we will make one pass through the entire dataset.

It's important to randomize the data and each epoch because running through the same samples in the same order each time could lead to undesirable patterns being learned as an exercise. You may want to try this out for yourself to prove that it is faster to set up your experiment. Here's what you should do in the TensorFlow 2.0 fit function.

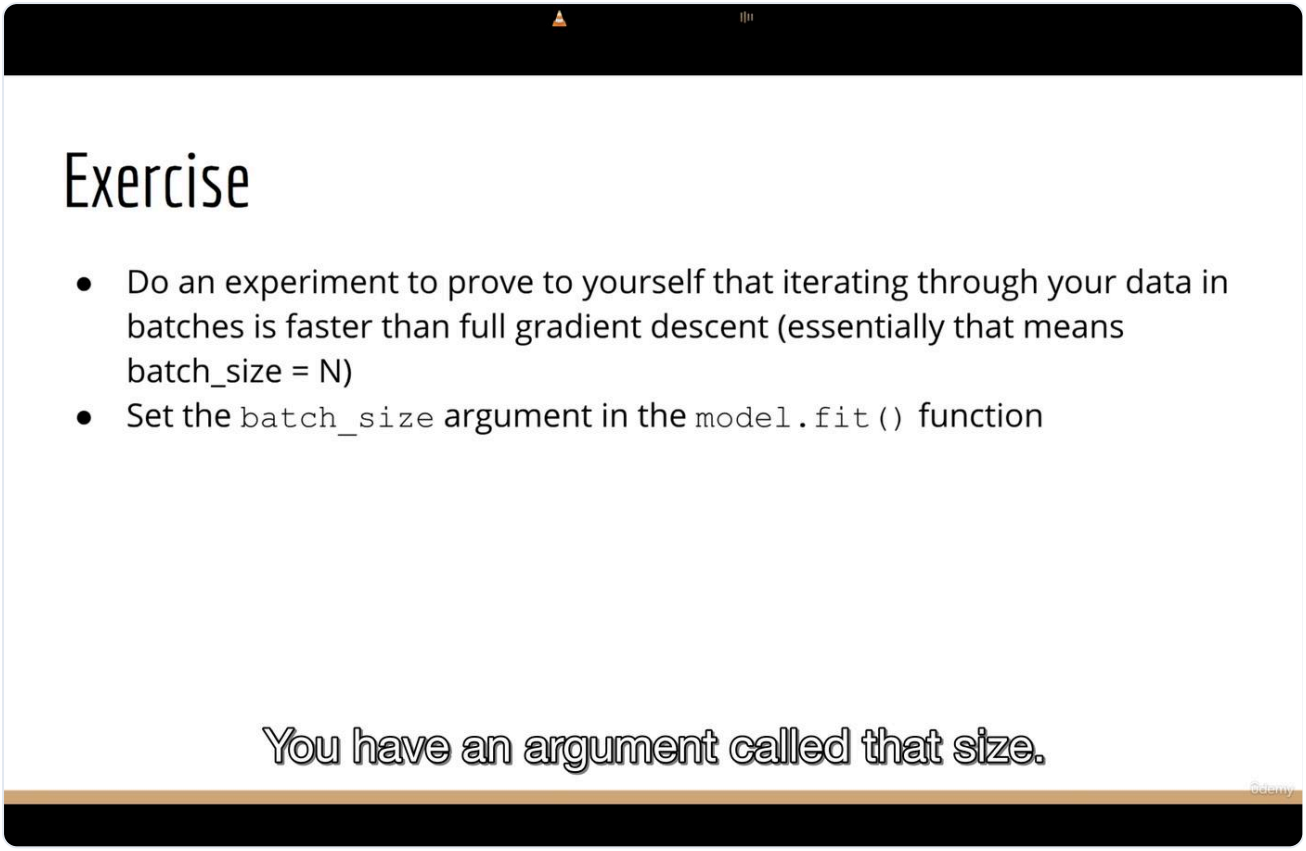

FIGURE 34.6

You have an argument called batch size. You can take any of the examples from this course and explicitly set the batch size to a small value such as 32 and explicitly set it to the size of the entire dataset. Whatever that may be from there. Compare the time it takes for each method to converge to a minimum. You should find that with batch gradient descent. The training process converges much faster and in fact sometimes your dataset may be so large that doing one step of gradient descent cannot even fit into memory.

LECTURE 128

Momentum Analogy

Everyone welcome back to this class modern Deep Learning in Python Deep Learning in Python. Part two in this lecture. We are going to look at what I consider to be one of the most effective improvements over plain gradient descent called momentum.

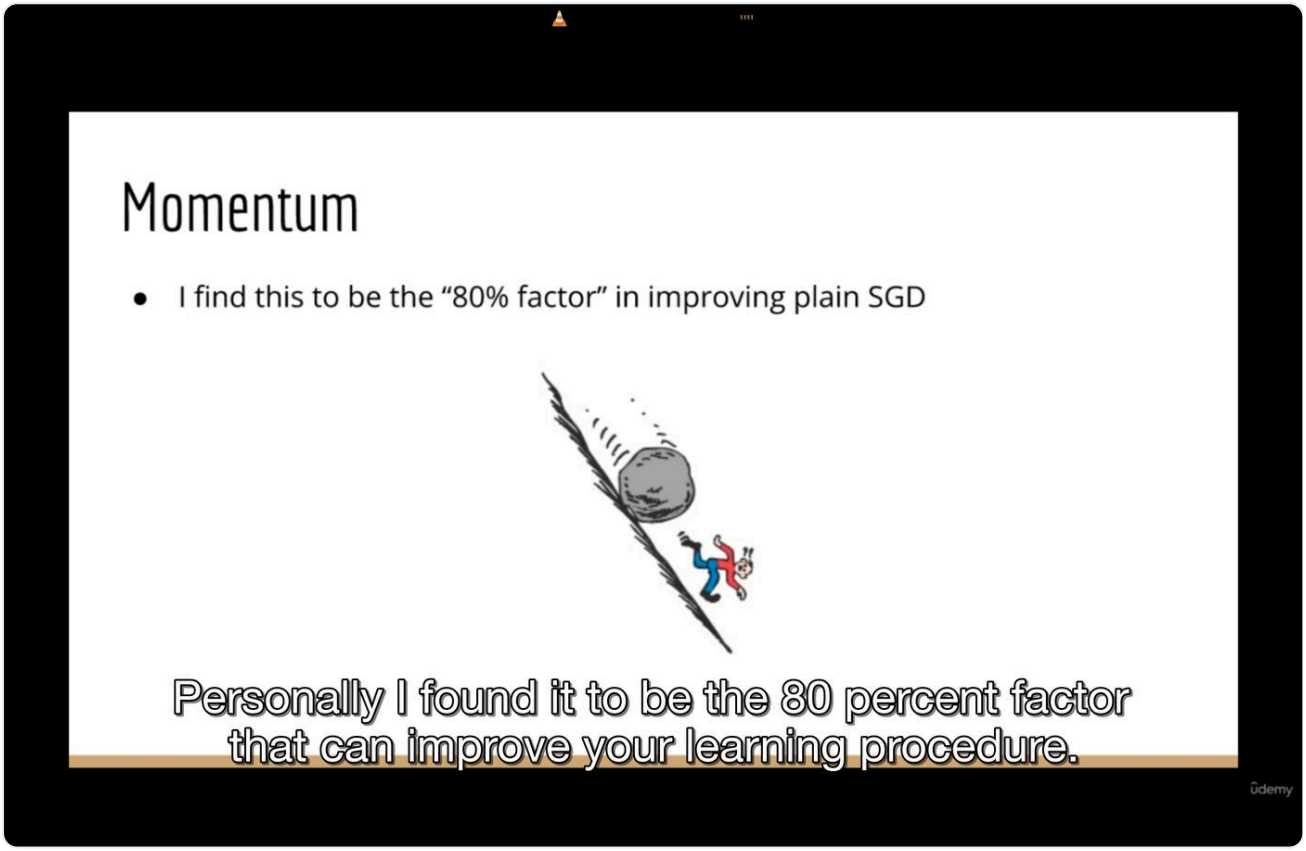

FIGURE 35.1

Personally I found it to be the 80 percent factor that can improve your learning procedure. So how does momentum work. I like the analogy of a ball rolling down a hill which is nice because during gradient descent we imagine the error surface as the bottom of a hill. But I think today an even better analogy is moving something on a frictionless surface like ice. So if you've ever gone skating before you know that you can easily glide along the ice very quickly and without much force because the momentum just carries you the way you were going before you can imagine pushing a box on ice and the box will move from one point to another quite easily because you can push it once and it's going to slide.

That's exactly what happens in physics.

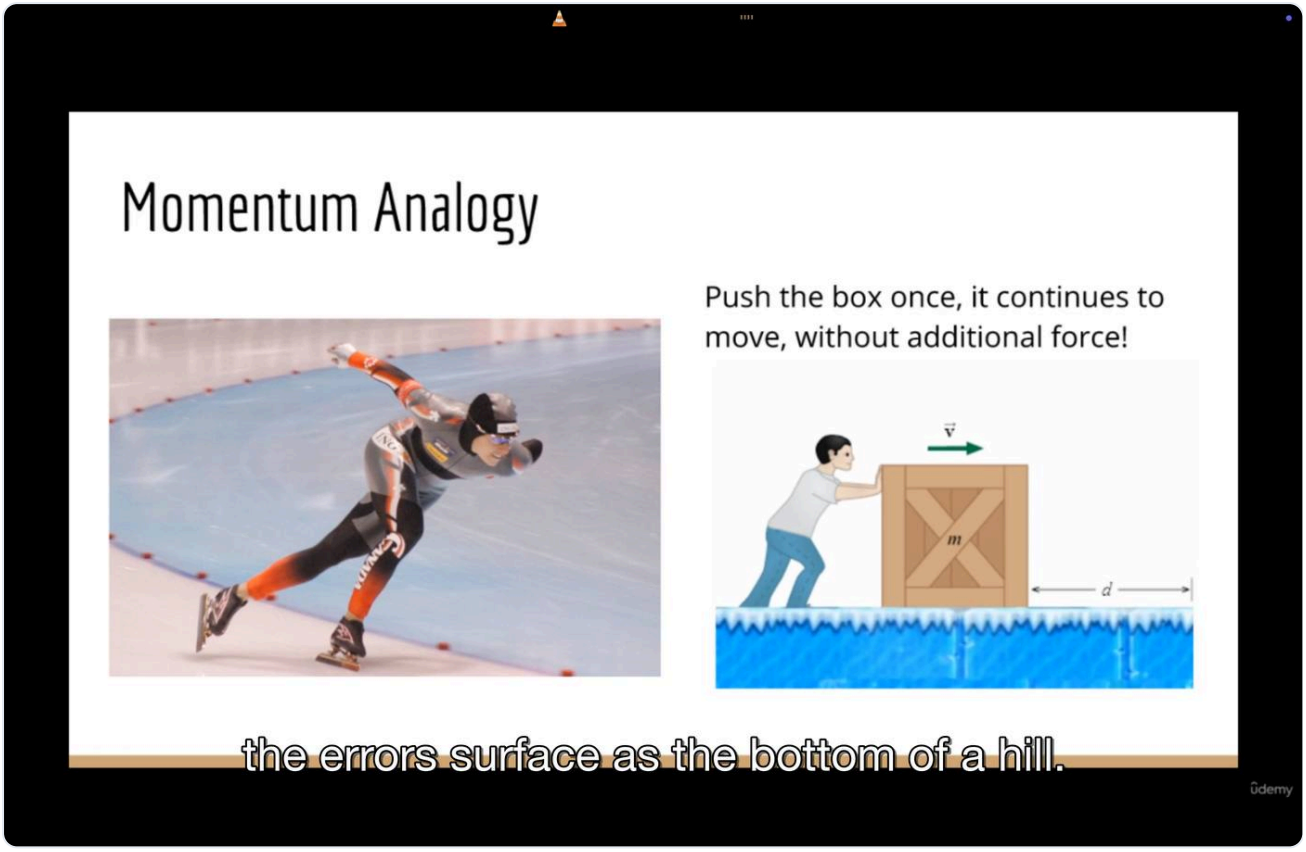

FIGURE 35.2 | Momentum Analogy

If there is no friction then an object will just continue to go in the direction it was already going of course ice still does have some friction. So eventually the box was slowed down and eventually stop as you'll see momentum in gradient descent also slows down after a while to to continue with this analogy. Let's try to think of a situation without momentum. Now keep in mind I'm talking about gradient descent momentum here not physics momentum since the definition of momentum and physics is a little different. So imagine now that instead of ice you're trying to push a box in sand. Now you can imagine that this is going to be very difficult because sand has lots of friction.

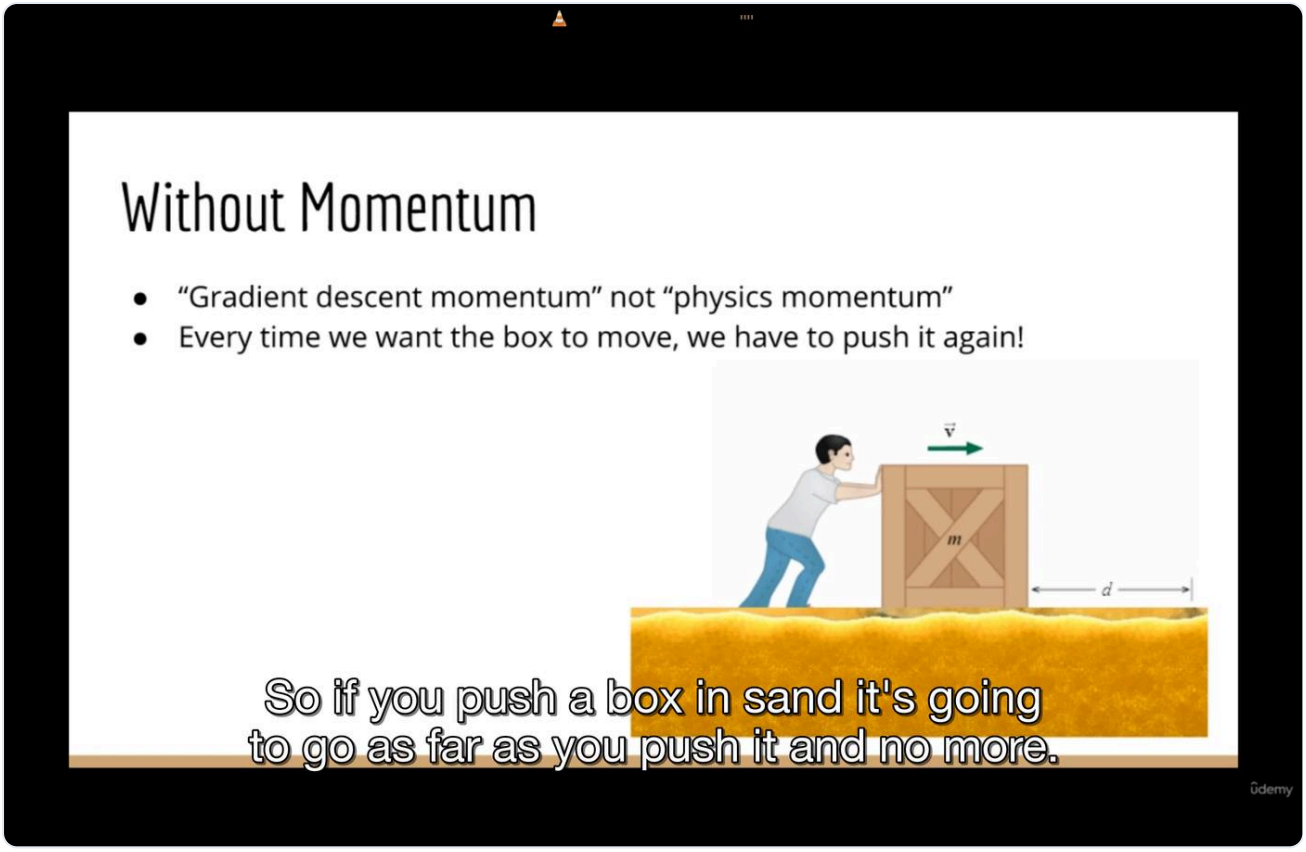

FIGURE 35.3 | Without Momentum

So if you push a box in sand it's going to go as far as you push it and no more. If you want the box to move again you have to push it again. That's like gradient descent without momentum each time. If we want to move there has to be a gradient so that we can move in the direction of the gradient. Now let's try to put these ideas into math. Let's start with regular gradient descent without momentum.

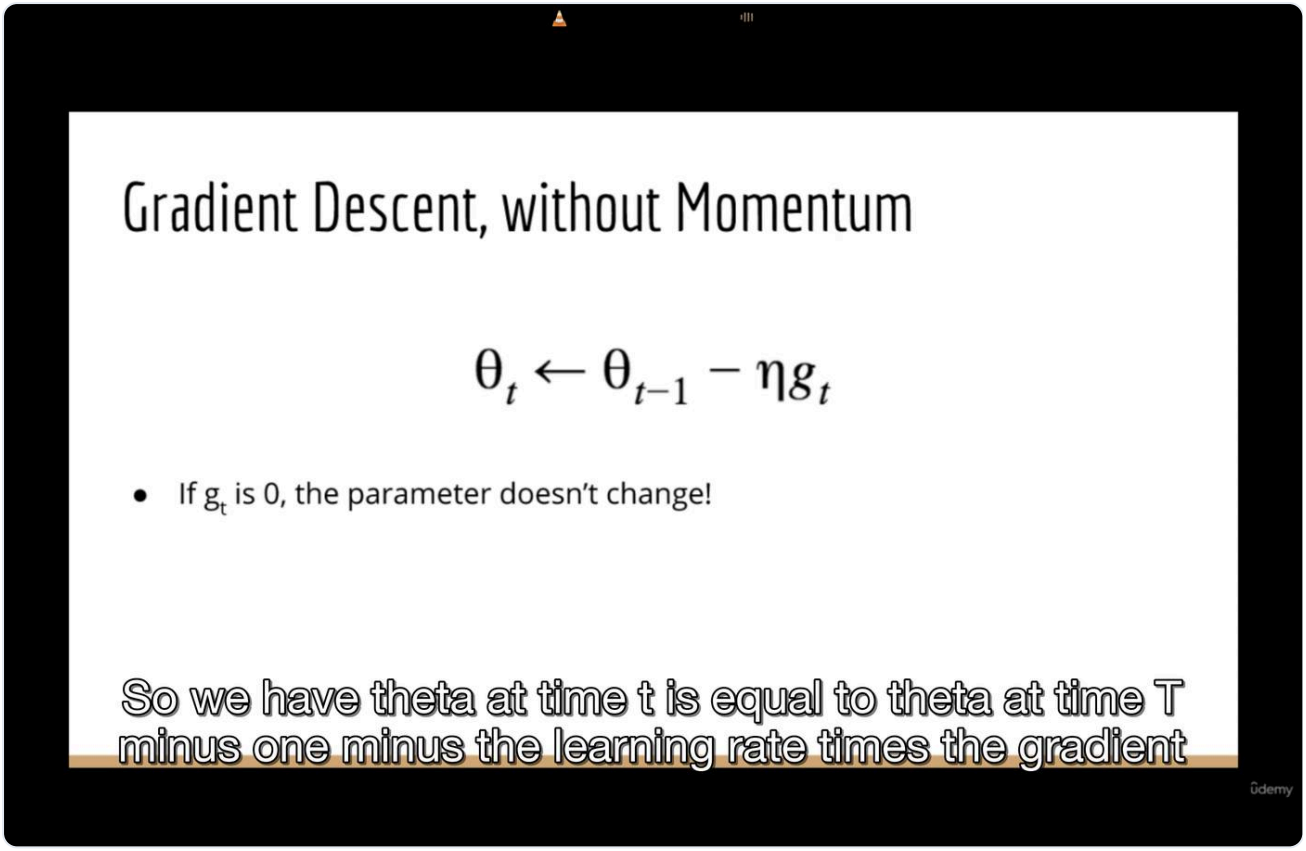

FIGURE 35.4 | Gradient Descent, Without Momentum

So we have theta at time t is equal to theta at time T minus one minus the learning rate times the gradient G. And from this we can see that if the gradient is zero nothing is going to happen to theta it just gets updated to its old value. It doesn't change now let's look at gradient descent with momentum.

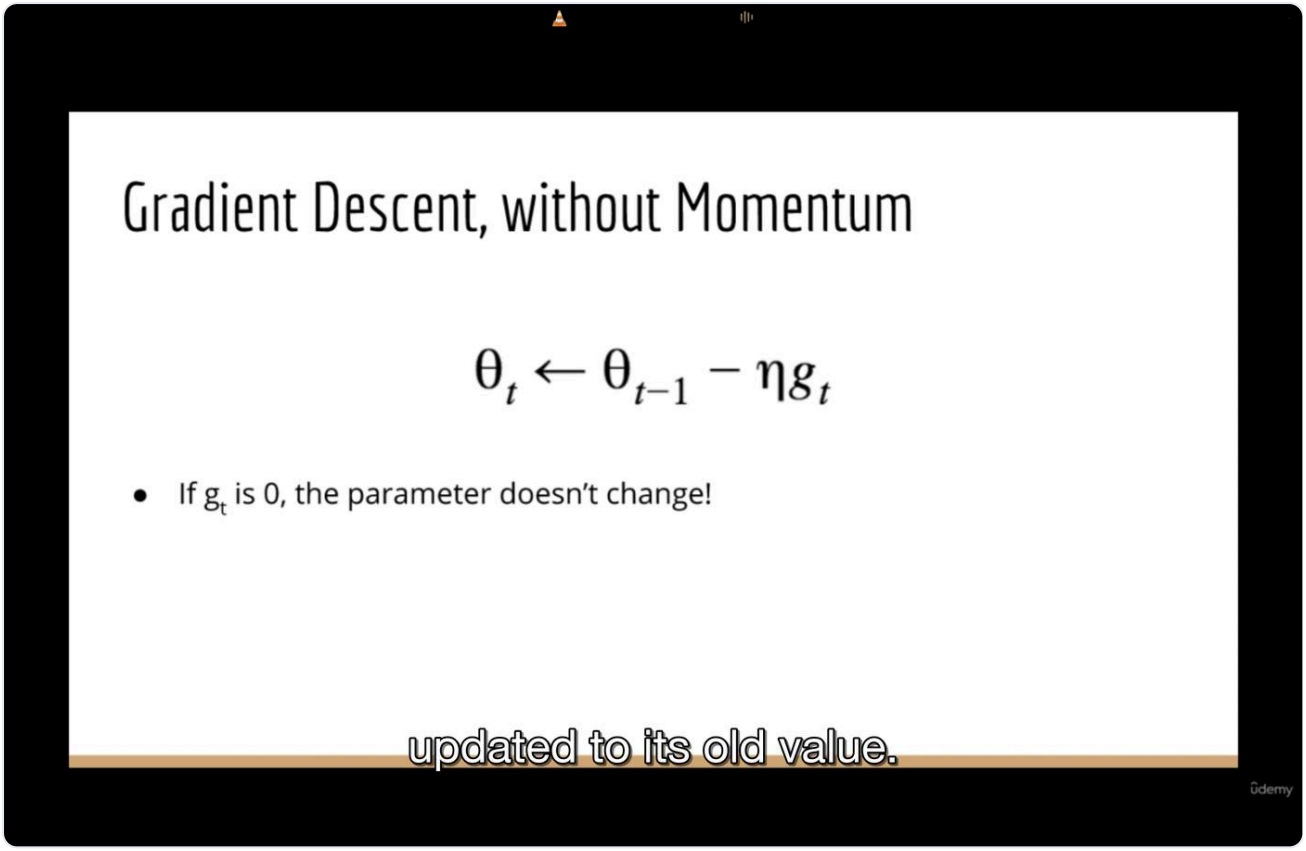

FIGURE 35.5 | Gradient Descent, Without Momentum

We use the term momentum very loosely here since it has nothing to do with actual physical momentum. So we create a new variable called V which stands for the velocity it's equal to mu times its old velocity minus the learning rate times the gradient. And this is exactly what we talked about earlier when we imagine pushing a box on ice. The G term is the effect of our pushing the box but the mu times V of T minus one term is the effect of continuing to move in the same direction as we were going before.

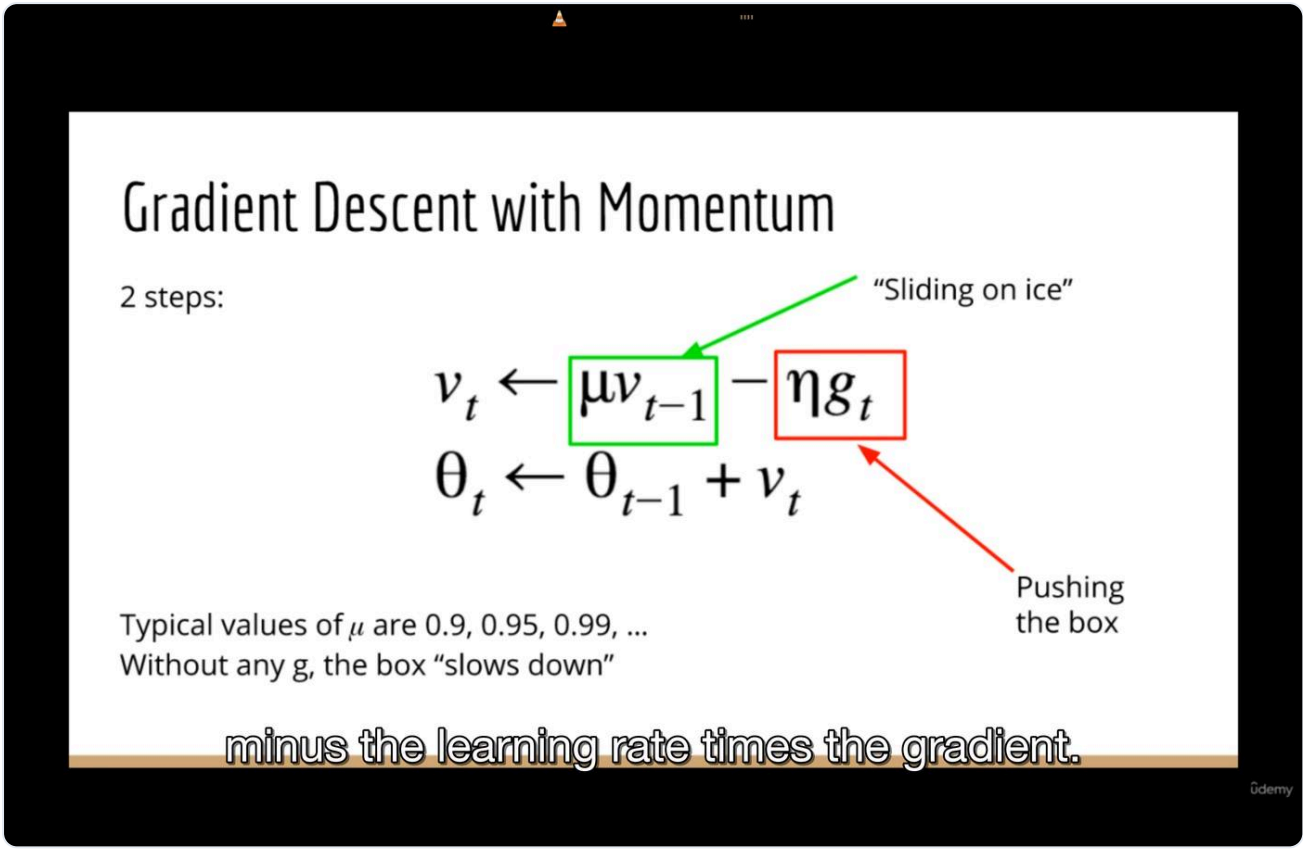

FIGURE 35.6 | Gradient Descent with Momentum

Now we talked about how when you push a box on ice even though there's less friction and the box can slide it's still going to stop eventually. So we want the new V to take on only a fraction of the old V and so a typical value for mu which we also call the momentum term is zero point nine or zero point nine five or zero point nine nine. Somewhere in that area one thing you can immediately see is that if we plug in the expression for V of t we can get the update for theta of T in terms of only the old theta the old V and the gradient G.

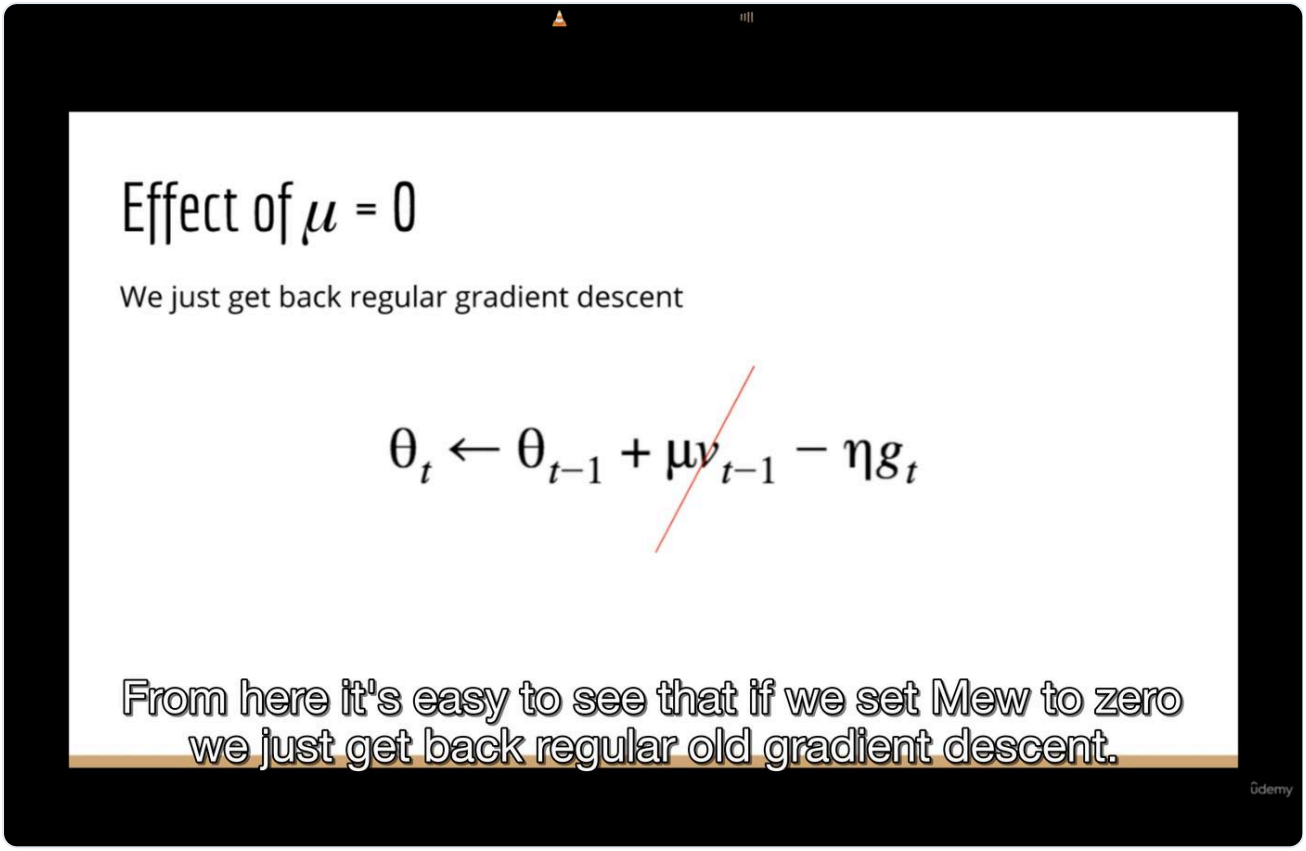

FIGURE 35.7 | We Just Get Back Regular Gradient Descent

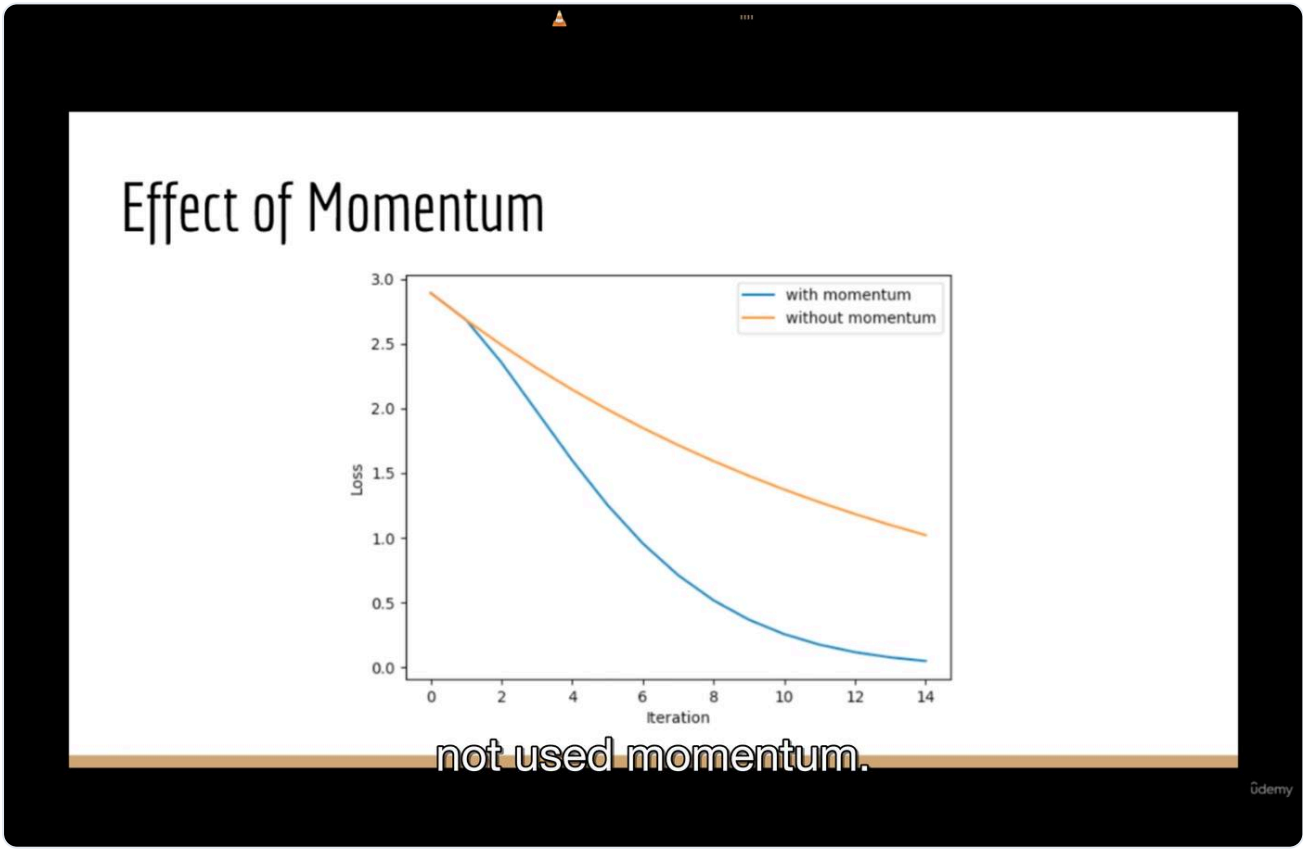

FIGURE 35.8 | Effect of Momentum

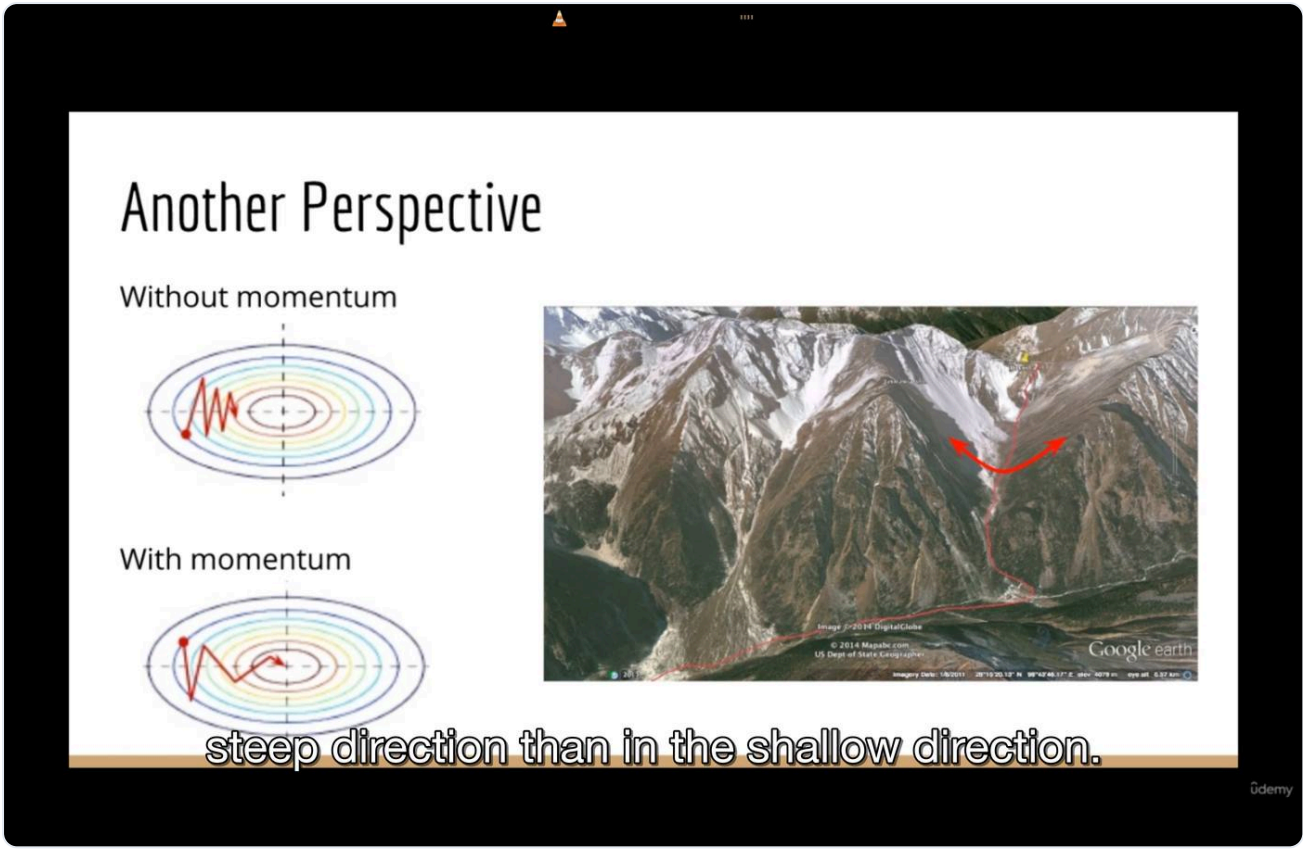

FIGURE 35.9 | Another Perspective

From here it's easy to see that if we set mu to zero we just get back regular old gradient descent. So what is the effect of using momentum. Well as you know in all of our scripts we like to plot the cost per iteration. And so what we usually see is that the cost converges to its minimum value much faster than if we had not used momentum. So using momentum is pretty great because it significantly speeds up training. Now I want to discuss one more perspective on momentum before we move on. This is what is typically taught in deep learning courses these days. So in this perspective the problem that momentum is solving is if we have unequal gradients in different directions.

So for visualization purposes let's assume we have two parameters to optimize the vertical parameter and the horizontal parameter. So the gradient in one direction is very steep and the gradient in the other direction is very shallow. The idea is if you don't have momentum then you rely purely on the gradient which points more in the steep direction than in the shallow direction. And this is just a property of the gradient. It's the direction of steepest descent. So since this gradient vector points more in the steep direction than the result is we're going to zigzag back and forth across this valley. And that's a really inefficient way of reaching the minimum.

So what happens when we add momentum to this situation. Well since in the shallow direction we move in the same direction every time those velocities are going to accumulate. So we'll have a portion of our old velocity added to our new velocity to help us along in that direction. The result is that we get there faster by taking bigger steps in the direction of the shallow gradient.

LECTURE 129

Variable Learning Rates

Hey everyone and welcome back to this class modern deep learning and Python Deep Learning in Python. Part two in this lecture.

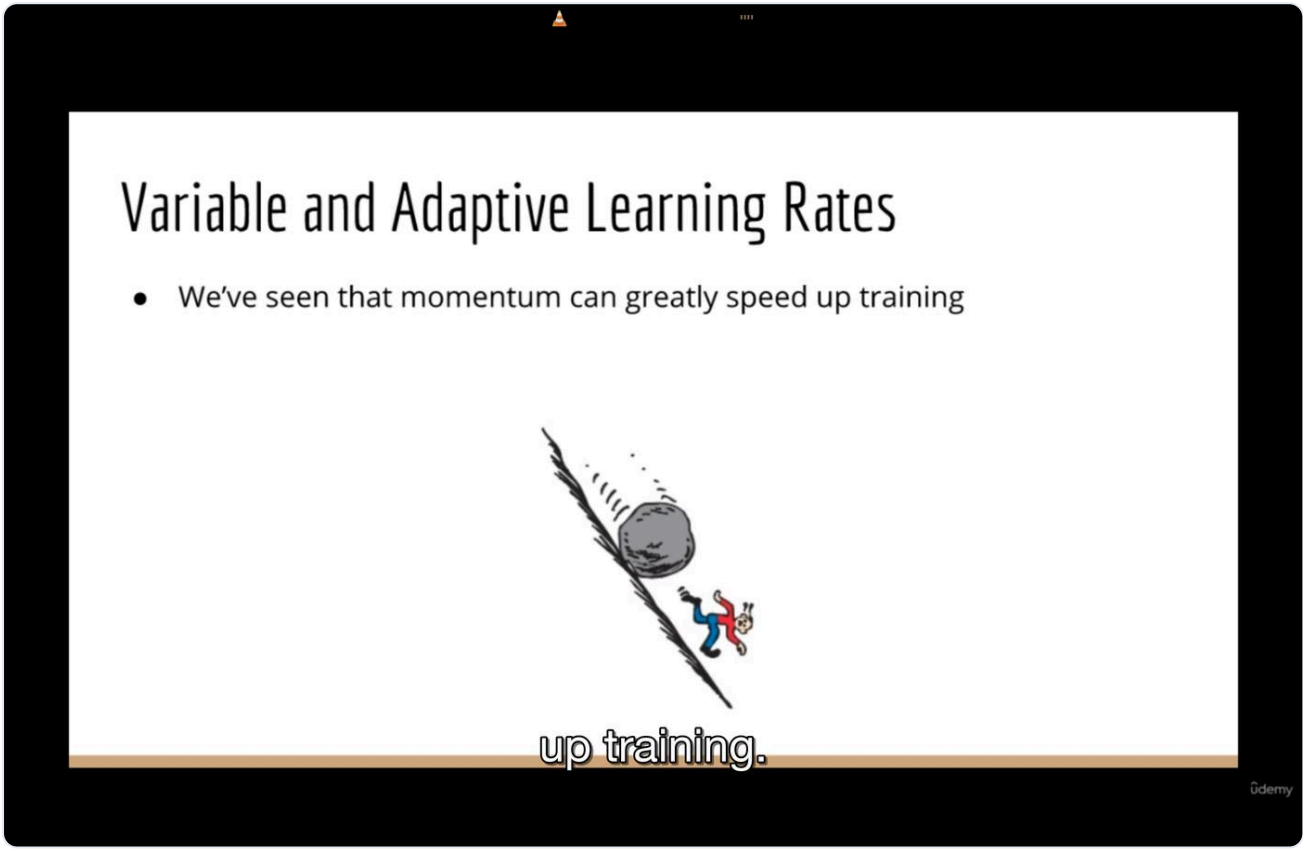

FIGURE 36.1 | Variable and Adaptive Learning Rates

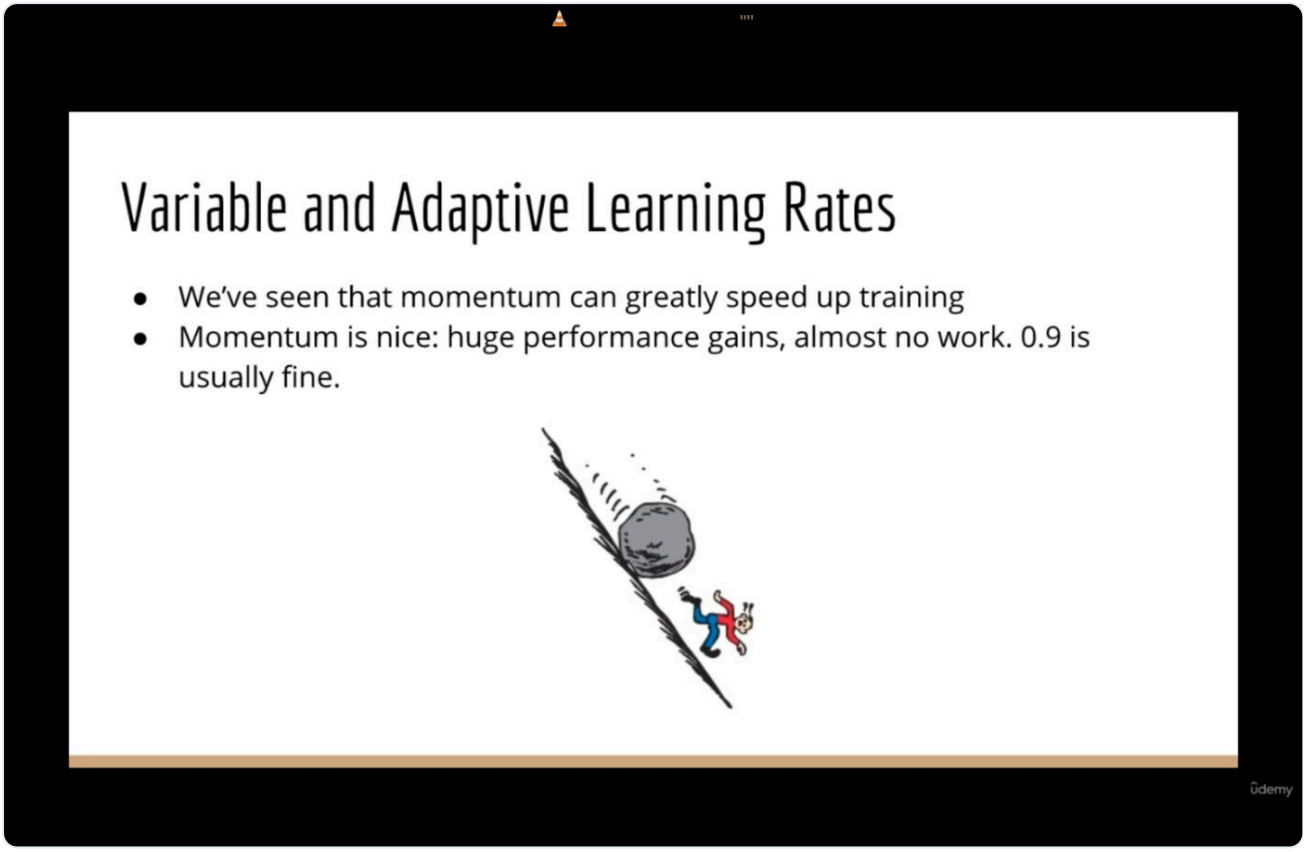

FIGURE 36.2 | Variable and Adaptive Learning Rates

We are going to discuss variable and adaptive learning rates. So far we've looked at momentum as a modification to vanilla back propagation that can greatly speed up training. And from my perspective I find momentum to be the most impactful. If you compare the loss per iteration for a standard gradient descent versus gradient descent with momentum the difference is huge. At the same time momentum is nice because you don't really have to play with the hyperparameters that much just go with zero point nine. And it's probably fine.

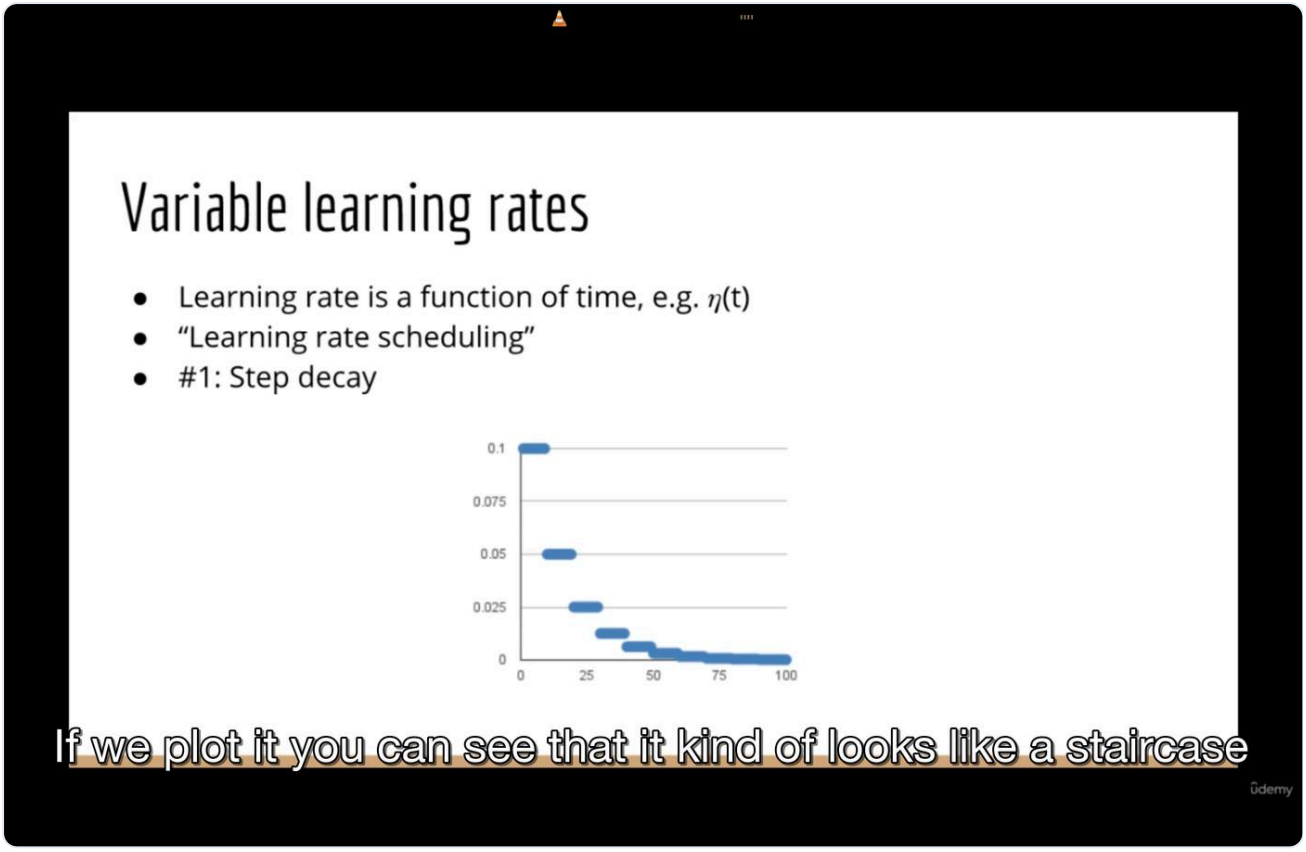

FIGURE 36.3 | Variable Learning Rates

So momentum is often my go to at the same time. Some of these adaptive learning rate techniques are very powerful. So let's have a look. The first thing we'll talk about is variable learning rates or in other words learning rates as a function of iteration or time. Sometimes people call this learning rates scheduling. They are very simple but they give you a lot to play with. So the first one we're going to look at is called step decay.

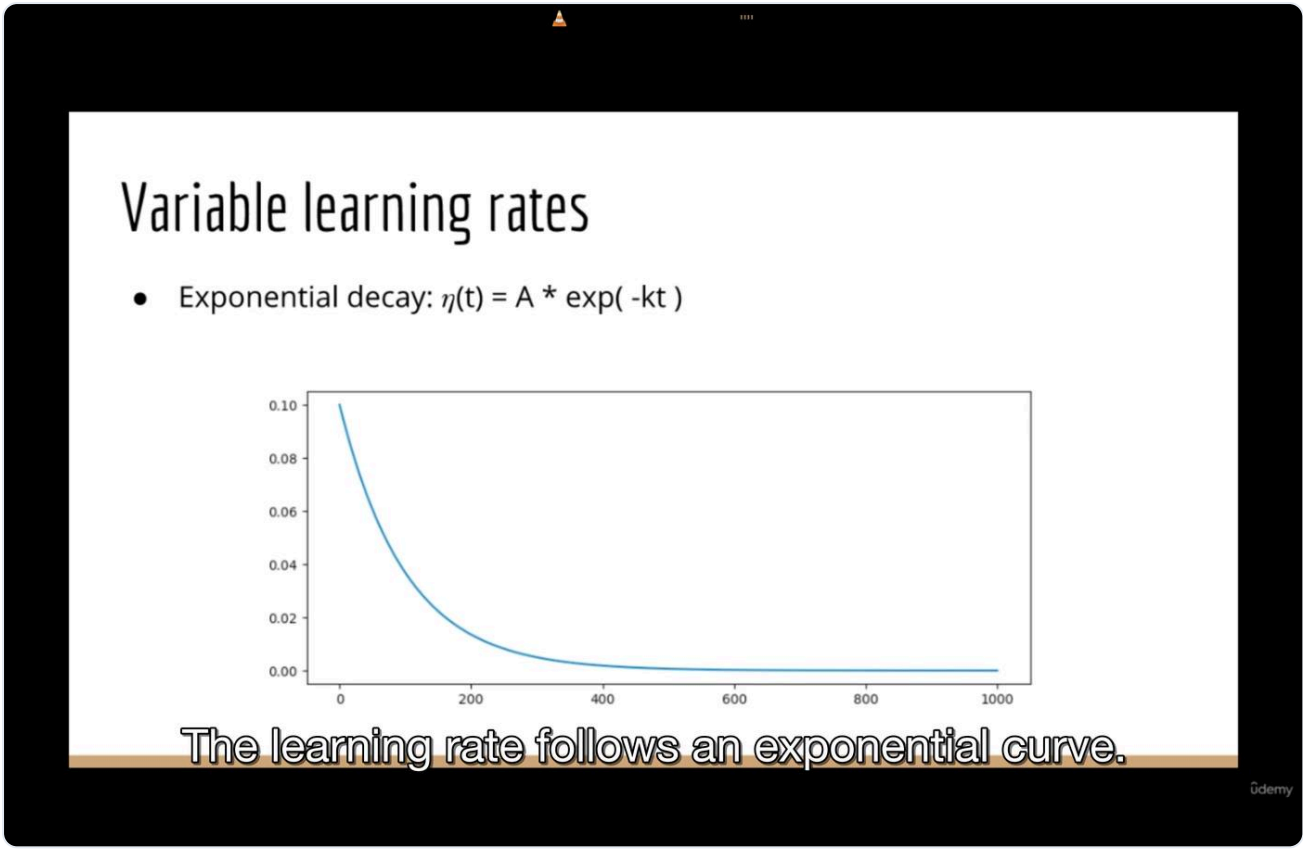

FIGURE 36.4 | Variable Learning Rates

Essentially what we do is periodically say every 100 steps we reduce the learning rate by a constant factor. So for example we divide by two or halve it each time. If we plot it you can see that it kind of looks like a staircase a second method is called exponential decay in this method. The learning rate follows an exponential curve. The third method is for the learning rate to decay proportionately to one over time. You can control how fast or slow the learning rate decays by changing the proportionality constant.

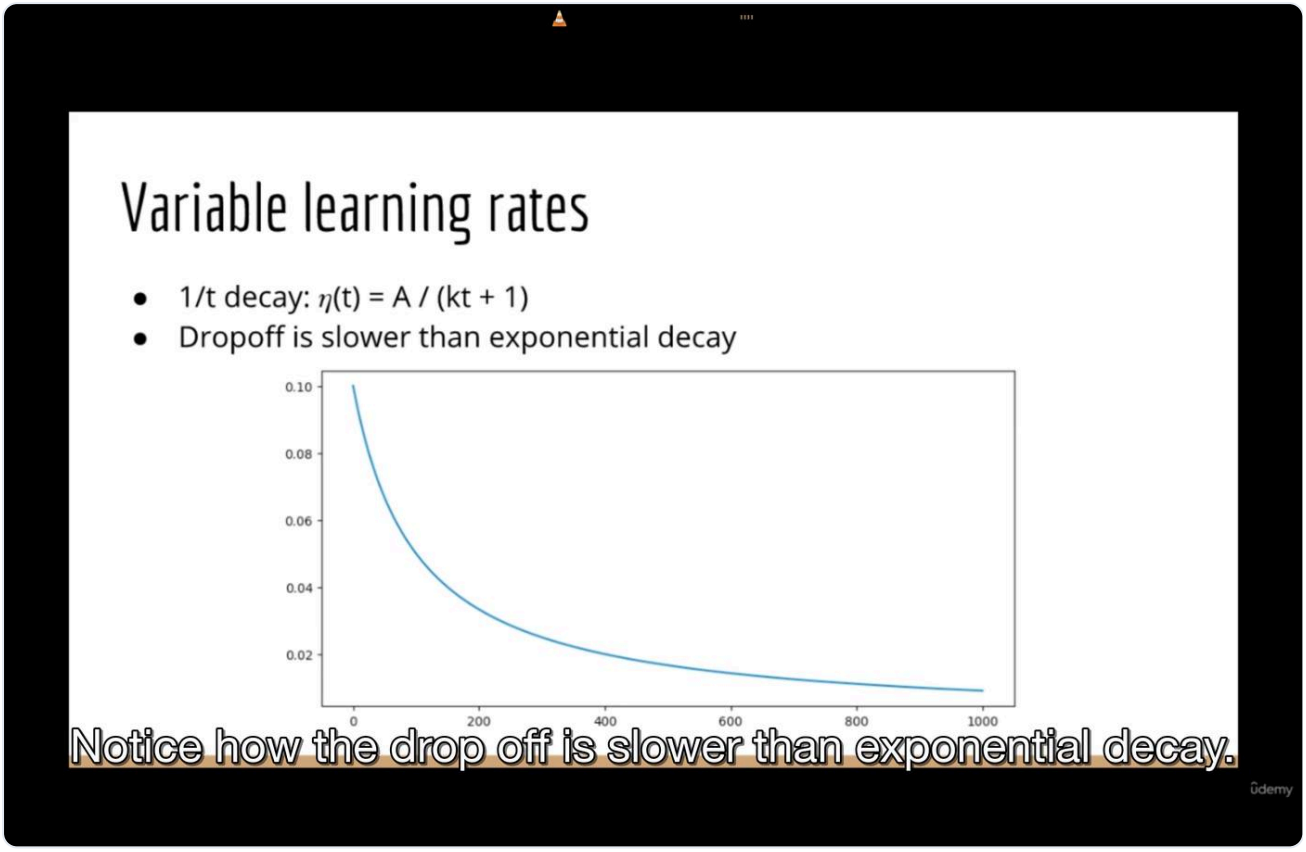

FIGURE 36.5 | Variable Learning Rates

Notice how the drop off is slower than exponential decay. So what do all of these have in common. Well you can see that they all decrease the learning rate with time. Why would we want to do that. Well generally speaking when we initialize the weights of a neural network randomly they're going to be very far from the optimal weights. So it's good to start with a large learning rate so that we can take bigger steps towards the goal. This is the motivation behind momentum as well. We want to pick up speed by accumulating past gradients because we know that if we are very far away from our goal then those gradients should be large. But when we get close to the goal the gradient is going to shrink.

In fact by definition the minimum of a function has a gradient of zero. That's how you solve for a minimum of a function in calculus. If you recall you find the derivative and then you set it to zero and then you solve for the parameter in question. So why might we want to slow down when we get close to the minimum. Well when you close to the minimum and you take too big of a step you're going to overshoot. And so what ends up happening is you just end up bouncing back and forth. In fact if your learning rate is too large you'll just bounce right out of the valley and your loss might actually increase. So in order to reduce all this bouncing around we would like to take small steps.

Here is another technique that people use which is related to these variable learning rate techniques. Sometimes researchers and machine learning practitioners will actually sit there and babysit the neural network training process so they'll do a couple of epochs and see how it goes if learning is happening too slowly. They might increase the learning rate if learning is starting to even out they might decrease the learning rate but you have to be careful because you're not

guaranteed to have a nice monotonically decreasing curve. Sometimes you might get stuck in a relatively flat portion of the error surface but this could just be temporary. And if you're patient enough you might end up getting a steep drop.

But at the same time this is not guaranteed as with most things in machine learning. The answer is that behavior is data dependent waiting longer is desirable. If you have the resources because then you get a full picture of the learning process. So manual learning rate scheduling is also an option for you to choose from now.

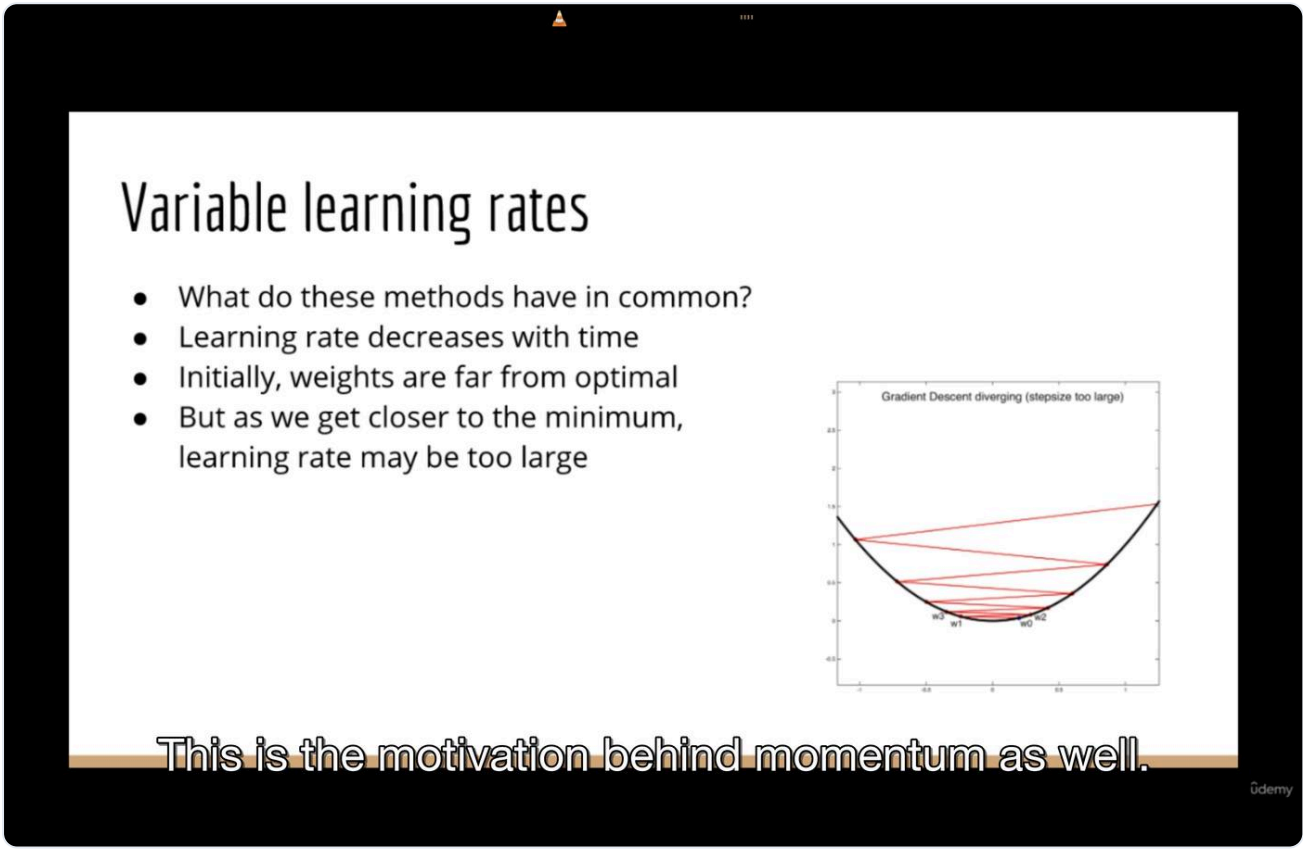

FIGURE 36.6 | Variable Learning Rates

Also important to think about what else to all these methods have in common. Well all of these methods add new hyperparameters to your list of things to optimize.

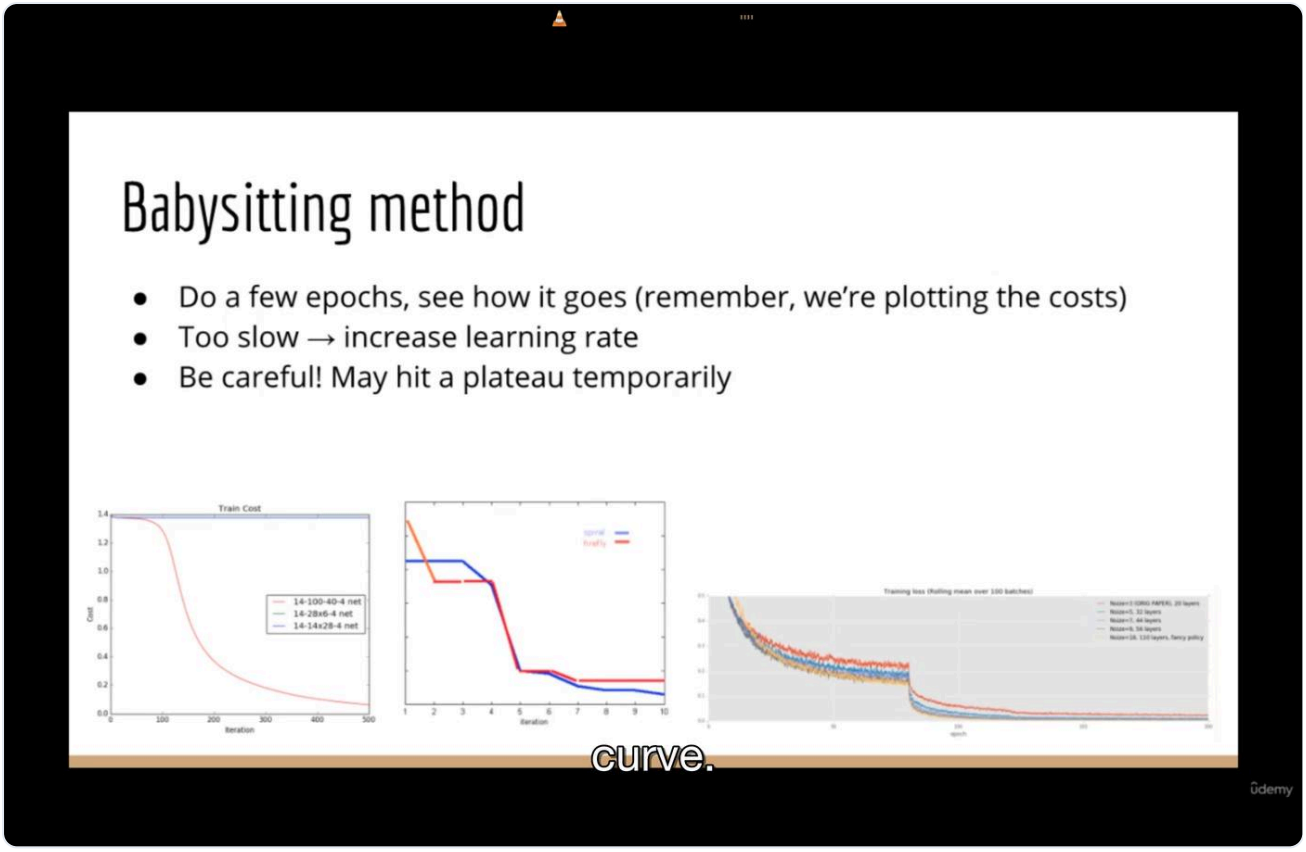

FIGURE 36.7

In fact choosing between each of these methods can also be seen as a hyperparameter optimization. So in one way knowing about these techniques can help you but from another perspective it adds more work to your plate the next couple of techniques I want to talk about are what I like to call adaptive learning rate techniques because they adapt to the training data that you've seen so far. The first one I want to discuss is called AdaGrad.

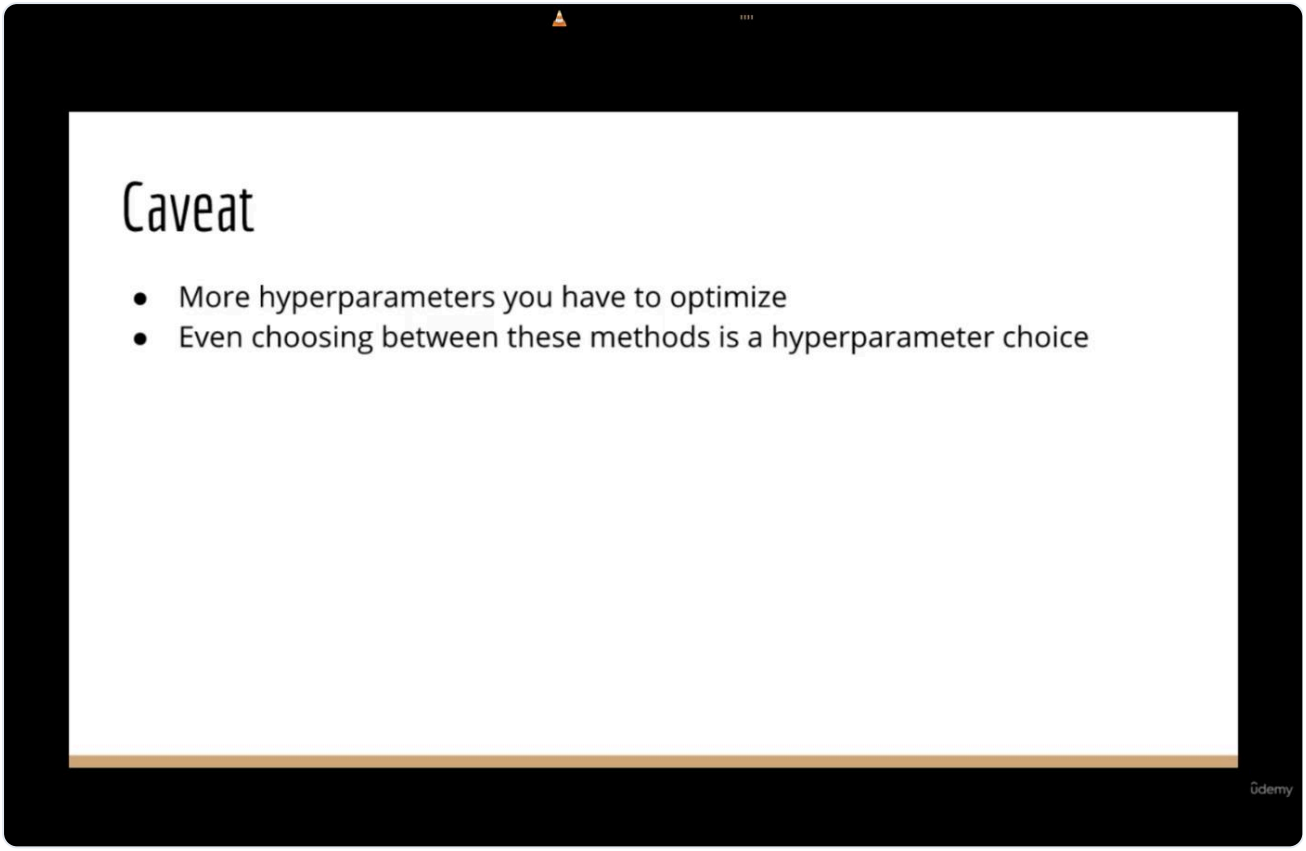

FIGURE 36.8

The basic idea is this we can't expect the dependence of the cost on each of the parameters to be the same. In other words in one direction the gradient might be really steep but in another direction the gradient might be really flat.

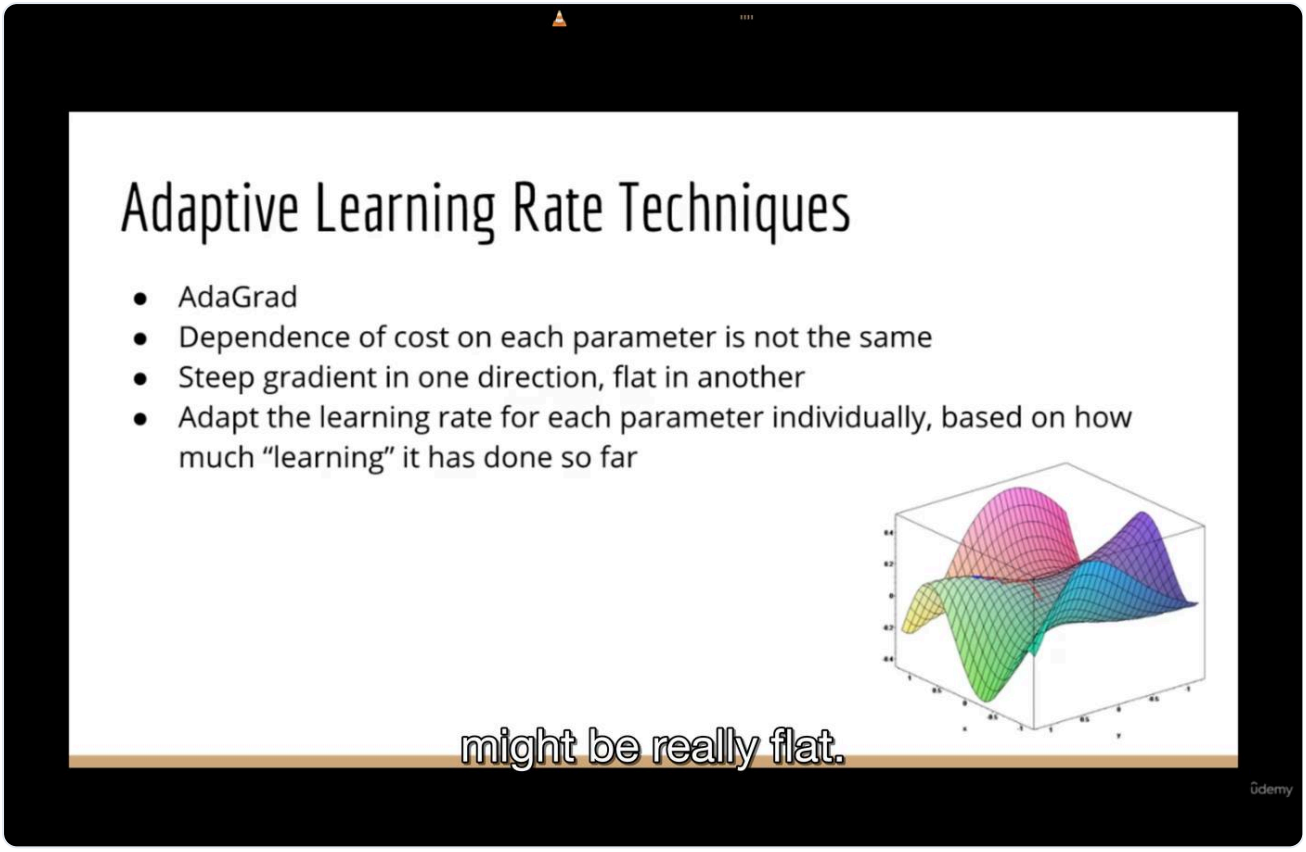

FIGURE 36.9 | Adaptive Learning Rate Techniques

So perhaps it may be beneficial to adapt the learning rate for each parameter individually based on how much it has changed in the past so in AdaGrad what we do is we introduce a variable called the cache each parameter of the neural network has its own cache.

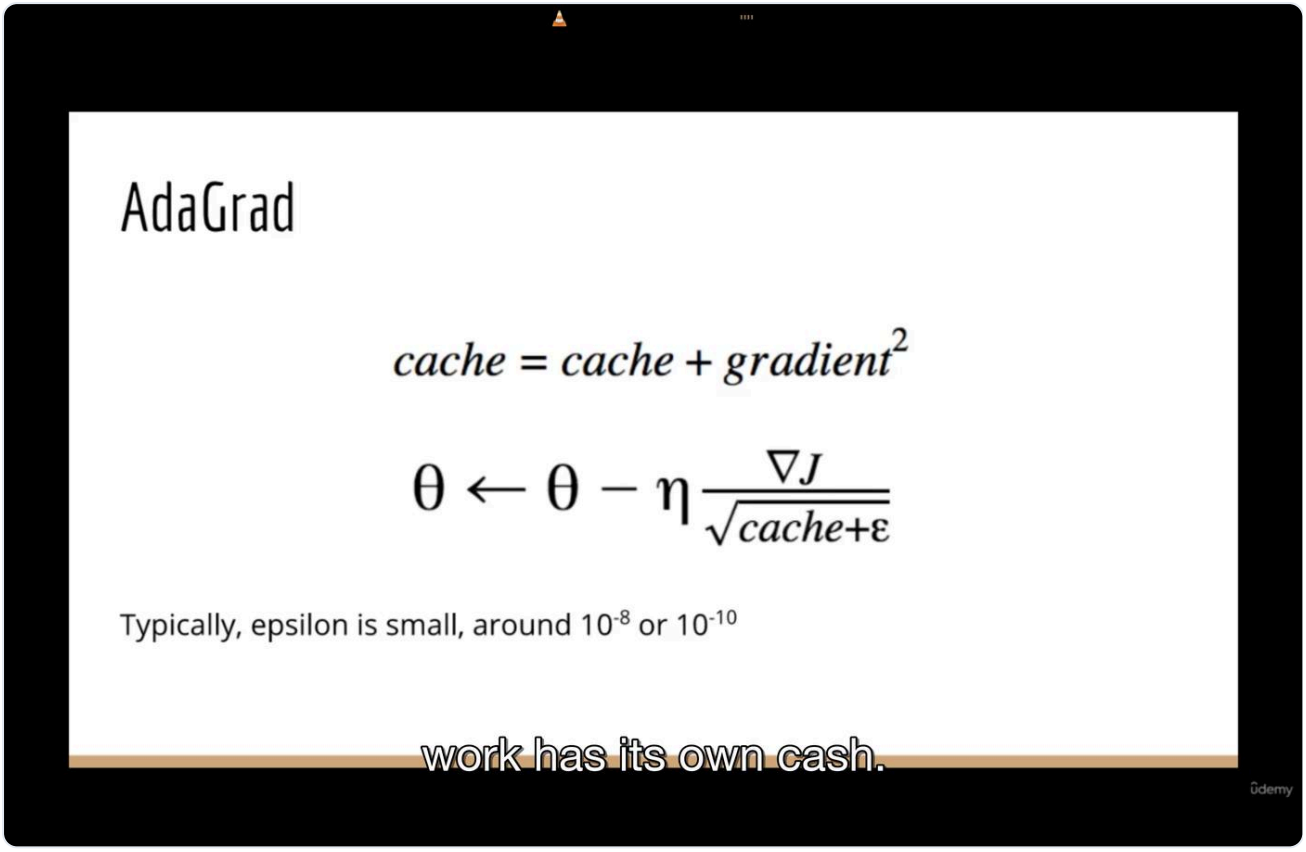

FIGURE 36.10

So for example if you have one weight matrix of size three by four then you'll also have a cache matrix of size three by four and the same thing goes for your bias vectors.

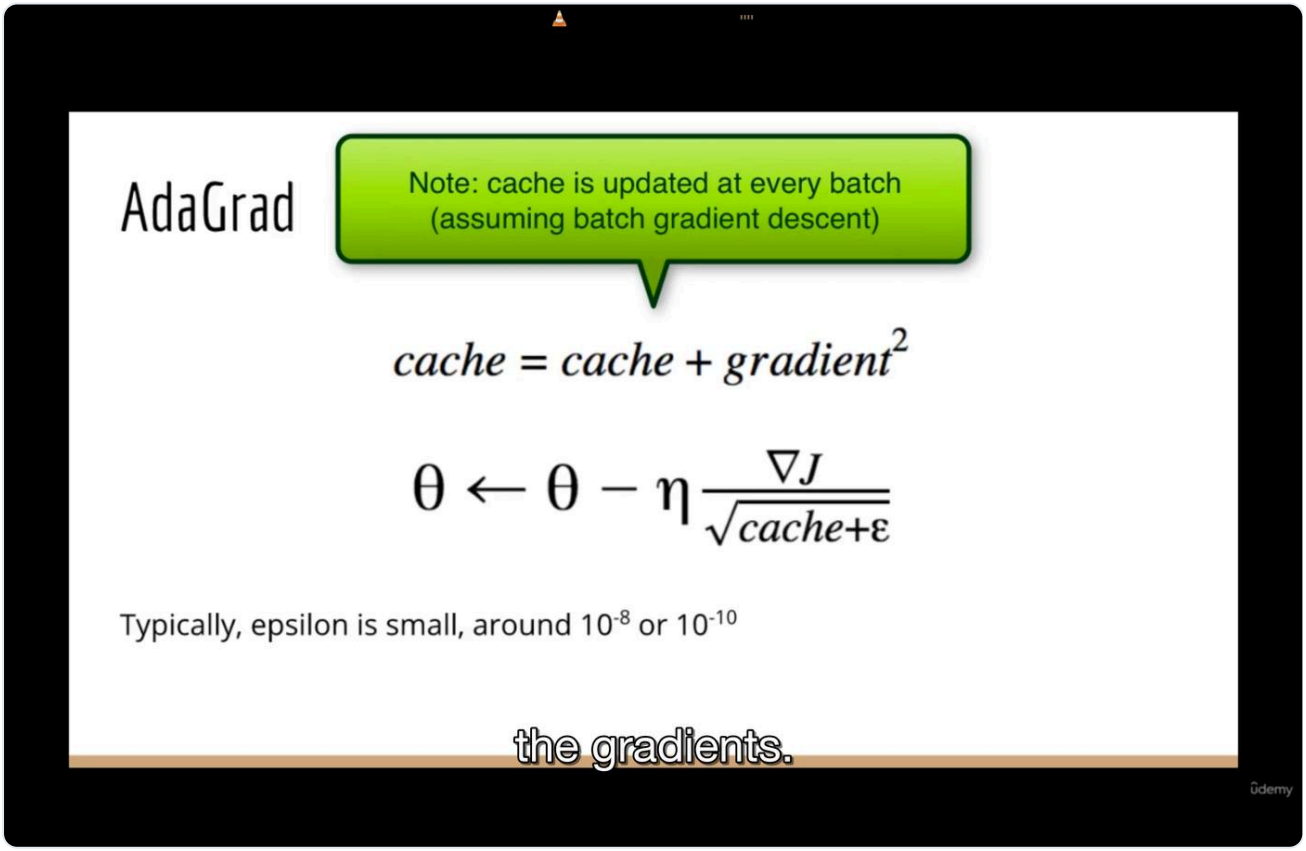

FIGURE 36.11 | Note: Cache Is Updated at Every Batch

The idea behind the cache is that it's going to accumulate the squared gradients because we're squaring the gradients. The cache is always going to be positive and because each parameter has its own cache then if one parameter has had a lot of large gradients in the past then its cache will be very large and its effective learning rate will be very small so it will change more slowly in the future.

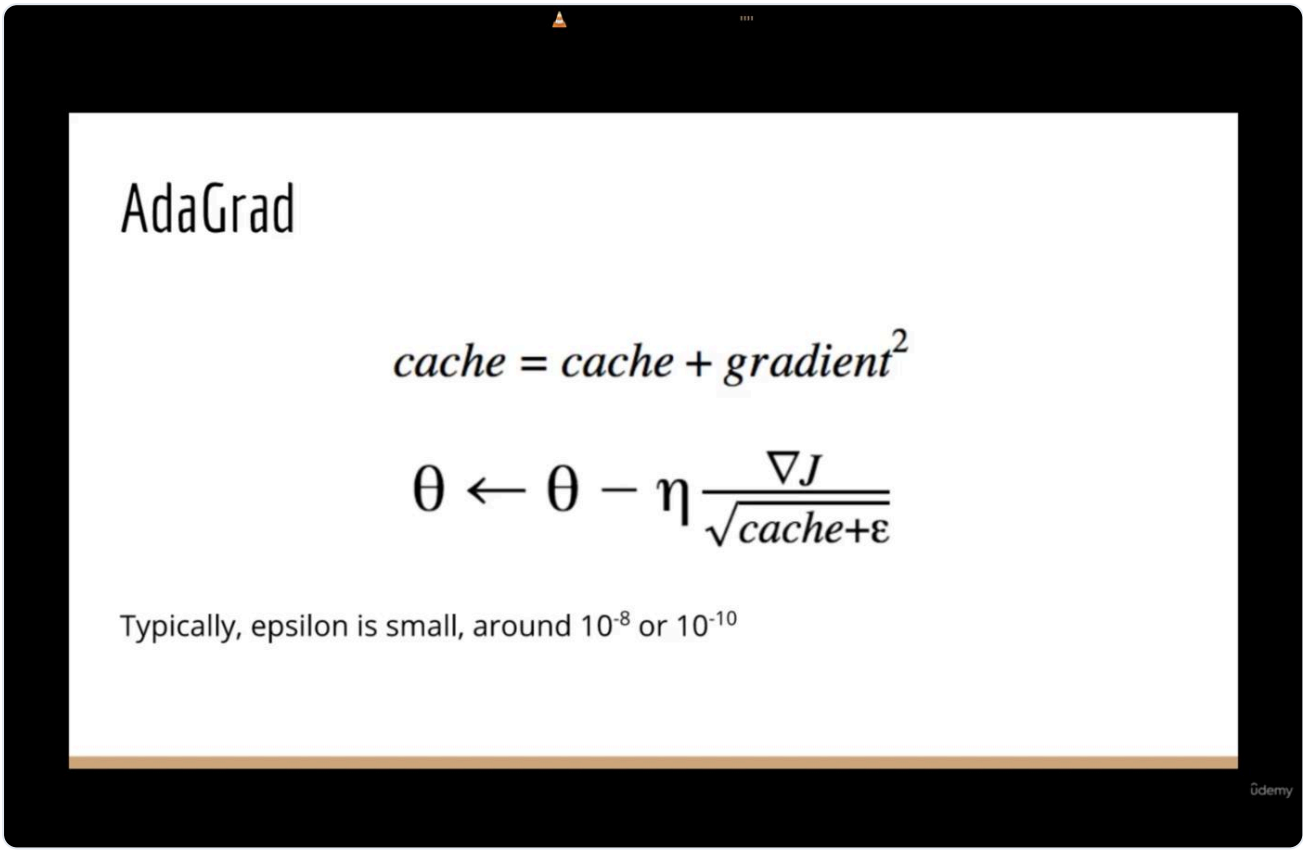

FIGURE 36.12

On the other hand if a parameter has had a lot of small gradients in the past then its cache will be small so it's effective learning rate will remain large and it will have more opportunity to change in the future one minor detail is that we usually add a small number Epsilon to the denominator to avoid dividing by zero.

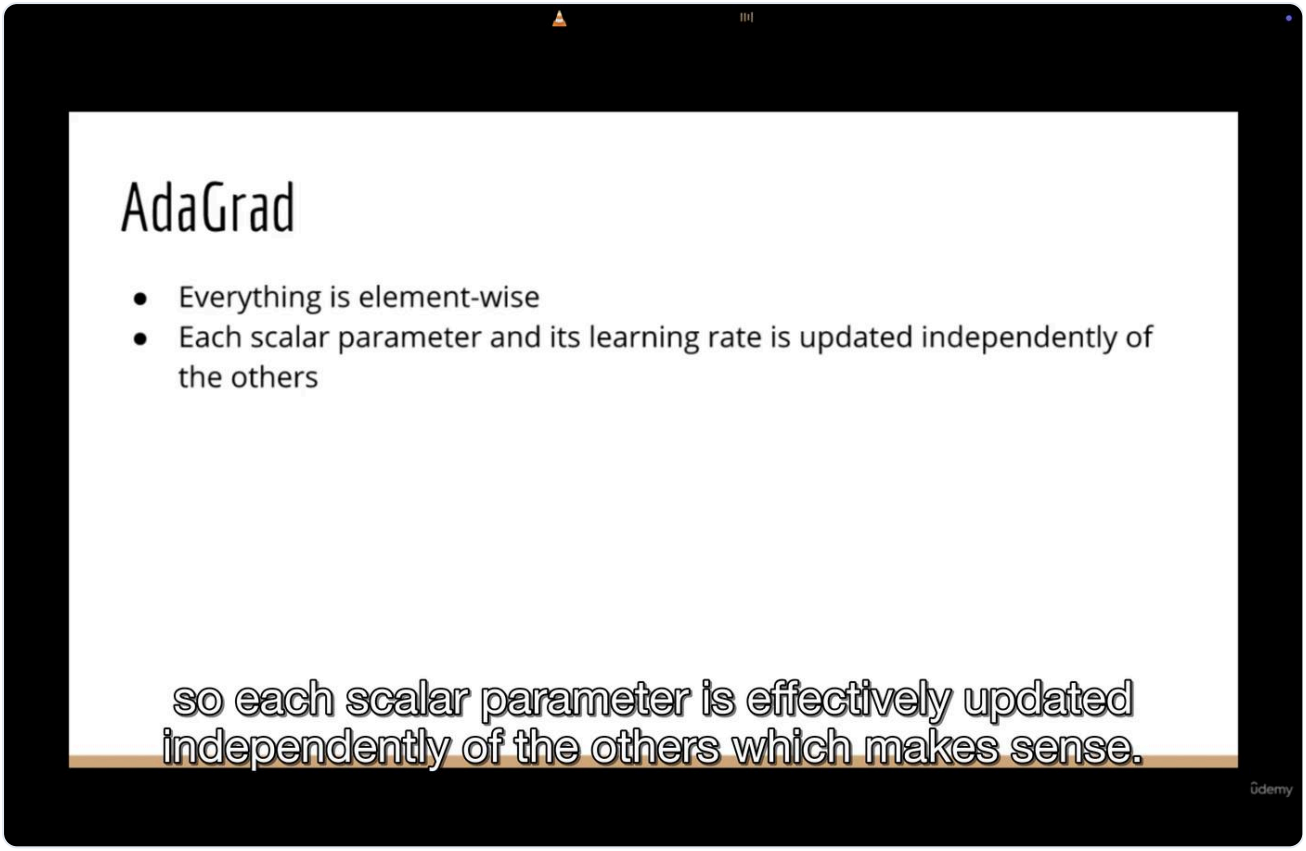

FIGURE 36.13

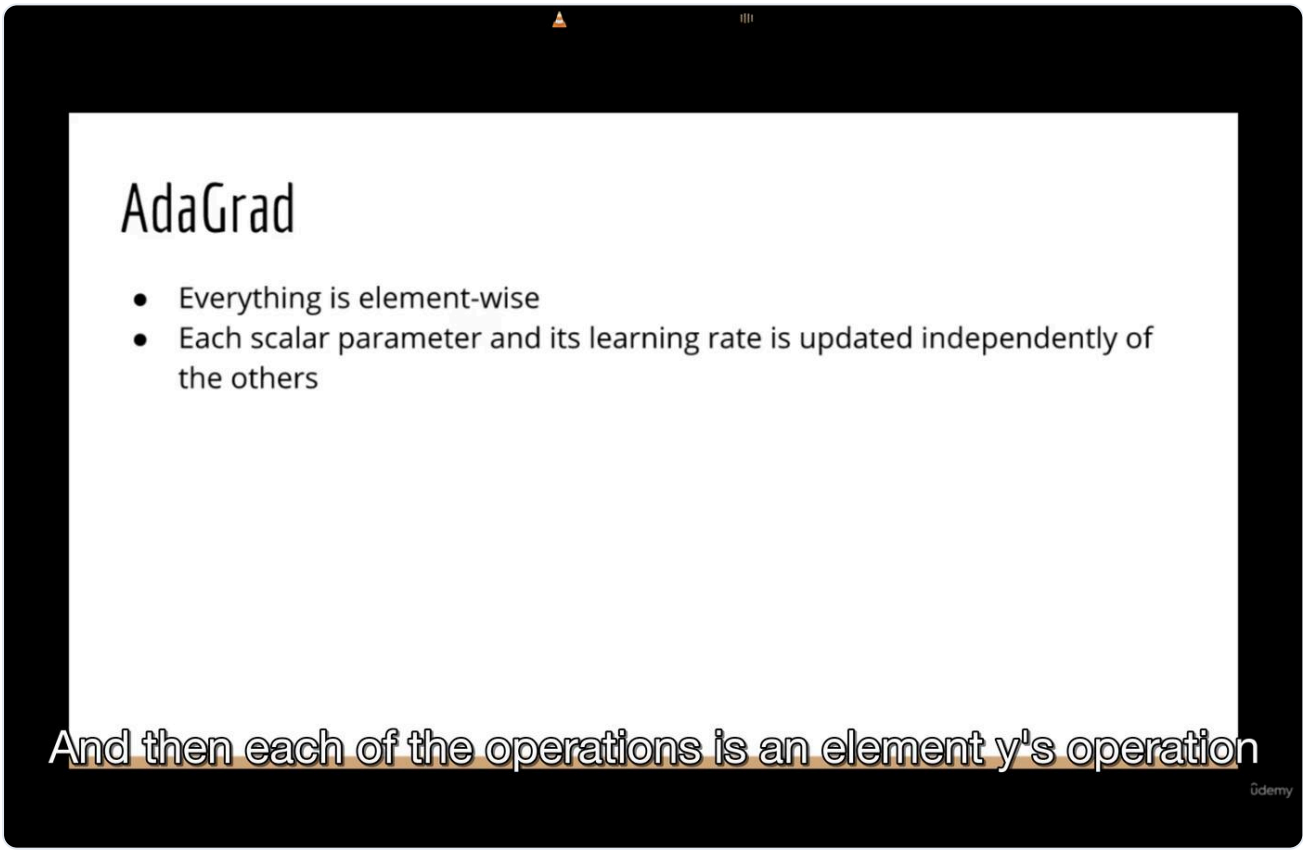

FIGURE 36.14

Typically this is set to 10 to the minus eight or 10 to the minus 10 one important point to stress about AdaGrad is that everything we're doing is an element wise operation so each scalar parameter is effectively updated independently of the others which makes sense. So you can look at the formulas we presented as scalar updates which apply to all the parameters or you can think of one huge parameter vector that contains all the neural network parameters. And then each of the operations is an element wise operation this next technique builds on the fact that researchers observed that AdaGrad decreases the learning rate too aggressively. So the learning rate would approach zero too quickly when in fact there was still more learning to be done.

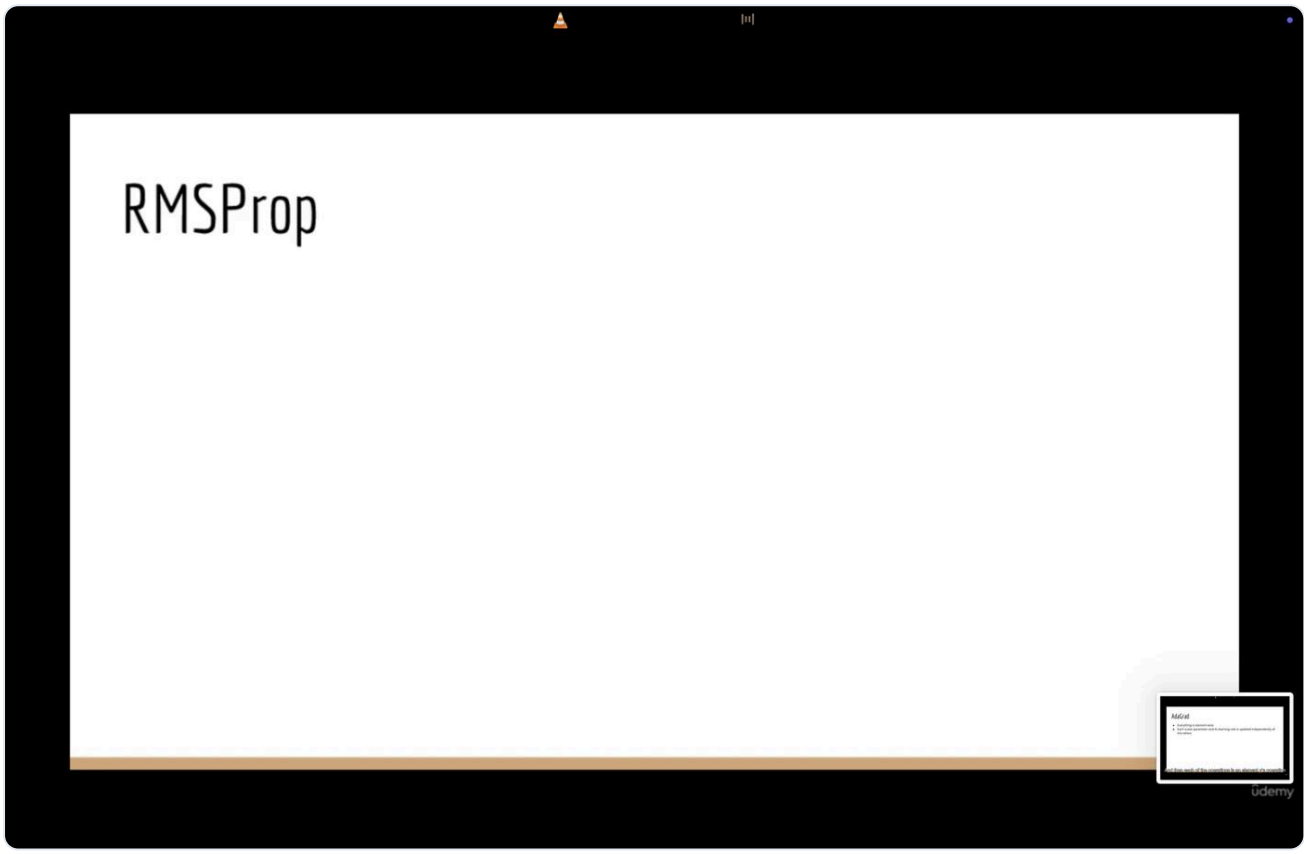

FIGURE 36.15

This technique is called RMSprop and was invented by Geoff Hinton and his team. So the way it works is this the reason AdaGrad decreases the learning rate too quickly is because the cache is growing too fast. So in order to make the cache grow less fast we actually decrease it. Each time we update it we do this by taking a weighted average of the old cache and the new squared gradient.

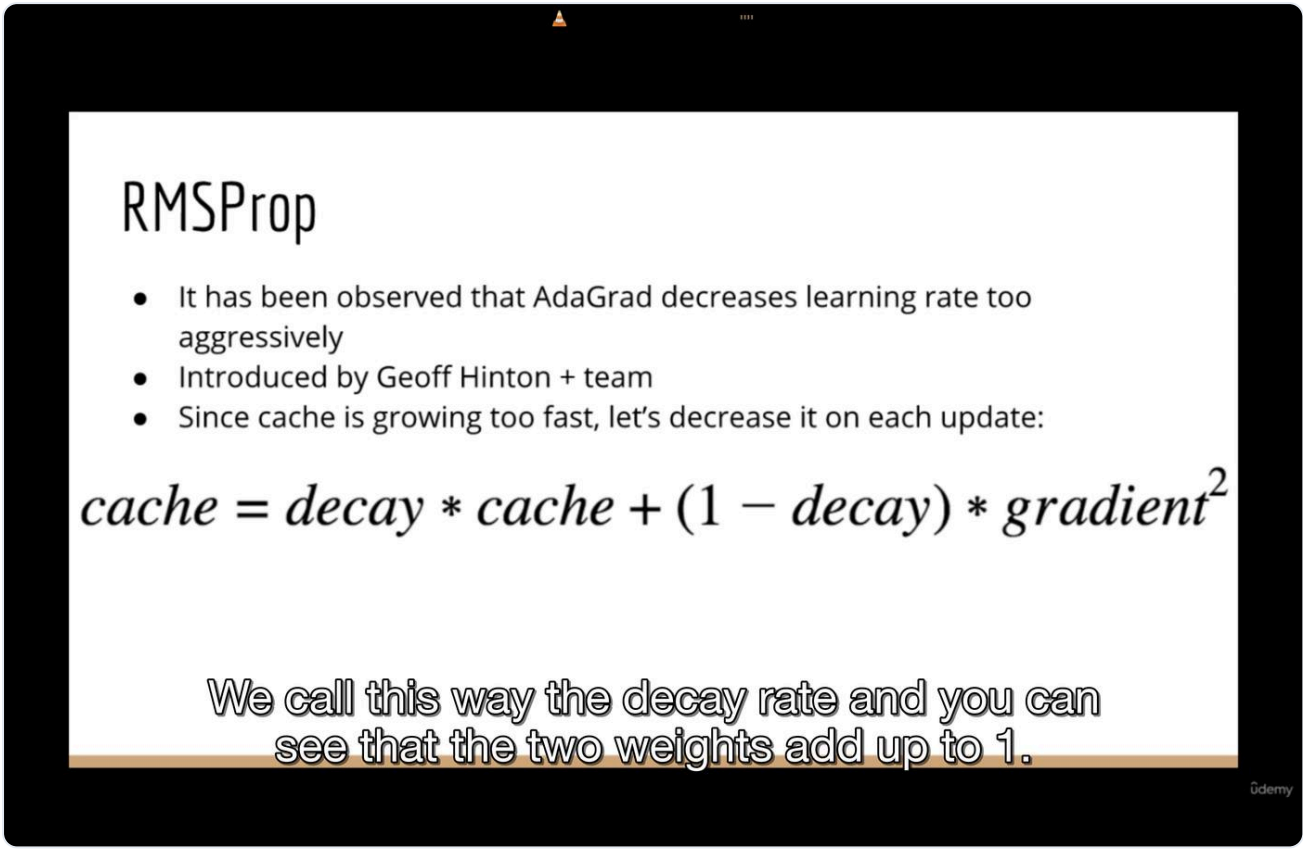

FIGURE 36.16 | Introduced by Geoff Hinton + Team

We call this weight the decay rate and you can see that the two weights add up to 1. We'll be looking at this form again and again throughout the course. So don't worry about knowing all there is to know right now. Just know that we'll be taking part of the old cache and part of the new squared gradient and adding them together to get the new cache typical values for the decay rate.

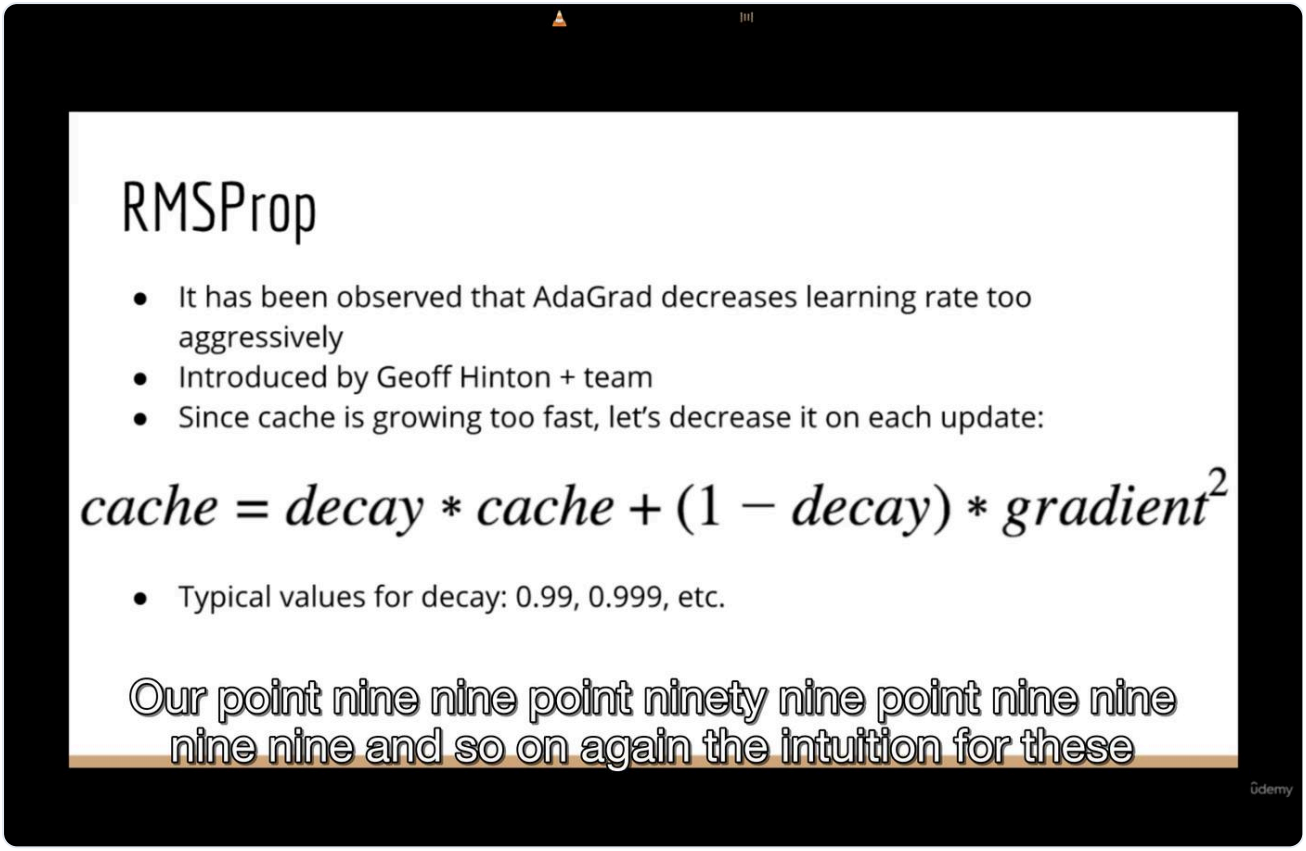

FIGURE 36.17 | Introduced by Geoff Hinton + Team

Our point nine nine point ninety nine point nine nine nine nine and so on again the intuition for these choices will be discussed in later lectures so by doing this we say that we're making the cache leaky.

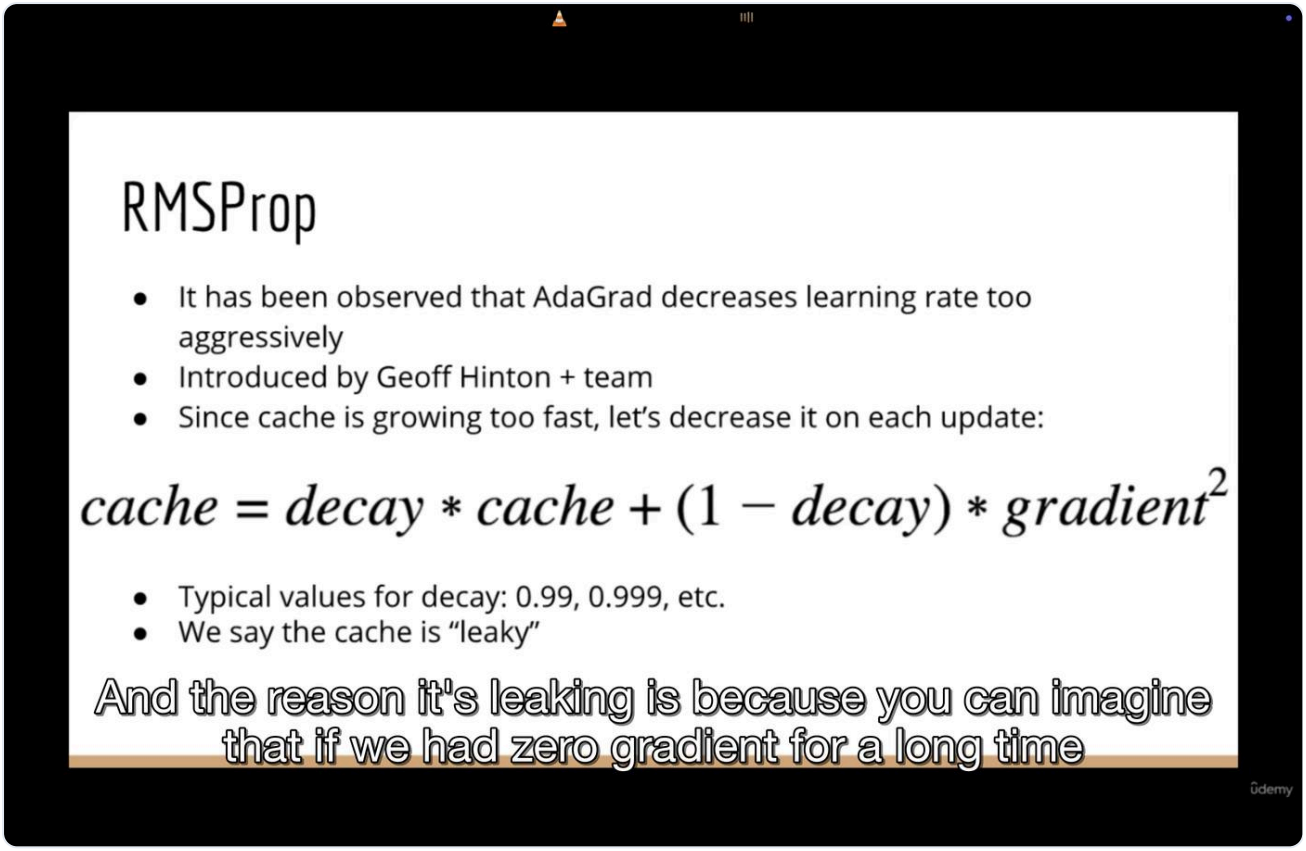

FIGURE 36.18 | Introduced by Geoff Hinton + Team

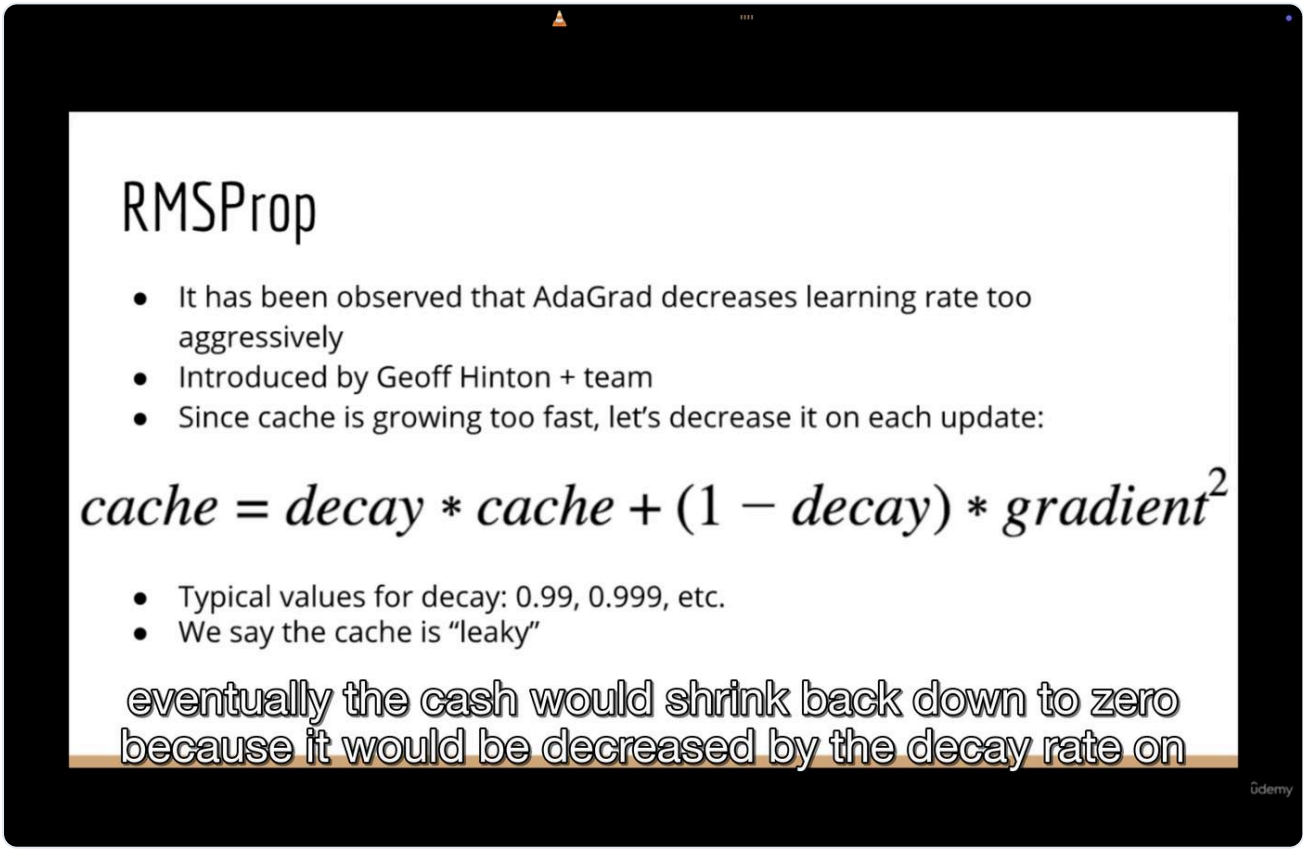

FIGURE 36.19 | Introduced by Geoff Hinton + Team

And the reason it's leaking is because you can imagine that if we had zero gradient for a long time eventually the cache would shrink back down to zero because it would be decreased by the decay rate on each round one obstacle. I realized sometime after this course was originally released is that there is actually some ambiguity in the RMSprop and AdaGrad algorithms. Specifically I discovered this in the context of RMSprop so one thing that's not specified in the RMSprop algorithm is the initial value of the cache. Now one might automatically just assume that zero is a good value. And I did this as well but this actually has a very strange effect.

Let's suppose you choose the decay rate of zero point nine nine nine. Then your initial update for the cache will be zero point zero zero one times G squared.

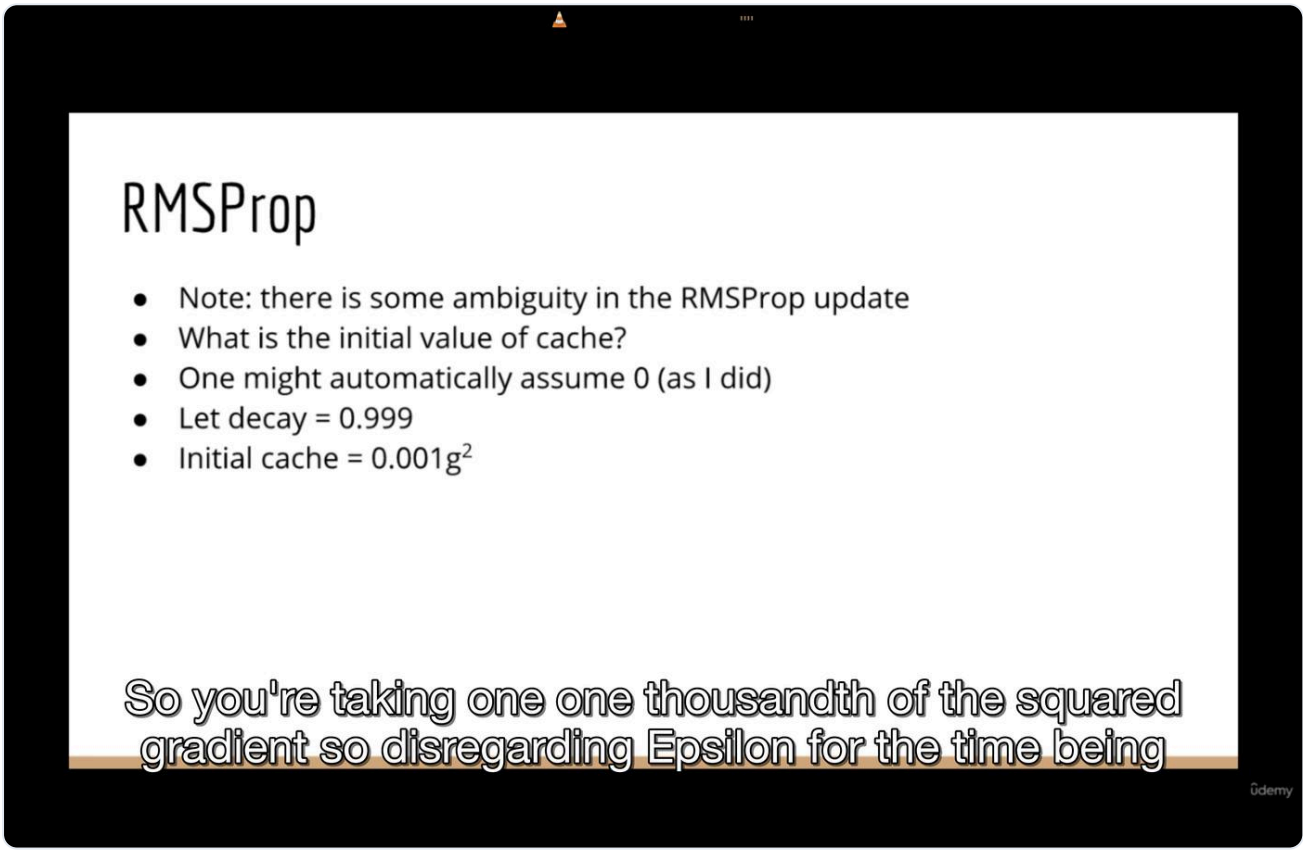

FIGURE 36.20 | Note

So you're taking one one thousandth of the squared gradient so disregarding Epsilon for the time being your effective learning rate actually becomes very large due to the fact that the denominator is very small. What you'd have to do is compensate by making your initial learning rate smaller than usual one solution to this is to set the cache to one instead.

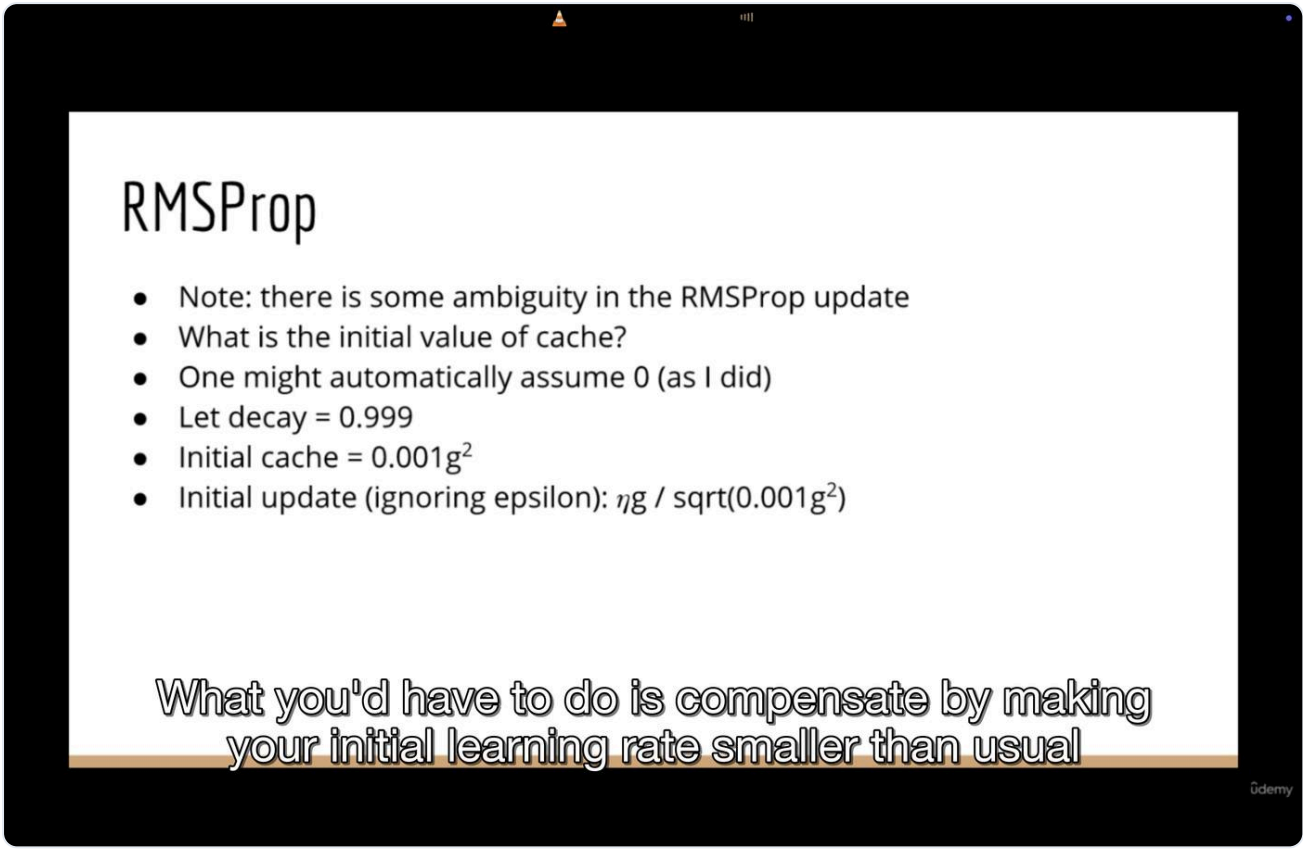

FIGURE 36.21 | Note

By manipulating the RMSprop update a little bit we can show that this gives us approximately what the update would be if we weren't doing RMSprop at all. Or in other words the effective learning rate is approximately equal to the initial learning rate divided by 1.

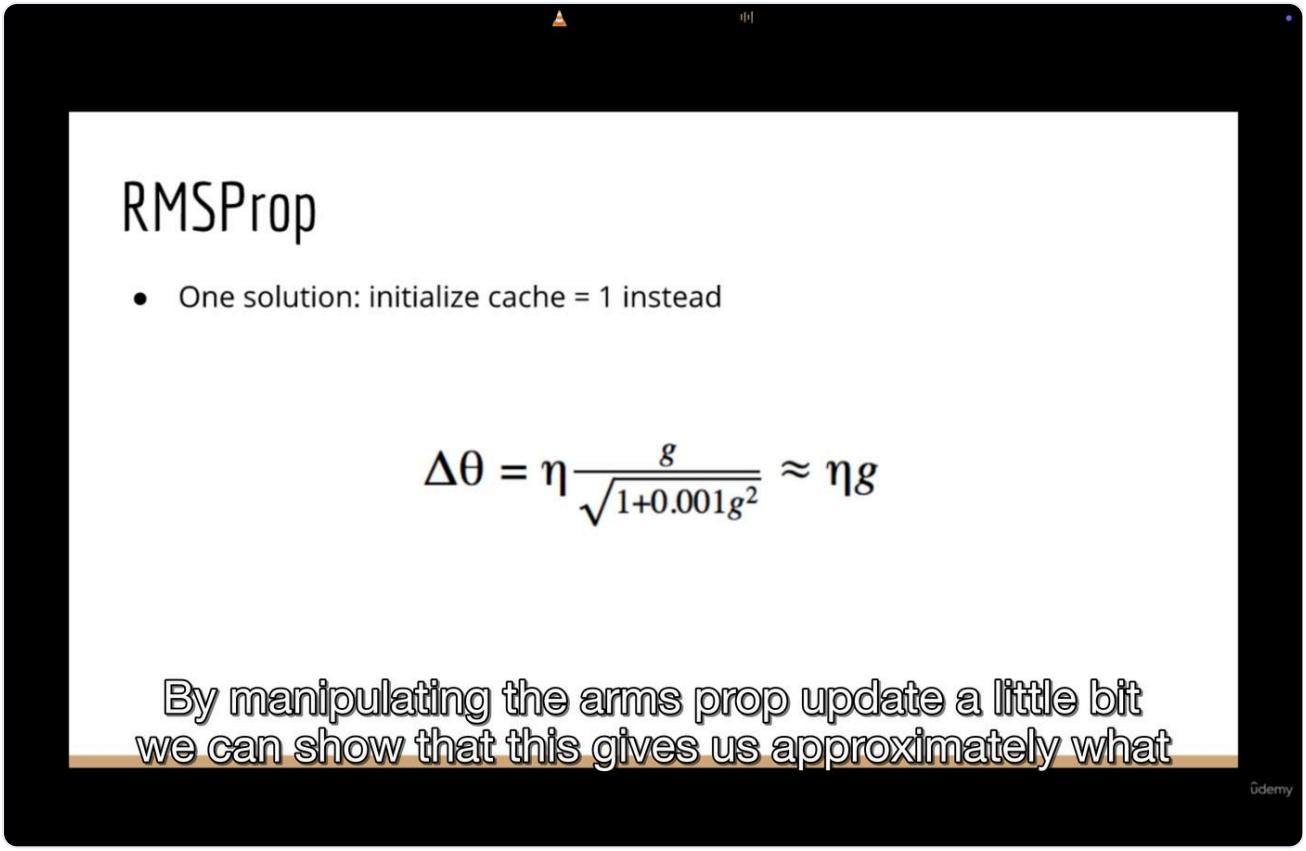

FIGURE 36.22

Now you might ask Well which one is the correct way. And we can't really say because it was never specified what I can tell you is that there are some very major packages that have implemented them both ways.

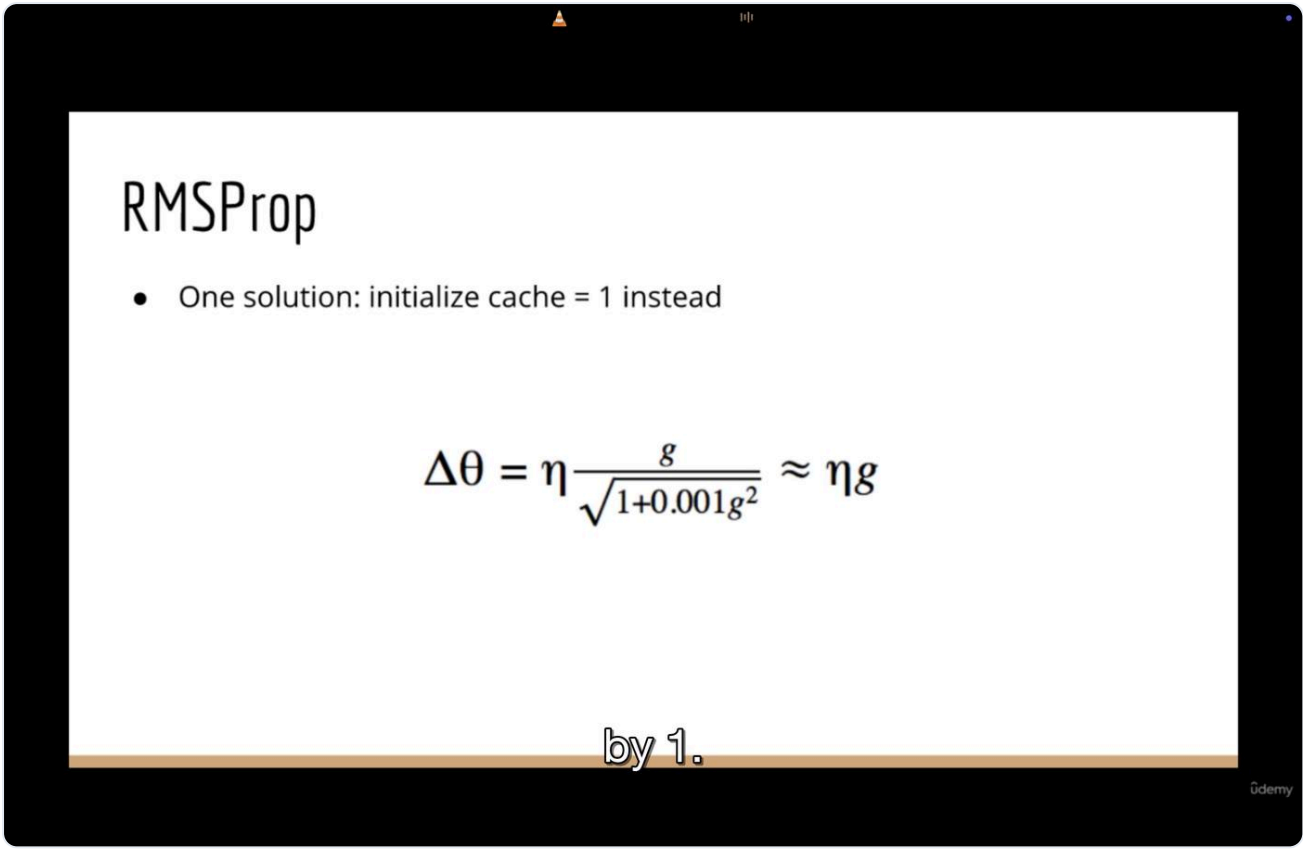

FIGURE 36.23

So it's not as if one is more correct than the other. For example in TensorFlow the cache is initialized to 1 in Keras the cache is initialized to 0.

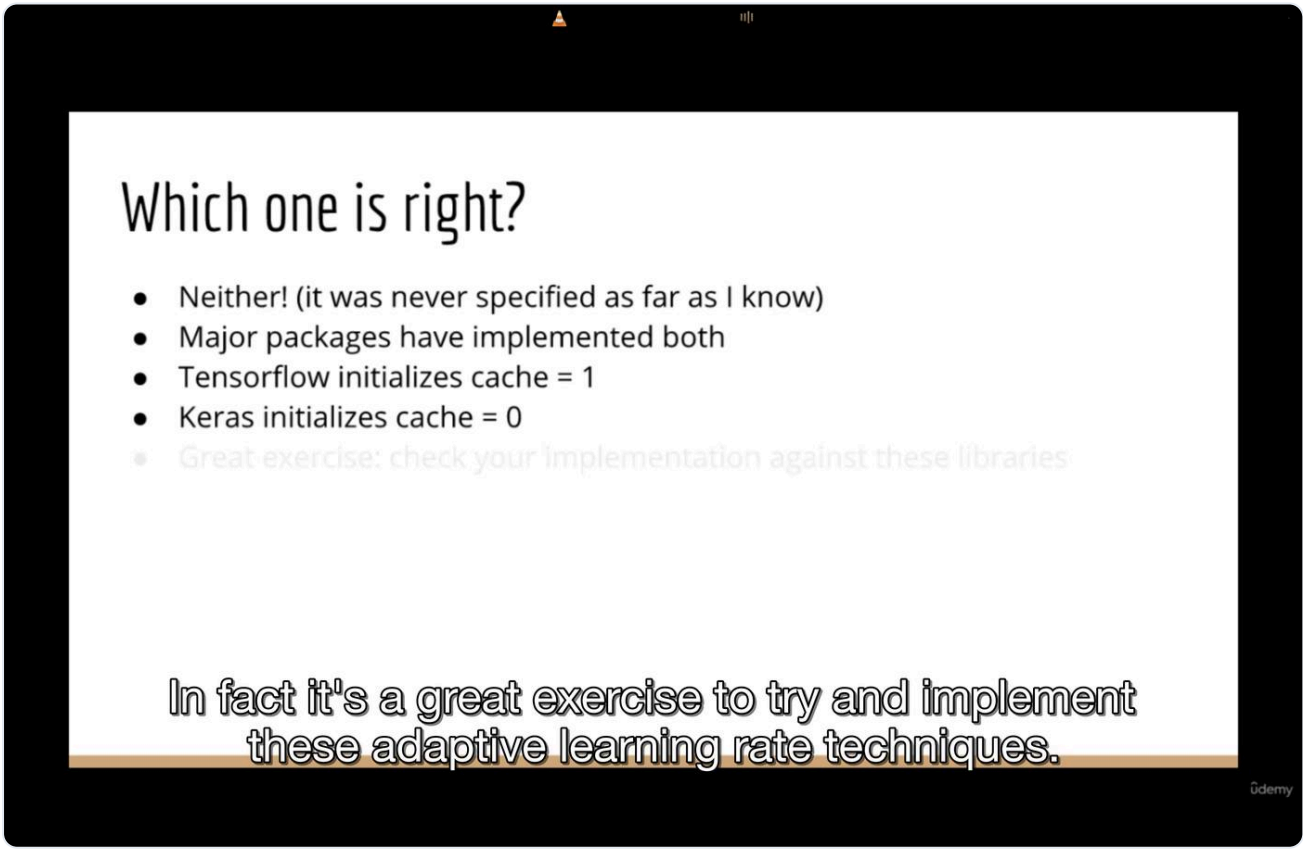

FIGURE 36.24 | Which One Is Right?

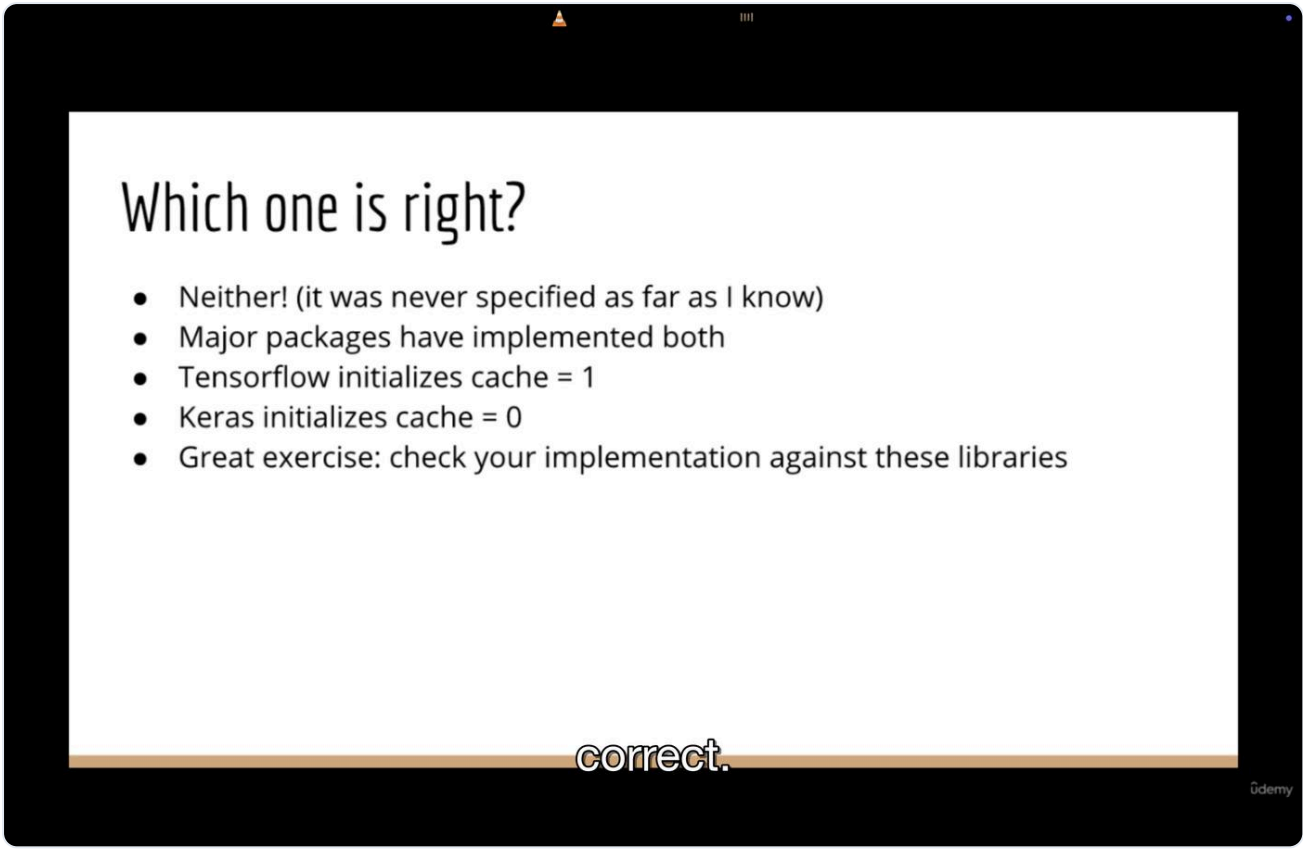

FIGURE 36.25 | Which One Is Right?

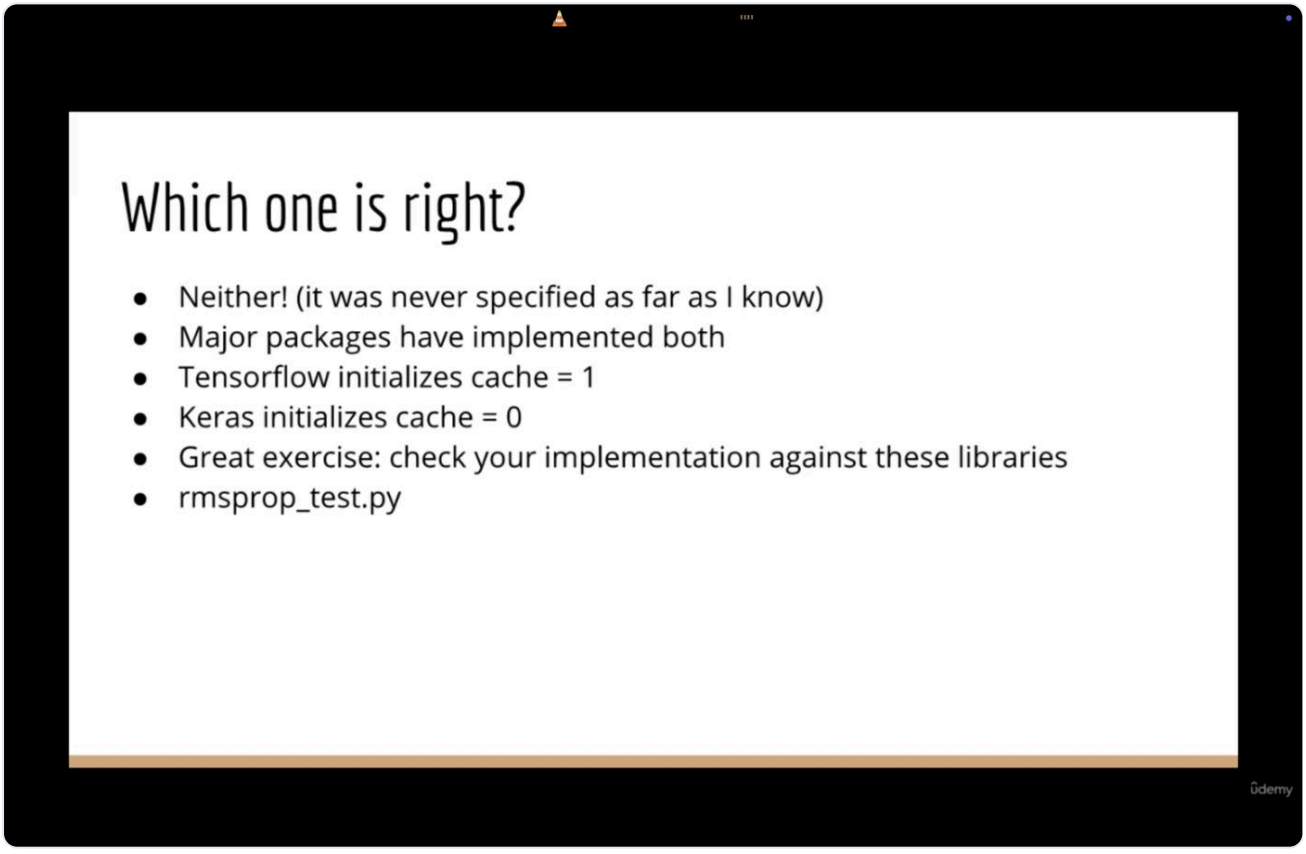

FIGURE 36.26 | Which One Is Right?

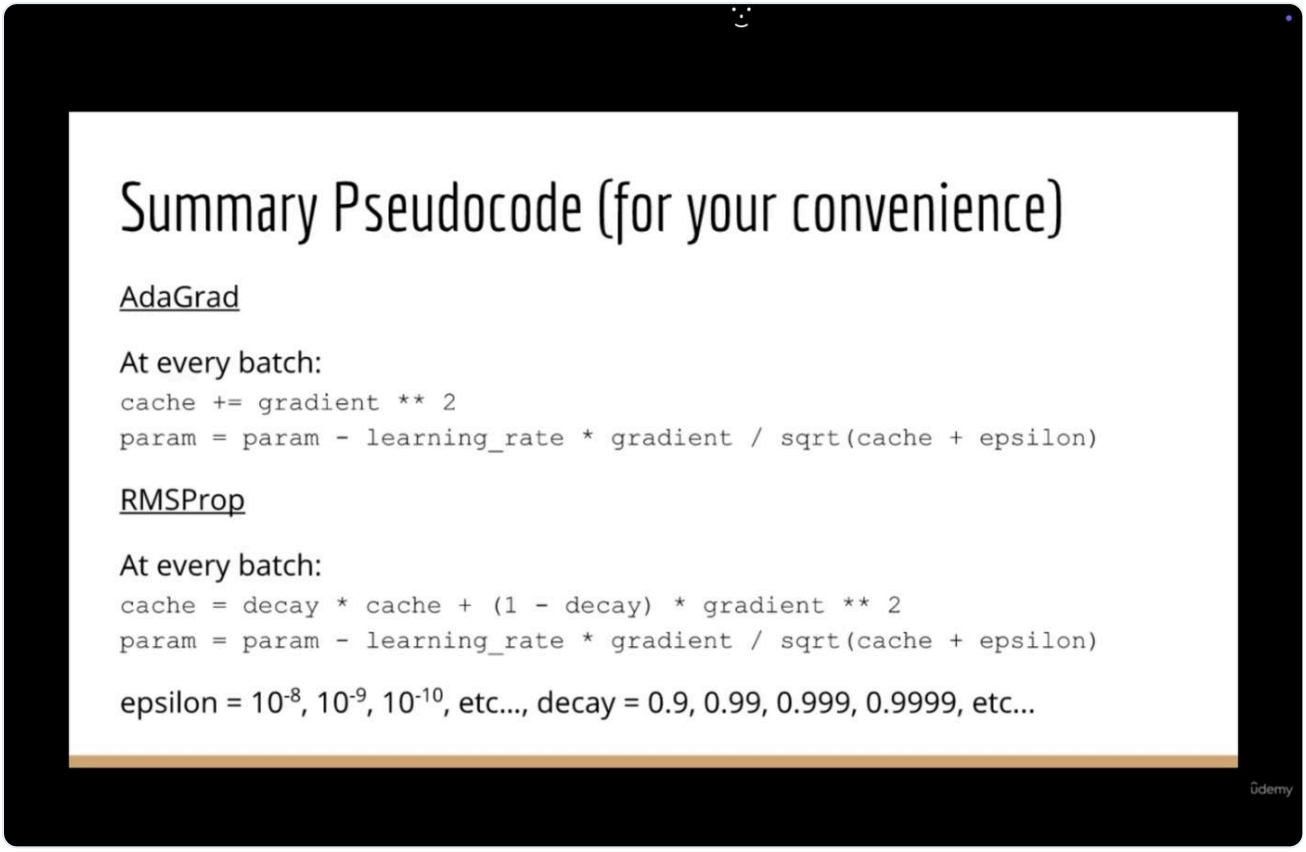

FIGURE 36.27

In fact it's a great exercise to try and implement these adaptive learning rate techniques. Then compare your implementation to these official implementations to check if your implementation is correct. If it is correct then your loss per iteration should match what the library gives you. And so I've done this in the file rmsprop_test.py in case you want to check it out.

LECTURE 130

Review So Far

In this lecture, we'll be looking at another way to modify gradient descent using a technique called adaptive moment estimation or Adam for short.

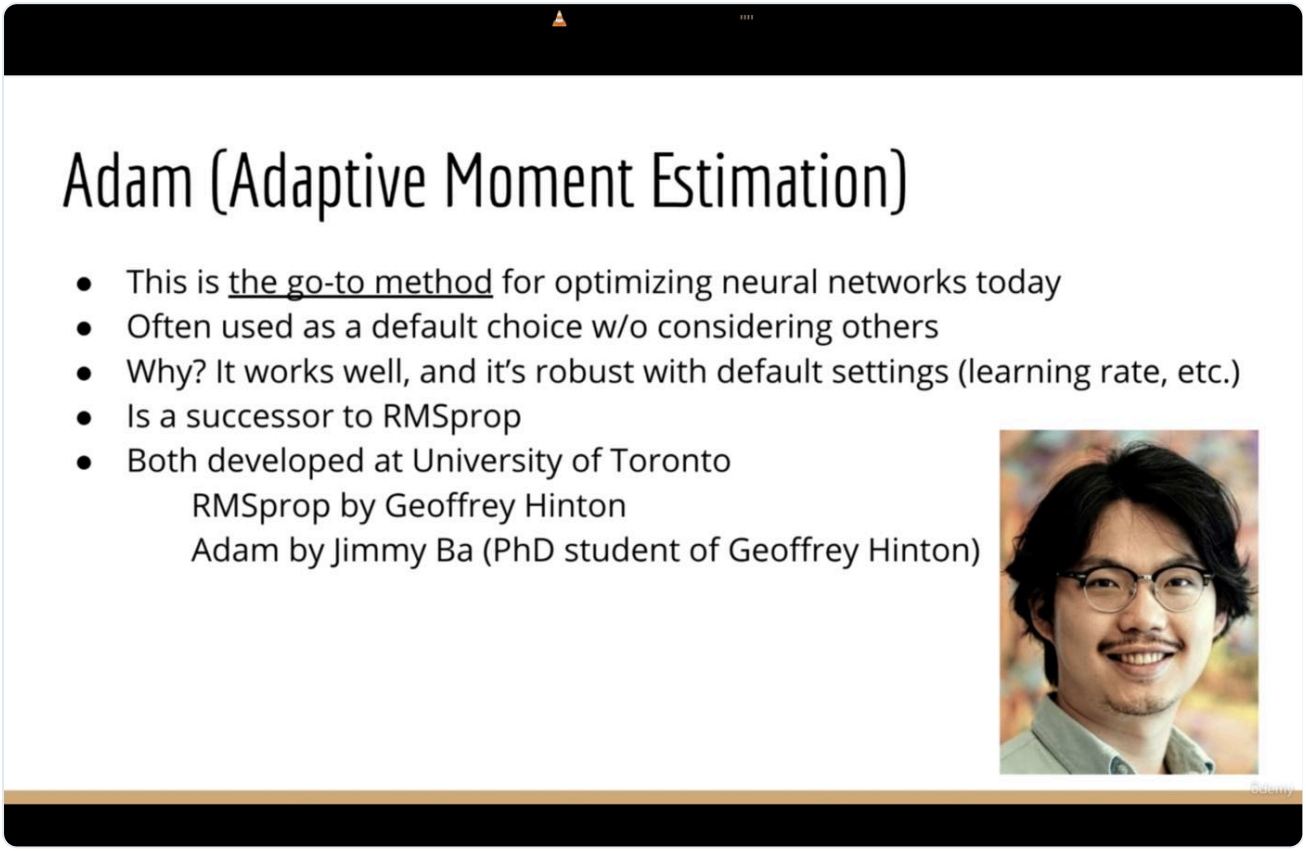

FIGURE 37.1 | Adam (Adaptive Moment Estimation)

Adam is a technique that deserves its own lecture because it is the go to method for optimizing neural networks today. That is, people most often use Adam as the default choice without really considering other options. The reason for this is simple. Adam tends to work well and it tends to work well with the default settings for the learning rate, momentum parameter and so on. In other words, Adam is robust. You can use it and not have to worry about tweaking the knobs to find values which are exactly right, since what you have by default is probably OK. In addition, Adam is somewhat of a successor to RMSprop. Both were developed at the University of Toronto RMSprop was developed by Geoffrey Hinton while Adam was developed by Jimmy Ba, who is a PhD student of Hinton's at the time.

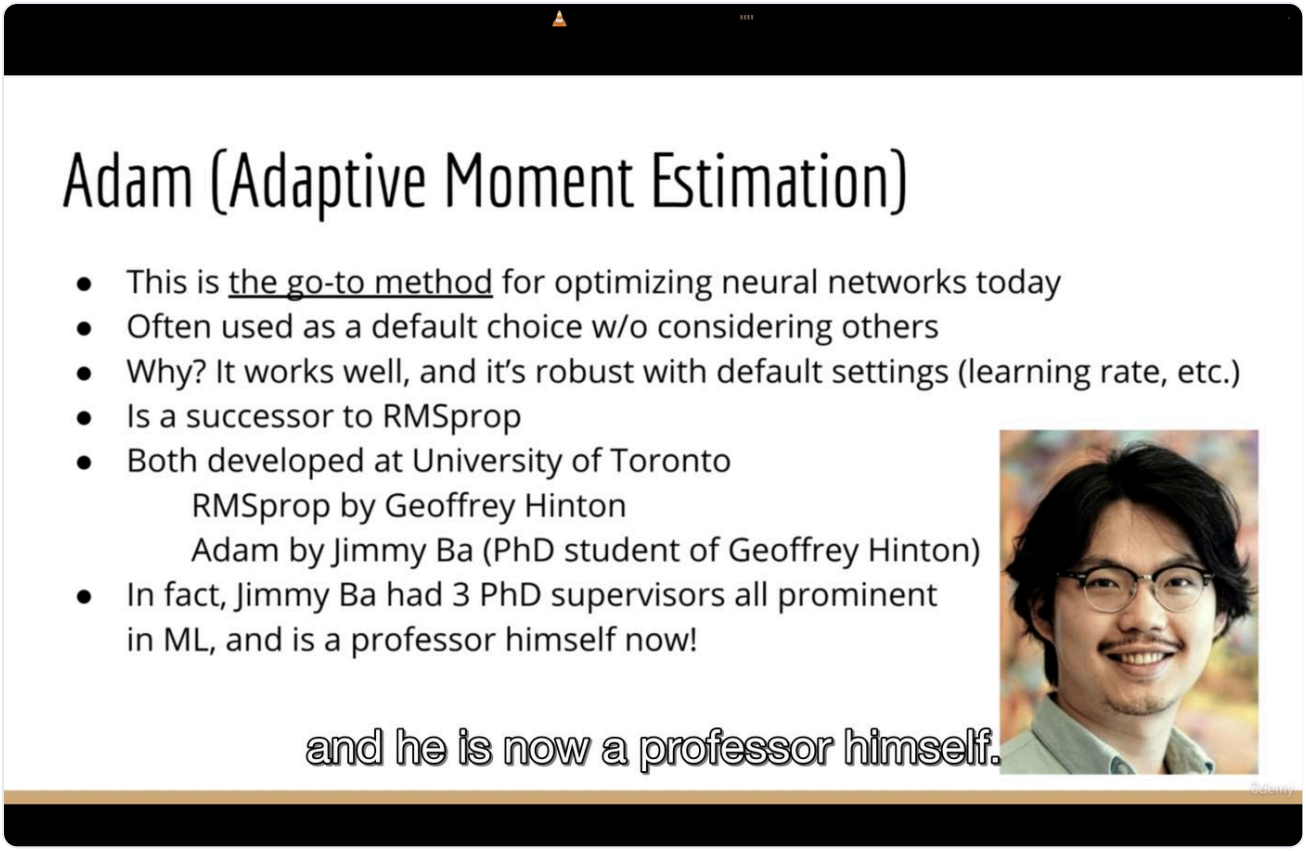

FIGURE 37.2 | Adam (Adaptive Moment Estimation)

In fact, Jimmy Ba had three PhD supervisors, all of whom are prominent machine learning researchers, and he is now a professor himself. Now, one way to think of Adam is that it's RMSprop with momentum. This is because it takes the same idea of having an adaptive learning rate, but it also adds momentum.

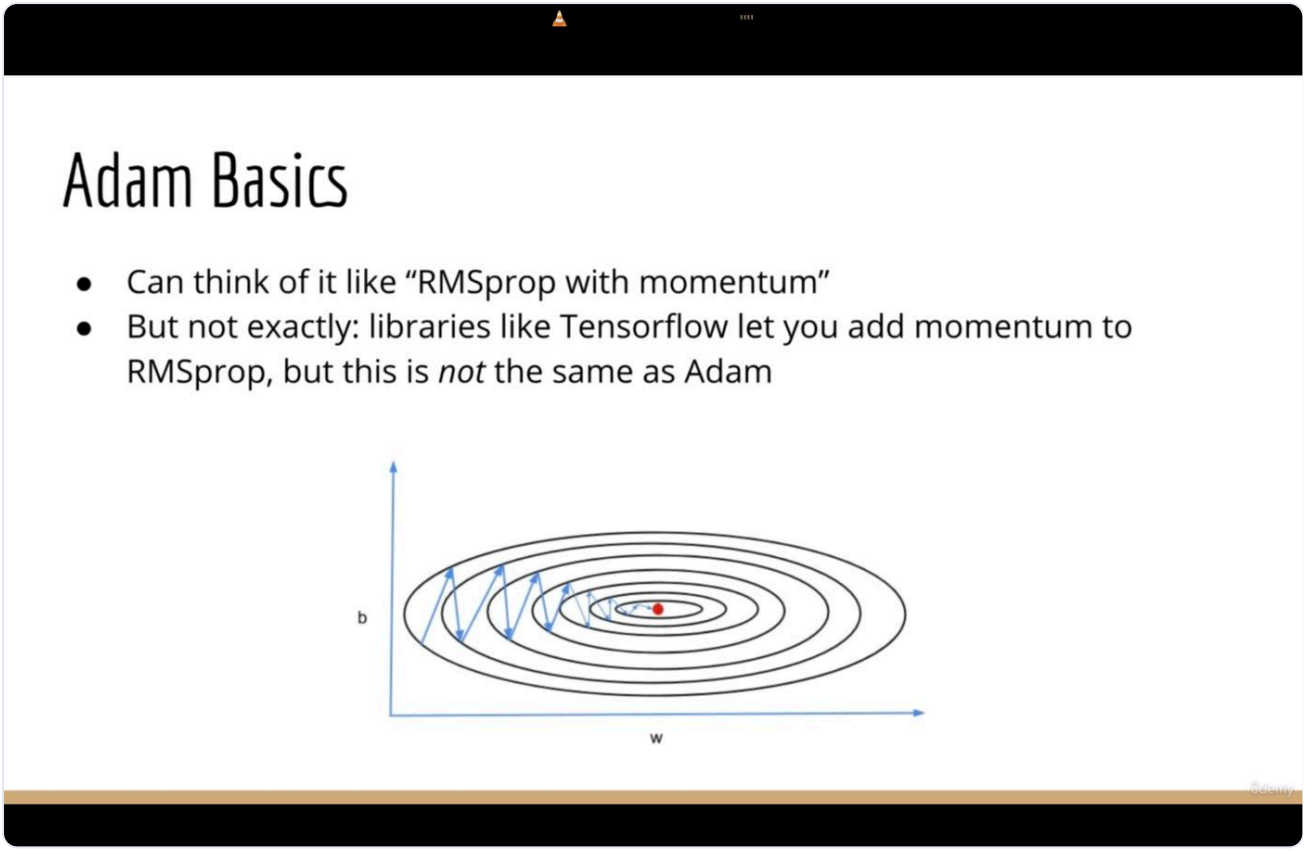

FIGURE 37.3 | RMSprop, but This Is Not the Same as Adam

This might be a little confusing since libraries like TensorFlow also provide you with an option to add momentum to RMSprop. However, these are slightly different. So the key point is Adam is like RMSprop with momentum, but it's not exactly the same. Let's start by reviewing what we have seen so far.

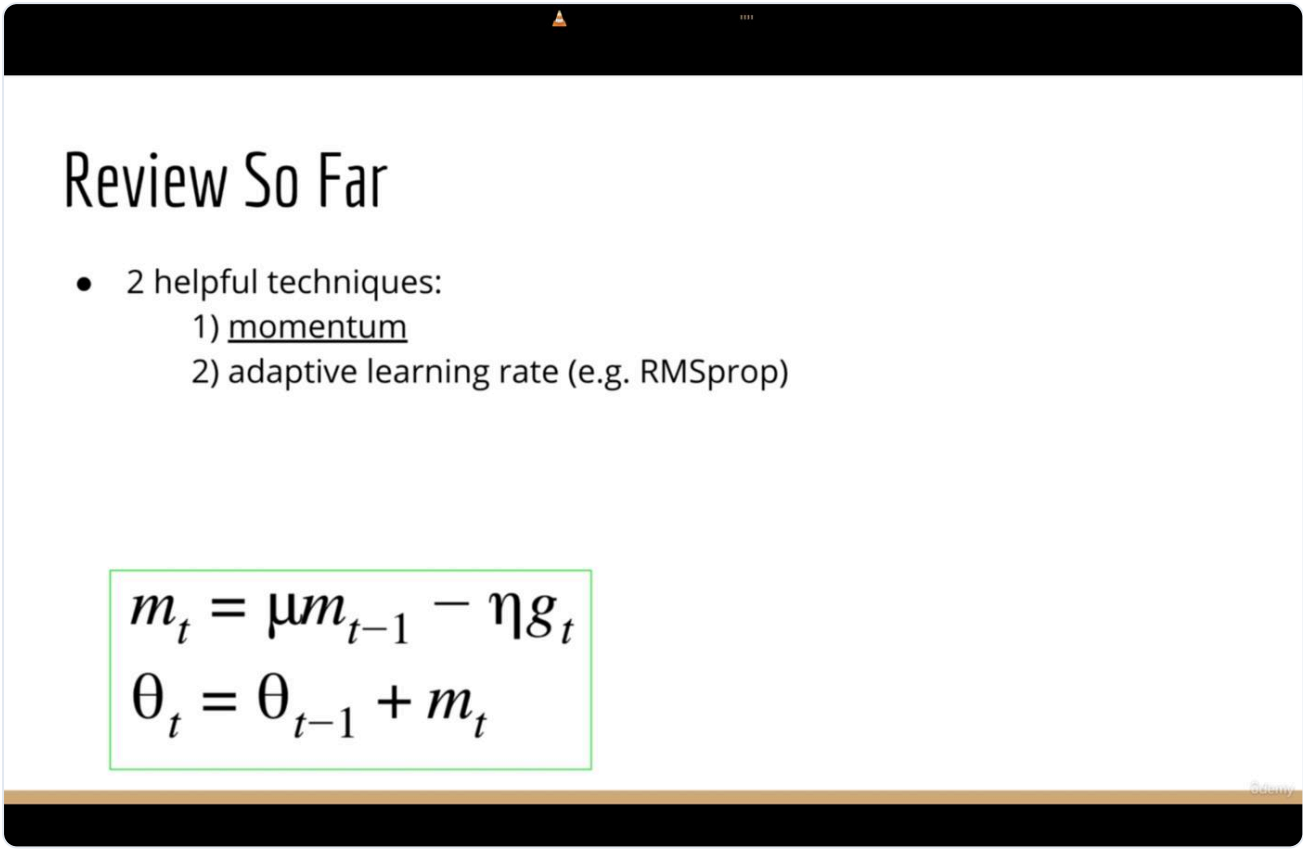

FIGURE 37.4 | Review So Far

Specifically, let's consider two helpful ways that we can improve vanilla gradient descent method. Number one is to use momentum. An object in motion stays in motion. Method number two is to use adaptive learning rate methods like RMSprop. Note that I'm using slightly different symbols than what you are probably used to.

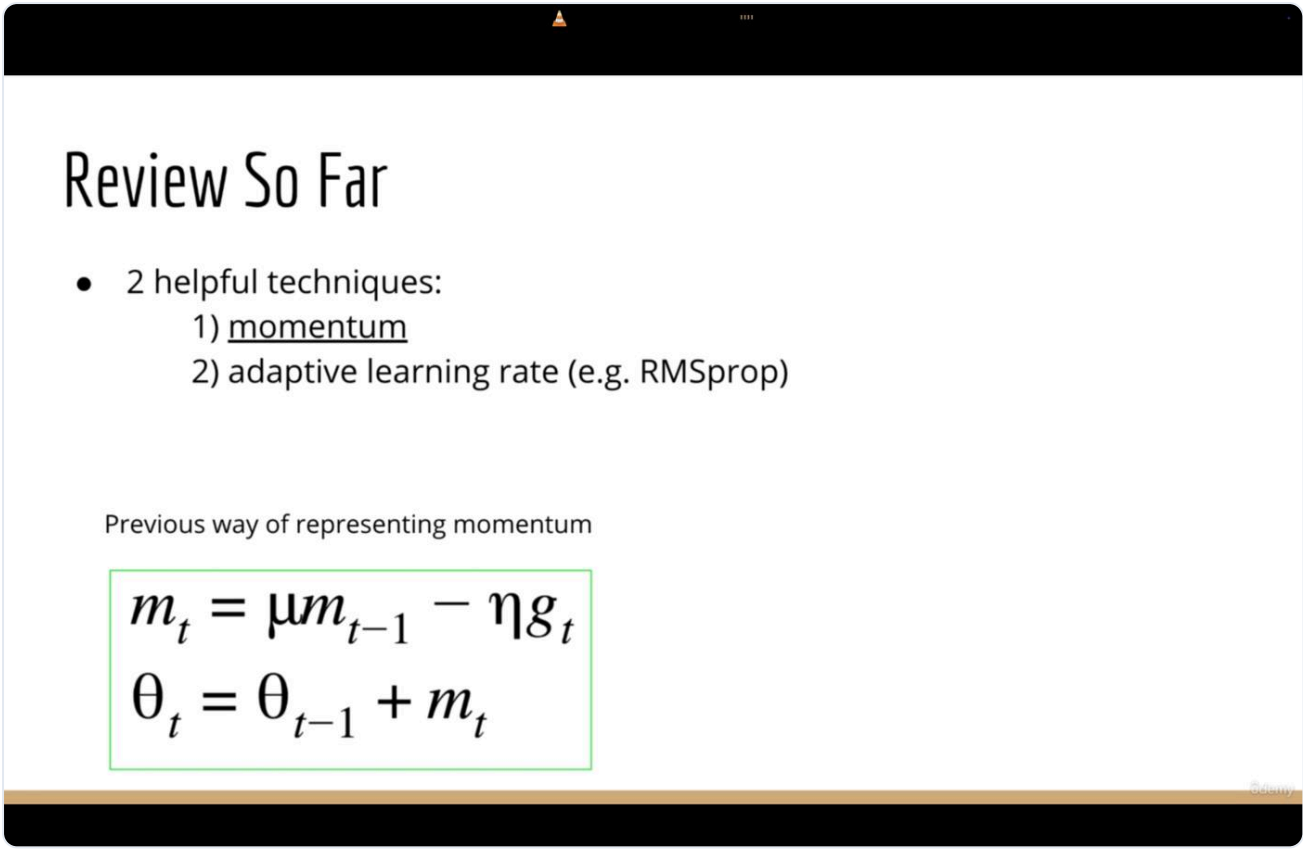

FIGURE 37.5 | Review So Far

This is just to set us up to learn Adam, which uses different symbols by convention. The parameter update for momentum proceeds in two steps. First, we update the momentum value, which I'm going to call em in this lecture. It's equal to some proportion of the previous momentum, minus the learning rate times the current gradient. Then we update the parameters themselves, which we call theta.

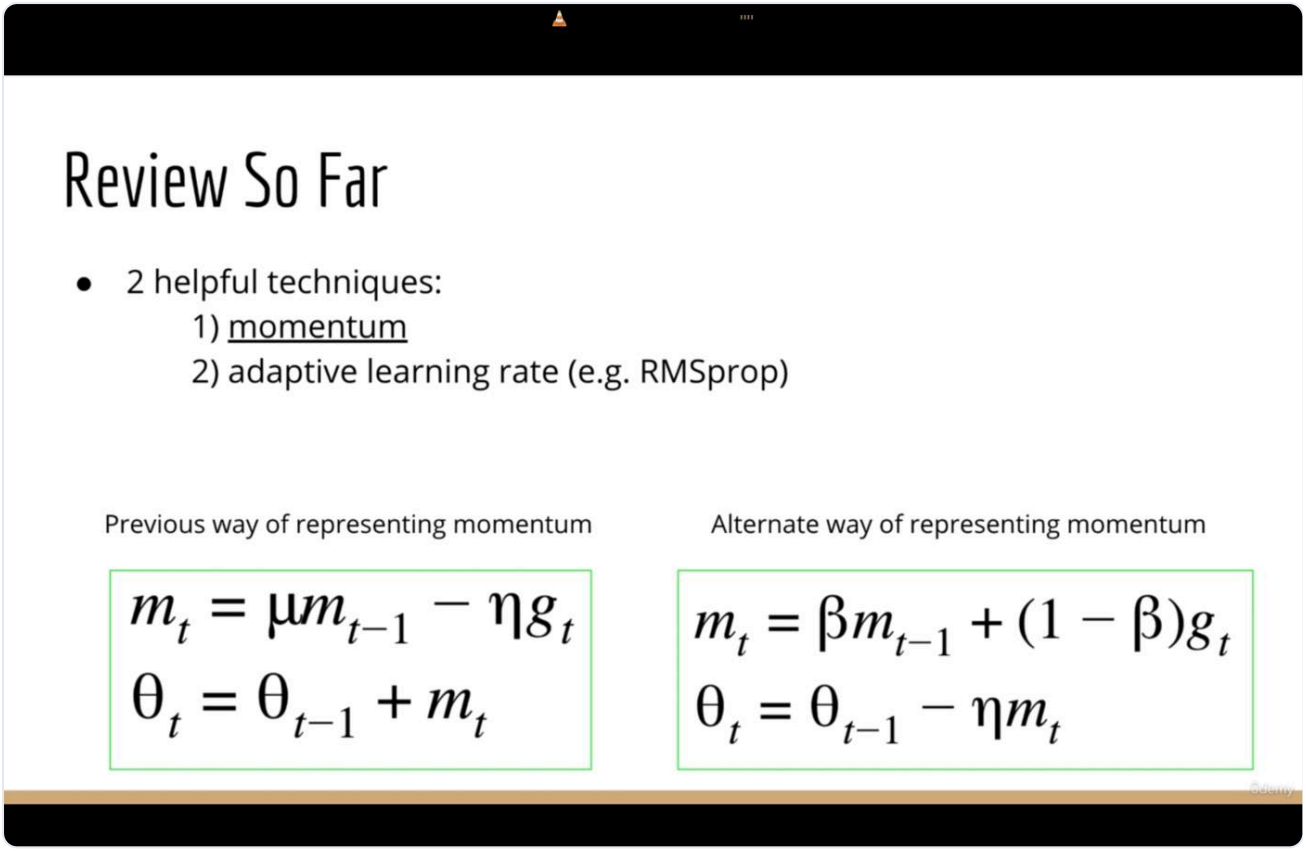

FIGURE 37.6 | Review So Far

The update is just the old value of theta, plus the new momentum. For this lecture, it's helpful to present momentum in a slightly different way. Specifically, we now treat the momentum value as kind of the negative of what it was before. The update for M of T is now the weighted sum of the previous M of T minus one and the current gradient G of T to update theta.

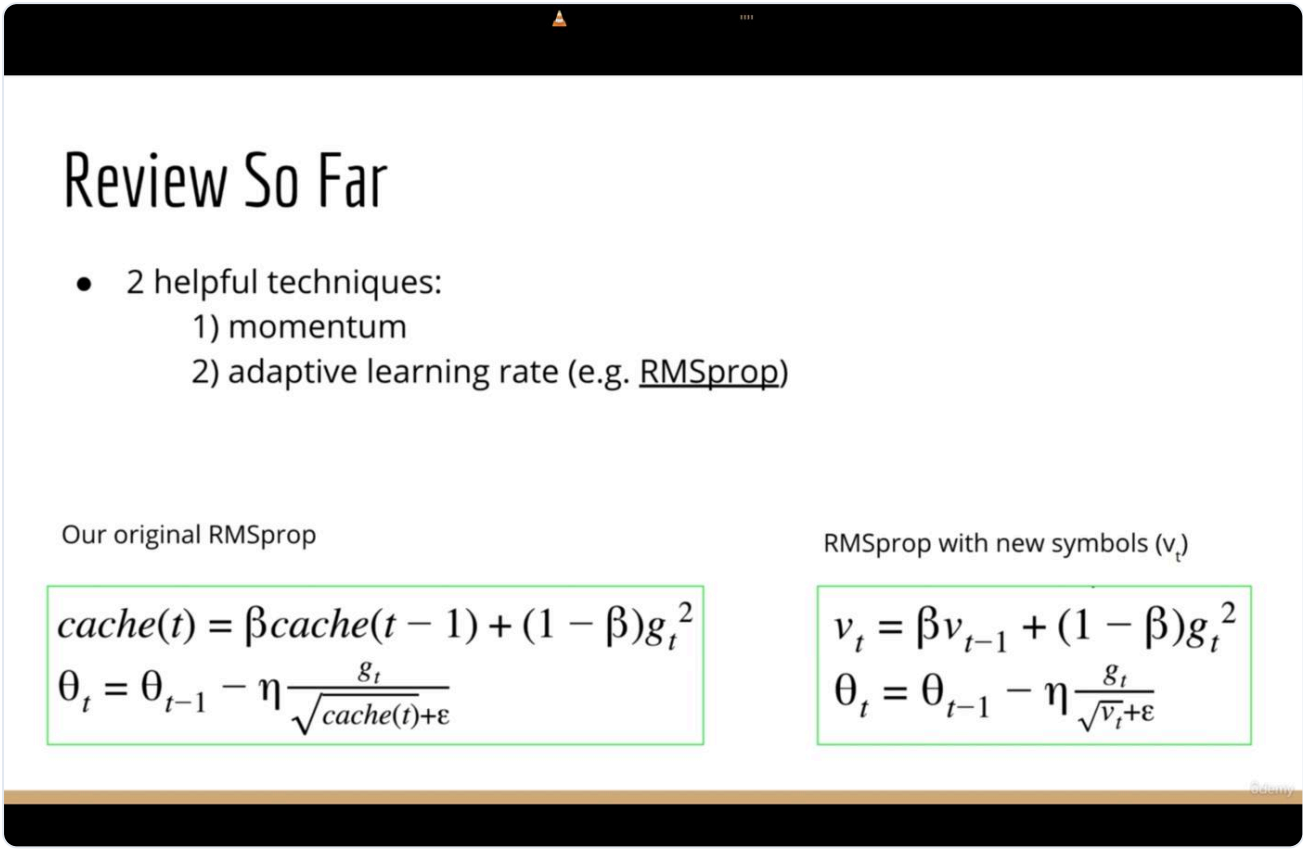

FIGURE 37.7 | Review So Far

We now subtract M of T times the learning rate eta. Now, since all these new symbols are just constants, you can show that the two methods of presenting momentum are equivalent. Let's also review RMSprop. RMSprop also proceeds in two steps.

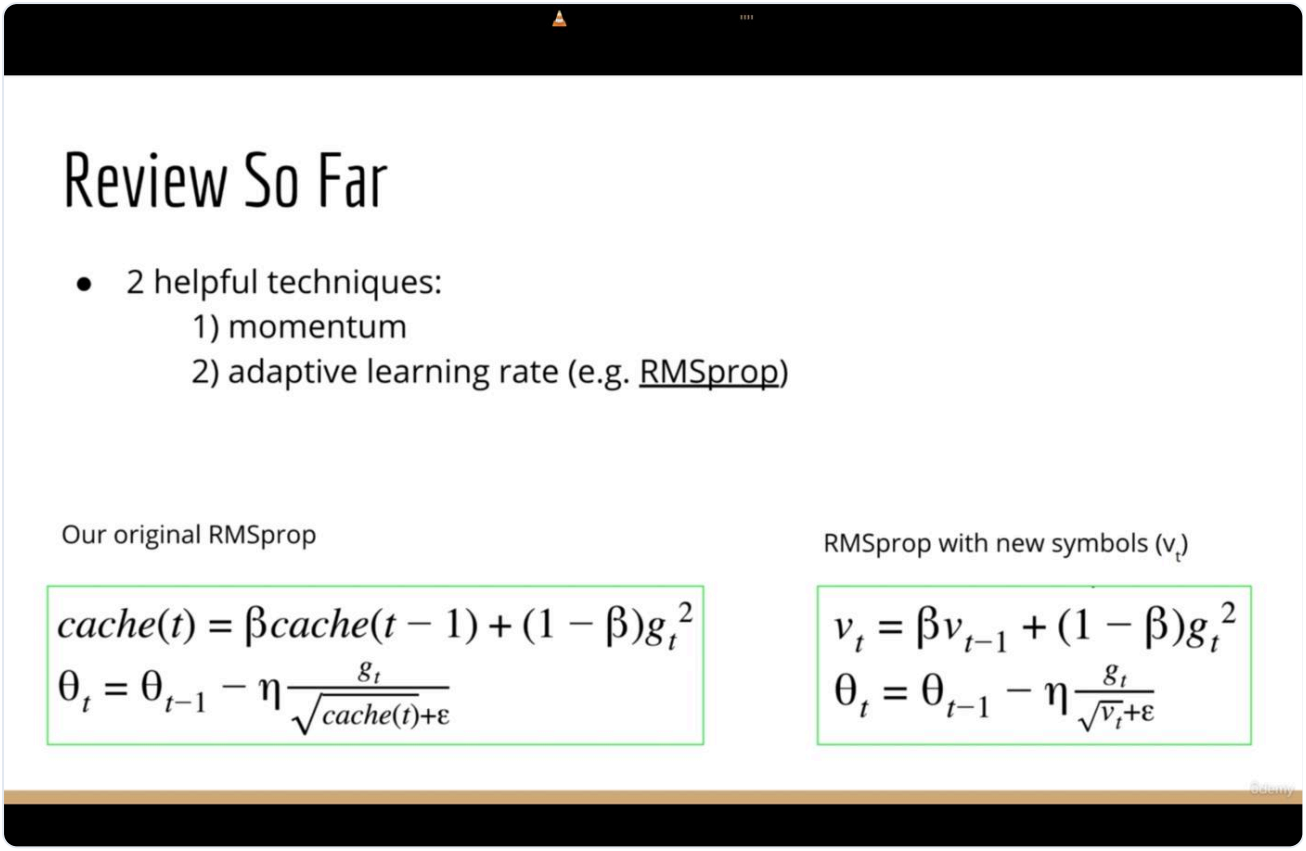

FIGURE 37.8 | Review So Far

First, we update our cache, which is now renamed V. Notice how similar this is to our modified version of momentum. But instead of being the weighted sum of the gradients, it's the weighted sum of the gradient squared. The second step is to update the parameter theta as usual, except instead of having the same learning rate for all time, we divide by the square root of the cache. OK, so for the next phase of this lecture, we're going to take a small digression and talk about moving averages.

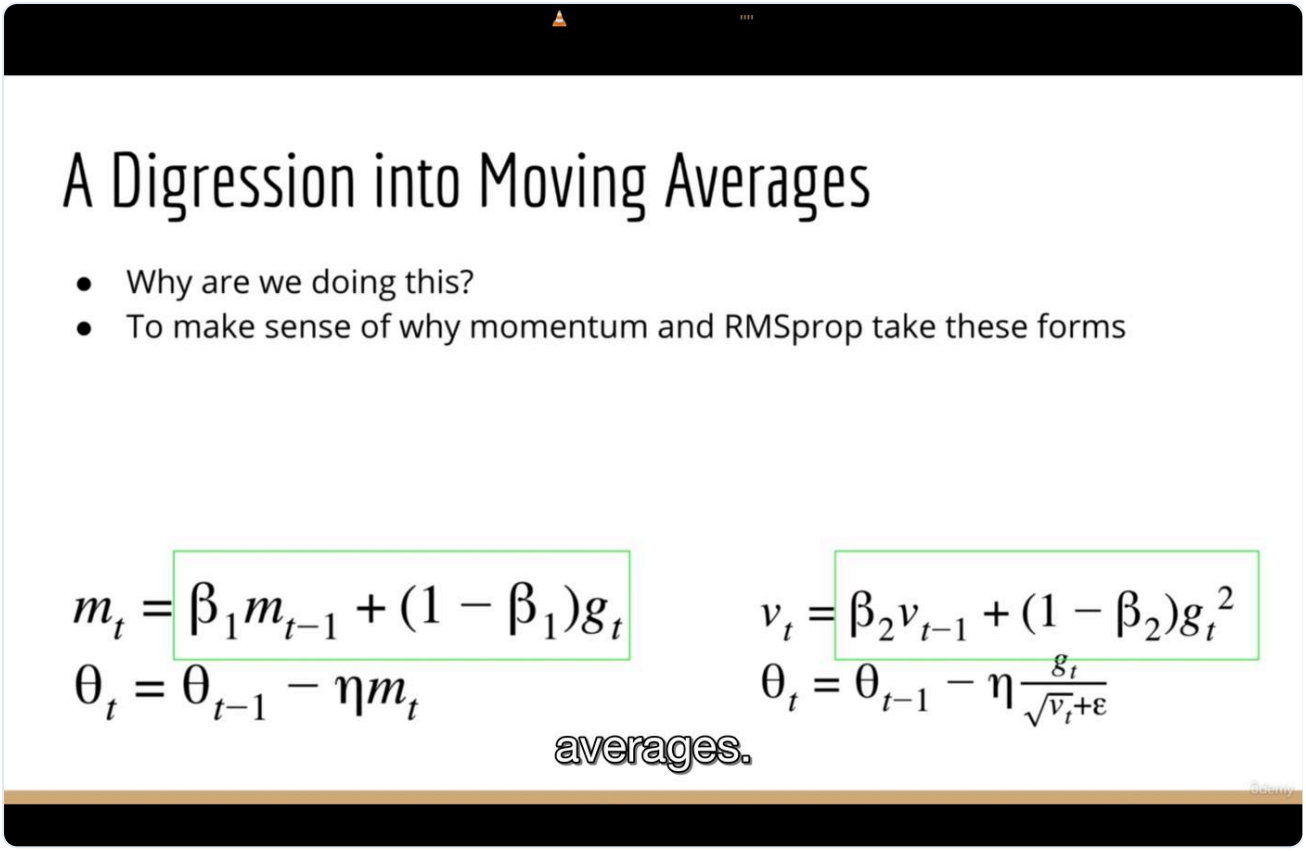

FIGURE 37.9 | A Digression into Moving Averages

By doing this, we'll begin to make sense of why momentum and RMSprop take the forms that they do to understand the moving average. A nice place to begin is just the regular average. Suppose I have a set of data points X one X two all the way up to X T. The average of these data points is exactly what you expect.

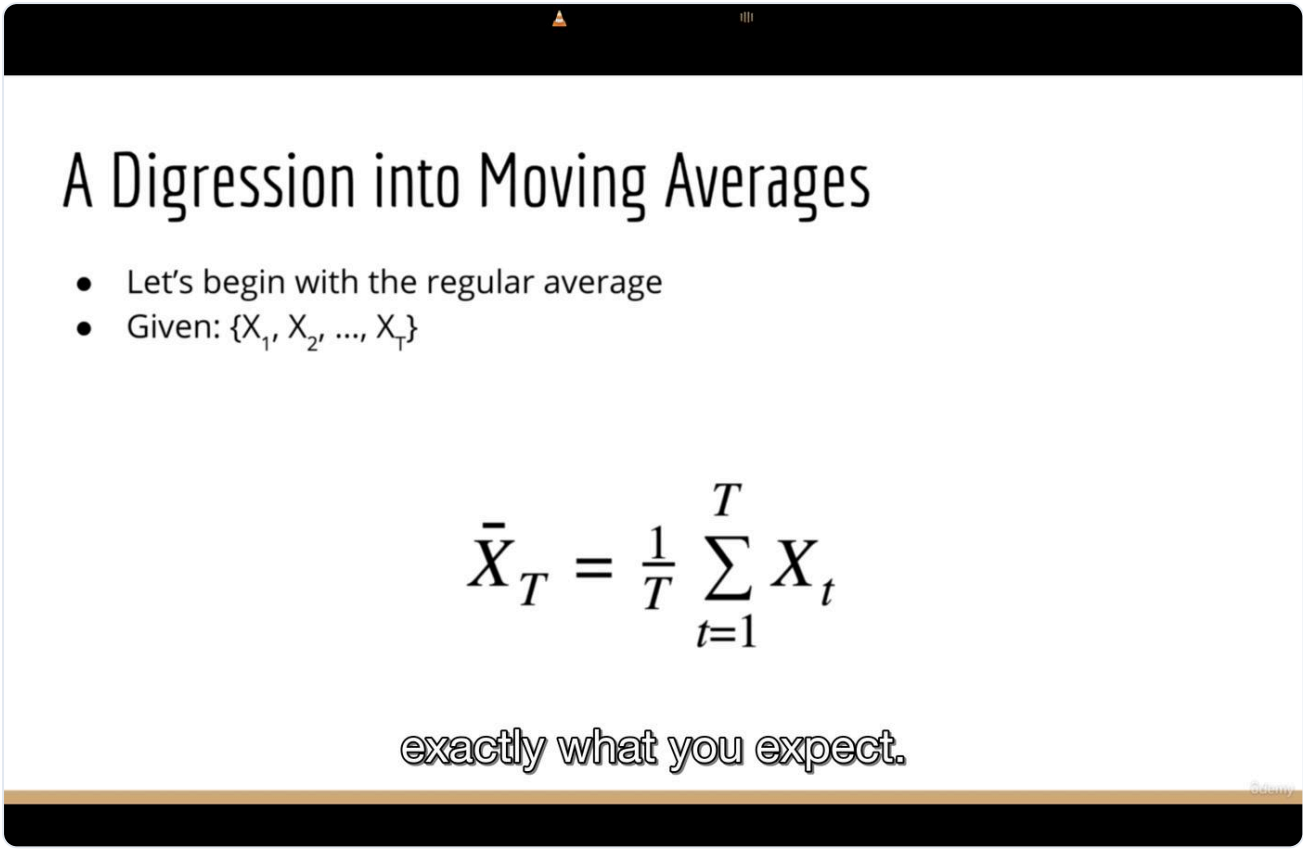

FIGURE 37.10 | A Digression into Moving Averages

Sum up all the X's and divide by the total number of data points, which is T. OK, so pretty simple so far. The next step is to consider what if I have so much data that I cannot fit it all into memory at once? For example, the data lives in a file that's one terabyte in size. Unfortunately, your computer is not capable of storing one terabyte of memory.

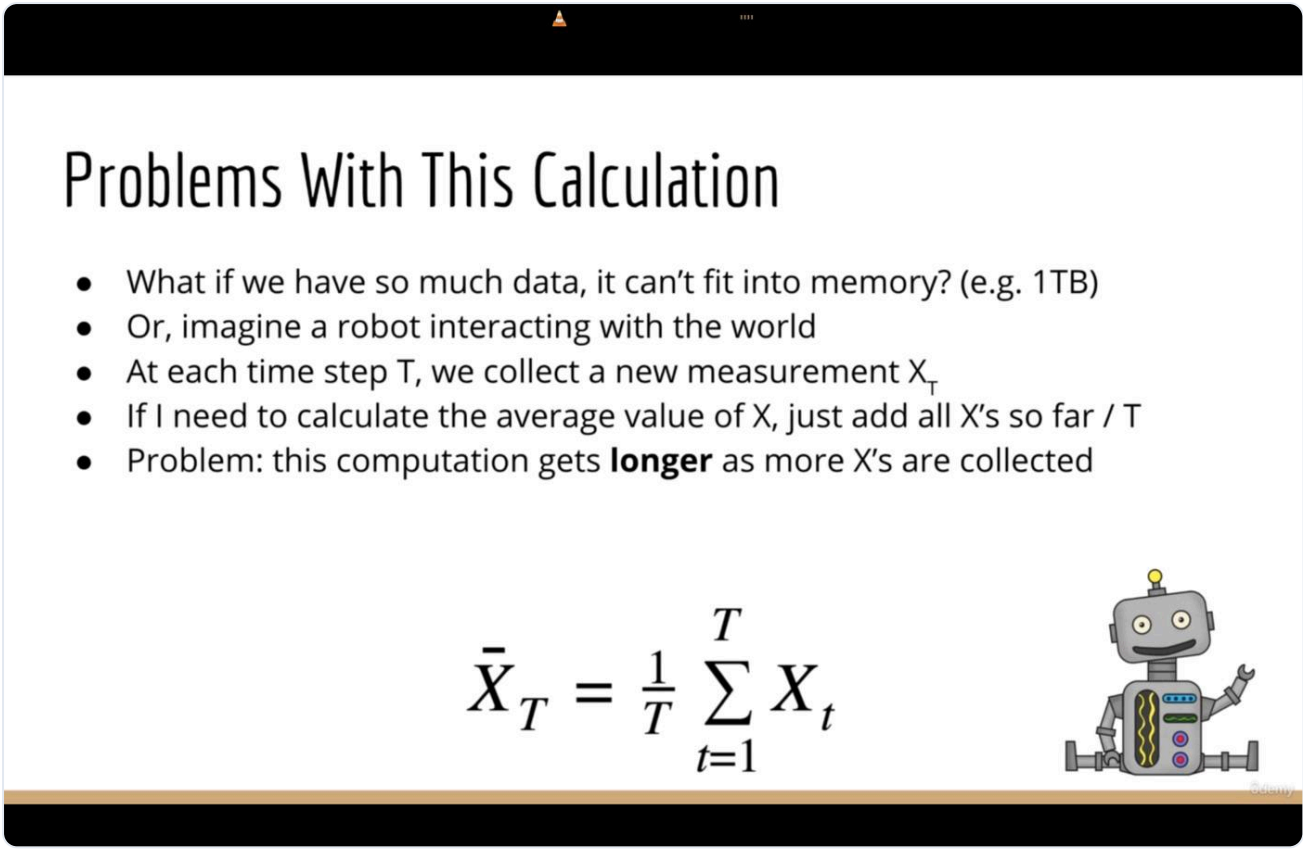

FIGURE 37.11 | Or, Imagine a Robot Interacting with the World

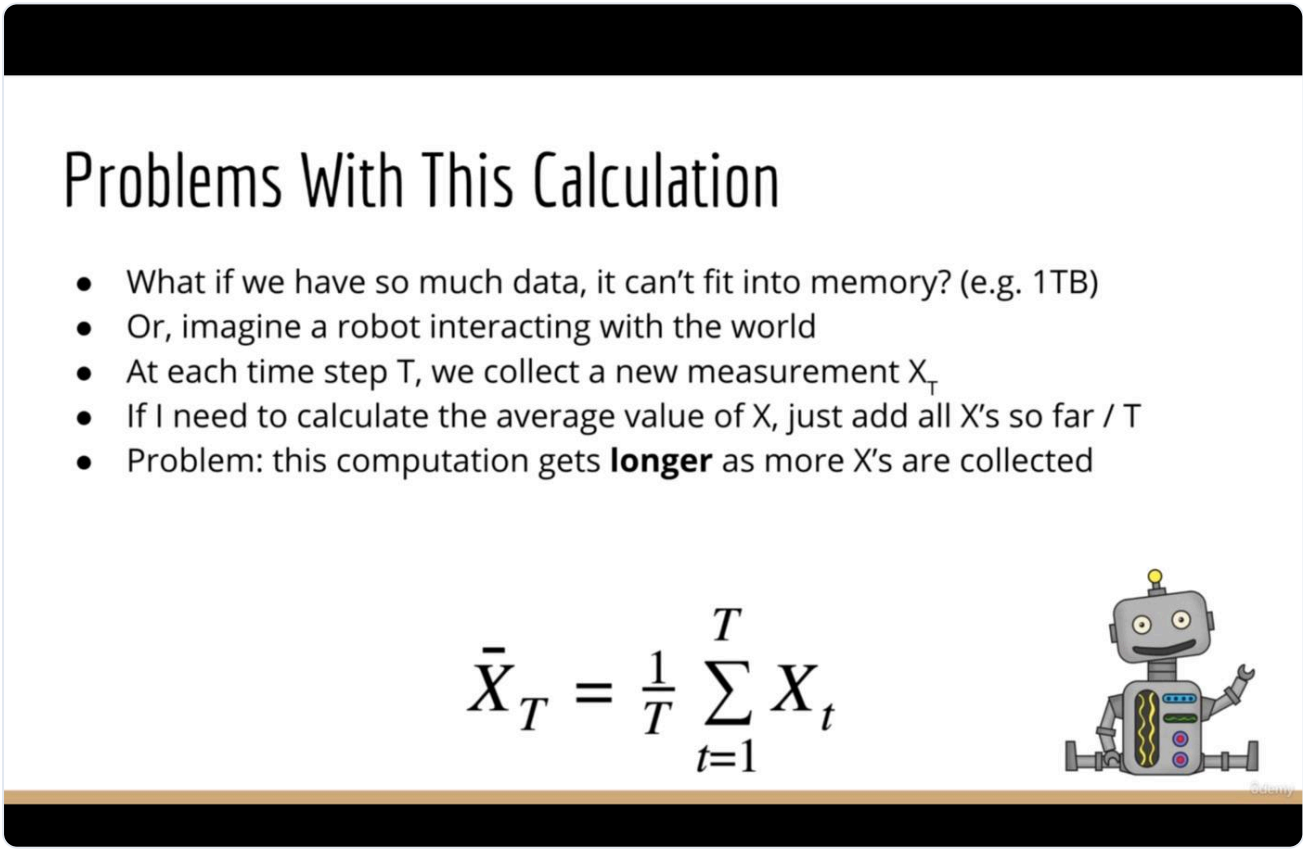

FIGURE 37.12 | Or, Imagine a Robot Interacting with the World

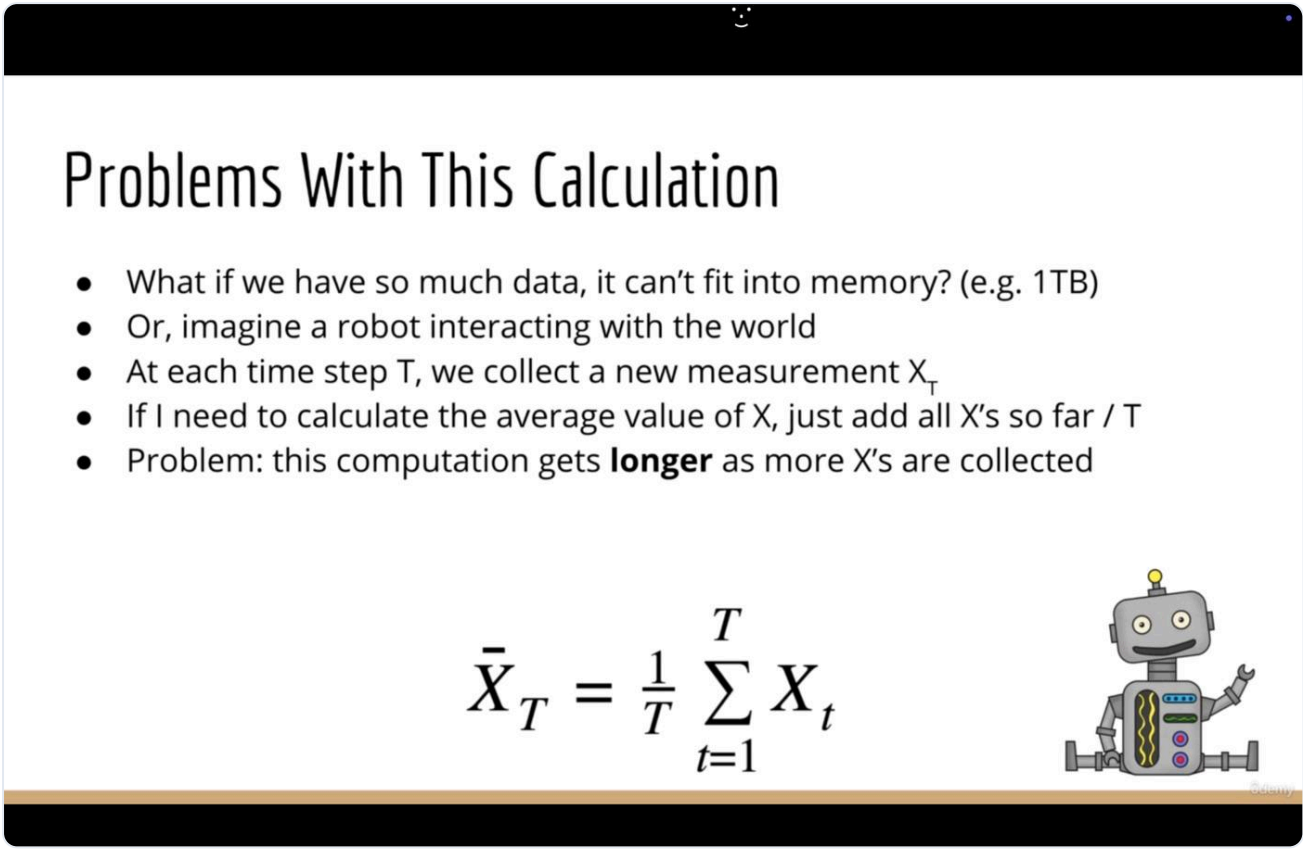

FIGURE 37.13 | Or, Imagine a Robot Interacting with the World

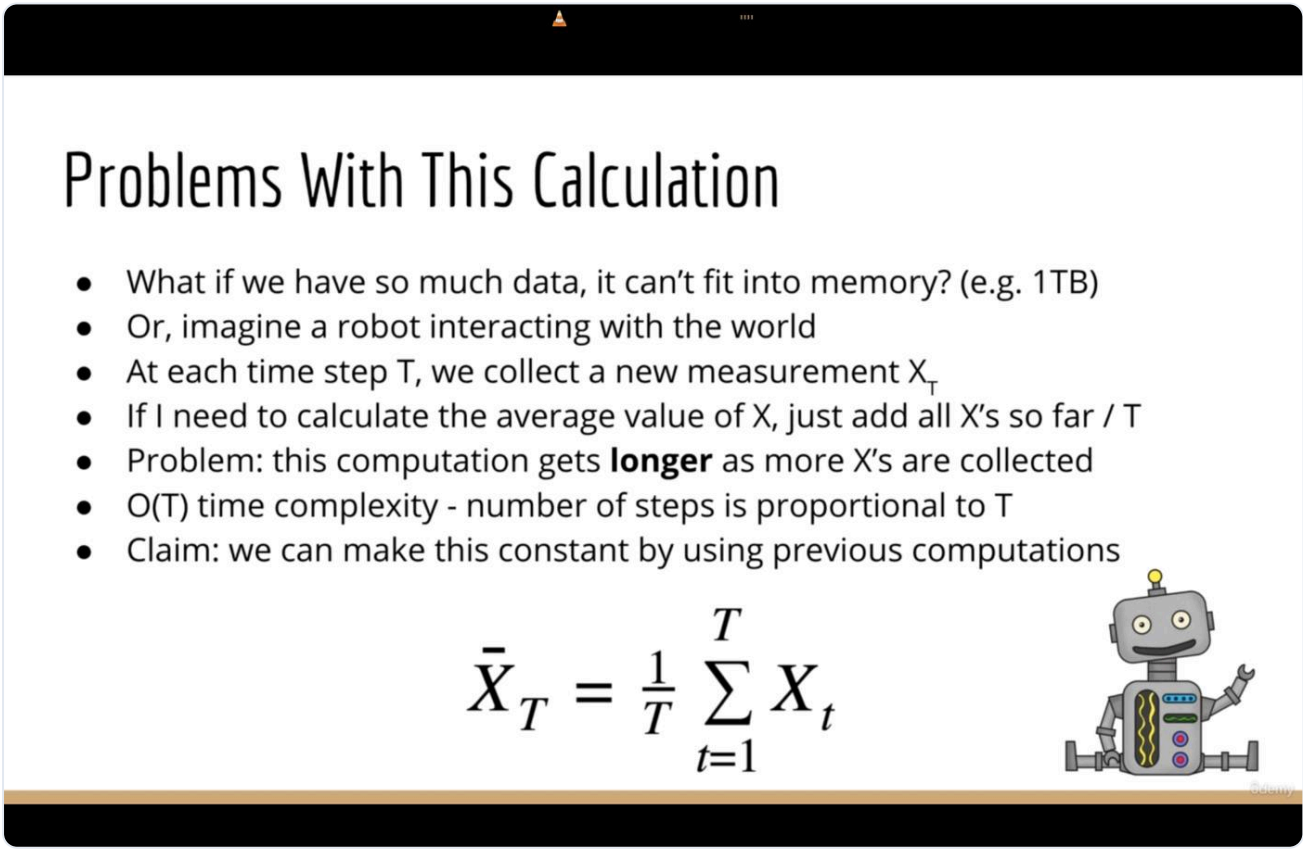

FIGURE 37.14 | Or, Imagine a Robot Interacting with the World

Or consider another scenario where you have a robot interacting with the world. In this case, each data point arrives at the robot's sensors at each instant in time. I can't see any of the future data points, so my average can only consist of the past data points and the current data point. If I want the average of all the data points I have seen so far, let's assume that the current time is t then sure I can add them all up and divide by T, but that seems wasteful. You can see that as T increases. The number of things I have to add together also increases. Therefore, on each step my robot will have to do more and more computation in order to compute this average.

We say that this computation is O of T because the number of steps to perform the computation is proportional to T. My claim is that if we have a robot streaming in data one point at a time, we can actually make this computation O of one.

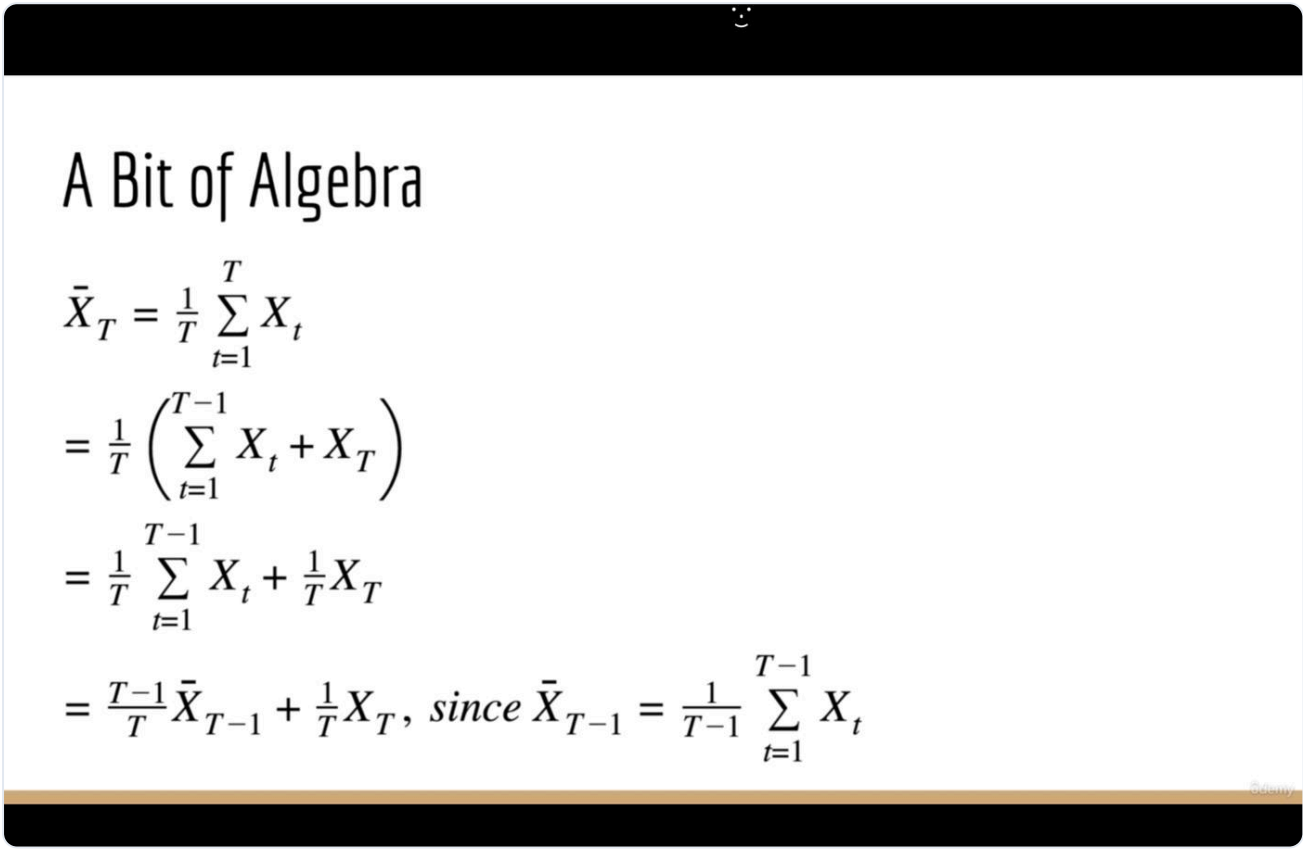

FIGURE 37.15

In other words, our computation is constant no matter how much data we've collected specifically that we can compute the current average by somehow making use of the previous computations we've done, and even more specifically, the previous average. To understand how we can do this, let's manipulate the equation for the sample mean, firstly, notice that our notation for the mean at time T is x bar subscript T, let's begin by splitting our sum by taking out only the final data point X of T. The next step is to multiply out the one over T term. The next step is to replace the sum from little T equals one up to Big T minus one specifically, we can replace this with the sample mean at time T minus one.

As you recall, the sample mean at time T minus one is just the sum divided by T minus one. OK, so now what do we have? We have exactly what we discussed previously. We have an equation that allows us to compute the current average X bar T in constant time. As you can see, it only in-

volves two terms the previous average X bar T minus one and the current sample X of T.

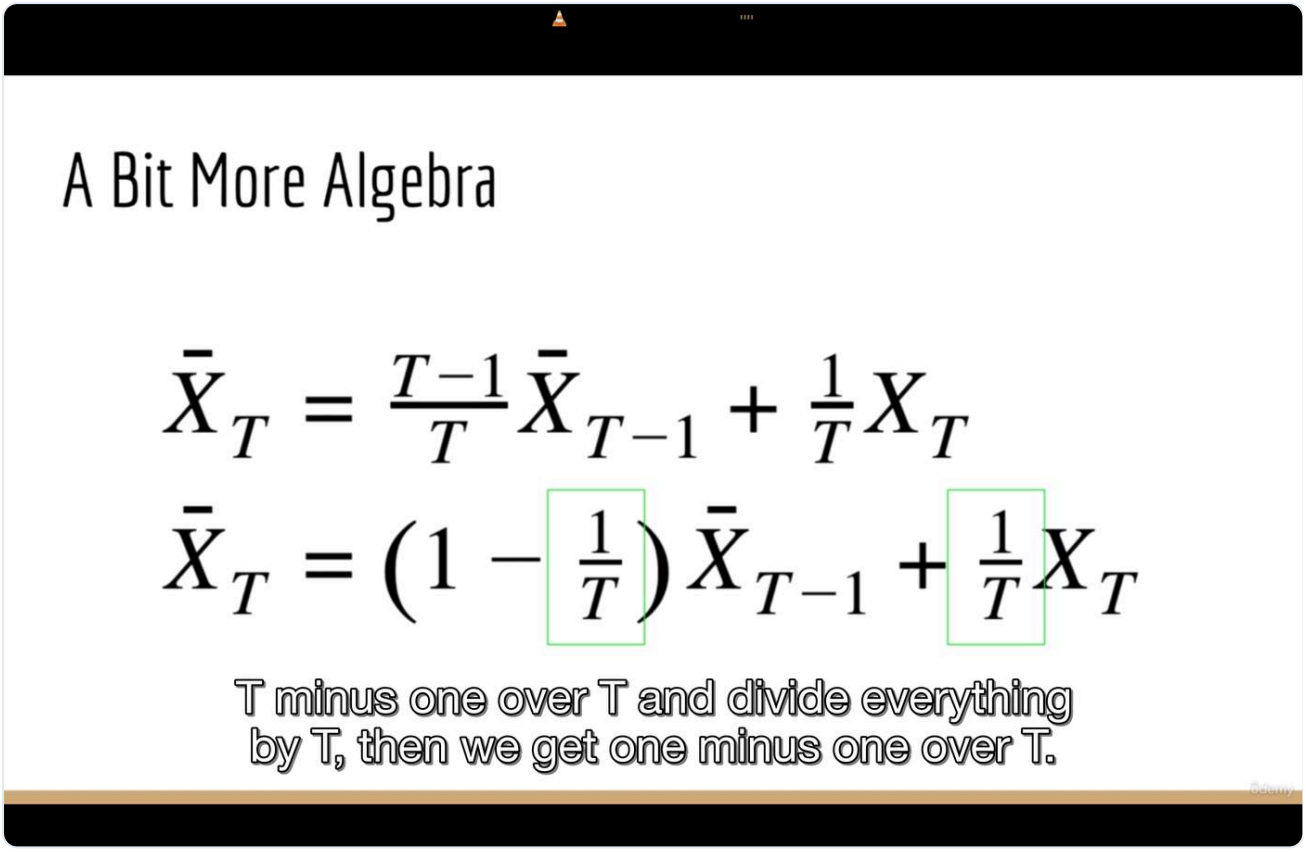

FIGURE 37.16

The next step is to manipulate our expression even more specifically, let's take the fraction T minus one over T and divide everything by T, then we get one minus one over T. This is helpful because now that one over T term appears twice. The next step is to ask the question, what if we replace one over T with a constant let's call it Alpha in this case, we are changing the answer, but we are doing so in a way that might be helpful later on when this was equal to one over T, we saw that this gave us the regular sample mean.

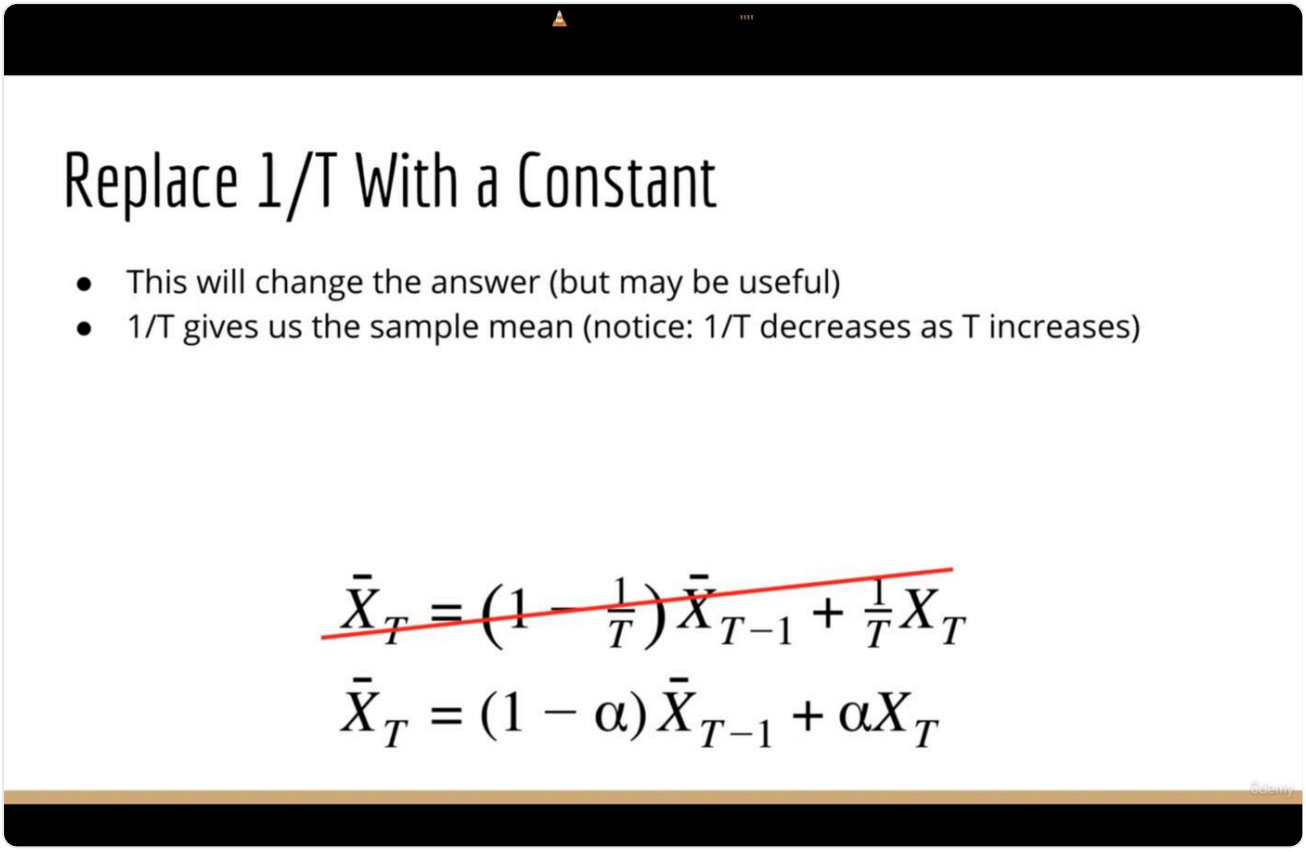

FIGURE 37.17 | Replace 1/T with a Constant

As you can see, the function one over T decreases as T increases. However, it turns out that this will weigh all of the X's just right so that every X we have considered so far is weighted by one over T note that we have just proven this and as you know, it leads exactly to the sample mean.

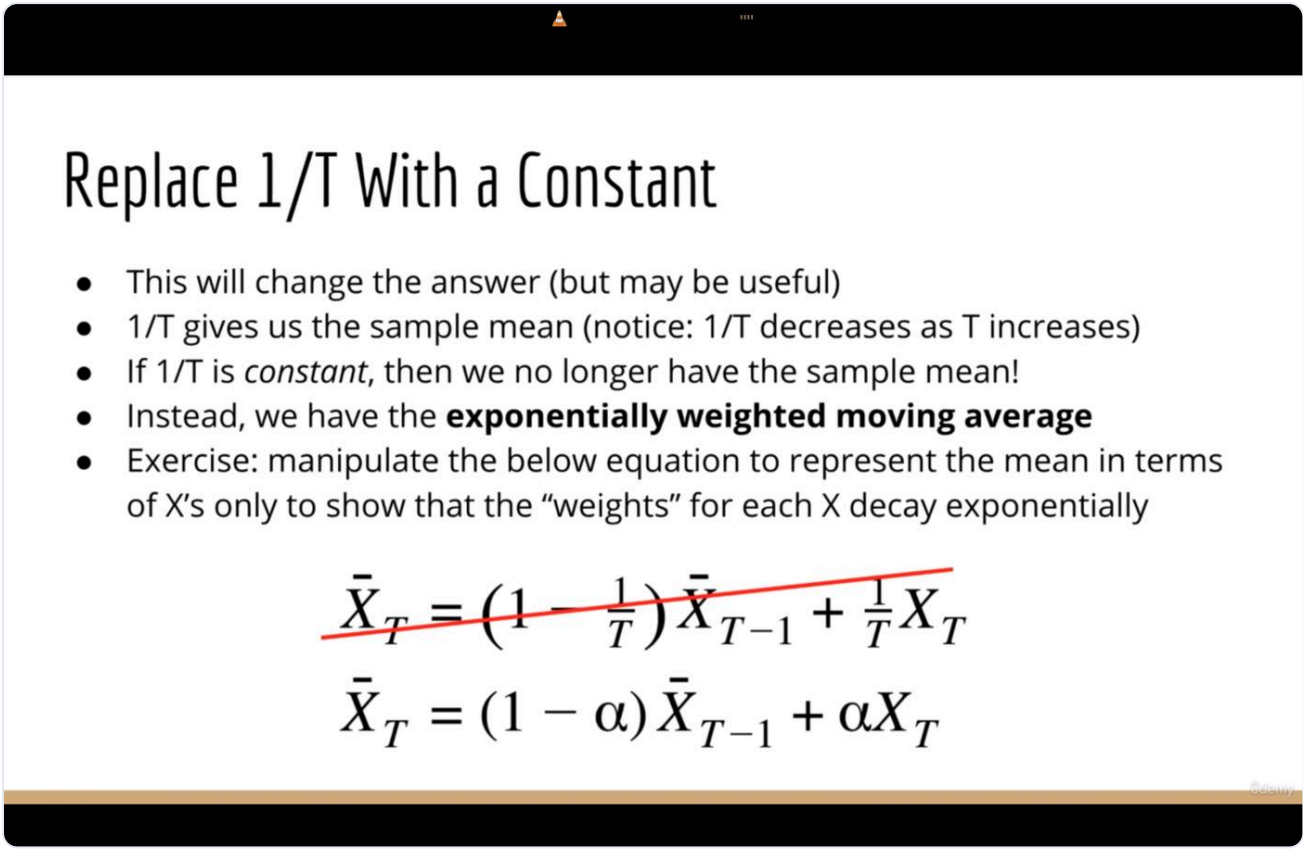

FIGURE 37.18 | Replace 1/T with a Constant

Importantly, this means that if we replace one over T with a constant, we will no longer have the sample mean. In fact, when we have a constant, this gives us the exponentially weighted moving average as an exercise. You might want to try manipulating the equation for constant alpha and put it back into a summation so that it depends only on the X's and not any previous averages.

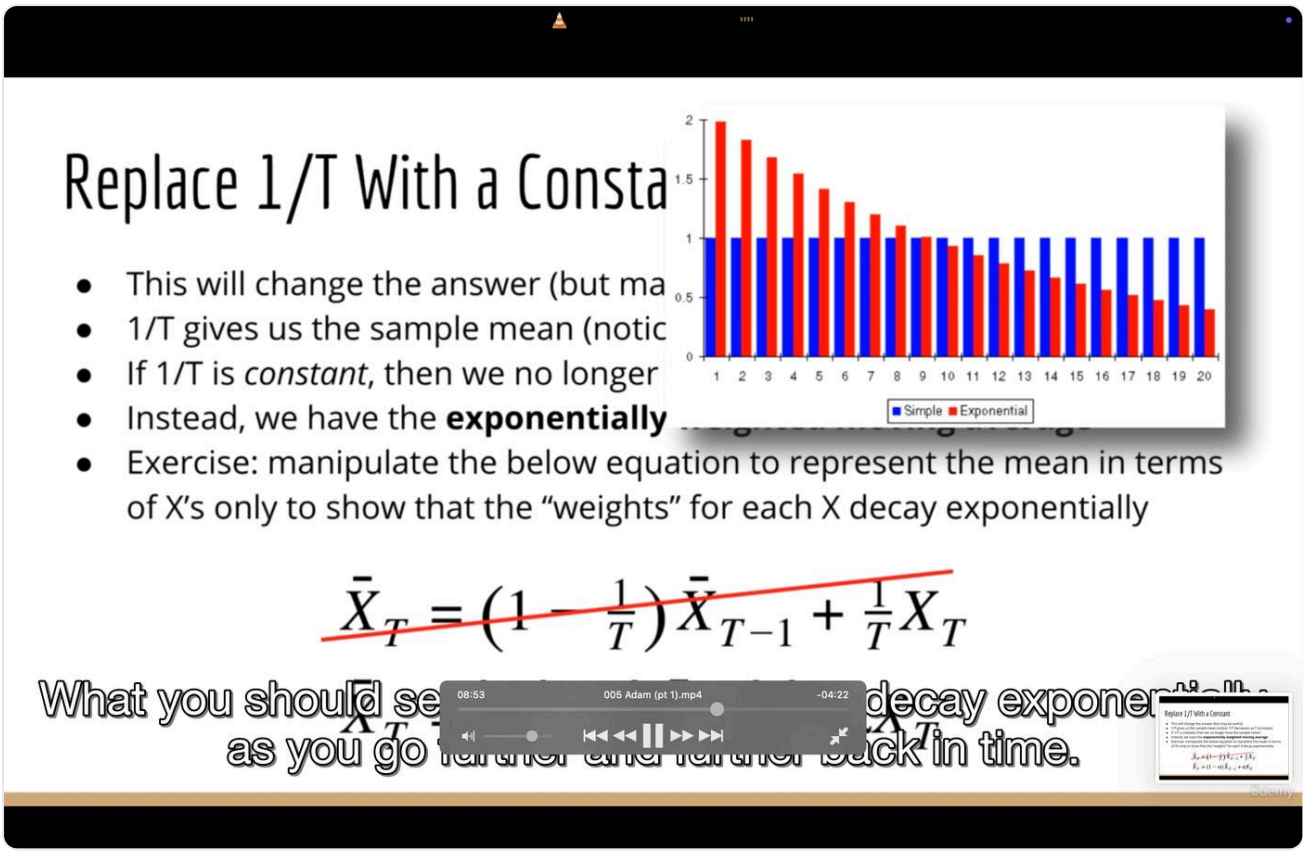

FIGURE 37.19 | Replace 1/T with a Consta:

What you should see is that the weights decay exponentially as you go further and further back in time. This is opposed to the regular average where all the weights are equal.

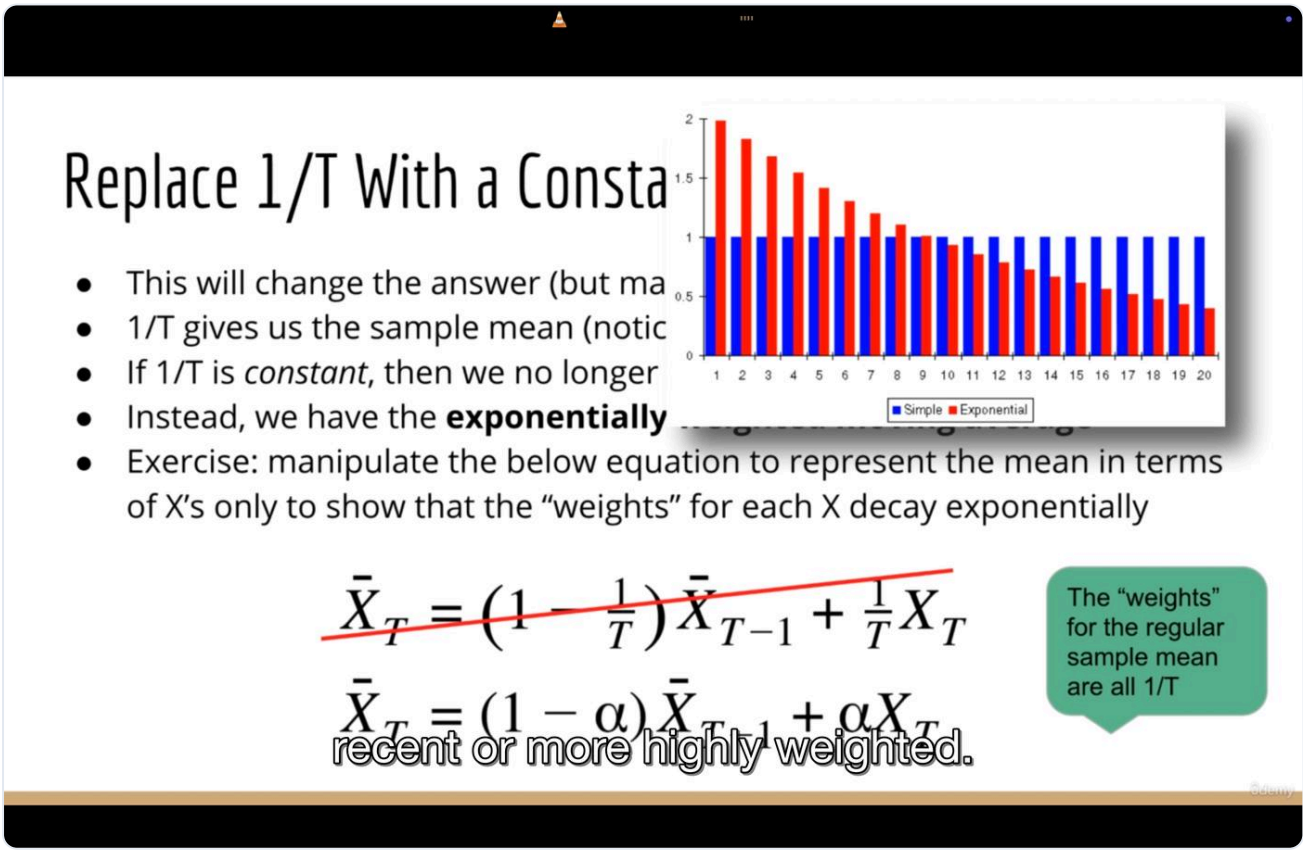

FIGURE 37.20 | Replace 1/T with a Consta'

Basically, this means that for the exponentially weighted moving average data points, which are more recent or more highly weighted. Now, just by convention, we can replace Alpha with the new symbol, specifically, let's introduce Beta, which is equal to one minus Alpha. We'll call this the decay.

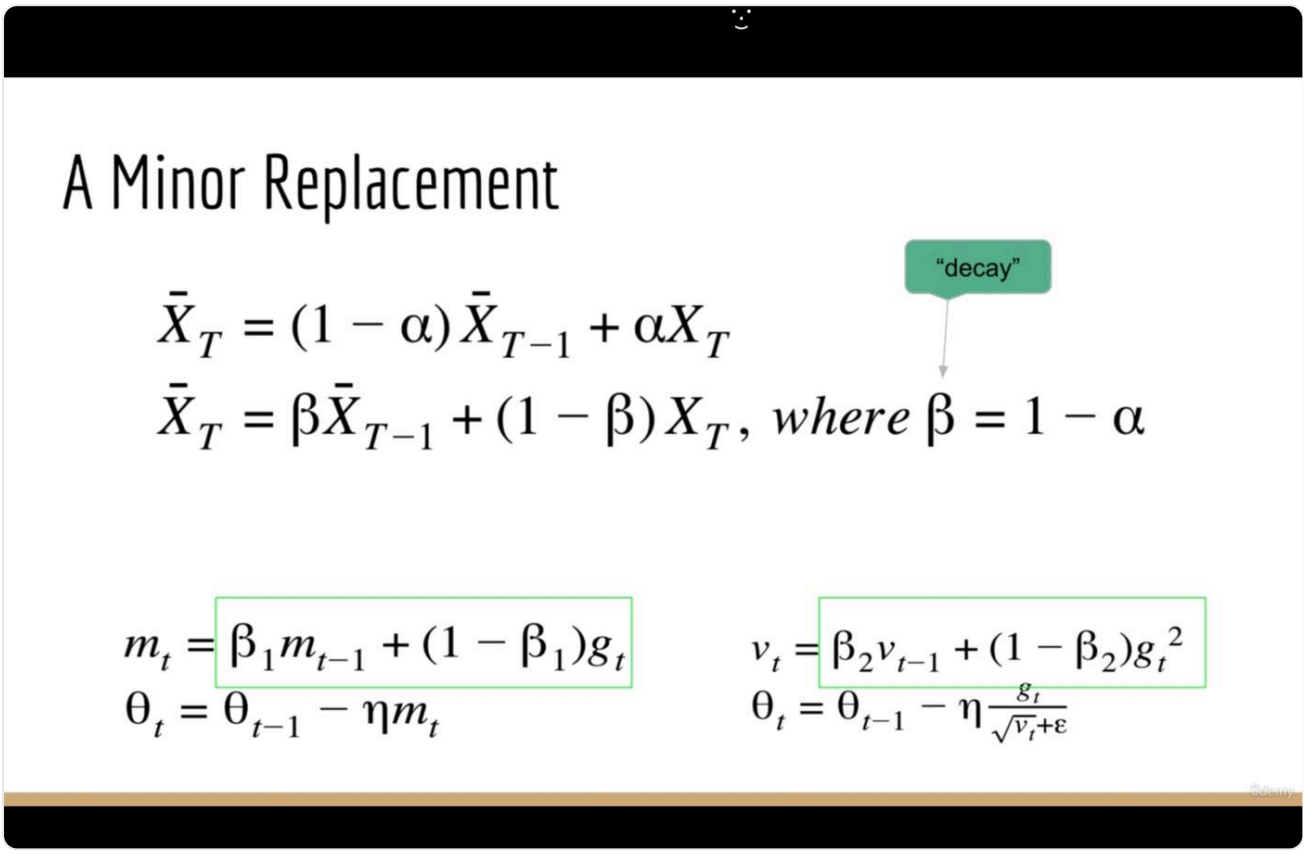

FIGURE 37.21

Note that this doesn't change anything. Only the symbols are different, but the functionality of the equation is still exactly the same.

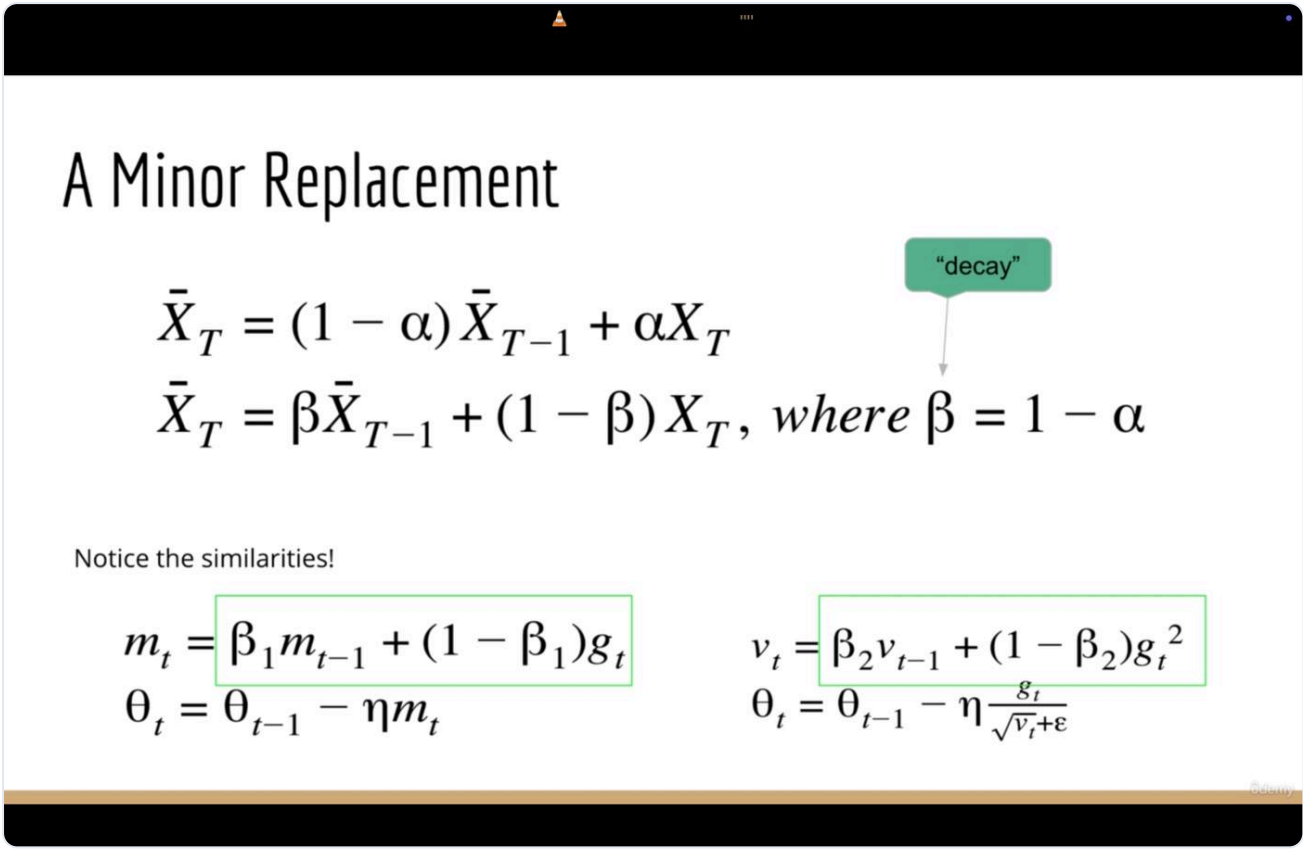

FIGURE 37.22

At this point, you should recognize that this is exactly the same as the cache update for RMSprop. It is also the same as our modified momentum update. So in RMSprop, where do we use this exponentially weighted moving average?

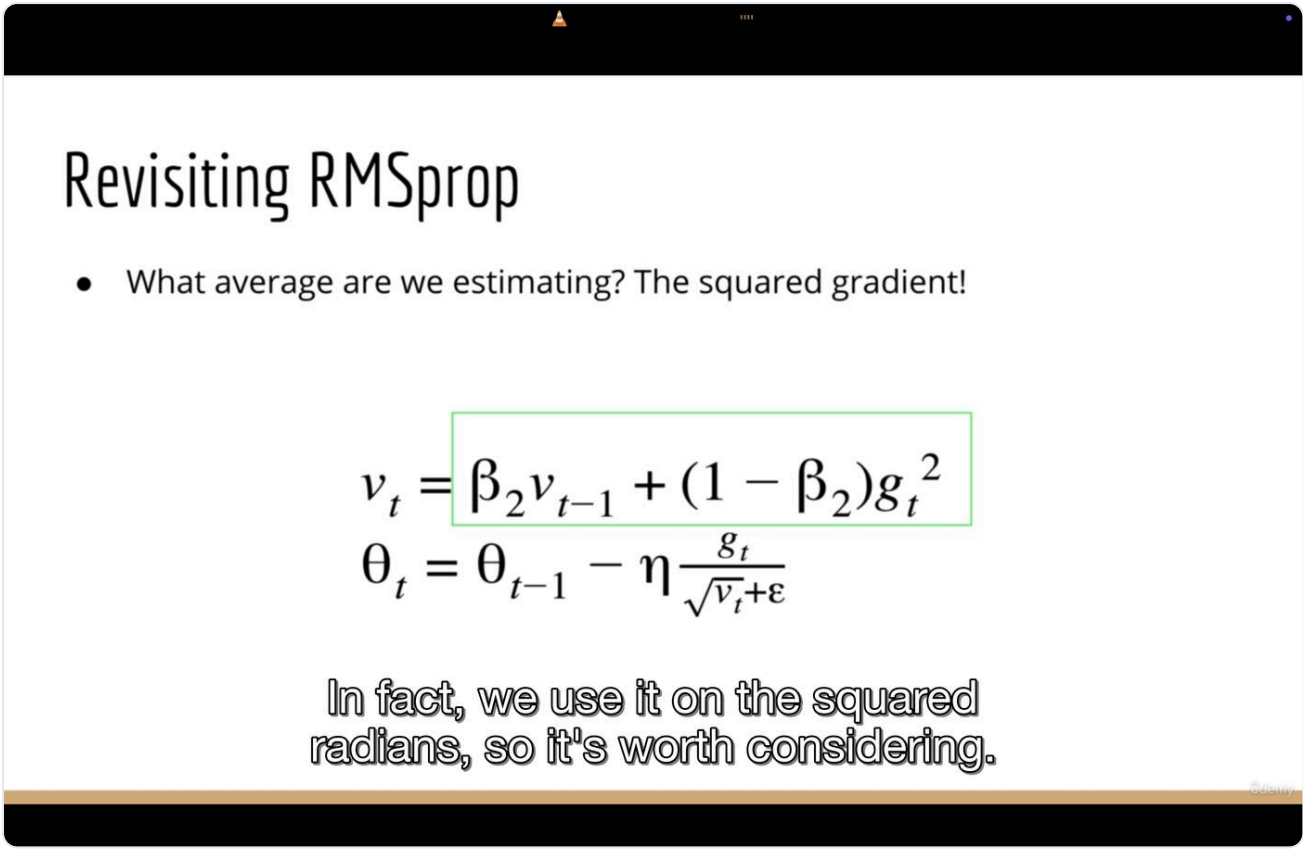

FIGURE 37.23

In fact, we use it on the squared gradients, so it's worth considering. What is this the average of what are we estimating now trivially? We are estimating the mean of the gradient squared. So in RMSprop, because we eventually end up using the square root of the cache. Now, the name RMSprop makes sense. RMS stands for root mean square. Specifically, we are estimating the square root of the mean of the square of the gradient.

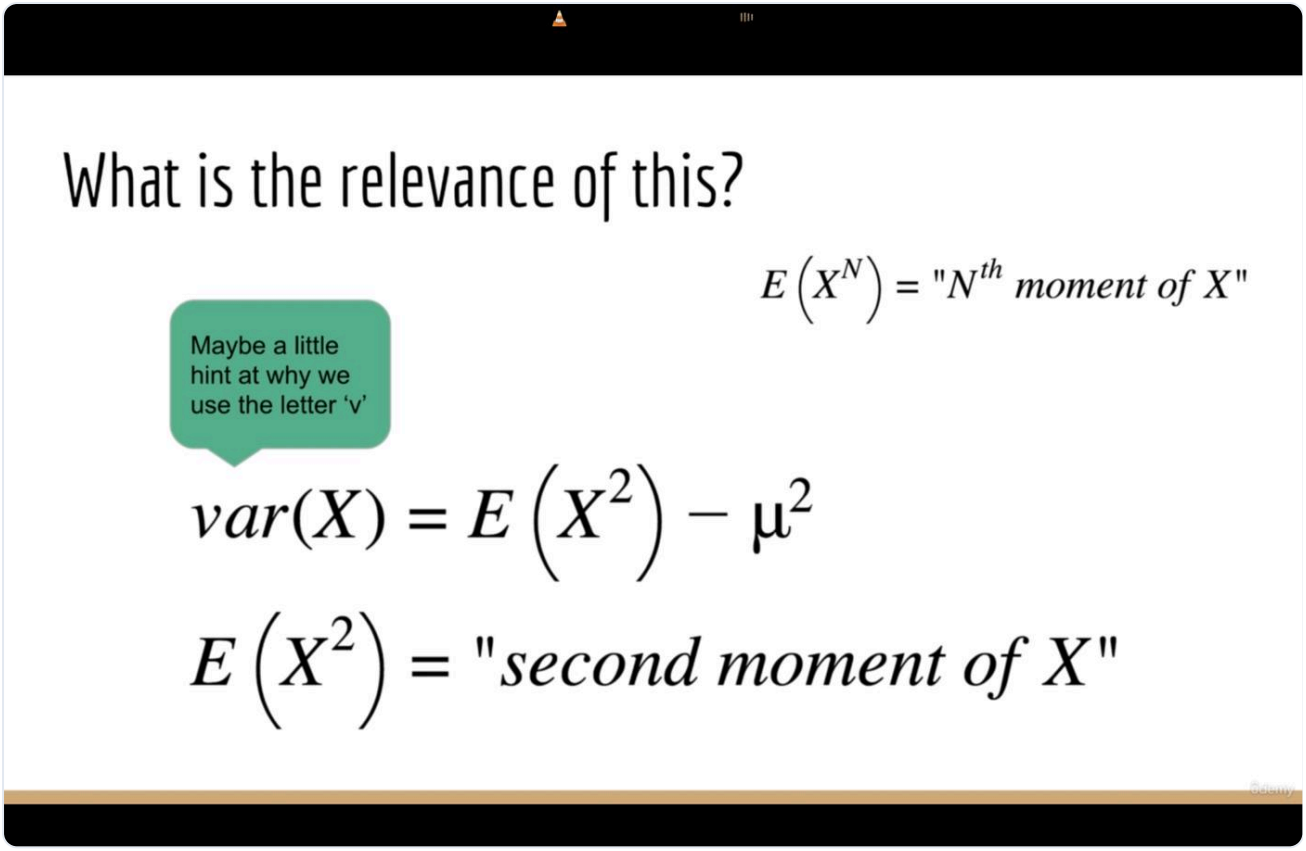

FIGURE 37.24 | What Is the Relevance of This?

However, we can go further than this. What is the sample mean? You may recall from statistics that the sample mean is used to estimate expected values. Now, what is the expected value of the square of a random variable? Well, it is related to the variance. Specifically, the variance is equal to the expected value of the random variable squared, minus the true mean squared. However, when we just have the expected value of the square, we often call this the second moment in general, the expected value of X to the power N is called the Nth moment.

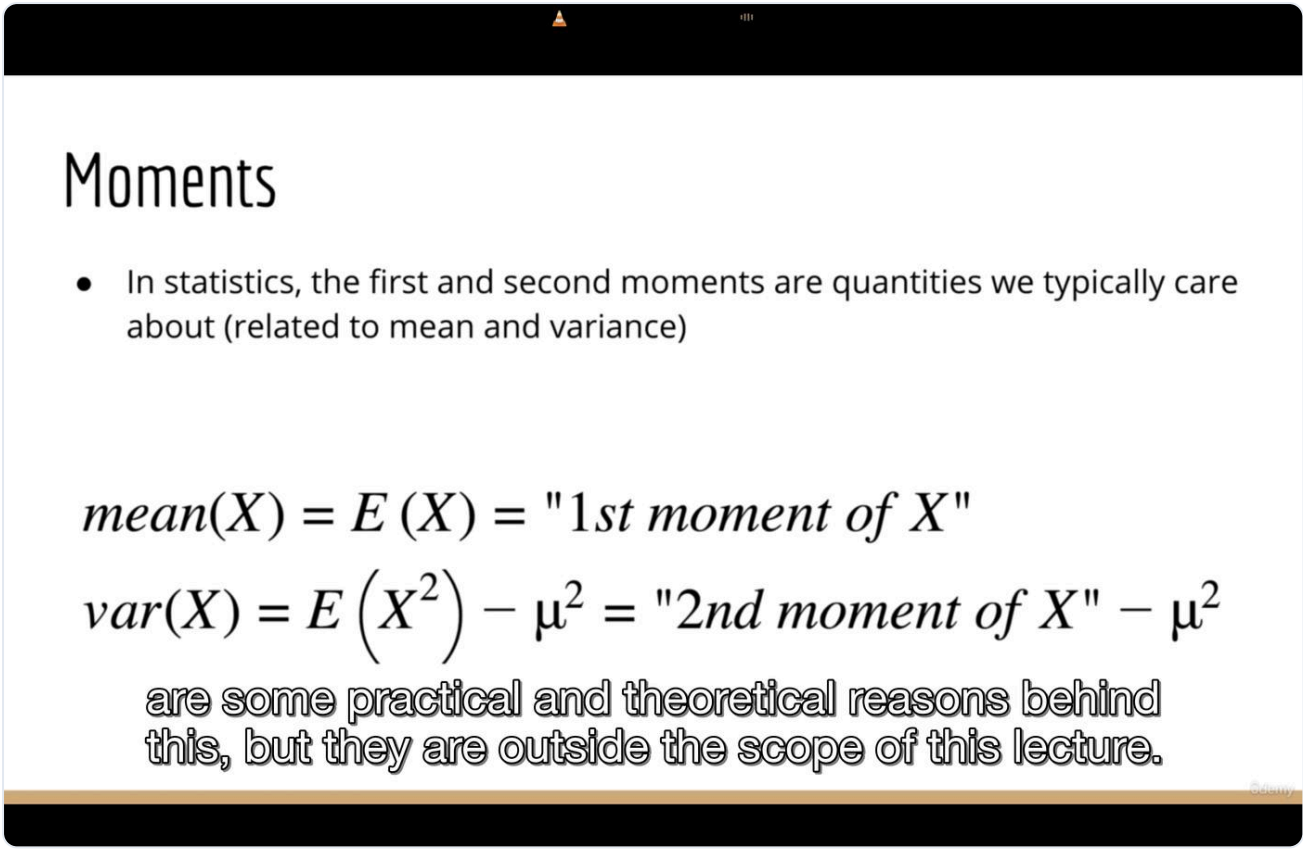

FIGURE 37.25

In statistics, it's often the case that what we care about are the first and second moments, there are some practical and theoretical reasons behind this, but they are outside the scope of this lecture. Intuitively, we know that what we often care about when it comes to something random is the mean and the variance. We know that the second moment is something like a variance. Similarly, the first moment is simply the mean.

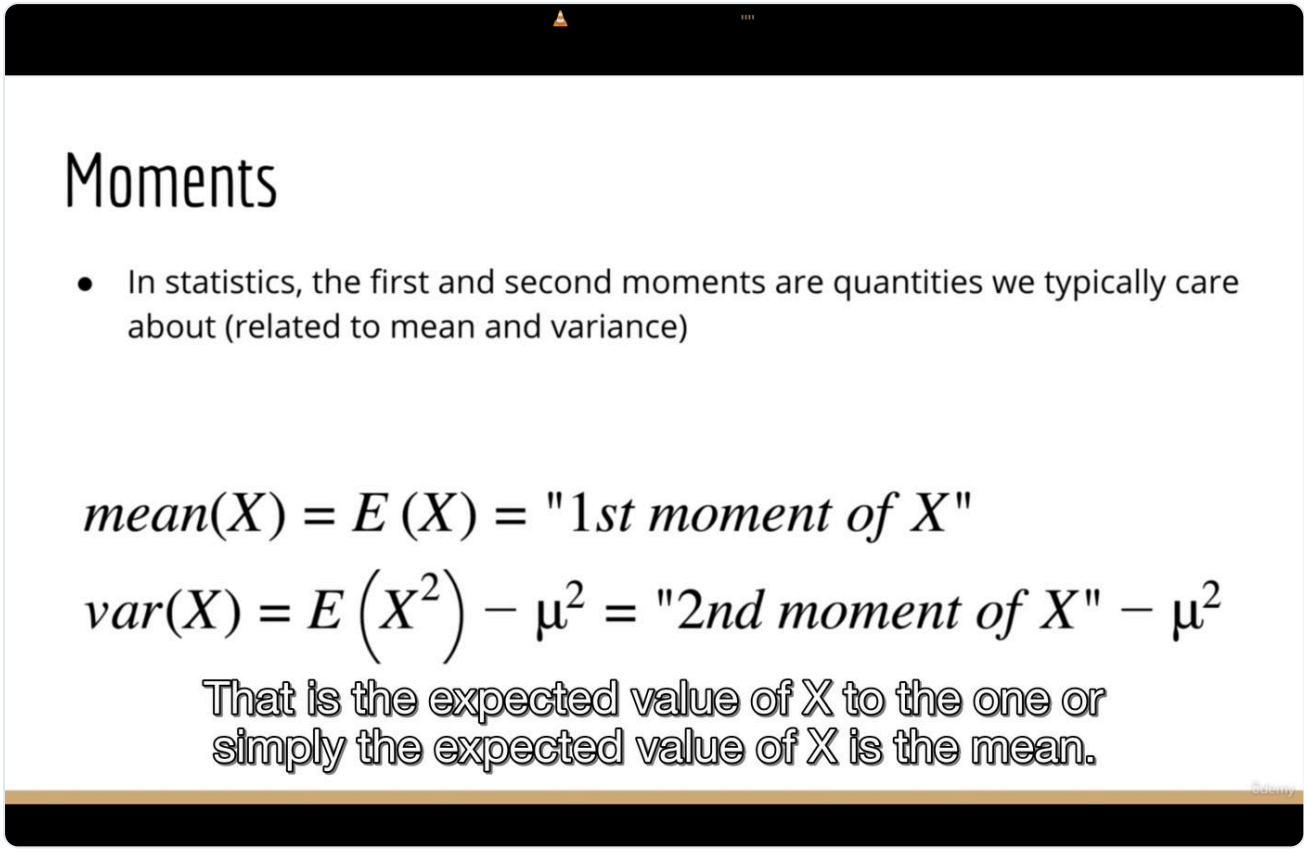

FIGURE 37.26

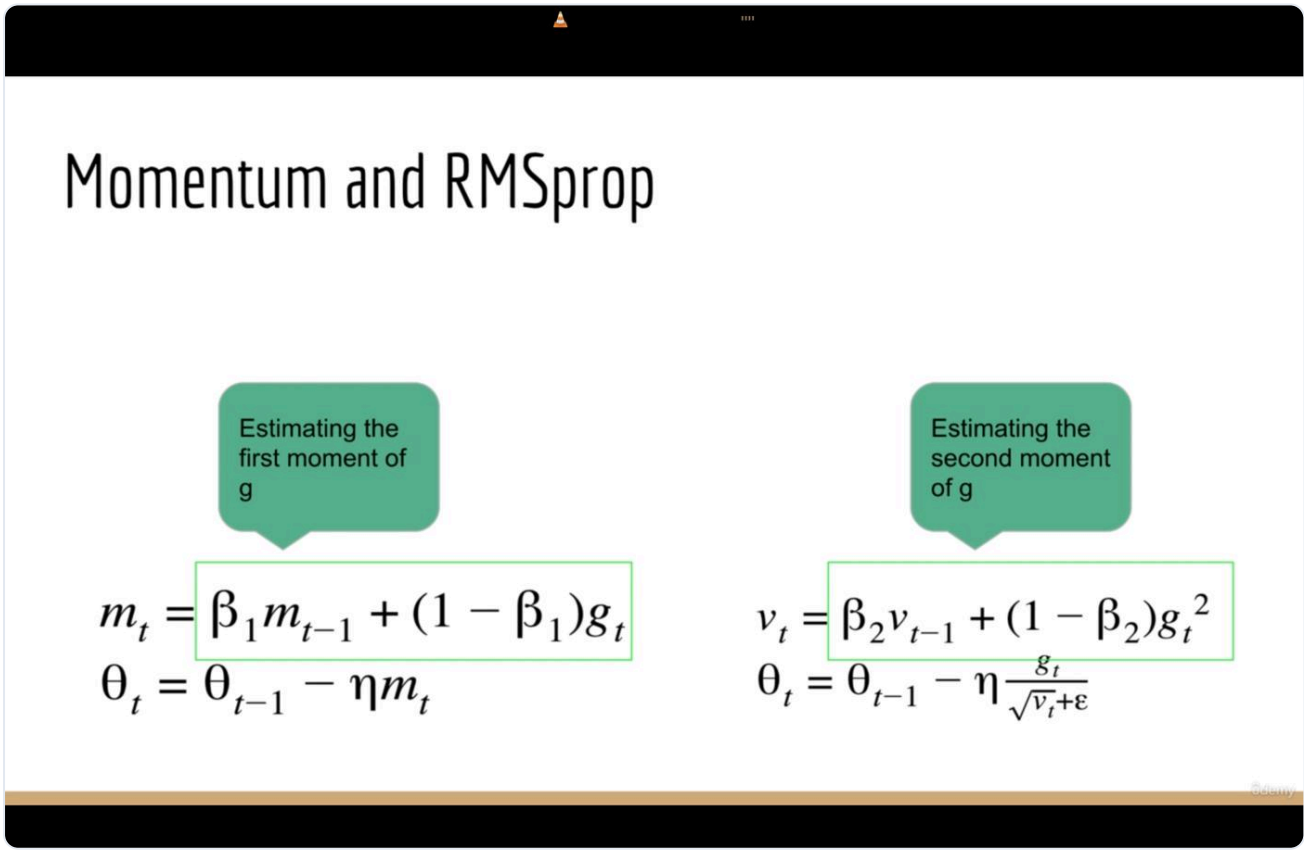

FIGURE 37.27 | Momentum and RMSprop

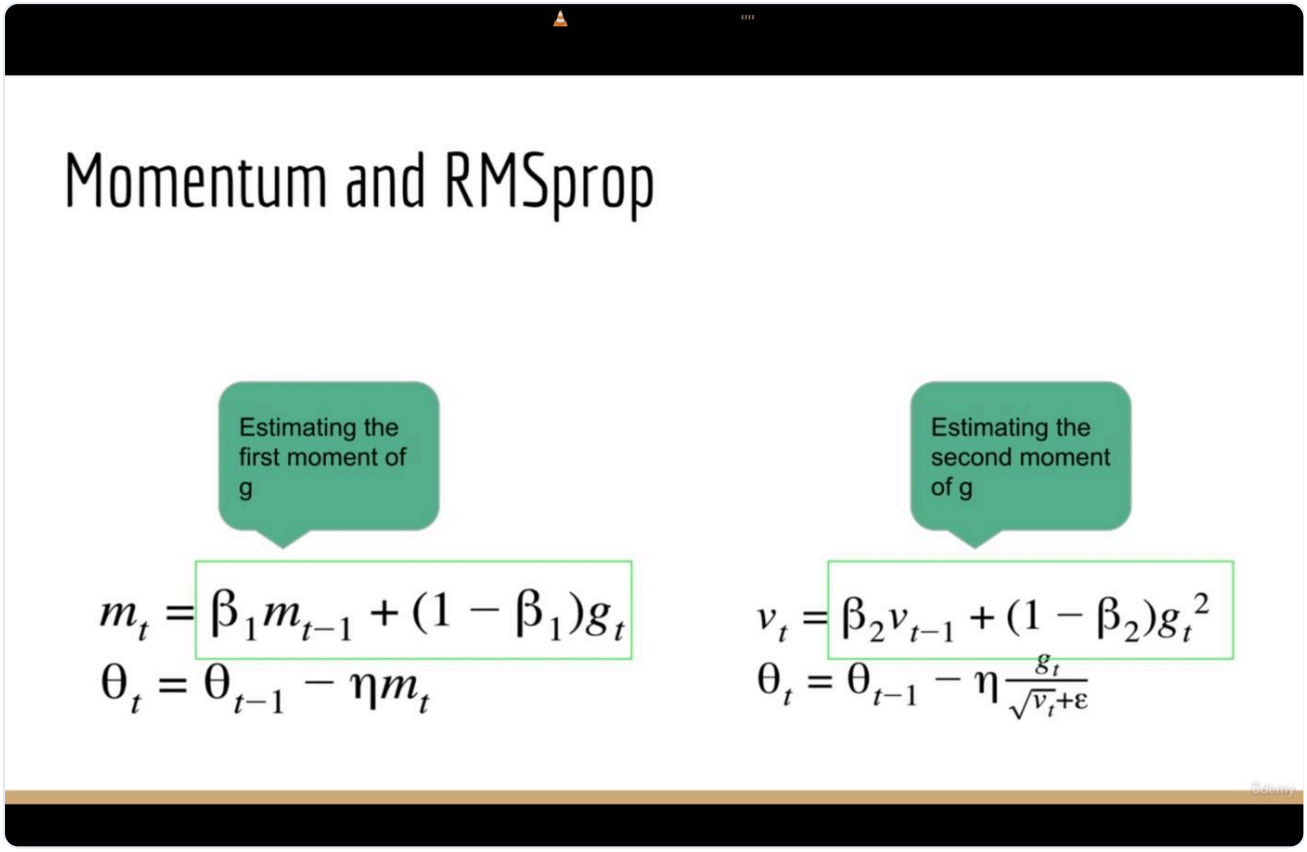

FIGURE 37.28 | Momentum and RMSprop

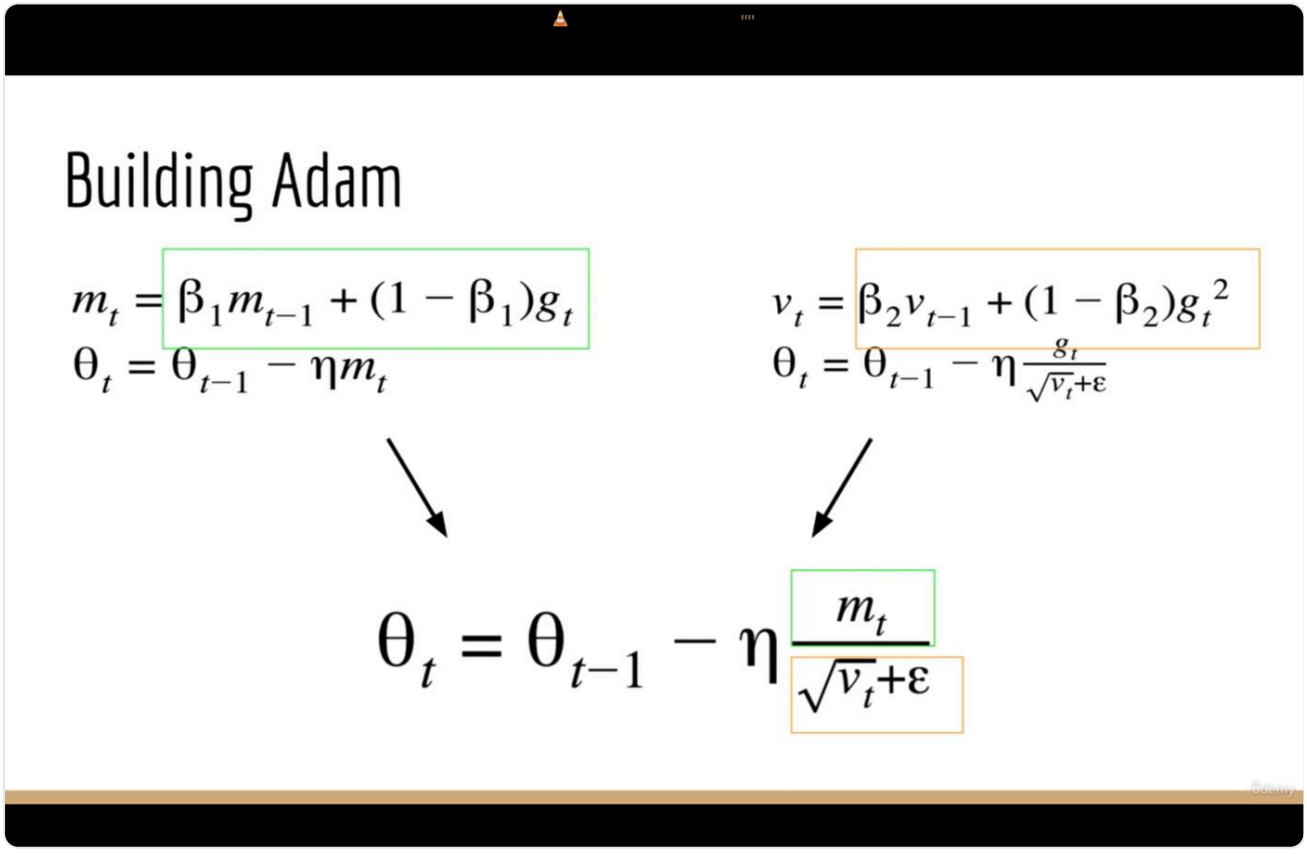

FIGURE 37.29

That is the expected value of X to the one or simply the expected value of X is the mean. At this point, we can recognize how this relates to momentum and RMSprop. Momentum essentially estimates the first moment of the gradient using an exponential moving average. Similarly, RMSprop essentially estimates the second moment of the gradient using an exponential moving average. Now, since we might want to tune these two values separately, we will give each of them a possibly different value of beta. We'll call the beta for the first moment beta one and we'll call the beta for the second moment beta two. The last step is to essentially put these two ideas together.

And that gives us Adam. In order to make sense of how to put these two ideas together, let's remind ourselves what momentum and RMSprop look like when they are apart. For momentum, we subtract the learning rate and multiply by M. For RMSprop, we subtract the learning rate multiplied by the gradient and divide by the square root of V plus some tiny value. Thus for Adam we simply combine these two together. That is to say, we subtract the learning rate, multiply it by M and divide by the square root of V, plus the usual tiny value to avoid dividing by zero. Now, this is the essence of the Adam optimizer, but note that there is still one more step before we are done, since this lecture has been quite long so far, and this is a different topic, we will save it for the next lecture.

LECTURE 131

One More Step to Adam

In this lecture, we'll learn about one more step to finalize our Adam optimizer equations to understand why we need to do this, we have to understand what the exponentially weighted moving average can be used for. One way to look at it is that it's just another way of computing the average.

FIGURE 38.1 | One More Step to Adam

Specifically, it's useful when our data is non stationary. That is, the true expected value might be changing over time.

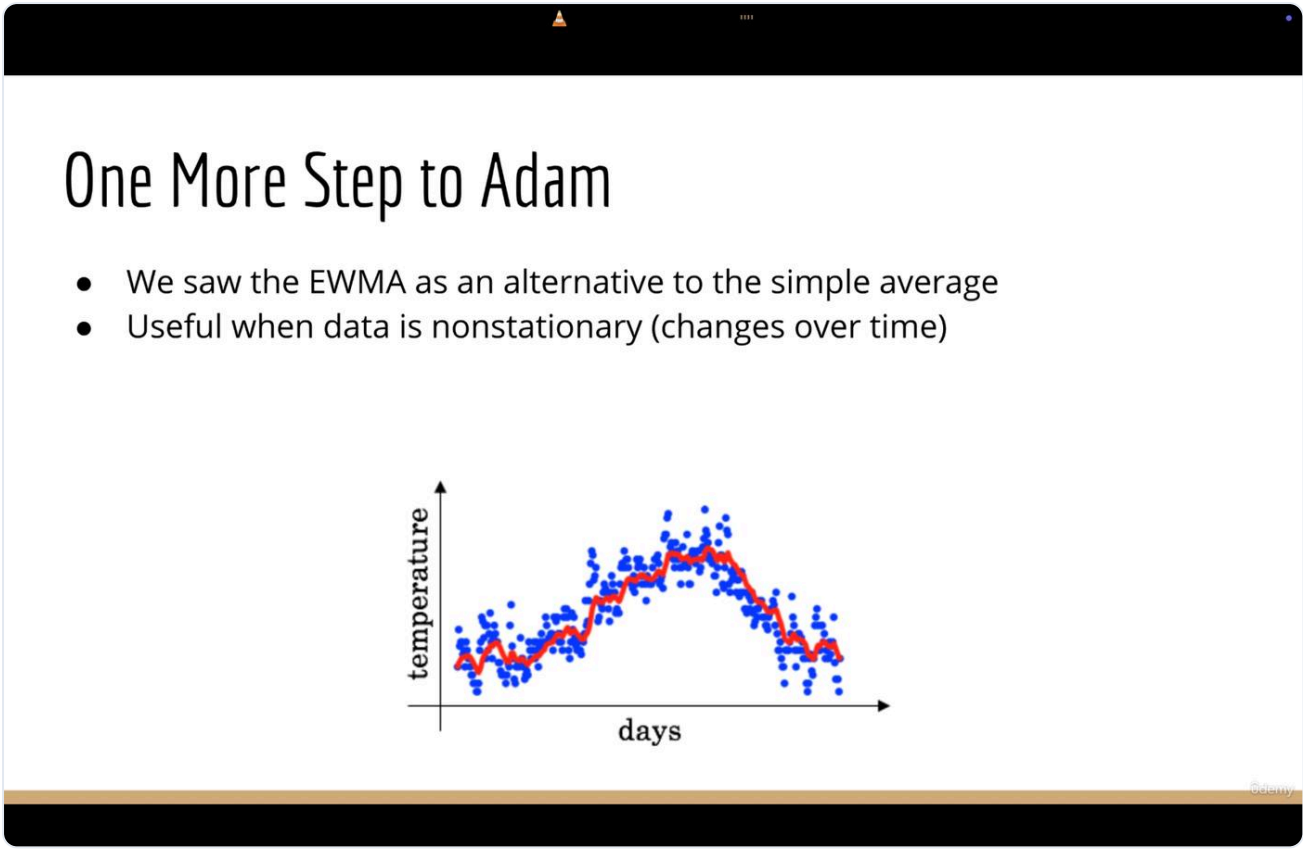

FIGURE 38.2 | One More Step to Adam

Therefore, more recent values are more useful than older values. Another way to look at the exponential moving average is this.

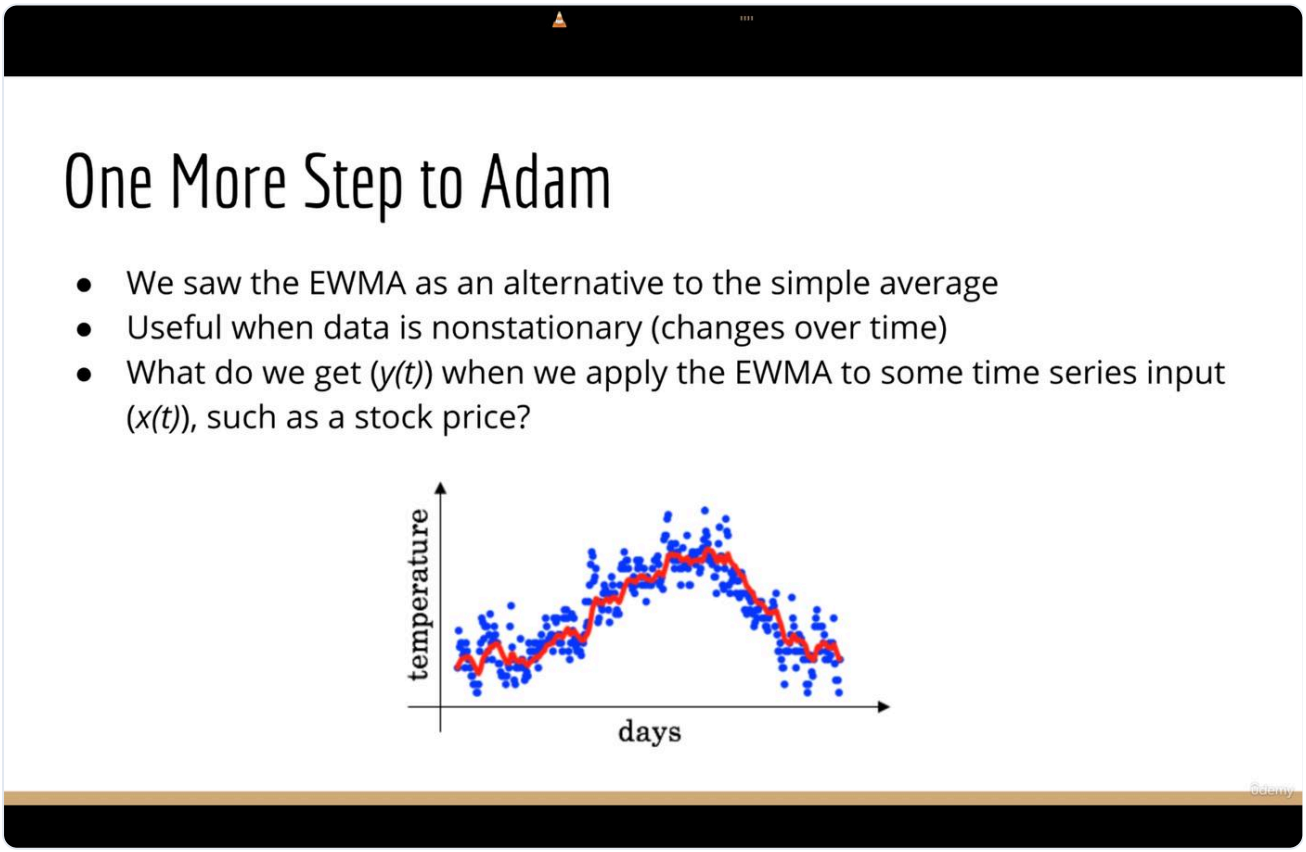

FIGURE 38.3 | One More Step to Adam

Suppose that you have some time varying signal such as a stock price. Clearly, this is a non stationary signal, so it's a good candidate for using this kind of average. Well, what happens if we apply our moving average to this time series signal. Let's call the input time series X of T and let's call the output time series Y of T. Well, what you get is you get a smoothed version of the input signal, that is, it generally follows the same trend, but in a much smoother manner without lots of random jumps up and down. This is useful in finance, for example, when you want to know the trend of a stock price, but you don't care about minor fluctuations. If you have experience in signal processing, you might know this by another name.

This is called the low pass filter. The reasoning behind that is this slow fluctuations have low frequencies, whereas fast fluctuations have high frequencies.

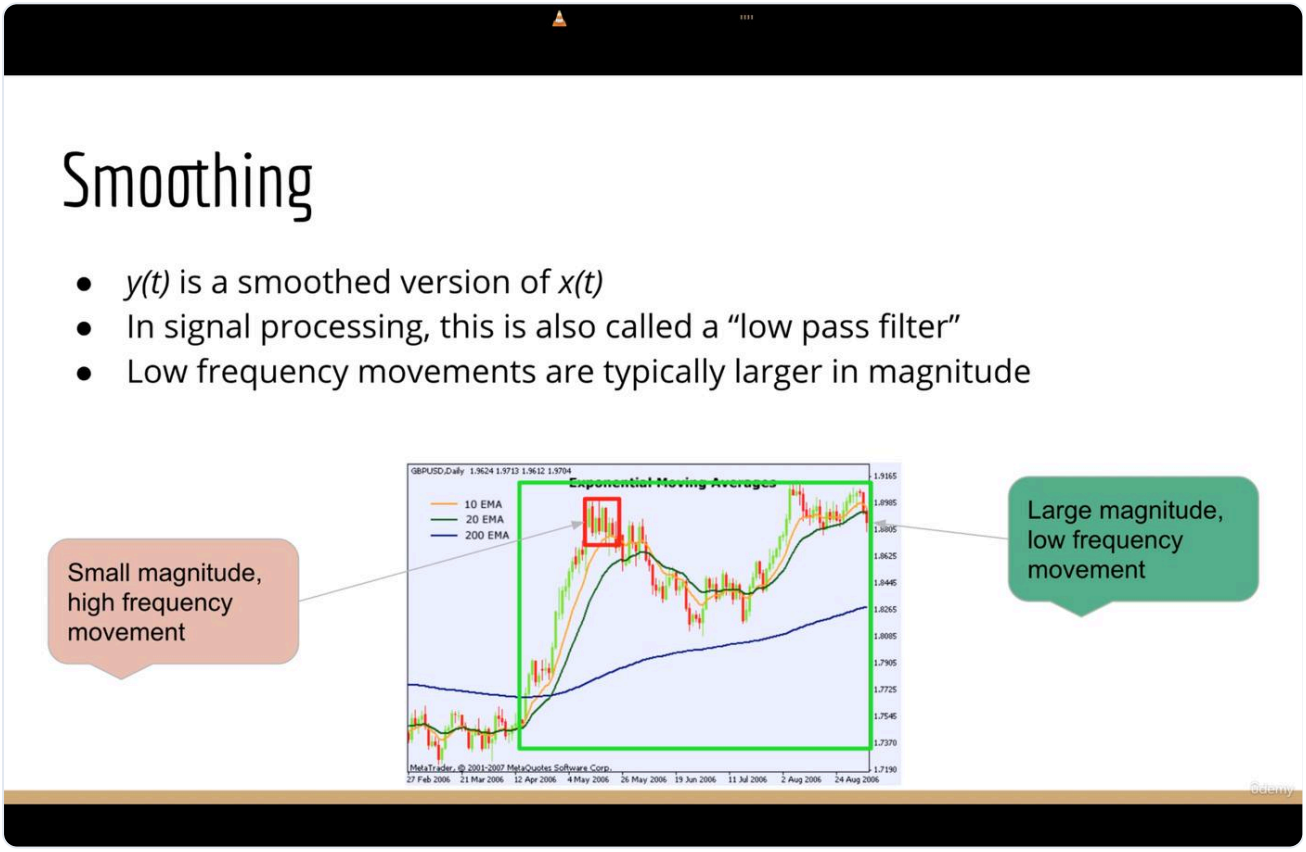

FIGURE 38.4

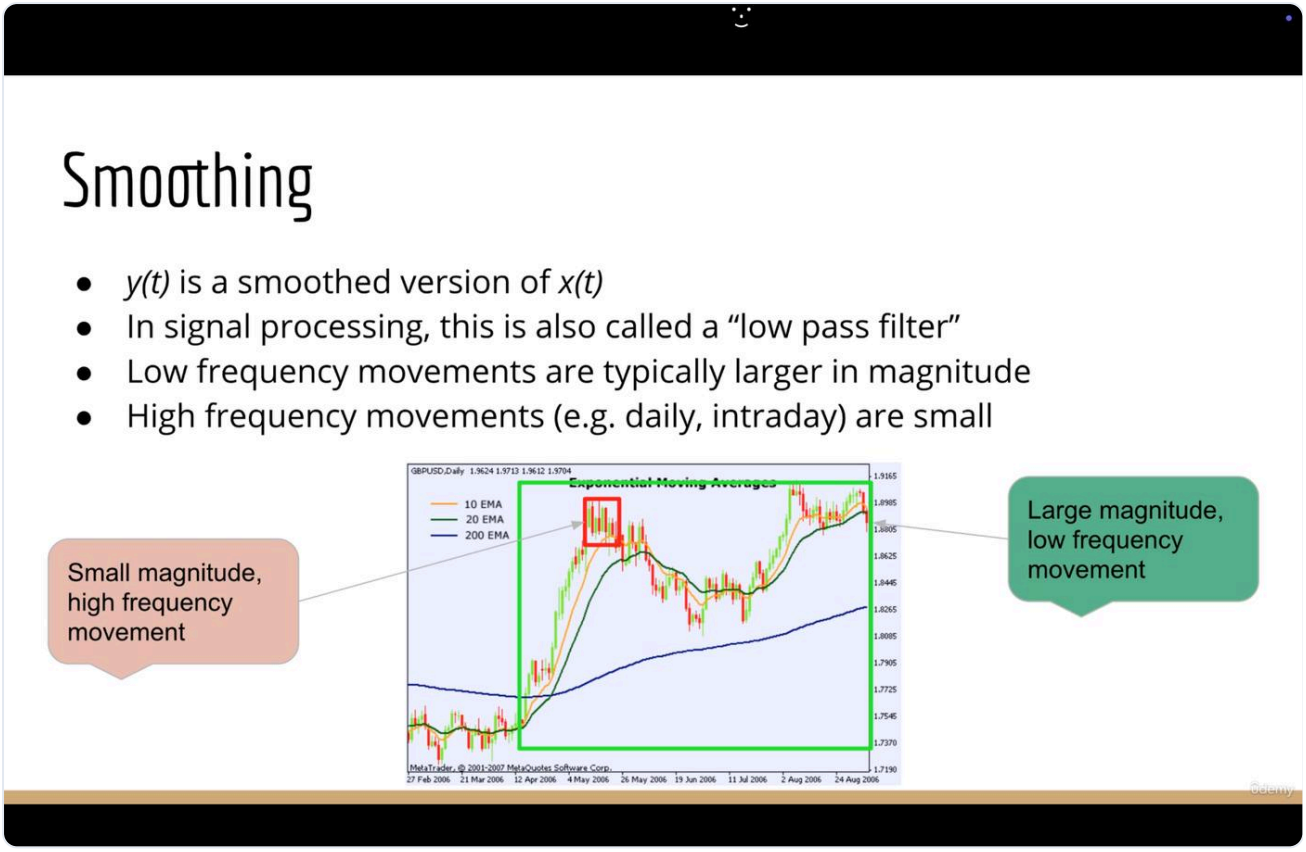

FIGURE 38.5

Generally speaking, the low frequency movements are what we actually care about because they're usually larger in magnitude. The high frequency movements are usually very tiny and tend not to be that significant. So by using a low pass filter, we are essentially passing our input signal into an algorithm that removes the high frequencies we do not care about. There is one small problem with our low pass filter. Note that our output Y of T always depends on the previous value Y of T minus one. But if we are trying to generate the first output Y of one, then what is Y of zero?

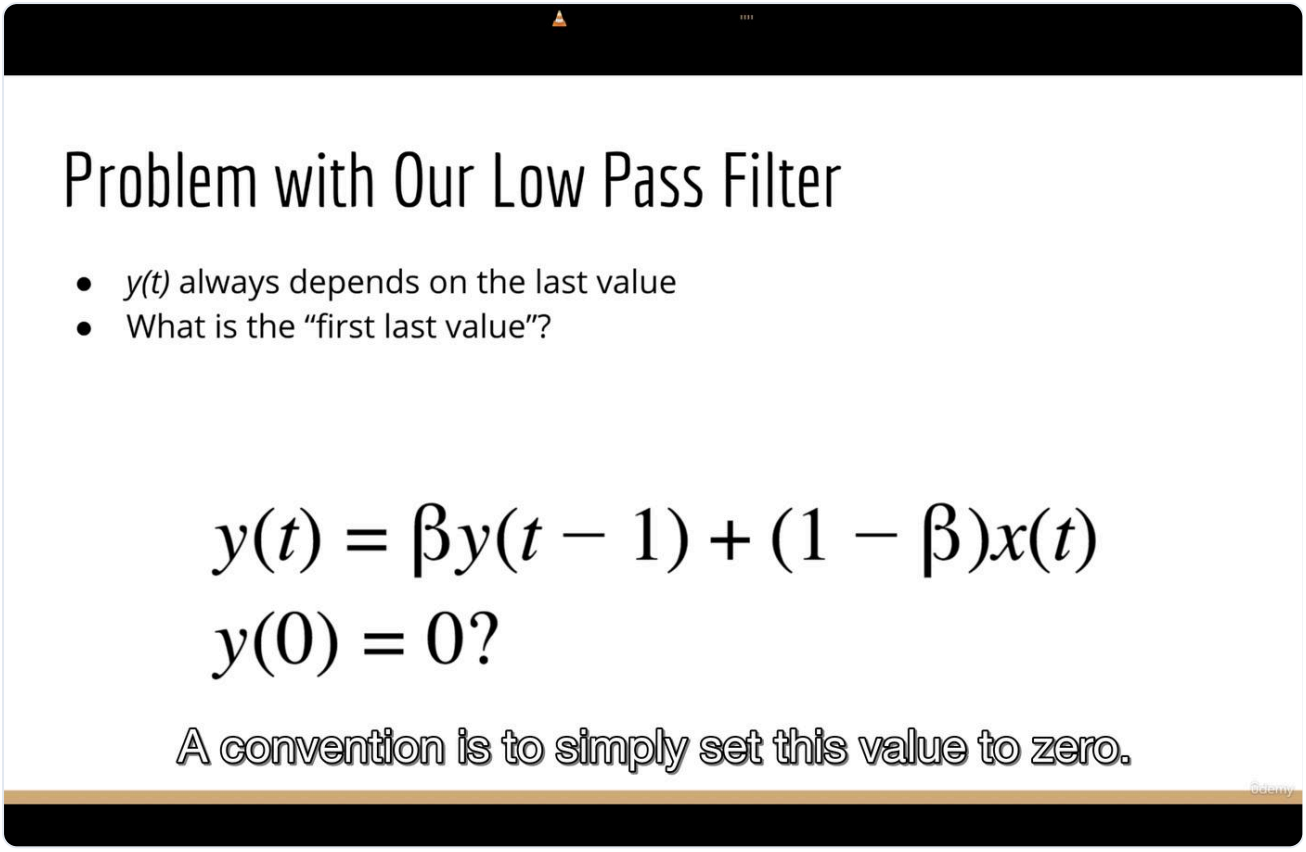

FIGURE 38.6 | Problem with Our Low Pass Filter

A convention is to simply set this value to zero. But what happens when we do this? The answer is that at the beginning of our output, all the values are biased toward zero.

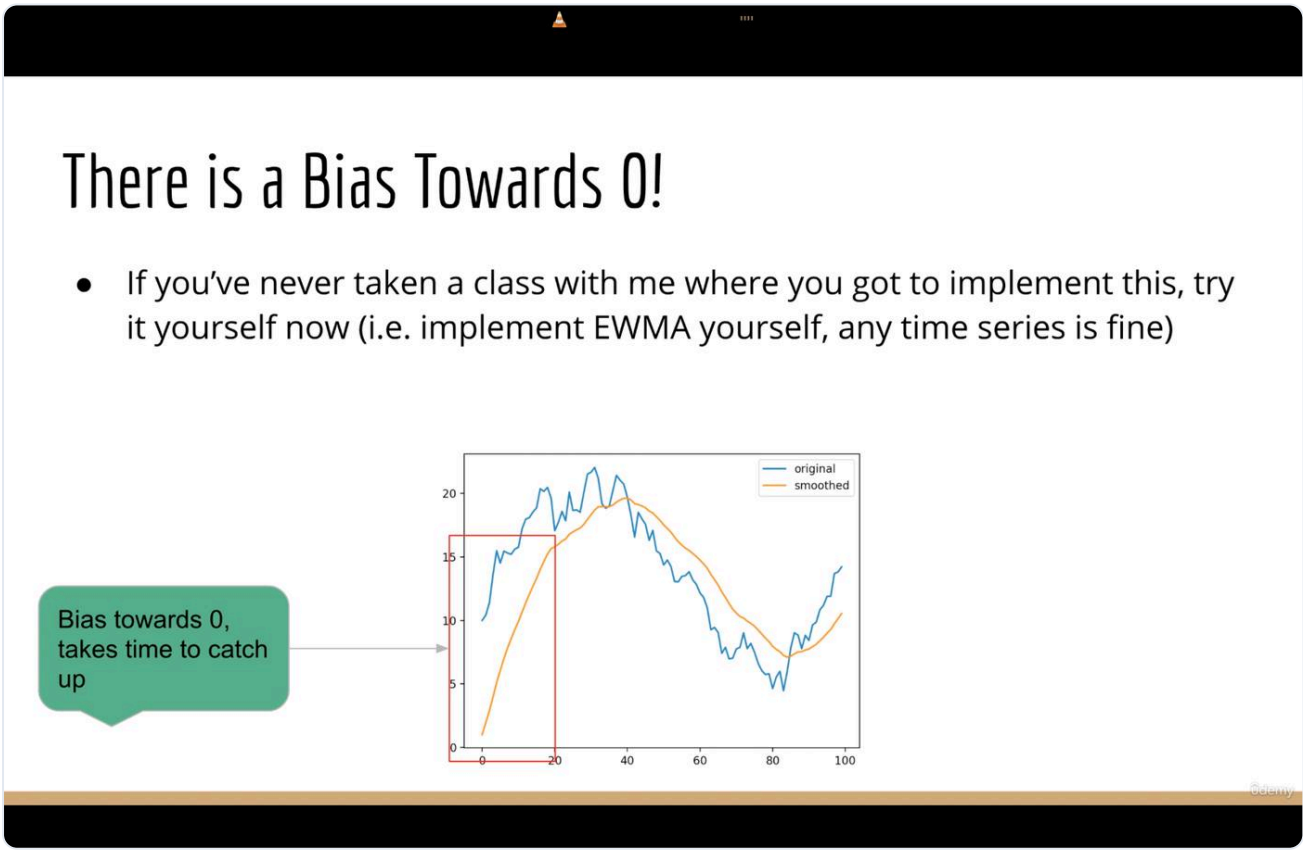

FIGURE 38.7

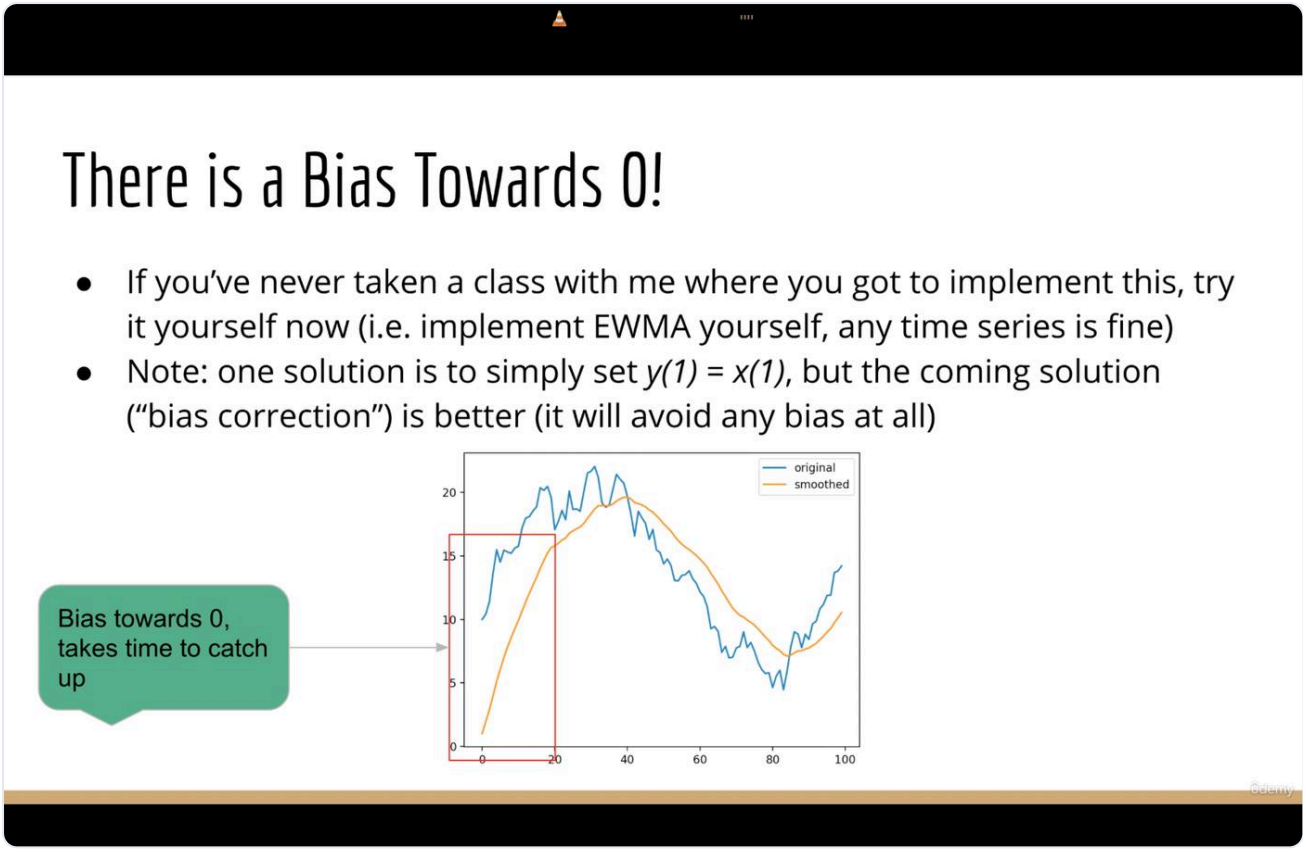

FIGURE 38.8

Now, if you never implemented this before or you've never taken one of my courses where you got an opportunity to implement this, it would be very useful to try and implement it yourself, to see what I'm talking about first hand. It should be a pretty simple exercise and you can try this on any time series you choose, such as the price of your favorite stock. Now, what does it mean that the beginning values are biased toward zero? Well, this is clearly not good because it takes our filter some time to catch up and to start producing useful outputs as a side note in other fields. There are simple solutions to this, such as simply setting Y of one to be equal to X of one instead of zero.

However, in deep learning we employ a method known as bias correction. So what is bias correction, bias correction means that instead of using what we would usually use Y of T, we adjust this value by some factor to get Y hat of T specifically, we divide by one minus beta to the power T.

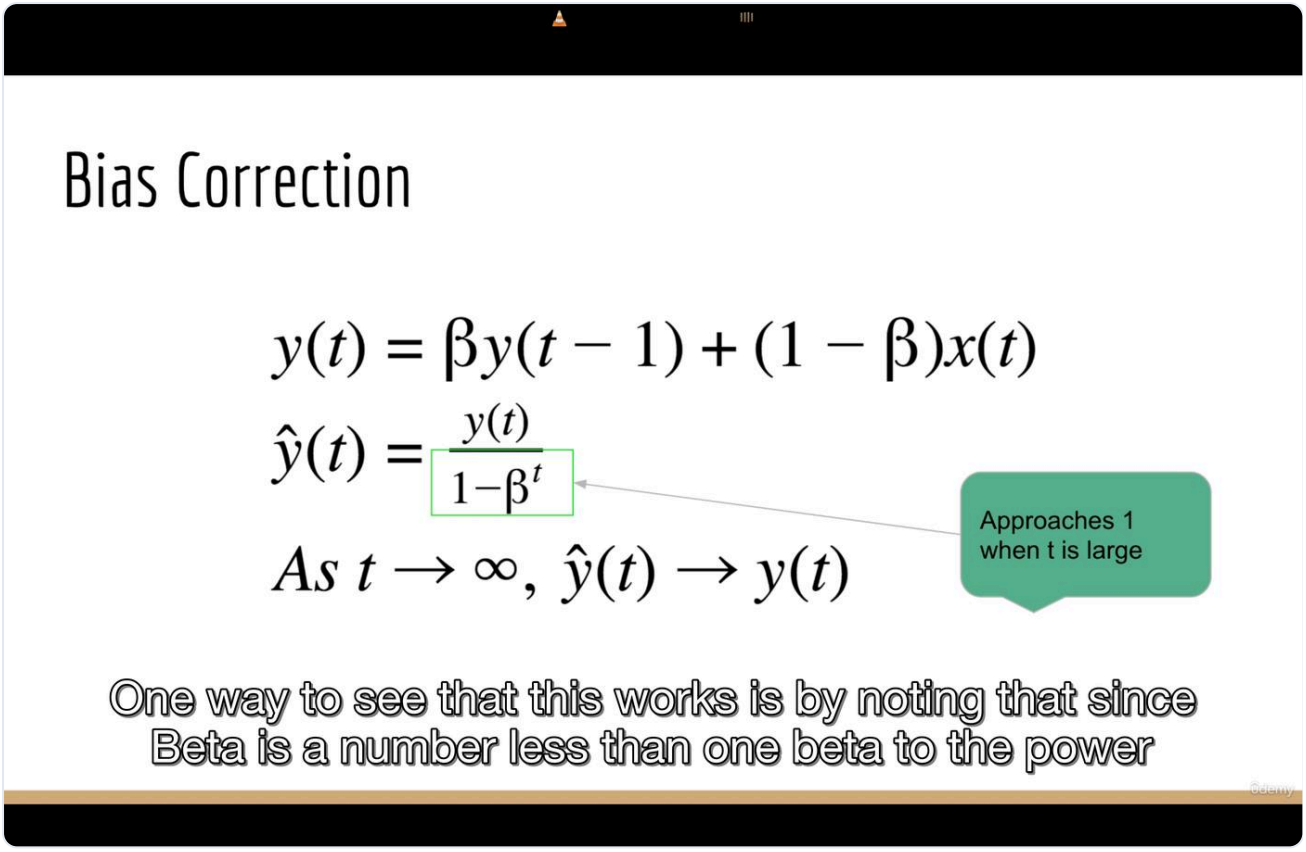

FIGURE 38.9 | Bias Correction

One way to see that this works is by noting that since Beta is a number less than one beta to the power T will approach zero as T gets large. Therefore, when T is large, we essentially divide by a number close to one and Y hat of T is very close to Y of T, which makes sense. This is because bias correction is only really needed when T is small. So what happens when T is small? The simplest way to see this is to plug some numbers in, let's suppose that beta is equal to zero point nine nine. Then Y of one is equal to beta times Y of zero plus one minus beta times X of one since Y of zero is zero, this is just zero point zero one times X of one. Clearly this is not good since we are only taking one percent of the true input X of one.

However, what do we get for Y hat of one? Well, we take Y of one and divide it by one minus beta to the power one.

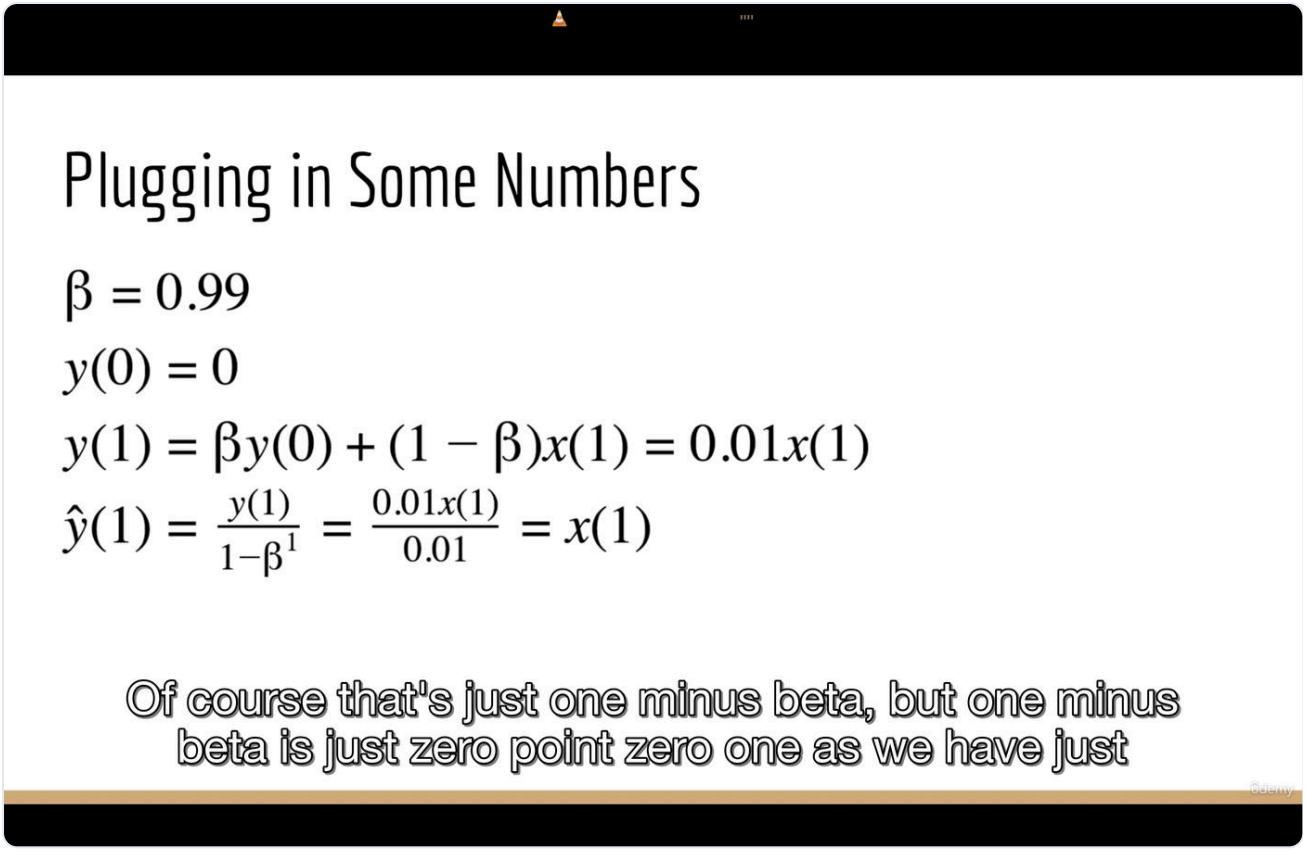

FIGURE 38.10 | Plugging in Some Numbers

Of course that's just one minus beta, but one minus beta is just zero point zero one as we have just seen. Therefore Y hat of one is just zero point zero one times X of one divided by zero point zero one, which is just X of one itself. Thus Y hat of one is no longer biased. It's exactly equal to X of one. How about when T is equal to two?

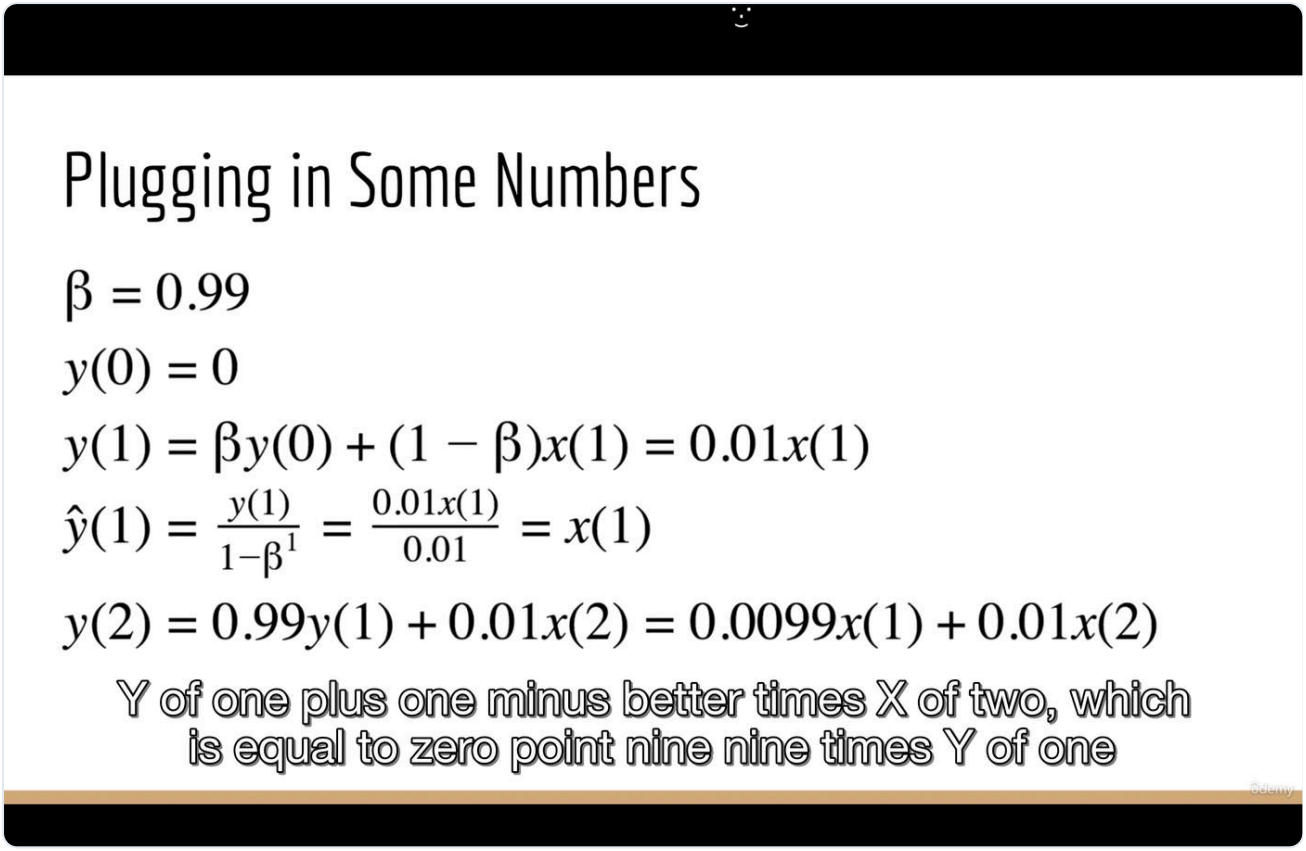

FIGURE 38.11 | Plugging in Some Numbers

In this case, Y of two is equal to beta times Y of one plus one minus beta times X of two, which is equal to zero point nine nine times Y of one plus zero point zero one times X of two. However, we know that Y of one is equal to zero point zero one times X of one, so we can plug that in. We then get Y of two equal to zero point zero zero nine nine times X of one plus zero point zero one times X of two. So now we basically have about one percent of X of one and about one percent of X of two. Still not a good estimate for Y of two. But what happens when we do bias correction? We take Y of two and divide that by one minus beta squared. Now the numbers get a little messy, so you'll have to do the calculations on your own at home.

But what you should end up with is this.

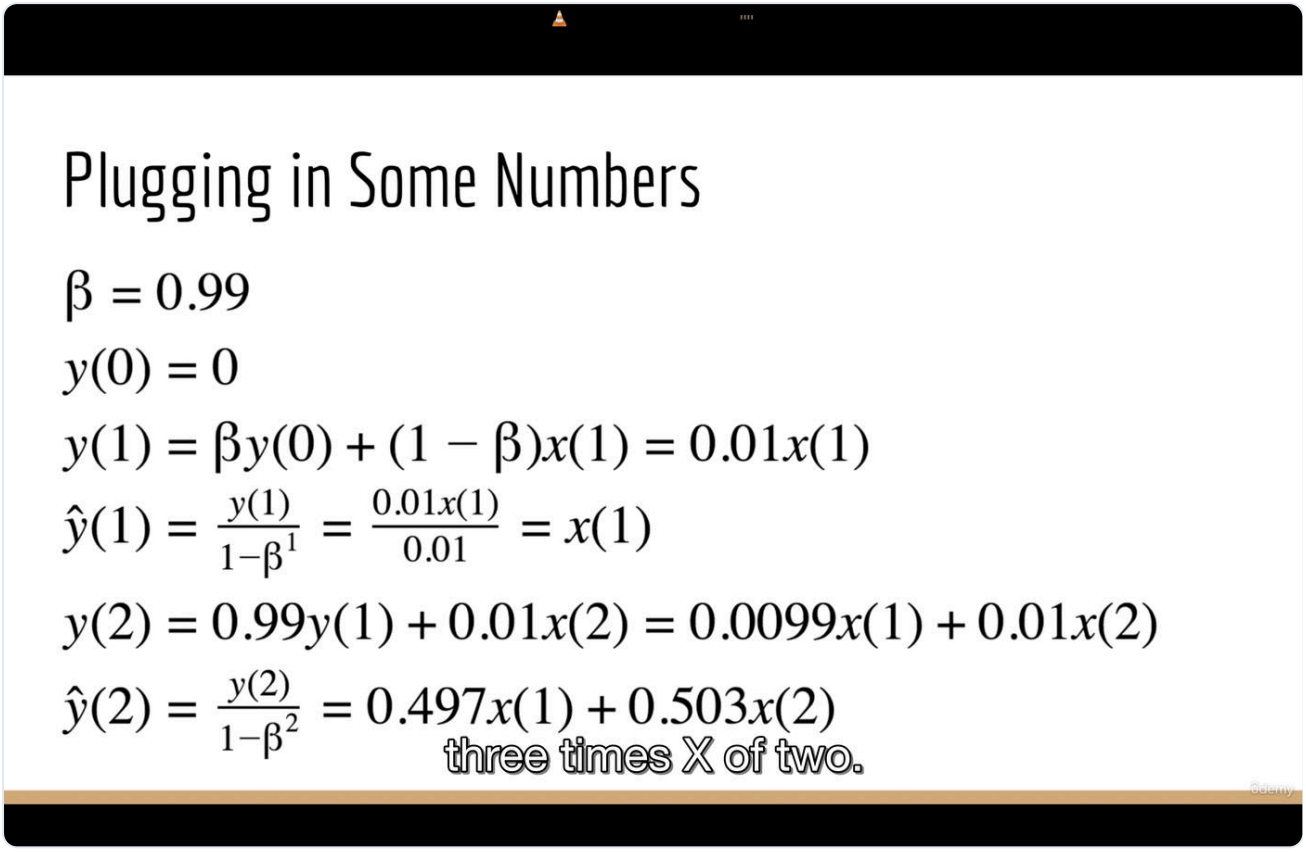

FIGURE 38.12 | Plugging in Some Numbers

You should get Y hat of two equal to zero point four nine seven times X of one plus zero point five zero three times X of two. So does this make sense? This makes perfect sense.

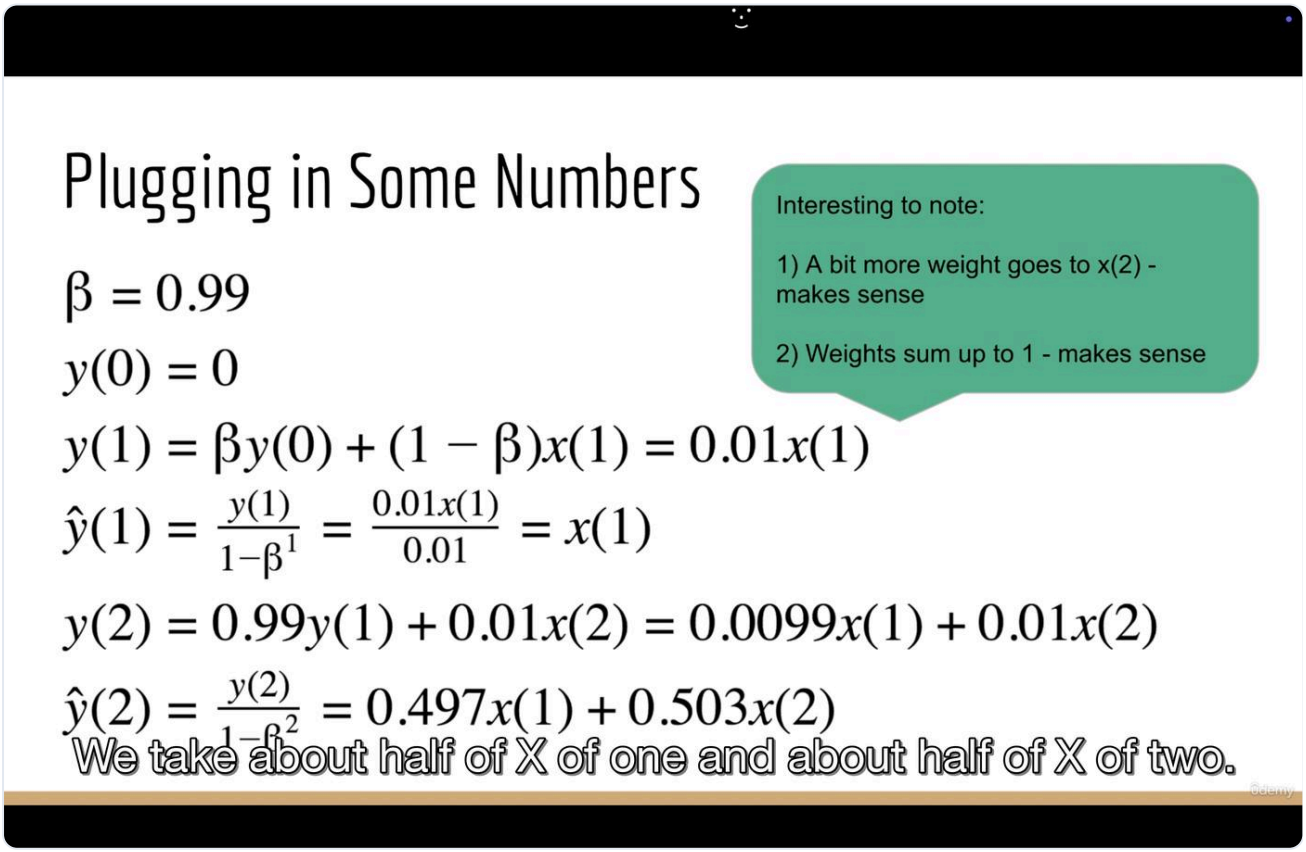

FIGURE 38.13

We take about half of X of one and about half of X of two. We take a little more from X of two since X of two is more recent than X of one. Furthermore, notice how these adjusted weights always add up to one, which makes a lot of sense. OK, so how can we incorporate this into our Adam optimization? Well, we know that we are doing two exponential moving averages, one for M and one for V. Therefore, we simply replace what we had before with these bias corrected versions of M and V.

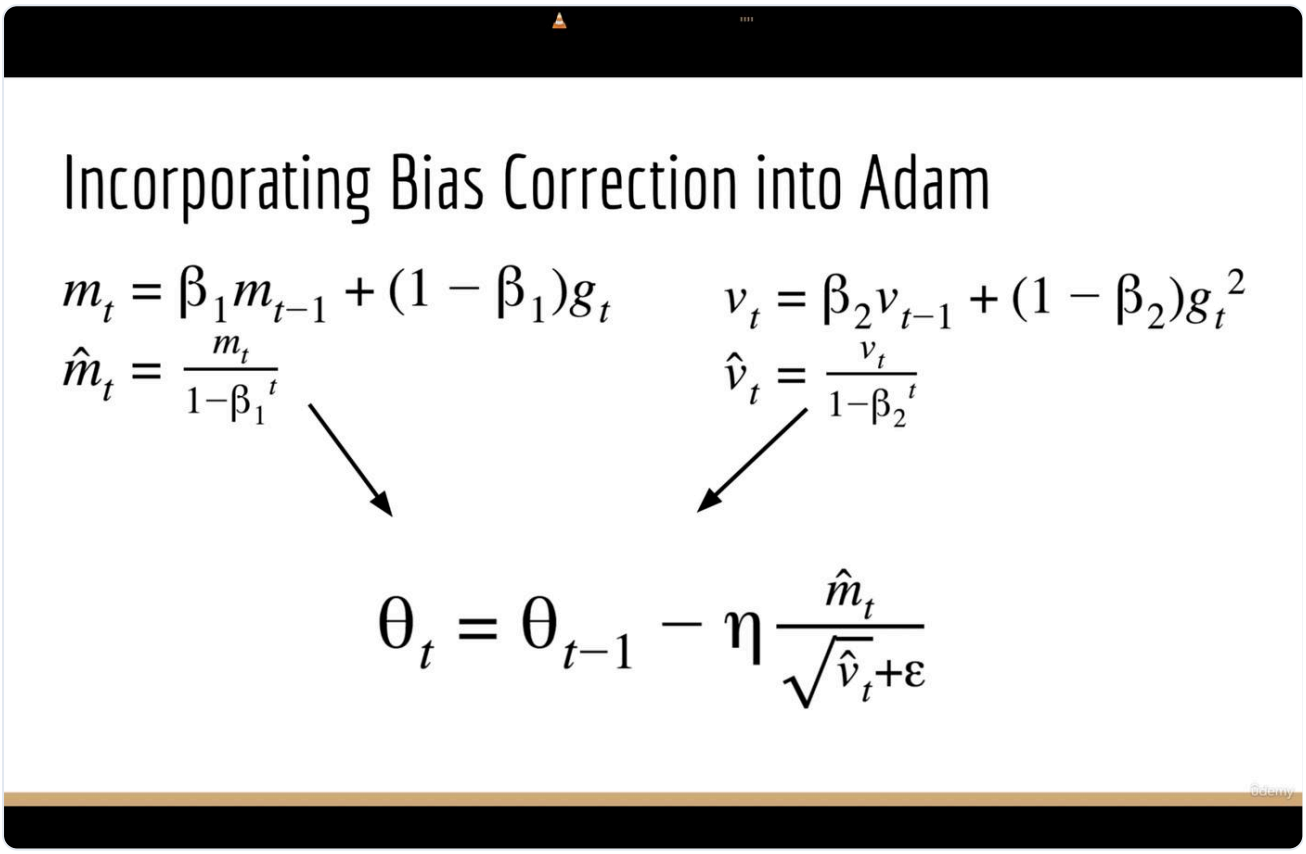

FIGURE 38.14 | Incorporating Bias Correction into Adam

So now, instead of subtracting the learning rate, multiplying by M and dividing by the square root of V, we simply replace M and V with their bias corrected versions. M hat and V hat. In total, our final algorithm is this before we start a training loop, we initialize M zero to zero V zero to zero and the time index T to zero. Then we enter our training loop inside the loop. We increment the time index by one. Next we find M of T using the usual recursive equation with the hyperparameter decay rate beta one. Next, we do the same thing for V of T using beta two.

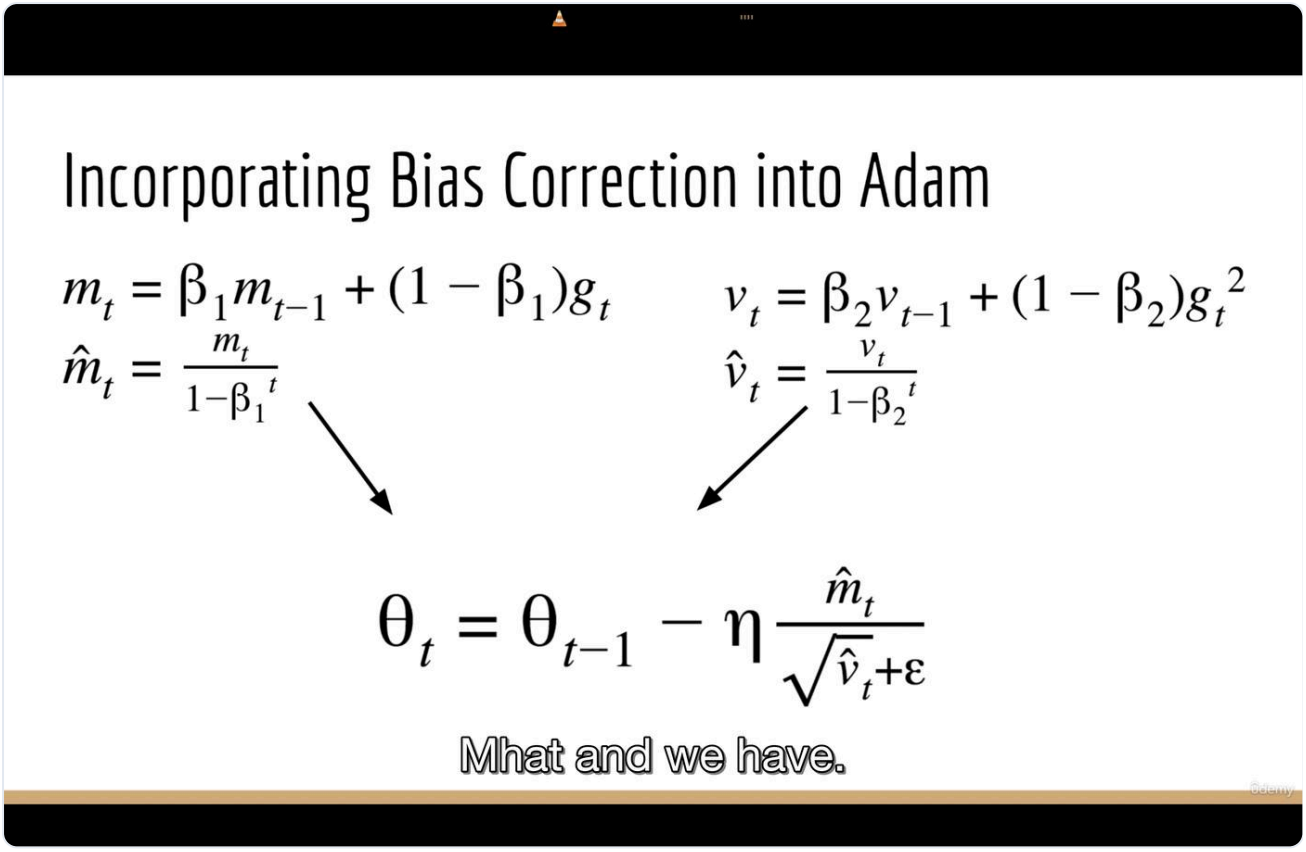

FIGURE 38.15 | Incorporating Bias Correction into Adam

Next, we perform bias correction to get M hat of T and V hat of T. Finally we perform our gradient update using the usual equation. Note again that this is for some generic parameter vector theta. In practice this represents a collection of all the parameters of your model. So you will need Ms and Vs of the same size as all of your parameter vectors matrices and tensors. The final thing I want to discuss in this lecture is what values do we choose for these hyperparameters?

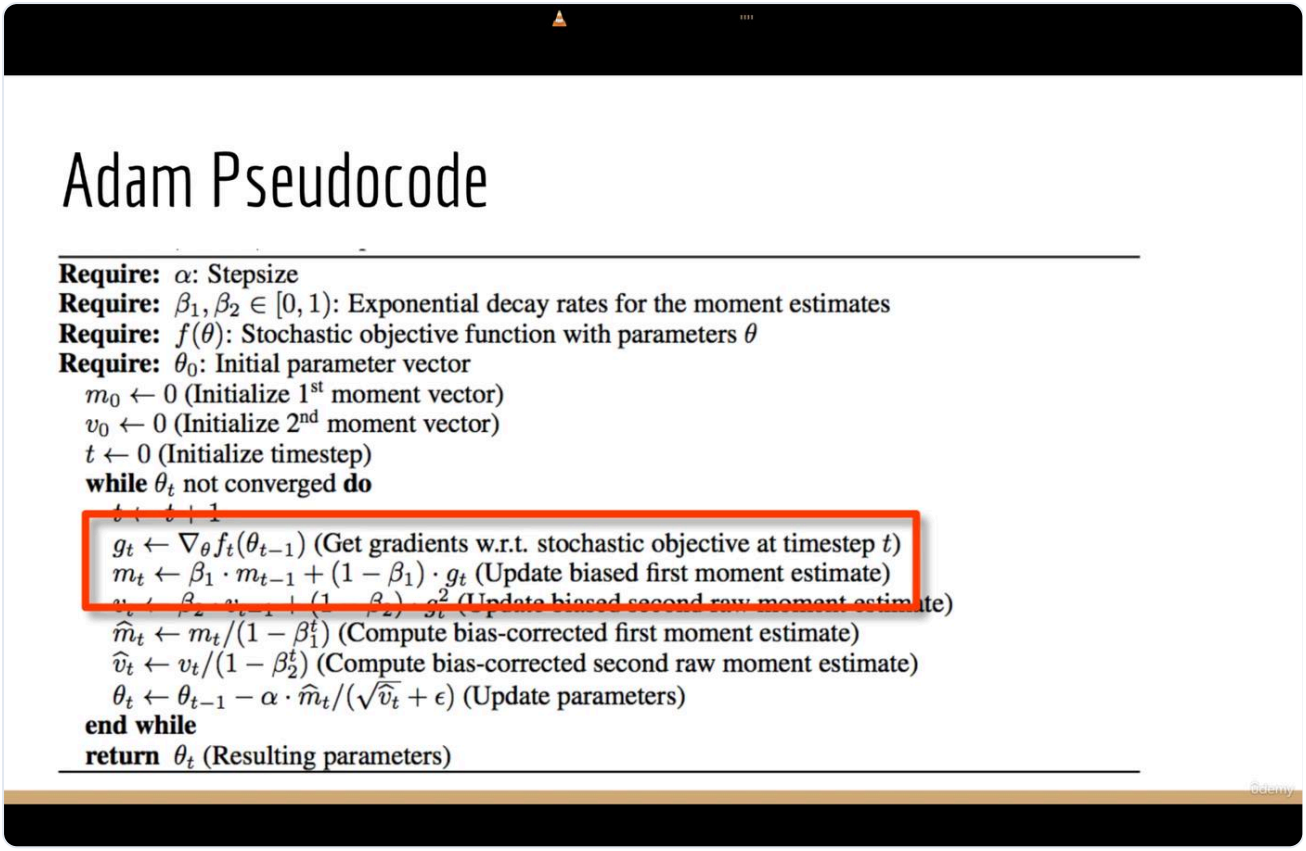

FIGURE 38.16

Previously, I told you that Adam optimization is very robust. That is the same values work for a wide range of problems with practice and experience. You may find that this is the case with TensorFlow and PyTorch. Note that these default values are built into the optimizers, so you rarely have to think about them. However, if you do ever end up implementing Adam from scratch, here are the suggested default values for the learning rate. A typical default value is ten to the minus three.

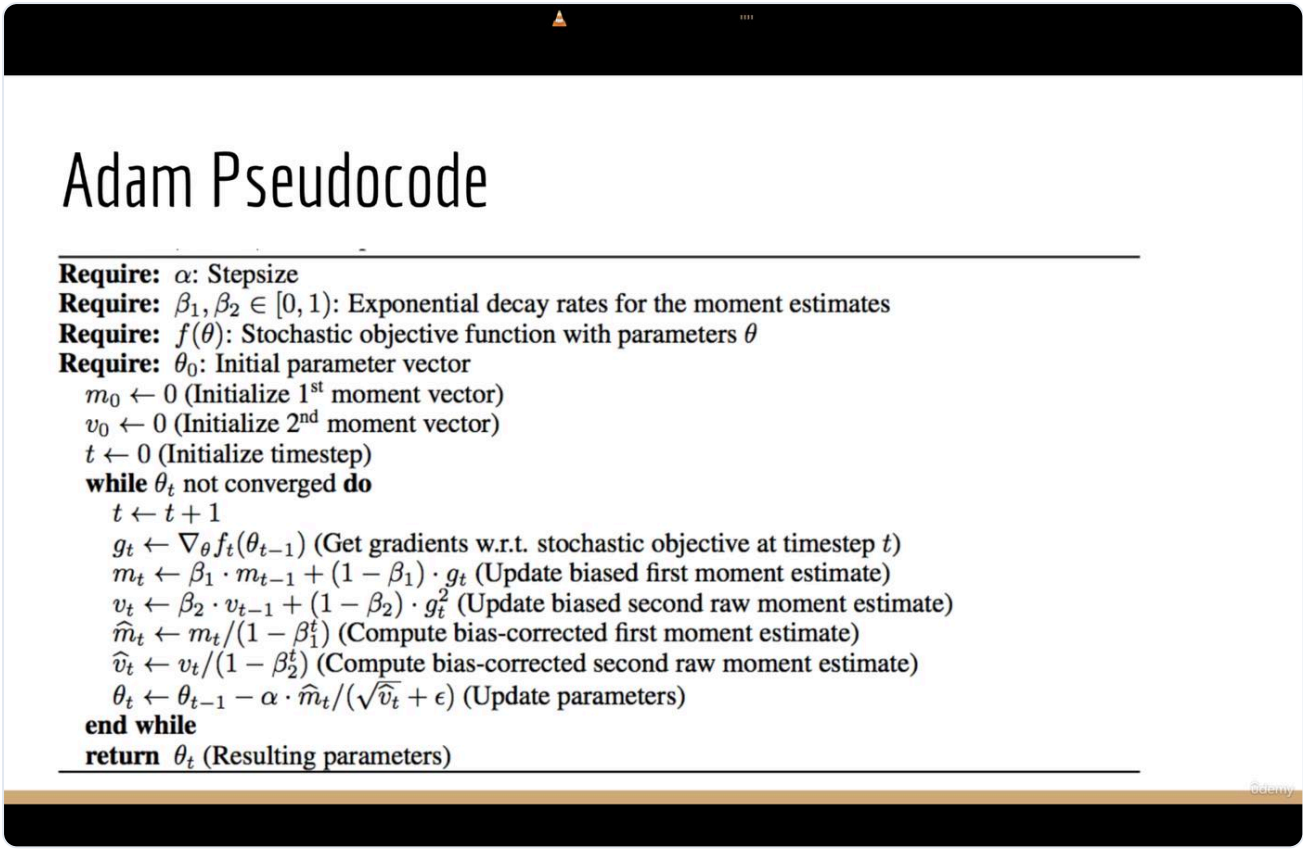

FIGURE 38.17

For Beta one, we use zero point nine. As you recall, this is a typical value. When we are using momentum for Beta two, we use zero point nine nine nine. As you recall, this is a typical value. When we are using RMSprop for Epsilon, a typical default value is ten to the minus eight. As a final thought to conclude this lecture, I want to mention that while Adam is a common default choice, you will still encounter scenarios where it is not the best choice.

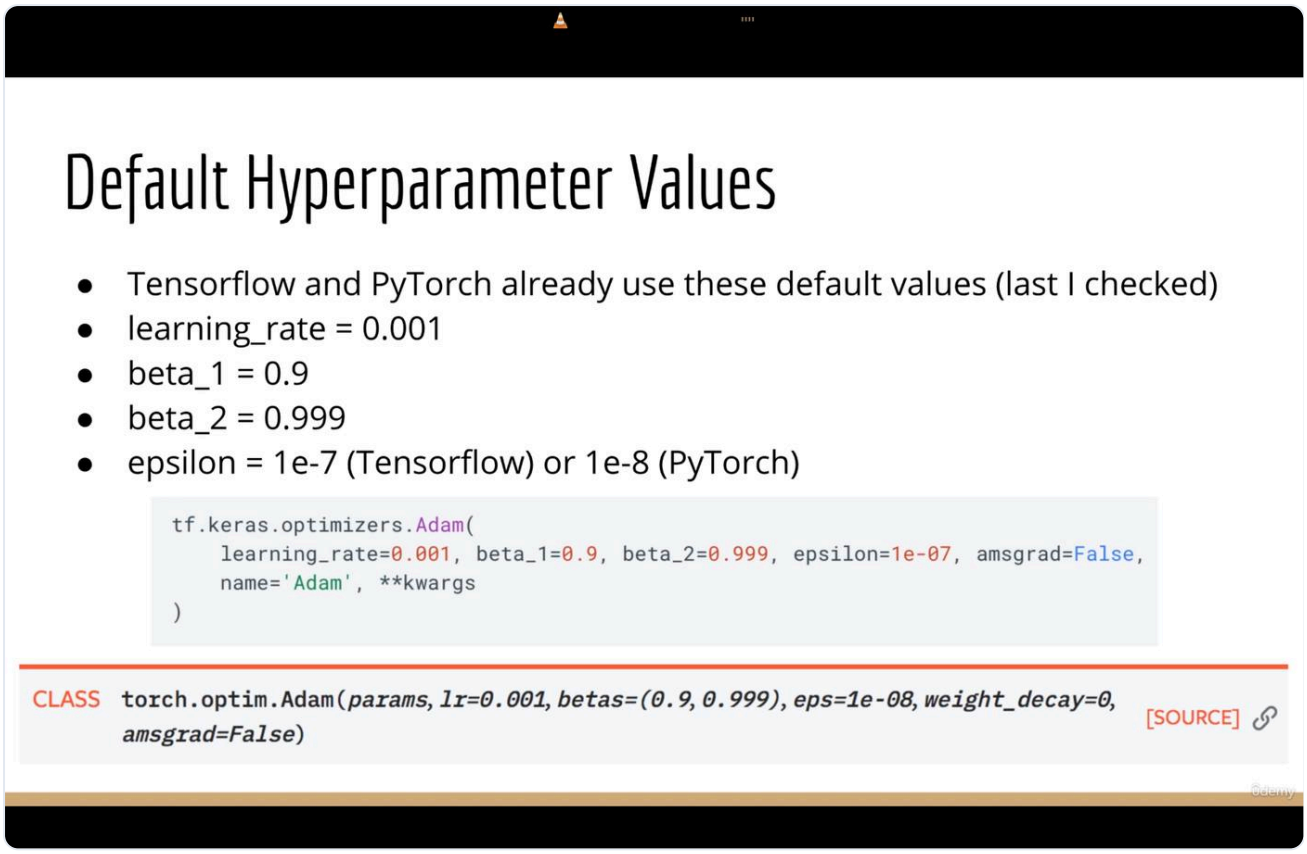

FIGURE 38.18 | Default Hyperparameter Values

In fact, sometime after many people had adopted Adam as their go to default choice of optimizer, a paper was written that showed a regular stochastic gradient descent with momentum performs better than Adam. This paper is called the Marginal Value of Adaptive Gradient Methods in Machine Learning. Personally, before the invention of Adam and even after the invention of Adam, SGD with momentum was often my own default choice. This reminds me of a rule that I often repeat to my students. This rule is that machine learning is experimentation, not philosophy. Oftentimes students just want to be told to one way of doing things so that they have less choices to make by themselves, which I suppose is more comfortable.

But the reality is, unless you try it yourself, you will never know what the real answer is to put this in a different way.

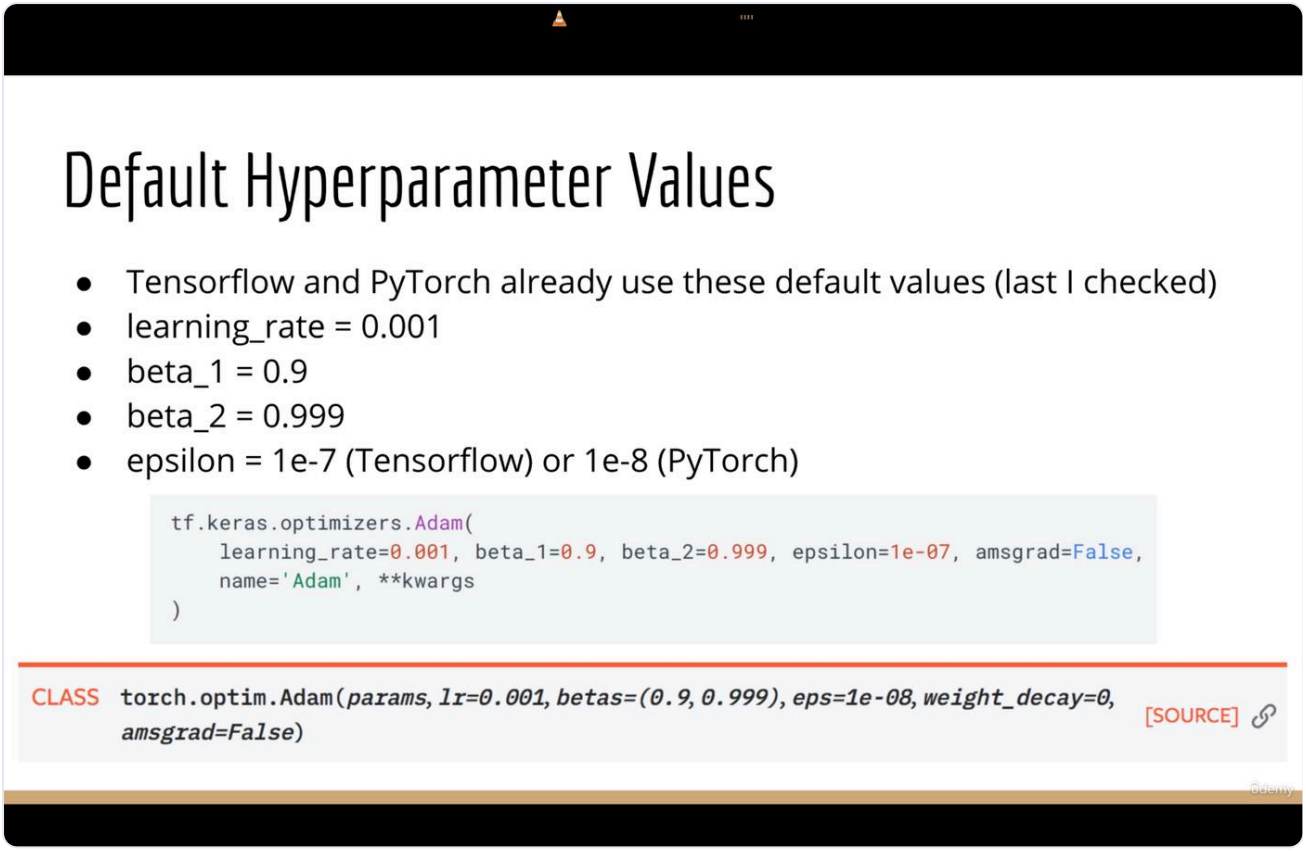

FIGURE 38.19 | Default Hyperparameter Values

Ask yourself this question. What should you do when you want to know the output of a computer program, specifically a computer program where you train your model on some data set using a variety of optimizers? Well, it would be silly and inefficient to ask me or anyone else what the output of the program will be. Similarly, it would be incorrect to try and guess what the output of the program will be instead, if you want to know the output of a computer program. Let me tell you an ancient secret that only true gurus know the answer is to simply run the computer program and look at the output yourself. As always, I am glad to be the source of many surprising and unusual facts.

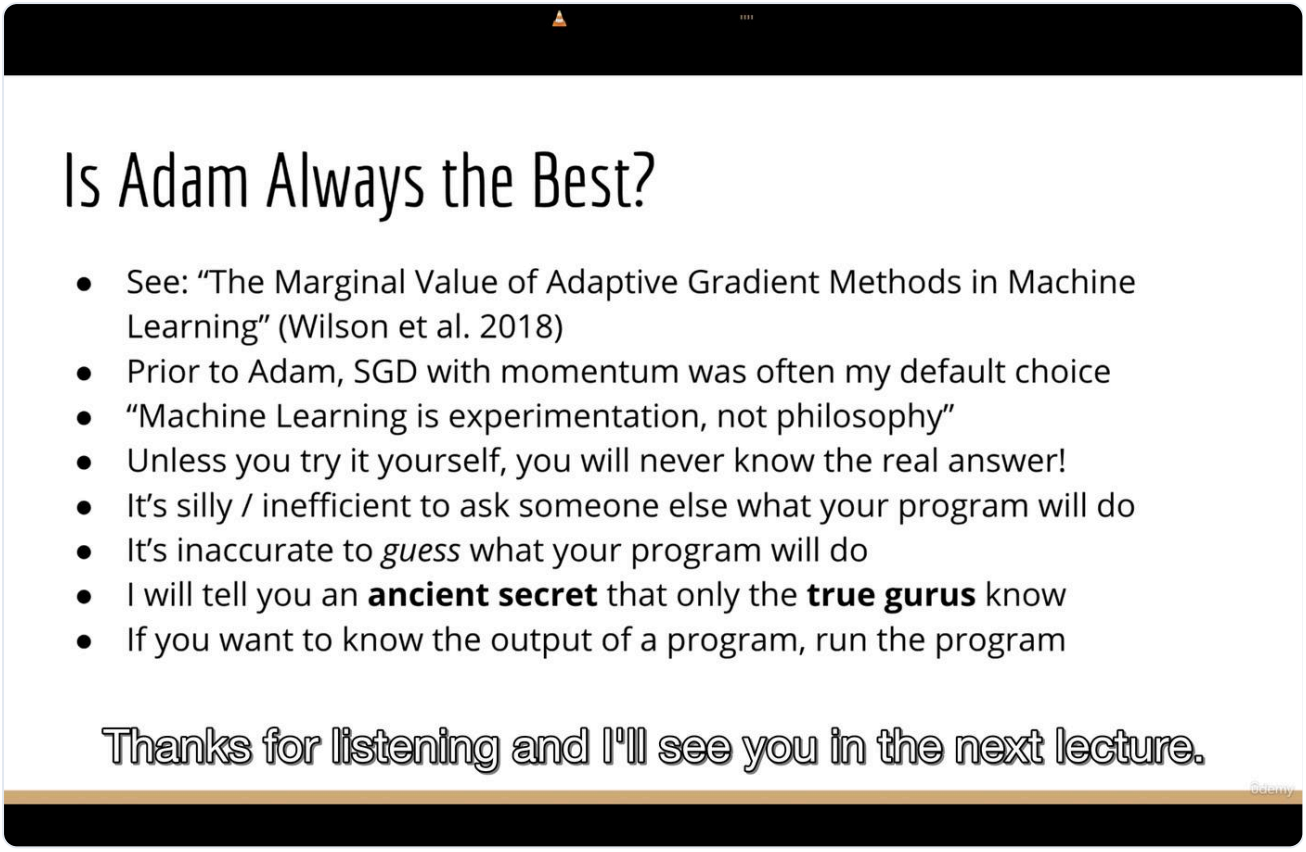

FIGURE 38.20 | Is Adam Always the Best?

Thanks for listening and I'll see you in the next lecture.